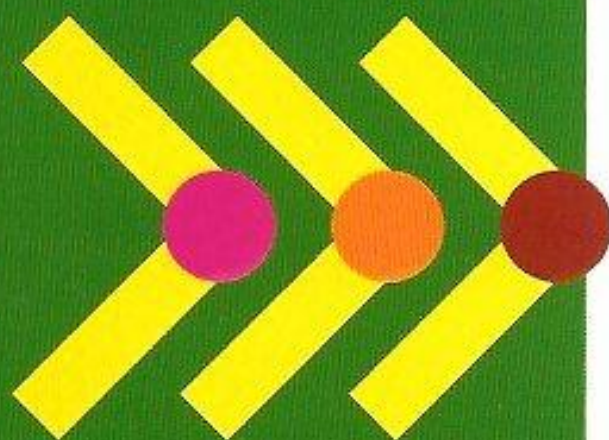


Herwig Mayr

Projekt Engineering

Ingenieurmäßige
Softwareentwicklung
in Projektgruppen

2. Auflage



mit Website

neu bearbeitete
Auflage



Fachbuchverlag Leipzig

Herwig Mayr
Projekt Engineering

„The scientist is chasing ,truth‘
and an engineer ,utility‘.
Two different directions.
Both require an intense mixture
of creativity and logic.“

(Richard Botting)

„Tell me and I forget.
Teach me and I remember.
Involve me and I learn.“

(Benjamin Franklin)

Herwig Mayr

Projekt Engineering

Ingenieurmäßige Softwareentwicklung in Projektgruppen

2., neu bearbeitete Auflage



Fachbuchverlag Leipzig
im Carl Hanser Verlag

FH-Prof. Dipl.-Ing. Dr. Herwig Mayr
Fachhochschule Hagenberg, Bereich Software Engineering

Alle in diesem Buch enthaltenen Programme, Verfahren und elektronischen Schaltungen wurden nach bestem Wissen erstellt und mit Sorgfalt getestet. Dennoch sind Fehler nicht ganz auszuschließen. Aus diesem Grund ist das im vorliegenden Buch enthaltene Programm-Material mit keiner Verpflichtung oder Garantie irgendeiner Art verbunden. Autor und Verlag übernehmen infolgedessen keine Verantwortung und werden keine daraus folgende oder sonstige Haftung übernehmen, die auf irgendeine Art aus der Benutzung dieses Programm-Materials oder Teilen davon entsteht.

Die Wiedergabe von Gebrauchsnamen, Handelsnamen, Warenbezeichnungen usw. in diesem Werk berechtigt auch ohne besondere Kennzeichnung nicht zu der Annahme, dass solche Namen im Sinne der Warenzeichen- und Markenschutz-Gesetzgebung als frei zu betrachten wären und daher von jedermann benutzt werden dürften.

Bibliografische Information Der Deutschen Bibliothek

Die Deutsche Bibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über <http://dnb.ddb.de> abrufbar.

ISBN 3-446-40070-2

Dieses Werk ist urheberrechtlich geschützt.

Alle Rechte, auch die der Übersetzung, des Nachdruckes und der Vervielfältigung des Buches, oder Teilen daraus, vorbehalten. Kein Teil des Werkes darf ohne schriftliche Genehmigung des Verlages in irgendeiner Form (Fotokopie, Mikrofilm oder ein anderes Verfahren), auch nicht für Zwecke der Unterrichtsgestaltung – mit Ausnahme der in den §§ 53, 54 URG genannten Sonderfälle –, reproduziert oder unter Verwendung elektronischer Systeme verarbeitet, vervielfältigt oder verbreitet werden.

Seit 1.1.2012 liegen sämtliche Rechte am Werk wieder beim Autor.

Fachbuchverlag Leipzig
im Carl Hanser Verlag

© 2005 Carl Hanser Verlag München Wien
Internet: <http://www.fachbuch-leipzig.hanser.de>

Druck und Bindung: Druckhaus „Thomas Müntzer“ GmbH, Bad Langensalza
Printed in Germany

Vorwort

Über dieses Buch

Moderne Softwareerstellung hat sich von der Kunst eines Einzelnen für einen Einzelnen (oft denselben) hin zu einer Ingenieursdisziplin entwickelt, in der ein Team von Entwicklern ein qualitativ adäquates Softwareprodukt zu planbaren Kosten und in abschätzbarer Zeit herstellt. Zusätzlich finden immer häufiger Bausteine und Bausteinsammlungen Verwendung, die zu immer größeren Softwaresystemen kombiniert werden und auf zunehmend komplexeren, vernetzten Rechnerarchitekturen Einsatz finden.

Dieses Buch gibt eine fundierte Einführung in die ingenieurmäßige Softwareentwicklung in Form von Projekten. Fokussiert werden dabei kleinere bis mittlere Entwicklungen (im Allgemeinen weniger als 10 Personenjahre) in Teamgrößen von 2 bis 15 Mitgliedern. Diese Struktur ist – zumindest im deutschsprachigen Raum – typisch für die moderne Softwareentwicklung, sowohl für aufbaubedingt kleine, aber sehr agile Softwareunternehmen als auch für vielfach bereits sehr stark projektorientiert umstrukturierte mittlere und größere IT-Unternehmen.

Dieses Buch ist *kein* Lehrbuch für Programmieren oder Algorithmenentwurf. Für diesen „klassischen“ Kernbereich der Softwareentwicklung wird auf die in großem Umfang vorhandene, vielfach exzellente Literatur verwiesen. Dieses Buch gibt sehr wohl detaillierte Anleitungen zu gutem Design und sauberer Gestaltung einer guten Implementierung, das Hauptaugenmerk liegt jedoch auf dem gesamten Entwicklungszyklus in einem Softwareprojekt von der initialen Ziel- und Anforderungsdefinitionsphase bis hin zur erfolgreichen Einführung des erstellten Produkts und einem für alle Beteiligten positiven Projektabschluss.

Speziell die in vielen anderen Büchern in diesem Bereich zu kurz kommenden Projektentwicklungsschritte der Zielfindung gemeinsam mit dem Auftraggeber, der (computerunterstützten) Anforderungsanalyse, des systematischen Testens sowie der Installation und Inbetriebnahme bis hin zu Betrieb, Wartung und Pflege eines Softwareprodukts finden in diesem Buch den Stellenwert, der ihnen auch im tatsächlichen Softwareentwicklungszyklus, zumindest kostenmäßig, zukommt. Zusätzlich werden Bereiche wie Projektführung und -planung, Qualitätssicherung und Risikobehandlung, die aus Technikersicht oft als Randgebiete der Softwareerstellung abgetan werden, im softwareentwicklungsspezifischen Kontext behandelt und damit die Notwendigkeit der Akquisition von Technik-, Wirtschafts- und Sozialkompetenz für einen erfolgreichen Know-how-Aufbau für die Softwareentwicklung im gebührenden Verhältnis betont.

Gliederung

Dieses Buch ist in vierzehn Kapitel unterteilt, die wiederum in drei Hauptabschnitte gruppiert sind:

- I. Einführung in die ingenieurmäßige Projektentwicklung,
- II. projektbegleitende Aufgabenbereiche,
- III. projektphasenspezifische Aufgabenbereiche.

Die Darstellung der Aufgabenbereiche erfolgt gemäß einem mnemonischen Schema (eingeführt am Ende des Abschnitts I), das eine klare Gliederung des Inhalts anhand eines anschaulichen Bildes ermöglicht. Bei der Beschreibung der Aufgabenbereiche wird besonderes Augenmerk gelegt auf konkrete Beispiele, die durch in der Praxis vom Autor gesammelte (oft auch subjektive) Detailtipps ergänzt werden. Die unmittelbare Umsetzbarkeit des Inhalts wird verstärkt durch konkrete Gliederungsempfehlungen für zu erstellende Dokumente sowie Checklisten für Prozesse und Ergebnisse. Ein ausführliches Inhaltsverzeichnis zu Beginn sowie ein detaillierter Index am Ende des Buches unterstützen das rasche Auffinden eines bestimmten Kontexts oder Problembereichs. Umfassende Literaturangaben ermöglichen bei Bedarf die rasche Vertiefung in bestimmten Interessengebieten.

Auf Wiederholungsfragen und kleine Übungsbeispiele am Ende jedes Kapitels wird bewusst verzichtet. Zum gründlichen Erlernen einer ingenieurmäßigen Softwareentwicklung eignet sich nur ein größeres, alle Aufgabenbereiche umfassendes Beispiel, das in seinen verschiedenen Fassetten durchleuchtet wird. Da ein solches Beispiel den Rahmen dieses Buches gesprengt hätte und das Internet ein viel geeigneteres Medium für das interaktive Durcharbeiten und Analysieren eines Beispielprojekts darstellt, wurde als Begleitmedium zu diesem Buch die Internetplattform <http://www.projekt-engineering.at> eingerichtet, die konkrete, durch den Autor im Übungsbetrieb eingesetzte Beispiele mit Realisierungsalternativen enthält.

Abschnitt I: Einführung in die ingenieurmäßige Projektentwicklung

In den ersten vier Kapiteln wird das Konzept des ingenieurmäßigen Projektentwicklungszyklus schrittweise entwickelt.

In *Kapitel 1* werden zuerst die grundlegenden Begriffe definiert bzw. bei mit mehreren Bedeutungen überlagerten Begriffen festgelegt, in welcher Bedeutung sie im Folgenden verwendet werden. Dies ist gerade im Fachgebiet des Software Engineering, in dem so wesentliche Begriffe wie Objekt und Komponente in einer Vielzahl von Bedeutungen Verwendung finden (oft im gleichen Kontext) und selbst über die Bedeutung (und manchmal sogar Sinnhaftigkeit) des Begriffs „Software Engineering“ in der Fachwelt fast 40 Jahre nach seiner Einführung noch heftig diskutiert wird, besonders angebracht.

Der Projektentwicklungszyklus in der Softwareentwicklung mit seiner typischen Ablaufstruktur wird in *Kapitel 2* erläutert. Auf den gesteuerten Kernprozess dieses Zyklus, die Produktentwicklung, wird in *Kapitel 3* im Detail eingegangen.

Durch Erweiterung des Produktentwicklungszyklus zu einem auch innerhalb eines Zyklus geregelten Prozess wird in *Kapitel 4* schließlich die ingenieurmäßige Projektentwicklung definiert. Diese Definition führt zu einer notwendigen Aufgabenteilung, deren Struktur aber auch die inhärenten Schwierigkeiten einer Softwareentwicklung klar darlegt. Ein mnemonisches Schema der ingenieurmäßigen Software-Projektentwicklung zusammen mit den integrierten Phasen schließt dieses Kapitel ab.

Abschnitt II: Projektbegleitende Aufgabenbereiche

Die moderne, qualitätsorientierte Softwareentwicklung ist sehr auf eine integrative Sicht der Entwicklungsschritte und eine inkrementelle Produkterstellung mit kurzen Regelkreisen ausgerichtet. In *Kapitel 5* wird daher eine detaillierte Einführung in für die Softwareentwicklung geeignete Formen der Aufbauorganisation von Projekten gegeben. Dabei werden allgemeine Organisationsformen den softwareentwicklungsspezifischen gegenüber gestellt. Weiters wird auf ökonomisch-soziale Aspekte einer Softwareentwicklung in Kleingruppen, wie Ausbildungsmix, Leistungsfähigkeit, Führungsverständnis und Motivation, eingegangen.

Kapitel 6 (Ablauforganisation) befasst sich ausführlich mit für die ingenieurmäßige Softwareentwicklung erstellten Vorgehensmodellen und Methoden sowie deren Bedeutung für den Projektlebenszyklus und den Softwarelebenszyklus. Neben der Vermittlung eines grundlegenden Verständnisses für Softwareentwicklungsprozesse wird auch auf die Diskussion des traditionellen versus des agilen Vorgehens eingegangen.

Der wachsenden Bedeutung der Kommunikation innerhalb eines Entwicklungsteams sowie zwischen Auftraggeber und Auftragnehmer wird in *Kapitel 7* Rechnung getragen. Zuerst wird dabei auf die Notwendigkeit und Arten der Kommunikation (schriftlich, mündlich) eingegangen. Anschließend werden der Dokumentationsprozess sowie die Dokumente von Aufbau- und Ablauforganisation beleuchtet. Große Bedeutung wird dabei der strukturierten Darstellung einer sauberen Dokumentation von Softwareprojekten (inklusive Inhaltsangaben und Checklisten) zugemessen.

In *Kapitel 8* wird auf die immer größer werdende Bedeutung der Qualität von Softwareprodukten eingegangen. Nach der Definition von Softwarequalität und Anmerkungen zu deren Messbarkeit werden konkrete Maßnahmen zu Qualitätsplanung, -prüfung und -lenkung vorgestellt. Wichtige Techniken und Modelle werden in ihrem Kontext der Softwareentwicklung präsentiert. Abschließend wird auf die Bedeutung von Normen und Richtlinien eingegangen.

Abschnitt III: Projektphasenspezifische Aufgabenbereiche

In diesem Abschnitt werden die Hauptphasen einer Softwareentwicklung beschrieben. Dabei wird betont, dass diese Phasen nicht sequenziell durchgeführt werden können und sollen, aber eine bestimmte Aktivität immer eindeutig einer Phase zugeordnet werden soll. Die Phasen werden in der Reihenfolge diskutiert, in der sie am ehesten chronologisch in einem Projekt beginnen, zumindest aber didaktisch sinnvoll vermittelt werden können.

In *Kapitel 9* wird die am Beginn jedes Projekts erforderliche Zielerforschung beschrieben. Konkret wird auf die notwendige Erhebung beim Auftraggeber (inklusive Techniken) sowie die anschließende Zielfindung durch den (potenziellen) Auftragnehmer eingegangen.

Kapitel 10 beschreibt die Hauptabschnitte der Analysephase eines Projektes, i. e. die Anforderungsanalyse, die (Detail-)Planung sowie die Risikobehandlung. Der historischen Entwicklung der Analysetechniken sowie deren sinnvollen Einsatzmöglichkeiten in modernen Softwareprojekten wird besonderes Augenmerk geschenkt.

Kapitel 11 (Design) und *Kapitel 12* (Implementierung) beschreiben die klassischen Kerngebiete der Softwareentwicklung. Dabei wird, wie bereits erwähnt, kein Design- oder Programmierkurs geboten, der Fokus liegt vielmehr auf der Beschreibung eines sauberen Entwicklungsprozesses, kombiniert mit Hinweisen für einen guten Design- und Implementierungsstil und wie dieser in einer gruppenorientierten Arbeitsweise durchgesetzt und aufrechterhalten werden kann. Der Bereich des Debuggens wird ebenfalls der Implementierung zugerechnet, da hierbei die Fehlersuche und Korrektur durch den Implementierer selbst erfolgt.

Testen steht im Mittelpunkt von *Kapitel 13*. Gerade der Bereich des Testens wird in vielen Software Engineering-Büchern zu wenig und zu unstrukturiert angesprochen (vor allem, wenn man den hohen Aufwandsanteil des Testens im gesamten Projektentwicklungszyklus in Betracht zieht). Zusätzlich finden gerade im Bereich des Testens viele Begriffe in der Literatur unterschiedliche Verwendung, sodass dieses Kapitel auch den Zweck der Einführung einer sauberen Nomenklatur des Testens verfolgt.

Kapitel 14 befasst sich ausführlich mit der Produkt-Einführung und beschreibt, nach Wissen des Autors, erstmalig alle notwendigen Tätigkeiten, die nach der Erstellung eines ausgetesteten Produkts bis zu einem erfolgreichen Projektabschluss (inklusive Evaluierung des Projektverlaufs) geleistet werden müssen. Auch hier konzentriert sich die Beschreibung auf Aufgaben, Techniken, Werkzeuge und Ergebnisse, die effektiv und effizient von einer eher kleinen Projektgruppe (2 bis 15 Personen) bei der Softwareentwicklung eingesetzt werden können und sollen.

Die Erläuterungen zu jeder Phase folgen einer einheitlichen Struktur und umfassen jeweils die Beschreibung von Ziel, Aufgaben, Techniken, Werkzeugen und Ergebnissen einer Phase. Dies ermöglicht einerseits die rasche Akquisition von Detailwissen zu einer Phase bzw. Kernaktivität, andererseits können hierdurch aber auch Anforderungskataloge, Checklisten etc. quer durch den gesamten Projektentwicklungszyklus sehr rasch erstellt werden. So ergibt sich zum Beispiel durch die Kombination der aufgelisteten Ergebnisse eine bereits zeitlich geordnete Liste zu erstellender Produkte und Dokumente für einen sauberen Projektentwicklungsprozess, der nur mehr für ein spezifisches Projekt maßgeschneidert werden muss.

Zielpublikum

Dieses Buch wurde als Lehrbuch für Studium und Selbststudium konzipiert mit dem expliziten Ziel, eine design- und implementierungsorientierte Softwareentwicklungsausbildung um eine Projektentwicklungsausbildung zu ergänzen, die den gesamten Entwicklungsprozess von der Zielerforschung bis zur Produkteinführung beim Kunden umfasst und neben dem technischen auch den wirtschaftlichen und sozialen Aspekt einer Softwareentwicklung im Team betont. Der Schwerpunkt liegt jedoch eindeutig auf einer ingenieurmäßigen Softwareentwicklung aus der Sicht eines praxiserfahrenen Technikers.

Im Folgenden wird skizziert, wie bestimmte Zielgruppen dieses Buch am effizientesten nutzen können.

Studierende, die sich Wissen über die projektorientierte Softwareentwicklung selbst aneignen wollen

Die spezielle Herausforderung dieses Buchs liegt darin, dass der Inhalt bewusst auch für Studierende gedacht ist, die am Beginn ihres Studiums stehen. Dies bedeutet vielfach, dass noch keine größeren Programmier- und Softwareentwicklungserfahrungen vorhanden sind. Die Entscheidung, den Bereich des Projekt Engineering am Beginn eines (Selbst-)Studiums anzusiedeln, wurde getroffen, da sich in anders aufgebauten Curricula gezeigt hat, dass Studierende (und Softwareentwickler allgemein), wenn sie sich einmal einen bestimmten „privaten“ Softwareentwicklungsprozess angeeignet haben, diesen nur schwer wieder aufgeben und daher eine teamorientierte Softwareentwicklungskultur nur begrenzt akzeptieren.

Studierenden im Selbststudium wird empfohlen, die einzelnen Kapitel sequenziell durchzuarbeiten, wobei Abschnitt II bei Zeitknappheit auch nur auszugsweise durchgegangen werden kann. Abschnitt I, obwohl allgemeiner Natur, sollte vor allem deshalb zu Beginn gelesen werden, um den im Buch neu eingeführten Begriffen, noch mehr aber den im Bereich des Software Engineering traditionell unscharf oder mehrdeutig verwendeten Fachbegriffen eine klare Bedeutung zuweisen zu können. Das in Kapitel 3 und 4 entwickelte mnemonische Schema für einen projektorientierten Softwareentwicklungsprozess ist sehr hilfreich, um für die weiteren Kapitel eine bessere Übersicht zu gewinnen und die Zusammenhänge zu erkennen.

Der prozessorientierte Aufbau des Buches sowie die einheitliche Gliederung aller phasenbezogenen Kapitel (Abschnitt III) ermöglichen es Studierenden mit noch geringer Erfahrung, dieses Buch als „Kochrezept“ für einen ersten, sauberen Projektentwicklungsprozess zu verwenden und diesen dann entsprechend dem Stand der eigenen Erfahrungen zu adaptieren. Der in diesem Buch vorgestellte und durchgehend verwendete Projektentwicklungsprozess kann, gerade für Studierende mit noch geringer Erfahrung in der Entwicklung umfangreicher Softwaresysteme, als Basismodell und Orientierungshilfe bereits zu Beginn ihres Studiums sowie für die ersten eigenen, kleineren Projekte verwendet werden. Mit wachsender Erfahrung und steigendem Einschätzungsvermögen für die Relevanz einzelner Vorgehensschritte kann der vorgestellte Prozess schrittweise abgeändert bzw. erweitert werden und so zu einem individuellen gruppen- bzw. unternehmensspezifischen Softwareentwicklungsprozess umgestaltet werden.

Lehrende, die eine Vorlesung über die teamorientierte Entwicklung von Softwareprojekten vorbereiten bzw. abhalten

Dieses Buch wurde in den letzten zehn Jahren für eine zweisemestrige, zweistündige Lehrveranstaltung (i. e. 30 Einheiten zu je 90 Minuten) für Studierende des Software Engineering an einer technischen Hochschule entwickelt und verwendet. Zusätzlich zu Vortrag und Selbststudium werden die Studierenden mit der Entwicklung eines Übungsprojekts beauftragt und auch aktiv betreut.

Im Rahmen dieses Übungsprojekts dient der Inhalt dieses Buches als Prozessmodell, das auch die grobe Projektgliederung sowie zu erbringende Leistungen festlegt. Als Trennstelle zwischen 1. und 2. Semester hat sich der Abschluss des Kapitels 9 bewährt, wobei Kapitel 8 (Qualitätssicherung) ins 2. Semester zurückgestellt wird. Zu Beginn des 2. Semesters haben die Studierenden in diesem Fall bereits eine klare Zielvorstellung vom Projekt zusammen

mit dem allgemeinen Wissen über einen ingenieurmäßigen Projektentwicklungsprozess. Die konkrete Produkterstellung (Kapitel 10 bis 14) führen sie im Sommersemester durch.

Der Autor bietet den Inhalt dieses Buches auch in gestraffter Form als einsemestrige, zweistündige Lehrveranstaltung an einer weiteren Hochschule an. Aus Gründen der Verfügbarkeit der Studierenden wird diese Lehrveranstaltung als reine Vorlesung ohne Übungsprojekt, jedoch mit großem Diskussionsanteil durchgeführt. Hierbei wird zuerst Abschnitt I in drei Einheiten präsentiert. Da es sich bei den Studierenden dieser Vorlesung durchwegs um höhersemestrige Studierende aus den Bereichen Informatik, Mathematik, Physik etc. handelt, wird Abschnitt II zuerst im Selbststudium erarbeitet und dann gemeinsam mit den Studierenden anhand deren konkreten Erfahrungen in Seminarform durchgesprochen (vier Einheiten). Die Kapitel des Abschnitts III werden dann jeweils von den Studierenden für eine Einheit vorbereitet und vom Vortragenden mit Ersuchen um unmittelbare Diskussion präsentiert. Zielsetzung dabei ist, dass die Studierenden drei Bereiche identifizieren, in denen sie ihr derzeitiges Vorgehen bei der Softwareentwicklung am entscheidendsten verbessern können.

(Software-)Entwickler, die ihr Wissen im Bereich der ingenieurmäßigen Abwicklung von Softwareprojekten sowie des Arbeitens in Projektteams erweitern wollen

Um für die weiteren Kapitel rasch ein klares Verständnis sowie eine bessere Übersicht zu gewinnen, ist es empfehlenswert, die Begriffsdefinitionen zu Beginn von Abschnitt I mit der eigenen Begriffswelt abzugleichen und sich das in Kapitel 3 und 4 entwickelte mnemonische Schema für einen projektorientierten Softwareentwicklungsprozess zu veranschaulichen.

Die Kapitel über Organisation, Information und Qualitätssicherung in Abschnitt II können bei Bedarf gelesen werden, wobei speziell Kapitel 5 über Gruppenführung und -organisation für Personen gedacht ist, die selbst eine Projektgruppe leiten bzw. Mitarbeiter koordinieren müssen.

Jedes Kapitel in Abschnitt III kann, je nach Einsatzschwerpunkt des Softwareentwicklers, separat gelesen werden und soll dadurch eine rasche Erweiterung des Wissens im unmittelbaren Einsatzbereich ermöglichen. Zwecks besseren Verständnisses der Bedeutung des Softwareprojekt-Entwicklungszyklus im Gesamten ist, je nach Verfügbarkeit der Zeit des Lesers, anzustreben, möglichst viele Kapitel des Abschnitts III durchzuarbeiten. Einen wesentlichen Vorteil zur raschen Auffindung benötigter Information sollte die einheitliche Gliederung aller Kapitel des Abschnitts III darstellen. Es ist damit für den Leser mit Bedarf an sehr spezifischer Information rasch möglich, z. B. eine Übersicht über für eine Softwareentwicklung sinnvolle Werkzeuge zu erhalten bzw. eine Aufgabenliste inklusive zu erstellender Dokumente für ein gesamtes Projekt zu generieren.

Eine Bitte an die Leser

Der Autor ist dankbar sowohl für jegliche Kommentare zur Struktur dieses Buchs als auch zur Organisation des Inhalts. Liebe Leserin, lieber Leser, bitte senden Sie mir Ihre Kommentare entweder per E-Mail an *Herwig.Mayr@fh-hagenberg.at*, oder richten Sie Ihre Zuschrift an meine postalische Adresse:

Herwig Mayr
Fachbereich Software Engineering – Projekt Engineering
Fachhochschule Hagenberg
A-4232 Hagenberg, Österreich

Bitte geben Sie an, für welchen Zweck Sie das Buch verwenden und welche Änderungen Ihnen behilflich wären, das Buch besser einzusetzen. Vielen Dank!

Dank

Der primäre Anstoß, eine Vorlesung im Bereich der Entwicklung von Softwareprojekten zu erarbeiten und damit dieses Buch zu schreiben, kam durch die Gruppe für Industrieprojekte am Forschungsinstitut für Symbolisches Rechnen (RISC) der Johannes Kepler Universität Linz, an deren Aufbau ich von 1987 bis 1996 beteiligt war. Ich möchte daher zuallererst dem Gründer und Vorstand des RISC, o.Univ.-Prof. Dr. Dr. h.c. mult. Bruno Buchberger, für die Bereitschaft zum Risiko danken, ein akademisches Institut durch einen Anhang zu bereichern, der wirtschaftlich und industriekonform denken und handeln musste. Ich glaube, wir haben alle sehr viel dabei gelernt.

Die Kombination von akademischen Einrichtungen mit industrienahen Arbeitsgruppen und mit der Zeit hinzugekommenen Unternehmen hat in Hagenberg das Magma gebildet, aus dem sich ein hochqualifizierter Forschungspark mit dem gemeinsamen Leitinhalt „Software“ entwickelt hat. Dies hat nicht zuletzt zur Gründung der Fachhochschule Hagenberg und deren inhaltlichem Schwerpunkt Software Engineering geführt. Für die nimmermüde Aufbauarbeit an der Fachhochschule möchte ich Univ.-Prof. Dr. Witold Jacak danken sowie allen Studierenden der Fachhochschule Hagenberg, die durch ihre kritischen Anmerkungen zu den Lehrunterlagen maßgeblichen Anteil an der Aufbereitung des Inhalts haben. Für die vielen angeregten inhaltlichen Diskussionen möchte ich meinen Kollegen FH-Prof. Dr. Heinz Dobler, FH-Prof. Dipl.-Ing. Hans Heinzlreiter und FH-Prof. Dipl.-Ing. (FH) Peter Kulczycki danken, die meine Vorstellungen sowohl mit akademischem Wissen als auch unbezahlbaren praktischen Erfahrungen schärften.

Besonders erwähnen möchte ich die sehr angenehme und freundschaftliche Atmosphäre in der Zusammenarbeit mit dem Fachbuchverlag Leipzig im Carl Hanser Verlag. Von allen dort unermüdlich an diesem Buchprojekt arbeitenden Mitarbeitern möchte ich insbesondere Frau Renate Roßbach von der Herstellung und Frau Dipl.-Ing. Erika Hotho von der Programmplanung danken. Gerade der von mir gewählte interfakultäre Zugang zum Buchthema mit aber doch deutlich erkennbarer Technikbetonung sowie meine Präferenzen bei der Titelgestaltung haben Frau Hotho sehr viel Engagement und manch verlängerten Arbeitstag abverlangt. Herzlichen Dank für alle guten Ratschläge!

Es ist schwierig, neben einem Vollzeitberuf die Zeit aufzubringen, ein Buch zu schreiben, vor allem wenn man eine Familie hat, auch wenn sie eine höchst verständnisvolle ist wie in meinem Fall. Ich betrachte mich jedoch als eine äußerst glückliche Person, da meine Frau Barbara nicht nur akzeptiert, dass ich vielfach Nächte und sogar Wochenenden mit Arbeit verbringe, sie ist außerdem die beste und genaueste Korrekturleserin der Welt, die ich kenne. Danke, Barbara, für dein Verständnis und für all die Zeit, die du mit dem Korrigieren meiner Sätze aufbrachtest. Nicht zuletzt möchte ich meinen Kindern Bernhard und Marlene dafür danken, dass sie mich vermisst, aber nicht vergessen haben, wenn ich sie auf Grund der Arbeit wieder einmal tagelang nicht gesehen habe.

Danke an euch alle!

Herwig Mayr, Mai 2001

Vorwort zur 2. Auflage

Auch Mitte des ersten Jahrzehnts des neuen Jahrtausends ist im Bereich der organisierten Softwareentwicklung von einer Konsolidierung weit und breit nichts zu spüren. Während der Hype der letzten Jahre, die agile Softwareentwicklung, bereits wieder an Glanz und (selbst auferlegtem) Glamour verliert, taucht mit Model-driven Development eine neues Schlagwort auf, das für heftige Kontroversen bezüglich Problemlösekapazität und zukünftiger Relevanz sorgt. Die Frage, die in einem primär für Weiterbildung und Lehre gedachten Buch gestellt werden muss, ist: Was bleibt? Was wird davon in den allgemeinen Erfahrungsschatz der Softwareentwicklung eingehen? Gerade das im Bereich der agilen Vorgehensmethoden lange Zeit praktizierte Anschwärzen Andersdenkender führte dazu, dass viele Fachleute gegenüber allem Agilen skeptisch blieben, wodurch die ursprünglichen Ziele – und erzielten Fortschritte – nur eine reduzierte Verbreitung bzw. Akzeptanz erlangten.

Die zweite Auflage dieses Buches trägt den aktuellen Strömungen durch zwei neue Kapitel Rechnung (mit dem Fokus auf Prozessen in Kapitel 6 und auf Information in Kapitel 7) und versucht, die Essenz der aktuellen Hypes und derer der letzten Jahre aufzuarbeiten. Dies und der allgemeine Fortschritt der Wissenschaft bedingte natürlich auch ein gründliches Überarbeiten aller anderen Kapitel. Wesentliches Ziel war es, das Buch kompakt zu halten. Durch Kürzungen in weniger relevanten Bereichen konnten daher die notwendigen Ergänzungen beinahe kompensiert werden. Der Bereich des Projekt Engineering wird jedenfalls auch in den nächsten Jahren einer der dynamischsten im Gebiet des Software Engineering bleiben.

Bedanken möchte ich mich für die vielen Rückmeldungen zur ersten Auflage. Ich war sehr überrascht, welch engagierte und konstruktive Kritik eingebracht wurde, die für die Überarbeitung eine große Hilfe darstellte. Bei der Finalisierung des Manuskripts wurde ich seitens des Carl Hanser Verlags in bewährter Weise durch Frau Dipl.-Ing. Erika Hotho und Frau Dipl.-Ing. Franziska Kaufmann unterstützt. Vielen Dank dafür! Auch diesmal gebührt der größte Dank meiner Familie. Die Familie ist Rückhalt und Basis erfolgreicher Arbeit. Ohne Familie gäbe es kein zufriedenes Leben. Danke, Marlene, Bernhard und Barbara!

Herwig Mayr, Mai 2005

Inhaltsverzeichnis

Vorwort	5
I Einführung in die ingenieurmäßige Projektentwicklung	25
1 Grundlegende Begriffe	25
1.1 System	25
1.1.1 Der Systembegriff	25
1.1.2 Systemdeterminanten	26
1.2 Organisation	26
1.2.1 Der Organisationsbegriff	26
1.2.2 Standardisierung, Programmierung und Problemlösung	27
1.2.3 Aufbauorganisation, Ablauforganisation	28
1.3 Projekt	28
1.3.1 Der Projektbegriff	28
1.3.2 Einteilung von Projekten	29
1.3.3 Das Spannungsfeld von Projekten	30
1.4 Software Engineering	32
1.5 Projekt Engineering	34
2 Der Projektentwicklungszyklus	35
2.1 Initialisierung	35
2.2 Angebotslegung und Auftragserteilung	36
2.3 Vorprojekt	37
2.4 Produkterstellung	39
2.4.1 Basisprodukt	39
2.4.2 Adaptierung und Optimierung	40
2.5 Nutzung	40
2.6 Lebensdauer von Software	41
3 Der Produktentwicklungszyklus	43
3.1 Planung	44
3.2 Durchführung	44
3.3 Überprüfung	45
4 Projekt Engineering	47
4.1 Blackbox/Whitebox-Sicht der Projektentwicklung auf die Produktentwicklung	47
4.2 Führung	49
4.2.1 Ziel	49

4.2.2	Aufgaben	50
4.2.3	Arten	50
4.2.4	Situierung der Projektführung	50
4.3	Entwicklung	51
4.3.1	Ziel	51
4.3.2	Aufgaben	51
4.3.3	Arten	52
4.4	Kontrolle	53
4.4.1	Ziel	53
4.4.2	Aufgaben	53
4.4.3	Arten	53
4.4.3.1	Termin- und Ablaufkontrolle	54
4.4.3.2	Leistungskontrolle	54
4.4.3.3	Kosten- und Finanzierungskontrolle	54
4.4.3.4	Integrierte Kontrolle	54
4.5	Schwierigkeiten des Projekt Engineering	55
4.5.1	Schwierigkeiten der Führung	55
4.5.2	Schwierigkeiten der Entwicklung	55
4.5.3	Schwierigkeiten der Kontrolle	56
4.5.4	Reifegrad der ingenieurmäßigen Projektentwicklung	56
4.6	Aufgabenschema des Projekt Engineering	57
II	Projektbegleitende Aufgabenbereiche	59
5	Aufbauorganisation	59
5.1	Allgemeine Organisationsformen	59
5.1.1	Reine Projektorganisation	59
5.1.2	Einfluss-Projektorganisation	60
5.1.3	Matrix-Projektorganisation	61
5.2	Softwareentwicklungsspezifische Organisationsformen	62
5.2.1	Besondere Anforderungen an Projektorganisationsstrukturen	62
5.2.2	Besondere Eigenschaften der Softwareentwicklung	62
5.2.3	Einzelkämpfer – Gruppe – Programmierheer	63
5.2.4	Hierarchische Teamorganisation	63
5.2.5	Das Chefprogrammierer-Team	64
5.2.6	Projektgruppe	66
5.3	Softwareentwicklung als humanzentrierte Aufgabe	67
5.3.1	Gruppendynamische Prozesse	67
5.3.2	Motivation der Mitarbeiter	68
5.3.3	Produktivität	69
5.3.4	Interdisziplinäre Zusammenarbeit	70
5.4	Organisationsmängel bei Softwareprojekten	71
6	Ablauforganisation	73
6.1	Prozessbegriff	73
6.1.1	Der Prozess in der Ablauforganisation	73
6.1.2	Definierter versus empirischer Prozess	74
6.1.3	Entwicklung der Prozessbedeutung	76

6.2	Softwarelebenszyklus und Projektlebenszyklus	77
6.3	Prozessmodelle in der Softwareentwicklung	77
6.3.1	Vorgehensmodelle und Prozessmodelle	77
6.3.2	Prozessmodell-Topologie	79
6.3.3	Prozessskalierung	81
6.4	Vorgehensmethoden	82
6.4.1	Sequenzielles Vorgehen	82
6.4.1.1	Phasenmodell	82
6.4.2	Inkrementelles Vorgehen	83
6.4.2.1	Wasserfallmodell	84
6.4.2.2	Objektorientiertes Modell	85
6.4.3	Iteratives Vorgehen	85
6.4.3.1	Exploratives Prototyping	86
6.4.3.2	Spiralmodell	89
6.4.4	Iterativ-inkrementelles Vorgehen	90
6.4.4.1	Evolutionäres Prototyping	91
6.4.4.2	Schablonenmodell	93
6.5	Prozessoptimierung	93
6.5.1	Prozessidentifikation: Bedeutung des Messens	94
6.5.2	Prozessverständnis: Feedback durch kurze Iterationszyklen	94
6.5.3	Prozessökonomie: Agiles Vorgehen	96
6.5.4	Prozessverbesserung: Bewährte Praktiken im 21. Jahrhundert	99
7	Informationsaustausch	101
7.1	Kommunikation in Softwareprojekten	101
7.1.1	Kundeneinbindung	102
7.1.2	Besprechung	103
7.1.3	Review	104
7.1.4	Werkzeugeinsatz	106
7.2	Dokumentation von Softwareprojekten	108
7.2.1	Dokumentationsplanung	108
7.2.2	Dokumentationsorganisation	110
7.2.3	Konfigurations- und Versionsverwaltung	110
7.2.4	Dokumentablage	111
7.3	Dokumente der Aufbauorganisation	112
7.3.1	Organigramm	112
7.3.2	Stellenbeschreibung	112
7.4	Dokumente der Ablauforganisation	115
7.4.1	Grundlegende Dokumente	115
7.4.2	Projekthandbuch	117
7.4.3	Projekttagebuch	118
7.4.4	Projektfortschrittsbericht	118
7.4.5	Arbeitsbericht	119
7.4.6	Projektbibliothek	119
8	Qualitätssicherung	121
8.1	Ziel	121
8.1.1	Qualität von Software	121
8.1.2	Qualitätsmerkmale von Software	121

8.1.2.1	Korrektheit	122
8.1.2.2	Zuverlässigkeit	122
8.1.2.3	Benutzerfreundlichkeit	122
8.1.2.4	Wartungsfreundlichkeit	124
8.1.2.5	Effizienz	124
8.1.2.6	Portabilität	125
8.1.3	Qualität in der Produktentwicklung	125
8.1.4	Qualitätsgesteuerter Softwareentwicklungsprozess	126
8.1.5	Qualitätssicherung	127
8.2	Aufgaben	127
8.2.1	Qualitätsplanung: Festlegen der Qualitätsmerkmale	127
8.2.2	Qualitätsprüfung: Prüfen der Projektgrundlagen	127
8.2.3	Qualitätslenkung: Maßnahmen zur Qualitätssicherung	127
8.2.3.1	Organisatorische Maßnahmen	127
8.2.3.2	Konstruktive Maßnahmen	128
8.2.3.3	Analytische Maßnahmen	128
8.3	Techniken	128
8.3.1	ISO 90003:2004	128
8.3.2	Das Capability Maturity Model Integration (CMMI)	129
8.3.3	BOOTSTRAP	131
8.3.4	ISO/IEC 15504 (SPICE)	132
8.4	Ergebnisse	132
8.4.1	Qualitätshandbuch	132
8.5	Normen und Richtlinien	132
8.5.1	Grundlegende Bedeutung von Normen und Richtlinien	132
8.5.2	Normungsgremien	133
8.5.2.1	Gremien für Normen	133
8.5.2.2	Gremien für Industriestandards	134
8.5.2.3	Gremien für Richtlinien	134
8.5.3	Normen und Richtlinien in der Softwareentwicklung	135
8.5.4	Normenauswahl	135

III Projektphasenspezifische Aufgabenbereiche 137

9 Zielerforschung 137

9.1	Erhebung	138
9.1.1	Ziel	138
9.1.2	Aufgaben	138
9.1.3	Techniken	139
9.1.3.1	Einteilung	139
9.1.3.2	Interview	139
9.1.3.3	Fragebogen	141
9.1.3.4	Dokumentausrwertung	143
9.1.4	Ergebnisse	144
9.1.4.1	Erhebungsausrwertung	144
9.2	Zielfindung	145
9.2.1	Ziel	145
9.2.2	Aufgaben	145

9.2.3	Techniken	145
9.2.3.1	Organisatorische Voraussetzungen für Kreativitätstechniken .	146
9.2.3.2	Brainstorming	147
9.2.3.3	Brainstorming mit Osborne-Verfremdung	148
9.2.3.4	Synektik	148
9.2.3.5	Morphologischer Kasten	150
9.2.4	Werkzeuge	151
9.2.4.1	Groupware	151
9.2.4.2	Die Unified Modeling Language (UML)	153
9.2.4.3	Diagramme der UML für die Zielerforschung	154
9.2.4.4	Vorgehen bei der Zielerforschung unter Verwendung der UML	155
9.2.5	Ergebnisse	157
9.2.5.1	Zielbeschreibung	157
9.2.5.2	Datenlexikon	157
10	Analyse	159
10.1	Anforderungsanalyse	159
10.1.1	Ziel	159
10.1.2	Aufgaben	160
10.1.2.1	Tätigkeiten	160
10.1.2.2	Eigenschaften	160
10.1.2.3	Schwierigkeiten	161
10.1.3	Techniken	162
10.1.3.1	Das Beherrschen der Komplexität	162
10.1.3.2	Zerlegungstechniken	164
10.1.3.3	Erstellen von Prototypen	171
10.1.3.4	Sprechdisziplin	173
10.1.4	Werkzeuge	174
10.1.4.1	Analysewerkzeuge	174
10.1.4.2	Diagramme der UML für die Analyse	176
10.1.4.3	Vorgehen bei der Analyse unter Verwendung der UML	176
10.1.4.4	Werkzeuge des Prototyping	177
10.1.5	Ergebnisse	180
10.1.5.1	Analysebericht	180
10.1.5.2	Lastenheft	180
10.1.5.3	Machbarkeitsstudie	182
10.2	Planung	183
10.2.1	Ziel	183
10.2.2	Aufgaben	183
10.2.2.1	Ziel- und Aufgabenplanung	183
10.2.2.2	Zeitplanung	184
10.2.2.3	Ablauf- und Terminplanung	184
10.2.2.4	Ressourcenplanung	185
10.2.2.5	Kosten- und Finanzierungsplanung	186
10.2.2.6	Begleitende und vorsorgende Maßnahmen	186
10.2.3	Techniken	187
10.2.3.1	Aufwandsschätzung	187
10.2.3.2	Netzplantechnik	187

10.2.3.3	Balkenplantechnik	189
10.2.4	Werkzeuge	189
10.2.5	Ergebnisse	191
10.3	Risikobehandlung	191
10.3.1	Ziel	191
10.3.1.1	Risiko und Projektrisiko	191
10.3.1.2	Risikoeigenschaften	192
10.3.1.3	Die zehn wichtigsten Risiken bei Softwareprojekten	193
10.3.1.4	Risikobehandlung	193
10.3.1.5	Situierung der Risikobehandlung in der Projektentwicklung	195
10.3.2	Aufgaben	196
10.3.2.1	Risiko-Identifikation	196
10.3.2.2	Risiko-Analyse	198
10.3.2.3	Risiko-Priorisierung	198
10.3.2.4	Risiko-Vorsorgeplanung	199
10.3.2.5	Risiko-Überwachung	199
10.3.2.6	Risiko-Überwindung	199
10.3.3	Techniken	200
10.3.3.1	Abschätzen des Risikoverhaltens	200
10.3.3.2	Risiko-Vermeidung	201
10.3.3.3	Risiko-Reduktion	201
10.3.3.4	Risiko-Annahme	201
10.3.3.5	Risiko-Übertragung	202
10.3.3.6	Verwendung von Standardprodukten	202
10.3.3.7	Behandlung der wichtigsten Risiken bei Softwareprojekten	203
10.3.4	Werkzeuge	205
10.3.5	Ergebnisse	206
10.3.5.1	Top-N-Liste der Risikoverfolgung	206
10.3.5.2	Risiko-Vorsorgeplan	207
11	Design	209
11.1	Ziel	209
11.2	Aufgaben	209
11.2.1	Systemdesign	210
11.2.1.1	Modellierung der Architektur	211
11.2.1.2	Modellierung der Schnittstellen	211
11.2.1.3	Modellierung der Datenhaltung	212
11.2.2	Komponentendesign	213
11.2.2.1	Modellierung der Komponentenstruktur	213
11.2.2.2	Modellierung des Komponentenverhaltens	214
11.3	Techniken	214
11.3.1	Erweitern der Analysemodelle	214
11.3.2	Erstellen von Prototypen	216
11.3.3	Modellgesteuertes Entwickeln	217
11.3.4	Anwenderseitiges Entwickeln	218
11.3.5	Wiederverwendung	219
11.3.5.1	Wiederverwenden von Bausteinen	220
11.3.5.2	Verwenden von Mustern	220

11.3.5.3	Abwandeln von Schablonen	221
11.3.5.4	Konkretisieren von Systemrahmen	221
11.3.6	Empfehlungen für ein gutes Design	222
11.4	Werkzeuge	223
11.4.1	Computergestützte Designwerkzeuge	223
11.4.2	Diagramme der UML für das Design	224
11.4.3	Vorgehen beim Design unter Verwendung der UML	225
11.4.4	Werkzeuge des Prototyping	226
11.4.4.1	Beschreibungssprachen für Benutzerschnittstellen	226
11.4.4.2	Grafische Generatoren zur Erstellung von Benutzerschnittstellen	226
11.4.5	Werkzeuge des Compilerbaus	227
11.5	Ergebnisse	227
11.5.1	Designbericht	227
11.5.2	Pflichtenheft	228
11.5.3	Benutzerdokumentation	229
12	Implementierung	233
12.1	Codieren	233
12.1.1	Ziel	233
12.1.2	Aufgaben	233
12.1.3	Techniken	234
12.1.3.1	Transformieren der Designmodelle	234
12.1.3.2	Instrumentieren des Codes	235
12.1.3.3	Einfügen logischer Bedingungen	236
12.1.3.4	Code-Restrukturierung	237
12.1.3.5	Erstellen von Testrahmen	238
12.1.3.6	Testgesteuerte Entwicklung	239
12.1.3.7	Kontinuierliche Integration	241
12.1.4	Werkzeuge	241
12.1.5	Ergebnisse	243
12.1.5.1	Quellcode	243
12.1.5.2	Systemdokumentation	243
12.2	Debuggen	245
12.2.1	Ziel	245
12.2.2	Aufgaben	246
12.2.3	Techniken	247
12.2.3.1	Statische Programmanalyse	247
12.2.3.2	Dynamische Programmanalyse	251
12.2.3.3	Effizienz der Techniken beim Finden von Fehlern	253
12.2.4	Werkzeuge	253
12.2.4.1	Statische Analysatoren	253
12.2.4.2	Listengeneratoren	253
12.2.4.3	Speicherprüfer	254
12.2.4.4	Testrahmensysteme	254
12.2.4.5	Debugger	255
12.2.5	Ergebnisse	256
12.2.5.1	Lauffähiges Programm	256

12.2.5.2 Fehlerliste	256
13 Testen	257
13.1 Ziel	257
13.2 Aufgaben	259
13.2.1 Erkennen von Analyse- und Designfehlern	259
13.2.2 Erkennen von Implementierungsfehlern	259
13.2.3 Bestimmen des Produktstatus	259
13.2.3.1 Alphatesten	259
13.2.3.2 Betatesten	260
13.2.4 Performanztest – Testen der Produktperformanz	260
13.2.5 Abnahmevortest – Vorbereiten der Abnahme	261
13.3 Techniken	262
13.3.1 Testfalldesign	262
13.3.2 Testsuitesdesign	263
13.3.2.1 Erstellen von Testfällen	263
13.3.2.2 Auswahl von Testfällen	264
13.3.3 Testablaufplanung	264
13.3.3.1 Testvorbereitung	264
13.3.3.2 Qualifikationstest	265
13.3.3.3 Testzyklus	265
13.3.4 Verifikation – Testen der Funktionalität	267
13.3.5 Validierung – Testen des Gesamtsystems	268
13.3.6 Regressionstesten	269
13.3.7 Testautomatisierung	270
13.4 Werkzeuge	271
13.5 Ergebnisse	273
13.5.1 Testplan	273
13.5.2 Testsuite	274
13.5.3 Testlisten	275
13.5.4 Fehlerbericht	275
13.5.5 Fehlerlogbuch	276
14 Produkt-Einführung	277
14.1 Ziel	277
14.2 Aufgaben	277
14.2.1 Lieferung	277
14.2.2 Installation und Inbetriebnahme	279
14.2.3 Schulung	279
14.2.4 Abnahme und Abschluss	280
14.2.5 Wartung	281
14.2.6 Evaluierung	285
14.2.7 Archivierung	286
14.3 Techniken	286
14.3.1 Umstellung	286
14.3.2 Probetrieb	287
14.3.3 Kontinuierliche Freigabe	288
14.3.4 Abnahmetest	289
14.3.5 Nachkalkulation	290

14.4	Werkzeuge	290
14.5	Ergebnisse	291
14.5.1	Betriebsversion	291
14.5.2	Installations- und Inbetriebnahmeprotokoll	291
14.5.3	Abnahmeprotokoll	292
14.5.4	Abschlussbericht	292
14.5.5	Projektarchiv	293
Literaturverzeichnis		294
Index		305

Abbildungsverzeichnis

1.1	Die Organisation als soziales System	26
1.2	Das traditionelle Spannungsfeld von Projekten	31
1.3	Das moderne Spannungsfeld von Projekten	31
2.1	Die Sättigungskurve der Weiterentwicklung in der Technik	41
3.1	Grundkomponenten der Produktentwicklung	43
4.1	Blackbox/Whitebox-Sicht der Projektentwicklung auf die Produktentwicklung	48
4.2	Technische und kaufmännische Sichtweise der Projektführung	51
4.3	Entwicklung der Erfolgsstatistik von Softwareprojekten in den USA	57
4.4	Aufgabenschema des Projekt Engineering	58
5.1	Ermittlung der (theoretisch) optimalen Gruppengröße bei Projekten	70
6.1	Geregelter Prozess aus technischer Sicht	74
6.2	Die Suche nach der goldenen Mitte	78
6.3	Komponenten eines modellbasierten Vorgehens	79
6.4	Klassifizierung der Prozessmodelle nach Gültigkeitsbereichen	80
6.5	Topologie der Prozessmodelle	80
6.6	Das Wasserfallmodell	84
6.7	Vorgehen nach dem objektorientierten Modell	86
6.8	Grundschrte einer Iteration des kontinuierlichen Verbesserungsprinzips . . .	86
6.9	Fehlerbehebungskosten im Laufe der Projektentwicklung	87
6.10	Das Spiralmodell	90
6.11	Das Twin-Peaks-Modell zur Visualisierung des dynamischen Zusammenhangs Anforderungen – Produkt	91
6.12	Der Zusammenhang von Iterationsgeschwindigkeit und Inkrementgröße	95
6.13	Geschwindigkeit versus Inkrementgröße in traditionellen und agilen Methoden	95
6.14	Spektrum der Bedeutung der Gesamtplanung im Projekt Engineering	97
7.1	Entwicklung der Zusammensetzung von Review-Teams	105
7.2	Essenzielle Dokumente bei traditionellem und bei agilem Vorgehen	109
7.3	Beispielorganigramm einer Projektgruppe	113
7.4	Checkliste für die grundlegenden Dokumente	115
8.1	Wechselwirkungen zwischen Qualitätsmerkmalen und ihre Wirkung auf Funk- tionalität, Kosten und Zeit	126
8.2	Grundlagen und Auswertungsmöglichkeiten der BOOTSTRAP-Methode . . .	131
9.1	Die Osborne-Verfremdung im Rahmen des Brainstorming	149

9.2	Modelle der UML	154
9.3	Komponenten eines Anwendungsfalldiagramms in UML-Darstellung	156
10.1	Mächtigkeit der Zerlegungstechniken in Bezug auf die Beherrschung der Komplexität	164
10.2	CRC-Karte	175
10.3	Haftnotizen-Objekt	175
10.4	Diagramme der UML für die Analyse	176
10.5	Die zehn wichtigsten Risiken bei Softwareprojekten Mitte der achtziger Jahre und Mitte der neunziger Jahre	194
10.6	Aufgabenschritte der Risikobehandlung	197
11.1	Diagramme der UML für das Design	224
13.1	Zuordnung der Testaktivitäten zu den Entwicklungsaktivitäten	269
14.1	Ergänzung des Projektentwicklungszyklus um Betrieb und Wartung	282

Teil I

Einführung in die ingenieurmäßige Projektentwicklung

1 Grundlegende Begriffe

In diesem Kapitel werden zuerst die grundlegenden Begriffe definiert bzw. bei mit mehreren Bedeutungen überlagerten Begriffen festgelegt, in welcher Bedeutung sie im Folgenden verwendet werden. Dies ist gerade im Fachgebiet des Software Engineering, in dem so wesentliche Begriffe wie Objekt und Komponente in einer Vielzahl von Bedeutungen (oft im gleichen Kontext) Verwendung finden und selbst über die Bedeutung (und manchmal sogar Sinnhaftigkeit) des Begriffs „Software Engineering“ in der Fachwelt mehr als 30 Jahre nach seiner Einführung noch heftig diskutiert wird, besonders angebracht.

1.1 System

1.1.1 Der Systembegriff

System stammt aus dem Griechischen und bedeutet „ein aus mehreren Teilen zusammengesetztes Ganzes“. Darauf aufbauend lässt sich System definieren als „Bezeichnung für jede Gesamtheit von Objekten, die sich in einem ganzheitlichen Zusammenhang befinden, wobei ihre Wechselwirkungen untereinander diejenigen mit der Umwelt im Allgemeinen stark überwiegen, sodass sie als ein von der Umwelt mehr oder weniger unabhängiges Ganzes (mit einer durch ihre Eigenschaften und Wechselwirkungen festgelegten Struktur) behandelt werden können“ [Meyers, 2004].

Ein System zeichnet sich daher durch die ganzheitliche Betrachtungsweise aus, aufbauend auf dem Grundsatz, dass das Ganze mehr ist als die Summe seiner Teile.

Ein System kann horizontal unterteilt werden in *Teilsysteme*, welche die Übersicht eines Systems unter bestimmten Gesichtspunkten (Aspekten) darstellen. Eine vertikale Unterteilung in *Untersysteme* gliedert ein System in – mehr oder weniger in sich geschlossene – Subkomponenten.

Eine Abbildung eines Systems im Computer wird oft als *Modell* (*model*) bezeichnet, die Darstellung eines Teilsystems als *Sicht* (*view*) auf das Modell. Die Kopplung von Untersystemen zum Gesamtsystem erfolgt via festgelegter *Schnittstellen* (*interfaces*).

1.1.2 Systemdeterminanten

Ein System wird bestimmt durch

- Systemziele,
- Systemelemente (Aufgaben, Menschen, Sachmittel, Information),
- Systembeziehungen (externe Beziehungen, interne Beziehungen),
- Systemverhalten (mit Beurteilungsmaßstäben für die Quantifizierung der Systemziele).

1.2 Organisation

1.2.1 Der Organisationsbegriff

In [Kieser und Kubicek, 1983] wird *Organisation* definiert als „soziales Gebilde, das dauerhaft ein Ziel verfolgt und eine formale Struktur aufweist, mit deren Hilfe die Aktivitäten der Mitglieder auf das verfolgte Ziel ausgerichtet werden sollen“. Eine Organisation ist also ein soziales System (Abbildung 1.1).

System	Organisation
Ziele	Ziel (dauerhaft)
Elemente	Mitglieder
Beziehungen	formale Struktur
Verhalten	Aktivitäten (der Mitglieder)

Abbildung 1.1: Die Organisation als soziales System

Die *Systemdeterminanten der Organisation* ermöglichen eine Anpassung an die jeweilige Realität:

- Unter dem *Ziel einer Organisation* versteht man die Vorstellung von dem zukünftigen Zustand, den die Organisation herzustellen oder zu erhalten sucht. Es ist also das Organisationsziel, das die Verhaltensmaximen für die Organisationsmitglieder bildet. Die

Organisationsstruktur ist ein Mittel, um dieses Ziel in konkretere Verhaltenserwartungen zu übersetzen.

- Ziele sind ursächlich persönliche Zielvorstellungen der Mitglieder, die sie durch die Organisation für sich erfüllt sehen möchten. Erst wenn die Zielvorstellungen von Mitgliedern in einem formalen, legitimierten Prozess als Ziele der Organisation deklariert werden, kann man von Zielen der Organisation sprechen.
- Die Zielgerichtetheit von Organisationen muss „auf Dauer“ angelegt sein. Die dauerhafte Verfolgung von Zielen bedeutet, dass die Organisationsziele in der Regel nicht davon beeinflusst werden, ob die Mitglieder an einzelnen Stellen ausgetauscht werden. Eine wesentliche Aufgabe der Organisationsstruktur besteht gerade darin, eine solche dauerhafte Zielverfolgung auch bei personellem Wechsel sicherzustellen.
- Andererseits bedeutet „auf Dauer angelegt“ nicht, dass sich Organisationen nicht ändern. Organisationsziele können sich ändern, und selbst bei unveränderten Zielen muss eine Organisation in einer sich wandelnden Umwelt oft verändert werden, um ihren Bestand zu sichern.

Der hier eingeführte Organisationsbegriff wird allgemein als *instrumentale Organisation* bezeichnet, vgl. z. B. [Grochla, 1982], da Organisation als Instrument zur Zielerreichung von Systemen verstanden wird. Organisation fokussiert also eine Tätigkeit, die als Ergebnis eine Vorgabe für die weiteren Tätigkeiten aufweist, wobei vielfach auch dieses Ergebnis mit dem Term Organisation synonymisiert wird. Im Gegensatz dazu steht der *institutionale Organisationsbegriff*, der das gesamte organisierte Gebilde mit dem Begriff „Organisation“ bezeichnet und vor allem in der Soziologie verbreitet ist. Im Folgenden verstehen wir unter „Organisation“ immer die instrumentale Organisation, sofern nicht ausdrücklich anderes angegeben wird.

1.2.2 Standardisierung, Programmierung und Problemlösung

Unter *Standardisierung* versteht man das antizipierende Durchdenken von Problemlösungswegen und die darauf aufbauende Festlegung von Aktivitätsfolgen, sodass diese im Wiederholungsfall mehr oder weniger routiniert und gleichartig ablaufen [Grochla, 1982].

Die Anzahl möglicher Verhaltensweisen, welche bei der Abwicklung eines bestimmten Prozesses zugelassen sind, wird *Varietät* genannt. Als *Programmierung* wird die Strukturierung von Aktivitätsfolgen bezeichnet, sodass zwischen der Tatsache der Aktivitätsstrukturierung an sich und der Standardisierung der Aktivitätsfolgen unterschieden werden kann.

Steht einer Person für die Beantwortung eines Reizes ein spezielles Programm zur Verfügung, so verhält sie sich *routiniert*; kann sie hingegen einen Reiz nicht als bekannt identifizieren (und muss, sofern vorhanden, auf allgemeine Programme zurückgreifen), so verhält sie sich *problemlösend*.

Ziel einer guten Organisation muss es sein, die Mitglieder im Hinblick auf die Aufgaben mit möglichst viel Routine zu versehen (durch Standardisierung), ohne dadurch die Kapazität der Problemlösung zu reduzieren, ja diese durch die Routinisierung sogar noch zu steigern.

1.2.3 Aufbauorganisation, Ablauforganisation

Die Aufgabe der *Aufbauorganisation* ist die Strukturierung von Systemen. Die Systemelemente und deren Beziehungen stehen im Vordergrund. Die *Ablauforganisation* befasst sich mit den Systembeziehungen und den dadurch bedingten Tätigkeiten und Veränderungen („Prozessen“) in Organisationen.

Für eine detailliertere Darstellung wird auf Kapitel 5 und Kapitel 6 verwiesen.

1.3 Projekt

1.3.1 Der Projektbegriff

Ein *Projekt* ist ein innovatives, komplexes Vorhaben mit definierten Randbedingungen wie Termine, Kosten, Ressourcen [DIN, 2000].

Projekte weisen im Allgemeinen folgende Gemeinsamkeiten auf (vgl. [Rausch-Schott und Retschitzegger, 1993]):

- innovative Aufgabenkomplexe,
- zeitlich abgegrenzte und komplexe Vorhaben,
- vorgegebenes Ziel bzw. vorgegebene Leistung,
- keine reine Wiederholung früherer Arbeiten,
- begrenzte verfügbare Mittel,
- im Voraus geplante oder festgelegte Kosten bzw. Budget,
- mehrere beteiligte Stellen, Zusammenarbeit erforderlich.

Projekte werden nach folgenden Eigenschaften klassifiziert und beurteilt:

- **Projektumfang:** Anzahl der Einzelaufgaben, Anzahl der beteiligten Personen/Abteilungen, Auftragswert, einzusetzende Mittel (Geld, Ressourcen).
- **Projektdauer:** Projekte sind auf ein bestimmtes Ziel ausgerichtet. Der Projektbeginn kann durch Ereignisse bestimmt werden (z.B. Auftragserteilung, Zeitpunkt der Ressourcenbewilligung). Mit dem Erreichen des Projektzieles ist der Projektzweck erfüllt und das Projekt beendet.
- **Projektbesonderheit:** keine Routinetätigkeiten, sondern Neuartigkeit oder Einmaligkeit des Vorhabens. Je geringer die Vertrautheit mit der Aufgabe ist, desto größer ist die Gefahr, dass die Ziele nicht befriedigend erreicht werden.
- **Projektkomplexität:** Anzahl und Verschiedenartigkeit der Projektteilaufgaben sowie ihre Verflechtung und gegenseitige Abhängigkeit. Einflussfaktoren sind der wissenschaftliche Neuheitsgrad, die Anzahl der beteiligten Organisationseinheiten und Fremdfirmen, die Anzahl der Abhängigkeiten zwischen den Einzelaufgaben sowie das Risiko der Zielerreichung.

- **Projektschwierigkeit:** Die Schwierigkeit des Problems ergibt das Projektrisiko; Einflussfaktoren für die Schwierigkeit sind technischer, wirtschaftlicher, organisatorischer, personeller oder sozialer Natur. Die Schwierigkeit von Projekten ist z. B. hoch bei knappem Budget, großen Widerständen der betroffenen Interessengruppen, unrealistischen Terminen oder beim Fehlen der nötigen Fachkenntnisse.
- **Projektbedeutung:** Die Bedeutung ist der Einfluss des Projekts auf Teil- oder Gesamtziele des Unternehmens. Auch Projekte geringen Umfangs können große Bedeutung haben, z. B. organisatorische Veränderungen, strategische Maßnahmen.
- **Projektrisiko:** Das Risiko ist Maß für die Höhe des Schadens für ein Unternehmen, wenn das Projektziel nicht erreicht wird. Teilrisiken sind z. B. das technische Realisierungsrisiko (kein Projektergebnis), Verwertbarkeitsrisiko (nicht verwertbares geplantes Ergebnis), Zeitrisiko (Termine nicht eingehalten), Aufwandsrisiko (Kosten oder Mittel überschritten) oder das Zielsetzungsrisiko (Produkt entspricht nicht den Anforderungen des Auftraggebers).
- **Projektkosten:** Diese müssen neben Zeit und Mittel meistens bereits im Planungsstadium quantifiziert werden; dies ist durch die Neuartigkeit oder Einmaligkeit des Projekts oft sehr schwierig.
- **Projektkontinuität:** Die Organisation des Projekts muss auf die Projektdauer ausgerichtet sein. Demgegenüber können Projektmitarbeiter kontinuierlich (hauptberuflich), variabel (Teilzeit) oder sporadisch mit der Projektarbeit befasst sein.
- **Projektintensität:** Diese charakterisiert die Beanspruchung der Mitarbeiter. Mitarbeiter können ausgelastet, unterbeschäftigt oder überlastet sein.
- **Projektanzahl:** Bei Multiprojekten oder großen Innovationsschüben werden mehrere Projekte gleichzeitig durchgeführt; die Belastung des Unternehmens durch nicht routinemäßige Aufgaben ist dementsprechend erhöht.
- **Organisations- und Führungsverständnis:** Dieses ist notwendig für eine moderne Unternehmensstruktur in Bezug auf Kooperationsbereitschaft, Konfliktlösungsfähigkeit sowie Flexibilität der Mitarbeiter.

1.3.2 Einteilung von Projekten

Die Einteilung von Projekten erfolgt einerseits nach der *Stellung des Auftraggebers*:

- **Internes Projekt:** Auftraggeber ist die Unternehmensleitung. Das Projektziel ist die Erneuerung bzw. Verbesserung des Leistungspotenzials des Unternehmens.
- **Externes Projekt:** Auftraggeber ist ein externer Kunde. Das Projektziel ist eine Marktleistung.

Andererseits können Projekte nach der *Art der Projektkooperation* von Auftraggeber und Auftragnehmer eingeteilt werden:

- **Consulting, Coaching:** Ziel ist die Projektbegleitung des Kunden; der Auftragnehmer ist Berater bzw. Fachexperte, der die Projektgruppe des Kunden, im Allgemeinen ohne Führungsaufgabe, verstärkt.
- **Turn-Key-Entwicklung:** Ziel ist die Erstellung eines Produkts durch den Auftragnehmer, das der Auftraggeber ohne zusätzlichen Aufwand einsetzen kann. Wesentliche Bedeutung wird dem Liefertermin des Produkts zugemessen. Voraussetzung dafür ist eine klare Aufgabenstellung sowie die Festlegung der Überprüfbarkeit des Zielerfüllungsgrades. Im Allgemeinen sind die Aufgaben des Auftraggebers und des Auftragnehmers klar abgegrenzt.
- **Joint Venture:** Ziel dieser „gemeinsamen Entwicklung“ ist eine Produkterstellung gemeinsam durch Auftraggeber und Auftragnehmer. Es kann also keinen festgelegten Liefertermin geben, wohl aber ein festgelegtes Projektende, für das beide Seiten verantwortlich sind. Im Gegensatz zum Consulting trifft hier auch den Auftragnehmer ein Teil der Projektverantwortung.

1.3.3 Das Spannungsfeld von Projekten

Betriebswirtschaftliche Sichtweise

Aus *betriebswirtschaftlicher Sicht* wird ein Projekt oft unter „verwaltungstechnischen“ Gesichtspunkten gesehen. Es soll – mit welchen Mitteln auch immer – das Projekt bei (vorab) genau definierter Funktionalität in einem Gleichgewicht zwischen verfügbarem Kostenrahmen, vorgegebener Zeit und zu erzielender Qualität des Produkts gehalten werden, vgl. Abbildung 1.2 [Elzer, 1994]. Die Mittel, die zur Erreichung dieses Zieles empfohlen werden, sind dementsprechend rein verwaltungstechnischer Art: Netzpläne, Kostenschätzverfahren, Soll-Ist-Vergleiche zwischen vorgegebenem und geleistetem Aufwand etc.

Technische Sichtweise

Aus *technischer Sicht* wird dagegen den Managementproblemen wenig Raum gewidmet. Der Erfolg eines Projekts wird hier fast ausschließlich von den technischen Eigenschaften her beurteilt. Ein entsprechendes Spannungsfeld der technischen Projektleitung besteht darin, dass die Lösung möglichst guten „Anwendungsbezug“ aufweisen muss, „flexibel“ und „wartbar“ sein muss und vor allem die Realisierung dem aktuellen Stand der Technik entspricht. Daraus ergibt sich in der Praxis, dass die gewünschte Funktionalität eines Produkts nicht (einmal annähernd) vollständig vorab festgelegt werden kann, sondern im Laufe der Entwicklung erkannt wird und auch häufig abgeändert wird („*evolvierende Funktionalität*“). Diese Erweiterung um die Variable „Funktionalität“ führt zum *modernen Spannungsfeld* von Projekten, vgl. Abbildung 1.3.

Das Integrieren evolvierender Funktionalität wird besonders von „agilen“ Ansätzen zur Softwareentwicklung zum Grundprinzip ernannt. Während ingenieurmäßige Ansätze immer danach trachten müssen, die Anforderungen zu stabilisieren [Fowler, 1999], sind Funktionalitätsänderungen und -erweiterungen in der Praxis unvermeidlich und daher im Prozess explizit vorzusehen („embrace change“, [Beck und Andres, 2005]).

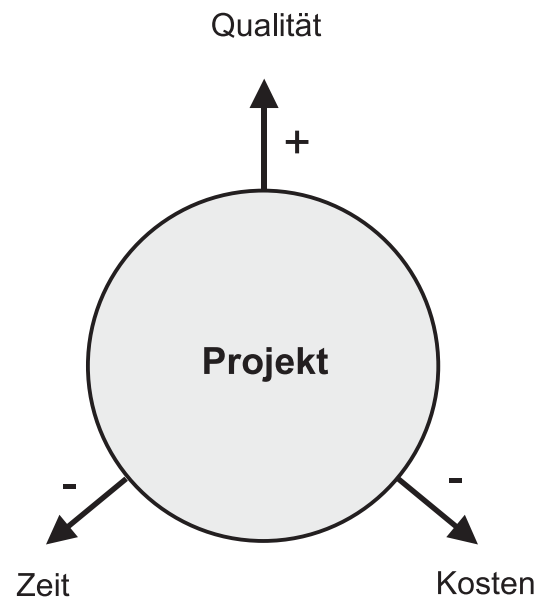


Abbildung 1.2: Das traditionelle Spannungsfeld von Projekten

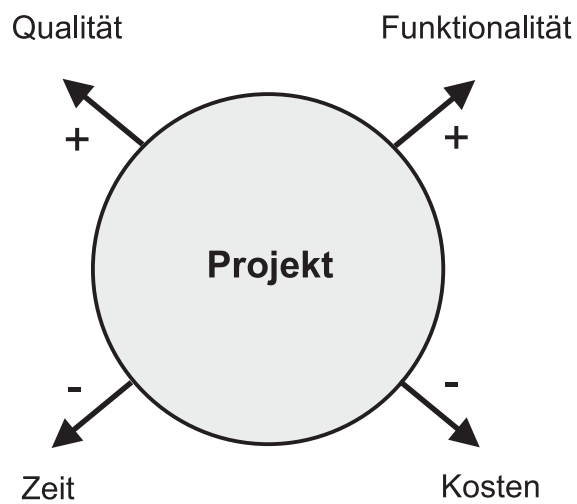


Abbildung 1.3: Das moderne Spannungsfeld von Projekten

[Alleman, 2002b] merkt hierzu jedoch an, dass „sich ändernde Anforderungen auch ein Indiz für sich ändernde Businesswerte, instabile Anforderungen oder fehlendes Problemverständnis sein können“. Software für derart unsichere Anforderungsszenarien zu entwickeln stellt vielfach ein unabwägbares Risiko dar. Entscheidend ist daher im Rahmen der Risikobehandlung, zu untersuchen, *warum* sich die Anforderungen ändern, sowie die Bereiche zu definieren, bei denen die größte Flexibilität bei der Anforderungsdefinition nötig ist [Alleman, 2002b].

Viele agile Vorgehensmethoden begegnen diesem Problem, indem explizit festgehalten wird, dass dem Kunden nicht ein definiertes *Produkt* (also Funktionalität) verkauft wird, sondern ein bestimmter *Umfang an Entwicklungsleistung* (vgl. z. B. [Highsmith, 2002], [Beck und McBreen, 2002]). Dieses Vorgehen entspricht dem einer Automobilwerkstätte, der ein PKW zur Reparatur übergeben wird und die nach acht Stunden Arbeit den PKW zwar nicht wieder zum Laufen gebracht hat, aber (wegen des redlichen Bemühens) die Arbeitszeit trotzdem in Rechnung stellt. Ob daher dieser „Lösungs-“ Ansatz mancher Verfechter des agilen Vorgehens von allen Auftraggebern akzeptiert wird, ist mehr als zweifelhaft.

Soziologische Sichtweise

Aus *soziologischer Sicht* werden die psychologischen Aspekte als hauptsächlich ausschlaggebend dargestellt und der Projektleiter dahingehend ausgebildet, wie man die notwendige Leistung mit einem Team erreichen kann, in dem für die einzelnen Teammitglieder das richtige Gleichgewicht zwischen „Teamgeist“ und „Selbstverwirklichung“ herrscht.

Wie [Elzer, 1994] anmerkt, ist die so genannte „*Softwarekrise*“, ein seit Jahren viel strapazierter Begriff für das in der Softwareentwicklung noch weit verbreitete Unvermögen, Projekte nichttrivialer Größenordnung erfolgreich abzuwickeln, hauptsächlich durch eine einseitige Denkweise entstanden. Projekte werden ausschließlich unter einem einzigen der oben genannten Gesichtspunkte gesehen, und zwar dem, der dem jeweiligen Projektleiter – oder seinen Vorgesetzten – gemäß seiner/deren Vorbildung am vertrautesten ist. In jüngerer Zeit setzt sich aber doch mehr und mehr die Einsicht durch, dass zur erfolgreichen Durchführung von Softwareentwicklungsprojekten alle genannten Aspekte gleichermaßen berücksichtigt werden müssen.

Es ist entscheidend, dem Kunden dieses *Spannungsfeld eines Projekts* von Beginn an vor Augen zu führen und ihn auf die Auswirkungen von Funktionalitäts- oder Terminänderungen, Kostenkürzungen etc. hinzuweisen. Dies kann vielfach nicht drastisch genug geschehen. Gerade bei Softwareprojekten, die oft auf Basis vager Anforderungen und unter hohem Termindruck abgewickelt werden müssen, geht der – einfache – Zusammenhang Kosten–Zeit–Funktionalität–Qualität oft im Trubel der Entwicklung unter.

1.4 Software Engineering

Der Begriff *Software Engineering* wurde auf einer NATO-Konferenz geprägt, die im Oktober 1968 in Garmisch, Deutschland, stattfand [Naur und Randell, 1969]. „Software Engineering“ war provokatorisch gemeint und sollte anzeigen, dass auch die Entwicklung von Software den Charakter einer Ingenieursdisziplin aufweisen muss. Bis heute gibt es noch keine allgemein anerkannte, feststehende Definition dieses Begriffs.

Softwareerstellung bedeutete früher die Erstellung von (kleineren) *Programmen* im Allgemeinen durch einen Programmierer für einen Anwender. Oftmals war sogar der Programmierer der Anwender; *Programmieren* wurde *als Kunst* angesehen. Die moderne Softwareerstellung umfasst komplexe Programmsysteme (*Softwaresysteme*), die aus mehreren bzw. vielen Komponenten (Programmen) zusammengesetzt sind und die von mehreren Programmierern für viele Anwender erstellt werden. Die Anwender sind nicht identisch mit den Programmierern, und ihr Programmzugang unterscheidet sich grundsätzlich von dem der Programmierer. *Programmieren wird als Ingenieursleistung, als Technik* angesehen.

Wir wollen die Facetten des Begriffs *Software Engineering* (bzw. mittlerweile seltener *Softwaretechnik*) anhand folgender Definitionen aus der Literatur veranschaulichen:

- [Fairley, 1985]: *Software Engineering* ist die technische und organisatorische Disziplin zur systematischen Herstellung und Wartung von Softwareprodukten, die zeitgerecht und innerhalb vorgegebener Kostenschranken hergestellt und modifiziert werden.
- [Sommerville, 2004]: *Software Engineering* befasst sich mit dem Bau von Softwaresystemen, die nicht von einem Entwickler alleine hergestellt werden können. Software Engineering beruht auf der Anwendung ingenieurmäßiger Prinzipien und umfasst sowohl technische als auch nichttechnische Aspekte.
- [Bauer, 1975]: Das Ziel der *Softwaretechnik* ist die wirtschaftliche Herstellung zuverlässiger und effizienter Software.

Die Entwicklung großer Softwaresysteme bringt neue, anders geartete Probleme mit sich als die Entwicklung kleiner Programme und weist viele Ähnlichkeiten mit der Entwicklung anderer technischer Produkte auf. Die Hauptprobleme sind [Pomberger und Blaschek, 1996]:

- Spezifikation der Anforderungen,
- Beherrschung der Komplexität,
- Zerlegung eines Systems in Untersysteme,
- Spezifikation der Schnittstellen zwischen den Untersystemen,
- Wiederverwendung/Wiederverwendbarkeit von Bausteinen, Entwicklung von Halbfabrikaten,
- Projektorganisation, vor allem im Hinblick auf arbeitsteilige Systementwicklung,
- Effizienz,
- Dokumentation und Wartbarkeit der Gesamtlösung,
- Änderbarkeit und Erweiterbarkeit,
- Übertragbarkeit auf und Anpassbarkeit an verschiedene Hardwaresysteme.

1.5 Projekt Engineering

Analog zum *Software Engineering* (wörtliche Übersetzung etwa *ingenieurmäßige Softwareentwicklung*) verbindet das *Projekt Engineering* (*ingenieurmäßige Projektentwicklung*) die technische Produktentwicklung mit der wirtschaftlichen Produktherstellung [Sanchez und Canton, 1997].

Im Bereich der Softwareentwicklung stimmen die in Abschnitt 1.4 genannten Eigenschaften des Software Engineering zum größten Teil mit den in Abschnitt 1.3.1 formulierten Eigenschaften von Projekten überein. Die Ziele einer erfolgreichen Software-Projektentwicklung („*Software-Projekt Engineering*“) und des Software Engineering sind also deckungsgleich.

2 Der Projektentwicklungszyklus im Software Engineering

Im Folgenden werden die Hauptschritte einer Softwareentwicklung überblicksmäßig beschrieben. Wie in den einzelnen Kapiteln angeführt wird, sind diese Schritte nicht rein sequenziell zu betrachten. Durch Erkenntnisse während eines Projekts kann das Zurückgehen auf frühere Schritte notwendig werden, ja sogar die gesamte Projektentwicklung erneut begonnen werden müssen. Man spricht daher von einem Projektentwicklungszyklus, den wir für den Bereich des Software Engineering diskutieren.

2.1 Initialisierung

Ziel der *Projektinitialisierung* (*Projektanbahnung*) ist, neben dem Kennen lernen der potenziellen Projektpartner, die möglichst genaue Zieldefinition, die Abgrenzung des Problems (gegenüber ähnlichen Problemen, gegenüber Problemen, die getrennt betrachtet werden müssen etc.) sowie das Erarbeiten der Projektrestriktionen (verfügbares Know-how, Zeit, Mittel, ...). Diese Zielformulierung ist schriftlich festzuhalten.

Wie in Abschnitt 1.3.2 besprochen, unterscheiden wir zwischen internen und externen Projekten, je nachdem ob der Auftraggeber die eigene Unternehmensleitung oder ein externer Kunde ist.

Tipp¹: Behandeln Sie ein internes Projekt so, als ob es ein externes wäre. Ordnen Sie jede Rolle eindeutig dem Auftraggeber oder dem Auftragnehmer zu. Keine Person darf Rollen des Auftraggebers und des Auftragnehmers in Personalunion übernehmen.

¹Im Folgenden mit „**Tipp**“ gekennzeichnete Absätze geben die persönliche Ansicht des Autors wieder. Diese Anregungen entstammen der praktischen Projekterfahrung des Autors und stehen manchmal sogar im Widerspruch zu einem Teil der Literatur. Die Anregungen haben sich in der Erfahrung des Autors bewährt, sollten jedoch nicht als allgemein gültig angesehen werden und durchaus oft kritisch gelesen bzw. angewendet werden.

Es können zwei Hauptvorgehensweisen unterschieden werden, wie ein Auftraggeber seine Anforderungen an ein neues Produkt formulieren kann:

1. Der Auftraggeber hat ein fertiges Lastenheft oder Pflichtenheft und möchte das darin beschriebene Produkt realisieren lassen.
2. Der Auftraggeber erklärt die Ziele des gewünschten Produkts und seine Anforderungen daran verbal (Besprechung, Workshop, Präsentation, ...).

Wichtig für das Projektverständnis ist, sich „vor Ort“ vertraut zu machen; man bekommt dadurch „ein Gefühl“ für das Problem. Gleichzeitig erhält man so eine Beschreibung des Istzustandes, i. e. wie der Auftraggeber derzeit das Problem löst bzw. ohne Lösung zurechtkommt. Ein weiterer Vorteil dieser Vorgehensweise ist die Ermittlung der Personalstruktur im Unternehmen (Erkennen der Experten, Ansprechpartner, Verantwortlichen).

Bereits bei der Projektanbahnung sollte jeder Schritt dokumentiert werden (durch Protokolle, Aktennotizen etc.). Die Dokumentation ist während des gesamten Projekts ein entscheidender Faktor für den Projekterfolg, denn nur die lückenlose schriftliche Dokumentation erlaubt das korrekte Argumentieren, v. a. bei Projektkonflikten.

Folgende projektvorbereitende Maßnahmen können bereits durchgeführt werden:

- Auswählen der Mitarbeiter für die Projektgruppe,
- Zusammenstellen verfügbarer Literatur, Experten und anderer Informationsquellen,
- Feststellen der terminlichen Auslastung der geplanten Projektgruppe und Reservierung von Arbeitszeit gemäß einer ersten groben Schätzung.

2.2 Angebotslegung und Auftragserteilung

Auf Basis der Problembeschreibung wird dem Kunden (potenziellen Auftraggeber) ein unverbindliches oder verbindliches Angebot unterbreitet („*Angebotslegung*“). Ein unverbindliches Angebot ist für den Kunden mit keinerlei Pflichten oder Kosten verbunden, kann allerdings auch nur als Absichtserklärung gewertet werden, das Problem seitens des potenziellen Auftragnehmers zu bearbeiten.

In einem verbindlichen Angebot ist der Leistungsumfang klar festzulegen (Rechte und Pflichten). Dabei ist vorerst festzulegen, ob dem eigentlichen Projekt ein „Vorprojekt“ vorgeschaltet wird (siehe Abschnitt 2.3) und damit eine Verfeinerung und Adaptierung des Angebots gemäß den Vorprojektsergebnissen erfolgen wird oder ob das Angebot bereits für das Gesamtprojekt Gültigkeit auch im Detail besitzt.

Folgende Hauptkomponenten sollten in einem Angebot enthalten sein:

- Projektbezeichnung,
- Angebotsleger (Auftragnehmer),
- Kunde (Auftraggeber),

- kurze Aufgabenbeschreibung,
- Termin für Projektbeginn (samt evtl. Bedingungen, wie z. B. Entwicklungsumgebung),
- Art des Projekts (Coaching, Turn-Key-Entwicklung, Joint Venture),
- Gliederung in Projektabschnitte (Phasen), denen einzelnen Projekteigenschaften (z. B. Dauer, Kosten, Ressourcenbindung, Leistung) zugeordnet werden; aus dieser Beschreibung ergibt sich das geplante Projektende,
- Mitarbeit des Auftraggebers,
- Festlegung, wann und wie die Detailplanung erfolgt (Pflichtenhefterstellung, Bereitstellung von Testdaten, Aufstellung von Abnahmekriterien, ...),
- definierte Ergebnisse (Programm – inkl. Quellcode? –, Dokumentation),
- Qualitätssicherungsmaßnahmen,
- Verteilung der Gesamtkosten auf das Projekt mit Zahlungszielen,
- organisatorische und juristische Abmachungen (z. B. Konsequenzen bei Projektverzug, Rechte am Ergebnis),
- Garantie, Konditionen für Wartung,
- Kosten für notwendige Schulung und Weiterbildung beim Kunden (soweit planbar).

Auf Basis des Angebots sowie der anschließenden Verhandlungen erfolgt im positiven Fall die – schriftliche – *Auftragserteilung*. Für die detailliertere Behandlung von Angebot und Auftrag, speziell aus juristischer Sicht, wird auf die entsprechende Literatur verwiesen (z. B. [Zahrnt, 2002]).

2.3 Vorprojekt

Das Ziel eines *Vorprojekts* (*pilot project*; manchmal auch – unsauber – *Vorstudie* genannt) ist die Verfeinerung der Angebotslegung, um damit die Abschätzung der Machbarkeit, Kosten und Qualität des Ergebnisses verbessern zu können. Je nach Darstellung des Problems seitens des Auftraggebers erfordert dies das Überprüfen des Lastenhefts des Auftraggebers (manchmal inkorrekterweise bereits als „Pflichtenheft“ bezeichnet) bzw. das Festschreiben der Anforderungen des Auftraggebers. Außerdem ist alles Informationsmaterial zusammenzustellen, das zum Projektthema bereits beim Auftraggeber vorhanden ist.

Erstes Ziel eines Vorprojekts ist also die Beschreibung des *Istzustands* (*Istanalyse*). Es ist dabei genau zu beschreiben, wie das Problem derzeit gelöst (bzw. umgangen) wird sowie welche Fehler und Schwierigkeiten es derzeit gibt. Die Istanalyse soll den derzeitigen Arbeitsablauf genau widerspiegeln und damit Einsicht in die derzeitige Arbeitsweise vermitteln. Die Istanalyse soll lediglich objektiv den aktuellen Zustand beschreiben. Es sollen keine Lösungs- oder Verbesserungsvorschläge enthalten sein.

Aus dem Erkenntnis des Istzustands sowie der Problembeschreibung des Auftraggebers ist der *Sollzustand* zu beschreiben (*Sollkonzept*). Diese Beschreibung umfasst die gewünschten

Änderungen und Verbesserungen im Arbeitsablauf, im Arbeitsergebnis, im Aufwand etc. Das Sollkonzept soll beschreiben, wie die Lösung (das Projektziel) aussieht, und *nicht*, wie die Lösung erreicht wird! Ergeben sich während eines Vorprojekts Zweifel, ob die gestellte Aufgabe vom Auftragnehmer (oder „überhaupt“) zu realisieren ist, sollte auf jeden Fall eine vorsichtige Haltung eingenommen werden. In dieser Projektphase ist ein Ausstieg aus dem Projekt meist noch relativ kostengünstig möglich. Eine Alternative dazu wäre, gemeinsam mit dem Auftraggeber eine Aufgabenstellung zu erarbeiten, die als machbar eingestuft wird, und für diese dann zu prüfen, ob sie das Problem des Auftraggebers zumindest in genügendem Umfang löst.

Tipp: Es wird manchmal vorgeschlagen, im Fall einer unsicheren Aufgabenstellung einen Teilauftrag anzunehmen („das, was leicht zu realisieren ist“) und die anderen Projektteile durch andere, vom Auftraggeber beigezogene Auftragnehmer (oder, noch schlimmer, eigene Subauftragnehmer) realisieren zu lassen. Von dieser Vorgehensweise wird dringend abgeraten. In der Praxis wird ein solches Projekt nämlich immer als ein Ganzes gesehen, und wenn ein Teilprojekt schief geht, wird dies allen Auftragnehmern angelastet, unabhängig von der Schuldfrage.

Oft ist ein Ziel des Vorprojekts das Entscheiden (oder zumindest Eingrenzen) der Entwicklungs- bzw. Zielumgebung (Hardware, Software, Werkzeuge). Hierbei sollte man sich unbedingt genau über die verschiedenen, in Frage kommenden Systeme informieren und sich nicht auf Prospektmaterial verlassen. Rasche Information und Produktvergleiche sind oft in Computernetzen verfügbar (Web-Foren, Newsgroups, ...). Bei Firmendemonstrationen sollte unbedingt ein projektspezifisches Detail zur „unvorbereiteten Bearbeitung“ vorbereitet werden. Eine gute Lösung ist es, zusammen mit dem Auftraggeber Testinstallationen durchzuführen.

Zusammenfassend ergeben sich folgende Ergebnisse des Vorprojekts:

- Feststellung der Machbarkeit,
- Darstellung des Istzustandes,
- (Grob-)Darstellung des Sollzustandes,
- grobe Anforderungsanalyse mit Ansätzen zum Design (bei realisierungskritischen Details),
- Aufwandsschätzung mit Arbeits-, Zeit- und Ressourcenplan,
- Beschreibung der Vorgehensweise bei der Projektdurchführung (Vorgehenshandbuch),
- Anforderung an die Entwicklungsumgebung,
- vom Auftraggeber zur Verfügung zu stellende Ressourcen (Hardware, Software, Experten, Daten, ...),
- Projektgliederung mit schrittweisem Realisierungsplan,
- Definition eines Erfüllungsgrades (Bedeutung von „brauchbarer“ bzw. „unbrauchbarer“ Software; Eingrenzung des Begriffs „Mangel“, projektbezogene Feindefinition der Qualitätskriterien, ...),
- Schulungsplan (soweit bereits zu diesem Zeitpunkt planbar).

Bereits im Vorprojektsbericht sollte explizit festgehalten werden, was alles im Projektergebnis *nicht* enthalten ist. Zur Absicherung des potenziellen Auftragnehmers sollte diese Beschreibung mindestens so detailliert sein wie die Funktionsbeschreibung des Projektergebnisses, auch wenn dadurch die Chancen auf Auftragserteilung reduziert werden sollten.

Je nach Gestaltung des Angebots bzw. des Auftrags hat das Vorprojekt eine Adaptierung (Verfeinerung) des Angebots oder eine erneute Angebotslegung zur Folge. Dabei hat ein erneutes Angebot für die Produkterstellung den Vorteil, dass der Auftragnehmer des Vorprojekts und der Auftragnehmer der Produkterstellung nicht unbedingt identisch sein müssen. Dies hat im Allgemeinen eine größere Unabhängigkeit des Vorprojektsergebnisses vom Auftragnehmer zur Folge und lässt damit dem Auftraggeber eine gewisse Unabhängigkeit und damit Projektsicherheit.

2.4 Produkterstellung

Für die detaillierte Behandlung der *Produkterstellung* sei auf die Kapitel 9 bis 14 verwiesen. Im Folgenden wird nur eine allgemein als günstig anzusehende Vorgehensweise der Produkterstellung präsentiert.

2.4.1 Basisprodukt

Bei fast jedem Projekt ist es sinnvoll, im Rahmen einer ersten „großen“ Produktentwicklungsphase dem Auftraggeber ein Produkt zur Verfügung zu stellen, das die notwendigsten Anforderungen erfüllt, ein so genanntes *Basisprodukt*. Diese Vorgehensweise zwingt den Auftraggeber dazu, seine Anforderungen zu priorisieren und, anstelle alles denkbar Mögliche in den Anforderungskatalog zu packen, die wichtigsten Grundanforderungen zu formulieren. Gerade die nicht unbedingt notwendigen Anforderungen und Wünsche ändern sich nämlich sehr häufig während der und auch bedingt durch die Produktentwicklung.

Es erweist sich daher als äußerst sinnvoll, den Projektablauf in kleinere überschaubare Abschnitte zu gliedern, wobei nach jedem Abschnitt entscheidungsreife Zwischenprodukte vorliegen sollen (*Meilensteintechnik*). Abhängig vom Ergebnis der Abnahme der Meilensteine können nun das weitere Vorgehen, die nächsten Teilziele sowie gegebenenfalls auch das Gesamtziel adaptiert werden. Diese Vorgehensweise ermöglicht beiden Seiten eine gewisse Absicherung des Entwicklungsfortschritts.

Häufig tritt während der Projekte eine bedeutende Erweiterung der Anforderungen und Wünsche an das Produkt seitens des Auftraggebers zu Tage. Meilensteine ermöglichen hier, zusätzliche Subprojekte zu initiieren (so genannte *Spin-offs*), weitere Meilensteine umzureihen oder das gesamte Projekt neu zu überdenken und – bei entscheidender Änderung der Zieldefinition – das Projekt neu zu formulieren.

Tipp: Bedenken Sie, dass die Abnahme eines Meilensteins immer eine gewisse Zeit beansprucht (im Wochen-, in seltenen Fällen sogar im Monatsbereich). Legen Sie für die Abnahme eine genügend große Zeitreserve an, und planen Sie Meilensteine nicht absolut, d. h., vom Projektbeginn an, sondern jeweils relativ zum Abschluss des vorigen Meilensteins.

2.4.2 Adaptierung und Optimierung

Ausgehend vom Basisprodukt ist es ratsam, Änderungs- und Erweiterungswünsche (*Adaptierung*) als einzelne *Zusatz-* oder *Folgeprojekte* zu definieren, die erst nach Abnahme des Basisprodukts bzw. – bei entsprechender Vorsicht – nach Abnahme des relevanten Meilensteins gestartet werden können.

Hierbei ist zu beachten, dass parallel laufende Teilprojekte immer gewisse Abhängigkeiten untereinander aufweisen und dass für die Koordination und Lösung der daraus entstehenden Abhängigkeiten genügend Zeitreserven angelegt werden. (Was eine Person in zwei Tagen erledigt, erledigt zwei im Allgemeinen nicht in einem!)

Speziell der Bereich der *Optimierung* sollte immer aus der Basisproduktentwicklung herausgehalten werden. Zum einen bestimmen erst das Basisprodukt und die (Pilot-)Anwendung, in welchen Bereichen und nach welchen Kriterien optimiert werden soll, zum anderen ist die parallele Entwicklung und Optimierung eines Softwareprodukts fast immer mit erheblicher gegenseitiger Behinderung der Softwareingenieure verbunden.

Anstelle von Folgeprojekten kann auch eine – im Allgemeinen nach Aufwand oder gemäß fixer Verfügbarkeit – Wartung des Produkts definiert werden, siehe dazu Abschnitt 14.2.5.

2.5 Nutzung

Entscheidend für den Einsatzerfolg eines Softwaresystems im Rahmen der *Nutzung* ist die Integration in das Betriebsführungskonzept des Auftraggebers. Ist diese nicht gegeben, können allein schon die organisatorischen Reibungswiderstände dafür sorgen, dass das neue System nie sinnvoll eingesetzt wird.

Man sollte sich auch darüber im Klaren sein, dass ein computerbasiertes System in viel stärkerem Maß als „konventionelle“ Techniken der Betriebsführung die Bereitstellung präziser Daten aus dem Prozess oder der Organisation verlangt. Das hat andererseits den positiven, für den Auftraggeber aber oftmals mit großer Anstrengung verbundenen Nebeneffekt, dass der Betreiber veranlasst wird, seinen Betrieb genauer zu verstehen und zu organisieren.

Auch die Notwendigkeit von Spezialpersonal für Nutzung und Wartung des Softwaresystems sollte nicht unterschätzt werden. Dies ist nicht allein ein Problem der Kosten, sondern auch der Verfügbarkeit solchen Personals.

Ein wichtiger Designgrundsatz ist auch die Vermeidung unnötiger Komplexität. Für die Benutzer eines Softwaresystems ist dieses ein Arbeitsmittel, dessen Bedienung nicht schwieriger sein darf als die Bewältigung der Aufgabe ohne die Unterstützung des Systems. Es sollte aber natürlich auch vermieden werden, das System aus technischem Ehrgeiz so aufwändig („elegant“) zu gestalten, dass es dadurch störanfällig wird oder nur noch von höchst qualifizierten Spezialisten bedient, gewartet und gepflegt werden kann.

Die *Motivation der Anwender*, die das System später benutzen sollen, ist ebenfalls ein wichtiger Erfolgsfaktor. Neben vorherigen sorgfältigen Überlegungen zur Gestaltung der Arbeitsprozesse, die durch das System bedingt oder verändert werden, sind gute Schulungsmaßnahmen ein geeignetes Mittel, um dieses Ziel zu erreichen.

2.6 Lebensdauer von Software

Die bewusste Verfolgung und Berücksichtigung von Trends ist – besonders bei der Entwicklung größerer Systeme – von großer Bedeutung. Die durchschnittliche *Produktlebensdauer von Hardware* beträgt zwei bis drei Jahre, ein Softwaresystem muss aber, besonders in seinen anwendungsorientierten Bestandteilen, zehn bis zwanzig Jahre tragfähig bleiben.

Man muss ein Softwaresystem also so auslegen, dass selbst kritische Komponenten ohne untragbaren Aufwand oder tief greifende Änderungen der Gesamtstruktur ausgetauscht werden können. Dazu müssen Planer und Entwickler z.B. abschätzen, welche Schnittstellenstandards, Programmiersprachen oder Rechnerfamilien mittel- bis langfristige „Überlebenschancen“ haben. Dies ist eine schwierige Aufgabe, die einerseits sehr viel Erfahrung und Einfühlungsvermögen in den Markt, andererseits aber Grundlagenwissen über Marktverhalten, Zeitkonstanten bei Technologieentwicklungen etc. erfordert. So dauern Basisinnovationen länger als gemeinhin angenommen. Im Technikbereich gelten 15 Jahre von der ersten Funktionsfähigkeit im Labor bis zur Produktreife als normaler Wert, in der Softwareentwicklung ist der Zeitraum noch länger [Potts, 1993]. Diese (Erfahrungs-)Werte sollte man sich vor Augen halten, wenn man sich möglicherweise darüber wundert, wie alt manche Techniken wirklich sind und weshalb sie bisher vielleicht so wenig angewandt wurden.

Von größerer Bedeutung ist auch die „Sättigungskurve der Weiterentwicklung“ (Abbildung 2.1, [Elzer, 1994]), gemäß der nach einer gewissen Zeit eine bestimmte Technik nur noch mit einem unangemessen hohen Aufwand weiterentwickelt werden kann. Es ist eine wesentliche Aufgabe eines Projektleiters, den richtigen Zeitpunkt für den Umstieg von einer Technik, die das Unternehmen gut beherrscht, die aber der Sättigung entgegengeht, auf eine neue Technik mit genügend Zukunftspotenzial zu finden.

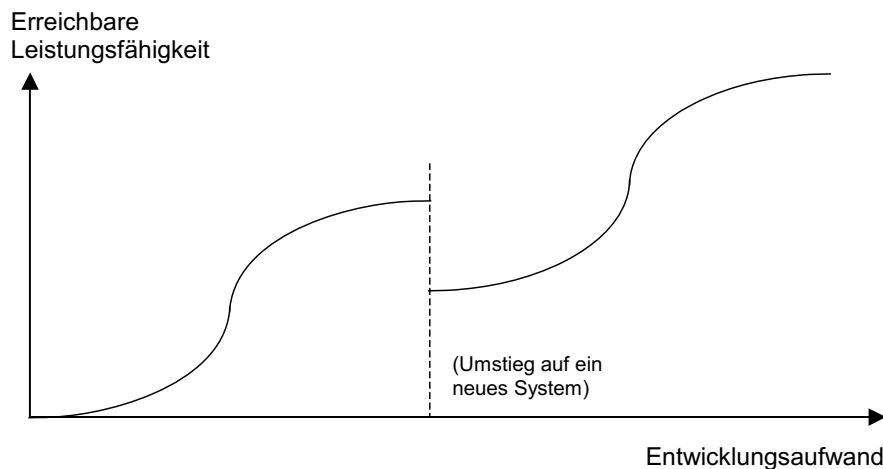


Abbildung 2.1: Die Sättigungskurve der Weiterentwicklung in der Technik

Der Einsatz einer bestimmten Technik in großem Maßstab bewirkt eine Stabilisierung, die dazu führen kann, dass sie viel länger erhalten bleibt, als man auf Grund der gleichzeitig weiterlaufenden Entwicklung annehmen sollte. Der Grund dafür ist, dass sie durch ihre weite Verbreitung einen hohen Anteil der verfügbaren Arbeitskräfte bindet. Damit erreichen konkurrierende Techniken mangels einer ausreichenden Zahl von Benutzern und Entwicklern nicht mehr die für eine Ausbreitung erforderliche kritische Masse. Es können auf diese Weise

sogar „Absorptionseffekte“ auftreten, bei denen neuere Techniken durch ältere, aber verbreitetere wieder verdrängt werden. Beispiele dafür sind FORTRAN im Vergleich zu „modernen“ Programmiersprachen oder MS-DOS/Windows gegenüber anderen Betriebssystemen. Dieser Effekt lässt sich aus der Technikgeschichte über Jahrhunderte hinweg belegen [Elzer, 1991].

3 Der Produktentwicklungszyklus im Software Engineering

Die *Durchführung* der Entwicklung eines Produkts setzt entsprechende *Planung* voraus. Es heißt zwar oft: „Planung ersetzt den Zufall durch den Irrtum!“, aber aus dem Irrtum kann man lernen. Ziel der Planung einer Produktentwicklung ist also nicht nur die Durchführung dieser einen Entwicklung, sondern auch der Aufbau von Know-how für weitere Produktentwicklungen.

Entscheidend für das Gelingen der Durchführung ist die Überprüfung der durch die Planung festgelegten Vorgaben. *Planung ohne Überprüfung ist sinnlos, Überprüfung ohne Planung unmöglich!* Abbildung 3.1 veranschaulicht diese drei Grundkomponenten der Produktentwicklung, deren Teilaufgaben im Folgenden diskutiert werden. Die Konzepte, Methoden und Werkzeuge der einzelnen Komponenten werden in den nachfolgenden Kapiteln erläutert.

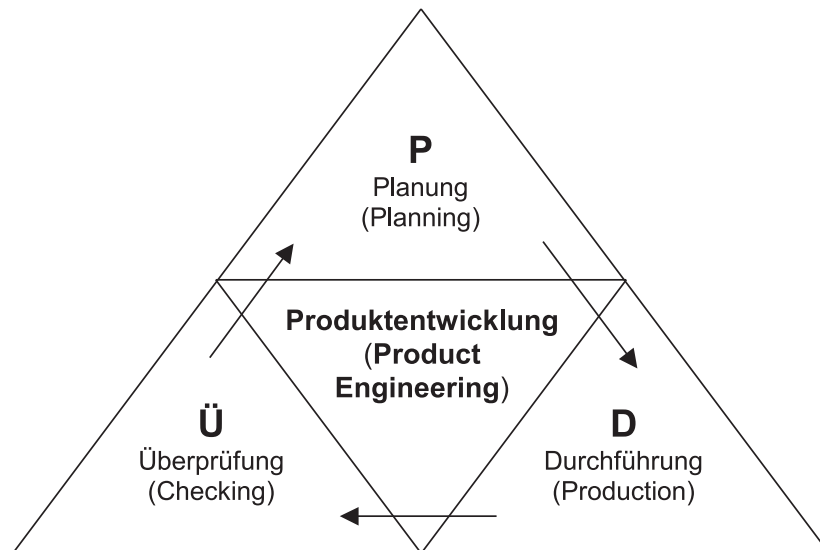


Abbildung 3.1: Grundkomponenten der Produktentwicklung

3.1 Planung

Das *Ziel der Planung (planning)* ist es, Vorgaben zur Entwicklung eines Softwareprodukts in Bezug auf Aufgaben, Aufwand, Termine, Ressourcen, Kosten, Finanzen sowie begleitende Maßnahmen (z. B. Qualitätssicherung) und vorsorgende Maßnahmen (z. B. Sicherungs- und Notfallplanung) festzulegen.

Dazu ist es notwendig, Softwareprodukte so zu zergliedern, dass die einzelnen Arbeiten planbar und mit überprüfbaren Resultaten von möglichst kleinen Teams mit wirtschaftlich vertretbarem Aufwand an Zeit und technischen Ressourcen durchgeführt werden können. Die *Aufgaben der Planung* ergeben sich entsprechend der Vorgabenklassifizierung.

Die Projektplanung ist – im Gegensatz zu den Ansichten in mancher Literatur – Aufgabe der gesamten Projektorganisation unter dem Verantwortungsbereich des Projektleiters. Die Komplexität der Projektplanung ergibt sich aus den Abhängigkeiten zwischen den Aufgaben. Eine sequenzielle Planung ist daher unmöglich, die einzelnen Planungsbereiche müssen gegenseitig ausbalanciert werden.

Im Projektverlauf sind durch zusätzliche Information oftmals Umplanungen notwendig. Dabei verliert das Konzept des *Masterplans* (i. e. eines umfangreichen, detaillierten Gesamtplans, der früh im Projekt erstellt wird) zunehmend an Bedeutung, da in der modernen Softwareentwicklung sowohl die geforderte Funktionalität als auch die einzuhaltenden Termine oftmals bewegliche Ziele darstellen (vgl. Abschnitt 1.3.3). Das Problem stellen dabei nicht die Änderungen an sich dar, sondern die unkontrollierten (und ungeplanten) Änderungen [Brown *et al.*, 2000]. Wenn die Anforderungen und Beschränkungen eines Projekts bewegliche Ziele darstellen, ist es entscheidend, nicht groß, sondern oft zu planen (“If you can’t plan well, plan often.” [Humphrey, 2005]).

Grundsätzlich ist Planung eine Steuerungsaufgabe. Durch – geplante – Überprüfungen wird daraus ein Regelkreis zur erfolgreichen Durchführung einer Produktentwicklung.

3.2 Durchführung

Ziel der Durchführung (production¹) ist die Produkterstellung unter Beachtung der durch die Planung vorgegebenen Vorgehensweise. Ein Projekt sollte dabei nicht in zu viele Projektschritte (Teilarbeitsbereiche) zergliedert werden, da im Allgemeinen zwischen solchen Komponenten nur sehr vage Schnittstellen definiert werden und dadurch der Kommunikationsaufwand umso mehr erhöht wird. Bei größeren Projekten ist es sinnvoll, „Unterprojekte“ zu definieren, die mit dem Rest des Projekts nur wenige, aber dafür exakt definierte Schnittstellen aufweisen.

Aufgaben der Durchführung sind:

- Abarbeitung der Projektschritte (parallel, seriell),
- (End-)Auswahl der Werkzeuge,

¹In [Coleman *et al.*, 1994] wird mit „production“ die Produkterstellung bis zur Produktion des Codes bezeichnet; die Integration der Codeteile zum Produkt wird als „assembly“ bezeichnet.

- Risikobehandlung,
- (Fein-)Design,
- Implementierung,
- Testen,
- Produkt-Einführung.

Ein Projekt sollte nie mit voller Teamstärke begonnen werden. Wenige, dafür aber hoch qualifizierte Mitarbeiter müssen die technischen Grundlagen erarbeiten, damit die Mehrzahl der (normalerweise weniger qualifizierten) Mitarbeiter konstruktiv und produktiv eingebunden werden können. Gleichzeitig kann so die Kommunikationsstruktur im Team sukzessive aufgebaut werden.

Die *Reihenfolgeplanung (Priorisierung)* der Projektschritte kann nach verschiedenen Gesichtspunkten erfolgen. Zwei bekannte Strategien sind [Heinrich und Lehner, 2002]

1. die Hardest-first-Strategie und
2. die Easiest-first-Strategie.

Die *Hardest-first-Strategie* geht davon aus, dass jene Aufgaben als erste bearbeitet werden, welche die größten (vorhersehbaren) Schwierigkeiten bei der Realisierung verursachen. Diese Strategie wird vor allem dann angewendet, wenn ein Projekterfolg ohne die Lösung dieser Aufgaben nicht möglich ist.

Die *Easiest-first-Strategie* geht davon aus, dass zunächst die Aufgaben bearbeitet werden, die ohne große Probleme lösbar sind und daher schnell abgearbeitet werden können. Diese Strategie bietet sich an, wenn gewährleistet werden soll, dass möglichst viele Teile der Projektaufgabe in einer begrenzten Zeit bearbeitet werden. Es wird damit vermieden, dass für schwierige, aber nicht wesentliche Aufgaben zu viel Zeit aufgewendet wird, sodass bei Projektende für einfach zu realisierende und wesentliche Aufgaben keine Zeit übrig bleibt.

3.3 Überprüfung

Ziel der Überprüfung (checking) der Produkterstellung ist es, bereits während der Durchführung und möglichst schon in ihren Ansätzen Abweichungen von den Projektzielen zu erkennen und gegebenenfalls korrigierende Maßnahmen einzuleiten.

Daraus ergeben sich folgende *Aufgaben der Überprüfung*:

- Anleiten und Motivieren der Mitarbeiter,
- Herbeiführen notwendiger Entscheidungen,
- Durchsetzen der geplanten Lösungen,
- Einleiten von Korrekturmaßnahmen bei Planabweichungen,

- Adaptieren der Ziele,
- Koordinieren der Zusammenarbeit zwischen Auftraggeber, Auftragnehmer und Projektgruppen,
- Durchführen der Validierung,
- Durchführen der Abnahme,
- (externer) Projektabschluss,
- Durchführen der (internen) Evaluierung und Nachkalkulation.

Die wichtigsten Hilfsmittel der Überprüfung sind daher die Anforderungsbeschreibungen, die Projektpläne sowie die Besprechungsprotokolle und Projektfortschrittsberichte.

4 Projekt Engineering

In diesem Kapitel wird Projekt Engineering hergeleitet durch die Erweiterung des Produktentwicklungszyklus zu einem auch innerhalb eines Zyklus geregelten Prozess der ingenieurmäßigen Projektentwicklung. Diese Definition führt zu einer notwendigen Aufgabenteilung, deren Struktur aber auch die inhärenten Schwierigkeiten einer Softwareentwicklung klar darlegt. Ein mnemonisches Schema der ingenieurmäßigen (Software-)Projektentwicklung zusammen mit den integrierten Phasen schließt dieses Kapitel ab.

4.1 Blackbox/Whitebox-Sicht der Projektentwicklung auf die Produktentwicklung

Der Leistungsprozess eines Unternehmens muss durch die Projektentwicklung so koordiniert werden, dass die Rahmenbedingungen für eine Produktentwicklung eingehalten werden. „Im Einzelnen muss das Management dazu die Ziele und Strategien für das Unternehmen festlegen, diese in Plänen operationalisieren, die Zielerreichung überwachen, eine adäquate Unternehmensorganisation für die Umsetzung der Ziele und Pläne entwickeln, die Mitarbeiter führen und motivieren, auf die Unternehmensprozesse steuernd einwirken sowie Entscheidungen treffen. ... [Projektentwicklung] kann somit als eine Tätigkeit definiert werden, welche die Gestaltung des Handelns anderer Personen zum Gegenstand hat.“ [Gernert und Ahrend, 2001]

Daraus ergeben sich folgende wirtschaftliche Aufgaben zur fristgemäßen und ökonomischen Herstellung eines Softwareprodukts (nach [Gernert und Ahrend, 2001], gegliedert gemäß der Struktur in Abbildung 4.1):

- *Führung (management)*: organisieren, steuern, Personal führen, informieren,
- *Entwicklung (development)*: Ziele setzen, planen, entscheiden,
- *Kontrolle (monitoring)*: überprüfen, überwachen.

Dabei ist auf eine klare Kompetenztrennung zu achten, die eine weitgehende Aufgabentrennung mit sich zieht. So ist z.B. Führung und Kontrolle eines Projekts in Personalunion

unzulässig, da erfahrungsgemäß Selbstkontrolle „den Umständen angepasst wird“ und somit ihr Ziel nicht vollständig erfüllen kann. Analoges gilt für die Zusammenhänge mit der Entwicklung.

In Anbetracht dieser Aufgabentrennungen sowie des Produktentwicklungszyklus ergibt sich somit eine *Blackbox/Whitebox-Sicht der Projektentwicklung* auf die Produktentwicklung¹, vgl. Abbildung 4.1:

- Die *Führung* umfasst die wirtschaftlichen Belange der Planung und Überprüfung, aber nicht der Durchführung.
- Die *Entwicklung* umfasst die wirtschaftlichen Belange der Planung und Durchführung, aber nicht der Überprüfung.
- Die *Kontrolle* umfasst die wirtschaftlichen Belange der Durchführung und Überprüfung, aber nicht der Planung.

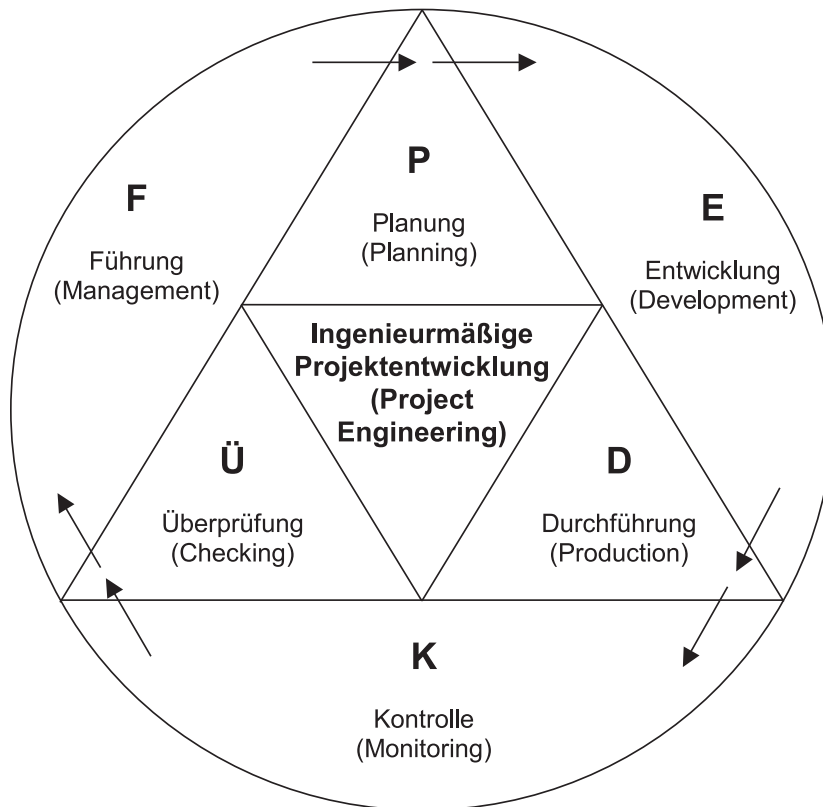


Abbildung 4.1: Blackbox/Whitebox-Sicht der Projektentwicklung auf die Produktentwicklung

Im Folgenden betrachten wir diese Hauptkomponenten im Überblick, die weiteren Kapitel behandeln dann deren Teilkomponenten im Detail.

¹Eine *Blackbox* ist eine Einheit, die keinen Blick in ihr Inneres erlaubt und damit nur durch ihr extern beobachtbares Verhalten analysiert werden kann. Eine *Whitebox* hingegen erlaubt auch die Analyse ihrer inneren Struktur und – bei Bedarf – die Einflussnahme darauf.

4.2 Führung

4.2.1 Ziel

Die *Führung* (*management*) eines Projekts umfasst die Projektplanung und -strukturierung sowie das Adaptieren der Planung auf Basis der Überprüfung.

Anmerkung: Der Begriff Führung wird im Deutschen in der Betriebswirtschaft oft einseitig mit den Begriffen der Führungsstile und Führungsarten verbunden. Andererseits hat die englische Übersetzung „management“ im Deutschen in den letzten Jahren geradezu eine Bedeutungs-inflation erlebt. Jeder Bereich, der etwas auf sich hält (oder mehr vorgeben möchte, als tatsächlich vorhanden ist?), versieht sich heutzutage mit dem Attribut „Management“. So wurde aus der ursprünglichen Qualitätssicherung das Qualitätsmanagement (vgl. Kapitel 8), aus der Produktionswirtschaft [Zäpfel, 1982] wurde das Produktionsmanagement (vgl. [Zäpfel, 2000a] bzw. [Zäpfel, 2000b]). Die Hauptbereiche, die vielfach mit „Management“ bezeichnet werden, sind:

- Projektführung,
- Organisation,
- (allgemeine) Projektentwicklung.

Um die Bedeutung präziser festzulegen, wird im Folgenden der Begriff „Management“ weitestgehend vermieden und durch die spezifischeren Termini ersetzt. Der Leser sei jedoch darauf hingewiesen, dass zum Auffinden entsprechender Literatur das Durcharbeiten des sehr umfangreichen Begriffs „Management“ im Allgemeinen unumgänglich ist.

Ziel der Führung ist es, die Projektaufgabe in Projektziele umzusetzen sowie die Realisierbarkeit der Projektaufgaben und damit der mit der Projektaufgabe verfolgten Ziele zu prüfen. Bei der Prüfung der Realisierbarkeit geht es in erster Linie um die Herausarbeitung der personellen, technischen, zeitlichen und finanziellen Ressourcen („Projektanforderungen“), die für die Durchführung des Projekts erforderlich sind. Die Projektführung ist daher so auszurichten, dass die gestellte Projektaufgabe mit begrenzten Ressourcen erfolgreich bearbeitet werden kann [Zäpfel, 2000a]. Dieses „Ausrichten“ ist kein einmaliger Prozess, sondern als Adaption der Projektplanung im Regelkreis der Projektentwicklung zu sehen, wobei die Regeldifferenzen durch die Überprüfung im Rahmen des Produktentwicklungszyklus ermittelt werden und damit eine *Projektsteuerung* (eigentlich *Projektregelung*!) ermöglichen.

Um geeignete strukturorganisatorische Rahmenbedingungen zur Abwicklung und Führung eines Projekts zu schaffen, wird vielfach die Projektführung in der bestehenden Organisation verankert (*institutionale Projektführung*, vgl. z. B. [Rausch-Schott und Retschitzegger, 1993]). Dies geschieht durch das Festlegen von grundlegenden organisatorischen Regelungen, wie

- Einrichten eines Projektträgers,
- Festlegen der Form der Projektorganisation,
- Festschreiben der Kompetenzen des Projektleiters.

4.2.2 Aufgaben

Die *Aufgaben der Führung* leiten sich aus der Planung, der Überprüfung sowie deren Kombination zur Steuerung/Regelung ab und umfassen:

- Projektleitung und Projektorganisation festlegen,
- Aufgaben, Kompetenzen und Verantwortung verteilen,
- Projekt und Projektziel strukturieren,
- Machbarkeit feststellen,
- Termine und Kosten planen und adaptieren,
- Durchführung organisieren,
- Information sicherstellen,
- Dokumentationserstellung planen.

Werden innerhalb einer (übergeordneten) Organisation mehrere Projekte gleichzeitig abgewickelt, so ist es Aufgabe des Projektträgers im Rahmen der *Multi-Projektführung*, die Projektpolitik festzulegen, die Projekte untereinander zu koordinieren sowie über die Verteilung des Budgets und des Personals zu entscheiden.

4.2.3 Arten

Je nach Schwerpunktsetzung der Sichtweise unterscheiden wir zwei *Arten der Führung* (vgl. Abbildung 4.2):

1. Die *technische Führung* basiert auf dem Denken in Projekten. Jedes Projekt wird vom Beginn bis zum Ende eigenständig betrachtet. Zwischenergebnisse und damit Fortschrittsbewertungen ergeben sich aus den – projektspezifischen – Meilensteinen.
2. Die *kaufmännische Führung* basiert auf dem Denken in Perioden. Jedes Projekt muss zu Beginn und am Ende einer betriebswirtschaftlichen Periode (z. B. Geschäftsjahr) bewertet und eingestuft werden. Für eine Periode wird also ein „Gesamtprojektfortschritt“ ermittelt, dessen Ermittlung betriebswirtschaftlich notwendig ist, dessen Sinnhaftigkeit von technischer Seite jedoch oftmals angezweifelt wird.

4.2.4 Situierung der Projektführung

In [Gareis, 1988] wird die Situation der Projektführung in den USA mit der in Österreich verglichen. Die darin gezogenen Schlüsse für die weitere Entwicklung und Situierung der Projektführung können auch heute durchaus noch als aktuell angesehen werden [Gareis, 2004]:

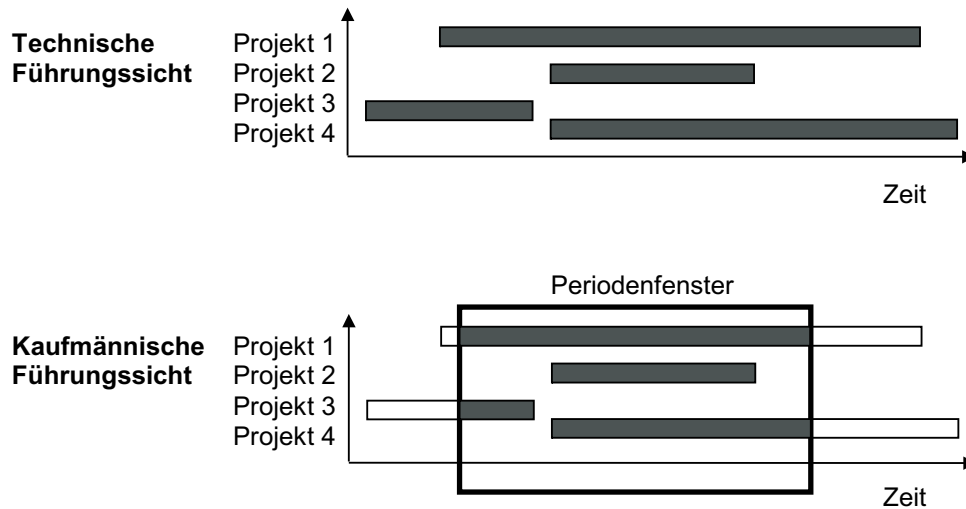


Abbildung 4.2: Technische und kaufmännische Sichtweise der Projektführung

- „Projektmanagement wird eine generelle Managementqualifikation darstellen. Das Berufsbild des Projektmanagers wird sich nur in wenigen Industrien ... ausprägen.
- Das Denken und Handeln in Projekten wird ein zentrales Konzept der Unternehmensführung der Zukunft darstellen. In flachen, flexiblen Organisationsstrukturen [*lean management*] werden in den Fachabteilungen die *skills* betreut, die *tasks* werden in den Projektteams durchgeführt.
- Projekte werden als Instrumente zur kontinuierlichen Entwicklung von Organisationen eingesetzt, da sie sowohl organisatorische Differenzierungen als auch organisatorische Integration ermöglichen.
- Projektmanagement-Training wird verstärkt mit Projektteams in aktuellen Projekten durchgeführt.“ [Gareis, 1988]

4.3 Entwicklung

4.3.1 Ziel

Das Ziel der *Entwicklung* (*development*) ist die Umsetzung des Projektziels gemäß der Planung.

4.3.2 Aufgaben

Die *Aufgaben der Entwicklung* umfassen:

- Verfeinerung der Analyse(modelle) zum inhaltlichen Design,

- Weiterentwicklung von Prototypen der Zielerforschung (Funktionserforschungsprototypen) zu – oder Neuentwicklung von – Schnittstellen- und Architekturprototypen,
- Aufstellen eines Ressourcenbelegungsplans inkl. Zuordnung der Mitarbeiter zu den entsprechenden Stellen,
- Erstellen des Pflichtenhefts auf Basis des Lastenhefts und des Designs,
- Durchführen der eigentlichen Implementierung,
- Erstellen/Abnehmen der Abnahmesuite,
- Erstellen der Produktdokumentation (System-, Benutzerdokumentation).

4.3.3 Arten

Folgende *Arten der Entwicklung* können unterschieden werden:

- **Einzelentwicklung:** Wenn die Entwicklung nur von einer einzigen Person durchgeführt wird, entsteht zwar kein zusätzlicher Kommunikationsaufwand, es existiert allerdings auch keine entwicklungsinterne Kontrollinstanz (in Form der Entwicklungskollegen, die Fehlwege der Entwicklung vielfach am raschesten erkennen können). Zusätzlich hängt die Entwicklung auf Gedeih und Verderb an einem Mitarbeiter und ist dadurch im Allgemeinen für ein Unternehmen schon allein aus diesem Grund viel zu risikoreich.
- **Verteilte Entwicklung:** Bereits im Rahmen der Projektplanung kann das Projekt in Unterprojekte zerlegt werden, die dann organisatorisch (und auch räumlich) verteilt abgearbeitet werden können. Voraussetzung ist, dass die Schnittstellen zwischen den Unterprojekten vorab festgelegt und möglichst klein gehalten werden, um so eine weitgehende Entkopplung der einzelnen Entwicklungsarbeiten zu ermöglichen.
- **Serielle Entwicklung:** Projektarbeiten, die voneinander abhängen, müssen entsprechend diesen Abhängigkeiten nacheinander ausgeführt werden. Dies führt zu einer Serialisierung der Arbeitsschritte. In der Praxis wird dieser Prozess vielfach durch die Begrenztheit bestimmter Ressourcen verstärkt, wobei besonders die Ressource „Mensch“ bzw. der menschliche Fachexperte in den – allgemein vorteilhaften – modernen Projektkleingruppen oftmals die entscheidende Beschränkung darstellt.
- **Parallele Entwicklung:** Wenn es die Abhängigkeit von Projektarbeiten zulässt, bewirkt eine Parallelisierung nicht nur eine Verkürzung der Entwicklungszeit, auch Schwächen der Vorarbeiten (Planung, Analyse, Design, ...) werden rascher erkannt und manche Fehlentwicklungen vermieden. In der Praxis gibt es jedoch kaum Projektarbeiten, die voneinander tatsächlich unabhängig sind, wie dies oft in der Theorie gefordert wird.
- **Konkurrierende Entwicklung** ist die parallele Entwicklung desselben Arbeitsschrittes. Diese kann bei genügend Ressourcen zur Zeitreduktion der Entwicklung eingesetzt werden („First-solution-wins“-Strategie). Häufiger dient diese Entwicklungstechnik allerdings dazu, verschiedene Werkzeuge oder verschiedene Vorgehen abzutesten, oder dazu, mehrere Lösungsvorschläge auszuarbeiten, die dann gegenseitig zur Optimierung verwendet werden können.

4.4 Kontrolle

4.4.1 Ziel

Ziel der Kontrolle (monitoring) ist die Erkennung von Planabweichungen während der Projektdurchführung. Dazu ist eine regelmäßige Kontrolle der Teilaufgaben nötig. Es müssen bei Planabweichungen deren Ursachen und Auswirkungen analysiert und korrigierende Maßnahmen (auch der Pläne!) gesetzt werden. Voraussetzung dafür ist eine gründliche Projektplanung sowie eine kontinuierliche, vollständige Erfassung der Istdaten (Start- und Endtermine, Mittelverbrauch, Zeitaufwand, ...).

Projektkontrolle muss aktiv durchgeführt werden. In der Regel geschieht dies durch die Auswertung der Projektfortschrittsberichte auf Basis der Planungsunterlagen (etwa alle zwei bis vier Wochen) sowie durch regelmäßige Reviews (alle ein bis zwei Monate) unter Einbindung der Projektaufsicht. Je detaillierter die Projektplanung durchgeführt wurde, desto einfacher ist die Projektkontrolle. Eine Kontrolle, die sehr detailliert durchgeführt wird, muss allerdings nicht automatisch hohe Kontrollqualität bedeuten.

4.4.2 Aufgaben

Die Projektkontrolle umfasst an *Aufgaben* die Kontrolle von

- Terminen und Abläufen,
- erbrachter Leistung,
- Kosten und Finanzierung,
- Vorhandensein und gleichmäßiger Auslastung von Ressourcen,
- Qualität,

wobei diese Komponenten projektspezifisch angepasst und gegebenenfalls erweitert werden müssen.

4.4.3 Arten

Gemäß den Aufgaben unterscheidet man zwischen

- Termin- und Ablaufkontrolle,
- Leistungskontrolle,
- Kosten- und Finanzierungskontrolle,
- Ressourcen- bzw. Kapazitätskontrolle, und
- Qualitätskontrolle.

Termin/Ablauf-, Kosten- und Leistungskontrolle werden im Folgenden näher erläutert, für Details zur *Kapazitätskontrolle* sei auf Abschnitt 10.2 verwiesen, für Details zur *Qualitätskontrolle* auf Kapitel 8.

4.4.3.1 Termin- und Ablaufkontrolle

Aufgabe der *Terminkontrolle* ist es, die zeitliche Einhaltung der im Terminplan festgelegten Entwicklungsziele zu prüfen. Dabei informiert sie über die tatsächlichen Anfangszeitpunkte, Endzeitpunkte und den Entwicklungsfortschritt und extrapoliert daraus die zu erwartenden Endzeitpunkte und Abweichungen. Die *Ablaufkontrolle* überprüft darauf aufbauend, ob Veränderungen in der Ablauflogik notwendig sind.

4.4.3.2 Leistungskontrolle

Die *Leistungskontrolle* (auch *Leistungsfortschrittskontrolle* genannt) überprüft, ob ausgeführte Vorgänge und erstellte (Zwischen-)Produkte das im Auftrag (bzw. in den dem Auftrag zu Grunde liegenden Dokumenten) festgelegte Ergebnis liefern.

Bei Engpässen können Vorgänge im Rahmen der Pufferzeiten verschoben, Vorgänge überlappt oder die benötigte Kapazität ausgeweitet werden.

4.4.3.3 Kosten- und Finanzierungskontrolle

Die *Kostenkontrolle* prüft, ob die im Projektplan angegebenen Kostenvorgaben überschritten worden sind. Dazu werden periodisch Plan- und Ist-Kosten gegenübergestellt. Im Rahmen der Kostenkontrolle wird vielfach auch gleich die *Finanzierungskontrolle* durchgeführt, welche die Finanzierung des Projekts überprüft bzw. mögliche Probleme in diesem Bereich im Vorhinein erkennbar machen soll. Eine Kostenkontrolle ist nur dann möglich, wenn eine entsprechende Kostenerfassung durchgeführt wird. Probleme ergeben sich immer wieder dadurch, dass Kostendaten oft erst Wochen nach dem Anfallen der Kosten verfügbar sind.

4.4.3.4 Integrierte Kontrolle

Planabweichungen sind häufig Kombinationen aus verschiedenen Problemen. Es sind daher alle Faktoren zu berücksichtigen und die Abweichungen untereinander in Beziehung zu setzen. Dies geschieht in der *integrierten Kontrolle*.

Wie in [Pomberger und Blaschek, 1996] festgestellt wird, ist die Kontrolle der Wirtschaftlichkeit eines Softwareprojekts in der Regel schwieriger als die technische Kontrolle. Während es nämlich normalerweise gelingt, irgendwann ein geplantes Programm in ausreichender technischer Qualität fertig zu stellen und zum Einsatz zu bringen, entsprechen die ursprünglichen Zeit- und Aufwandsschätzungen sehr selten auch nur annähernd der Realität.

4.5 Schwierigkeiten des Projekt Engineering

Grundlegende Schwierigkeiten des Projekt Engineering treten in allen drei Aufgabenbereichen der ingenieurmäßigen Projektentwicklung auf.

4.5.1 Schwierigkeiten der Führung

- **Die Einmaligkeit von Softwaresystemen:** Softwaresysteme werden in der Regel nur einmal entwickelt. Die Anzahl der Erfahrungswerte aus vorangegangenen Projekten ist daher zu gering, um zuverlässige Aufwandsschätzungen zu machen. Die Aufwandsschätzungen beruhen meist auf dem von erfahrenen Softwaretechnikern geschätzten Umfang (vielfach der Anzahl der Programmanweisungen) des zu entwickelnden Softwaresystems. Klarerweise ist dies eine sehr unzuverlässige Kalkulationsgrundlage.
- **Die sehr technische Führungssicht:** Die Entwicklung der Projektführung wurde lange Zeit stark von Praktikern, insbesondere von Ingenieuren geprägt. Dadurch wurde vielfach die Organisationsentwicklung zu Gunsten der Softwareentwicklung vernachlässigt. Zudem tendieren Ingenieure schon allein aus Zweckoptimismus, aber auch aus der Begeisterung für die Aufgabe heraus, zu einer zu optimistischen Planung. Erst in den letzten Jahren tragen auch Wissenschaftler und Praktiker mit betriebswirtschaftlichem, soziologischem und psychologischem Background zur Weiterentwicklung der Projektführung und zu realistischeren (holistischeren) Planungen bei.
- **Mangelhafte Planung:** Viele Softwareprodukte werden charakterisiert durch unklare oder unvollständige Planungsziele, häufige Änderung der Planungsziele und Mängel in der Projektorganisation. Mangelhafte Planung ist also häufig die Hauptursache für Projektrisiken. Meist sind die Planungsmängel auf Defizite in der Anwendung des Planungsinstrumentariums (Methodik, Methoden und Werkzeuge), nicht auf Defizite in der Verfügbarkeit des Planungsinstrumentariums zurückzuführen. Dieser Umstand zeigt die Notwendigkeit einer besseren Ausbildung für alle an einer Projektentwicklung Beteiligten (Projektträger, Projektleiter, Projektmitarbeiter und Anwender).
- **Die hohe Zahl von Lösungsmöglichkeiten:** Es gibt in der Softwareentwicklung praktisch eine unbegrenzte Anzahl von Möglichkeiten, eine bestimmte Aufgabe zu lösen. Die Entwicklung von Softwareprodukten unterliegt nicht jenen engen Schranken, wie sie für andere technische Produkte (z.B. durch Normen und Materialeigenschaften) gegeben sind. Die Schranken sind primär durch die Komplexität gegeben und lassen sich daher nur schwer im Vorhinein ermitteln.

4.5.2 Schwierigkeiten der Entwicklung

- **Die Individualität der Programmierer:** Auch heute wird noch vielfach die Softwareentwicklung mehr als Kunst denn als Technik gesehen. Softwaretechniker sind Individualisten mit großen Leistungsunterschieden; deshalb ist es besonders schwierig, den tatsächlichen Personalaufwand abzuschätzen. Außerdem lassen sich Individualisten meist nur ungern in ein organisatorisches Korsett zwingen.
- **Die raschen technologischen Veränderungen:** Die schnelle technologische Entwicklung der Hardware- und der Softwarebasis erschwert sowohl die Planung als auch

die Organisation von Softwareprojekten. Nicht selten kommen während der Entwicklung großer Softwaresysteme neue, leistungsfähigere Komponenten (z. B. erweiterte Grafik-Funktionalitäten oder Window-Managementsysteme) auf den Markt, welche die Konzeption des Systems bereits nach dem Design veraltet erscheinen lassen und eine Designänderung erzwingen. Dadurch wird die Planung oft vollständig über den Haufen geworfen. Andererseits werden häufig neue Softwarewerkzeuge eingesetzt, deren Nutzen ebenfalls schwer abzuschätzen ist.

- **Das Fehlen genormter Bauteile und Baugruppen:** In beinahe allen technischen Bereichen ist das Modellieren komplexer Systeme unter Zuhilfenahme getesteter und für tauglich befundener Bauteile und Baugruppen (Module) gang und gäbe. Es wird kein Produkt in voller Tiefe neu entwickelt, das Risiko mangelnder Produktqualität („Kinderkrankheiten“) wäre viel zu hoch. Diese Ansicht wird zwar im Software Engineering häufig diskutiert, neben der fehlenden Bereitschaft zur Verwendung von Baugruppen (Systemrahmen, Klassenbibliotheken) sind jedoch auch die verfügbaren Komponentenbibliotheken untereinander inkompatibel oder weisen immer dieselben Lücken auf.

4.5.3 Schwierigkeiten der Kontrolle

- **Das Immaterielle von Softwareprodukten:** Die „Unsichtbarkeit“ von Softwareprodukten erschwert die Kontrolle. Es ist nur schwer festzustellen, wie viel von einem Softwareprodukt tatsächlich fertig ist, und dem Programmierer stehen allerlei Möglichkeiten offen, den tatsächlichen Entwicklungsstand zu verschleiern. Die Kontrollaufgaben sind wesentlich schwieriger auszuüben als bei anderen technischen Projekten. Darüber hinaus sind vielfach die Qualitätsziele, so sie überhaupt definiert sind, nicht quantifiziert und damit nicht überprüfbar.

4.5.4 Reifegrad der ingenieurmäßigen Projektentwicklung

Verfügbare Zahlen verschiedener Monitoringeinrichtungen über Projekterfolge der amerikanischen Softwareindustrie sind in Abbildung 4.3 zusammengefasst. Den Daten liegt je Untersuchung eine Basis von einigen hunderttausend allgemeinen Softwareprojekten zu Grunde. Aus Sicht dieser Erfolgsstatistiken ergibt sich auf den ersten Blick ein ernüchterndes Bild: Die Untersuchungsergebnisse zeigen, dass sowohl Mitte der neunziger Jahre als auch zur Jahrtausendwende nur ca. ein Viertel aller Projekte als erfolgreich eingestuft wurde. Drei Viertel waren entweder beträchtlich über Budget oder über Zeit (im Durchschnitt ca. + 70 %) oder wurden wegen Erfolglosigkeit abgebrochen.

Als mögliche Ursachen für diese durchaus als katastrophal zu bezeichnende Situation werden in [Yourdon, 1997] angegeben, dass noch immer bei mehr als zwei Drittel der Softwareunternehmen keinerlei Praktiken des Software Engineering, wie z. B. Projektorganisation oder Festlegen von Projektphasen, eingesetzt werden oder dass keinerlei bewährte Praktiken („best practices“), wie z. B. Reviews, Definition des Begriffs „gut genug“ oder evolutionäres Prototyping, Anwendung finden.

Diese Ergebnisse werfen auf den ersten Blick kein gutes Licht auf das Gebiet der Softwareentwicklung. Man muss aber bedenken, dass der Umfang von Softwareprojekten kontinuierlich steigt und die Randbedingungen (Termine, Ressourcen etc.) immer schärfer werden. Dies

Erfolgsuntersuchungen an Softwareprojekten in den USA	Projekt erfolgreich	Projekt über Budget oder über Zeit	Projekt abgebrochen
Standish Group (1994) (http://www.standishgroup.com)	16 %	53 %	31 %
Center for Project Management (1995) (http://www.center4pm.com)	25 %	50 %	25 %
Standish Group (2000) (http://www.standishgroup.com)	28 %	49 %	23 %
Cutter Consortium (2000) (http://www.cutter.com)	16 %	63 %	21 %
Gartner Group (2000) (http://www.gartner.com)	24 %	51 %	25 %
Standish Group (2004) (http://www.standishgroup.com)	29 %	53 %	18 %

Abbildung 4.3: Entwicklung der Erfolgsstatistik von Softwareprojekten in den USA

war gerade in den neunziger Jahren in hohem Maß der Fall [Paulson, 2001]. Ohne Fortschritte im Projekt Engineering wäre die Erfolgsrate daher vermutlich beträchtlich gesunken!

Unter Berücksichtigung der laufenden Verschärfung der Rahmenbedingungen von Projekten, gepaart mit der explosionsartigen Entwicklung der Technologie, sollte die annähernd gleich bleibende Erfolgswahrscheinlichkeit moderner Softwareprojekte daher nicht als Stagnation, sondern durchaus als – wenn auch sehr bescheidener – Erfolg gewertet werden.

4.6 Aufgabenschema des Projekt Engineering

Wenn Produktentwicklungszyklus und Projektentwicklungszyklus kombiniert betrachtet werden, ergeben sich folgende *Aufgabenbereiche des Projekt Engineering*, koordiniert durch die Projektorganisation (Abbildung 4.4, vgl. [Mayr, 1997]):

- Zielerforschung,
- Analyse,
- Design,
- Implementierung,
- Testen,
- Produkt-Einführung,
- Reengineering.

Wie beispielsweise in [Pfleeger, 2001] angemerkt wird, herrscht in der Literatur eine große Übereinstimmung über die Existenz dieser Aufgabenbereiche, die oftmals als *Projektphasen* bezeichnet werden, vor, allerdings werden deren Bezeichnungen sowie die exakte Grenzziehung zwischen den Phasen (so eine solche überhaupt möglich ist) heftig diskutiert. In den folgenden Kapiteln werden diese Aufgabenbereiche im Detail diskutiert, beginnend mit der Projektorganisation.

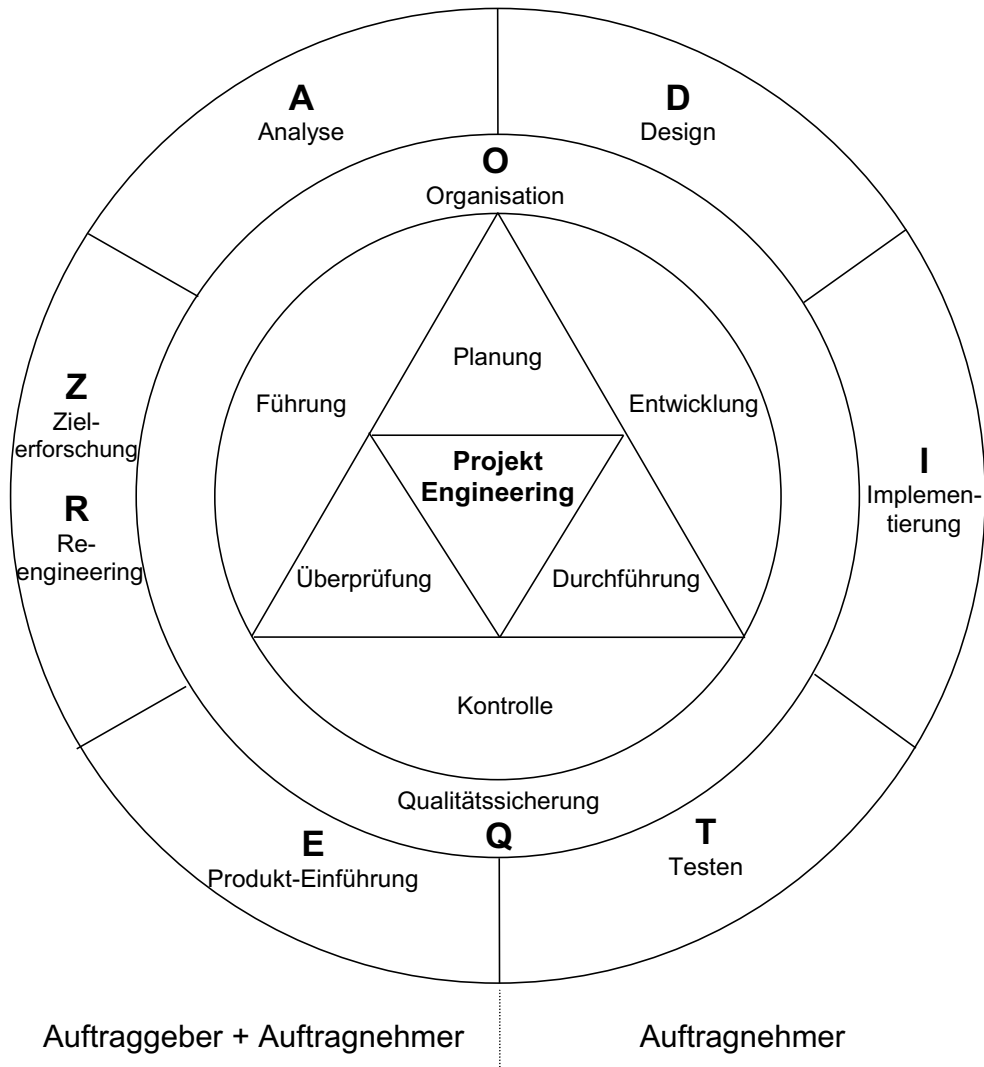


Abbildung 4.4: Aufgabenschema des Projekt Engineering

Teil II

Projektbegleitende Aufgabenbereiche

5 Aufbauorganisation

In diesem Kapitel wird eine detaillierte Einführung in für die Softwareentwicklung geeignete Formen der Aufbauorganisation von Projekten gegeben. Die *Aufbauorganisation* beschreibt die Strukturen eines Systems. Im Folgenden werden zuerst die „klassischen“ Organisationsstrukturen aus der Organisationslehre vorgestellt. Nach einer Analyse, warum diese Formen für eine Softwareentwicklung nicht unmittelbar übernommen werden sollten, werden besser für diese Zwecke adaptierte Organisationsformen vorgestellt. Abschließend werden ökonomisch-soziale Aspekte einer Softwareentwicklung in Kleingruppen behandelt.

5.1 Allgemeine Organisationsformen

5.1.1 Reine Projektorganisation

Die *reine Projektorganisation* (*Taskforce*) ist eine ausschließlich projektbezogene Organisation, die parallel zur Unternehmensorganisation errichtet wird. Die reine Projektorganisation besitzt daher ein hohes Maß an Eigenständigkeit, da der Organisationsleiter (Projektleiter) im Allgemeinen der Unternehmensleitung direkt unterstellt ist.

Der Projektleiter leitet die Projektmitarbeiter direkt und ist seinem Vorgesetzten (der Unternehmensführung) für die Projektdurchführung verantwortlich. Er verfügt über Entscheidungskompetenzen und Anordnungsbefugnisse in Bezug auf die Projektmitarbeiter und alle zur Verfügung stehenden Mittel.

Die Projektmitarbeiter bilden für die Projektdauer eine eigene Organisationseinheit. Sie werden aus der bestehenden Organisation herausgelöst oder neu aufgenommen. Für die Projektdauer sind sie dem ursprünglichen Vorgesetzten *nicht* unterstellt.

Die Vorteile der reinen Projektorganisation sind:

- eindeutige Zuordnung von Aufgaben und Kompetenzen an den Projektleiter,
- volle Konzentration auf das Projekt,
- Vorhandensein einer Anlaufstelle in allen Projektbelangen sowohl für den Auftraggeber als auch für die Projektmitarbeiter in Person des Projektleiters,
- Identifikation der Projektmitarbeiter mit der Aufgabe,
- Möglichkeit einer raschen projektbezogenen Entscheidungsfindung,
- Möglichkeit der flexiblen Reaktion auf veränderte Situationen.

Die vollzeitliche Abstellung von Mitarbeitern für ein Projekt bringt aber auch Probleme mit sich:

- qualitative und quantitative Schwächung der betroffenen Abteilung,
- Abstellung jener Mitarbeiter, die in der Abteilung am wenigsten fehlen,
- stärkere Belastung der in der Abteilung verbleibenden Mitarbeiter,
- bei der Beschaffung notwendiger Informationen von Projektmitarbeitern aus ihrer angestammten Abteilung manchmal Störungen und Konflikte,
- Wiedereingliederung nach Projektende,
- unterschiedliche Auslastung der Projektmitarbeiter in den einzelnen Phasen.

In allgemeinen Organisationsbereichen wird die reine Projektorganisation für (sehr) kleine Projekte eingesetzt, die von besonderer Schwierigkeit sind.

5.1.2 Einfluss-Projektorganisation

Die *Einfluss-Projektorganisation* lässt die funktionale Hierarchie im Unternehmen unverändert. Der Projektleiter übernimmt eine Stabsfunktion im traditionellen Sinn. Dadurch werden seine Kompetenzen stark eingeschränkt. Die Aufgaben des Projektleiters sind daher eine reine Koordinations- und Informationsfunktion zwischen am Projekt beteiligten Stellen und Mitarbeitern. Er ist für Planung, Verfolgung des Projektverlaufs und die Meldung von Planabweichungen zuständig, kann allerdings durchzuführende Maßnahmen an Linieninstanzen nur vorschlagen.

Die Vorteile der Einfluss-Projektorganisation sind:

- Die bestehende Organisation wird nicht verändert.
- Die Mitarbeiter verbleiben in ihren Abteilungen.

Probleme der Einfluss-Projektorganisation sind:

- Nichtübereinstimmung von Autorität und Verantwortung beim Projektleiter,
- Zeitverzögerung bei nicht lösbaren Konflikten (der Projektleiter muss Probleme „nach oben“ melden),
- Fehlen eines zugleich kompetenten und entscheidungsberechtigten Gesprächspartners beim Auftraggeber.

Die Einsatzbereiche der Einfluss-Projektorganisation beschränken sich daher auf gut strukturierte Aufgaben, deren Vorgehensweisen den Mitarbeitern bekannt sind. Sie ist dementsprechend für Projekt Engineering im Softwarebereich im Allgemeinen nicht geeignet.

5.1.3 Matrix-Projektorganisation

Die *Matrix-Projektorganisation* ist eine zweidimensionale Organisationsform: Sie besteht aus einer vertikalen Dimension (Hierarchie) und einer horizontalen (Projekte). Der Abteilungsleiter ist Personalvorgesetzter, der Projektleiter ist Fachvorgesetzter. Es gibt daher „Punkte des beabsichtigten Konflikts“ in den Kreuzungsfeldern der Matrix. Die Mitarbeiter unterliegen nämlich einer Doppelunterstellung (horizontal und vertikal). Dies soll Konflikte möglichst früh offen legen, um rechtzeitige und insgesamt bessere Lösungen zu finden. Von allen Beteiligten sind dazu hohes Führungsverständnis und große Konfliktlösungsfähigkeit erforderlich.

Die Vorteile der Matrix-Projektorganisation sind:

- Nutzung der Projektmitarbeiter als „Frühwarnsystem“,
- Verfolgung der Projektziele vom Projektleiter selbstständig und direkt,
- gezielter Einsatz von Spezialwissen und besonderen Erfahrungen,
- Stärkung des Sicherheitsgefühls der Projektmitarbeiter.

Probleme der Matrix-Projektorganisation sind:

- Verzicht der Vorgesetzten auf Ausschließlichkeitsansprüche (Verunsicherung bei Vorgesetzten),
- Mehrfachunterstellung der Mitarbeiter (Verunsicherung der Mitarbeiter),
- Kompetenzkonflikte,
- Möglichkeit, die Vorgesetzten gegeneinander auszuspielen.

Durch die Matrix-Projektorganisation ergibt sich ein organisatorischer Zwang zur Gemeinsamkeit: Weder der funktionale Vorgesetzte noch der Projektleiter allein entscheiden und handeln. Dies soll in der Theorie zu besseren Lösungen führen, verzögert in der Praxis die Lösungen aber häufig. Die Matrix-Projektorganisation führt zu einer beträchtlichen Schwächung der Projektleistung durch inadäquat lange Entscheidungsfindungsprozesse und ist daher für Softwareprojekte eher ungeeignet.

5.2 Softwareentwicklungsspezifische Organisationsformen

5.2.1 Besondere Anforderungen an Projektorganisationsstrukturen

Projekte sind zeitlich begrenzt. Daher sind auch projektbezogene organisatorische Regelungen in ihrer Dauer im Allgemeinen auf die Projektzeit bezogen.

Änderungen der Projektumwelt sowie der internen Projekterfordernisse bedingen eine hohe Flexibilität der gesamten Projektstruktur. Daher ist auch die Organisationsstruktur laufend anzupassen.

Der Abschluss eines Projekts bedeutet meist auch die Auflösung der Projektorganisation. Um einerseits eine personelle Kontinuität des Unternehmens, andererseits aber auch projektnachfolgende Tätigkeiten (z.B. Wartung) zu ermöglichen, ist eine projektübergreifende Planung notwendig.

5.2.2 Besondere Eigenschaften der Softwareentwicklung

Seit Beginn der Programmierarbeit wird in der Softwarebranche diskutiert, ob die Programmentwicklung eine Produktionstätigkeit wie jede andere ist oder ob sie etwas Besonderes ist, auf welches die herkömmlichen Regeln der Organisation nicht anwendbar sind. In [Elzer, 1994] wird dazu angemerkt, dass Programmentwicklung wahrscheinlich eher mit herkömmlichen Planungsaufgaben vergleichbar ist und weniger als Produktionsprozess gesehen werden sollte. Es wird daher angeregt, dass die Softwareentwicklung nach dem Muster von Planungsbüros, wie etwa denen von Architekten, organisiert werden sollte.

In [Elzer, 1987] wird ein weiterer spezieller Aspekt der Softwareentwicklung betont: „Die Mehrheit der Softwarespezialisten (zumindest in Europa) haben einen akademischen Grad erworben, wenn auch vielfach nicht in Software. Das bedeutet, dass sie in einer Tradition ausgebildet wurden, wo sie an der Fähigkeit gemessen wurden, einmalige oder zumindest außerordentliche Ergebnisse zu erreichen. Außerdem haben gewöhnlich Hochschulabsolventen nie die Notwendigkeit gelernt, Regeln strikt einzuhalten. Diese beiden Ausgangsvoraussetzungen machen es sehr schwer, mit diesem Menschentyp größere Teams zu bilden. Andererseits sind aber bis heute bei Softwareprojekten die wiederholbaren und feststehenden Arbeitsanteile wesentlich kleiner als in traditionellen Konstruktionsprojekten.“ Wie in [Elzer, 1994] dazu angemerkt wird, ist gerade im Planungsbüro der Anteil an Akademikern traditionell gering. Diese werden von einer weitaus größeren Zahl von Detailkonstrukteuren, Zeichnern etc. unterstützt.

Aus diesen Eigenheiten lassen sich Argumente für die Unzulänglichkeit traditioneller Organisationsformen ableiten:

- Hierarchische Kommunikationswege beeinträchtigen Qualität, Quantität und Schnelligkeit des Informationsflusses.
- Der Problemstau bei Führungskräften sowie die Überlastung der Unternehmensspitze führt zu längeren Entscheidungsprozessen.
- Gegenseitige Abhängigkeit von Stellen bedeutet hohes Konfliktpotenzial.
- Mangelhaftes Ineinandergreifen wichtiger Unternehmensbereiche führt zu Uninformiertheit und Unkoordiniertheit.
- Realisierbarkeit, Kosten und voraussichtlicher Zeitbedarf werden falsch beurteilt.

Es ist daher notwendig, für das Projekt Engineering im Softwarebereich spezielle Organisationsstrukturen zu bilden.

5.2.3 Einzelkämpfer – Gruppe – Programmierheer

Während es noch vor 15 Jahren üblich war, in Klein- und Mittelunternehmen auf den für alles zuständigen und auf sich selbst gestellten „EDV-Universalexperthen“ zu treffen, während sich große Hardware- und Softwarehersteller damit brüsteten, wie viele hunderte (oder sogar tausende) Mannjahre in ihren Produkten steckten, ist in den letzten Jahren ein allgemeiner Trend zur Mitte zwischen „*Einzelkämpfertum*“ und „*Programmierheeren*“ – der *Arbeitsgruppe* – zu bemerken. Tabelle 5.1 soll diese Entwicklung anhand wichtiger Projektqualitätskriterien begründen.

5.2.4 Hierarchische Teamorganisation

Die Organisation von Softwareprojekten orientierte sich lange Zeit an der in anderen Industriezweigen üblichen hierarchischen Organisation. Dabei wird der Zerlegung der Softwareentwicklung in einzelne Phasen besondere Bedeutung zugemessen. Für alle Phasen werden verantwortliche Leiter eingesetzt, die von einem Projektleiter geführt und kontrolliert werden und die ihrerseits – je nach Größe des Projekts – Mitarbeiter oder Gruppenleiter führen und kontrollieren. Dem Projektleiter wird in der Regel noch ein Projektstab mit beratenden und administrativen Aufgaben zur Seite gestellt. Je größer das Projekt ist, desto größer ist die Anzahl der Hierarchiestufen im Organisationsschema.

Diese Organisationsform weist für die Softwareentwicklung einige gravierende Schwächen auf [Pomberger und Blaschek, 1996]:

- Der Projektleiter ist zu weit von der eigentlichen Programmierung entfernt und kann dadurch seine Planungs- und Kontrollaufgaben nur mangelhaft wahrnehmen.
- Die mehrstufige Organisationshierarchie behindert die Kommunikation und Projektfortschrittskontrolle. Gerade in Softwareprojekten ist aber eine reibungslose Kommunikation der Projektmitarbeiter von besonderer Bedeutung für einen erfolgreichen Projektabschluss.

Kriterium	Einzelkämpfer	Gruppe	Programmierheer
<i>Organisation</i>	Improvisation, hohes Ausfallrisiko	gemeinsames Arbeiten, Gruppendynamische Effekte	Aufgaben(zer)-teilung, geringes Produktrisiko
<i>Motivation</i>	extrem hoch, Selbstausbeutung	hoch (Klima ist entscheidend), Tendenz zur Ausbeutung	niedrig, „Mitschleppen“ unproduktiver Mitarbeiter
<i>Vorgehensweise</i>	innovativ bis verspielt, unreflektierte Entscheidungen	innovativ, direkte Kommunikation, rasche Fehlerkorrektur	starr, indirekte Kommunikation, komplexe Methoden und Werkzeuge
<i>Dokumentation</i>	stark entwicklerbezogen, primär für Eigenbedarf	standardisiert, knapp	standardisiert, umfangreich
<i>Testen/Kontrolle</i>	ungenügend (Personalunion!)	führungsabhängig, Tendenz zu gering	umfangreich, sehr indirekt (Ablehnung!)
<i>Projektgröße</i>	klein	klein bis mittel	„immer“ groß

Tabelle 5.1: Personalorganisation und Qualitätskriterien

- In einer Hierarchie wird sehr leicht das so genannte *Peterprinzip* wirksam, d.h., jeder Mitarbeiter hat die Tendenz, bis zur Stufe seiner Unfähigkeit aufzurücken. Die Projektmitarbeiter streben also nach der Stufe ihrer Inkompetenz, und ihre eigentliche Qualifikation kann nicht ausgenutzt werden.

Moderne Ansätze der Organisation im Softwareentwicklungsbereich verzichten daher weitgehend auf Hierarchien und verfolgen einen eher demokratischen Organisationsstil.

5.2.5 Das Chefprogrammierer-Team

Dieses Organisationsmodell wurde von Baker [Baker, 1972] erstmals angegeben, indem positive Erfahrungen bei der Zusammenarbeit von Chirurgen präzisiert wurden. Im *Chefprogrammierer-Team* wird versucht, die Nachteile hierarchischer Organisationsmodelle zu vermeiden. Es ist im Wesentlichen gekennzeichnet durch:

- den Verzicht auf einen Projektleiter, der nicht an der Systementwicklung selbst beteiligt ist,
- den Einsatz von sehr guten Spezialisten,
- die Beschränkung der Teamgröße.

Das Chefprogrammierer-Team setzt sich zusammen aus [Baker und Mills, 1979]:

- dem Chefprogrammierer,
- dem Projektassistenten,
- dem Projektsekretär
- und den Spezialisten (Sprachspezialisten, Programmierer, Testspezialisten, ...).

Der *Chefprogrammierer* ist sowohl in den Planungs-, Analyse- und Designprozess und idealerweise auch in den Implementierungsprozess aktiv eingebunden. Er kontrolliert den Projektfortschritt, entscheidet in allen wichtigen Fragen und ist für alles verantwortlich. Die fachlichen Anforderungen an den Chefprogrammierer sind dementsprechend hoch.

Der *Projektassistent* ist der engste technische Mitarbeiter des Chefprogrammierers. Seine Aufgabe ist es, dem Chefprogrammierer bei allen wesentlichen Tätigkeiten zu assistieren, ihn zu unterstützen und im Bedarfsfall zu ersetzen. Seine fachliche Qualifikation soll daher gleich hoch sein wie die des Chefprogrammierers.

Der *Projektsekretär* hat die Aufgabe, den Chefprogrammierer und auch alle anderen Programmierer von Verwaltungsaufgaben zu entlasten. Er verwaltet alle Programme und Dokumente und wird auch zur Projektfortschrittskontrolle herangezogen. Seine Hauptaufgabe ist die Verwaltung der Projektbibliothek.

Die Anzahl und Art der *Spezialisten* wird vom Chefprogrammierer nach Bedarf festgelegt. Sie werden für die Auswahl der Entwicklungsumgebung, die Implementierung der einzelnen Systemkomponenten, die Auswahl und den Einsatz von Softwarewerkzeugen und die Durchführung der Tests herangezogen.

Die Vorteile dieser Organisationsform sind:

- Der Chefprogrammierer ist direkt in die Systementwicklung involviert und kann daher seine Kontrollfunktion besser wahrnehmen.
- Die Kommunikationsschwierigkeiten, die bei rein hierarchischer Organisation auftreten, werden abgeschwächt.
- Der Berichtsweg über den Projektfortschritt ist institutionalisiert.
- Kleine Teams sind in der Regel produktiver als große Teams.

Die Nachteile dieser Organisationsform sind:

- Sie ist auf kleine Teams beschränkt. Nicht jedes Projekt kann mit wenigen Mitarbeitern realisiert werden.
- Die Personalanforderungen sind nahezu unerfüllbar. Es gibt wenige Softwaretechniker, welche die Qualifikation für einen Chefprogrammierer oder für einen Projektassistenten haben.

- Der Projektsekretär hat eine äußerst schwierige und verantwortungsvolle Aufgabe zu erfüllen, die aber zum Großteil aus Routinearbeiten besteht; er hat daher eine untergeordnete Stellung. Dies hat große psychologische Nachteile. Wegen seiner zentralen Stellung kann der Projektsekretär leicht zum Engpass werden.
- Das Organisationsmodell sieht keinen Ersatz für den Projektsekretär vor. Ein Ausfall des Projektsekretärs hätte aber schwerwiegende Folgen für die weitere Projektabwicklung.

Ungeachtet der Nachteile des Chefprogrammierer-Organisationsmodells haben die Erfahrungen gezeigt, dass kleine Teams, bestehend aus erfahrenen Softwareingenieuren, wesentlich produktiver, zuverlässiger und termingerechter arbeiten als große „Programmierheere“.

5.2.6 Projektgruppe

Folgende Änderungen in der Softwareerstellung prägten die Entwicklung vom Chefprogrammierer-Team zum modernen Konzept der *Projektgruppe* entscheidend:

- Ein Softwareingenieur trennt in der modernen Auffassung seines Arbeitsbereiches nicht mehr zwischen Design, Implementierung und Testen. Er muss alle diese Techniken beherrschen und stark ineinander verwoben verwenden können.
- In Notfällen muss für jeden Mitarbeiter kurzfristig – und zumindest für kurze Zeit – ein Stellvertreter gefunden werden können, der über den aktuellen Projektstand informiert ist. Durch den breiteren Zuständigkeitsbereich der Experten sowie dieser Stellvertreterfunktion (Möglichkeit des Rollenwechsels) wird im Allgemeinen nicht mehr von Teamarbeit (die ja die Spezialisierung betonte) gesprochen, sondern von Gruppenarbeit (die den holistischen Projektansatz für jeden Mitarbeiter betont).
- Ein Chefprogrammierer, der perfekte Managementqualitäten aufweist und optimal an der Implementierung mitarbeiten kann, ist in der Praxis nicht zu finden. Ein moderner Projektleiter sollte inhaltliches Verständnis für die Implementierung mitbringen, ist aber nicht in der Implementierung tätig.
- Die undankbare – und inhaltlich oft weniger interessante – Aufgabe eines Projektsekretärs als „Chefdokumentierer“ und Kontrollor macht diese Stelle auf die Dauer unbesetzbar. Speziell die Dokumentationsaufgabe muss anfallsbezogen verteilt werden.

Eine Projektgruppe besteht daher aus einem Projektleiter und Gruppenmitgliedern, welche die zu besetzenden Rollen vom Projektleiter zugeteilt bekommen, jedoch auch zumindest ein fundiertes Grundverständnis der Rollen der anderen Gruppenmitglieder aufweisen müssen („*Leitergruppe*“). Für die zu besetzenden Rollen sei auf Abschnitt 7.3 verwiesen.

Voraussetzungen für die erfolgreiche Arbeit einer Projektgruppe sind:

- Identifikation aller Gruppenmitglieder mit den Projektzielen,
- Fähigkeit zur (demokratischen) Zusammenarbeit,

- Fachwissen und fachbezogenes Allgemeinwissen der Gruppenmitglieder,
- Führungsvermögen des Projektleiters,
- Fähigkeit des Projektleiters zur dauernden Motivation und Konfliktlösung.

Durch das bewusste Zulassen gruppendynamischer Prozesse zur Selbstordnung (vgl. [Schwaber und Beedle, 2001]) ist vor allem zu Projektbeginn entsprechende Projektzeit vorzusehen, unter anderem zur Formierung der Gruppe und zur Abklärung des Verhältnisses der Gruppenmitglieder zueinander. Da Softwareentwicklung großteils eine Planungstätigkeit – und damit geistiger Natur – ist, muss der Mitarbeiterführung eine sehr große Bedeutung zugemessen werden und der Projektleiter entsprechend eine große soziale Kompetenz aufweisen. In der Praxis zeigt sich, dass die Identifikation mit den Projektzielen in einer Organisation als Projektgruppe höher ist als im Chefprogrammierer-Team.

Eine Integration der Projektgruppe in die klassischen Organisationsformen ist auf zwei Arten möglich:

1. **Als projektorientierte Teilorganisation:** Die Projektgruppe wird aus Mitarbeitern einer einzigen Abteilung gebildet. Die Linieninstanz überträgt gewisse Kompetenzen an einen ihr ohnehin unterstellten, der Abteilung jedoch übergeordneten Projektleiter. Die eindimensionale Organisationsform bleibt somit erhalten.
2. **Durch Projektleitung in der Linienorganisation:** Die Verantwortung für das Projekt wird der Fachabteilung mit dem größten Anteil am Projekt zugewiesen. Die eindimensionale Organisationsform bleibt größtenteils erhalten.

5.3 Softwareentwicklung als humanzentrierte Aufgabe

5.3.1 Gruppendynamische Prozesse

Bedingt durch die laufende Evolution, der eine Softwareentwicklung unterworfen ist, und dem Faktum, dass moderne Softwareentwicklungen durchwegs von Personengruppen und kaum mehr von Einzelkämpfern durchgeführt werden, ist die *Kommunikation der Mitglieder* untereinander und deren Stimulation und Koordination durch den Projektleiter ein, wenn nicht der entscheidende Faktor für den Projekterfolg. Softwareentwicklung wird daher vielfach als *humanzentrierte Tätigkeit* bezeichnet [Humphrey, 1997].

Dabei ist zu beachten, dass gerade technische Führungskräfte im Allgemeinen dazu neigen, die psychologischen und sozialen Aspekte in Arbeitsgruppen zu unterschätzen: Konflikte müssen, sobald sie erkannt werden, angesprochen und behoben werden. Im Bereich technischer Entwicklungen gibt es oftmals nicht die Möglichkeit zum Kompromiss: es gibt „Gewinner“ und „Verlierer“, was erneut Konflikte auslösen kann. Auf die Dauer ist es jedoch für die Mitarbeiter und das Projekt untragbar, Konflikte um jeden Preis zu vermeiden.

Durch das bewusste Zulassen *gruppendynamischer Prozesse* zur Selbstordnung ist vor allem zu Projektbeginn entsprechende Projektzeit vorzusehen, unter anderem zur Formierung der Gruppe und zur Abklärung des Verhältnisses der Gruppenmitglieder zueinander. Wie Jens

Coldewey dazu anmerkt, ist es „... denkbar, Software weitgehend ohne definierten Prozess ... zu entwickeln. Es ist aber völlig unmöglich, sie ohne ein qualifiziertes Team zu schreiben, auch wenn diese Erkenntnis noch nicht zu jedem Projektmanager durchgedrungen zu sein scheint. Auch der beste Prozess wird aus einem chaotischen Haufen von Anfängern kein professionelles Team machen. Auf der anderen Seite zeigen praktisch alle professionellen Teams einen hohen Grad an Selbstorganisation, ...“ [Coldewey, 2002]. Barry Boehm stellt dazu (im Rahmen einer Diskussion über agile Methoden und deren Fundierung auf gute Entwickler) zynisch fest, dass unvermeidlicherweise 49,99 % der Softwareentwickler unterdurchschnittlich qualifiziert sind [Boehm, 2002].

Da Softwareentwicklung großteils eine Planungstätigkeit – und damit geistiger Natur – ist, muss der Mitarbeiterführung eine sehr große Bedeutung zugemessen werden und der Projektleiter entsprechend eine große soziale Kompetenz aufweisen. Durch die heterogene Zusammensetzung vieler Entwicklergruppen ist diese Herausforderung bei der Entwicklung moderner Software oft sogar noch größer als in anderen technischen Disziplinen.

Gerade in der Softwareentwicklung ist die Bedeutung von Ethik und Moral äußerst hoch. Oftmals sind Überprüfung und Kontrolle bei Projekten zu gering ausgeprägt bzw. nur in ungenügendem Ausmaß möglich. Dadurch ist die Verpflichtung des Mitarbeiters, von selbst seine Arbeit so gut wie möglich zu machen, ein auch ökonomisch äußerst wichtiger Faktor (vgl. den Entwurf einer „Profession des Softwareingenieurs“ in [Ford und Gibbs, 1996] sowie erste Erfahrungen damit in [Meyer, 2001]).

5.3.2 Motivation der Mitarbeiter

Der Bereich der *Mitarbeitermotivation* ist äußerst vielschichtig und Fächer übergreifend. Für eine umfassende Behandlung sei daher auf die Literatur verwiesen; z. B. gibt [Turkle, 1986] eine grundlegende Einführung. Folgende *Faktoren* sollten jedoch zur Motivation von Mitarbeitern besonders beachtet werden:

- Planvorgaben müssen realistisch sein.
- Kompetenzen müssen klar festgelegt sein.
- Jedes Gruppenmitglied muss sich individuell wichtig und für den Projekterfolg bedeutsam fühlen.
- Akzeptable Arbeitsbedingungen müssen vorhanden sein.
- Der Projektleiter muss (in einem vernünftigen Rahmen) auch Anlaufstelle für persönliche Probleme der Mitarbeiter sein.

Wie z. B. in [Elzer, 1994] angemerkt wird, führt dauernde leichte *Überforderung* zu besserer Arbeit eines Mitarbeiters. Dieser Standpunkt wird von Menschen ohne Berufserfahrung meist sehr skeptisch gesehen. Die Erfahrung bestätigt aber, dass Menschen besser arbeiten und sich zufriedener fühlen, wenn sie dazu gebracht werden, etwas mehr zu leisten, als sie ursprünglich von sich selbst erwartet hatten [Elzer, 1994].

Allerdings ist es auch für erfahrene Projektleiter oftmals schwer, den richtigen Forderungsgrad für jeden Mitarbeiter festzulegen; bei zu großer Überforderung sinkt die Leistung nämlich

extrem rasch ab. Auch ist ein gewisses Maß an *Freiraum* (*slack*) für jeden Mitarbeiter notwendig, um seinen Aufgabenplan nach Bedarf umstrukturieren bzw. auf unvorhergesehene Ereignisse rascher reagieren zu können [DeMarco, 2004].

5.3.3 Produktivität

Die *Leistungsfähigkeit* einzelner, gleich gebildeter Programmierer ist einer erheblichen Streuung unterworfen. In einer immer wieder zitierten Studie [Schnupp und Floyd, 1976] werden folgende Verhältnisse zwischen jeweils bester und schlechtester Programmierleistung angegeben (jeweils bezogen auf die gleiche Aufgabe):

- Programmumfang: 5 : 1,
- Programmierzeit: 25 : 1,
- benötigte Testzeit: 26 : 1,
- Laufzeit des Ergebnisses: 13 : 1.

Das Kalkulieren mit Durchschnittsleistungswerten von Programmierern hat also nur sehr begrenzte Bedeutung.

Eine wesentliche Einflussgröße für die *Produktivität einer Projektgruppe* ist die Gruppengröße. [Brooks, 1975] gibt an, dass der Zeitbedarf für jede in der Gruppe geleistete Arbeit aus zwei Grundkomponenten besteht:

1. der produktiven Arbeit (*Nutzleistung*),
2. der Kommunikation und der gegenseitigen Abstimmung der Mitarbeiter untereinander (*Bindleistung*).

Wäre keine Kommunikation zwischen den Gruppenmitgliedern notwendig, so würde der Zeitbedarf mit steigender Anzahl der Mitglieder linear abnehmen. Nimmt man aber an, dass jeder Mitarbeiter mit jedem anderen Informationen austauschen muss, so ergibt sich für die Entwicklungszeit folgende Näherungsformel (t = Projektentwicklungszeit, n = Anzahl der Mitarbeiter, k = durchschnittliche Kommunikationszeit):

$$t \approx 1/n + kn^2/2.$$

Die Konsequenz dieses *brooksschen Gesetzes* ist: Ein Projekt, das – bei ausreichender Mitarbeiteranzahl – in Verzug ist, gerät durch Hinzufügen neuer Mitarbeiter nur noch mehr in Verzug!

Weiters kann aus diesem Gesetz eine optimale Gruppengröße abgeleitet werden, die nicht überschritten werden darf, wenn ein Softwareentwicklungsprojekt innerhalb einer vernünftigen Zeitspanne durchführbar sein soll (vgl. Abbildung 5.1, aus [Rausch-Schott und Rettschitzegger, 1993]). Als Näherungsformel wird an selber Stelle vorgeschlagen (MM bedeutet „Mannmonate“ bzw. Personenmonate):

$$n_{\text{optimal}} = \sqrt{t}, \text{ wobei } [t] = MM.$$

Wie in [Elzer, 1994] angemerkt wird, kann im Allgemeinen ein Projekt mit wesentlich weniger Aufwand durchgeführt werden, wenn man eine geringfügig höhere Zeitdauer zulässt.

Tipp: Eine bewährte Erfahrung aus der Praxis ist das „Zusammenschweißen“ von je zwei Mitarbeitern zu Subgruppen und das Bilden der Projektgruppe aus 2, 2×2 oder 2×4 Mitarbeitern plus Projektleiter.

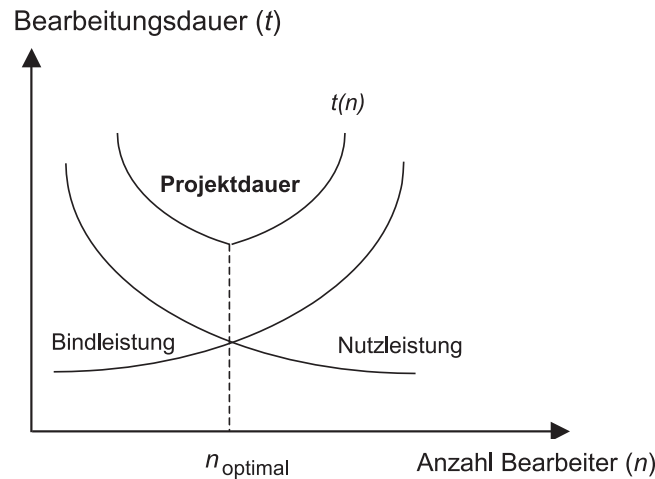


Abbildung 5.1: Ermittlung der (theoretisch) optimalen Gruppengröße bei Projekten

5.3.4 Interdisziplinäre Zusammenarbeit

Die Schlüsselaufgabe für den Leiter moderner Softwareprojekte ist, dass er ein Team von Personen mit verschiedensten Fertigkeiten führen muss. Die Teammitglieder sind oft Spezialisten mit unterschiedlichsten Kenntnissen, Gewohnheiten und Wertigkeitsskalen. Vielen – häufig jungen – Entwicklern sind klassische Vorgehensmodelle und -methoden zur Sicherung/Erhöhung der Entwicklungsreife (wie etwa CMMI oder ISO/IEC 15504; vgl. Abschnitt 8) nicht bekannt. Zeit, sich in diese Vorgehensmodelle einzuarbeiten bzw. diese sogar anzuwenden, ist sowieso nicht vorhanden. Prozessentwicklung, Disziplin oder Schätzen wird als unnötiger Ballast abgeworfen.

Jeder Entwicklertyp scheint Probleme zu haben, die Beiträge von Entwicklern mit anderem Ausbildungshintergrund anzuerkennen und wertzuschätzen [McDonald und Welland, 2001]. Umgekehrt tun sich Experten schwer, Umfänge von Aufgaben, die nicht in ihre Domäne fallen, auch nur einigermaßen brauchbar abzuschätzen. Dieses Verhalten ist in den verschiedensten, häufig stark anwendungsbezogenen Softwareentwicklungsbereichen anzutreffen und wurde vom Autor beispielsweise im Rahmen der Betreuung einer Gruppe von Spieleentwicklern festgestellt, bei der Experten wie Game Designer, Asset Manager und Implementierer äußerst konfliktär miteinander umgingen. Vielfach fällt dem Projektleiter hier die Rolle eines Übersetzers und Mediators zu, der sowohl Inhalt als auch Beweggründe für Entscheidungen von einer Domäne und deren Sprache in die andere übersetzen muss.

Entscheidend ist, dass jeder Mitarbeiter seine übernommenen Rollen und Verantwortlichkeiten versteht. Bei Überlappungen ist es gleichermaßen Aufgabe des Mitarbeiters und des Projektleiters, daraus entstehende Konflikte auf die für das Projekt am besten geeignete Weise zu lösen. Auf Grund der kurzen Entwicklungszeiten bei modernen Softwareprojekten ist es wichtig, die Konflikte rasch zu lösen, auch wenn dadurch aus der Gesamtsicht suboptimale Lösungen entstehen.

Viele junge Entwickler zeigen auf Grund ihrer jugendlichen Begeisterung großes Interesse an neuen Werkzeugen und Technologien, aber auch entsprechende Ignoranz gegenüber bestehendem Wissen und bewährten Praktiken. Sie probieren gerne neue Werkzeuge und Softwaresysteme aus, das Update auf jeweils letzte Versionen ist ein Muss. Dies führt dazu, dass oft eine Vielzahl von Werkzeugen und Technologien gleichzeitig in derselben Organisationseinheit benutzt wird, ohne dafür klare Begründungen liefern zu können außer Entwicklerpräferenz, vgl. [McDonald und Welland, 2001]. Andererseits können ältere Mitarbeiter oft kaum mehr freiwillig dazu gebracht werden, ihre „vertraute Umgebung“ zu verlassen und auf moderne Softwareentwicklungsumgebungen umzusteigen.

Aufgabe des Projektleiters ist es, die Begeisterung der jungen Mitarbeiter zur Auswahl geeigneter Technologien und Werkzeuge zu nutzen. Die Bereitschaft zur Schulung und Weiterbildung ist gerade bei diesen Mitarbeitern äußerst groß, wobei es unerheblich ist, ob diese Schulungen in der Dienst- oder Freizeit angeboten werden. Nach dem Festschreiben der Entwicklungsumgebung und der Vorgehensmodelle ist jedoch eine geordnete Änderungs- und Updatepolitik festzulegen sowie dafür zu sorgen, dass diese auch eingehalten wird. Die vorgegebenen Werkzeuge müssen auch tatsächlich verwendet werden und es darf auf keine anderen umgestiegen werden, bevor diese geprüft und freigegeben wurden.

5.4 Organisationsmängel bei Softwareprojekten

Es kommt immer wieder vor, dass Softwareprojekte nicht termingerecht und mit wesentlich höheren Kosten als zu Projektbeginn angenommen fertig gestellt werden oder dass Softwareprojekte überhaupt scheitern. Die Ursache dafür ist vielfach auf *Organisationsmängel* zurückzuführen, wie in [Pomberger und Blaschek, 1996] angemerkt wird. Die wesentlichsten, dort angeführten Organisationsmängel sind:

- **Fehlen von Projektplanung, Projektorganisation allgemein sowie von Projektstandards:** Softwareprojekte müssen wegen ihrer hohen Komplexität besonders sorgfältig geplant und organisiert werden. Programmierer sind Individualisten, sie bezeichnen sich selbst oft als Künstler und legen ihre Karten nicht gerne auf den Tisch. Diese Tatsache erweist sich bei arbeitsteiliger Softwareentwicklung als sehr störend, und das Fehlen eines systematisch erstellten Projektplans, die mangelnde Vorgabe und Durchsetzung einer Vorgehensmethode sowie das Fehlen von Projektstandards und Meilensteinen zur Projektfortschrittskontrolle wirkt sich in großen Softwareprojekten oft verheerend aus, weil mangelnde Qualität der einzelnen Systemkomponenten und Terminverzug dann erst spät offenbar werden.
- **Inkompetenz der Projektführung:** Die Projektführung ist meist streng hierarchisch organisiert, und die Leitung obliegt nicht selten entweder dem Softwaretechniker

mit wenig organisatorischer und kalkulatorischer Erfahrung oder einem qualifizierten Manager mit wenig oder keiner Erfahrung auf dem Gebiet des Software Engineering.

- **Fehlen aktueller Dokumente:** Die Vernachlässigung einer projektbegleitenden Dokumentation kommt sehr häufig vor. Sie behindert die Kommunikation der am Projekt beteiligten Personen und verhindert die Projektfortschrittskontrolle.
- **Mangel an Projektfortschritts- und Qualitätskontrolle:** Komplexe Softwaresysteme bestehen aus Subsystemen und diese wiederum aus einzelnen Komponenten, die in der Regel von verschiedenen Personen implementiert werden. Zu spät erkannte Qualitätsmängel einzelner Komponenten erfordern meist aufwändige Nachbesserungen, die Kostensteigerungen und Terminüberschreitungen nach sich ziehen. Die Vernachlässigung der Projektfortschritts- und Qualitätskontrolle kann dazu führen, dass wegen der nicht zeitgerechten Fertigstellung einzelner Systemkomponenten die weiteren Entwicklungsschritte unterbrochen und damit die gesamte Planung über den Haufen geworfen wird.
- **Fehlen einer Kontrolle der Kostenentwicklung:** Fehlt in einem Projekt die laufende Kontrolle des Aufwandes, so ist es unmöglich, Fehler in der Kostenplanung und Schwachstellen der Projektentwicklung frühzeitig erkennen und geeignete Maßnahmen dagegen ergreifen zu können.

6 Ablauforganisation

Dieses Kapitel befasst sich ausführlich mit der Ablauforganisation der ingenieurmäßigen Softwareentwicklung und den damit verbundenen Prozessen, erstellten Vorgehensmodellen und -methoden sowie deren Bedeutung für den Projektlebenszyklus und den Softwarelebenszyklus. Ein Überblick über aktuelle und bewährte Praktiken in Softwareentwicklungsprozessen schließt dieses Kapitel ab.

6.1 Prozessbegriff

6.1.1 Der Prozess in der Ablauforganisation

Unter *Ablauforganisation* versteht man die Aneinanderreihung systeminterner Elemente (so genannter *Arbeitsabläufe*), die räumlich und zeitlich so angeordnet sind, dass das System funktionsfähig ist und zielgerichtet arbeitet. Arbeitsabläufe dienen dazu,

- den Ablauf von Arbeitsvorgängen zu organisieren,
- Verantwortlichkeiten innerhalb der Vorgänge zu formalisieren,
- Möglichkeiten zu bieten zum Rationalisieren, Vereinfachen und Standardisieren.

Da sich speziell in der Softwareentwicklung Arbeitsabläufe häufig ändern, spricht man oftmals von „Prozessen“. Gemäß [Webster's, 2005] wird *Prozess* definiert als eine „kontinuierliche Entwicklung mit vielen Änderungen“. Graphentheoretisch stellt ein Prozess (hier oft *Prozesskette* genannt) einen gerichteten Graph von Aktionen dar.

Ein (*geregelter*) *Prozess* im technischen Sinn (vgl. Abschnitt 4) ist in Abbildung 6.1 dargestellt: Eingabegrößen werden unter Berücksichtigung von Störgrößen verarbeitet und ergeben Ist-Ausgabegrößen (Ergebnisse). Diese werden mit den Soll-Ausgabegrößen verglichen und die Differenz zur Eingabe rückgeführt, um in der Verarbeitung auf die Differenz reagieren zu können.

Im Bereich der Softwareentwicklung wird der Begriff „Prozess“ von wesentlichen normativen Einrichtungen wie folgt präzisiert:

- ISO (International Standardization Organization, ISO/IEC 90003:2004): „[Ein Prozess ist] eine Menge von in Wechselwirkung oder Wechselbeziehung stehenden Tätigkeiten, die Eingaben in Ausgaben transformieren.“ [ISO, 2004]
- SEI (Software Engineering Institute der Carnegie Mellon Universität, USA): „Ein Prozess ist eine Menge von Aktivitäten, Methoden, Praktiken und Vorschriften, die Menschen verwenden, um Software und die damit verbundenen Produkte zu entwickeln und zu erhalten.“ [Kulpa und Johnson, 2003]
- IEEE (International Institute of Electrical and Electronic Engineers): „Ein Prozess ist eine Sequenz von Schritten, die durchgeführt werden, um einen vorgegebenen Zweck zu erfüllen.“ [IEEE, 1996]

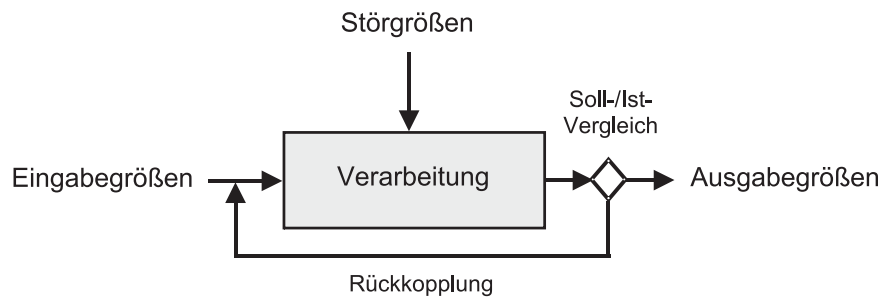


Abbildung 6.1: Geregelter Prozess aus technischer Sicht

Eine Kernaufgabe der Ablauforganisation ist es, nicht nur Prozesse in Gang zu setzen, sondern sie auch kontinuierlich zu analysieren und zu verbessern [Coleman *et al.*, 1994]. Ergebnisse von (Teil-)Prozessen können daher nicht nur produktbezogen („statischer“ Natur), sondern auch prozessbezogen („dynamischer“ Natur) sein.

6.1.2 Definierter versus empirischer Prozess

In einem *definierten Prozess* sind sämtliche Aktivitäten vorab vollständig bekannt und verstanden [Ogunnaike und Harmon, 1994]. Für jede zulässige Eingabe wird jedes Mal dasselbe Ergebnis (innerhalb eines akzeptablen Rahmens) erzeugt. Charakteristische Eigenschaften eines definierten Prozesses sind daher Wiederholbarkeit und Vorhersehbarkeit. Ein derartiger Prozess besteht aus vorher definierbaren, abhängigen Einzelschritten, deren Beginn und Ende vorab realistisch planbar ist.

Aufgaben von Projekten, die einem definierten Prozess folgen, lassen sich bis in ihre Einzelschritte *schrittweise verfeinern* [Wirth, 1971] und linear gemäß dem erstellten Plan abarbeiten. Folgende Aussagen treffen für derartige Projekte zu [Alleman, 2002a]:

- Projektzweck, Beginn und Ende können vor dem Projekt oder zumindest frühzeitig im Laufe eines Projekts identifiziert werden.
- Entscheidungsprozesse sind vorhersehbar und (meist) reversibel.
- Sobald die Anforderungen definiert sind, können genaue Vorhersagen über Projektdauer und Ressourcenbedarf gemacht werden.

- Wenn Projekte scheitern, so geschieht dies auf Grund von fehlendem technischem Verständnis oder Managementvermögen, nicht durch unklare Machbarkeit, Brauchbarkeit oder Akzeptanz der Lösung.

Viele technische Prozesse beinhalten aber ein so großes Maß an Komplexität und Unvorhersehbarkeit, dass sie nicht vollständig verstanden und damit definiert werden können. Derartige Prozesse, die in [Schwaber und Beedle, 2001] als *empirische Prozesse* bezeichnet werden, wurden wissenschaftlich umfassend im Bereich der chemischen Industrie untersucht (DuPont Experimental Station, Wilmington, Delaware/USA [Ogunnaike und Harmon, 1994]). Die Hauptidee dieser Untersuchungen ist, dass Prozesse erfolgreich gesteuert und geregelt werden können, selbst wenn sie nicht (vollständig) verstanden werden! [Schwaber und Beedle, 2001]

Modelle für empirische Prozesse „erwarten das Unerwartete“. Prozesse werden durch häufige Inspektionen und unmittelbare Anpassungen geregelt, selbst wenn sie unvollständig definiert sind und unvorhersehbare und unwiederholbare Ergebnisse erzielen [Ogunnaike und Harmon, 1994]. Ein Beispiel hierfür ist die Fruchtsaftherstellung, bei der die Qualität des Rohmaterials (Zuckergehalt der Früchte, Reifegrad etc.) ständig anders, aber für die Qualität des Ergebnisses entscheidend ist. Jeder Produktionsprozess läuft unter anderen Eingabewerten und oft auch anderen Störgrößen (z. B. Umgebungstemperatur, Verunreinigung des Rohmaterials) ab. Wichtig bei der Regelung derartiger Prozesse ist, die natürliche Streuung der Eingabegrößen von unvorhergesehenen Einflüssen (Störfaktoren) zu unterscheiden und die Anpassungen korrekt vorzunehmen.

Der *Prozess der Softwareentwicklung* ist viel häufiger empirischer Natur denn ein definierter Prozess [Schwaber und Beedle, 2001]. Konsequenterweise sind Techniken zur Regelung empirischer Prozesse (inhaltsbetonte Meilensteine, häufige Iterationen mit Inspektionen und Anpassung der Verarbeitung von Eingabe- und Störgrößen sowie ggf. des Prozesses selbst) viel geeigneter für die Softwareentwicklung als die klassischen Techniken, die für definierte Prozesse Anwendung finden (zeitbetonte Meilensteine, Masterpläne, lineare Umsetzung). Wie in [Ogunnaike und Harmon, 1994] formuliert, ist in der Softwareentwicklung „zu viel Denken und Kreativität“ involviert, als dass der Prozess vollständig definiert sein könnte.

Folgende Paradigmenwechsel finden bei der Transition von definierten Prozessen zu empirischen Prozessen statt:

- **Planen** stellt einen kontinuierlichen Prozess dar, dessen mittel- und langfristige Werte und Änderungen durch die Reaktion des Kunden auf die gelieferten Zwischenprodukte gesteuert werden.
- **Schrittweise Verfeinerung** ist nicht mehr für das gesamte Projekt bis zur Tiefe eines detaillierten Verständnisses möglich (und auch nicht mehr nötig). Die Realisierung des Produkts wird begonnen, sobald genügend Anforderungen bekannt sind, ungeachtet deren Proportionalität zum „Gesamtprodukt“.¹
- **Stabilität** und damit Treue zum ursprünglichen Plan geht verloren. Plantreue um jeden Preis bedeutet vielfach vergebene Geschäftschancen oder ungenutzte mögliche Flexibilität der Rahmenbedingungen (z. B. technologischer Fortschritt).

¹In [Cockburn, 2004] wird diese Fähigkeit als „die Kunst des Möglichen“ („the art of the possible“) bezeichnet.

- **Anforderungsänderungen** sollen und können gar nicht vermieden werden; sie stellen auch keine Projektschwäche dar. Die Fähigkeit, auf Änderungen der Kundenanforderungen (und vielfach auch interner Beschränkungen) rasch reagieren zu können, stellt ein wichtiges Qualitätsmerkmal des Entwicklungsprozesses dar.

Jede Lieferung eines Zwischenprodukts erzeugt neue und/oder abgeänderte Anforderungen. Diese nicht-lineare Rückkopplung führt zu neu entstehendem, emergentem Verhalten (*emergent behavior*, vgl. [Alleman, 2002a]) sowie der Generierung zusätzlichen Werts des Projektergebnisses (*value-added software*, vgl. [Boehm und Huang, 2003]). „Softwareprojekte sind nicht wiederholbar, stabil oder linear. Sie sind einzigartig, gesteuert durch instabile Anforderungen, Technologien und Marktkräfte, und enthalten viele nicht-lineare Aktivitäten.“ [Alleman, 2002a]

6.1.3 Entwicklung der Prozessbedeutung

Im Laufe der – noch relativ kurzen – Geschichte der prozessorientierten Softwareentwicklung kann folgende Verschiebung der Bedeutung von Prozessen in der Entwicklung beobachtet werden:

1. **Normative Bedeutung:** Normatives Prozessdenken [Kuhn, 1996] nimmt an, dass die Mitglieder einer Organisation das gemeinsame Ziel genau kennen und der Prozess das gemeinsame Vorgehen so lenken muss, dass dieses Ziel möglichst optimal erreicht wird (z. B. das gemeinsame Zusammenbauen eines Puzzles durch mehrere Personen). Der Prozess stellt ein regulierendes Werkzeug zur effizienten und effektiven Erreichung des (vorher bekannten) Ziels dar und umfasst „rationale Methoden“, deren Anwendung – unabhängig von der Anwendungsdomäne – den Projekterfolg positiv beeinflusst. Dies setzt voraus, dass die zu Grunde liegenden Prozesse stabil sind und nicht durch die Art und Weise der Verwendung durch die Mitarbeiter (oder die Anwendung der Prozesse selbst) geändert werden.² Normative Prozesse definieren daher, WAS zu tun ist, und geben in nur sehr geringem Maß an, WAS NICHT zu tun ist.
2. **Transformierende Bedeutung:** Bald wurde in der Softwareentwicklung erkannt, dass normatives Prozesswissen (vgl. z. B. die verschiedenen „bodies of knowledge“ im Projektmanagement, etwa das PMBOK [PMI, 2005]) zwar sehr hilfreich ist, ohne Transformation in die Domäne der Anwendung aber kaum brauchbar [Alleman, 2002a]. Es ist entscheidend zu wissen, WIE in einer bestimmten Domäne Kundenwünsche als Anforderungen definiert werden können, WIE aus Anforderungen Testfälle generiert werden können, WIE Testpläne mit dem Kunden umgesetzt werden können etc. Dieses transformationale Prozesswissen stammt aus dem Bereich der Wirtschaftswissenschaften, in dem die Bedeutung von Geschäftsprozessen (und deren domänenspezifische Optimierung) bereits sehr früh erkannt wurde, vgl. [Porter, 1998].
3. **Wertgenerierende Bedeutung:** Der Fokus des wertgenerierenden Prozesswissens liegt auf der Auslieferung des wertvollsten Ergebnisses für den Kunden (vgl. die „lean

²Ein Beispiel für eine solche Einflussnahme wäre ein Imker, der ein Schmerzmittel gegen Bienenstiche entwickeln möchte, aber durch die laufenden Selbstversuche schmerzunempfindlich wird.

construction“ in [Koskela, 2000]). Der Wert eines Produkts wird in der Softwareentwicklung vorrangig durch den Kunden definiert und kaum durch die Entwickler. Dementsprechend ist eine enge Kundeneinbindung in den laufenden Prozess der „Wertdefinition“ entscheidend für den Erfolg moderner Softwareentwicklung. Die Entscheidungen über Priorisierung der Anforderungen sowie deren Umsetzung und das Testen werden mittelfristig immer stärker von dem Kunden gemäß dem jeweilig für ihn damit verbundenen Wert erfolgen [Boehm und Huang, 2003].

Wie [Alleman, 2002a] anmerkt, hat der Prozessbegriff in der Softwareentwicklung in den letzten zwei Jahrzehnten zwar den angesprochenen Wandel vollzogen, die Bedeutungen dürfen aber nicht exklusiv gesehen werden: Entscheidend ist, das transformationale Prozesswissen gemäß dem Kundenwert zu priorisieren. Darüber hinaus macht es jedoch sehr wohl Sinn, normatives Prozesswissen in Form bewährter Praktiken zu extrahieren und darauf in nachfolgenden Projekten aufzubauen (vgl. auch [Ambler, 1999a], [Ambler, 1999b]).

6.2 Softwarelebenszyklus und Projektlebenszyklus

Wie andere Projekte unterteilt man auch Softwareprojekte in einzelne Tätigkeitsbereiche, die Phasen, vgl. Abschnitt 4.6. Diese Phasen zusammengefasst und die Ordnung ihrer zeitlichen Abfolge bezeichnet man als *Softwarelebenszyklus* (*software life cycle*). Diese Bezeichnung impliziert die Vorstellung von einer Zeitspanne, in der ein Softwareprodukt entwickelt und eingesetzt wird, bis zum Ende seiner Benutzung.

Wichtig ist dabei anzumerken, dass mit dem Begriff „Phase“ in der Softwareentwicklung oft ein sequenzielles Vorgehen assoziiert wird, dies aber in der beabsichtigten Bedeutung (und auch der ursprünglichen, vgl. [Pfleeger, 2001]) jeglicher Grundlage entbehrt. Auch in stark iterierenden und/oder iterativen Vorgehensmodellen wird davon gesprochen, dass sich ein (Teil-)Produkt in der Testphase befindet oder im Rahmen einer Iteration eine Analysephase durchgeführt wird. In diesem Sinn kann auch die Gliederung der Aufgabenbereiche der ingenieurmäßigen Projektentwicklung (vgl. Abschnitt 4.6) als *Projektlebenszyklus* (*project life cycle*) verstanden werden, ohne damit Vorgaben über die Art des Vorgehens im Rahmen der Projektentwicklung zu machen.

6.3 Prozessmodelle in der Softwareentwicklung

6.3.1 Vorgehensmodelle und Prozessmodelle

Die Dokumentation der Ablauforganisation wird in der Softwareentwicklung oft als *Vorgehensmodell* bezeichnet. Solche Vorgehensmodelle (z. B. das V-Modell, vgl. [Versteegen, 1994]) waren daher lange Zeit vorrangig schriftlich festgehaltene Rahmenmodelle in der Form umfangreicher Dokumentiersysteme. [Elzer, 1994] nennt diese Modelle daher „Checklisten für eine geordnete Abwicklung der Arbeit“, wobei die Vielzahl der zu erstellenden Dokumente eher eine Kostensteigerung als eine Kostensenkung bewirkte. Gerade in den neunziger Jahren des letzten Jahrhunderts wurden manche Vorgehensmodelle immer umfangreicher und damit schwerer und träger handhabbar, ohne jedoch Projekte erfolgreicher damit abwickeln

zu können [Versteegen, 1994]. Diese Situation war – besonders in Bezug auf die Papierflut – sowohl für Auftragnehmer als auch Auftraggeber unzumutbar.

Diese dokumentationszentrierten Vorgehensmodelle waren der Auslöser für die Gegenbewegung der „*agilen Softwareentwicklung*“, manifestiert in [Fowler und Highsmith, 2001]. Mit der zeitweise sehr heftig geführten Diskussion „agil“ (modern, leichtgewichtig, fragil, ...) versus „rigoros“ (traditionell, schwergewichtig, stabil, ...) wurde geradezu ein religiöser Krieg begonnen (vgl. [Orr, 2002]), der mittlerweile auch auf das Projektmanagement übertragen wird. So wird ein „Extreme Project Management“ angeboten [Thomsett, 2001], das sich bei näherer Betrachtung als recht konventionell herausstellt. Andererseits zeigt *Scrum* (vgl. [Highsmith, 2002] oder [Schwaber und Beedle, 2001]), dass die Übertragung von agilen Ideen auf das Projektmanagement durchaus von Erfolg gekrönt sein kann, wenn die Konzepte nicht zu radikal umgesetzt werden.

Der Hauptzweck der Vertreter des agilen Vorgehens, nämlich das Einbremsen der Prozessnormierung und Reduzieren der „Prozessmonster“ wie des Rational Unified Process (RUP; [Kruchten, 2000]) oder des V-Modells [Versteegen, 1994] auf ein vernünftiges Maß, wurde durch die oft provokanten Aussagen jedoch erreicht [Stephens und Rosenberg, 2003]. Das Pendel, das von sehr rigorosen Methoden (z. B. CMM³) zu sehr agilen (z. B. XP⁴) geschwungen wurde (Abbildung 6.2), wird sich in absehbarer Zeit wieder – irgendwo in der Mitte – stabilisieren, wie dies auch bei früheren Methodendisputen der Fall war („Datenkrieg“ der achtziger Jahre, „OO-Krieg“ der neunziger Jahre, vgl. [Orr, 2002]).

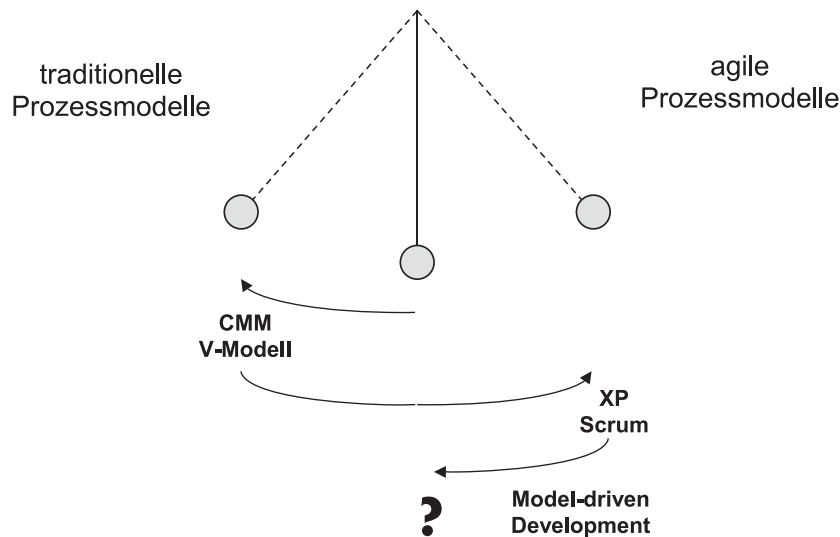


Abbildung 6.2: Die Suche nach der goldenen Mitte

Ein weiterer Trend ist, dass Manager zunehmend versuchen, induktiv aus ihren Prozesserfahrungen zu lernen und diese bewährten Praktiken weiterzugeben. Der *Pattern*-Begriff wurde ins Projektmanagement übertragen [Ambler, 1999a] [Ambler, 1999b] und wird mittlerweile durch Sammlungen von Negativbeispielen ergänzt (*Anti-Patterns*, vgl. [Brown *et al.*, 2000]). Zunehmend wird daher unter einem Vorgehensmodell vorrangig die *systematische Gliederung*

³Capability Maturity Model; siehe Abschnitt 8.3.2

⁴eXtreme Programming; vgl. z. B. [Beck und Andres, 2005]

einer Lösung verstanden. Gemäß [Kuhrmann, 2005] beschreibt „... ein Vorgehensmodell ... systematische, ingenieurmäßige und quantifizierbare Vorgehensweisen, um Aufgaben einer bestimmten Klasse wiederholbar zu lösen. Ein Vorgehensmodell beruht auf einer Reihe von Prinzipien“. Ein Vorgehensmodell enthält je ein *Entwicklungsmodell* für den Auftraggeber und den Auftragnehmer sowie ein *Qualitätsmodell* für die gesamte Organisation.

Ein solches modellbasiertes Vorgehen wird durch ein *Prozessmodell* geregelt (vgl. Abbildung 6.3, modifiziert aus [Broy und Rausch, 2005a]) und unterstützt den Ablauf des Lösungsprozesses. Ein Prozessmodell definiert die grundlegende Struktur des Projektverlaufs sowie die Reihenfolge der (schrittweisen) Produkterstellung.

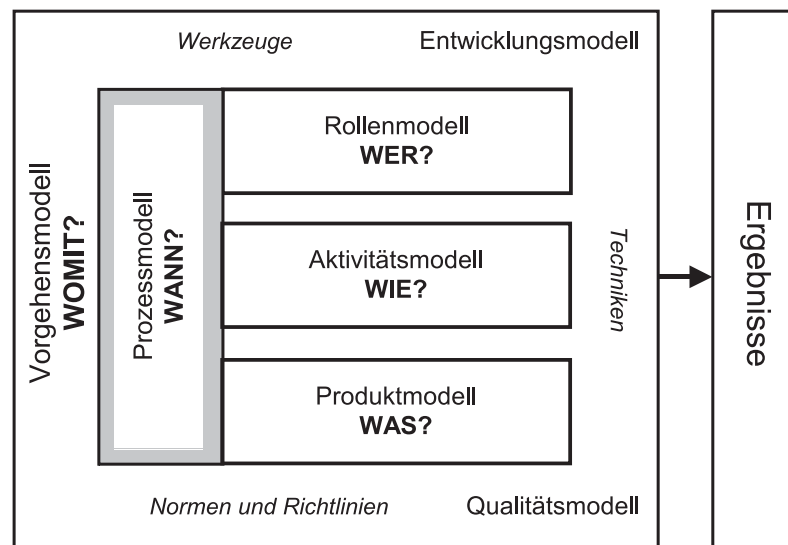


Abbildung 6.3: Komponenten eines modellbasierten Vorgehens

Ein Prozessmodell unterteilt das Vorgehen in überschaubare Abschnitte und soll dadurch eine schrittweise Planung, Durchführung und Überprüfung ermöglichen. Die ingenieurmäßige Erfahrung, dass allgemeine Designentscheidungen dadurch in Frage gestellt werden können, dass bestimmte Detaillösungen nicht wie geplant realisierbar sind, führte zur Erkenntnis, dass ein Prozessmodell auch Rückkopplungsschleifen enthalten muss, durch die Einsichten, die auf „tieferen Ebenen“ gewonnen werden, „höhere Ebenen“ beeinflussen können („*prozess-orientiertes Vorgehen*“). Prozessmodelle können zusätzlich projekt- und situationsspezifisch angepasst werden, wobei diese Anpassungen zu dokumentieren sind.

6.3.2 Prozessmodell-Topologie

Beim Einsatz von Prozessmodellen (bzw. von Methoden, die solche Modelle implementieren) in der Softwareentwicklung ist zu beachten, dass die verschiedenen verfügbaren Prozessmodelle verschiedene Gültigkeitsbereiche im Gesamtumfeld des Projekt Engineering umfassen können. Dem Gültigkeitsbereich entsprechend kann eine Klassifikation in folgende drei Bereiche vorgenommen werden (Abbildung 6.4; [Mayr, 2004a]):

1. **Unternehmensprozess-Modelle** fokussieren auf die Kultur der Softwareentwicklung im jeweiligen Unternehmen. Hierzu gehören beispielsweise Teambildungsstrategien,

Modell	Gültigkeitsbereich
Unternehmensprozess-Modell U	Unternehmenskultur (QS, Teambildung, Arbeitszeit, ...)
Projektprozess-Modell P	gesamter Projektentwicklungszyklus
Entwicklungsprozess-Modell	Zyklus Design–Implementieren–Testen

Abbildung 6.4: Klassifizierung der Prozessmodelle nach Gültigkeitsbereichen

Arbeitszeitregelungen, Zulieferer- und Releasepolitik sowie die Festlegung unternehmensweiter Richtlinien, z. B. für projektübergreifende Qualitätssicherung, Kundenbindungsmaßnahmen (customer relationship management, CRM) u. dgl.

2. **Projektprozess-Modelle** fokussieren auf einen gesamten Projektzyklus, von der Initiierung, Anforderungsanalyse, Shareholder-/Stakeholderaspekten über die Entwicklung zu Lieferung, Betrieb und Wartung des Produkts.
3. **Entwicklungsprozess-Modelle** konzentrieren sich auf den Design–Implementieren–Testen-Zyklus.

Jedes Modell einer höheren (umfangreicheren) Ebene benötigt eine Instanz eines geeigneten Modells auf der nächstniedrigeren Ebene, um für ein Projekt konkret angewendet werden zu können. Beispiele für die sich daraus ergebende *Prozessmodell-Topologie* sind in Abbildung 6.5 angegeben. So ist etwa „1. ISO 9000f., 2. RUP, 3. Wasserfall“ ein Beispiel für eine kombinierbare traditionelle Prozesshierarchie, während „1. APM/SW-Ecosystems, 2. Scrum, 3. XP“ ein gängiges Beispiel für eine agile Hierarchie darstellt.

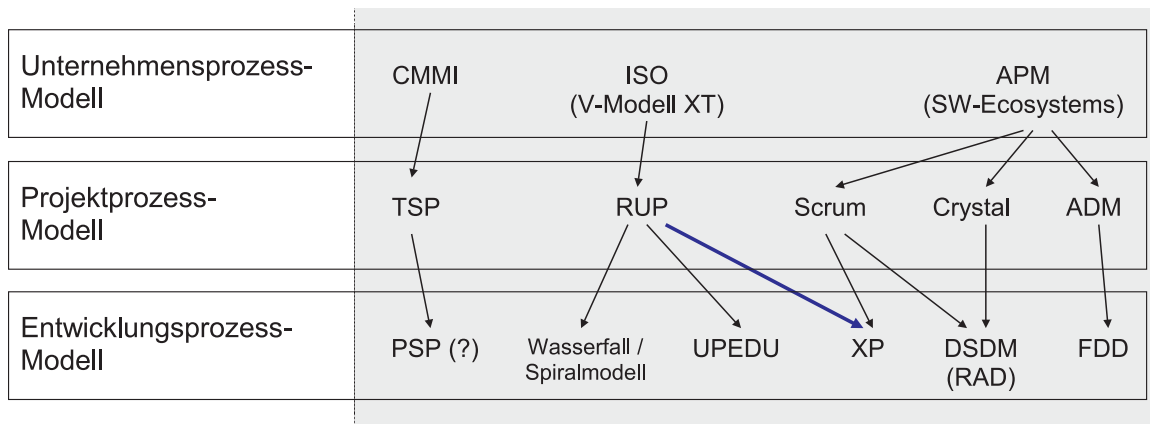


Abbildung 6.5: Topologie der Prozessmodelle

Dieses Klassifizierungsschema macht klar, warum Vergleiche verschiedener Prozessmodelle in der Literatur (wie z. B. CMM versus XP [Orr, 2002] oder RUP versus XP [Engels *et al.*, 2003]) wenig brauchbare Ergebnisse – und oft sogar falsche Schlussfolgerungen!⁵ – bringen:

⁵So wird z. B. in [Beck und Andres, 2005] abgestritten, dass es sich bei XP um eine Abwandlung des Spiralmodells handelt; oder es wird in [Jeffries *et al.*, 2000] und [Engels *et al.*, 2003] RUP als „nicht-agile“

Sie vergleichen Prozessmodelle mit verschiedenen Gültigkeitsbereichen! Die Existenz eines XP-Plugins für die Rational RUP-Suite (<http://www.rational.com/products/rup/xprog.jsp>), die häufig Studierende und sogar Professionisten und Hochschullehrer irritiert, ist ein gutes Beispiel für die Anwendbarkeit des oben genannten Klassifizierungsschemas.

In [Coldewey, 2002] klassifiziert Jens Coldewey agile Prozessmodelle in einem Überblick über die Welt der agilen Softwareentwicklung sehr ähnlich wie in Abbildung 6.5: Projektprozess-Modelle werden darin „Meta-Prozesse“ genannt und die Entwicklungsprozess-Modelle „spezifische Prozesse“. Die dritte Ebene der Unternehmensprozessmodelle bleibt in [Coldewey, 2002] unbehandelt. Jedoch werden detaillierte Eigenschaften zur Unterscheidung der ersten zwei Topologieebenen gegeben sowie die Wichtigkeit der Ebenentrennung betont. Aus Entwicklersicht wird nämlich oft der gesamte Bereich der Prozessmodelle als unnötiger Ballast gesehen und damit werden sämtliche Modelle in einen Topf geworfen, vgl. z. B. [Alleman, 2002a]. Dieses kurzsichtige Denken führt spätestens beim nächsten Projekt dazu, dass erfolgreiche Prozessschritte nicht zuverlässig reproduziert werden können, dafür aber bereits gemachte Fehler ein zweites Mal begangen werden.

6.3.3 Prozessskalierung

Im Rahmen der Diskussion über traditionelle versus agile Prozesse wurde auch erkannt, dass Prozessmodelle nie exakt für ein neues Projekt passen können [Cockburn, 2001] und neben der inhaltlichen Adaptierung vor allem die *Skalierung eines Prozessmodells* auf die Größe des Projekts einen entscheidenden Erfolgsfaktor darstellt. Beispielsweise zielt die Umstellung des auf Grund seiner Dokumentationslastigkeit sehr in Verruf geratenen *V-Modells* auf das *V-Modell XT* sehr stark auf die Anpassbarkeit des Modells und seiner Werkzeuge auf verschiedene Projektgrößen ab [Kuhrmann, 2005].

Der grundlegende Gedanke der *Prozessskalierung* (*process tailoring*, auch *Prozessanpassung*) ist, auf der Basis von – vorab erhebbaren – Projektmerkmalen ein Produktprofil zu erstellen. Mittels diesem werden dann auf der schablonen- oder bausteinartig aufgebauten Prozessdefinition die verpflichtend zu verwendenden Elemente festgelegt. Neben projektrelevanten Aspekten (Liefertermin, Gruppengröße, Projektvolumen) ist dabei auch die Anwendungsdomäne relevant. So wird z. B. der Crystal-Projektprozess nicht nur nach der Größe des Projekts (und damit der Anzahl der Projektmitglieder) gewählt und verfeinert, sondern auch nach der Kritikalität der Anwendungsdomäne (Sicherheitsaspekte, potenzielle Benutzergefährdung etc.) [Cockburn, 2004].

Oftmals wird nicht nur der Prozess angepasst, es werden auch die benötigten Werkzeuge ausgewählt bzw. auf das rechte Maß zurecht geschneidert [Broy und Rausch, 2005a]. Eine weitere Möglichkeit der Vereinfachung eines Prozessmodells ist es, das Modell für die verschiedenen Benutzerrollen durch unterschiedliche Sichten (z. B. Auftraggeber, Auftragnehmer, Projektleiter, Entwickler) einzuschränken. In der Rational RUP-Suite geschieht dies durch für das Projekt dynamisch generierte HTML-Leitfäden, die durch den Projektprozess führen [Kruchten, 2000].

In einem Prozessmodell können Art und Umfang von Rollen, Produkten und Aktivitäten angepasst werden. Zusätzlich können Werkzeuge, Techniken und Normen geeignet ausgewählt

Entwicklungsmethode bezeichnet.

werden, vgl. Abbildung 6.3. Das V-Modell XT beispielsweise verpflichtet als Minimum nur zur Wahl der folgenden Ergebnispakete [Broy und Rausch, 2005b]:

- Projekthandbuch,
- Projektplan,
- Qualitätshandbuch,
- Produktbibliothek (das Produkt selbst mit seinen Varianten und Zwischenergebnissen).

Wichtig ist, dass das Prozessmodell so in Bausteine untergliedert ist, dass diese als Einheit eigenständig verwendet werden können und unabhängig voneinander veränder- und weiterentwickelbar sind [Kuhrmann, 2005]. Dies ist nicht nur zur Anpassung des Prozessmodells für nachfolgende Projekte wichtig, sondern ggf. auch während der Laufzeit eines Projekts. Prozessanpassungen, die kollektiv vom Auftragnehmer als projektfördernd angesehen werden (und im Fall der Sichtbarkeit für den Auftraggeber durch diesen genehmigt werden), werden im Projekthandbuch dokumentiert und ab diesem Zeitpunkt einheitlich umgesetzt.

6.4 Vorgehensmethoden

Jedes Prozessmodell umfasst zur Umsetzung des *Vorgehensprozesses* (*engineering process*) eine oder mehrere – oft kombiniert angewendete – *Vorgehensmethoden* (*engineering practices*). Bei einer Vorgehensmethode steht immer die *Methodik des Vorgehens* im Mittelpunkt im Gegensatz zur *Methodik der Beschreibung*. Beides wird aber in der Literatur oft unsauber als „Vorgehensmodell“ bezeichnet, was zu Verwirrungen und falschen Schwerpunktsetzungen führen kann [Orr, 2002].

Nachfolgend konzentrieren wir uns auf die Methodik des Vorgehens und erläutern die wesentlichen Vorgehensmethoden näher. Es sei bereits an dieser Stelle darauf hingewiesen, dass diese Methoden im Allgemeinen einander nicht ausschließen, sondern oft erst in ihrer Kombination zum Erfolg führen.

6.4.1 Sequenzielles Vorgehen

Die Anwendung einer streng sequenziellen Vorgehensweise in der Softwareentwicklung sollte dazu führen, dass Softwareprojekte besser planbar, organisierbar und kontrollierbar sind. Die Realität zeigte jedoch, dass eine rein sequenzielle Vorgehensweise nur in den seltensten Fällen durchführbar ist. Das nachfolgend erläuterte Phasenmodell ist daher aus historischen Gründen erwähnenswert, besitzt jedoch keine aktuelle Praxisrelevanz.

6.4.1.1 Phasenmodell

Das *Phasenmodell* wurde ursprünglich aus rein organisatorischen Gründen für die Entwicklung aller Arten komplexer technischer Systeme entwickelt – hauptsächlich um Projekte der öffentlichen Hand nach einheitlichen Richtlinien abwickeln zu können. Später stellte sich

heraus, dass es auch auf Softwareprojekte anwendbar war und vor allem einen nützlichen Rahmen für Konstruktion und Klassifikation von „Entwicklungswerkzeugen“ abgab. Daraufhin wurde es einige Zeit lang als technisches Dogma betrachtet und vor allem gelehrt.

Die dem Phasenmodell zu Grunde liegende Vorgehensweise bei der Softwareentwicklung beruht auf dem Prinzip, dass für jede der Phasen klar zu definieren ist, welche Ergebnisse erzielt werden müssen, und dass eine Phase erst dann in Angriff genommen werden darf, wenn die vorhergehende vollständig abgeschlossen ist („sequenzielles Vorgehen“).

Als Ausgangsbasis für die einzelnen Phasen werden wohldefinierte „Produkte“ (meist in Form von Dokumenten) verlangt. Diese Produkte werden in phasenbezogenen Prozessen – für die bestimmte Methoden und Werkzeuge angewandt werden – verarbeitet und die Resultate an die nächste Phase weitergeleitet. Die Phasen sind klar gegeneinander abgegrenzt und dürfen erst dann verlassen werden, wenn in einem Verifikations-/Validierungsschritt die Ergebnisse akzeptiert worden sind.

In der Praxis störte von Anfang an, dass bei unkritischer Anwendung des Phasenmodells ein psychologischer Druck dahingehend entsteht zu glauben, dass es wünschenswert oder möglich sei, eine Entwicklungsphase vollständig abzuschließen, bevor man die nächste beginnen könne. Besonders schädlich wirkte sich diese Fehlinterpretation dann aus, wenn sie dazu noch an eine missverstandene Form des „Topdown-Designs“ gekoppelt war. Dieses ist aber eine Auffassung, die allen Regeln und Erfahrungen des Ingenieurwesens widerspricht. Wer jemals ein nichttriviales technisches System entwickelt hat, weiß, dass oft technische Probleme im Detail wesentliche vorher getroffene Designentscheidungen hinfällig machen können.

Die streng sequenzielle Vorgehensweise hat auch zur Folge, dass erst sehr spät ein Produkt oder Produktteile vorliegen, die „greifbare“ Ergebnisse darstellen. Die Erfahrung zeigt, dass man beim Validierungsprozess in der Regel nicht ohne realitätsnahe Experimente auskommt. Das verhindert aber die phasenorientierte Entwicklungsmethode. Dies hat auch zur Folge, dass Änderungswünsche von den Benutzern erst sehr spät geäußert und dann unter Umständen nur mit hohem Aufwand berücksichtigt werden können.

6.4.2 Inkrementelles Vorgehen

Der fehlende Kundenbezug und die äußerst späte Verfügbarkeit eines Produkts beim rein sequenziellen Vorgehen führte dazu, dass Alternativen entwickelt wurden, in denen diese streng sequenzielle Vorgehensweise aufgeweicht wird. Außerdem wurden neue Methoden und Techniken zur Verbesserung der Softwareentwicklung selbst entwickelt (z. B. Prototyping zur besseren Darstellung des geplanten Produkts dem Kunden gegenüber oder die objektorientierte Programmierung zur Verbesserung des Design- und Implementierungsprozesses), die ihrerseits auf die Vorgehensmethoden im gesamten Softwareentwicklungsprozess Auswirkungen hatten und eine Abweichung vom klassischen Phasenmodell erforderten.

Den nachfolgend dargestellten *inkrementellen Vorgehensmethoden* liegt die Gemeinsamkeit zu Grunde, dass das Produkt *schrittweise* (= inkrementell) entwickelt wird und die Validierung von Zwischenergebnissen eine Überprüfung auf Brauchbarkeit für den Kunden ermöglicht.

6.4.2.1 Wasserfallmodell

Das *Wasserfallmodell* stellt eine Verfeinerung des klassischen sequenziellen Phasenmodells dar und wurde in den siebziger Jahren des letzten Jahrhunderts entwickelt (vgl. Abbildung 6.6; [Royce, 1970]). Es enthält im Wesentlichen zwei Erweiterungen:

1. die Einführung von Rückkopplungen zwischen den Phasen zusammen mit der Einschränkung, Rückkopplungen möglichst nur bei aufeinanderfolgenden Phasen vorzusehen, um die „teure“ Nacharbeit, die infolge von Iterationen über mehrere Phasen hinweg entsteht, zu vermindern,
2. die Einbindung der (möglichst experimentellen) Validierung der Phasenergebnisse in den Softwarelebenszyklus.

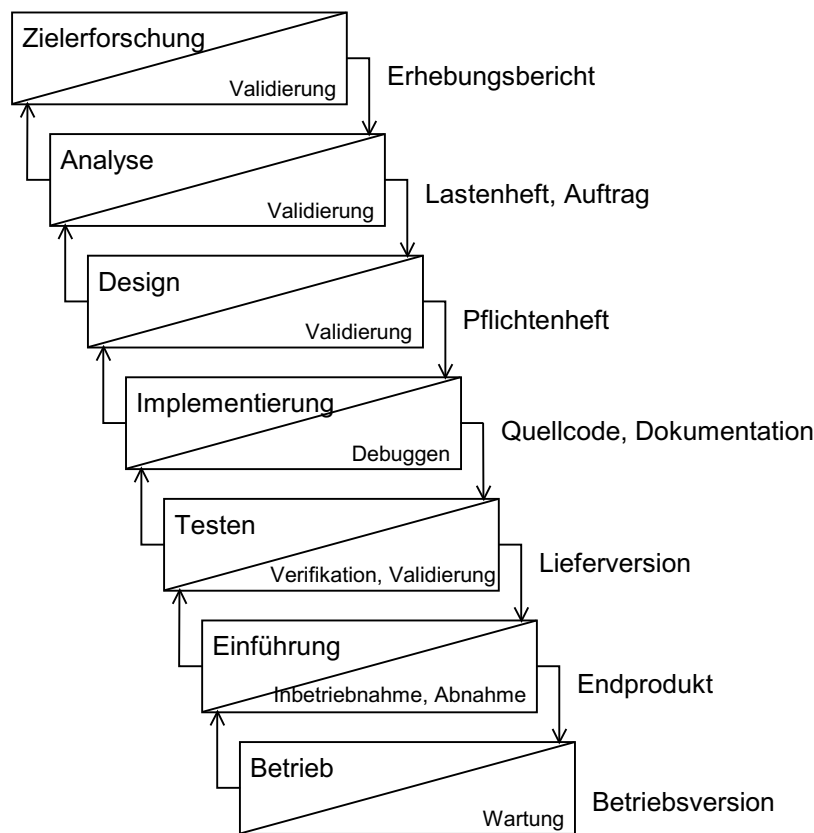


Abbildung 6.6: Das Wasserfallmodell

Mit diesem Ansatz wird die streng sequenzielle Vorgehensweise, die das Phasenmodell vorschreibt, aufgeweicht und eine inkrementelle Entwicklungsstrategie in den Mittelpunkt gerückt.

Die Einbindung einer schrittweisen Entwicklungsstrategie für das (System-)Design sowie einer phasenweisen Validierung soll helfen, die Auswirkungen von Fehlentscheidungen besser in den Griff zu bekommen und den Softwareentwicklungsprozess besser kontrollierbar zu gestalten.

Die Hauptnachteile des Wasserfallmodells sind, dass die Erweiterung kaum Flexibilität ins Modell bringt und dass Änderungswünsche „teuer“ sind.

6.4.2.2 Objektorientiertes Modell

Das *Tool-Smithing* (*Werkzeugschmieden*) ist eine bottomup-orientierte Softwareentwicklungsmethode. Die Grundidee ist die Produkterstellung vorrangig durch das Zusammenfügen bereits existierender Bausteine. Das bedeutet natürlich auch, Bausteine so zu fertigen, dass sie möglichst gut wieder verwendet werden können. Dies ist erst durch die objektorientierten Konzepte (wie Generizität, Vererbung, Polymorphismus, dynamische Bindung) akzeptabel ermöglicht worden, die daher auch dem Tool-Smithing so richtig zum Durchbruch verhelfen.

Folgende Konsequenzen ergeben sich aus dieser Vorgehensweise:

- Das Design kann nicht mehr losgelöst von der Implementierung durchgeführt werden, weil schon in dieser Phase berücksichtigt werden muss, welche Bausteine zur Lösung dieses Problems zur Verfügung stehen. Design und Implementierung rücken dadurch näher zusammen, und schon die Wahl der Programmierungsumgebung kann zu einer völlig anderen Programmstruktur führen.
- Die Dauer der Implementierungsphase wird verkürzt. Insbesondere liegen schon sehr früh (Teil-)Ergebnisse vor, an denen man die Korrektheit des Designs kontrollieren kann.
- Die (Klassen-)Bibliothek, aus der die Bausteine stammen, muss ständig gewartet werden. Einsparungen bei der Implementierung werden teilweise durch eine neue Aufgabe kompensiert, die Archivierung. Aufgabe des Archivars (Bibliothekars) ist es, dafür zu sorgen, dass die Bibliothek auf effiziente Weise benutzt werden kann.
- In der Testphase wird nicht nur die Funktion des neuen Produkts geprüft, sondern auch die Funktion der dabei verwendeten Bausteine. Festgestellte Mängel müssen genauestens protokolliert werden. Die daraus resultierenden Änderungen müssen zentral in der Bibliothek vorgenommen werden, damit sie sich auch auf andere (sowohl parallel ablaufende als auch zukünftige) Projekte auswirken.
- Neu entstandene Bausteine (Klassen) müssen im Hinblick auf ihre allgemeine Brauchbarkeit geprüft werden. Wenn Aussicht besteht, dass ein Baustein auch in anderen Projekten verwendet werden kann, muss er in die Klassenbibliothek aufgenommen, entsprechend dokumentiert und den anderen Projekt-/Organisationsmitgliedern bekannt gemacht werden.

Das objektorientierte Vorgehensmodell ist in Abbildung 6.7 dargestellt. Die (Klassen-)Bibliothek dient dabei als projektübergreifendes Wissensarchiv, sodass sich das durch die Projekte erarbeitete Wissen zunehmend produktivitätssteigernd auf Folgeprojekte auswirkt.

6.4.3 Iteratives Vorgehen

Die Grundidee des *iterativen Vorgehens* ist die der kontinuierlichen Verbesserung durch wiederholte (= iterative) Ausführung der Basisschritte jeder Produktentwicklung (Abbildung 6.8

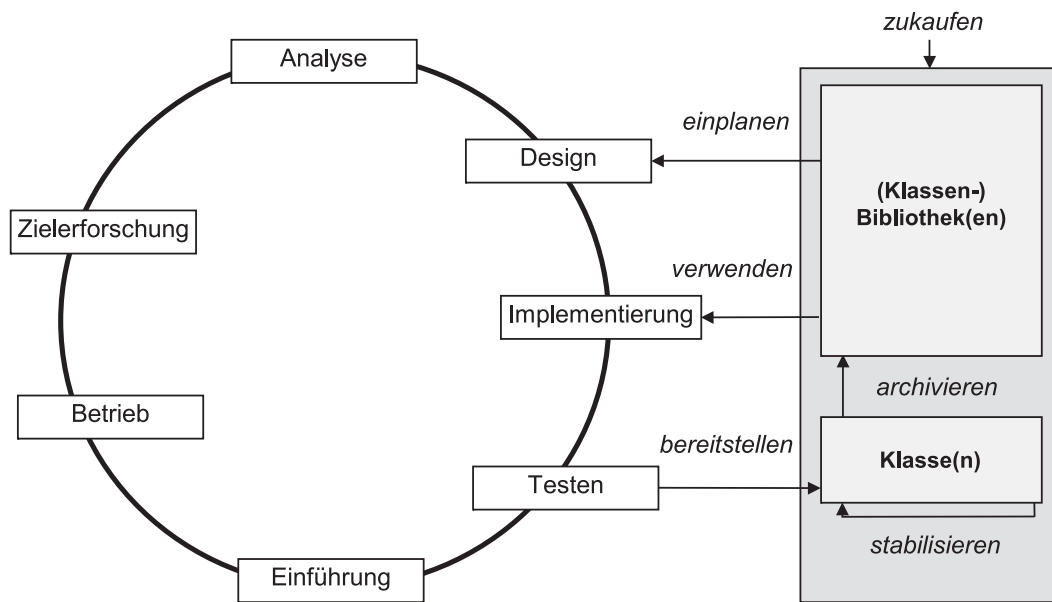


Abbildung 6.7: Vorgehen nach dem objektorientierten Modell

[Deming, 1986]). Dieses Konzept wurde von Deming im vergangenen Jahrhundert zur Steigerung der Produktivität der japanischen Wirtschaft eingesetzt (*Kaizen*-Prinzip); Abbildung 6.8 ist daher auch unter der Bezeichnung „Deming-Rad“ bekannt. Das Kaizen-Prinzip basiert auf der Annahme, dass jeder Mensch Fehler macht und dass es viel sinnvoller ist, deren Ursachen zu suchen als Schuldige [Deming, 1986]. Durch ein iteratives Vorgehen können viele Fehler früher erkannt und behoben werden, was die Gesamtkosten für die Fehlerkorrektur beträchtlich senkt (vgl. Abbildung 6.9 [Spalt, 2004]).

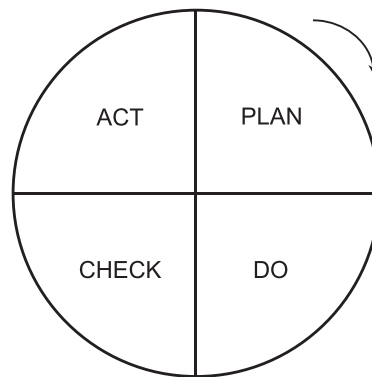


Abbildung 6.8: Grundschrte einer Iteration des kontinuierlichen Verbesserungsprinzips

6.4.3.1 Exploratives Prototyping

Prototypen und Prototyping

Eine Analyse der Literatur zeigt, dass durchaus keine Einigkeit darüber besteht, was man unter „*Prototyp*“ und „*Prototyping*“ im Software Engineering versteht [Pomberger und Bla-

Aktivität	Analyse	Design	Codieren	Debuggen	Testen	Betrieb
Relative Anzahl der entstandenen Fehler	10%	40%	50%			
Relative Anzahl der erkannten Fehler	3%	5%	7%	25%	50%	10%
Kosten pro Fehlerkorrektur (€)	250	250	250	1000	3000	12500

Abbildung 6.9: Fehlerbehebungskosten im Laufe der Projektentwicklung

schenk, 1996]: „Einigkeit herrscht offensichtlich darüber, dass Softwareprototypen ausführbar sein müssen. Ob diese Ausführbarkeit direkt gegeben sein muss oder simuliert werden kann, ist bereits kontrovers. Nahezu Einigkeit herrscht auch darüber, dass Softwareprototypen – im Gegensatz zu anderen Prototypen, z. B. in der Maschinenindustrie – schnell und billig realisiert werden müssen. Die einen schlagen vor, echte Prototypen im Sinn von Mustern zu entwickeln, die alle wesentlichen Eigenschaften eines geplanten Produkts aufweisen, und diese dann als Spezifikation für den eigentlichen Produktentwicklungsprozess zu verwenden – also herkömmliches Prototyping wie in anderen technischen Disziplinen. Andere wiederum sind der Meinung, Prototyping sei ein inkrementeller Entwicklungsprozess: man implementiert schnell einige wenige, von vornherein klare Basisfunktionen, lässt diese durch den Benutzer erproben und beurteilen, verbessert sie und implementiert weitere Benutzeranforderungen, bis das Produkt fertig ist – also evolutionäre Softwareentwicklung.“

In Anlehnung an [Connell und Shafer, 1989] definieren wir daher wie folgt [Pomberger und Blaschek, 1996]:

- Ein (*Software-*)*Prototyp* ist ein – mit wesentlich geringerem Aufwand als das geplante Produkt hergestelltes – einfach zu änderndes und zu erweiterndes ausführbares Modell des geplanten Softwareprodukts, das nicht notwendigerweise alle Eigenschaften des Zielsystems aufweisen muss, jedoch so geartet ist, dass vor der eigentlichen Systementwicklung der Anwender die wesentlichen Systemeigenschaften erproben kann.
- *Prototyping* umfasst alle Arbeiten, die zur Herstellung solcher Prototypen notwendig sind. Während Prototypen also auf jeden Fall Software enthalten müssen, kann Prototyping durchaus auch nicht unmittelbar computerbezogene Tätigkeiten umfassen (z. B. die Erstellung grafischer Benutzerschnittstellen mit Bleistift und Papier).

Prototyping ist also eine Vorgehensmethode, die nicht notwendigerweise Prototypen als (Zwischen-)Ergebnisse liefern muss! Prototypen entstehen oft aus der Notwendigkeit heraus, dem Kunden rasch einen Eindruck des erwarteten Produkts zu vermitteln; sie können durchaus auch durch andere Methoden als Prototyping entstehen!

Prototypingorientiertes Vorgehen

Ansätze der Idee, Softwareprodukte prototypingorientiert zu entwickeln, sind bereits im Wasserfallmodell enthalten. Neu ist, dass das Vorgehen nicht als lineares, sondern als iteratives

Modell anzusehen ist und dass angegeben wird, wo und in welcher Weise diese Iteration nicht nur möglich, sondern sogar notwendig ist. Da bei Prototyping die Phasen nicht mehr Abschnitte einer stetigen Entwicklung sind, wird oft der Begriff „Phase“ durch „Aktivität“ ersetzt.

Der entscheidende Unterschied zwischen Prototyping und dem klassischen Phasenmodell ist der, dass beim klassischen Vorgehen so spät wie möglich implementiert wird, bei prototypingorientierter Entwicklung jedoch so früh wie möglich. Die Praxis zeigt, dass man durch Letzteres viel eher zum Ziel kommt, weil man das Produkt schrittweise anhand eines Modells entwickelt, das es gestattet, auch das dynamische Verhalten eines Systems darzustellen, also durch ausführbare Prototypen und nicht bloß durch Beschreibungen. Prototyping ist daher eine topdown-orientierte Softwareentwicklungsmethode, die schneller realitätsnahe Funktionstests ermöglicht und so den Kunden früher und stärker in die Entwicklung einbinden kann.

Durch die Überlappung der einzelnen Aktivitäten und durch Ergebnisse in Form lauffähiger Prototypen wird das Risiko, eine Fehlentscheidung zu treffen, wesentlich verringert, und die Effekte, die mit den Zwischenergebnissen durch Experimente erzielt werden können, erleichtern die Qualitätssicherung.

Arten des explorativen Prototyping

Exploratives Prototyping wird in der Literatur unterteilt in [Floyd, 1984]:

- **Revelationäres Prototyping:** Das Ziel des revelationären Prototyping ist eine möglichst vollständige Problembeschreibung. Der Zweck besteht darin, den Entwicklern einen Einblick in den Anwendungsbereich zu ermöglichen, mit den Anwendern verschiedene Lösungsansätze zu diskutieren und die Realisierbarkeit des geplanten Systems in einem gegebenen organisatorischen Umfeld abzuklären (Revelation = Offenbarung).

Ausgehend von ersten Vorstellungen über das geplante System wird ein Prototyp entwickelt, der es erlaubt, diese Vorstellungen anhand von Anwendungsbeispielen zu prüfen und die erwünschte Funktionalität zu ermitteln. Maßgeblich dabei ist nicht die Qualität des Prototyps, sondern seine Funktionalität, die leichte Veränder- und Erweiterbarkeit und auch die Kürze der Entwicklungszeit. Revelationäres Prototyping findet vor allem während der Zielerforschungsphase und der Analysephase statt.

- **Experimentelles Prototyping:** Das Ziel des experimentellen Prototyping ist, eine vollständige Beschreibung von Teilsystemen zu erstellen als Grundlage zur Machbarkeitsfeststellung und damit für Designentscheidungen und letztlich die Implementierung. Der Zweck besteht darin, die Tauglichkeit von Teilmodellen des Designs, von Architekturmodellen und von Lösungsideen für einzelne Systemkomponenten experimentell nachzuweisen.

Ausgehend von ersten Vorstellungen über die Zerlegung des Systems wird ein Prototyp entwickelt, der es erlaubt, die Wechselwirkungen zwischen den Systemkomponenten zu simulieren sowie anhand von Anwendungsbeispielen die Schnittstellen der einzelnen Systemkomponenten und die Flexibilität der Systemzerlegung im Hinblick auf Erweiterungen zu erproben. Für die Qualität der Prototypen gilt gleiches wie beim revelationären Prototyping. Realisiert werden die Prototypen für die Durchführung

von Experimenten mit Architekturmodellen in der Hauptsache von den Entwicklern. Experimentelles Prototyping findet vor allem in der Designphase, aber auch in der Zielerforschungsphase und der Analysephase (zur Analyse der Machbarkeit) statt.

Exploratives Prototyping führt nicht nur zu einer Verminderung der Risikofaktoren im Softwareentwicklungsprozess, sondern bringt auch neue Risikofaktoren mit sich [Pomberger und Blaschek, 1996]:

- Beim Einsatz von Prototyping kann es dazu kommen, dass der Analyse- und Designprozess unnötig aufgebläht wird und in manchen Fällen nicht mehr zu enden droht (je mehr der Benutzer hat, desto mehr will er).
- Die Allgemeinheit und Flexibilität der Lösung kann auch übertrieben werden, sodass Eleganz und Effizienz einer Lösung vernachlässigt werden.
- Die Kostenrechtfertigung ist oft schwierig: Bei konventioneller Softwareentwicklung dauert es sehr lange, bis der Anwender mit lauffähigen Programmen konfrontiert ist, bei prototypingorientierter Softwareentwicklung ist das Umgekehrte der Fall. Der Benutzer kann sehr früh (mit Prototypen) experimentieren und findet einen Teil der Funktionalität schon in befriedigendem Ausmaß vor. Es entsteht der falsche Eindruck, dass die Herstellung eines Softwareprodukts viel einfacher geworden und mit viel geringerem Aufwand als bisher möglich ist. Dies führt beim Auftraggeber zu Unverständnis im Hinblick auf den tatsächlichen Aufwand, um aus einem Prototyp ein Produkt zu machen.

Um zu vermeiden, dass niemand ernsthaft explorative Prototypen testet, „da es ja eh nur Prototypen sind“, und auch um den in der Industrie im Softwarebereich bereits negativ besetzten Begriff „Prototyp“ zu vermeiden, wird in [Gregorio und Wakefield, Jr., 1994] vorgeschlagen, von einer *Arbeitsfortschritts-Demonstration* (*work-in-progress demonstration*) zu sprechen, selbst wenn es sich um einen Wegwerfprototypen handelt.

6.4.3.2 Spiralmodell

[Boehm, 1988] definierte die grundlegende Struktur des Ablaufes einer Softwareentwicklung als einen zyklischen Ablauf, der durch das *Spiralmodell* charakterisiert wird (Abbildung 6.10).

Im Spiralmodell stellt die radiale Ausdehnung den Gesamtaufwand dar, der bis zu einem bestimmten Zeitpunkt geleistet wurde, und die Winkeldimension bezieht sich auf den Projektfortschritt in den einzelnen Spiralzyklen. Jeder Zyklus umfasst die gleiche Schrittfolge für jeden Teil des zu entwickelnden Produkts und für jede Ausarbeitungsstufe.

Ein Spiralzyklus beginnt mit der Festlegung der Ziele und Anforderungen an das Produkt, der Ausarbeitung der Realisierungsalternativen (neue Lösung, Wiederverwertung, Zukauf) sowie der Auflistung der Nebenbedingungen und Einschränkungen. Im nächsten Schritt wird eine Beurteilung der vorgeschlagenen Lösungsvarianten im Hinblick auf die Projektziele und gegebenen Nebenbedingungen vorgenommen. Dabei geht es vor allem darum, mögliche Risikoquellen und Unklarheiten aufzudecken. Werden solche gefunden, schließt sich daran die Überlegung von Maßnahmen und Strategien zur Verminderung der Risikoquellen und deren potenziellen Auswirkungen an.

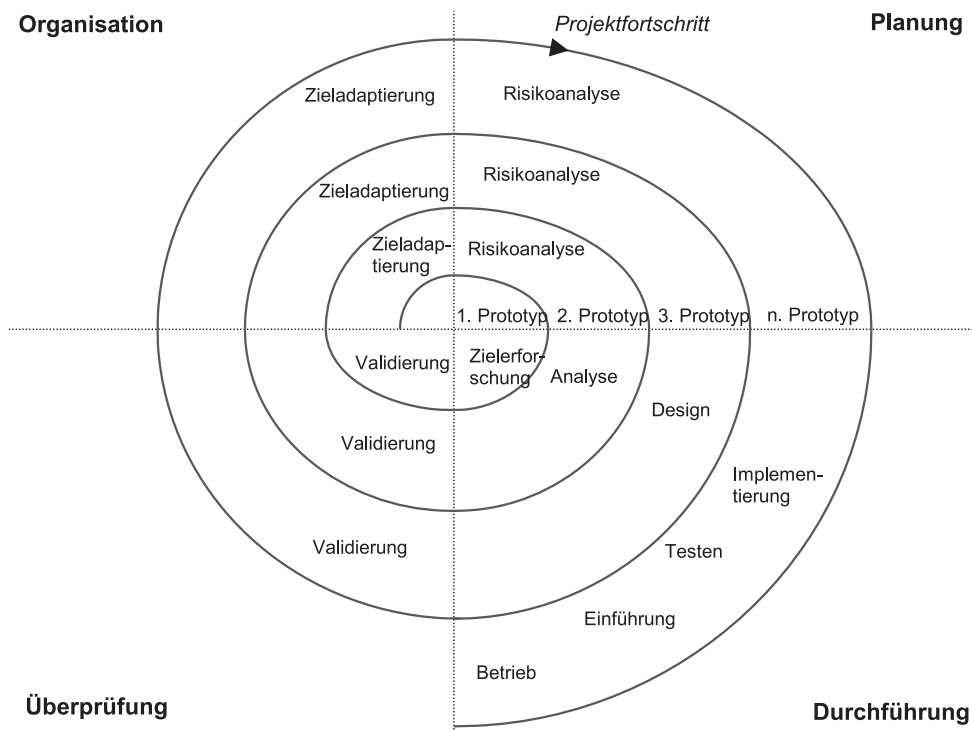


Abbildung 6.10: Das Spiralmodell

Diese Vorgehensweise setzt sich im Uhrzeigersinn fort, d.h., es erfolgt der jeweils nächste Schritt entsprechend dem klassischen Phasenmodell, dem wiederum eine Risikoanalyse vorausgeht und der durch eine Validierung, die gegebenenfalls zu einer Planadaptierung führt, abgeschlossen wird. Das Hauptziel des Validierungsschritts ist, dass sich alle Beteiligten über das bisher Geleistete und das im nächsten Zyklus Geplante einig sind. Die nächste Iteration setzt nicht notwendigerweise auf den Zwischenprodukten der vorherigen Iteration auf. In der Praxis ist dies jedoch oft der Fall, wodurch aus einer rein iterativen Entwicklung eine iterativ-inkrementelle Entwicklung wird.

Das Spiralmodell eignet sich gut für die Einbettung von Qualitätssicherungsmaßnahmen in den Softwareentwicklungsprozess, vor allem da Lösungsalternativen abgewogen werden sollen und frühzeitig Fehler erkannt und korrigiert werden können. Außerdem kann das Modell auch in einem fließenden Übergang für die Wartung angewendet werden. Der Nachteil des Spiralmodells ist seine starke Verknüpfung mit dem Wasserfallmodell. In jedem Zyklus wird das Projekt wasserfallartig durchlaufen; Schritte „quer über die Spirale“ sind unzulässig.

6.4.4 Iterativ-inkrementelles Vorgehen

Beim *iterativ-inkrementellen Vorgehen* erfolgt die Entwicklung ebenfalls in Zyklen („iterativ“), wobei jeweils auf den Ergebnissen des letzten Zyklus aufgebaut wird („inkrementell“). Diese Vorgehensweise bedingt einen laufenden Abgleich des entwickelten (Zwischen-) Produkts mit den (sich ebenfalls oft laufend ändernden) Anforderungen. Insbesondere können durch ein Zwischenprodukt die gewünschten Anforderungen seitens des Kunden geändert und/oder neu priorisiert werden.

Dieser dynamische Zusammenhang zwischen Anforderungen und Produkt wurde von Nuseibeh im so genannten *Twin-Peaks-Modell* dargestellt (Abbildung 6.11, adaptiert aus [Nuseibeh, 2001]). Entscheidend dabei ist, wie weit die *gelieferte Funktionalität* für den Kunden ein brauchbares Produkt darstellt und ob die Relation zur *geforderten Funktionalität* stimmt. Da aber durch das iterativ-inkrementelle Vorgehen häufig Zwischenprodukte erstellt werden, kann durch deren Abgleich mit dem Kunden die Entwicklungsrichtung laufend angepasst werden.

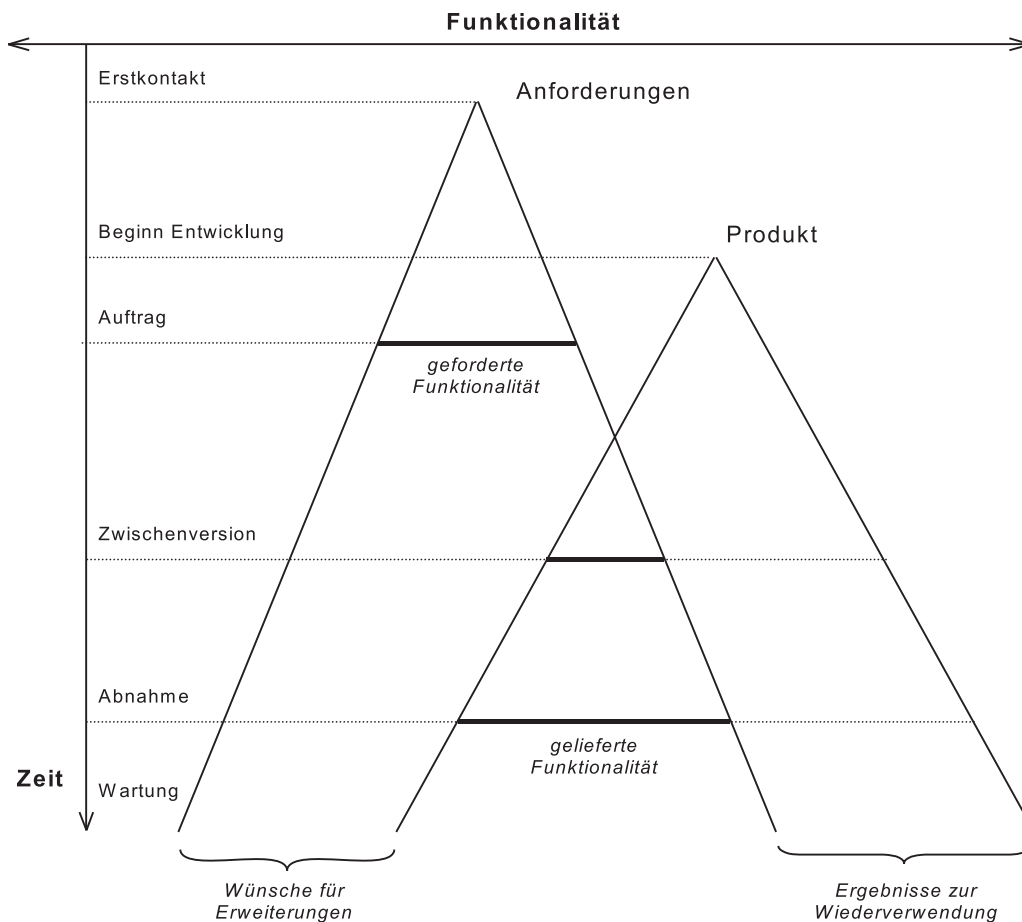


Abbildung 6.11: Das Twin-Peaks-Modell zur Visualisierung des dynamischen Zusammenhangs Anforderungen – Produkt

6.4.4.1 Evolutionäres Prototyping

Evolutionäres Prototyping beginnt mit der Entwicklung eines Prototyps für die von vornherein klaren Benutzeranforderungen. Das Ergebnis dient als Basissystem für den Anwender und für den nächsten Schritt, in dem neue Benutzeranforderungen integriert werden und der Prozess von neuem beginnt.

Damit verliert die Systementwicklung den Charakter eines abgeschlossenen Projekts und wird ein Prozess, der die Anwendung ständig begleitet. Das ist leichter gesagt als getan. Dadurch wird zwar das Analyseproblem zum Teil entschärft, aber das Problem des Designs

keineswegs gelöst, weil eine unvollständige Analyse gerade der Grund dafür sein kann, dass die Brauchbarkeit einer Zerlegungsstruktur nicht beurteilt werden kann.

Evolutionäres Prototyping beginnt idealerweise bereits in der Analysephase, jedenfalls aber in der Designphase und stellt den Kernpunkt einer prototypingorientierten Implementierungsphase dar. Bei dieser Vorgehensweise lässt sich eigentlich nicht mehr zwischen Prototyp und Produkt unterscheiden, die Bezeichnung Prototyping ist aber deswegen angebracht, weil die ersten Versionen sicher nicht als praktisch einsetzbare Produkte zu sehen sind.

Beim evolutionären Prototyping werden die Prototypen nicht simuliert und in der Regel auch nicht weggeworfen, sondern Schritt für Schritt zum Produkt ausgebaut. *Objektorientiertes Denken* eignet sich aus folgenden Gründen sehr gut als Basis für (evolutionäres) Prototyping [Coad und Yourdon, 1990]:

- Die Wiederverwendbarkeit von Konzepten ist durch die Verfügbarkeit von Klassenbibliotheken, Komponenten, Schemata/Schablonen etc. hervorragend gegeben.
- Die Konzepte der Spezialisierung und Aggregation bedeuten eine rasche Adaptierbarkeit und Erweiterbarkeit von Prototypen.
- Durch die für eine Programmiersprache explizite Beschreibung der Problemsemantik kann die Funktionalität eines Problems exakt beschrieben werden, d. h., ein programmierter Prototyp ist im Allgemeinen viel besser in der Funktionalitätsbeschreibung als eine papierbasierte Darstellung.

Auch evolutionäres Prototyping ist mit einer Reihe von Risikofaktoren behaftet [Pomberger und Blaschek, 1996]:

- Es besteht die Gefahr der Auslieferung von Prototypen statt von Produkten. Wenn die Anwender während des Analyseprozesses mit Prototypen konfrontiert werden, die bereits einen Großteil der gewünschten Funktionalität aufweisen, dann ist häufig zu beobachten, dass die Anwender lieber weniger und noch nicht vollständig ausgefeilte Funktionalität sofort einsetzen wollen als auf eine saubere und effiziente Implementierung, für die der Prototyp nur das Modell darstellt, zu warten. Dies führt dann zu einer Vernachlässigung softwaretechnischer Prinzipien in der Systemimplementierung und damit zur Gefahr des Verlusts von Qualitätsmerkmalen, allen voran Struktur, Erweiterbarkeit und Effizienz.
- Ab einem gewissen (im Allgemeinen eher frühen) Zeitpunkt erfordert Prototyping Demonstrationen. Diese bedeuten vielfach ein hohes Risiko, dass der Kunde das Verhalten des Prototypen falsch einschätzt (sowohl positiv bzgl. vorgetäuschter Funktionalität als auch negativ bzgl. planmäßig noch nicht realisierter Funktionalität). Meilensteine sind daher Risikopunkte, und Kunden verlassen sich sehr stark auf gezeigte Ergebnisse. "Show quality or show nothing!" soll daher auch für die Präsentation von Prototypen gelten [Gregorio, 1996].
- Gängige Softwaremaße sind vielfach für Prototypen nicht anwendbar bzw. schlecht adaptierbar. Spezielle Maße für Prototypen (speziell im Hinblick auf Meilensteinerreichung) sind bislang noch nicht definiert worden und vermutlich schwer allgemein festzulegen.

6.4.4.2 Schablonenmodell

Die Vorgehensweise des *schablonenmäßigen Vorgehens* ist das Erkennen und Anwenden vorgefertigter (vorgedachter) Denkschemata (*patterns*) bzw. Problemlöseschablonen (*templates*). Durch das Klassifizieren eines neuen Problems (*pattern matching*) in Bezug auf einen vorhandenen *Schablonenkatalog* kann das neue Problem „im Wesentlichen“ routiniert gelöst werden.

Diese Methode reduziert bzw. eliminiert zwei der Hauptprobleme des (objektorientierten) Tool-Smithing und des Prototyping, die gerne in Kombination angewendet werden:

1. Im Tool-Smithing können nur vorgefertigte Bausteine verwendet werden. Abänderungen führen zu einer laufenden Erweiterung der Bausteinanzahl, auch wenn diese hierarchisch geordnet sind. Mit Hilfe von Schablonen können ganze Teilbäume dieser Hierarchie gleichsam parametrisiert und schematisiert werden.
2. Gerade das „Lauffähig Machen“ von Prototypen ist vielfach mehr Arbeit als vorher angenommen, speziell bei Prototypen, die nicht nur eine Schnittstelle abklären sollen, sondern das innere Programmverhalten oder gar ein bestimmtes Zeitverhalten. Schablonen können auch eine (schablonenartige) Beispielsimplementierung beinhalten (vgl. z. B. [Gamma *et al.*, 1997]), die sehr effizient für Prototyping verwendet werden kann.

Derzeit wird das *Schablonenmodell* hauptsächlich in einem sehr eng gekoppelten Design- und Implementierungsprozess verwendet, die Schablonen sind aber genauso in der Analyse anwendbar (vgl. [Fowler, 1997]) und könnten dadurch auch eine entscheidende Annäherung der Analyse zur Implementierung mit sich bringen.

6.5 Prozessoptimierung

Die Verwendung *definierter Prozesse* (oder die Entwicklung eigener Prozesse) muss darauf abzielen, diesen Prozess laufend anpassen und damit verbessern zu können. Erst dadurch macht sich die Investition in die systematische Prozessanwendung im Laufe nachfolgender Projekte bezahlt. Dies gilt bei traditionellen Vorgehensmethoden, noch viel mehr aber bei agilen Methoden, bei denen Änderungen nicht nur innerhalb eines Projekts, sondern natürlich auch von Projekt zu Projekt inhärent sind. Folgende vier Schritte sind für jede Adaptierung/Optimierung eines Prozesses entscheidend:

1. Prozess identifizieren,
2. Prozess verstehen,
3. Prozess wirtschaftlich machen,
4. Prozess verbessern.

Obwohl viele agile Methoden in ihrer Definition keine Schritte zur Prozessverbesserung vorsehen (oder zumindest nicht besonders betonen), ist Prozessverbesserung mit agilen Methoden durchaus kompatibel, wie [Paulk, 2001] im Rahmen seiner Prüfung von XP auf CMM-Tauglichkeit feststellt.

6.5.1 Prozessidentifikation: Bedeutung des Messens

Wenn der Projektstatus gemessen wird, ist es essenziell, zwei Aspekte zu unterscheiden:

1. Messen des Projektfortschritts,
2. Messen des Grades der Zielerreichung.

Diese beiden Aspekte werden häufig verwechselt oder vermischt, gerade von unerfahreneren Softwareentwicklern. Während der Projektfortschritt am besten *entwicklerseitig* gemessen werden kann, und zwar in einer Metrik⁶, die für die Entwickler Bedeutung besitzt (lines of code etc., [Humphrey, 2005]), ist für das Messen des Grades der Zielerreichung ein umfassendes Verständnis des Auftraggeberinteresses (bzw. des Benutzerinteresses) entscheidend [Grünbacher, 2003].

Gerade agile Vorgehensmethoden zeichnen sich dadurch aus, dass die sich aus dem Projektziel ergebenden Anforderungen häufig geändert werden können. Manche Methoden motivieren den Kunden sogar explizit, seine Anforderungen häufig zu ändern, vgl. [Beck und Andres, 2005].

Agiles Vorgehen muss daher mit dem Anvisieren eines beweglichen Ziels verglichen werden. Dies impliziert, dass das Erreichen eines Projektfortschritts – das für sich selbst bereits schwierig zu messen ist, vgl. [Goldberg und Rubin, 1995] – eine Bewegung in die falsche Richtung bedeuten kann und daher noch weniger mit dem Grad der Zielerreichung korreliert als bei traditionellen Methoden.

Konsequenterweise muss sich ein Softwareentwicklungsteam, wenn es agile Methoden verwendet, in raschen, kurzen Schritten fortbewegen, wobei das Ziel laufend neu anvisiert werden muss (vgl. Abbildung 6.12; die durch die Spirale überdeckte Fläche gibt die realisierte Funktionalität an).

Viele Metriken des Projekt Engineering wurden mittlerweile für agile Vorgehensmethoden angepasst (siehe z. B. [Olsina *et al.*, 2002] für eine gute Übersicht). Entscheidend für gute Abschätzungen sind – wie bei jeder Metrik – sauber definierte Zählrichtlinien. Das Hauptproblem der meisten Metriken bleibt allerdings, dass entwicklungsbezogene Designkomponenten gezählt werden (Funktionen, Objekte, Klassen etc.), der Input also mehr entwicklerseitig als kundenseitig gegeben wird. Fortschritt lässt sich damit gut messen. Ob damit auch der – schlussendlich entscheidende – Grad der Zielerreichung steigt, bleibt offen.

6.5.2 Prozessverständnis: Feedback durch kurze Iterationszyklen

Durch kurze Iterationszyklen ist es häufiger möglich, Feedback vom Auftraggeber zu erhalten und damit genauer festzustellen, wo man sich als Auftragnehmer in Relation zum Ziel befindet [Fowler, 2004a]. Durch das bewusste Anwenden einer Vorgehensmethode sowie die kleinen Schritte, nach denen jeweils eine Validierung durch den Auftraggeber durchgeführt wird, sind agile Methoden auch klar vom *Hackertum* abgegrenzt (Abbildung 6.13). Allerdings ist es,

⁶Eine *Metrik* ist ein quantitatives Maß für den Grad, in dem eine Systemkomponente oder ein Systemprozess eine bestimmte Eigenschaft besitzt [IEEE, 1991].

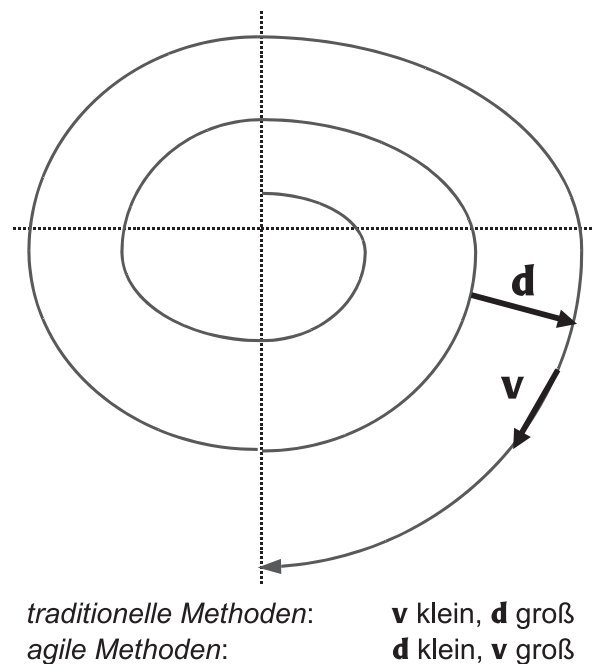


Abbildung 6.12: Der Zusammenhang von Iterationsgeschwindigkeit und Inkrementgröße

um den Zyklus geordnet abwickeln zu können, notwendig, in jedem Zyklus die Aufgabenkette „Planen–Durchführen–Überprüfen–Adaptieren“ zu durchlaufen. Dies bedeutet, dass ein Zyklus nicht beliebig kurz (und damit ein Inkrement nicht beliebig klein) gemacht werden kann, ohne einen sehr hohen Organisations-Overhead (Planung, Kontrolle, Besprechungen, ...) in Kauf zu nehmen.

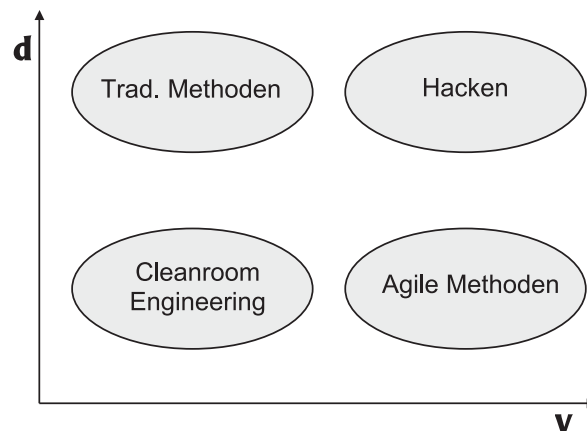


Abbildung 6.13: Geschwindigkeit versus Inkrementgröße in traditionellen und agilen Methoden

Die Vorgehensweise der kurzen, kontrollierten Iterationszyklen ist identisch mit der des *Rapid Application Development (RAD)*, einer in der klassischen Softwareentwicklung seit mehr als 20 Jahren bekannten und eingesetzten Technik (vgl. z. B. [Coad und Yourdon, 1990]). Neu ist, dass kaum mehr versucht wird, Anforderungen im Vorhinein möglichst umfassend abzuschätzen und damit natürlich auch Projektgröße und -kosten [Reifer, 2002]. Dazu kommt,

dass vielfach starker Druck hinsichtlich eines definierten, sehr kurzfristigen Liefertermins gemacht wird⁷, sodass bei agilem Vorgehen „an Stelle der Kosten eines wohlspezifizierten Systems vielmehr die Qualität eines zeitfixierten Produkts ... abgeschätzt werden solle“ [Pressman, 1998].

6.5.3 Prozessökonomie: Agiles Vorgehen

Die Hauptziele traditioneller Vorgehensmethoden sind Vorhersehbarkeit, Wiederholbarkeit und Optimierbarkeit [Boehm, 2002]. Dies führt zum Streben nach vollständigen, konsistenten, präzisen, testbaren und im Lauf der Entwicklung verfolgbaren Anforderungen, um das Produktrisiko zu reduzieren. Dies geschieht planfokussiert durch umfangreiche Projektpläne und Architekturmodelle, die mehrmals vor Beginn der Implementierung reviewt werden. Einsatzgebiete, in denen dieses Vorgehen wichtig (und auch bezahlbar) ist, sind Systementwicklungen, die für Menschen sicherheitskritisch sind (z. B. im Gesundheitsbereich oder im Transportwesen).

Durch das Erstellen umfangreicher Pläne und Designmodelle entsteht aber ein neues Risiko, nämlich dass Änderungen der Projektanforderungen oder des Ablaufs die laufende Aktualisierung der Pläne und Modelle sehr aufwändig gestalten oder diese sogar vollständig unbrauchbar werden. [Highsmith, 2000] stellt dazu fest, dass, wer „... in einem komplexen System einen Plan [bedingungslos] verfolgt, das Produkt erhält, das er geplant hat, nicht das Produkt, das er am Projektende braucht“.

Agile Vorgehensmethoden reduzieren daher die angefertigten Pläne und Modelle auf das Mindestmaß, das Auftragnehmer und Auftraggeber unbedingt brauchen. Das von den Hauptvertretern dieser Methoden herausgegebene „Agile Manifest“ ([Fowler und Highsmith, 2001]; vgl. dazu auch Abschnitt 6.3.1) wurde ursprünglich bewusst sehr provokativ formuliert⁸ und kann einen unerfahrenen Softwareentwickler dazu verleiten anzunehmen, dass Verträge, Pläne, Dokumente und Werkzeuge in der modernen Softwareentwicklung obsolet geworden wären. In [Coldewey, 2002] wird daher das Agile Manifest verständlicher und gemäßigter formuliert:

„Kollaboration statt Formalismus,
kurze Inkremente statt jahrelanger Meditation,
Flexibilität statt Planungsorgien,
Einbindung des Kunden statt Absicherung.“ [Coldewey, 2002]

Auch die Autoren des Agilen Manifests betonen mittlerweile regelmäßig, dass das Ignorieren der „rechten Seite“ der Aussagen (Verträge, Pläne etc.) eine Fehlinterpretation darstellt (vgl. z. B. [Jeffries *et al.*, 2000]). Trotzdem bleibt die anfängliche, gewollte Provokation in aller Erinnerung und lässt kaum gemäßigte, balancierte Meinungen zu. Beispielsweise merkt Steven Raitkin an, dass aus seiner Erfahrung „... die Aussagen auf der rechten Seite [des Manifests] die essenziellen Teile darstellen, während die Aussagen auf der linken Seite meist nur als Ausreden für Hacker dienen, Code auf unverantwortliche Weise zusammenzuschustern“ [Raitkin, 2001].

⁷In den letzten zehn Jahren sank die durchschnittliche Laufzeit von Softwareprojekten in den USA von zwei Jahren auf sechs Monate. [Reifer, 2002]

⁸[Orr, 2002] stellt dazu fest: “If you want to start a religious or software war, issue an edict or manifesto.”

Ohne die provokative Formulierung des Agilen Manifests und das forsche Auftreten insbesondere der Vertreter der XP-Vorgehensmethode wäre aber dem Gedanken des agilen Vorgehens sicherlich kaum genügend Gehör verschafft worden. Es war nur auf diese Weise möglich, „sich von den Prozess-Dinosauriern der achtziger und neunziger [Jahre] ab[zu]grenzen und wieder etwas Umsetzbares [zu] schaffen“ [Coldewey, 2002]. Vor allem war kaum ein Softwareentwickler bereit, die hunderte oder gar tausende von Seiten Beschreibung einzelner Vorgehensmodelle (wie RUP, CMM) durcharbeiten oder gar umzusetzen.

Selbst relativ junge Vertreter/Adaptierungen dieser Modelle, wie etwa das CMMI⁹, nehmen noch kaum Rücksicht auf die effiziente Möglichkeit, agile Vorgehensweisen umzusetzen, wie [Boehm, 2002] betont. Die gesamte derzeitige Bandbreite des Planungsspektrums ist in Abbildung 6.14 dargestellt (adaptiert aus [Boehm, 2002]). Undiszipliniertes Vorgehen („Hacken“) stellt dabei das linke Extrem dar, während exzessives Mikromanagement („Erbsen zählen“) das rechte Extrem charakterisiert.

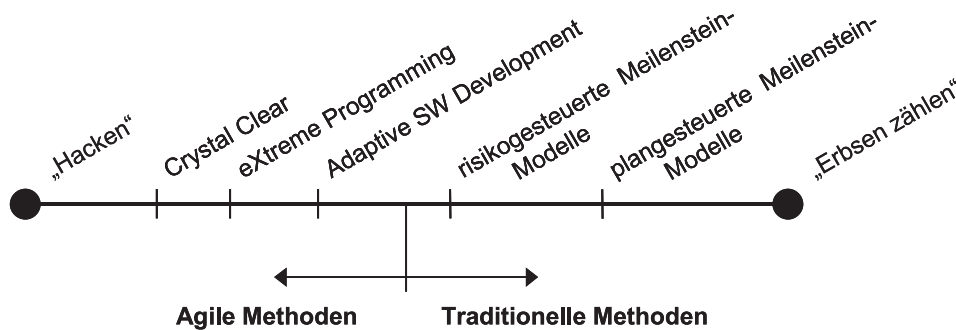


Abbildung 6.14: Spektrum der Bedeutung der Gesamtplanung im Projekt Engineering

Mit der Transition zum agilen Vorgehen wird aber nicht nur die Prozessökonomie verbessert, also das Verfolgen eines Prozesses wieder wirtschaftlich möglich gemacht, es werden auch insbesondere dem (nichttechnischen) Projektmanagement lieb gewordene Vorstellungen verabschiedet [Coldewey, 2002]:

- Ein einheitlicher, wiederholbarer Entwicklungsprozess ist nicht definierbar.
- Projekte lassen sich nicht sinnvoll detailliert planen, egal wie viel Aufwand man treibt.
- Eine ausreichende Projektkontrolle ist ohne Kooperation der Entwickler unmöglich.

[Coldewey, 2002] betont aber, dass rigide Praktiken notwendig sind, den (agilen) Entwicklungsprozess selbst unter Kontrolle zu halten. Auch [Beck und Andres, 2005] stellen fest, dass das Scheitern eines XP-basierten Projekts meistens auf Grund zu lockerer Anwendung der Vorgehensmethode erfolgt.

⁹Capability Maturity Model Integration; vgl. z. B. [Kulpa und Johnson, 2003]

Folgende Vorteile ergeben sich durch agiles Vorgehen (vgl. dazu auch [Humphrey, 2002]):

- iterativ-inkrementelle Entwicklung mit kurzen Releasezyklen,
- Beachtung überprüfbarer Qualität von Anfang an,
- intensive Einbindung des Auftraggebers/Kunden,
- ko-lokiertes Entwicklerteam mit direkter, intensiver Kommunikation,
- gleichmäßige Belastung der Mitarbeiter (Vermeidung von Stress-Spitzen),
- Aufwandsschätzungen durch Entwickler (dadurch Vertrauen in und Identifikation mit den Schätzungen),
- Erstellen und Anwenden von (Programmier-)Standards,
- möglichst einfaches Design,
- laufende Reviews (zumindest des Codes),
- durch Systemtests vorangetriebene Entwicklung,
- kontinuierliche Integration (vgl. Abschnitt 12.1.3.7).

Demgegenüber stehen folgende Nachteile, die durch agiles Vorgehen bedingt sind bzw. entstehen (vgl. dazu beispielsweise [Stephens und Rosenberg, 2003] und [Humphrey, 2002]):

- nur hoch qualifizierte Mitarbeiter brauchbar,
- unzureichende Skalierbarkeit für größere Teams,
- mangelnde Transparenz des Vorgehens für das Management,
- Prozessverbesserungen nur zufällig und nicht wiederholbar,
- fehlende Migrationsmöglichkeit aus bestehenden Prozessen,
- fehlender Qualitätsplan,
- beinahe kontinuierliche Anforderungsänderungen,
- keine Design-Reviews (bei Code-Reviews fehlt Übersicht),
- stark inkrementelles Design (oft ohne Zeit für Refactoring),
- fehlende Designdokumentation („Big-Picture“-Design), damit fehlende Design-Reviews und umfangreiches, teures Testen,
- Code-Zentriertheit statt Design-Zentriertheit (rasch steigende Änderungsproblematik),
- fehlende Quellen für Systemtests (wie etwa Pflichtenheft etc.).

Speziell das Fehlen eines „Big-Picture“-Designs (und damit eines Grobübersichts) und die mangelnde Einbindung des Managements haben in den letzten Jahren zu einer Neubewertung der Vorteile des agilen Vorgehens und eine Diskussion über dessen Grenzen geführt. Gerade die Gruppierungen im Bereich der *modellgesteuerten Entwicklung* (*model-driven development* – *MDD*) werfen diesbezüglich die kritischsten Fragen auf („Wann wird agile Softwareentwicklung zu fragil?“ u. dgl.; [Mellor *et al.*, 2003]).

Auch das Projekt, im Rahmen dessen Kent Beck die fundamentalen Grundlagen von XP erarbeitet hat (das mittlerweile berühmt-berüchtigte Chrysler C3 Projekt), wurde nach mehreren Jahren Arbeit erfolglos abgebrochen, weil die Entwickler den Überblick über das Design verloren hatten und als Konsequenz die Unterstützung des Managements verloren ging [Coldewey, 2002] [Stephens und Rosenberg, 2003].

6.5.4 Prozessverbesserung: Bewährte Praktiken im 21. Jahrhundert

Was wird nach dem (absehbaren) Ende des Agilen Hypes übrig bleiben? Auf den ersten Blick scheint das eher wenig zu sein, da die Dissonanz der Verfechter der verschiedenen Methoden deutlich größer ist als deren Konsonanz. Dennoch lassen sich eine Anzahl von Praktiken entdecken, die mittlerweile auch von den Vertretern traditioneller Methoden aufgegriffen wurden (etwa die intensive Kundeneinbindung) oder sich über alle Methodengrenzen hinweg mehr oder weniger gleichzeitig entwickelt haben (etwa die iterativ-inkrementelle Entwicklung). Zusätzlich wird bereits ernsthaft über Kombinationsmöglichkeiten beider „Vorgehenswelten“ diskutiert (vgl. z. B. [Boehm, 2002]).

Folgende Gemeinsamkeiten aller (oder zumindest der meisten) derzeit aktuellen Vorgehensmethoden können festgestellt werden und stellen damit einen Mindestdatz bewährter Projekt Engineering-Praktiken dar, die im Rahmen einer Prozessanalyse/Prozessverbesserung übernommen werden können:

- Das Projektziel wird als eine Art Vision definiert, die nicht aus den Augen verloren werden darf.
- Der Auftraggeber/Kunde wird laufend in die Entwicklung eingebunden.
- Die Entwicklung erfolgt iterativ-inkrementell mit möglichst kurzen Zyklen.
- Durch die potenziellen häufigen Änderungen nimmt eine laufende Risikobehandlung höchsten Stellenwert ein.
- Neu hinzugekommene Anforderungen werden zuerst durch zugehörige Testfälle präzisiert und erst dann implementiert.
- Dokumente werden nicht als Selbstzweck (oder „des Prozesses“ wegen) erstellt, sondern nur dann, wenn sie dem Auftragnehmer unmittelbar nützen oder vom Auftraggeber explizit verlangt werden.
- Langfristige Planung wird durch laufende kurzfristige (Um-)Planung ersetzt.
- Zu Projektbeginn werden nur die minimalen Mindestanforderungen festgelegt, deren Realisierung so rasch wie möglich begonnen wird.

- Anforderungsänderungen sind kein lästiges Übel, sondern alltäglich und können ins Design und in den Code eingearbeitet werden.
- Das Design ermöglicht einen groben Überblick und ist möglichst einfach, dabei aber gut erweiterbar.
- Einzeltests werden von den Entwicklern, Systemtests gemeinsam mit dem Kunden definiert. Beide werden laufend – möglichst automatisiert – durchgeführt.

Insgesamt muss festgestellt werden, dass noch immer erschreckend wenige „professionelle“ Softwareentwickler diese Praktiken auch nur teilweise umsetzen [Cockburn, 2001]. Unabhängig vom Streit, welche Vorgehensmethode nun die geeignetste ist, muss es daher die gemeinsame Aufgabe sein, jedem Softwareentwickler wenigstens *irgendeine* definierte Vorgehensmethode eigen zu machen. Jede Methode wird eine Hilfe darstellen [Humphrey, 2002], eine Prozessadaptierung und -verbesserung kann dann später erfolgen.

7 Informationsaustausch

Dieses Kapitel gibt einen Überblick über Bedeutung und Art des Informationsaustauschs in Softwareprojekten. Zuerst wird der durch die verstärkte Kundeneinbindung steigenden Bedeutung gut vorbereiteter und strukturierter Besprechungen Rechnung getragen. Anschließend wird die Dokumentation von Projekten und deren effiziente Organisation diskutiert. Zum Schluss werden die wichtigsten Dokumente der Aufbau- und Ablauforganisation vorgestellt.

7.1 Kommunikation in Softwareprojekten

Die *Kommunikation in Projekten* erfolgt sowohl mündlich – strukturiert in Form von Besprechungen und natürlich auch unstrukturiert – als auch schriftlich in Form verschiedenster Dokumente. Dabei ist zu unterscheiden, ob der Informationsaustausch intern beim Auftragnehmer erfolgt oder zwischen Auftraggeber und Auftragnehmer (extern). In beiden Fällen ist es – besonders im Fall getroffener Entscheidungen – ratsam, die Ergebnisse unbedingt schriftlich festzuhalten, da mündlich getroffene Entscheidungen später schwer nachzuvollziehen (und ggf. nachzuweisen) sind. [Gilb, 1996] merkt dazu pointiert an: „Wenn du es nicht in geschriebener Form hast, dann habe ich es nicht gesagt.“

Die Bedeutung gut strukturierter Projektkommunikation steigt auch durch zwei Trends, die für moderne Softwareprojekte beobachtbar sind:

1. Durch agile Vorgehensmethoden steigt die Häufigkeit der Kundeneinbindung (bis hin zur durchgehenden Präsenz eines Auftraggebervertreters vor Ort beim Entwicklerteam). Eine saubere Gesprächs- und Dokumentationskultur ist auf Grund der Transparenz für den Auftraggeber unumgänglich.
2. Andererseits steigt die Anzahl der Projekte, bei denen die Mitarbeiter zwar gemeinsam an einem Projekt arbeiten, aber räumlich über weite Distanzen (bis hin zu global) verteilt sind. Eine effiziente Kommunikation und ein sauber definierter Daten- und Dokumentaustausch via Telekommunikations- und Computernetze sind erfolgskritisch für diese Art der Kooperation.

7.1.1 Kundeneinbindung

Die Beteiligten und Betroffenen eines Projekts werden allgemein unter dem Begriff *Stakeholder* zusammengefasst. Stakeholder sind Personen oder Organisationen, die durch das Projektergebnis betroffen sind und einen direkten oder indirekten Einfluss auf die Projektanforderungen oder Rahmenbedingungen nehmen [Kotonya und Sommerville, 1998]. Wichtige Stakeholder(-gruppen) sind:

- Kunde,
- Anwender (Benutzer),
- Auftraggeber,
- Entwickler,
- leitende Mitarbeiter des Auftragnehmers (Projektleiter, Vertriebsleiter, Leiter der Qualitätssicherung, ...),
- Eigentümer des Auftragnehmers,
- Subauftragnehmer, Zulieferer u. dgl.

Für jeden Stakeholder ist festzustellen, was er sich als Projektergebnis *wünscht* und was er wirklich *braucht*. Dabei ist es Aufgabe des Projektleiters, die Wünsche und Erfordernisse der Stakeholder (sowie die Stakeholder selbst) nach ihrer Relevanz und Erfolgskritikalität zu priorisieren.

Traditionell werden Kundenwünsche in stark formalisierten Dokumenten wie dem Lastenheft (vgl. Abschnitt 10.1.5.2) oder dem Pflichtenheft (vgl. Abschnitt 11.5.2) festgehalten. Im Rahmen der agilen Entwicklung ergänzen Prototypen/Zwischenprodukte und Protokolle von Iterationsreviews diese Dokumente oder ersetzen sie sogar. Aktuelle Ansätze der Anforderungsanalyse beziehen daher den Kunden viel intensiver und vor allem viel längerfristig in den Entwicklungsprozess mit ein. Es ist mittlerweile klar, dass Anforderungen nicht durch simple Befragung der Stakeholder gewonnen werden können, sie sind vielmehr das Ergebnis eines gemeinsamen Lern- und Konsensbildungsprozesses [Lowe und Eklund, 2002]. Statt „... der Chimäre der vollständigen Spezifikation nachzujagen, bezieht [man] ... den Kunden direkt in das Projekt ein“ [Coldewey, 2002].

Zusätzlich steigt die Bedeutung erfolgreicher Kundeneinbindung dadurch, dass bei der Priorisierung der Funktionalitäten einer Produktentwicklung immer stärker deren Wert für den Auftraggeber im Verhältnis zu den eingesetzten Mitteln im Vordergrund steht (*value-driven requirements engineering*, vgl. [Boehm, 2000]). Dies bedingt aber, dass der Auftraggeber für den gesamten Verlauf der Entwicklung inhaltlich kompetent und zusätzlich entscheidungsbefugt sein muss [Boehm, 2002]. Ein derartiger Auftragnehmer(-vertreter) ist in der Praxis nur selten anzutreffen und wenn doch, dann oft terminlich schwer verfügbar.

Gerade bei agilem Vorgehen ergibt sich in der Praxis, dass ein Fortschritt in den Arbeiten kaum möglich ist, wenn nicht ein fachlich kompetenter Entscheidungsträger kurzfristig für Fragen zur Verfügung steht, vgl. [Coldewey, 2002]. Der Druck auf den entwicklerinternen Pilotanwender (vgl. dazu Abschnitt 7.3.2) als Vertreter der Auftraggeberinteressen ist daher

bei agilen Projekten in der Praxis oft sehr groß. Eine geeignete *Werkzeugunterstützung der Anforderungsanalyse* ist daher gerade bei agilem Vorgehen mit enger Kundeneinbindung für ein Projekt überlebensnotwendig, vor allem um die – während des Projekts laufend erhobenen – Anforderungen zu sammeln, zu priorisieren sowie Änderungen zu erkennen und verfolgen zu können (*traceability* von Anforderungen, vgl. [Grünbacher, 2003]).

7.1.2 Besprechung

Projektbesprechungen (auch *Statusbesprechungen*, *Arbeitsbesprechungen*) haben eine besondere Bedeutung für die Ablauforganisation. Einerseits ist gerade in kleinen Projekten die unmittelbare Kontrolle durch den Projektleiter bei regelmäßigen *internen Projektbesprechungen* sinnvoll und möglich, andererseits ist bei *externen Projektbesprechungen*, an denen Vertreter des Auftraggebers beteiligt sind, der Informationsaustausch zwischen Auftraggeber und Auftragnehmer rasch und problembezogen möglich.

Eine externe Projektbesprechung besteht aus drei Besprechungsteilen:

1. interne *Vorbesprechung*,
2. externe *Besprechung*,
3. interne *Nachbesprechung*.

Aufgaben der *Vorbesprechung* sind:

- Sicherstellen des gleichen Wissensstandes bezüglich des Projekts,
- Analysieren der gewünschten Resultate des Auftragnehmers (Eine besondere Abstimmung ist notwendig, um Meinungsverschiedenheiten nicht extern auszutragen!),
- Analysieren der (vermutlich) erwarteten Resultate des Auftraggebers,
- Erarbeiten der Besprechungsstruktur (Tagesordnungspunkte); Erstellen der Tagesordnung (Agenda) und Verteilen dieser an alle Besprechungsteilnehmer.

Im Allgemeinen leitet der Projektleiter die *Besprechung*. Seine Hauptaufgaben sind:

- Sorgen für das Einhalten der Tagesordnung und des zeitlichen Rahmens,
- Gliedern des Gesprächs,
- Vorhersehen der Gesprächsrichtung,
- Anbieten von Auswegen und Alternativen bei verfahrenen Besprechungen,
- Ausüben der Moderatorfunktion; insbesondere sprechen, wenn keiner etwas sagt, bzw. eingreifen, wenn das Gespräch abschweift.

Entscheidungen in einer externen Projektbesprechung trifft nur der Projektleiter. (Größere) Entscheidungen oder Abschätzungen seitens des Auftragnehmers sollten nie in einer externen Besprechung direkt getroffen werden, sondern schriftlich oder per E-Mail nach der Nachbesprechung.

Aufgaben der *Nachbesprechung* sind:

- Aufbereiten des Besprechungsergebnisses,
- Durchgehen des Protokolls,
- Versenden des Protokolls an alle Besprechungsteilnehmer,
- Planen der Termine und Einteilen der Arbeit durch den Projektleiter.

Jede Besprechung ist durch einen Protokollführer zu protokollieren, der nicht mit dem Leiter der Besprechung identisch sein darf. Im Allgemeinen wird in der Projektentwicklung kein Volltextprotokoll (wie z. B. bei Gericht), sondern nur ein Stichwortprotokoll angefertigt.

Besprechungen werden von Entwicklern oftmals als unproduktiv und unangenehm gesehen. Bedingt durch die Beteiligung mehrerer bzw. aller Projektmitarbeiter sind sie auch relativ teuer. Besprechungen sollen daher wohlüberlegt angesetzt und die Teilnehmersauswahl gut überlegt werden. Selten ist nämlich die Teilnahme aller Teammitglieder an einer Besprechung erforderlich. Vor allem sollte aber bei Besprechungen keine Zeit mit Abschweifungen vergeudet werden. Eine geplante, vorher bekannt gegebene maximale Besprechungsdauer ist hier sehr hilfreich.

Tipp: Für jede Besprechung sollte nicht nur eine Beginnzeit, sondern auch eine (einzuhaltende!) Endzeit festgelegt und bereits in der Agenda vermerkt werden.

7.1.3 Review

Ein *Review* ist eine mehr oder weniger formale Überprüfung von Zwischenergebnissen oder Ergebnissen (z. B. Lastenheft, Pflichtenheft, Design, Code, Testsuite) einzelner Projektphasen [Jones, 1994]. Reviews werden in Gruppen durchgeführt und sind sehr breit einsetzbar. Beispiele für Anwendungen sind

- das Überprüfen, ob die Funktionalität¹ des Programms die Anforderungen aus der Analyse abdeckt,
- das Prüfen des Designs auf Verständlichkeit und Realisierbarkeit (Plausibilität),
- das Prüfen der Benutzerdokumentation auf Vollständigkeit und Übereinstimmung mit der aktuellen Programmversion,
- das Analysieren von entwickeltem Code,

¹Von einem Programm wird *Funktionalität* verlangt, unabhängig davon, welche *Funktionen* das Programm umfasst.

- das Testen der Installierbarkeit und möglichen Inbetriebnahme des Programms in der Zielumgebung.

Vorteile von Reviews sind

- die Anwendbarkeit in jeder Projektphase, insbesondere auch auf halbfertige Programme oder nur Analyse- oder Designergebnisse,
- genaue Information des Autors durch dessen Teilnahme am Review, aber zusätzliche Sichtweisen durch die weiteren Teilnehmer,
- die Erklärung des diskutierten Ergebnisses für andere Personen als den Autor; das Know-how der Projektgruppe steigt, Vertretungen sind leichter möglich.

Die Teamzusammensetzung für Reviews änderte sich Hand in Hand mit der veränderten Zusammensetzung und den geänderten Anforderungen an moderne Projektgruppen, vgl. Abbildung 7.1:

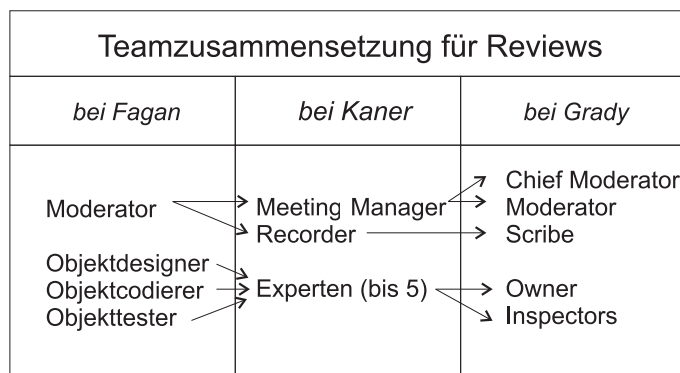


Abbildung 7.1: Entwicklung der Zusammensetzung von Review-Teams

1. In [Fagan, 1976] wird ein Team von vier Personen vorgeschlagen, bestehend aus einem Moderator (einem erfahrenen, nicht am Projekt beteiligten Softwareingenieur), dem Designer, dem Implementierer sowie dem Tester des zu analysierenden Objekts.
2. Für [Kaner *et al.*, 1999] wird ein Reviewtreffen von einem Meeting Manager („facilitator“) und einem Schreiber („recorder“) organisiert. Keiner dieser Personen kommentiert den Reviewinhalt. Der Meeting Manager ist zuständig für die Organisation des Treffens, d. h., Festlegen und Organisieren des Raumes, Steuern des Redeflusses, Abwehr von Unterbrechungen und Abschweifungen und Vorbereiten des zusammenfassenden Berichts. Speziell muss er die Gruppe davon abhalten, mögliche Lösungen für die erkannten Probleme zu diskutieren.

Der Schreiber notiert alle wesentlichen Kommentare so, dass sie vom ganzen Team durchgehend zu sehen sind (z. B. auf Flip Charts). Wichtig ist, dass alle Entscheidungen festgehalten werden. Wird eine Aufgabe für eine spätere Diskussion offen gelassen, so ist diese explizit auf der Liste der verschobenen Aufgaben zu notieren.

Die eigentliche Reviewarbeit leisten bei [Kaner *et al.*, 1999] bis zu fünf Softwareingenieure, wobei einer von denen der Ersteller des zu analysierenden Objekts ist. Diese Experten sind bei der Diskussion gleichberechtigt.

3. Bei [Grady, 1992] wird neben dem Manager (hier „moderator“), dem Schreiber (hier „scribe“), dem Autor („owner“) und den Experten („inspectors“) ein „chief moderator“ eingeführt, der bei allen Reviews (auch projektübergreifend) anwesend ist und über die Reviews Berichte und Statistiken anfertigt. Dieser Chefmoderator ist für die Verbesserung des Reviewprozesses zuständig und überwacht die Aktualisierung von Review-Checklisten und Standards.

Wie [Elzer, 1994] anmerkt, ist für den Erfolg von Reviews die Akzeptanz bei den Mitarbeitern ganz wesentlich. Es ist nicht damit getan, Reviews einfach anzuordnen und formal Ergebnisse festzulegen bzw. einzufordern. Besonders zu Anfang des Einsatzes von Reviews müssen ihre Durchführung und ihr Erfolg sorgfältig beobachtet werden, um eventuell notwendige personelle Konsequenzen ziehen zu können.

Termine für Reviews müssen so festgelegt werden, dass das Budget und der Zeitrahmen für den jeweiligen Abschnitt noch nicht aufgebraucht sind. Im Fall von Termin- oder Kostendruck kann auf die Reviewer allzu leicht psychologischer Druck dahingehend ausgeübt werden, möglichst wenige Fehler zu finden, da „sowieso kein Geld mehr vorhanden sei“ oder ein Meilenstein gefährdet werden könnte.

7.1.4 Werkzeugeinsatz

Wie in anderen Projekten zur Herstellung eines technischen Produkts benötigt man auch für die Herstellung eines Softwareprodukts *Werkzeuge* zur Unterstützung des Herstellungsprozesses. Nach [Rechenberg, 1985] verstehen wir unter Werkzeugen Programme, welche die Herstellung, Prüfung, Wartung und Dokumentation von Programmen vereinfachen, beschleunigen oder in ihrer Qualität verbessern.

Eine integrierte Sammlung solcher Werkzeuge wird als *Softwareentwicklungsumgebung* oder *Computer Aided Software Engineering-System (CASE-System)* bezeichnet. CASE-Systeme sollen jedoch koordiniert, d. h., in eine Entwicklungsstrategie eingebettet, eingesetzt werden.

Eine gute Softwareentwicklungsumgebung integriert einzelne (anpassbare und austauschbare) Werkzeuge derart, dass sie nicht nur einzeln eingesetzt werden können, sondern auf einer gemeinsamen (logischen) Datenbasis operieren und sich den Entwicklern als geschlossenes Entwicklungssystem präsentieren. Ein solches Entwicklungssystem umfasst:

- Planungswerkzeuge („Projektmanagement“-Werkzeuge),
- Prototypingwerkzeuge,
- Analysewerkzeuge und Designwerkzeuge (typischerweise UML-basiert), die eine automatische Codegenerierung ermöglichen sowie aus Code wieder Designmodelle extrahieren können (sog. *Roundtrip Engineering*),
- Implementierungswerkzeuge wie Editoren, Compiler, Interpreter, Debugger, Browser,
- Werkzeuge zur Testfallgenerierung, -durchführung und Testautomatisierung,
- ein Werkzeug zur Konfigurations- und Versionsverwaltung (vgl. Abschnitt 7.2.3),
- Dokumentationswerkzeuge und Werkzeuge zur Dokumentationsverwaltung.

Wichtig dabei ist, dass die Werkzeuge einfach zu bedienen und effizient sind, eine möglichst einheitliche Benutzerschnittstelle aufweisen, dass Daten und Ergebnisse zwischen verschiedenen Werkzeugen ausgetauscht werden können und die Systemumgebung auf einer modernen Hardware und einem leistungsfähigen Betriebssystem aufsetzt. Ein gutes Zusammenspiel dieser Werkzeuge (möglichst eine Integration) und einfache Erweiterungsmöglichkeiten sind für eine Effizienzsteigerung bei der Entwicklung unabdinglich (vgl. den Erfolg des auf dieser Idee basierenden *Eclipse*-Projekts, <http://www.eclipse.org>). Dabei ist aber zu beachten, dass der Entwicklungsprozess von den verwendeten Werkzeugen und Technologien unabhängig sein muss [McDonald und Welland, 2001], um Änderungen und Optimierungen unabhängig voneinander zuzulassen.

Folgende Vorgehensweise wird für eine sinnvolle Werkzeugnutzung empfohlen [Mayr, 2003]:

1. Trenne den Entwicklungsprozess klar von den Werkzeugen, Modellen und Sprachen.
2. Achte auf durchgehend nutzbare, integrierbare Werkzeuge.
3. Beginne frühzeitig mit dem Werkzeugeinsatz. Ein späteres Nachziehen der Modelle ist mühsam und unbefriedigend.
4. Verwende Werkzeuge und Prozesse, die das iterativ-inkrementelle Vorgehen sowie die Kundenrückmeldungen effizient unterstützen.
5. Plane Schulungen und Einarbeitungszeit für jedes Werkzeug ein.
6. Prüfe vor jedem Werkzeug- oder Releasewechsel dessen Notwendigkeit und Konsequenzen.
7. Miss nicht nur den Projektfortschritt, sondern auch den Grad der Zielerreichung.

Obwohl moderne, agile Softwareentwicklungsmethoden sich selbst gerne als „schlank“ oder gar „minimalistisch“ bezeichnen (vgl. z. B. [Beck und Andres, 2005]), setzen sie doch auf eine Anzahl von Werkzeugen, die für Anforderungsanalyse, Konfigurations- und Versionsverwaltung, Testplanung und -durchführung, Fehlerverfolgung etc. eingesetzt werden. Obwohl alle diese Werkzeuge auch in der traditionellen Softwareentwicklung angebracht wären, ist deren Beherrschung und aktiver Einsatz gerade bei agilem Vorgehen unerlässlich [Mayr, 2003]. In [Cutter, 2000] wird dazu ausgeführt, dass „... ohne Werkzeuge zur Fortschrittsmessung, Änderungsmanagement und Ergebnisüberprüfung ... sowie ... sorgfältiger Überwachung der Anforderungen sowohl durch den Entwickler als auch den Auftraggeber Iterationen zu Ad-hoc-Entwicklungen werden können“ [Cutter, 2000]. Krücken auf Basis von Standardwerkzeugen (z. B. eine Excel-Liste der getätigten Änderungen anstelle eines Versionsverwaltungswerkzeugs) führen gerade bei den häufigen Änderungen iterativ-inkrementeller Entwicklungen rasch zum Scheitern des Projekts (vgl. [Stephens und Rosenberg, 2003]).

Oft wird beim Umstieg auf agile Vorgehensweisen aber übersehen, dass diese Werkzeuge nicht nur verfügbar sein müssen; der Umgang mit ihnen muss auch erlernt und geübt werden. Vielfach ist diese Einarbeitungszeit im Rahmen der eher kurzen Iterationszyklen bzw. Gesamtprojektlaufzeiten nicht vorgesehen bzw. kann kostenmäßig im Rahmen der ersten Projekte gar nicht gerechtfertigt werden. Das individuelle „Selbsterlernen“ agiler Methoden und der benötigten Werkzeuge durch die meist eher unerfahrenen Entwickler führt dann erst recht dazu, dass alles subjektiv Unwichtige weggelassen wird und das Hackertum Einzug hält [Beck und McBreen, 2002].

7.2 Dokumentation von Softwareprojekten

Die *Dokumentation* eines Softwareprojekts besteht aus der (organisatorischen) *Projektdokumentation*, die den Ablauf eines Projekts dokumentiert, und der (technischen) *Produktdokumentation*, die das Projektergebnis dokumentiert. Für die einzelnen Dokumente bestimmter Projektphasen wird auf die nachfolgenden Kapitel verwiesen, im Folgenden werden grundsätzliche Vorgehensweisen und die grundlegenden Dokumente der Projektorganisation näher besprochen.

Die Dokumentation dient dazu, während der Entwicklungsphasen die Kommunikation zwischen den an der Entwicklung beteiligten Personen zu ermöglichen und nach Abschluss der Entwicklungsarbeit den Einsatz und die Wartung von Softwareprodukten zu unterstützen. Sie soll außerdem den Projektverlauf zum Zweck der Kalkulation des tatsächlichen Herstellungsaufwands und zur besseren Planung zukünftiger Projekte dokumentieren.

7.2.1 Dokumentationsplanung

Aufgabe der *Dokumentationsplanung* ist die Erstellung von Standards, Strukturen und Vorlagen für die Dokumentation.

Ziele einer einheitlichen Dokumentation sind:

- einheitliche Struktur der Dokumente,
- rasche, einfache Erstellung neuer Dokumente,
- parallele Erstellung bzw. Zusammenführung von Dokumenten,
- Möglichkeiten zur Präsentation des Projektstandes,
- Möglichkeiten zur Projektkontrolle.

Folgende Entscheidungen sind für jedes Dokument zu treffen:

- Wann ist es zu erstellen?
- Wer erstellt es?
- Wer ist der Empfängerkreis?
- Wie oft ist ein Update notwendig? (Terminkontrolle z. B. via Projektmanagementwerkzeug, Terminkalender oder händisch)
- Ist eine Versionsverwaltung notwendig? (Versionsverwaltung per Hand, CASE-unterstützt)
- Welches Textformat ist am geeignetsten? (für Datenextraktion, Zusammenführung, Suche etc.)

Eine weitere wesentliche Aufgabe der Dokumentationsplanung ist die Planung des projektbezogenen *Berichtswesens*. Dabei steht der Projektverlauf im Mittelpunkt. Das Berichtswesen soll gewährleisten, dass die jeweiligen Informationen, die für die Durchführung des Projekts notwendig sind, richtig, rasch und vollständig zur Verfügung stehen. Soweit im Unternehmen allgemeine Richtlinien bestehen, ist zu prüfen, inwieweit diese im Projekt Anwendung finden.

Bei der Festlegung der Organisationsstruktur und der Zuständigkeiten ist bereits festzustellen, wer von wem zu welchem Zeitpunkt welche Informationen zu erhalten hat. Es müssen also die verschiedenen Berichtsarten, der Zeitraum und der Verteilerschlüssel bestimmt werden. Damit wird gleichzeitig ein großer Teil der formalen Kommunikationsbeziehungen im Projekt festgelegt.

Ein Überblick über essenzielle Dokumente bei traditionellem und agilem Vorgehen (Abbildung 7.2) zeigt, dass die Anzahl der bei agilem Vorgehen produzierten Dokumente nicht wesentlich geringer ist als bei traditionellem Vorgehen [Mayr, 2003]. Aus Sicht der verwendeten Werkzeuge sowie der erstellten (Zwischen-)Ergebnisse ist jedoch eine deutliche Transition vom *Dokumenten-getriebenen Vorgehen* der traditionellen Softwareentwicklung hin zu hochgradig *Werkzeug-unterstütztem Vorgehen* bei agilen Entwicklungen zu erkennen [Orr, 2002]. Durch die gestiegene Verwendung von Werkzeugen bei agilem Vorgehen werden viele Dokumente automatisch generiert oder konvertiert, und es wird viel weniger Zeit benötigt, um sie zu erstellen bzw. auf dem aktuellen Stand zu halten, als bei traditionellen Methoden [Mayr, 2004b].

Wichtiger bei traditionellem Vorgehen	Gleich wichtig	Wichtiger bei agilem Vorgehen
	Organigramm, Stellenbeschreibungen	
	Projektbibliothek, Projekttagbuch, Protokolle	
Fortschrittsberichte	Zwischenprodukte, Prototypen (Protokolle)	
	Konfigurationsmanagement (Werkzeug!)	
	Qualitätsmerkmale	
	Zielbeschreibung	
Lastenheft	Auftrag, Anforderungssuite (Werkzeug!)	
Gesamtplan	Grobplan, kurzfristiger Feinplan (Werkzeug!)	
	Risiko-Vorsorgeplan	
	Benutzerdokumentation	Zwischenprodukte, Prototypen
Pflichtenheft	Designmodell (Werkzeug!), durchg. Modellsprache	
	Quellcode	
	Systemdokumentation	(Anwendungs-)Daten
	Lauffähiges Programm	
		Testplan, Testsuite
Fehlerliste, Fehlerbericht	Fehlerbehandlung (Werkzeug!)	
	Fehlerlogbuch	
	Betriebsversion	
	Installations-, Inbetriebnahmeprotokoll	
	Abnahmeprotokoll	
Abschlussbericht	Wartungs-, Pflegeplan	
	Projektarchiv	

Abbildung 7.2: Essenzielle Dokumente bei traditionellem und bei agilem Vorgehen

7.2.2 Dokumentationsorganisation

Es ist wichtig, dass sämtliche Dokumente, der Dokumentstatus und der Verteiler für die erstellten Dokumente systematisch festgelegt werden. Die Hauptaufgaben der *Dokumentationsorganisation* lassen sich folgendermaßen zusammenfassen:

- Identifizieren der Dokumentarten,
- Festlegen der Dokumentationsanforderungen und Standards,
- Erstellen eines Dokumentablagensystems,
- Freigeben und Verteilen der Dokumente,
- Überwachen des Dokumentstatus,
- Organisieren und Führen des Archivs.

Durch die Wahrnehmung dieser Tätigkeiten ist sicherzustellen, dass die richtigen Dokumente rechtzeitig an die richtigen Empfänger weitergeleitet werden.

Für eine wirkungsvolle Projektdokumentation sind folgende Prinzipien zu beachten:

- **Führen mitlaufender Dokumentation:** Die Dokumentation muss ständig aktualisiert werden und bereits während der Arbeitsabläufe erfolgen; lediglich Ergänzungen oder Erläuterungen werden nachträglich vorgenommen.
- **Vermeiden von Redundanz:** Um ein mehrfaches Erfassen gleicher Daten, Beschreibungen etc. zu vermeiden, sind Querverweise oder automatische Dokumentationssysteme gemeinsam mit einer zentralen Projektbibliothek zu verwenden.
- **Verwenden einheitlicher Bezeichnungen:** Für Fachbegriffe, speziell für projektspezifische Termini, muss eine einheitliche Sprachregelung festgeschrieben werden, vgl. dazu Abschnitt 9.2.5.2. Dadurch können auch gezielt Dokumente gesucht oder beschlagwortet werden. Allgemein ist auf exaktes korrektes Schreiben zu achten (Verwenden von Rechtschreibhilfen!).
- **Systematisieren der Ablage:** Privatablagen neben der offiziellen Dokumentation sind zu vermeiden. Die Ordnung der Ablage muss nach einem einheitlichen und eindeutigen Schema erfolgen. Nicht eindeutig zuordenbare Dokumente sind in zeitlicher Folge abzulegen.

7.2.3 Konfigurations- und Versionsverwaltung

Wenn mehrere Personen² an einem Projekt arbeiten, kann die Integration der einzelnen Subkomponenten (Dokumente, aber auch Programme) in ein Gesamtsystem zu einem wesentlichen organisatorischen Problem werden. Es müssen nämlich *Konfigurationen* des Systems

²**Tipp:** Auch Einzelentwicklern wird dringend geraten, ein System zur Konfigurations- und Versionsverwaltung einzusetzen. Im Laufe der Zeit können auch Einzelpersonen eine erstaunliche Zahl an Varianten und Revisionen von Dokumenten bzw. Dateien eines Projekts erstellen. Der Umstieg auf ein Konfigurationsverwaltungssystem ist dann bereits sehr aufwändig.

erstellt und verwaltet werden. Da dies per Hand rasch unübersichtlich wird, wurden bereits frühzeitig Softwaresysteme zur automatischen Konfigurations- und Versionsverwaltung entwickelt (vgl. z. B. [Tichy, 1988]).

Die Aufgaben der computerunterstützten *Konfigurations- und Versionsverwaltung* (*configuration & versioning management – CVM*) sind daher:

- Übersicht Halten über die Dokumentstruktur und Änderungsgeschichte („*Revisionen*“),
- Festhalten der Unterschiede zwischen Dokumenten verschiedener Konfigurationen („*Varianten*“),
- Bereitstellen aller oder bestimmter Dokumente einer (auch älteren) Konfiguration,
- Übersicht Halten über Änderungen, die zur Zeit in Arbeit sind,
- Auflisten aller von einer bestimmten Änderung betroffenen Dokumente,
- Regeln des Zugriffs auf gemeinsam genutzte Dateien (einschauen, ändern).

Diese Aufgaben werden geeigneterweise durch Werkzeuge unterstützt, welche die Versionierung und Konfiguration der Programme, der Testdaten und der Dokumentation übernehmen [Dart, 2000]. Solche *Konfigurations- und Versionsverwaltungssysteme* automatisieren Ablage, Wiederherstellung und Identifikation von Versionen und stellen einen Auswahlmechanismus zur Zusammenstellung von Konfigurationen verschiedener Module zu einem System zur Verfügung. Sie ermöglichen außerdem festzulegen, wer was wann ändern darf, sowie die Kontrolle über diese Änderungen und die Verfolgung von deren Geschichte.

Ideal für die Zugriffssicherheit und Integrität der Daten ist es, wenn Konfigurations- und Versionsverwaltungssysteme mit einem Datenbanksystem zusammenspielen, wie z. B. beim Open-Source-Produkt *Subversion* [Will, 2004]. In manchen Systemen wird jede Version als Volltext gespeichert, in anderen wird nur der Unterschied zur vorangehenden Version gespeichert („*Deltaspeicherung*“). Dem Platzvorteil bei dieser Alternative steht das größere Risiko eines Dokumentenverlusts gegenüber, da bereits ein fehlerhaft gespeichertes Delta das Dokument unbrauchbar machen kann.

Da viele Projekte mit kleinem Umfang beginnen und dann sukzessive wachsen, ist es entscheidend, dass Konfigurations- und Versionsverwaltungssysteme bereits von Beginn an eingesetzt werden (auch wenn das Projekt dann noch „zu klein“ für eine werkzeugunterstützte Konfigurationsverwaltung ist). Ein späterer Umstieg ist mit großem Aufwand und Zeitverzug verbunden, außerdem kann die Entwicklungsgeschichte vor dem Einsatz des Werkzeugs nicht mehr nachvollzogen werden. Gerade wenn ein Projekt auf viele Teilprojekte aufgeteilt wird, deren Ergebnisse für die einzelnen (Zwischen-)Produkte häufig zusammengeführt werden müssen, wird ein Konfigurations- und Versionsverwaltungssystem rasch zum unverzichtbaren Bestandteil der Entwicklungsumgebung [Baskerville und Levine, 2001].

7.2.4 Dokumentablage

Die *Dokumentablage* ermöglicht das sichere Ablegen von Dokumenten jeder Art (elektronisch und in Papierform) mit dem Zweck, dass die richtige Information am richtigen Ort

zur richtigen Zeit zur Verfügung steht. Um eine effiziente elektronische Dokumentablage zu ermöglichen, sind folgende grundlegende Werkzeuge notwendig:

- Werkzeuge zur Dateisicherung, Dateisuche und zum Dateivergleich,
- Werkzeuge zum raschen Auffinden gewünschter Dokumente durch effiziente Recherchemöglichkeiten (z. B. Volltextsuche) oder geeignete Beschlagwortung,
- Werkzeuge zur Dateikompression und -expansion sowie zur Formatkonvertierung,
- Werkzeuge zum Festlegen von Zugriffsrechten,
- Werkzeuge zur Konfigurations- und Versionsverwaltung (siehe voriges Kapitel).

Abhängig von den Berechtigungen des Suchenden wird ein Dokument zum Ändern mit einem dazu geeigneten Programm, nur zum Einsehen oder gar nicht zur Verfügung gestellt. Gleichzeitig ist es auch die Aufgabe des entsprechenden Werkzeugs, jeweils nur eine gültige Version eines Dokuments zu erlauben.

Eine Sammlung derartiger Werkzeuge wird oft unter der Bezeichnung *Dokumentenmanagement-System* bzw. *elektronisches Archivierungssystem* angeboten. Elektronische Archivierungssysteme umfassen zusätzlich die Langzeitspeicherung von Dokumenten auf dafür geeigneten Medien sowie Replikationsmechanismen zur lokalen oder weltweiten Verteilung von Dokumenten.

7.3 Dokumente der Aufbauorganisation

7.3.1 Organigramm

Ein bewährtes Mittel zur Darstellung der Rollen in der Aufbauorganisation ist das *Organigramm*. Es beschreibt grafisch die Gesamtstruktur eines Systems als Beziehungsgraph der einzelnen Funktionen. Diejenigen Funktionseinheiten im Organigramm, denen *Rollen* (Mitarbeiteraufgaben) zugewiesen werden, werden *Stellen* genannt. Ein Mitarbeiter kann gleichzeitig mehrere Rollen übernehmen und damit mehrere Stellen (teilweise) besetzen.

Ein Organigramm ist für jedes Projekt spezifisch zu erstellen. Das Organigramm in Abbildung 7.3 soll dazu eine im Allgemeinen brauchbare Schablone für die Gestaltung einer Projektgruppe im Software Engineering darstellen (zur Erklärung der Abkürzungen sei auf Abschnitt 7.3.2 verwiesen).

7.3.2 Stellenbeschreibung

Die *Stellenbeschreibung* ist die Beschreibung der Aufgaben, Kompetenzen und Verantwortlichkeiten des Inhabers einer Stelle einer Organisation mit dem Ziel der Schaffung klarer Zuständigkeiten. Darüber hinaus werden so die Anforderungen an den Stelleninhaber, die notwendige Qualifikation sowie eine Stellvertretung festgelegt.

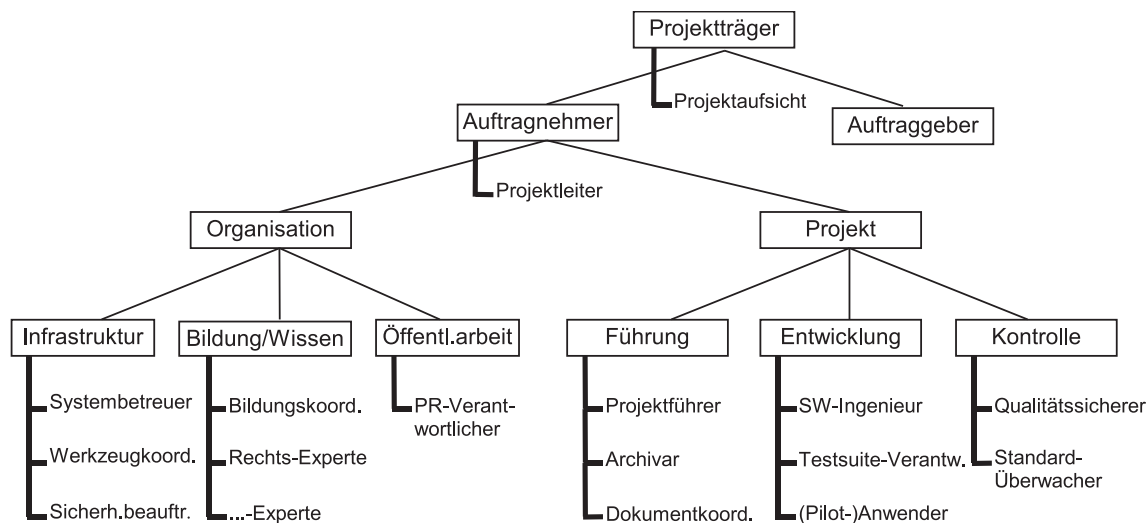


Abbildung 7.3: Beispielorganigramm einer Projektgruppe

Im Folgenden werden die in Abbildung 7.3 dargestellten Stellen genauer beschrieben, wobei die Beschreibung größtenteils nur beispielhaften Charakter hat und bei konkreten Projekten detaillierter ausfallen muss und auch durchaus im Laufe eines Projekts adaptiert werden kann und soll.

- **Projektaufsicht (Projektkoordinator):** Die Projektaufsicht repräsentiert den Projektträger, also jene Einheit, der die gesamte Projektorganisation unterstellt ist. In der Projektaufsicht sollten Auftraggeber und Auftragnehmer vertreten sein. Die Projektaufsicht sollte im Hintergrund bleiben, sich jedoch den Projektfortschritt durch regelmäßige Berichte dokumentieren lassen. Durch das eher seltene Auftreten wird der Projektaufsicht gewöhnlich viel zu wenig Bedeutung zugemessen.

Aufgaben der Projektaufsicht sind unter anderem die Formulierung des Projektauftrages mit Grobziel und Rahmenbedingungen, Festlegen von Projektpolitik, Projektbudget und Projektprioritäten, Koordination paralleler Projekte (Multi-Projektorganisation), Ernennung des Projektleiters, Regelung von Kompetenzen, Abbruchentscheidungen und Terminverlängerungen sowie die Rolle der Schiedsstelle bei Differenzen zwischen Auftraggeber und Auftragnehmer.

- **Projektleiter:** Der Projektleiter ist technischer, wirtschaftlicher und sozialer Leiter der Projektgruppe. In verschiedenen Ländern existieren dazu unterschiedliche Rollenverständnisse. So gibt [Gareis, 2004] an, dass die Rolle des Projektleiters in den USA autoritärer gesehen wird als in Österreich. Der Projektleiter ist in den USA der „doer“ und weniger der „facilitator“ (der, der macht, und nicht der, der's möglich macht). Außerdem werden in den USA Projektleitungs- und Projektsteuerungsaufgaben häufig an externe Konsulenten übertragen.

Für den technisch vorgebildeten Projektleiter ist es wohl am naheliegendsten, Projektleitung als regelungstechnische Aufgabe zu begreifen: Um regeln zu können, muss man sowohl die Sollwerte kennen, d. h., zuverlässig schätzen, als auch die Abweichungen von diesen Sollwerten durch angemessene Maßnahmen zur Projektkontrolle rechtzeitig und zuverlässig erkennen, um korrigierend eingreifen zu können.

Die Hauptaufgaben des Projektleiters sind daher das Führen der Projektgruppe, das Vorbereiten und Treffen von Projektentscheidungen sowie die Überprüfung von Führung (insbesondere Plausibilität der Planung), Entwicklung und Kontrolle.

- **Systembetreuer:** Hardware, Systemsoftware, Releasepolitik (Updates).
- **Werkzeugkoordinator:** CASE-Auswahl, Installation, Übersicht über Werkzeuge.
- **Sicherheitsbeauftragter:** Datenschutz, Zugangsschutz, Datensicherung, Erarbeitung eines Notfallplans und eines Katastrophenplans.
- **Bildungskoordinator:** Information über (Ausbildungs- und) Weiterbildungsmöglichkeiten, Weiterbildungsplan, Organisation entsprechender Veranstaltungen.
- **Experte:** Einbringung von projektnotwendigem, externem Expertenwissen (z. B. aus dem juristischen Bereich).
- **PR-Verantwortlicher:** Erstellen einer Corporate Identity, Vorgaben für die Dokumentgestaltung, Unterstützung der Präsentation und Dokumentation.
- **Projektführer (administrativer Projektleiter, stellvertretender Projektleiter, Projektmanager):** Koordination, stellvertretende Entscheidungen für den Projektleiter, Kontakt zu Auftraggeber.
- **Archivar (Klassenmanager, librarian):** Anregungen und Maßnahmen zur Steigerung der Wiederverwendbarkeit von Ergebnissen und Know-how, Klassenmanagement bei objektorientierter Vorgehensweise.
- **Dokumentkoordinator:** Korrespondenzsammlung, Dokumentensammlung, Verwaltung der Projektbibliothek, Protokollführung.
- **Softwareingenieur:** Analyse, Design, Implementierung, Testen.
- **Testsuite-Verantwortlicher:** Hilfe bei der Testsuite-Erstellung, Abnahme der Testsuite.
- **(Pilot-)Anwender:** Testen aus Anwendersicht, direkte Rückmeldungen, Abklärung der Ergonomie und allgemein der Produktqualität aus Anwendersicht. Bei iterativ-inkrementeller Entwicklung und der damit verbundenen engen Kundeneinbindung wird die Rolle des Pilotanwenders stark aufgewertet: Er vertritt den Kunden in dessen Abwesenheit („on-site customer“ [Beck und Andres, 2005] bzw. „virtual customer“ [Schwaber und Beedle, 2001]) und treibt durch von ihm vorab erstellte – und laufend überprüfte – Systemtests die Implementierung voran.
- **Qualitätssicherer (Qualitätsmanager):** Qualitätssicherung (siehe Kapitel 8).
- **Standard-Überwacher:** Überwachung der Einhaltung vorgegebener Standards, Koordination der Aufstellung projektspezifischer und projektinterner Standards, Überwachung der Portierbarkeit bzw. des Portierens (porting maintainer).

7.4 Dokumente der Ablauforganisation

7.4.1 Grundlegende Dokumente

Ziel der Vorgabe von *Dokumentstrukturen* ist die einheitliche Gliederung des Inhalts der Dokumente und damit die raschere Informationsbeschaffung. In Abbildung 7.4 sind die grundlegenden Dokumente und die darin enthaltenen Attribute angeführt. Die wichtigsten dieser Dokumente werden nachfolgend kurz charakterisiert.

Dokumentart							
Attribut	Brief	Fax	Aktennotiz	Telefon-/ Gespr.notiz	Agenda	Protokoll	Bericht
Erstellungs-/Änderungsdatum	✓	✓	✓	✓	✓	✓	✓
Revisionsnummer					✓	✓	(✓)
Autor/Protokollführer	✓	✓	✓	✓	✓	✓	✓
Kontaktadresse/Absender/Institution	✓	✓	✓	✓	✓	✓	✓
Telefonnummer Absender	✓	✓			✓	✓	(✓)
Faxnummer Absender	(✓)	✓					(✓)
E-Mail-Adresse Absender	(✓)	(✓)					(✓)
Projekt/Betreff	✓	✓	✓	✓	✓	✓	✓
Anlass (inkl. Beginn, Ende)	✓	✓	✓	✓	✓	✓	✓
Datum des Anlasses	(✓)	(✓)	(✓)		✓	✓	
Teilnehmer					✓	✓	
Tagesordnung					✓	✓	
Verteiler/Gesprächspartner/Adressat	✓	✓	(✓)	(✓)	✓	✓	(✓)
Ablage (Papierform)	✓	✓	(✓)	(✓)	(✓)	(✓)	(✓)
Ablage (elektronisch)	(✓)	(✓)	✓	✓	✓	✓	✓

Abbildung 7.4: Checkliste für die grundlegenden Dokumente

Aktennotiz

Ziel einer *Aktennotiz* ist die Wiedergabe des persönlichen Eindrucks eines Sachverhalts („subjektiv“) – im Allgemeinen ausgelöst durch gezogene Schlussfolgerungen bzw. Erkenntnisse.

Eine Aktennotiz dient

- der Dokumentation eines Sachverhalts,
- (ggf.) der Vorbereitung von Entscheidungen sowie
- (ggf.) der Veranlassung von Aktivitäten.

Telefonnotiz, Gesprächsnotiz

Zweck einer *Telefonnotiz* bzw. einer *Gesprächsnotiz* ist die Wiedergabe des Sachverhalts *ohne* eigene Schlussfolgerung bzw. Erkenntnis (also „objektiv“). Eine Telefonnotiz/Gesprächsnotiz entspricht einem eingelangten Brief.

Eine Telefonnotiz/Gesprächsnotiz dient

- der Nachvollziehung,
- der Nachbesprechung und
- der Veranlassung von Aktivitäten.

Agenda

Eine *Agenda* ist die schriftliche Planung einer Besprechung. Sie dient der Vorbereitung und Strukturierung einer Besprechung. Das Erstellen der Agenda ist Aufgabe des Besprechungsleiters. Sie ist zeitlich so zu erstellen, dass sie den Besprechungsteilnehmern rechtzeitig vor der Besprechung übermittelt werden kann.

Protokoll

Ein *Protokoll* ist eine rechtsgültige Projektunterlage (ab dem Zeitpunkt der Genehmigung durch Unterschrift). Es ergänzt bzw. ersetzt also Abmachungen z. B. aus dem Pflichtenheft (im Allgemeinen aber nicht Abmachungen aus Angebot/Auftrag auf Grund rechtlicher Zuständigkeiten).

Das Ziel des Protokolls ist daher die chronologische Dokumentation eines Ereignisses (z. B. Besprechung) in knapper, aber vollständiger (heißt nicht ausführlicher!) Darstellung. Der Inhalt hat sich daher zu konzentrieren auf

- Erkenntnisse,
- Entscheidungen (positive wie negative!),
- Aktivitäten (Zuständiger, Beteiligte, Aufgabe, Termin, Priorität).

Bei einer Besprechung ist es aber nicht nur die Aufgabe des Protokollführers mitzuschreiben. *Jeder* Mitarbeiter muss auch für sich selbst die ihn betreffenden Aktivitäten notieren. Allgemein ergibt sich durch das Mitschreiben auch ein psychologischer Vorteil, da das Gespräch automatisch bedachter erfolgt.

Bericht

Projektberichte legen formale Kommunikationsbeziehungen fest und dienen der Darstellung eines Sachverhalts *für andere*. Man unterscheidet Berichte nach dem Berichtsfluss (Verteilerlisten), den Berichtsinhalten sowie der Berichtshäufigkeit.

Häufig ist es sinnvoll, zwischen einem Bericht im engeren Sinn und den Beilagen zum Bericht zu unterscheiden. Details, die zur Erläuterung und Untermauerung der Berichtsaussagen dienen oder die erforderlich sind, um Berichtsaussagen nachvollziehen zu können, werden in einem Anhang (als Beilagen) dargestellt.

7.4.2 Projekthandbuch

Im *Projekthandbuch* (auch *Organisationshandbuch*, *Vorgehenshandbuch*) wird die Projektorganisation für alle Beteiligten verbindlich festgelegt. Dargestellt werden auch die Ziele und Arbeitsweisen im Projekt, und es werden den beteiligten Stellen Aufgaben, Leistungen, Befugnisse und Verantwortung zugewiesen. Das Projekthandbuch bildet die Grundlage zur Erteilung detaillierter Aufträge im Rahmen des Projekts, zur Ausarbeitung notwendiger Verträge, und es dient während der Projektbearbeitung als verbindliche Richtlinie für die Zusammenarbeit [Zäpfel, 2000a].

Inhalte des Projekthandbuches sind:

- Organigramm,
- Stellenbeschreibungen,
- Stellenzuordnungen,
- Projektleitung und Berichtswesen,
- Sitzungskonzept,
- Dokumentationsvorgaben (für jede Phase),
- Projektstandards,
- Führungsgrundsätze,
- Projektplanung,
- Finanzierung und Beschaffung,
- Qualitätssicherung,
- Vorgehensmethode,
- verwendete Methoden und Werkzeuge.

Die Inhalte des Projekthandbuches hängen im Einzelfall stark vom konkreten Projekteinhalt ab und können vielfach nur in Form einer *Checkliste* dargestellt werden. Als zweckmäßig hat sich herausgestellt, wenn in einer Beilage zum Projekthandbuch Muster aller Dokumente angefügt werden, die in den einzelnen Projektphasen auszuarbeiten sind. Auch die Berichtsinhalte sollten zumindest im Entwurf oder als Beispiel vorgegeben werden.

Das Projekthandbuch gilt grundsätzlich für die gesamte Dauer eines Projekts. Es kann im Laufe der Projektbearbeitung vor einer neuen Phase oder Iteration revidiert und an geänderte Erfordernisse angepasst werden. Vor dem Beginn jeder Phase/Iteration sollten folgende Aktivitäten durchgeführt werden:

- Beurteilen der Organisation der vergangenen Phase/Iteration,
- Abschätzen der wichtigsten Probleme der kommenden Phase/Iteration,
- Erarbeiten einer zweckmäßigen Organisation für die kommende Phase/Iteration.

Das Projekthandbuch wird schrittweise erstellt und sukzessive weiterentwickelt. Wie alle anderen Organisationsanweisungen sollten auch die im Projekthandbuch enthaltenen Regelungen durch die Projektaufsicht genehmigt werden. Dies ist deshalb besonders wichtig, weil bei Meinungsverschiedenheiten im Projektverlauf oder bei Kompetenzproblemen auf die Regelungen im Projekthandbuch zurückgegriffen werden muss.

Ein Projekthandbuch kann seinen Zweck nur erfüllen, wenn es stets auf dem aktuellen Stand gehalten wird. Die Verantwortung für die Erstellung und Aktualisierung liegt bei der Projektleitung bzw. der innerhalb des Projekts mit der Aktualisierung beauftragten Stelle.

7.4.3 Projekttagebuch

Ziel des *Projekttagebuchs* ist die chronologische Aufzeichnung aller Projektstätigkeiten (entwicklerintern!). Das Projekttagebuch gibt Aufschluss über den Projektverlauf, hilft dem Projektleiter bei der Projektfortschrittskontrolle und enthält wertvolle Informationen über Entscheidungen, die länger zurückliegen und deren Begründung die Projektmitarbeiter bereits vergessen haben. Ein Eintrag im Projekttagebuch enthält deshalb,

- Datum,
- Aktivität,
- Beteiligte,
- spezielle Ressourcen,
- Aufwand.

Das Projekttagebuch ist von allen Mitarbeitern gemeinsam zu führen und jeden Arbeitstag zu aktualisieren.

7.4.4 Projektfortschrittsbericht

Der *Projektfortschrittsbericht* (*Leistungsbericht*) stellt den Zusammenhang zwischen den Projektaktivitäten und der Projektplanung her. Er ist daher ein reiner Terminbericht, der einen raschen Soll-Ist-Vergleich ermöglichen soll. Der Projektfortschrittsbericht enthält deshalb,

- welche Aufgaben termingerecht beendet wurden,
- welche Meilensteine erreicht wurden,
- welche Arbeiten vorgezogen wurden und warum,

- welche Arbeiten verschoben wurden und warum sowie
- eine Aktualisierung der Projektplanung (v. a. Termine).

Ein Projektfortschrittsbericht ist regelmäßig (ca. alle zwei bis vier Wochen) vom Projektleiter zu erstellen, jedenfalls aber bei Erreichen eines Meilensteins oder eines anderen markanten Punkts des schrittweisen Vorgehens. Der Bericht ergeht an den Projektträger (Projektaufsicht) sowie ggf. an die Qualitätssicherung, falls diese projektextern (unternehmenszentral) angesiedelt ist.

7.4.5 Arbeitsbericht

Ein *Arbeitsbericht* ist die Tätigkeitsdokumentation einer abgeschlossenen Aufgabe eines Mitarbeiters (bzw. einer von ihm übernommenen Rolle) oder einer Gruppe. Ziele sind unter anderem

- Übergabe der Tätigkeit an andere,
- Feststellung des Arbeitsabschlusses,
- Erreichen eines Zahlungszieles.

Arbeitsberichte werden z. B. erstellt von

- externen Mitarbeitern,
- temporären Mitarbeitern,
- beigezogenen Experten.

Ein spezieller Arbeitsbericht ist der *Abschlussbericht* eines Projekts (vgl. dazu Abschnitt 14.5.4).

7.4.6 Projektbibliothek

Die *Projektbibliothek* ist ein Verzeichnis aller zum Projekt gehörenden Dokumente. Dieses Verzeichnis enthält für jedes Dokument Informationen über

- den Entwicklungsstand (geplant, in Arbeit, freigegeben, ...),
- den Autor,
- die aktuelle Version,
- die Zugriffsrechte sowie
- seine Archivierung.

Die strukturierende Wirkung der Projektbibliothek fördert methodisches Arbeiten, z. B. durch die Übergabe definierter Zwischenergebnisse an die Bibliothek. Damit wird die Kommunikation zwischen den Projektmitgliedern gefördert, Dokumente können leichter wieder verwendet werden, und unterschiedliche Projekte können verglichen werden.

Für das Führen der Projektbibliothek ist eine eigene Stelle vorzusehen (*Dokumentkoordinator*, siehe Abschnitt 7.3.2).

8 Qualitätssicherung

Nach der Definition von Softwarequalität und Anmerkungen zu deren Messbarkeit werden in diesem Kapitel konkrete Maßnahmen zu Qualitätsplanung, -prüfung und -lenkung vorgestellt. Techniken und Modelle, wie etwa CMMI, ISO 90003 oder ISO/IEC 15504, werden in ihrem Kontext der Softwareentwicklung präsentiert. Abschließend wird auf die Bedeutung von Normen und Richtlinien eingegangen.

8.1 Ziel

8.1.1 Qualität von Software

Qualität in der Projektentwicklung ist gegeben, wenn gestellte Anforderungen erfüllt werden. Der Begriff Qualität wird jedoch sehr häufig verwechselt mit *Spitzenqualität*. Das Ziel eines Projekts ist es nicht, bestmögliche Qualität zu erreichen, sondern die geplante Qualität [Goldberg und Rubin, 1995]. Unrealistische Versprechungen oder falsche Erwartungen führen zu Unzufriedenheit. Dies ist dann Nichterfüllung von Anforderungen, also keine Qualität.

Die Qualitätsanforderungen an Softwareprodukte fanden ihren Niederschlag in der Definition von Qualitätsmerkmalen. Dabei stößt man aber in der Praxis vielfach auf ungelöste Probleme, die vor allem auf zwei Bereiche zurückzuführen sind: Es fehlt eine allgemein verbreitete und anerkannte Definition der Begriffe *Qualität* und *Qualitätsmerkmal*, und es ist eine eindeutige, objektive Qualitätsbewertung eines Softwareprodukts nicht möglich. Qualität muss aber notwendigerweise ein Ziel der Softwareentwicklung sein und wirkt sich sowohl auf die Planung als auch auf die Durchführung und Überprüfung der Softwareentwicklung aus.

8.1.2 Qualitätsmerkmale von Software

Über die *Qualitätsmerkmale von Software* ist sich die Fachwelt einigermaßen einig, vgl. z. B. [Wallmüller, 2001]. Qualitätsmerkmale von Software so wie die von Maschinen zu messen, ist jedoch bis heute kaum möglich – es fehlen die entsprechenden Methoden und Maße. Im Folgenden diskutieren wir die wichtigsten und auch in der Praxis anerkannten Qualitätsmerkmale von Softwareprodukten gemäß ISO 9126 ([ISO, 1991], adaptiert).

8.1.2.1 Korrektheit

Unter der *Korrektheit* eines Softwaresystems verstehen wir die Eigenschaft, dass das Softwaresystem die der Programmentwicklung zu Grunde liegende Spezifikation erfüllt. Die Korrektheit eines Softwaresystems bezieht sich also auf die Übereinstimmung zwischen Spezifikation und Programmtext und ist daher unabhängig von der tatsächlichen Verwendung des Softwaresystems.

Besonders kritisch ist die Korrektheit eines Programms, das in ein komplexes Softwaresystem eingebettet werden soll. Da sich die Wahrscheinlichkeit der Korrektheit eines Softwaresystems durch die Multiplikation der Einzelwahrscheinlichkeiten ergibt, bewirkt auch eine hohe Einzelwahrscheinlichkeit des Programms von fast 1 sehr rasch eine Gesamtwahrscheinlichkeit des Softwaresystems, die sich nur unwesentlich von 0 unterscheidet! [Kopetz, 1976]

8.1.2.2 Zuverlässigkeit

Die *Zuverlässigkeit* eines Programms wird durch die Korrektheit und durch die Verfügbarkeit bestimmt. Die Korrektheit eines Softwaresystems haben wir im vorigen Kapitel ohne jede Aussage über das Zeitintervall, in dem das Softwaresystem eine vorgegebene Spezifikation erfüllt, definiert. Das zeitliche Verhalten der Erfüllung einer vorgegebenen Spezifikation hängt von der Zuverlässigkeit des Softwaresystems ab.

Die Zuverlässigkeit eines Softwaresystems kann daher definiert werden als die Wahrscheinlichkeit, dass dieses System während eines vorgegebenen Zeitraums eine (durch die Spezifikation festgelegte) Funktion für eine vorgegebene Anzahl von Eingabefällen unter festliegenden Eingabebedingungen erfüllt (vorausgesetzt, Hardware und Eingabe sind fehlerfrei). Ein Softwaresystem kann als zuverlässig angesehen werden, wenn es eine geringe Fehlerrate aufweist. Die *Fehlerrate* (d. h., die Wahrscheinlichkeit, dass in einem bestimmten Zeitintervall ein Fehler auftritt) hängt von der Häufigkeit der Eingaben ab und von der Wahrscheinlichkeit, dass eine einzelne Eingabe zu einem Fehler führt.

Viele Anwendungen stehen heutzutage im Dauerbetrieb („24×7 operation“), z. B. Datenbanken, Webportale, Virens Scanner. Dies stellt hohe Anforderungen an die Qualität dieser Anwendungen in Bezug auf Zuverlässigkeit, aber auch an die zu Grunde liegende Hardware und Netzwerke. Weiters bedeutet dies eine große Herausforderung für die Wartung und Pflege solcher Anwendungen, da diese entweder am laufenden System oder parallel zum laufenden System durchgeführt werden müssen. In letzterem Fall ist die Synchronisation von Entwicklungs- und Zielsystem sowie das Testen (bei sich dynamisch verändernden Daten) besonders schwierig.

8.1.2.3 Benutzerfreundlichkeit

Unter *Benutzerfreundlichkeit* wird der übergeordnete Begriff für Adäquatheit, Erlernbarkeit und Robustheit eines Softwaresystems verstanden.

Adäquatheit

Die Forderung nach *Adäquatheit* (*Verständlichkeit*, *Benutzbarkeit*, *Bedienbarkeit*) bezieht sich auf

- die vom Benutzer verlangte *Eingabe*: Die erforderlichen Eingaben sollen sich auf das Notwendigste beschränken. Informationen sollen nur dann vom Softwaresystem erwartet werden, wenn sie für die vom Benutzer gewünschten Funktionen erforderlich sind. Das Softwaresystem soll dem Benutzer eine flexible Dateneingabe gestatten und Plausibilitätskontrollen der Eingabedaten durchführen. In dialogorientierten Softwaresystemen kommt der Einheitlichkeit, Klarheit und Einfachheit der Benutzerführung besondere Bedeutung zu;
- die vom Softwaresystem angebotene *Leistung*: Die Leistungsfähigkeit eines Softwaresystems soll unter Berücksichtigung der Erweiterbarkeit den Wünschen des Benutzers angepasst sein, d. h., die Funktionen sollen sich auf die in der Spezifikation vorgegebenen Funktionen beschränken („*Vollständigkeit*“);
- die vom Softwaresystem produzierten *Ergebnisse*: Die von einem Softwaresystem gelieferten Ergebnisse sollen übersichtlich und gut strukturiert ausgegeben werden und einfach zu interpretieren sein. Das Softwaresystem soll dem Benutzer Flexibilität bezüglich des Umfangs, des Detaillierungsgrades und der Art der Präsentation der Ergebnisse bieten. Eventuelle Fehlermeldungen müssen in einer für den Benutzer verständlichen Form bereitgestellt werden („*Brauchbarkeit*“).

Erlernbarkeit

Die *Erlernbarkeit* eines Softwaresystems hängt unmittelbar von der Ausgestaltung der Benutzerschnittstellen und der Klarheit und Einfachheit des Benutzerhandbuchs ab. Die Benutzerschnittstelle soll so geartet sein, dass sie die Information möglichst realitätskonform präsentiert und eine effiziente Nutzung der angebotenen Funktionen unterstützt. Lernen durch Probieren (durch Verfügbarkeit einer „Undo“-Funktion) muss möglich sein.

Robustheit

Unter der *Robustheit* eines Softwaresystems versteht man die Eigenschaft, die Auswirkungen von Bedienungsfehlern, falschen Eingabedaten und Hardwarefehlern abzuschwächen. Ein Softwaresystem wird als robust bezeichnet, wenn die Folgen eines Fehlers in der Bedienung, der Eingabe oder der Hardware in Bezug auf eine gegebene Anwendung umgekehrt proportional zu der Wahrscheinlichkeit des Auftretens dieser Fehler in der gegebenen Anwendung sind. Häufig zu erwartende Fehler (z.B. fehlerhafte Kommandos, Tippfehler) müssen also mit besonderer Umsicht behandelt werden. Seltener erwartete Fehler (z.B. Stromausfall) können großzügiger behandelt werden, dürfen aber trotzdem keine irreparablen Folgen nach sich ziehen.

8.1.2.4 Wartungsfreundlichkeit

Unter der *Wartungsfreundlichkeit* eines Softwaresystems versteht man seine Eignung für die Lokalisierung von Fehlerursachen, für die Durchführung von Fehlerkorrekturen und die Eignung zur Veränderung oder Erweiterung der Programmfunktionen. Diese Definition deutet an, dass die Wartungsfreundlichkeit von der Lesbarkeit, der Erweiterbarkeit und der Testbarkeit eines Softwaresystems abhängt.

Lesbarkeit

Die *Lesbarkeit* eines Softwaresystems wird bestimmt durch die Darstellungsform, den Programmierstil und seine Konsistenz, durch die Lesbarkeit der Implementierungssprache, die Strukturiertheit des Systems, die Qualität der Dokumentation, aber auch durch die Werkzeuge zur Inspektion eines Softwaresystems.

Erweiterbarkeit

Unter *Erweiterbarkeit* (*Änderbarkeit*) versteht man die Möglichkeit, gewünschte Änderungen an den passenden Stellen ohne unerwünschte Nebenwirkungen einzufügen. Dies ist ganz besonders abhängig von der Modularität und der Strukturiertheit des Softwaresystems und von den Möglichkeiten, welche die Implementierungssprache dafür zur Verfügung stellt, aber natürlich auch von der Lesbarkeit (zum Auffinden der passenden Stellen), der Verfügbarkeit einer verständlichen Programmdokumentation und von den zur Verfügung stehenden Werkzeugen.

Testbarkeit

Unter *Testbarkeit* eines Softwaresystems versteht man seine Eignung für die Verfolgung des Programmablaufs (Laufzeitverhalten unter vorgegebenen Bedingungen) und für die Lokalisierung von Fehlern. Die Testbarkeit hängt im Wesentlichen von der Modularität und der Strukturiertheit des Softwaresystems ab. Modulare, gut strukturierte Programme eignen sich besser zum systematischen, stufenweisen Testen als monolithische, unstrukturierte Programme. Testwerkzeuge und die Möglichkeit der Formulierung von Konsistenzbedingungen (Assertionen) im Quellcode reduzieren den Testaufwand und bilden wichtige Voraussetzungen für einen umfangreichen, systematischen Test aller Systemkomponenten.

8.1.2.5 Effizienz

Unter *Effizienz* (*Wirtschaftlichkeit*) eines Softwaresystems versteht man die Eigenschaft des Softwaresystems, seinen Zweck unter bestmöglicher Ausnutzung aller benötigten technischen Ressourcen zu erfüllen. Der Begriff „Ressource“ muss dabei sehr weit gefasst werden und bezieht sich auf Zeit, Speicherplatz, Übertragungskanäle und periphere Einheiten.

8.1.2.6 Portabilität

Unter *Portabilität* (*Übertragbarkeit*) eines Softwaresystems versteht man die Leichtigkeit, mit der es auf andere Rechner übertragen werden kann. Die Portabilität eines Softwaresystems hängt also vom Grad seiner Rechnerunabhängigkeit ab. Die Rechnerunabhängigkeit ist z. B. bestimmt durch die Wahl der Implementierungssprache und durch den Grad der Ausnutzung spezieller Systemfunktionen und Hardwareeigenschaften. Die Portabilität hängt damit in hohem Maße davon ab, ob das Softwaresystem so organisiert ist, dass die systemabhängigen Teile zu leicht austauschbaren Programmkomponenten zusammengefasst sind. Ein Softwaresystem kann als portabel bezeichnet werden, wenn der Änderungsaufwand für die Übertragung wesentlich kleiner ist als der Aufwand für eine neue Implementierung [Wallmüller, 2001].

Moderne Softwareentwicklung sieht normalerweise vor, dass Systemstrukturen, die aus verschiedenartigen, teilweise zugekauften Komponenten bestehen, gleichzeitig auf möglichst allen Zielhardwaresystemen entwickelt bzw. portiert werden. Das Konzept des „automatischen Portierens“ sieht vor, den Code eines Tagwerkes z. B. nach Dienstschluss automatisch über das Netzwerk auf alle Zielmaschinen zu übertragen und dort zu übersetzen, sodass praktisch jederzeit derselbe Projektstand auf allen Zielplattformen verfügbar ist.

8.1.3 Qualität in der Produktentwicklung

Die Qualitätsanforderungen an ein Softwareprodukt beziehen sich nicht nur auf das fertige (benutzerreife) Produkt. Die *Qualität des Endprodukts* hängt vielmehr von der *Qualität der Zwischenprodukte* ab, d. h., die Qualitätsanforderungen beziehen sich auf alle Ebenen der Softwareerstellung. Schlechte Qualität von Zwischenprodukten (z. B. Architekturdesign) hat immer schlechte Qualität des Endprodukts zur Folge [Pomberger und Blaschek, 1996]. Hinsichtlich der Qualitätsmerkmale und ihrer Bedeutung für den Entwicklungsprozess unterscheidet man daher zwischen

- Qualitätsmerkmalen, welche die *Zwischenprodukte* betreffen, und
- Qualitätsmerkmalen, welche das *Endprodukt* betreffen.

Die folgenden Gliederungen gehen zurück auf [Bons und van Megen, 1982].

Qualitätsmerkmale von Zwischenprodukten gliedern wir in:

- Qualitätsmerkmale, welche die *Transformation* betreffen: sie wirken auf die Eignung eines Zwischenprodukts für seine unmittelbare Transformation in ein nachfolgendes (höherwertiges) Produkt (Korrektheit, Lesbarkeit und Testbarkeit);
- Qualitätsmerkmale, die sich auf die *Qualität des Endprodukts* auswirken: sie wirken unmittelbar auf die Qualität des Endprodukts (Korrektheit, Zuverlässigkeit, Adäquatheit, Lesbarkeit, Erweiterbarkeit, Testbarkeit, Effizienz und Portabilität).

Qualitätsmerkmale von Endprodukten gliedern wir in:

- Qualitätsmerkmale, welche die *Anwendung* betreffen: sie wirken auf die Eignung des Produkts für die geplante Anwendung (Korrektheit, Zuverlässigkeit und Benutzerfreundlichkeit);

- Qualitätsmerkmale, welche die *Wartung* betreffen: sie wirken auf die Eignung des Produkts für Funktionsveränderungen und -erweiterungen (Lesbarkeit, Erweiterbarkeit und Testbarkeit);
- Qualitätsmerkmale, welche die *Übertragung* betreffen: sie wirken auf die Eignung des Produkts für die Übertragung in eine andere Einsatzumgebung (Portabilität und Testbarkeit).

8.1.4 Qualitätsgesteuerter Softwareentwicklungsprozess

Die Forderung, eine bestimmte Qualitätsanforderung zu erfüllen, löst Maßnahmen im Softwareentwicklungsprozess aus, die unter Umständen mit Funktionalitätsänderungen, höherem Kosten- und/oder Zeitaufwand verbunden sind (z.B.: die Erhöhung der Korrektheit, Zuverlässigkeit, Benutzerfreundlichkeit, ...), aber in der Regel eine Kosten- und Zeitersparnis für die Wartung und den Betrieb des Softwareprodukts (z.B. durch bessere Lesbarkeit und Testbarkeit) bewirkt. Sie kann jedoch im Widerspruch zu anderen Qualitätsmerkmalen stehen (so wirkt sich in der Regel die Zuverlässigkeit negativ auf die Effizienz aus). Eine entsprechende Priorisierung der Qualitätsmerkmale ist daher für einen *qualitätsgesteuerten Softwareentwicklungsprozess* (*quality-driven software development*) unumgänglich.

Die Wechselwirkungen zwischen Qualitätsmerkmalen und ihre Wirkung auf Funktionalität, Kosten und Zeit zeigt Abbildung 8.1 [Bons und van Megen, 1982]. „+“ in Abbildung 8.1 bedeutet eine Verstärkung, „-“ eine Abschwächung und „0“ ein neutrales Verhalten.

Merkmal	wirkt auf																
		Korrektheit	Zuverlässigkeit	Adäquatheit	Erlernbarkeit	Robustheit	Lesbarkeit	Änder-/Erweiterbarkeit	Testbarkeit	Effizienz	Portabilität	Entwicklungszeit	Lebenszeit	Entwicklungskosten	Betriebskosten	Wartungskosten	Übertragungskosten
Korrektheit			+	0	0	+	0	0	0	0	0	-	+	-	+	+	0
Zuverlässigkeit		0		0	0	+	0	0	0	-	0	-	+	-	+	+	0
Adäquatheit		0	0		+	0	0	0	0	+	-	-	0	-	+	+	-
Erlernbarkeit		0	0	0		0	0	0	0	-	0	-	0	-	+	0	0
Robustheit		0	+	+	0		0	0	+	-	0	-	+	-	+	+	0
Lesbarkeit		+	+	0	0	+		+	+	-	+	+	+	+	0	+	+
Änder-/Erweiterbarkeit		+	+	0	0	+	0		+	-	+	-	+	+	0	+	+
Testbarkeit		+	+	0	0	+	0	+		-	+	+	+	+	0	+	+
Effizienz		-	-	+	-	-	-	-	-		-	-	+	-	+	-	-
Portabilität		0	0	-	0	0	0	+	0	-		-	+	-	-	+	+

Abbildung 8.1: Wechselwirkungen zwischen Qualitätsmerkmalen und ihre Wirkung auf Funktionalität, Kosten und Zeit

8.1.5 Qualitätssicherung

Das *Ziel der Qualitätssicherung* (auch *Qualitätsmanagement* mit einer etwas umfassenderen Bedeutung genannt) ist die Sicherung der Produktqualität über den ganzen Projektentwicklungsprozess [ISO, 2004]. In der Regel kann in allen Folgephasen maximal die Qualität der vorausgehenden Phasen erreicht werden. Das bedeutet, dass mit Qualitätssicherung bereits bei der Projektinitiierung begonnen werden muss!

8.2 Aufgaben

8.2.1 Qualitätsplanung: Festlegen der Qualitätsmerkmale

Im Rahmen der *Qualitätsplanung* sind einerseits die Prioritäten der Qualitätsmerkmale festzulegen, andererseits Möglichkeiten, wie die Qualität überprüft werden kann. Dies geschieht durch quantitativ messbare Vorgaben (z. B. maximale Ausfallszeit pro Jahr) oder durch qualitative Vorgaben (z. B. Programm muss unter Linux und Microsoft Windows laufen).

8.2.2 Qualitätsprüfung: Prüfen der Projektgrundlagen

Die Voraussetzung für eine qualitativ hochwertige Projektentwicklung sind widerspruchsfreie und vollständige Unterlagen, welche die Projektgrundlage bilden und vorab geprüft werden müssen. Im Rahmen dieser *Qualitätsprüfung* ist Widerspruchsfreiheit noch eher überprüfbar als Vollständigkeit, da die Übersicht über die Problematik vom Auftragnehmer ja erst erarbeitet werden muss.

8.2.3 Qualitätslenkung: Maßnahmen zur Qualitätssicherung

Softwarequalität entsteht nicht von selbst. Es bedarf einer Reihe von Maßnahmen, um sicherzustellen, dass das Endprodukt eine akzeptable Qualität aufweist. Die wichtigsten Maßnahmen dieser *Qualitätslenkung* sind:

8.2.3.1 Organisatorische Maßnahmen

Organisatorische Maßnahmen werden im Laufe des gesamten Projektentwicklungszyklus eingesetzt und umfassen beispielsweise:

- Einsetzen von Vorgehensmodellen,
- kontinuierliches Weiterbilden der Produktentwickler,
- Institutionalisieren der Qualitätssicherung,
- Erstellen von Dokumentationsrichtlinien.

8.2.3.2 Konstruktive Maßnahmen

Konstruktive Maßnahmen werden bei der Erstellung eines (Zwischen-)Produktes angewendet und umfassen beispielsweise:

- konsequentes Anwenden von Methoden in allen Phasen des Entwicklungsprozesses,
- Einsetzen adäquater Entwicklungswerkzeuge,
- Entwickeln von Software auf der Basis hochwertiger Halbfabrikate,
- konsequentes Fortschreiben der Entwicklungsdokumentation.

8.2.3.3 Analytische Maßnahmen

Analytische Maßnahmen werden nach der Erstellung eines (Zwischen-)Produktes verwendet und umfassen beispielsweise:

- Durchführen statischer Programmanalyse,
- Durchführen dynamischer Programmanalyse,
- systematisches Auswählen geeigneter Testfälle,
- konsequentes Protokollieren der Analyseergebnisse,
- systematisches Melden und Behandeln von Fehlern,
- Prüfen und Beurteilen des Qualitätssicherungssystems selbst, z. B. im Hinblick auf eine Zertifizierung nach ISO 9000 ff.

8.3 Techniken

Im Gegensatz zu der früher in der Qualitätssicherung üblichen produktorientierten Betrachtungsweise sind die im Folgenden präsentierten aktuellen *Techniken der Qualitätssicherung* (Modelle, Verfahren und Normen) *prozessorientiert*, d. h., es wird versucht, mit Hilfe geeigneter Kriterien zu erfassen, wie gut eine Institution den Softwareentwicklungsprozess beherrscht.

Eine Bestätigung eines Unternehmens, dass es ein bestimmtes Modell oder Verfahren einsetzt (z. B. „ISO 9000-zertifiziert“ ist), bedeutet also nicht automatisch, dass das Unternehmen höchste Qualität produziert. Es bedeutet lediglich, dass es die Qualität des Produktionsprozesses (und damit des Produkts) offen legt.

8.3.1 ISO 90003:2004

Mit der Globalisierung der Produktion und des Handels stand die Wirtschaft verstärkt vor dem Problem, ihre Produktqualität und Qualitätssicherungsmethoden vor nationalen und

internationalen Kunden zu rechtfertigen. Daher wurde 1980 von der International Standardization Organization (ISO) ein technisches Komitee eingesetzt, um eine internationale Qualitätsnorm zu entwickeln. Der Entwurf war 1987 fertig gestellt und ist der Kern der *ISO 9000 ff.* Qualitätssicherungsnorm. De facto ist daher ISO 9000 ff. eine weltweite Kompilation der besten Qualitätssicherungspraktiken, die eine allgemeine Zustimmung fanden.

Die Anwendung von ISO 9000 ff. auf Software ist streng genommen als Nebeneffekt zu betrachten, wird aber durch eine Ergänzung zu ISO 9000 ff. gesondert geregelt und erhält damit in gewisser Weise doch eine Sonderstellung: „*ISO/IEC 90003:2004* – Qualitätsmanagement- und Qualitätssicherungsnormen: Leitfaden für die Anwendung von ISO 9001 auf die Entwicklung, Lieferung und Wartung von Software“ (bislang letzte Überarbeitung im Jahr 2004, vgl. [ISO, 2004]).

Um nach ISO 9000 ff. (genauer: ISO 9001:2000) zertifiziert werden zu können, sind 8 Schritte notwendig:

1. Evaluierung der existierenden Qualitätssicherungsmaßnahmen in Bezug auf die Anforderungen des ISO 9000 ff. Standards,
2. Identifikation der notwendigen Korrekturmaßnahmen, um mit ISO 9000 ff. konform zu werden,
3. Ausarbeitung eines Qualitätssicherungsprogramms,
4. Definition, Dokumentation und Implementierung der neuen Prozeduren,
5. Ausarbeitung eines Qualitätshandbuches,
6. Vor-Audits mit einem geprüften Auditor, um die Qualitätssicherungsunterlagen zu analysieren,
7. Audit von einer geprüften Zertifizierungsstelle,
8. Zertifizierung.

Die Zertifizierung ist „unumkehrbar“, d. h., ein Unternehmen kann nicht ohne weiteres wieder darauf verzichten, die Maßstäbe von ISO 9000 ff. anzuwenden, sondern muss sich in regelmäßigen Abständen neu überprüfen lassen (im Allgemeinen jährlich). Man kann sich leicht vorstellen, dass eine Nichtverlängerung der Zertifizierung für das betroffene Unternehmen sehr nachteilige Konsequenzen haben kann.

8.3.2 Das Capability Maturity Model Integration (CMMI)

Der Ansatz des *Capability Maturity Models (CMM)* zur Beurteilung der Qualität von Software geht auf eine Initiative der US-Luftwaffe zurück, die Richtlinien suchte, welche es ihr erlauben würden, die Leistungsfähigkeit und Vertrauenswürdigkeit von Softwarelieferanten schon während der Angebotsphase zu beurteilen. Das CMM wurde vom SEI entwickelt ([Humphrey, 1989], [Humphrey, 1987]) und im Rahmen von Adaptierungen immer weiter an den gesamten Softwareentwicklungszyklus angepasst (*Software-CMM* bzw. *SW-CMM*, vgl. [Paulk *et al.*, 1993]).

Im Rahmen der Erweiterungen wurde erkannt, dass für eine wirkungsvolle Prozessverbesserung verschiedene Bereiche von Geschäftsprozessen betrachtet und integriert werden müssen. In dem im Jahr 2000 vorgestellten *Capability Maturity Model Integration* (CMMI, manchmal auch als *Capability Maturity Model Integrated* bezeichnet; [Kulpa und Johnson, 2003]) werden die Bereiche Systems Engineering, Software-Akquisition und Software Engineering integriert betrachtet. Multidisziplinäre Prozessverbesserungen sind damit möglich, allerdings macht die Flut an Dokumenten und Vorschriften das Modell äußerst unhandlich [Ebert, 2004].

Eine CMMI-Klassifikation erfolgt nach fünf „Reifegraden“ (*maturity levels*), die den Grad der Beherrschung des Entwicklungsprozesses charakterisieren (Tabelle 8.1). Der Reifegrad wird auf Grund bestimmter Kriterien ermittelt (in CMMI 25), die alle Schlüsselbereiche des Entwicklungsprozesses (*key process areas*) abdecken. Allerdings ist das CMMI sehr vage gehalten, was laut [Ebert, 2004] leicht zu unpräziser Verwendung und nicht vergleichbaren Assessmentresultaten führen kann.

Reifegrad		Eigenschaften	Notwendige Verbesserungen
1	„Initial“	Prozessablauf „chaotisch“ und Leitung „ad hoc“	Projektführung, Projektplanung, Konfigurationsmanagement, Qualitätssicherung
2	„Managed“	Prozess intuitiv und personenabhängig beherrscht	Ausbildung, technische Praktiken (Reviews, Tests), Konzentration auf Normen und Teams
3	„Defined“	Prozess qualitativ erfasst, institutionalisiert und projektspezifisch anpassbar	Analyse und „Messung“ des Prozesses, quantitative Qualitätspläne
4	„Quantitatively Managed“	Prozess quantitativ erfasst und verstanden	Wechsel in der Technologie, Problemanalyse, Vermeidung von Problemen
5	„Optimizing“	Rückwirkung der Verbesserungen auf den Prozess	Organisation der Produktion auf optimierter Ebene

Tabelle 8.1: Reifegrade des Capability Maturity Model Integration

Das CMMI schlägt nach der Klassifizierung einer Institution konkrete Maßnahmen zur Verbesserung des Beherrschungsgrades des Softwareentwicklungsprozesses vor. Dabei wird pragmatisch vorgegangen, d. h., es wird jeweils nur empfohlen, dass eine Institution den nächsten „Level“ der Beherrschung anstreben soll, anstatt zu versuchen, in einem Anlauf die „totale Qualität“ zu erreichen.

Da das Assessment bei CMMI typischerweise vom Unternehmen selbst durchgeführt wird, ist jedenfalls kritisch zu hinterfragen, wer assessiert hat, welche Unternehmensbereiche (in welchem Umfang) assessiert wurden und ob das Assessment von akkreditierten Assessoren durchgeführt wurde.

8.3.3 BOOTSTRAP

Um nach ISO 9000 ff. zertifiziert werden zu können, ist eine Bestandsaufnahme des Unternehmens notwendig. Um auch diese Bestandsaufnahme geordnet durchführen zu können, wurde CMM(I) an die speziellen Erfordernisse von ISO 9000 ff. angepasst und erweitert. Dieses Verfahren, BOOTSTRAP genannt, ermöglicht nicht nur die Messung der Prozessreife, sondern auch das Ableiten präziser, ISO 9000 ff.-tauglicher Maßnahmen zur Zertifizierung und weiteren Prozessverbesserung [Frühauf und Lebsaft, 1995].

Um für eine Organisation ihren Qualitätsgrad zu ermitteln, werden Assessments (Bewertungen auf der Basis von Befragungen) durchgeführt. Die BOOTSTRAP-Methode, ein Ergebnis eines ESPRIT-Projekts der EU, verwendet dazu standardisierte Fragebögen, die sehr stark an CMM angelehnt sind und bei denen jede Frage einem „Reifegrad“ zugeordnet ist. Durch die Auswertung der Fragebögen wird für eine Softwareproduktionseinheit ein Reifegrad ermittelt, der zwischen 1 (niedrigstes Niveau) und 5 (höchstes Niveau) liegt und in Zehntelstufen angegeben wird. Zusätzlich wird ein Aktionsplan mit empfohlenen Sofortmaßnahmen erstellt, der für jeden Aktionsschritt ein „Miniprojekt“ definiert, um die empfohlenen Verbesserungsschritte umzusetzen. Im Unterschied zu CMMI wird jedoch auch ein Stärken-Schwächen-Profil erstellt, aus dem für die Qualitätskriterien des CMMI hervorgeht, auf welchem Entwicklungsstand sich das Unternehmen im Hinblick auf jedes Kriterium befindet.

Abbildung 8.2 zeigt die Grundlagen des BOOTSTRAP-Verfahrens sowie die möglichen Auswertungen. Der entscheidende Vorteil von BOOTSTRAP gegenüber ISO 9000 ff. ist der, dass BOOTSTRAP differenzierte Resultate liefert (und somit eine Qualitätsbewertung ermöglicht), während das Ergebnis eines Zertifizierungsaudits im Rahmen von ISO 9000 ff. ein binäres ist: ein Unternehmen besitzt ISO 9000 ff.-Niveau oder es besitzt es nicht.

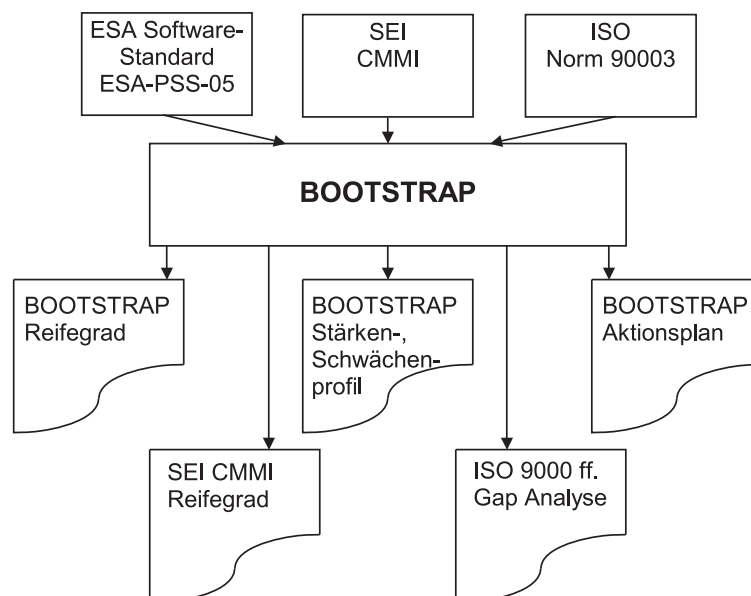


Abbildung 8.2: Grundlagen und Auswertungsmöglichkeiten der BOOTSTRAP-Methode

Die in einem BOOTSTRAP-Assessment anfallenden Beurteilungen lassen die Erstellung einer Analyse des Abweichungsgrades von ISO 9000 ff. zu („ISO 9000 ff. Gap-Analyse“). Gleich-

zeitig ist der erzielte BOOTSTRAP-Reifegrad ein Indikator für die Erfolgchance einer Zertifizierung. Liegt dieser in der Nähe oder oberhalb von 3, so kann man davon ausgehen, dass eine ISO-Zertifizierung ohne allzu großen Zusatzaufwand möglich ist.

8.3.4 ISO/IEC 15504 (SPICE)

ISO/IEC 15504 ([ISO, 1999]; auch unter *Software Process Improvement and Capability dEtermination – SPICE* bekannt) entstand aus der verstärkten Notwendigkeit, internationale Projekte nach gemeinsamen Qualitätsnormen abzuwickeln. Nachdem einige Zeit lang nach der Devise “follow CMM(I), certify ISO” verfahren wurde, wurde im Juni 1992 eine gemeinsame Arbeitsgruppe der ISO und IEC mit der Entwicklung einer Suite für Standards zur skalierbaren Softwareprozessmessung und -verbesserung beauftragt.

Die Kernpunkte von ISO/IEC 15504 sind ein Modus zur Zustandsbestimmung, ein Modus zur Prozessverbesserung und ein Modus zur Selbstbefragung und -auswertung, um für ein neues Projekt rasch die Qualitätsfähigkeiten bestimmen zu können. Entscheidende Vorteile *eines* internationalen Modells und Standards sind, dass Zulieferer international nach einem gemeinsamen Qualitätsstandard ausgesucht werden können (internationale Ausschreibungen!) und dass Softwareanbieter bei ihrer Qualitätsbestimmung einem einheitlichen Vorgehensmodell folgen können.

Mit weiteren Verfahren, etwa dem auf statistischen Methoden basierenden *Six Sigma* [Harry und Schroeder, 1999], der aus der Logistik kommenden Methode der *Balanced Scorecard* [Kaplan und Norton, 1996] oder Anwendungen des *Total Quality Management (TQM)* [Lowell, 1993]) besteht noch zu wenig Erfahrung in der Anwendung bei Softwareprojekten, um bereits schlüssige Aussagen machen zu können.

8.4 Ergebnisse

8.4.1 Qualitätshandbuch

Ein *Qualitätshandbuch* enthält die in der Qualitätsplanung festgelegte Priorisierung der Qualitätsmerkmale sowie die Beschreibung der Qualitätsziele der Projektentwicklung in messbaren Größen dieser Merkmale. Diese Merkmale können sowohl das Produkt als auch den Entwicklungsprozess betreffen. Auch die Maßnahmen zur Qualitätslenkung sind anzuführen. Entscheidend ist dabei die Festlegung von Kompetenzen und Verantwortungen für alle Qualitätssicherungsmaßnahmen.

8.5 Normen und Richtlinien

8.5.1 Grundlegende Bedeutung von Normen und Richtlinien

Normen und *Richtlinien* (im Deutschen wird oft auch der englische Begriff *Standard* verwendet) spielen in der industriellen Praxis eine bedeutende Rolle [Elzer, 1994]. Ihre Erarbeitung durch die zuständigen Gremien ist zwar meist ein mühseliger und langwieriger Prozess, in

dem neben technischen Gesichtspunkten und denen der Produktkosten auch die Konkurrenzsituation zwischen den beteiligten Firmen und selbst nationale Interessen eine Rolle spielen. Dennoch, oder vielleicht gerade auch wegen der Kompliziertheit des Entscheidungsprozesses, sind die Ergebnisse vielfach brauchbar und vor allem langlebig.

Erstens stellen sie sicher, dass die entstehenden technischen Lösungen gewissen Qualitätsansprüchen genügen. Deshalb kann die Verletzung einschlägiger Bestimmungen im Fall eines Schadens, der dadurch entsteht, recht unangenehme Folgen haben – besonders unter dem Gesichtspunkt der Produkthaftung. Weiters verbessern sie die Konkurrenzchancen kleinerer Firmen, die es sich nicht leisten können, komplette Systemlösungen anzubieten, und darauf angewiesen sind, z. B. „steckerkompatible“ Komponenten zu liefern.

Aus diesem Grund versuchen manche Großfirmen wiederum, eigene Richtlinien auf dem Markt durchzusetzen, um möglichst viele Teile eines Systems aus einer Hand liefern zu können. Auch für dieses Vorgehen gibt es gute technische Gründe: Eine Richtlinie kann durchaus einen neueren Stand der technischen Entwicklung darstellen und (auch für den Kunden) zu kosteneffizienteren Lösungen führen als eine (inter)nationale Norm, die wesentlich mehr Einflüsse berücksichtigen muss. Außerdem hat, wer haften muss, gerne alle Einflussgrößen in der Hand.

Normen und Richtlinien sind daher bereits bei der Zielerforschung durchzudenken, und zwar in Bezug auf

- Normen, welche die Projektgrundlage darstellen,
- Normen, die bei der Lösungserarbeitung angewendet werden müssen,
- Normen bzw. Richtlinien, die adaptiert bzw. neu erarbeitet werden müssen.

8.5.2 Normungsgremien

8.5.2.1 Gremien für Normen

Normen mit weltweiter Gültigkeit werden hauptsächlich von der *International Standardization Organization (ISO)* und – speziell im IT-Bereich – der *International Electrotechnical Commission (IEC)* herausgegeben. Diese Normen haben in vielen Ländern grundsätzlich Vorrang vor nationalen Normen (so z. B. in allen Ländern der EU). Rein aus Gründen des technischen und wirtschaftlichen Potenzials hat das amerikanische *American National Standards Institute (ANSI)* in der ISO den größten Einfluss, obwohl es in den Gremien auch nur jeweils mit einer Stimme vertreten ist. Inzwischen haben jedoch durch verbesserte Koordination der europäischen Aktivitäten auch die Organisationen der EU, *Comité Européen de Normalisation (CEN)* und *Comité Européen de Normalisation Electrique (CENELEC)*, an Bedeutung gewonnen.

Die in Deutschland für Normung auf dem Gebiet der Computerhardware und -software maßgeblichen Organisationen sind das *Deutsche Institut für Normung (DIN)* und die *Deutsche Elektrotechnische Kommission (DEK)*. Diese Organisationen haben (neben Frankreich und Großbritannien) auch den größten Einfluss in den Normungsgremien der EU. Das *Österreichische Institut für Normung (ÖNORM)* übernimmt größtenteils Normen von DIN bzw. DEK mit kleineren, für Österreich spezifischen Adaptierungen.

Neben den Normen dieser „quasistaatlichen“ Normungsinstitutionen gibt es die (inter-)national akkreditierten Normen starker Benutzerverbände, z. B. die der Post (*Comité Consultatif International de Téléphonie et Télégraphie – CCITT*), der Automobilerzeuger (in Deutschland *Verein der Deutschen Automobilhersteller – VDA*) oder des Militärs (in den USA *Mil.Stdd.*).

Die Erstellung einer Norm ist, wie bereits angesprochen, sehr zeitraubend. Der durchschnittliche Wert für Normierungen im IT-Bereich liegt bei fünf bis sieben Jahren [Elzer, 1994]. Dies erscheint nicht akzeptabel, da z. B. die Dauer einer „Hardwaregeneration“ meist kürzer ist (zwei bis vier Jahre). Da aber bisher alle Versuche, den Normierungszeitraum zu verkürzen, scheiterten, hat man, um wenigstens einige Jahre zu gewinnen, ein altes Prinzip der Normung außer Kraft gesetzt, nämlich eine Entwicklung erst zur Normung anzumelden, wenn sie sich in der Praxis als lebensfähig erwiesen hat.

Inzwischen wird, sobald sich z. B. genügend Interesse einiger Hersteller und Benutzer an einem Interfacestandard abzeichnet, mit der Standardisierungsarbeit begonnen. Die Praxisbewährung muss dann durch entsprechend intensivere Labortests ersetzt werden. Dass dies nicht immer zielführend ist, zeigt z. B. die von IBM forcierte (und normierte) „Microchannel“-Architektur, die sich am Markt nicht durchsetzen konnte.

8.5.2.2 Gremien für Industriestandards

Alle von den bisher genannten Organisationen herausgegebenen Normen sind sehr stark bindend und haben in manchen Ländern sogar Gesetzeskraft.

Weniger stark offiziell bindend, jedoch wegen der dahinter stehenden finanziell motivierten Freiwilligkeit der beteiligten Initiatoren in der Praxis manchmal erheblich wirksamer, sind die *Industriestandards*. Sie entstehen teilweise auch auf streng kanalisierte Weise in den offiziell zuständigen Gremien der Verbände, wie z. B. der *European Computer Manufacturers Association (ECMA)*, meist aber als Ergebnis loser Vereinigungen von Herstellern und Großanwendern (z. B. das „Ethernet“-Konsortium).

8.5.2.3 Gremien für Richtlinien

Offiziell auch nicht so stark bindend wie eine Norm, aber in der Praxis meist genauso angesehen und entsprechend beachtet, sind die Richtlinien der nationalen und internationalen Ingenieurverbände. Sie entstehen üblicherweise auch mit gleicher Sorgfalt und erfordern entsprechenden Aufwand.

Die in Deutschland traditionell am höchsten angesehenen Richtlinien sind die des *Vereins Deutscher Ingenieure (VDI)* und des *Verbandes Deutscher Elektrotechniker (VDE)*. Auf dem Gebiet der Informationstechnik haben sich inzwischen international die „Guidelines“ des *International Institute of Electrical and Electronic Engineers (IEEE)* hohes Ansehen erworben, obwohl es sich dabei ursprünglich um eine rein US-amerikanische Organisation handelte.

8.5.3 Normen und Richtlinien in der Softwareentwicklung

In der Softwareentwicklung sollen im Hinblick auf die Erhöhung der Qualität, Produktivität und Portabilität von Produkten möglichst viele Normen und Richtlinien Verwendung finden. Standardisierungsaspekte sind zusätzlich zur Auswahl von Entwicklungs- und Zielplattformen auch bei der Verwendung von Werkzeugen, Halbfabrikaten und Bibliotheken zu berücksichtigen.

Oft erscheint die Einführung bzw. Anwendung bestimmter Normen viel mehr „politisch“ motiviert und technisch zumindest hinderlich. Wie aber [Elzer, 1994] anmerkt, ist die Beherrschung der Normierungsmechanismen und das Verstehen ihrer Ergebnisse im Kontext oft entscheidend für Erfolg, Ansehen und Überlebenschancen einer Entwicklung. Außerdem wird in die Erarbeitung von Normen und Richtlinien so viel technischer Sachverstand investiert, dass es nie schaden kann, diese Dokumente zumindest zu konsultieren, auch wenn man sie dann im Projekt nur teilweise berücksichtigt. Die „Neuerfindung des Rades“ um jeden Preis ist immer schädlich.

Dazu kommt, dass Normen und Richtlinien, die von starken Kundengruppen und ihren Verbänden aufgestellt wurden, im Eventualfall von diesen Kunden zwangsweise (über die Ausschreibungsbedingungen) durchgesetzt werden. Beispiele dafür sind wieder die Post, das Militär, aber auch die Elektrizitätswirtschaft (in Deutschland VDEW – Verband der Deutschen Elektrizitätswirtschaft).

8.5.4 Normenauswahl

Offiziell ist bei der Auswahl der Normen und Richtlinien wie folgt vorzugehen:

1. internationale Normen,
2. Normen von Staatenverbänden (z. B. EU),
3. länderspezifische Normen,
4. Normen internationaler Vereinigungen und Verbände,
5. Kundenrichtlinien,
6. unternehmensinterne Richtlinien.

In der Praxis sieht die Reihenfolge meist so aus:

1. unternehmensinterne Richtlinien und Vorschriften,
2. alle übrigen Normen, Industriestandards und Richtlinien, die man innerhalb des Unternehmens der Geschäftsleitung, dem mittleren Management *und* den Mitarbeitern als sinnvoll und nutzbringend darstellen kann.

Die Schwierigkeit für einen Projektleiter (bzw. Standardisierungsbeauftragten) besteht oft darin, diese zwei Vorgehensweisen zu kombinieren, etwa derart, dass nach erster vorgegangen wird, dies aber als Vorgehen nach letzterer „verkauft“ wird.

In der Praxis bewahrheitet sich oft der Satz “The beauty of standards is that there are so many to choose from!” [Tanenbaum, 2001].

Teil III

Projektphasenspezifische Aufgabenbereiche

9 Zielerforschung

Die *Zielerforschung* (*objectives¹ determination*) dient der Festlegung des *Gesamtziels* (bzw. der *Grobziele*). Diese Grobziele eines Systems stellen die Zielvorgabe eines Projekts dar, die sich „im Groben“ während des Projekts nicht ändern darf. Im Rahmen der Analyse sind die Grobziele Gegenstand der Untersuchung und werden in Detailziele zerlegt. Diese Ziele sind dann in den weiteren Phasen Vorgaben.

Ziele und Anforderungen ändern und entwickeln sich im Laufe eines Projekts. Dies ist keine Schwäche der Projektführung, sondern Projektbestandteil, wie bereits in [Fischer, 1989] angemerkt wird: „Wir müssen sich ändernde Anforderungen als eine Tatsache der Realität annehmen und sie nicht als ein Produkt ungenauen Denkens verdammen.“ Mittlerweile wird diese Erkenntnis in fast allen modernen iterativ-inkrementellen Entwicklungsmethoden übernommen, vgl. [Highsmith, 2002].

Wenn sich Detailziele ändern, kann im Allgemeinen das Projekt durch Umplanung angepasst werden; wenn sich Grobziele ändern, muss das Projekt abgebrochen und ggf. neu definiert werden. Da sich Ziele meistens schleichend ändern, ist es eine der schwierigsten Aufgaben des Projektleiters, zu erkennen, wann die Projektentwicklung vom Gesamtziel so weit abgewichen ist, dass das Projekt durch Umplanen nicht mehr fortgeführt werden kann und ein Projektabbruch notwendig ist.

¹Im Englischen wird zwischen „goals“ und „objectives“ unterschieden, wobei ein objective messbar und mit einem Zeitziel versehen ist, vgl. [Coleman *et al.*, 1994]. Objectives werden daher genauer definiert als goals.

Als Ergebnis der Zielerforschung soll daher eine möglichst kompakte, klare Beschreibung (*Vision* bzw. *vision statement*) erarbeitet werden, was das unveränderliche Gesamtziel des Projekts ist, d. h., was durch das zukünftige Projekt erreicht werden soll. Die Phase der Zielerforschung wird daher unter anderem auch als *Systemanalyse* bezeichnet (im Unterschied zur *Anforderungsanalyse*, vgl. Abschnitt 10.1).

9.1 Erhebung

9.1.1 Ziel

Die Zielerforschung ist meist nicht Teil der (bezahlten) eigentlichen Projektarbeit, sondern dient der Projektanbahnung, die entweder eigens finanziert wird (im Rahmen eines Vorprojekts) oder die im Hinblick auf den zu erwartenden Projektauftrag als Vorleistung erbracht wird. Der erste Schritt dabei ist die *Erhebung* (*objectives capturing*), deren *Ziel* die Feststellung der Projektausgangslage durch den (potenziellen) Auftragnehmer im Umfeld des (potenziellen) Auftraggebers ist.

9.1.2 Aufgaben

Die *Erhebung* umfasst folgende *Aufgaben*:

- Zusammenstellen der *Projektvorarbeiten* seitens des potenziellen Auftraggebers und
- Definieren des *Projektumfeldes* (Organisation, Eingliederung in das Unternehmen des Auftraggebers und dessen operationale und strategische Entwicklung).

Dabei tritt der Auftragnehmer als Außenstehender dem Unternehmen – bei externen Projekten – gleichsam als Forscher oder Beobachter im weitesten Sinn gegenüber. Diese Sichtweise mag bei internen Projekten vielfach in weit geringerem Maß gegeben sein. Dennoch sollte allzu große Problemvertrautheit und internes „Problemselbstverständnis“ vermieden werden, um nicht den Gefahren der Betriebsblindheit ausgesetzt zu sein.

Aus dieser Situation ergibt sich eines der größten Probleme in der Projektentwicklung: Man arbeitet in einem Problemraum, der neu und unvertraut ist, in dem man aber den Anwendungsbereich studieren muss und zu neuen Erkenntnissen (Problemlösungen) kommen soll. Der Problemraum muss in Umfang und Inhalt begriffen werden, man muss erkennen, wo man welche Tiefe des Verständnisses erarbeiten muss. Dies alles muss so schnell wie möglich geschehen – ein Projekt soll „an Land gezogen werden“, und im Allgemeinen wird man nicht oder zumindest nicht nach Aufwand bezahlt. Auch im glücklichen Fall, dass man ein Problem eines vertrauten Bereichs erarbeiten muss, bleibt das annähernd gleich große Problem, das eigene Wissen und die Erfahrungen effizient an das eigene Team weiterzugeben.

9.1.3 Techniken

9.1.3.1 Einteilung

Die verschiedenen *Techniken der Erhebung* können nach mehreren Kriterien klassifiziert werden:

1. Nach der Art der Interaktion:

- auditiv: Interview (persönlich oder telefonisch),
- schriftlich: Fragebogen zur Selbstbearbeitung bzw. Auswertung existierender Dokumente.

2. Nach dem Strukturierungsgrad:

- nicht strukturiert („offen“),
- halb strukturiert (Fragen vorgegeben),
- strukturiert („geschlossen“; Fragen und Antworten vorgegeben).

3. Nach dem Objekt der Erhebung:

- Primärerhebung: Befragung und Beobachtung,
- Sekundärerhebung: Dokumentenauswertung.

4. Nach der Anzahl der Erhebungsobjekte:

- Einzelerhebung (ein Element der Gesamtheit),
- Fallstudie (einzelne Elemente der Gesamtheit),
- repräsentative Erhebung (eine Gruppe, die für die Gesamtheit repräsentativ ist),
- Totalerhebung (alle Elemente der Gesamtheit).

Die drei Haupttechniken – das Interview, der Fragebogen und die Dokumentenauswertung – werden im Folgenden näher vorgestellt. Die Darstellungen folgen in Teilen [Heinrich und Lehner, 2002].

9.1.3.2 Interview

Das *Interview* ist ein Interaktionsprozess mit einer Rollenaufteilung zwischen Interviewer und Interviewtem. Beide Rollen werden in der Regel durch je eine Person besetzt, können aber auch durch mehrere Personen repräsentiert werden. So genannte Gruppeninterviews, bei denen die Interviewten in der Gruppe auftreten, sind vor allem im Beratungsbereich zu finden. Ein Beispiel für eine Gruppe von Interviewern ist das Bewerbungsgespräch, bei dem im Allgemeinen mehrere Experten den Kandidaten/die Kandidatin interviewen.

Normalerweise ist die Interaktion bei einem Interview von Angesicht zu Angesicht. Beim Telefoninterview ist der Kommunikationsprozess auf den auditiven Kanal beschränkt. Wegen dieser Beschränkung und wegen der zeitlichen Beschränkung, die sich aus der besonderen Situation am Telefon ergibt, wird das Telefoninterview praktisch nur strukturiert geführt.

Im Folgenden wird hauptsächlich das nicht strukturierte bzw. das halb strukturierte Interview behandelt. Strukturierte Interviews werden nur insoweit angesprochen, als sie sich von der Erhebung mittels Fragebogen (siehe Abschnitt 9.1.3.3) unterscheiden.

Nicht und halb strukturierte Interviews

Nicht strukturierte Interviews bedeuten ein offenes Gespräch zwischen Interviewer und Interviewtem, bei dem im Allgemeinen nur ein Thema vorgegeben ist. Ein guter Interviewer muss folgende Qualitäten besitzen:

- Er stellt Fragen, die interessante Antworten versprechen.
- Er ist in der Lage, auf Antworten zu reagieren, d. h., die weiteren Fragen richten sich auch nach den vorangegangenen Antworten.
- Er nimmt nicht nur das gesprochene Wort wahr, er registriert auch nonverbale Signale („Körpersprache“) des Interviewten.

Obwohl beim nicht strukturierten Interview der Inhalt während des Gespräches erarbeitet wird, muss das Interview doch gründlich vorbereitet werden. So sollten dem Interviewer der Themenbereich sowie die Person und das Umfeld des Interviewten vertraut sein. Allerdings darf dem spezifischen Wissen über den Interviewten nicht zu sehr vertraut werden. Die Gesprächsrichtung könnte unvorhergesehene Wege gehen, oder an Stelle der geplanten Person muss das Gespräch mit einer Vertretung geführt werden.

Es ist daher wichtig, für ein nicht strukturiertes Interview einen *Interviewleitfaden* zu formulieren, der solche Themen enthält, die unabhängig vom spezifischen Wissen über den Interviewten Antworten und Erzählungen in Richtung der Zielinhalte auslösen. Im Fall eines *halb strukturierten Interviews*, bei dem Fragen vorgegeben werden, setzt dies auch eine klare und verständliche Formulierung der Fragen voraus. Werden zusätzlich zu den Fragen noch Antwortalternativen durch den Interviewer angeboten, dann wird aus dem halb strukturierten ein strukturiertes Interview.

Strukturiertes Interview

Strukturierte Interviews sind eine Verbindung der Interaktionsart des Interviews mit der Technik des Fragebogens. Die Ausfüllarbeit des Fragebogens wird vom Interviewer übernommen. Die Struktur des Interviews wird durch den Interviewleitfaden, der einem Fragebogen gleichkommt, völlig vorgegeben. Gleichzeitig dient meist der Fragebogen auch als Interviewprotokoll, d. h., die Antworten werden direkt in den Fragebogen eingetragen.

Die Besonderheit des Interviews mit Fragebogen gegenüber dem reinen Fragebogen zum Selbstausfüllen ist, dass Interviewer speziell geschult sind. Im Allgemeinen ist es daher nicht notwendig, einen Fragebogen, der nur als Interviewleitfaden und Interviewprotokoll dient, mit demselben Aufwand zu erstellen, wie dies bei Fragebögen zur Selbstbearbeitung notwendig ist. Unklarheiten im Fragebogen können während des Interviews ausgeräumt werden, und der Interviewer weiß, welche Antworten er auf Fragen des Interviewten zu geben hat. Weiters können auch komplexere Verzweigungen für bedingte Fragen eingebaut werden.

Fragetypen

Fragen lassen sich auf folgende Weise klassifizieren:

- **Aufschließende versus abschließende Fragen:** Aufschließende Fragen skizzieren bzw. beginnen ein Thema („Welche Probleme gibt es bei Ihrer Arbeit?“). Abschließende Fragen beenden einen Abschnitt („Könnten Sie mir nochmals die drei wichtigsten Probleme nennen?“).
- **Offene versus geschlossene Fragen:** Offene Fragen (*Ergänzungsfragen*, *W-Fragen*) erlauben eine inhaltliche, ergänzende Antwort des Interviewten („Welche Probleme gibt es bei Ihrer Arbeit?“). Geschlossene Fragen (*Entscheidungsfragen*) erlauben die Entscheidung zwischen einer Anzahl vorgegebener Antworten, im Minimalfall zwischen „Ja“ und „Nein“ („Fühlen Sie sich bei Ihrer Arbeit gestresst?“).
- **Direkte versus indirekte Fragen:** Direkte Fragen sprechen ein Team direkt an („Fühlen Sie sich bei Ihrer Arbeit gestresst?“). Indirekte versuchen, Fakten über einen Umweg herauszufinden („Wie schätzt man in Ihrer Abteilung die Aufgabe ein?“).

9.1.3.3 Fragebogen

Fragebögen zählen zu den Methoden mit hohem Strukturierungsgrad und dienen wegen der relativ hohen Entwicklungskosten und relativ niedrigen Durchführungskosten vor allem zur Befragung größerer Teilgruppen oder zur Totalerhebung in Organisationen.

Fragebögen dienen einerseits zur Unterstützung von Interviews, andererseits werden sie zur direkten Bearbeitung durch den Befragten konstruiert. Im Folgenden gehen wir davon aus, dass der Fragebogen durch den Befragten selbst ausgefüllt wird. Daraus ergibt sich der Grundsatz der Fragebogenkonstruktion: Ein Fragebogen soll so gestaltet sein, dass alle Personen des Kreises der Befragten grundsätzlich in der Lage sind, diesen selbstständig, schnell und ohne Rückfragen zu bearbeiten.

Folgendes schrittweises Vorgehen ist bei der Fragebogenerhebung zu befolgen:

1. **Zielgruppe auswählen:** Im Vorhinein weiß man zwar meist wenig über die Gruppe der Befragten, allein aus der Art der Zielgruppe (Stichprobe) lassen sich allerdings Mindestfähigkeiten ableiten, welche die Bearbeitung gewisser Fragestellungen im Fragebogen ermöglichen. Mehr als diese Mindestfähigkeiten sollte nicht abverlangt werden.
2. **Fragen formulieren:** Die eigentliche Frage sollte kurz, prägnant und eindeutig sein. Eine kompakte Anforderungsdefinition an Fragen stellt die *KROKUS-Regel* dar:

Kurze Fragen stellen,
Redundante Fragen vermeiden,
Offene Fragen stellen,
Konkrete Fragen stellen,
Unter- und Kettenfragen vermeiden,
Suggestivfragen vermeiden.

Die Eindeutigkeit einer Frage wird oft erst durch die vorgegebenen Antwortalternativen sichergestellt (geschlossene Fragestellung). Auch in diesem Fall ist aber häufig noch ein Bearbeitungshinweis notwendig. Mögliche Bearbeitungsformen sind:

- **Ankreuzen von Alternativen** (bzw. Nennung von Alternativen) in einem dafür vorgesehenen Feld, wobei klargestellt werden muss, ob nur eine Nennung oder mehrere möglich sind. Die angegebenen Alternativen sollten vollständig sein, d. h., es sollte nicht vorkommen, dass der Befragte seine Antwort nicht einordnen kann (gegebenenfalls ist eine Kategorie „Sonstiges“ mit einem frei gelassenen Feld vorzusehen; dies bringt aber oft Schwierigkeiten bei der Auswertung mit sich).
- **Reihung von Alternativen** durch Vergabe von Ordnungszahlen. Dazu ist eine genaue Anweisung notwendig, wie die Reihung vorzunehmen ist. So ist die Bedeutung der Ordnungszahlen anzugeben (was ist besser, was ist schlechter), es ist festzulegen, ob mehreren Alternativen die gleiche Ordnungszahl zugewiesen werden darf etc.
- **Skalierung** durch Markieren auf einer vorgegebenen Skala. In der Regel werden die Endpunkte, eventuell auch die Mitte, seltener alle Ausprägungen auf der Skala benannt. Dabei ist zu berücksichtigen, dass bei Skalen ein Trend zur Mitte und zur Vermeidung der Extreme beobachtet werden kann; manchmal kann es sogar sinnvoll sein, durch die Vorgabe einer geraden Anzahl von Skalenwerten die Auswahl der Mitte zu verhindern. Skalen mit drei bis sechs Unterteilungen sind allgemein gut handhabbar (vertrautes Schema der Schulnoten!), nur in seltenen Ausnahmefällen sollte eine Skala mehr als neun Unterteilungen haben.

Wie bereits angesprochen, wird ein Fragebogen zu einer halb strukturierten Methode, wenn vermehrt offene Fragen verwendet werden. Nachdem der Fragebogen kein Nachfragen erlaubt, ist die sinnvolle Nutzung von offenen Fragen nur in eingeschränktem Ausmaß möglich.

3. **Fragebogen zusammenstellen:** Die Bearbeitung einer Frage beeinflusst die Antwort auf die folgenden Fragen. Daher ist ein wesentlicher Teil der Fragebogenkonstruktion das Finden der geeigneten Fragenreihenfolge (*Fragebogendramaturgie*):

Bevor mit Fragen begonnen wird, muss der Befragte zur Mitarbeit motiviert werden, und es müssen allfällige Widerstände abgebaut werden. Dazu sind in einer Einleitung folgende Punkte zu berücksichtigen:

- Angaben über die erhebende Institution,
- Ziel der Erhebung,
- Zusicherung über die Anonymität bzw. genaue Angaben über Datenverwendung.

Um Widerstände gegen die Arbeit am Fragebogen abzubauen, sollte nicht unbedingt mit den schwierigsten bzw. heikelsten Fragen begonnen werden. Vielmehr stehen am Anfang so genannte Einleitungsfragen, die primär zum Ziel haben, Interesse am Fragebogen zu fördern (z. B. eigenes Interesse oder Zugang zum Thema). Als Abschluss eines Fragebogens eignen sich gut Angaben zum Ausfüller (Geschlecht, Alter etc.) oder ein generelles Statement bzw. Resümee.

Die notwendige Arbeitszeit, um einen Fragebogen auszufüllen, sollte 20 Minuten nicht überschreiten [Heinrich und Lehner, 2002].

4. **Fragebogen testen:** Auch bei genauester Planung eines Fragebogens werden in den meisten Fällen einzelne Formulierungen für die Befragten nicht verständlich oder vorgegebene Antwortmöglichkeiten unvollständig sein. Deshalb ist für eine fundierte Fragebogenkonstruktion ein Vortest mit einer ersten Version des Fragebogens notwendig, mit dem nur einige wenige Personen befragt werden. Dieser Vortest ist nur in einer Interviewsituation sinnvoll möglich, da sonst die Unklarheiten unter Umständen nicht dokumentiert werden können. Mit den Erkenntnissen aus dem Vortest wird dann die endgültige Version des Fragebogens konstruiert.
5. **Fragebogen einsetzen:** Ein Fragebogen wird entweder für ein strukturiertes Interview verwendet oder ausgeteilt bzw. versendet. In letzterem Fall ist ein entsprechendes Begleitschreiben beizulegen.

Tipp: Allgemein sollte bereits bei der Planung einer Fragebogenerhebung bedacht werden, dass die Anzahl der retournierten Fragebögen im Verhältnis zu den versendeten (Rücklaufquote) in vielen Fällen sehr niedrig ist. So ist beispielsweise eine Rücklaufquote von mehr als 10 % bei einer Fragebogenerhebung bei Unternehmen bereits als Erfolg zu werten.

6. **Fragebogen auswerten:** Die Auswertung der retournierten Fragebögen hängt naturgemäß stark mit der Fragebogenstruktur zusammen. Zusätzlich ist eine gute Abschätzung der Vertrauenswürdigkeit der gegebenen Antworten (*Zuverlässigkeitsgrad*) sehr wichtig.

Fast immer ist es notwendig, ungenaue oder unvollständige Information zu ergänzen, um Antworten besser vergleichbar zu machen. Sich nachträglich als unwichtig oder als nicht sensitiv (d. h., im Allgemeinen gleich beantwortete) herausstellende Fragen müssen eliminiert werden; die Liste der für eine bestimmte Zielklassifizierung notwendigen Bedingungen („Sine-qua-non-Bedingungen“) muss adaptiert werden.

Allgemein sind geschlossene Fragestellungen rascher und vergleichbarer auswertbar als offene Fragestellungen.

9.1.3.4 Dokumentauswertung

Im Allgemeinen wird von einem Auftraggeber ein Projekt nicht ohne (interne) Vorarbeiten in Auftrag gegeben. Die Auswertung der Dokumentation dieser internen Vorarbeiten ist daher eine entscheidende Unterstützung der Erhebung. Allerdings werden diese Daten zu Beginn manchmal sehr schwer zugänglich gemacht (z. B. wenn ein Unternehmen eine Problemlösung selbst nicht geschafft hat und daher extern vergibt). Oft wird von den Auftragnehmern auch schlicht vergessen, nach entsprechenden Dokumenten zu fragen.

Diese Dokumente liefern im Allgemeinen Informationen über den ein Problem/Projekt betreffenden Daten- und Informationsfluss. Beispiele solcher Dokumente können alle im Rahmen dieses Buchs angesprochenen Ergebnisse einer Projektentwicklung sein, aber auch Bestellschein, Lieferscheine, Arbeitsanweisungen, Ablaufdiagramme, Hinweissammlungen für Mitarbeiter (Vorschläge im Rahmen eines Verbesserungswesens), Konkurrenzprodukte bzw. Vorgehensweisen oder Lösungen vergleichbarer Organisationen etc.

Eine spezielle Art der Dokumentauswertung ist der nachfolgend angesprochene *Produktvergleich*.

Produktvergleich

In einem Projekt fallen laufend Entscheidungen an, bei denen zwischen Alternativen gewählt werden muss. So müssen z. B. Werkzeuge ausgewählt werden, oder es muss entschieden werden, ob eine externe Klassenbibliothek zugekauft oder eine vorhandene adaptiert werden soll. Vielfach ist es auch entscheidend, die Kriterien festzulegen, durch die sich das neu zu erstellende Produkt von anderen, bereits (kommerziell) erhältlichen Produkten abhebt. Falls die Konkurrenzprodukte gewisse zusätzliche Funktionalitäten aufweisen, muss entschieden werden, welche davon ins eigene Produkt übernommen werden.

Um solche Entscheidungen bewusst und nachvollziehbar machen zu können, müssen die verschiedenen Alternativen nach einer Bewertungsmethodik vergleichbar gemacht und verglichen werden. Gearbeitet wird dabei mit Arbeitsversionen der zu vergleichenden Produkte, mit Demonstrationsversionen oder mit schriftlichem Informationsmaterial, sofern keine lauffähige Software zur Verfügung steht. Die bekanntesten Techniken des Produktvergleichs sind die *Bewertungsmatrix* sowie die *Nutzwertanalyse*.

Grundsätzlich basieren diese Techniken auf einer Zerlegung der Produktaufgaben in Teilaufgaben, der Auflistung von Vor- und Nachteilen sowie der gegenseitigen Gewichtung dieser Komponenten (immer subjektiv!). Die Gewichtungsstrategie sollte daher immer dem Auftraggeber transparent gemacht und von diesem mitgetragen werden. Die Ergebnisse eines Produktvergleichs müssen immer Plausibilitäts- und Sensibilitätsüberprüfungen unterzogen werden.

Ein Produktvergleich wird von verschiedenen Personen/Abteilungen des Auftraggebers verschieden gelesen und verwendet. Für den zuständigen Projektleiter seitens des Auftraggebers sind z. B. die Adressen der verschiedenen Anbieter samt Kontaktpersonen von Interesse; der Leiter der Beschaffung verlangt eine einseitige Übersicht aller (finanziell) entscheidenden Faktoren etc.

9.1.4 Ergebnisse

9.1.4.1 Erhebungsauswertung

Die *Erhebungsauswertung* ist ein Dokument, das die Auswertung von zur Projektanbahnung notwendigen Interviews und Besprechungen, Fragebogenaktionen, vorhandenen Dokumenten etc. umfasst. Vom Dokumenttyp her ist es ein Bericht, der dem Auftraggeber zur Verfügung zu stellen ist und auf den in den nachfolgenden Phasen sowie im Angebot Bezug genommen werden kann.

Die Erhebungsauswertung gibt die Problembeschreibung in der Interpretation des (potenziellen) Auftragnehmers wieder und kann somit kein bindendes Dokument darstellen. Erst durch die Abnahme der nachfolgenden Dokumente, wie Lastenheft oder Auftrag, in welche die Ergebnisse der Erhebungsauswertung einfließen, werden die Festlegungen für das Projekt bindend.

9.2 Zielfindung

9.2.1 Ziel

Vielfach ist das Projektziel des Auftraggebers sehr nebulös formuliert, oder es ergeben sich durch die Erhebung zusätzliche Aspekte, die das Projektziel ändern oder erweitern. *Ziel der Zielfindung (objectives elicitation)* ist es daher, auf Basis der erhobenen Projektgrundlage das *Gesamtziel* des Projekts und die damit verbundenen *Grobziele* zu erarbeiten.

9.2.2 Aufgaben

Die *Zielfindung* umfasst folgende *Aufgaben*:

- Ziele müssen gewichtet und klassifiziert werden in *Mussziele (essential objectives)*, *Sollziele (conditional objectives)* und *Wunschziele (optional objectives)*². Dabei muss auf Zielkonflikte geachtet werden.
- Die Beweglichkeit der Ziele und deren Auswirkungen müssen analysiert werden.
- Die verschiedenen Organisationsvorgaben (Unternehmensziel, Abteilungsziel, Projektziel) sind festzulegen.
- Die unterschiedlichen Zielarten (technische Ziele, organisatorische Ziele, finanzielle Ziele, ...) sind herauszuarbeiten.

Für alle Ziele gilt, dass deren Erreichungsgrad bestimmbar und überprüfbar sein muss. Ziele, deren Erreichung nicht oder nur ungenau festlegbar bzw. überprüfbar ist („Programm muss schnell laufen“), sind zu verfeinern oder fallen zu lassen.

Die Zielfindung ist (zusammen mit der sie verfeinernden Analyse) der eigentlich kreative Prozess der Projektentwicklung. Dies bedingt die zentrale Rolle menschlicher Experten in dieser Phase, ein methodisches Vorgehen bei der Zielfindung lässt die Erfolgchancen allerdings beträchtlich steigen. Man bedient sich daher bei der Zielfindung der Methoden der kreativen Ideenfindung (Kreativitätstechniken), deren wichtigste Vertreter nachfolgend vorgestellt werden.

9.2.3 Techniken

Der Begriff „kreativ“ bedeutet schöpferisch. Wenn Neues entwickelt wird, so ist dies ein wesentliches Kennzeichen für Kreativität. Kreatives Denken ist fixer Bestandteil im Problemlösungsprozess. Jeder Mensch ist grundsätzlich kreativ. Wieweit dieses Kreativitätspotenzial ausgenutzt wird, hängt aber sehr von der Motivation bzw. dem Umfeld ab.

²[Yourdon, 1997] weist darauf hin, dass Auftraggeber gerne alle ihre Ziele als „Mussziele“ klassifizieren. Als Lösung wird vorgeschlagen, den Auftragnehmer nach Erstellung der Liste aller Ziele die Auswahl selbst vornehmen zu lassen, wobei jedoch jeder der drei Kategorien (Muss, Soll, Kann) ein Drittel der Ziele zugeteilt werden muss („Triage“).

Der Projektleiter muss die kreativen Leistungen seiner Mitarbeiter verstärken, indem er sie ausreichend mit Informationen versorgt und selbst auch kreativ ist. Kreative Ideen sind oft originell und unkonventionell. Man darf a priori keinen Vorschlag verwerfen, auch wenn dadurch schon vermeintlich geklärte Fragen neu bearbeitet werden müssen.

Ein Problemlösungsprozess ist immer interaktiv und iterativ, d. h., Lösungen müssen immer wieder einer Überprüfung in Richtung weiterer Verbesserungen unterzogen werden. Die *Techniken der Zielfindung*, die diesen Problemlösungsprozess unterstützen, nennt man *Kreativitätstechniken*.

9.2.3.1 Organisatorische Voraussetzungen für Kreativitätstechniken

Zur Bewältigung der Projektaufgaben sind nicht nur Kenntnisse und Fähigkeiten der Aufgabenträger (z. B. zur Anwendung von Methoden und Werkzeugen) erforderlich, sondern auch schöpferische Eigenschaften, also Kreativität. Kreativität ist besonders dann gefragt, wenn es gilt, Alternativen zu entwickeln. Dies ist in den früheren Phasen der Projektentwicklung in größerem Rahmen der Fall als in den späteren Phasen.

In den frühen Phasen einer Projektentwicklung ist der Handlungsspielraum groß; er nimmt mit dem Fortschreiten der Projektarbeit ständig ab. Ein großer Handlungsspielraum lässt aber nicht nur mehr Raum für kreatives Handeln, er muss auch ausgeschöpft werden, wenn die Projektziele effizient erreicht werden sollen. Kreatives Handeln wird durch die Anwendung von Kreativitätstechniken wirkungsvoll unterstützt.

Kreativitätstechniken sind Methoden zur Definition und zur Lösung schlecht strukturierter Probleme durch Anwendung intuitiver Probiervverfahren. Aus diesen generellen Überlegungen zur Möglichkeit und zur Notwendigkeit kreativen Handelns und seiner Unterstützung durch Kreativitätstechniken ergibt sich, dass die wichtigste Aufgabe der Kreativitätstechniken das Generieren alternativer Systemkonzepte darstellt.

Allen Kreativitätstechniken gemeinsam sind die Anwendungen in Gruppensitzungen mit Teilnehmern aus verschiedenen Tätigkeitsbereichen, verschiedenen hierarchischen Ebenen und mit unterschiedlichen Kenntnissen und Fähigkeiten („interdisziplinäre Kreativitätsgruppe“). In der Projektentwicklung sind daher Gruppen zweckmäßig, deren Teilnehmer aus verschiedenen Entwicklungsbereichen und/oder Anwendungsbereichen kommen, die teilweise eher theoretische oder eher praktische Kenntnisse besitzen. Gemäß [Heinrich und Lehner, 2002] besitzt eine interdisziplinäre Kreativitätsgruppe mit fünf bis zwölf Personen die ideale organisatorische Größe. Es kann daher zweckmäßig sein, dass sich mehrere Projektgruppen zu Organisationssitzungen zusammenfinden. Mindestens ein Moderator steuert den Gruppenprozess und dokumentiert die Ergebnisse. Die Sitzungsdauer sollte 30 Minuten nicht wesentlich überschreiten; im Allgemeinen sind daher mehrere Sitzungsrounden erforderlich.

Gemeinsame *Merkmale der Kreativitätstechniken* sind also:

- das Zerlegen von Problemen in Teilprobleme,
- das Generieren von Lösungsvorschlägen zu den Teilproblemen,
- das Auswählen der besten Teillösungen und deren Synthese zur Problemlösung und
- das Steuern des Gruppenprozesses durch einen Moderator.

Die wichtigsten Kreativitätstechniken sind:

- Brainstorming,
- Brainstorming mit Osborne-Verfremdung,
- Synektik,
- Morphologischer Kasten.

Brainstorming (mit und ohne Osborne-Verfremdung) ist eher für Probleme mit niedriger Komplexität geeignet. Es kann mit eingeschränkter Erfolgswahrscheinlichkeit auch von Einzelpersonen angewendet werden. Die Methoden der Synektik und die des Morphologischen Kastens sind auch für Probleme mit höherer Komplexität geeignet. Sie sind nur in der Gruppe sinnvoll anwendbar. Im Folgenden werden diese Kreativitätstechniken genauer dargestellt.

9.2.3.2 Brainstorming

Das Konzept des *Brainstorming* wurde von A. F. Osborne entwickelt und geht von der Annahme aus, dass jeder Mensch sein Kreativitätspotenzial auch in sachfremden Bereichen nutzbringend einsetzen kann (vgl. [Backerra *et al.*, 2002]). Voraussetzung für die Durchführung von Brainstorming-Sitzungen ist eine präzise Problemdefinition und eine geringe Komplexität des Problems. Im Rahmen einer Sitzung der Kreativitätsgruppe werden unter Leitung des Moderators Problemgliederungen und Lösungsvorschläge erarbeitet (*Ideengenerierungsphase*). Dabei sind folgende Regeln einzuhalten:

- Während der Sitzung ist keine Kritik erlaubt. Gegenüber eigenen und fremden Ideen muss eine positive Einstellung vorherrschen; auch abwegig erscheinende Problemlösungen dürfen nicht kritisiert werden.³
- Jeder darf (und soll) frei mit seinen Gedanken spielen. Die Problemlösungen sollen möglichst originell und neuartig sein. Vernunft und Logik sind zunächst nicht gefragt.
- Das Ziel ist es, möglichst viele Ideen zu produzieren (vorerst ohne Beachtung der „Qualität“ bzw. der „Sinnhaftigkeit“ der Ideen).
- Die Ideen eines Mitglieds sollen durch andere weiterentwickelt werden.

In einem vorgegebenen Zeitraum (10 bis 15 Minuten) produziert die Kreativitätsgruppe unter Einhaltung der vereinbarten Regeln eine Anzahl von Ideen durch freie gedankliche Assoziation und Zurufe zum Moderator. Der Moderator dokumentiert die Ideen, motiviert und steuert den gruppendynamischen Prozess. Das Ergebnis dieses Arbeitsschritts ist eine ungeordnete Menge von Ideen zur Problemlösung. In einer Abschlussrunde (*Ideenbewertungsphase*) werden dann diese Ideen in „brauchbar“, „vielleicht brauchbar“ und „unbrauchbar“ gegliedert, wobei unter anderem Termine, Kosten und Auswirkungen als Beurteilungskriterien gelten.

³Bei großen interdisziplinären Kreativitätsgruppen empfiehlt es sich, situationsbezogene Verhaltensregeln zu vereinbaren, die zur Regelung des Gruppenprozesses beitragen. Beispiele dafür sind: andere Gruppenmitglieder nicht unterbrechen, zuhören können, Aggressionen vermeiden, Störungen sofort aufarbeiten.

Jede Idee wird dabei von der Kreativitätsgruppe unter der Leitung des Moderators verbal, ordinal oder durch Vergabe von Punkten anhand der Kriterien bewertet.

Die Vorteile dieses Verfahrens sind:

- Es werden viele Ideen in kurzer Zeit produziert.
- Verschiedene Personen sehen Probleme aus verschiedener Sichtweise und produzieren daher sehr unterschiedliche Ideen.
- Das Verfahren ist einfach.

Die Nachteile dieses Verfahrens sind:

- Manche Ideen sind realitätsfremd, da sie von fachlich nicht kompetenten Mitarbeitern eingebracht werden.
- Der Verlauf der Ideenfindung kann das Ergebnis nachhaltig beeinflussen (d. h., die Gruppe kann sehr leicht in eine bestimmte Richtung abdriften, die nicht unbedingt den Kern des Problems darstellt).

9.2.3.3 Brainstorming mit Osborne-Verfremdung

Osborne hat sein Brainstorming-Verfahren um eine so genannte *Verfremdungsphase* erweitert, die zwischen Ideengenerierungsphase und Ideenbewertungsphase eingeschoben wird. Die in der Generierungsphase gewonnenen Ideen zur Problemlösung werden dabei mit Hilfe der *Osborne-Verfremdung* verändert (vgl. Abbildung 9.1, adaptiert aus [Backerra *et al.*, 2002]).

Die Prüfliste für Verfremdungen nach Osborne umfasst vier Gruppen der *Gesetze der Verfremdung*:

1. Gesetze der Assoziation (kombinieren, anders verwenden),
2. Gesetze der Ähnlichkeit (anpassen, kleiner machen, größer machen),
3. Gesetze der Abwandlung (abändern, ersetzen, neu anordnen),
4. Gesetz des Gegensatzes (umkehren).

Einzelne Ideen können miteinander kombiniert, angepasst, neu angeordnet, ersetzt, vergrößert, verkleinert, umgekehrt oder anders verwendet werden. Sowohl einzelne Verfremdungsmöglichkeiten als auch Kombinationen können angewendet werden.

9.2.3.4 Synektik

Bei der Methode der *Synektik* wird ein Problem zuerst spontan definiert (“problem definition as seen”). Durch Fragen zu dieser Definition wird das Problem danach umformuliert, bis die Problemdefinition zufrieden stellend erfolgt ist. Bedingt durch die Wichtigkeit geeigneter

Fragen (Warum, Wie, Wozu etc.) wird diese Methode auch *W-Technik* genannt. Die Synektik legt ihren Schwerpunkt auf die Problemdefinition, Lösungen werden nur erarbeitet, solange sie im Rahmen der Problemumformulierung artikuliert werden. Die Vorgehensweise dieser Methode kann in folgende Schritte untergliedert werden:

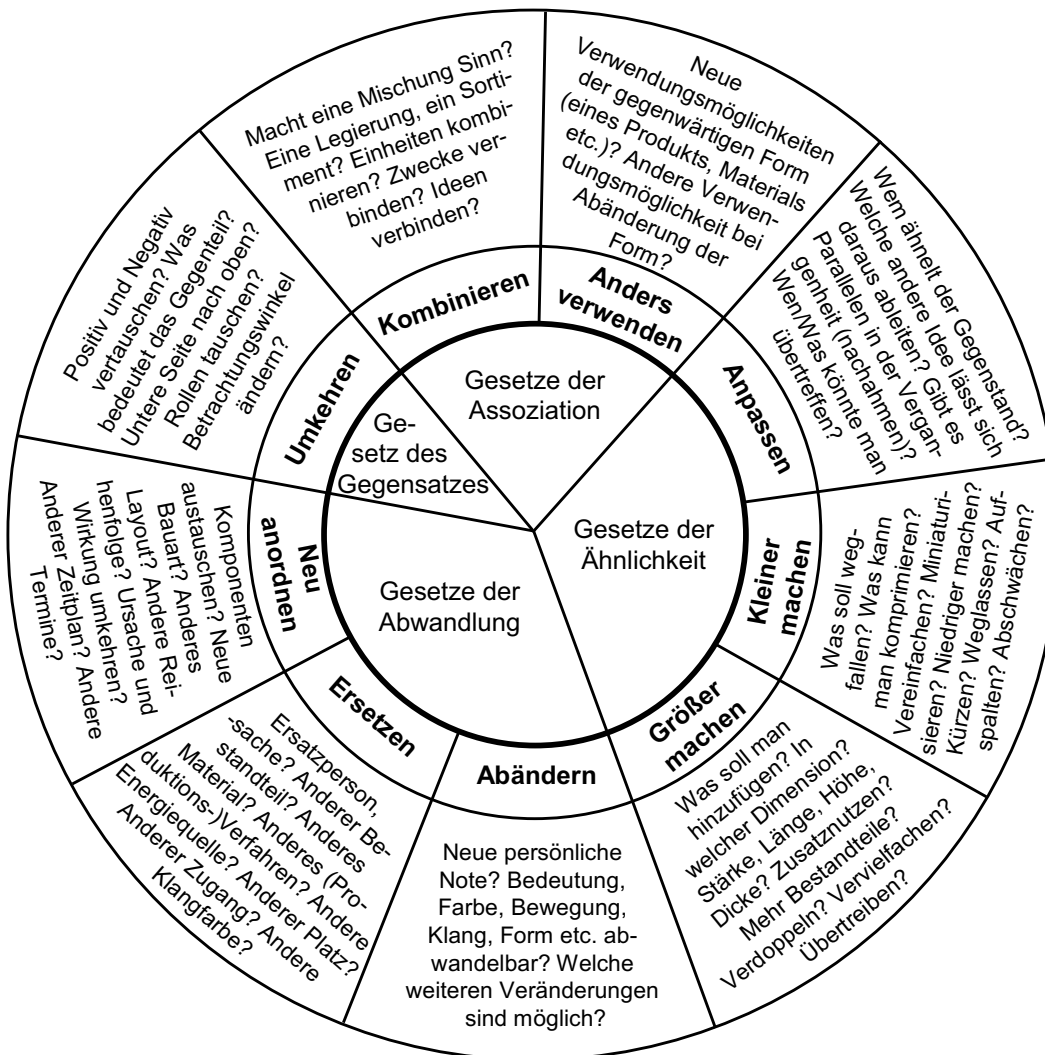


Abbildung 9.1: Die Osborn-Verfremdung im Rahmen des Brainstorming

1. **Spontandefinition:** Das Problem wird spontan definiert, wie es zunächst symptomatisch wahrgenommen wird. Die Problemdefinition wird erleichtert, wenn sie mit „Wie kann ich erreichen, dass ...“ eingeleitet wird.
2. **Umformulierung:** Die Spontandefinition wird zu einer Frage umformuliert. Die Frage wird mit „Warum ...“ eingeleitet.
3. **Antwortenermittlung:** Zur Frage aus dem zweiten Schritt werden durch freie gedankliche Assoziationen drei einfache, griffige Antworten ermittelt.
4. **Problemdefinition:** Die drei Antworten aus dem dritten Schritt werden zu neuen Problemdefinitionen unter Verwendung der Einleitung „Wie kann ich erreichen, dass ...“ umformuliert.

5. **Eignungsprüfung:** Es wird geprüft, ob eine der im vierten Schritt gefundenen Problemdefinitionen geeignet ist, das Problem ausreichend (konstruktiv) zu definieren. Ist dies nicht der Fall, kann durch Verallgemeinerung einer oder mehrerer Problemdefinitionen aus dem vierten Arbeitsschritt eine neue Problemdefinition gefunden werden.
6. **Rückkopplung:** Sind alle in Schritt 5 gefundenen Problemdefinitionen unbefriedigend, wird eine neue Iteration mit den drei im vierten Schritt generierten Problemdefinitionen durchgeführt. Hieraus ergeben sich neun neue Problemdefinitionen, mit denen wiederum analog verfahren wird.

Erfahrungsgemäß wiederholen sich bei steigender Iterationsanzahl immer mehr Problemdefinitionen. Dies wird als Konvergenz der Synektik betrachtet, d. h., die ermittelten Problemdefinitionen werden als optimal betrachtet und der Prozess wird abgebrochen.

Die Vorteile dieses Verfahrens sind:

- Man beginnt bei einer (Spontan-)Problemdefinition. Die Gefahr, vollständig am Ziel vorbei zu arbeiten, ist damit weitgehend ausgeräumt.
- Das Verfahren ist auch bei größeren Problemen anwendbar, da man immer einen Gedanken verfolgt und variiert, die Gefahr unüberschaubar vieler Alternativen wird also durch die Methode vermieden.

Der Nachteil dieses Verfahrens ist:

- Es werden mehrere Personen benötigt, die mit dem Fachgebiet vertraut sein müssen.

9.2.3.5 Morphologischer Kasten

Beim *Morphologischen Kasten* wird systematisch eine Vielzahl von Lösungsmöglichkeiten gesucht. Das Problem (Objekt) wird in Teilprobleme (Parameter) zerlegt; anschließend werden für jeden Parameter unterschiedliche mögliche Werte (Ausprägungen) festgehalten.

Die Vorgehensweise dieser Methode kann in folgende Schritte untergliedert werden:

1. Das Problem wird präzise definiert und beschrieben, gegebenenfalls verallgemeinert.
2. Die Problemelemente werden bestimmt (Zerlegung).
3. Für jedes Element werden kreative Lösungen gesucht.
4. Die Lösungsmöglichkeiten werden untereinander kombiniert und bewertet.
5. Die bestmögliche Lösung wird ausgewählt und realisiert.

Die Vorteile dieses Verfahrens sind:

- Es fördert systematisches Vorgehen.
- Es verhindert voreilige Entscheidungen.

- Das fachübergreifende Arbeiten wird erleichtert.
- Die Lösung wird klar dargestellt.

Die Nachteile dieses Verfahrens sind:

- Bei zu komplexen Aufgabenstellungen besteht die Gefahr der Unübersichtlichkeit.
- Die Auswahl der optimalen Lösung kann sich schwierig gestalten.

9.2.4 Werkzeuge

9.2.4.1 Groupware

Traditionellerweise wird gerade in der Phase der Zielfindung (wie auch der Erhebung) neben der verbalen Kommunikation verstärkt mit klassischen Dokumentationsmitteln gearbeitet. Diese umfassen neben Bleistift und Papier auch Tafel, Overheadprojektoren mit Folien, Flip-Charts u. dgl. Diese Werkzeuge sind nahe liegend und sehr sinnvoll für erste Vorbereitungsarbeiten, weisen aber Schwächen auf in der Adaptierbarkeit und Änderungsfreundlichkeit (geschriebene Texte), in der Dokumentablage (Text auf der Tafel) oder in der Anwendbarkeit für Telekonferenzen.

Die Computerunterstützung der Phasen der Erhebung und der Zielfindung ist Aufgabe des Forschungsgebiets *computer supported cooperative work (CSCW)*. Ziel ist es, die Zusammenarbeit von Menschen durch den Einsatz von Informations- und Kommunikationstechnik zu verbessern, d. h., effizienter und flexibler, aber auch humaner und sozialer zu gestalten. Derartige Softwaresysteme zur Unterstützung kooperativen Arbeitens werden unter dem Begriff *Groupware* bzw. *Groupwaresysteme* (auch im Deutschen) zusammengefasst. Dieser Begriff lässt sich von Bürokommunikationssystemen dadurch abgrenzen, dass im Bereich der Bürokommunikation v. a. Prozesse der Übertragung (Kommunikation) sowie der Verbreitung und Speicherung von Informationen im Mittelpunkt stehen, während bei Groupwaresystemen in erster Linie Aspekte der Zusammenarbeit und Koordinationsprobleme betrachtet werden, wobei Kommunikation als Mittel zur Realisierung der Koordination zwischen Aufgabenträgern angesehen wird [Favela und Decouchant, 2004].

Folgende Anforderungen müssen *Groupwaresysteme* erfüllen:

- **Effizienz:** Entscheidungsprozesse sollen beschleunigt, Koordinationstätigkeiten vereinfacht und die Qualität der Entscheidungen verbessert werden.
- **Transparenz:** Das System muss für den Menschen durchschaubar bleiben, d. h., der Mensch muss die Systemprozesse verstehen können, und die Möglichkeit zur Kontrolle der durch das System unterstützten Aktionen muss laufend gegeben sein.
- **Offenheit und Integrierbarkeit:** Gerade in der Anfangsphase eines Projekts ist Information in verschiedenster Form und aus verschiedensten Quellen verfügbar. Existierende Informations- und Kommunikationssysteme müssen eingebunden werden können.
- **Angemessenheit:** Aus einer aktiven Systemunterstützung darf keine „Bevormundung“ des Benutzers resultieren.

Folgende Komponenten können beispielsweise in einem Groupwaresystem enthalten sein (vgl. [Favela und Decouchant, 2004]):

- Erstellung der Tagesordnung (Agenda),
- elektronisches Brainstorming (electronic brainstorming),
- Kategorisierer (categorizer) zu Listenerstellung,
- Abstimmung (vote),
- Themenkommentator (topic commenter) zur Ideensammlung,
- Gliederungsentwurf (group outliner),
- Umfrage (survey),
- Alternativenbewertung (alternative analysis),
- Utensilien (briefcase utilities) wie Taschenrechner, Kalender, Uhr, Notizblock,
- Ereignismonitor (event monitor) zum Festhalten der Tagesordnungsaktivitäten,
- persönliches Tagebuch (personal log),
- Meinungsbarometer (opinion meter),
- Informationsmaterial-Verteiler (handout dispatcher),
- elektronische Tafel (whiteboard),
- Teilnehmerinformation (people info),
- Aktivitäten-Modellierer (activity modeler).

Folgende Vorteile ergeben sich durch den Einsatz von Groupwaresystemen:

- Alle Teilnehmer können gleichzeitig und gleichberechtigt ihre Ideen und Meinungen äußern; paralleles Arbeiten wird ermöglicht.
- Per Telekonferenz/Computernetzwerk können wichtige dislozierte Teilnehmer (Experten u. dgl.) zugeschaltet werden.
- Der Besprechungsprozess wird besser strukturiert und genauer nachvollziehbar.
- Informationen können auch noch während einer Besprechung recherchiert werden.
- Die Anonymität bewirkt größere Offenheit sowie Unabhängigkeit von Macht- und Statusunterschieden.
- Eine synchrone Sitzungsprotokollierung wird ermöglicht. Ideen, Abstimmungsergebnisse etc. lassen sich speichern und können in der gleichen oder einer weiteren Sitzung wieder verwendet werden.
- Ergebnisse und Zwischenergebnisse sind besser dokumentiert und weiter verwendbar.

Dem ist entgegenzuhalten, dass sich derzeitige Groupwaresysteme noch stark in Entwicklung befinden. Daraus ergeben sich eine mangelnde Normung der Bedienung, des Funktionsumfangs und der Abläufe sowie eine sehr uneinheitliche Art und Gestaltung der erstellten Dokumente.

9.2.4.2 Die Unified Modeling Language (UML)

Nach der enormen Methodenvielfalt zu Beginn der neunziger Jahre (die zum „*Methodenkrieg*“ führte) entstand durch die Methodenentwickler selbst, vor allem aber durch die Anwender, die für sich oft nur unwesentlich unterscheidende Methoden jeweils andere Darstellungsarten erlernen mussten, der Drang nach Vereinheitlichung. Dies wurde im Wesentlichen dadurch erreicht, dass die Firma Rational (jetzt IBM) die Hauptakteure Grady Booch, James Rumbaugh und Ivar Jacobson zusammenführte. Diese „Drei Amigos“ vereinheitlichten ihre Methoden und ergänzten sie zur *Unified Modeling Language (UML)*, vgl. z. B. [Oestereich, 2004].

Die Bestrebungen zur Vereinheitlichung wurden bestärkt durch die Wahl von UML zur *Standardbeschreibungssprache für Softwareentwicklungen* durch die Object Modeling Group (OMG). Mit der UML wurde jedoch nur eine Standardsprache, eine Notation festgelegt. Der zur Erweiterung zu einer Methode notwendige Prozess ist nicht spezifiziert. Die aktuelle, offiziell verfügbare Version stellt *UML 2* dar, vgl. z. B. [Oestereich, 2004] für eine Einführung.

Die UML umfasst in ihrem Problembereichsmodell die Phasen der Zielerforschung, Analyse und Design, für die sie jeweils entsprechende Modelle bereithält, die mit einer Vielzahl von Diagrammen dargestellt werden können. Dabei ist zu bedenken, dass viele der Modelle in mehreren Phasen Verwendung finden können (und sollen) und entsprechend bei den Diagrammen auch phasenübergreifende Zusammenhänge dargestellt werden können. Folgende Modelle werden in der UML unterschieden (Abbildung 9.2)⁴:

- **Problembereichsmodell:** Das Problembereichsmodell repräsentiert das konzeptionelle Modell des Problemereichs in einer für den Auftraggeber verständlichen Sprache und Gliederung. Es dient der Kommunikation zwischen Auftraggeber und Auftragnehmer in Zielerforschung, Analyse und Design und umfasst die Modelle für diese drei Phasen. Implementierungsspezifische Details sind daher noch nicht enthalten.
- **Zielerforschungsmodell:**
 - **Anwendungsfallmodell (use case model):** Das Anwendungsfallmodell dient der Beschreibung des gewünschten Systemverhaltens aus Anwendersicht in Form von Anwendungsfällen (use cases). Für Details wird auf Abschnitt 9.2.4.3 verwiesen.
- **Analysemodell:** Das Analysemodell beschreibt die *logische Sicht (logical view)* des Systems aus Anwendersicht (siehe Abschnitt 10.1.4.2 für Details). Es umfasst:
 - **Strukturmodell (structure model):** Das Strukturmodell beschreibt die Systemarchitektur aus Anwendersicht unter Berücksichtigung der vorgegebenen Be-

⁴Die englischen Begriffe wurden von den UML-Entwicklern im Wesentlichen einheitlich festgelegt; im Deutschen gibt es Bestrebungen, einheitliche Übersetzungen festzulegen, vgl. [Oestereich, 2004]. Die hier verwendeten deutschen Begriffe berücksichtigen diese Vereinheitlichungsbestrebungen.

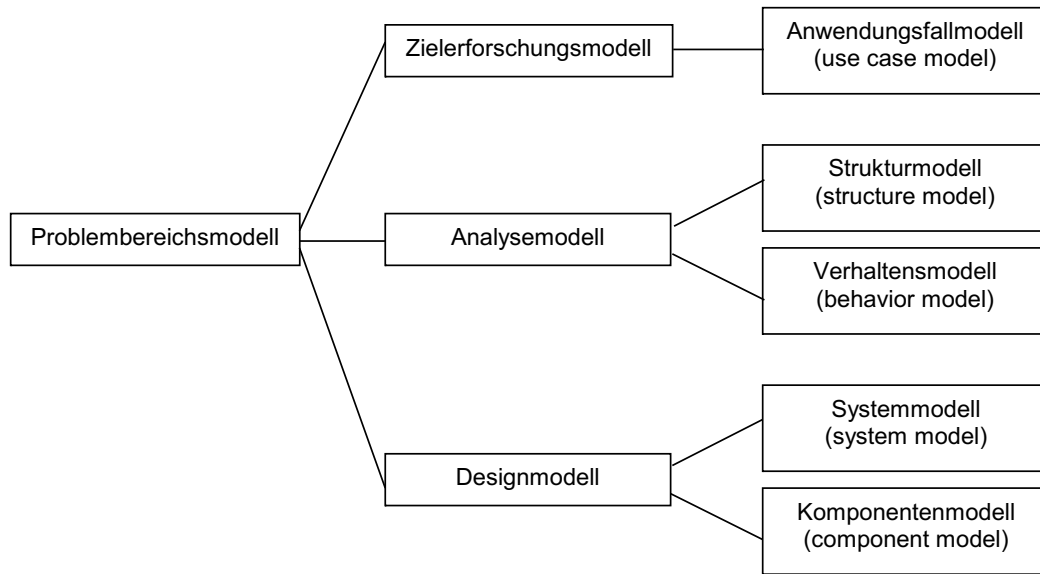


Abbildung 9.2: Modelle der UML

schränkungen. Designentscheidungen dürfen dabei noch nicht vorweggenommen werden. Komplexe Strukturmodelle können in UML in Pakete unterteilt werden.

- **Verhaltensmodell (behavior model):** Das Verhaltensmodell legt durch Definition von Verantwortlichkeiten und Interaktionsstruktur das systemglobale Verhalten fest (v. a. das nach außen wirksame). Lokales Verhalten wird, so es nicht im Einzelfall eine Anforderung explizit verlangt, erst im Design spezifiziert.
- **Designmodell:** Das Designmodell beschreibt die *physische Sicht (physical view)* des Systems aus Entwicklersicht. Für Details wird auf Abschnitt 11.4.2 verwiesen.

9.2.4.3 Diagramme der UML für die Zielerforschung

Die *Anwendungsfallmodellierung* stellt die durch das Produkt verwirklichten Ziele in Form von Anwendungsfällen dar („use case-driven“). Laut [Oestereich, 2004] ist ein *Anwendungsfall (use case)* ein Dialog (i. e. eine Abfolge von Transaktionen) zwischen dem zu erstellenden System und einem Anwender, dem *Akteur (actor)*. Auf diese Weise wird festgelegt, wie das System mit seiner Umwelt kommuniziert. Das System wird funktional in konkrete Anwendungsfälle zerlegt, die durch (außerhalb des Systems stehende) Akteure initiiert werden, indem sie das System benützen (*aktive Akteure*) oder vom System benützt werden (*passive Akteure*, bei denen das System die Anwendungsfälle initiiert).

Die Anwendungsfälle beschreiben einen Ausschnitt der Kundenanforderungen aus deren Sicht und in deren Ablauf und Umfeld. Die Beschreibung erfolgt ausgerichtet auf das Projektziel (Was wird mit dem zu erstellenden Produkt bezweckt?) und nicht lösungsorientiert.

Durch das Beschreiben von gewünschtem Systemverhalten sowie der Identifikation von Kundenwünschen lässt sich eine textuelle Zielbeschreibung erstellen. Diese Beschreibung stellt im Allgemeinen keine vollständige Systembeschreibung dar, auf Konsistenz zwischen den beschriebenen Anwendungsfällen ist jedoch zu achten.

Das *Anwendungsfalldiagramm* (*use case diagram*) ist ein Diagramm der UML zur Darstellung von Anwendungsfällen. Es dient der Beschreibung des gewünschten Systemverhaltens aus Anwendersicht. Ein Anwendungsfalldiagramm beschreibt eine Sammlung zusammenhängender Anwendungsfälle, die während der Zielfindungsphase generiert werden. Eine Zielbeschreibung kann mehrere Anwendungsfalldiagramme umfassen. Solange weitere Anwendungsfälle gefunden werden, werden die entsprechenden Anwendungsfalldiagramme erweitert und adaptiert.

Ein *Anwendungsfallmodell* umfasst Anwendungsfalldiagramme (in Paketen strukturiert), Anwendungsfallbeschreibungen in textueller Form für jeden Anwendungsfall sowie Schnittstellenbeschreibungen zu jedem Akteur. Diese Beschreibungen werden nachfolgend detailliert dargestellt.

9.2.4.4 Vorgehen bei der Zielerforschung unter Verwendung der UML

Darstellung des Anwendungsfalldiagramms

Ein Anwendungsfalldiagramm wird in UML-Notation wie folgt modelliert (Abbildung 9.3):

- Ein Anwendungsfall wird durch eine Ellipse dargestellt, die als Text eine prägnante Kurzbeschreibung enthält. Eine prozessorientierte Beschreibung (ein Wortpaar Substantiv-Verb, z. B. „Bericht drucken“) ist dafür sehr empfehlenswert.
- Für den Akteur gibt es zwei Darstellungsarten. Ein Akteur, der eine von einem Menschen übernommene Rolle darstellt (*menschlicher Akteur*), wird als stilisiertes Männchen dargestellt; als Bezeichner wird der Rollename angegeben. Ein *maschineller Akteur* (im Allgemeinen eine andere Softwarekomponente; aber auch ein Hardwaregerät etc.) wird durch ein Rechteck dargestellt, das neben dem Schlüsselwort `<actor>` die Funktion des maschinellen Akteurs enthält.
- Ein System wird durch eine stilisierte Karteikarte dargestellt, die im Bezeichnerfeld nach dem Schlüsselwort `<system>` den Systemnamen enthält. Das Rechteck der Karteikarte stellt die Systemgrenzen dar. Akteure befinden sich außerhalb des Systems und werden daher neben die Karteikarte gezeichnet.
- Ein komplexes System kann in Subsysteme gegliedert werden, die als benannte Karteikarte, jedoch ohne weitere Details, in der Karteikarte des Supersystems dargestellt werden. Die Details des Subsystems werden in seiner eigenen Karteikarte beschrieben. Diese hierarchische Gliederung in Supersysteme und Subsysteme wird in UML als *Paketierung* (*packaging*) bezeichnet, die einzelnen, nichtatomaren Systeme werden als *Pakete* (*packages*) bezeichnet.

Bei der Paketierung ist darauf zu achten, dass die Gliederung in Subsysteme überschneidungsfrei erfolgt, also eine Partitionierung erfolgt. Partitionierungskriterien können funktionale Abhängigkeiten sein, aber auch von einem Akteur ausgelöste Anwendungsfälle, Verwendung von maschinellen Akteuren in Form von kritischen Ressourcen etc.

- Anwendungsfälle können zueinander auf zweierlei Weise in Beziehung gesetzt werden (die Leserichtung entspricht der Pfeilrichtung):

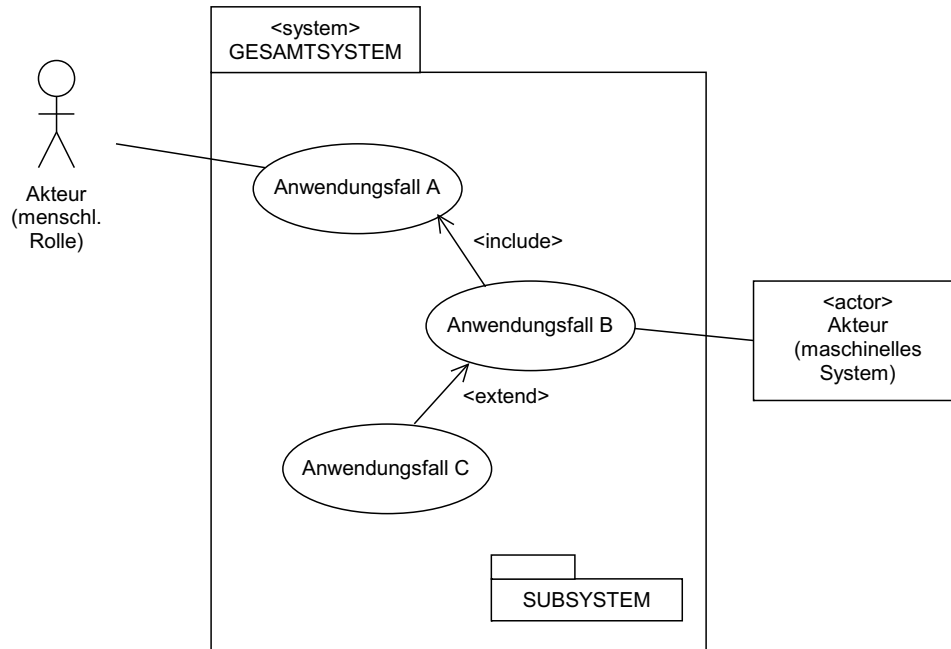


Abbildung 9.3: Komponenten eines Anwendungsfalldiagramms in UML-Darstellung

1. Ein Anwendungsfall kann das Verhalten eines anderen verwenden (*Enthält-Beziehung*; Schlüsselwort **<include>**). Der enthaltene Anwendungsfall muss in diesem Fall jedenfalls vorhanden (modelliert) sein, um den enthaltenden zu ermöglichen.
2. Ein Anwendungsfall kann das Verhalten eines anderen erweitern (*Erweitert-Beziehung*; Schlüsselwort **<extend>**). Diese Erweiterung kann zur Spezialisierung des erweiterten Anwendungsfalls verwendet werden, dieser ist aber auch ohne Erweiterung verwendbar.

Strukturierung einer Anwendungsfallbeschreibung

Die *Anwendungsfallbeschreibung* ist eine detaillierte Beschreibung in Freitext, die für jeden Anwendungsfall erstellt wird. Obwohl in der UML nicht speziell geregelt, wird in [Cockburn, 1997] zur Vereinheitlichung eine formilere Beschreibung vorgeschlagen, die Folgendes beinhaltet:

- Name,
- Kurzbeschreibung,
- Vorbedingung (Voraussetzung für einen erfolgreichen (Standard-)Ablauf),
- Nachbedingung (Systemzustand nach erfolgreichem (Standard-)Ablauf),
- Fehlerfälle (inkl. Systemzustand je Fehlerfall),
- Akteure (die mit dem Anwendungsfall kommunizieren),

- Trigger (Auslöseereignisse für den Anwendungsfall),
- Standardablauf (aufgelöst in Schritte, Beziehung zu anderen Anwendungsfällen),
- Alternativabläufe (mögliche Abweichungen vom Standardablauf).

Die Anwendungsfallbeschreibung ist jedenfalls in der Sprache des Kunden und für diesen transparent zu erstellen.

Schnittstellenbeschreibungen zu Akteuren

Eine *Schnittstellenbeschreibung* soll während der Phase der Zielfindung nur in dem Umfang erfolgen, in dem sie zur Klärung der Interaktion der Akteure mit dem System beiträgt. Im Detail erfolgt die Schnittstellenbeschreibung in der Analyse und, vor allem, im Design.

Für jeden Akteur, der einer menschlichen Rolle entspricht, wird eine *Benutzerschnittstelle* spezifiziert (falls notwendig für jeden Anwendungsfall). Dies wird in UML nicht explizit unterstützt. Von graphischen Skizzen bis hin zu Prototypingwerkzeugen ist jede Computerunterstützung zugelassen und sinnvoll.

Die Spezifikation von maschinellen *Systemschnittstellen* erfolgt in UML mittels der Darstellung von *Schnittstellen*, vor allem im *Komponentendiagramm* (für Details siehe Abschnitt 11.4.2).

9.2.5 Ergebnisse

9.2.5.1 Zielbeschreibung

Die *Zielbeschreibung* ist ein Bericht des Auftragnehmers an den Auftraggeber, der die Beschreibung des Gesamtziels als Problemdefinition mit skizzierter Lösung darstellt. Er fasst somit das Ergebnis der Kreativitätstechniken zusammen. Eine lösungsbezogene Strukturierung ist nicht enthalten. Anwendungsfalldiagramme können in der Zielbeschreibung verwendet werden, sofern diese Darstellung für den Auftraggeber verständlich ist.

Die Zielbeschreibung gibt die Sichtweise des (potenziellen) Auftragnehmers wieder und kann somit kein für das Projekt ursächlich bindendes Dokument darstellen. Bei manchen Projekten stellt der Auftraggeber dem Auftragnehmer eine (extern oder intern erstellte) Zielbeschreibung zur Verfügung, die vom Auftraggeber geprüft und adaptiert wurde. Dieses Dokument kann dann eine verbindliche Projektgrundlage darstellen.

9.2.5.2 Datenlexikon

Das *Datenlexikon* (*Datenverzeichnis*, *data dictionary*⁵; auch *Datenkatalog* genannt) ist ein Verzeichnis aller im Projekt relevanten Fachbegriffe (Terme, Konzepte, Beschränkungen,

⁵Der Begriff „*repository*“ ist eine herstellereigenspezifische Bezeichnung für ein Datenlexikon (vgl. [Heinrich und Roithmayr, 1992]).

Komponenten, Klassen/Objekte, Nachrichten, Methoden, Variablen etc.). Zur Verwaltung eines Datenlexikons werden Werkzeuge benötigt, die Funktionen zum Eintragen, Löschen, Suchen und Verändern von Eintragungen bereitstellen. Datenlexika größeren Umfangs werden hierarchisch aufgebaut, mit entsprechenden Querverweisen versehen und in einem Datenbanksystem verwaltet (vgl. [Heinrich und Lehner, 2002]). Auch eine mehrbenutzerfähige Versionsverwaltung ist unumgänglich (vgl. die Ausführungen zur Dokumentation in Abschnitt 7.2).

Viele CASE-Systeme unterstützen die Erstellung und Aktualisierung eines Datenlexikons, in dem aus den verschiedenen Diagrammen automatisch alle Bezeichner (und, soweit möglich, deren Beschreibungen) extrahiert werden. Bei der Verwendung großer Werkzeugvielfalt ergibt sich jedoch oft das Problem, welches Datenlexikon das zentrale sein soll (kann) und wie mit diesem Daten ausgetauscht werden können.

Die Erstellung eines Datenlexikons ist bereits im Rahmen der Zielerforschung zu beginnen, während der weiteren Projektentwicklungsphasen sukzessive fortzuführen, und das Datenlexikon ist laufend zu aktualisieren. Für den Auftraggeber relevante Einträge des Datenlexikons werden in Form eines Glossars ins Lastenheft bzw. ins Pflichtenheft übernommen sowie in die Produktdokumentation eingebunden.

10 Analyse

Dieses Kapitel beschreibt die Hauptabschnitte der Analysephase eines Projektes: die Anforderungsanalyse, die (Detail-)Planung sowie die Risikobehandlung. Den verschiedenen Analysetechniken sowie deren sinnvollen Einsatzmöglichkeiten in modernen Softwareprojekten wird besonderes Augenmerk geschenkt.

10.1 Anforderungsanalyse

10.1.1 Ziel

Bereits Descartes bemerkte 1637: „Man soll jede schwierige Frage in so viele Teilfragen als möglich zerlegen, damit man jede einzelne lösen kann.“ Heute bezeichnet man diesen Grundsatz als Prinzip der *hierarchischen Strukturierung* oder auch als Konzept der *Problemzerlegung* bzw. als *Analyse* (= zergliedern, in Einzelteile zerlegen) der Anforderungen (*Anforderungsanalyse*, *Bedarfsanalyse*, *requirements analysis*; im Folgenden vielfach oft nur kurz *Analyse* genannt). Diese Phase wird manchmal auch als *Systemspezifikation* bezeichnet, vgl. z. B. [Pomberger und Blaschek, 1996]. Diese Bezeichnung kann jedoch zu Missverständnissen führen, da im Englischen mit *system specification* das Pflichtenheft bezeichnet wird (vgl. dazu Kapitel 11) und die *Problemspezifikation* der Mathematik eine formal korrekte Problembeschreibung darstellt.

Eine *Anforderung* (*requirement*) beschreibt eine zu erfüllende Eigenschaft oder zu erbringende Leistung eines Systems. Eine Anforderung ist daher „... eine Fähigkeit, die eine Software erfüllen oder besitzen muss, um einen Vertrag, eine Norm oder ein anderes, formell bestimmtes Dokument zu erfüllen“ [IEEE, 1991]. Häufig wird zwischen *funktionalen* und *nichtfunktionalen Anforderungen* unterschieden. Funktionale Anforderungen betreffen die Dienste eines Systems und werden daher auch *explizite Anforderungen* genannt. Nichtfunktionale Anforderungen (*implizite Anforderungen*) beschreiben Qualitätsaspekte oder auch Termin- und Kostenziele [Grünbacher, 2003]. Dabei ist zu beachten, dass nichtfunktionale Anforderungen vielfach vom Kunden nicht geäußert, aber trotzdem erwartet werden. So wird ein Kunde nicht explizit erwähnen, dass ein Softwaresystem stabil laufen muss; dies setzt er voraus!

Im Bereich des Software Engineering stammt die erste Definition der *Anforderungsanalyse* von DeMarco [DeMarco, 1978]: „Analyse ist das Studieren eines Problems, noch bevor Aktionen getätigt werden.“ In [Coad und Yourdon, 1990] wird Analyse beschrieben als „das Studieren eines Problemraumes, das zu einer Beschreibung des extern beobachtbaren Systemverhaltens führt, eine vollständige, konsistente und brauchbare Beschreibung dessen, was benötigt wird“.

In der Analyse steht daher der Prozess des Extrahierens der Notwendigkeiten eines Systems im Vordergrund, also WAS ein System zur Erfüllung der Kundenanforderungen leisten muss und *nicht*, WIE das geschieht (*WAS/WIE-Trennung*). Zu jeder geforderten Notwendigkeit ist es jedoch im Hinblick auf spätere Betrachtungen entscheidend festzuhalten, WARUM diese Notwendigkeit gefordert wird.

10.1.2 Aufgaben

10.1.2.1 Tätigkeiten

Die wichtigsten Tätigkeiten der Anforderungsanalyse sind (vgl. [Pomberger und Blaschek, 1996]):

- die Erhebung des Istzustands und des Sollzustands mit Abgrenzung des Problemereichs; diese erfolgt nun lösungsorientiert, aufbauend und in Erweiterung der Zielerforschung,
- die grobe Skizzierung der Architektur und der Komponenten des geplanten Systems,
- die Feststellung der Machbarkeit, i. e. der Lösbarkeit des Problems durch den Auftragnehmer,
- eine erste Abschätzung des Projektumfangs und der Wirtschaftlichkeit des geplanten Projekts sowie die Erstellung eines groben Projektplans.

Anforderungen müssen einer *Validierung* („Haben wir das Richtige festgelegt?“) sowie einer *Verifikation* („Haben wir es richtig festgelegt?“) unterzogen werden. Dabei können verschiedene Techniken wie etwa Reviews, Inspektionen oder Prototyping zum Einsatz kommen, vgl. [Halling *et al.*, 2003]. In der iterativ-inkrementellen Entwicklung erfolgt dies laufend parallel zur Entwicklung (vgl. Abschnitt 6.4.4).

10.1.2.2 Eigenschaften

Folgende Eigenschaften charakterisieren dabei eine gute Anforderungsanalyse:

- **Modularität:** Eine Analyse muss modular sein. Das hat sowohl technische als auch organisatorische Gründe. Vom technischen Standpunkt aus ist bekannt, dass ein modulares System leichter zu entwerfen und zu warten ist als ein monolithisches. Unter organisatorischen Gesichtspunkten ist es wichtig, sich von Anfang an bewusst zu machen, dass die Arbeit in der Regel in einer Gruppe durchgeführt werden wird und deshalb so gegliedert sein muss, dass gut trennbare Arbeitsabschnitte verschiedenen Personen oder Untergruppen zugewiesen werden können.

- **Änderbarkeit:** Änderbarkeit bezieht sich nicht nur auf „Änderungen nach Auslieferung“ (*design for change*, siehe [Parnas, 1972]), sondern genauso auf Änderungen, die während der Entwicklung auftreten. Es ist in der Regel so, dass man erst während der Entwicklung Klarheit darüber gewinnt, welche Funktionalität die zukünftigen Benutzer von der in Entwicklung befindlichen Software genau erwarten. Das *Verwalten von Anforderungen* (*requirements management*) unterstützt sowohl Änderungen bestehender Anforderungen als auch das Hinzufügen neuer Anforderungen.
- **Abgrenzung des Projektumfangs, Randbedingungen:** Sehr wesentlich ist die Abgrenzung des Projektumfangs. Man hat nur dann eine Chance, ein Projekt erfolgreich in einem überschaubaren Rahmen abzuwickeln, wenn vorher klar ist, wo die Grenzen der Leistung eines neuen Systems liegen sollen. Auch die Feststellung der Randbedingungen gehört zu einer sachgerechten Systemanalyse. Dabei kann es sich um technische, wirtschaftliche, aber auch soziale Wechselwirkungen des zu entwickelnden Systems mit seiner Umgebung handeln.
- **Realisierbarkeit:** Fast schon trivial mag es erscheinen, die Abschätzung der Realisierbarkeit zu nennen. Gerade sie wird aber oft aus Wunschdenken vernachlässigt, was dann allenfalls zu ineffizienten Lösungen führt, meist aber zum Abbruch des Projekts oder zur Nichtbenutzung des Endprodukts.
- **Schnittstellen:** Eine sauber beschriebene Grobstruktur des zu entwickelnden Systems mit den wesentlichen Komponenten und den Datenflüssen ist das Mindeste, was als Ergebnis einer fachgerechten Systemanalyse vorliegen sollte. Dabei ist besondere Sorgfalt auf eine klare und eindeutige Schnittstellenfestlegung zu verwenden, da diese später eine der entscheidendsten Voraussetzungen für die organisatorische Durchführbarkeit des Projekts ist.

10.1.2.3 Schwierigkeiten

Wie in [Coad und Yourdon, 1990] angemerkt wird, werden die Schwierigkeiten der Anforderungsanalyse sehr oft unterschätzt und überfordern viele, die sich damit auseinander setzen. Folgende Kernproblematiken haben sich in der Praxis ergeben, welche die großen Herausforderungen der Projektanalyse darstellen:

- **Das Verständnis für das Anwendungsgebiet:** Eines der größten Probleme, die es bei der Analyse von Systemen zu bewältigen gilt, ist die Untersuchung des Anwendungsgebiets und das Bestreben, möglichst viel über das zu behandelnde Thema in Erfahrung zu bringen. Dabei ist es notwendig, das Anwendungsgebiet regelrecht zu erforschen. Der Analytiker hat sich so genau mit der ihm gestellten Aufgabe zu befassen, dass er auch Feinheiten und Nuancen entdeckt, die selbst diejenigen noch nicht in Betracht gezogen haben, die täglich in diesem Bereich arbeiten.
- **Die zwischenmenschliche Kommunikation:** Die Herausforderungen der Anforderungsanalyse machen eine effektive Kommunikation unumgänglich. Der Analytiker muss Gespräche
 - mit dem Auftraggeber führen, um Informationen über das Anwendungsgebiet und die Systemaufgaben zu erhalten,

- mit seinen Kollegen und Vorgesetzten führen, um über das Projekt gleichen Wissensstand und gleiches Verständnis zu erlangen,
 - mit dem Auftraggeber führen, um ihm das erworbene Wissen und das gewonnene Verständnis über das Anwendungsgebiet und die daraus folgenden Systemaufgaben näher zu bringen.
- **Sich ständig ändernde Anforderungen:** Die Anforderungen an ein System unterliegen ständigen Veränderungen. Diese Anforderungen werden von vielen Beteiligten bestimmt und/oder beeinflusst:
 - vom Kunden,
 - von der Konkurrenz,
 - vom Gesetzgeber,
 - von Prüfern,
 - vom Fortschritt der Technik.

Analytiker müssen demnach ihre Vorgehensweisen und ihre Notationen so gestalten, dass die Ergebnisse der Anforderungsanalyse bei Veränderungen der ursprünglichen Anforderungen (möglichst) einfach angepasst werden können. [Coad und Yourdon, 1990] stellen dazu fest: „Wir müssen es als eine Tatsache hinnehmen, dass Anforderungen sich ständig verändern, und wir sollten nicht glauben, dies wäre das Ergebnis nachlässiger Arbeit.“

- **Die mangelnde Verfolgbarkeit von Anforderungen im Projekt:** Anforderungen müssen vom Lastenheft/Pflichtenheft über Designmodell und Code bis in die Testfälle verfolgbar sein (*traceability*), insbesondere auch bei und nach Änderungen. In der Praxis ist dazu eine Werkzeugunterstützung in Form von integrierten Entwicklungsumgebungen unumgänglich.
- **Die Wiederverwendbarkeit der Ergebnisse:** Bei der Anforderungsanalyse bedeutet „Wiederverwendung“ das Einbinden früherer Analyseergebnisse in die aktuelle Analyse. Dies ist jedoch vielfach nur domänenspezifisch möglich, vgl. das Konzept der *Analysemuster (analysis patterns)* [Fowler, 1997]. Wieder verwendbare Analyseergebnisse bergen das größte Potenzial, die Systementwicklung zu verbessern, um Menschen in die Lage zu versetzen, festgelegte Systemfähigkeiten innerhalb eines gegebenen Zeit- und Finanzplans zu entwickeln.

10.1.3 Techniken

10.1.3.1 Das Beherrschen der Komplexität

Die in den nächsten Kapiteln vorgestellten Analysetechniken basieren auf nachfolgenden Konzepten¹ für den Umgang mit komplexen Systemen [Kulczycki, 1996]:

¹Diese Konzepte stammen ursprünglich aus der Erforschung des menschlichen Sozialverhaltens im Zeitalter der Aufklärung.

- **Abstraktion:** Abstraktion ist das Prinzip, jene Aspekte des untersuchten Gegenstandes zu ignorieren, die nicht für die Aufgabe relevant sind, um sich dafür umso mehr auf die relevanten Aspekte konzentrieren zu können.

Als *prozedurale Abstraktion* wird jenes Prinzip bezeichnet, in welchem jede Operation, die einen definierten Zweck erfüllt, von ihren Benutzern als eine Einheit gesehen werden kann, obwohl sie selbst vielleicht wieder aus mehreren, untergeordneten Operationen besteht.

Als *Datenabstraktion* wird jenes Prinzip bezeichnet, welches einen Datentyp anhand der Operationen definiert, die auf Objekte dieses Typs angewandt werden. Oft wird zur sauberen Definition verlangt, dass die Werte dieser Objekte nur durch die Ausführung der Operationen verändert oder überwacht werden.

- **Kapselung:** Kapselung ist das „Verstecken“ von Information (*information hiding*) speziell bei jenen Teilen eines Systems, die für Veränderungen während der Entwicklungszeit eines Projekts anfällig sind. Schnittstellen zu Modulen sollen so definiert werden, dass möglichst wenige der inneren Festlegungen nach außen transparent gemacht werden. Der Sinn der Kapselung ist daher das Trennen des Anwenders vom Autor [Parnas, 1972].
- **Vererbung:** Das Prinzip der Vererbung bildet die Grundlage für eine wichtige Technik, nämlich die ausdrückliche Beschreibung von Gemeinsamkeiten. Der Analytiker ist dadurch in der Lage, gemeinsame Attribute und Methoden einmal zu definieren und diese bei Bedarf zu erweitern oder zu verändern.

Die Vererbung kann in der Analyse von Anfang an verwendet werden, um Gemeinsamkeiten ausdrücklich darzustellen. Dies geschieht durch den Aufbau von Hierarchien (so genannten „*Taxonomien*“), innerhalb deren übergeordnete Komponenten (die „Vorfahren“) Eigenschaften besitzen, die alle untergeordneten Komponenten (die „Nachfahren“) erben.

- **Assoziation:** Assoziationen sind Verbindungen von Ideen. Der Mensch verwendet Assoziationen, um bestimmte Dinge, die sich zu einem bestimmten Zeitpunkt oder unter vergleichbaren Umständen ereignen, miteinander in Verbindung zu setzen.
- **Nachrichtenkommunikation:** Die *Kommunikation mittels Nachrichten*, besonders im Zusammenhang mit den beteiligten „Schnittstellen“, ist ein Grundprinzip für den Umgang mit komplexen Problemen. Die Kommunikation mittels Nachrichten entspricht dem Imperativ in der menschlichen Sprache.
- **Organisationsmethodik:** Es gibt drei primäre Organisationsmethoden, die das Denken des Menschen durchziehen:

1. das Unterscheiden von Wahrnehmungen in einzelne Objekte und ihre Eigenschaften,
2. das Unterscheiden zwischen ganzen Objekten und ihren einzelnen Bestandteilen,
3. das Klassifizieren von Objekten und die Unterscheidung zwischen diesen Klassen.

In der Praxis bringt die Kombination aller drei Methoden eine wesentlich größere Einsicht in das betreffende Anwendungsgebiet und in die Aufgaben des Systems, als wenn man nur eine dieser Methoden anwenden würde.

- **Verhaltenskategorisierung:** Eine Klassifizierung gemäß „Ursache und Wirkung“ erlaubt häufig das Ableiten von Ähnlichkeiten. Folgende Klassifizierungen sind möglich:
 - die Verhaltensklassifizierung auf Grund unmittelbarer Verursachung,
 - die Verhaltensklassifizierung auf Grund einer ähnlich evolutionären Geschichte (Veränderungen im Laufe der Zeit),
 - die Verhaltensklassifizierung auf Grund der Ähnlichkeit der Funktionalität.

Die im Folgenden besprochenen Zerlegungstechniken,

- funktionale Dekomposition,
- strukturierte Analyse,
- Information Modeling sowie
- objektorientierte Zerlegung,

stellen die historische Entwicklung der Analyse dar und bedeuten somit nicht einen Katalog von Methoden, aus denen äquivalent gewählt werden kann. Im Allgemeinen sind zuerst genannte Methoden in späteren enthalten bzw. in der breiten Anwendung später genannten Methoden unterlegen, vgl. Abbildung 10.1. Für spezielle Anforderungen ist es hingegen durchaus sinnvoll, eine Analyse rein funktional orientiert (funktionale Dekomposition) oder datenorientiert (strukturierte Analyse) durchzuführen.

Komplexitäts-kriterium	Zerlegungstechnik			
	Funktionale Dekomposition	Strukturierte Analyse	Information Modeling	Objektorientierte Zerlegung
Abstraktion	✓	✓	✓	✓
Kapselung	✓	✓	✓	✓
Vererbung				✓
Assoziation		✓	✓	✓
Nachrichtenkommunikation				✓
Organisationsmethodik			✓	✓
Verhaltenskategorisierung	✓	✓	✓	✓

Abbildung 10.1: Mächtigkeit der Zerlegungstechniken in Bezug auf die Beherrschung der Komplexität

10.1.3.2 Zerlegungstechniken

Funktionale Dekomposition

Die Vorgehensweise der *funktionalen Dekomposition* ist die der hierarchischen Zerlegung eines Problems in seine Schritte und Unterschritte. Die dieser Analysetechnik zu Grunde liegenden Bausteine sind:

- Funktionen,
- Unterfunktionen und
- Funktionsschnittstellen.

Die grundlegende Strategie der funktionalen Dekomposition besteht aus der Bestimmung der Bearbeitungsschritte (Bearbeitungsfunktionen), die für ein geplantes System notwendig sind, samt deren Unterschritten, unter Verwendung der Erfahrungen aus vorangegangenen, ähnlichen Arbeiten. Im Zentrum des Interesses steht, welche funktionalen Prozesse für das neue System notwendig sind. Der Analytiker spezifiziert dazu die Prozessfunktionen und die funktionalen Schnittstellen. Bedingungen (z. B. Invarianten) können als textuelle Anmerkungen festgehalten werden.

Die funktionale Dekomposition erfordert von Menschen, Abläufe aus dem Problemraum auf Funktionen und Unterfunktionen abbilden zu können. Die daraus resultierende Spezifikation kann allerdings nur indirekt (durch die Funktionsergebnisse für den Anwender) in die Problemwelt zurück abgebildet werden. Die Ergebnisse müssen vom Anwender im Licht des Verständnisses der Problemwelt interpretiert werden. Als Darstellungsmittel werden in der funktionalen Dekomposition *Flussdiagramme* bzw. *Struktogramme* verwendet.

Diese Vorgehensweise bringt folgende Probleme mit sich:

- Es ist für den Analytiker schwierig festzustellen, ob die beschriebenen funktionalen Anforderungen die Anforderungen der Problemwelt genau und vollständig (bzw. ausreichend) beschreiben.
- Es lässt sich nicht direkt überprüfen, ob die Problemwelt überhaupt bzw. genau genug verstanden worden ist. Ähnlichkeiten sowie subtile Unterschiede in der Problemwelt werden oftmals nicht entdeckt.
- Das Zerlegen in Funktionen und Unterfunktionen ist – durch die indirekte Abbildung – schwierig zu bewerkstelligen und für die Problemlösung sehr unzuverlässig. Es hat sich nämlich gezeigt, dass sich gerade der Bereich der funktionalen Anforderungen im Laufe eines Projekts am meisten ändert.
- Funktionen, die in eine Hierarchie eingebettet sind, tendieren dazu, nur eine spezifische übergeordnete Funktion zu unterstützen. Dies macht Wiederverwendung schwierig oder gar unmöglich [Coleman *et al.*, 1994].

Aus diesen Problemen darf nicht geschlossen werden, dass die funktionale Dekomposition grundsätzlich schlecht oder nicht notwendig sei, denn jedes System besteht unter anderem aus Funktionalität. Warum sich die Techniken der Analyse dennoch entscheidend weiterentwickelt haben, war die Suche nach einem stabileren Ausgangspunkt für die Analyse und die Erstellung der Problembeschreibung. Dieser wurde in den *Daten* gefunden.

Strukturierte Analyse

Die Vorgehensweise der *strukturierten Analyse* ist die des Zerlegens eines Problems gemäß den Daten, die am Prozess beteiligt sind. Daher wird diese Methode oft auch als *Datenflussanalyse* bezeichnet. Die dieser Analysetechnik zu Grunde liegenden Bausteine sind:

- Datenflüsse,
- Datentransformationen,
- Datenspeicher,
- Terminatoren (Prozessrandbedingungen und -abbruchbedingungen),
- Prozessspezifikationen und ein
- Datenlexikon.

Diese Methode bildet Probleme der realen Welt in Datenflüsse und Datenknoten ab. Um dies bewerkstelligen zu können, muss der Analytiker aus den zu lösenden Problemen den Fluss der Daten extrahieren können. Die Erfahrung hat aber gezeigt, dass das „Verfolgen eines Datenflusses“ keine Methode ist, zu welcher der Mensch einen natürlichen Zugang besitzt. Während das Zerlegen in Unterfunktionen der funktionalen Dekomposition für die meisten Menschen intuitiv zumindest nachvollziehbar ist (die Beschreibung ist vergleichbar mit einem strukturierten Kochrezept), ist jedoch das Strukturieren in Datenknoten und das Bestimmen von Datenflüssen zwischen den Knoten für den ungeschulten Betrachter vielfach nicht nachvollziehbar.

Der größte Vorteil der strukturierten Analyse gegenüber der funktionalen Dekomposition ist jedoch, dass mit ihrer Hilfe auch Probleme größerer Dimension überschaubar gemacht und strukturiert werden können. Gerade mit der Entwicklung der Programmierung weg von einfachsten Daten (v. a. Zahlen) hin zu komplexeren Strukturen konnte die strukturierte Analyse ihre großen Vorzüge ausspielen.

Die ursprüngliche Methode der „strukturierten Analyse“ wurde Ende der siebziger Jahre entwickelt (vgl. [DeMarco, 1978], [Gane und Sarson, 1979]) und bedient sich der Darstellung durch *Datenflussdiagramme* (*data flow diagrams*, *DFDs*). Dabei werden zuerst die Datenknoten und Datenflüsse auf Basis der realen Welt gezeichnet, dann in die Problemwelt abstrahiert, logische Bedingungen ergänzt und in die Lösung umgesetzt.

Das Kernproblem dieses Ansatzes ist, dass es sich als ungemein schwierig erwiesen hat, das Zerlegen in Datenknoten sinnvoll zu begrenzen. Allgemein tendieren Analytiker dazu, unheimlich komplexe Graphen mit sehr unübersichtlichen Datenflüssen zu erstellen. Da die Methode ein nachfolgendes Vereinfachen bzw. Zusammenfassen der Flüsse nicht direkt unterstützt, werden ausufernde strukturierte Analysen meist nicht beendet, sondern abgebrochen.

Bereits zu Beginn der achtziger Jahre des letzten Jahrhunderts wurde die strukturierte Analyse in Richtung ereignisorientierter Abläufe hin erweitert (vgl. z. B. [McMenamin und Palmer, 1984]). Dieses Konzept wurde von Yourdon Ende der achtziger Jahre unter dem Namen „*moderne strukturierte Analyse*“ vereinheitlicht. Die Ziele waren folgende:

- Der Analytiker sollte bei der Auswahl der Datenstrukturen unterstützt werden.
- Durch die systematische und beschränkte Auswahl der Datenstrukturen sollte sich „automatisch“ eine Reduktion der Datenflüsse ergeben.

Die Grundidee der modernen strukturierten Analyse ist die Festlegung der das Problem beschreibenden *Ereignisse* (*events*), um damit die anschließende Festlegung der Daten zu unterstützen. In [Yourdon, 1989] werden Ereignisse definiert als Vorkommnisse in der äußeren „realen“ Welt, auf die das erstellte System in geplanter Weise reagieren muss. Die tatsächliche Interaktion zwischen dem Anwender und dem System ist dabei nachrichtenbasiert. Die Ereignisliste, die dadurch bestimmt wird, ist im Wesentlichen die Dokumentation der individuellen Anforderungen menschlicher Anwender, die den Zusammenhang zwischen Ereignissen und Datenflüssen darstellen. Diese Darstellung erfolgt mittels *Kontrollflussdiagrammen* (*control flow diagrams* – *CFDs*).

Die Vorgehensweise der modernen strukturierten Analyse bringt folgende Probleme mit sich:

- Die Hauptschwierigkeit ist die geeignete Wahl der Knoten, wobei das Voreinteilen in Ereignisse und Ereignisantworten nur eine bedingte Hilfe darstellt.
- Da die Strukturierung in Datenbeziehungen nicht natürlich verständlich ist, ist das Kommunizieren des Analyseergebnisses oft sehr schwierig, sogar innerhalb der Analysegruppe.
- Bei realen Problemstellungen wird die Anzahl der Datenbeziehungen so groß, dass die Übersichtlichkeit verloren geht. Auch die Unterstützung durch Datenlexika (mittels Browser, Abstraktionsebenen etc.) hilft hier nur beschränkt.
- Die Trennung der Daten und Funktionen führt, vor allem bei nicht vorrangig daten(bank)orientierten Problemen, zu schlechten Analyseergebnissen.
- Die Daten werden bei der strukturierten Analyse zur Problemstrukturierung verwendet; die Möglichkeiten, die Daten selbst zu strukturieren (i. e. Erstellung und Einsatz von Datenstrukturen, Datentypen), sind nur sehr schwach bis gar nicht ausgebildet.

Datenflussanalysen konnten sich in der Praxis nur sehr beschränkt durchsetzen. Analytiker wechselten eher gleich zu moderneren Techniken (wie der des Information Modeling oder der objektorientierten Zerlegung), oder sie blieben bei der funktionsorientierten Zerlegung.

Information Modeling

Das *Information Modeling* (vgl. z. B. [Coad und Yourdon, 1990]) basiert auf einer datenorientierten Zerlegung, wobei nicht nur „Grunddatentypen“ (Zahlen, Zeichenketten etc.) als Daten zur Verfügung stehen, sondern eigene komplexe Datentypen (so genannte „*Objekte*“ bzw. „*Entitäten*“; *entities*) die Grundlage der Zerlegung bilden. Die dieser Analysetechnik zu Grunde liegenden Bausteine sind:

- Objekte (Entitäten),
- Attribute,

- Beziehungen (Relationen),
- Supertypen, Subtypen,
- assoziative Objekte (Beziehungsobjekte).

Die Darstellung der Modellierung erfolgt mit *Entity-Relationship-Diagrammen* (ERD; [Chen, 1977]). Diese Darstellungen haben sich allerdings im Laufe der Zeit von rein datenbankorientierten „Tabellen“-Darstellungen hin zu *semantischen Datenmodellen* [Darwen und Date, 1998] entwickelt.

In der ursprünglichen, aus dem Datenbankbereich motivierten Vorgehensweise beginnt man mit der Erstellung einer Liste aller relevanten Attribute. Diese werden dann in Komponenten gruppiert, den Entitäten, zwischen welchen nun Relationen definiert werden inklusive der Kardinalität der Relationen (Anzahl der Instanzen der verbundenen Komponenten). Sodann wird eine Hierarchie mittels Supertypen und Subtypen eingeführt (Faktorisieren der gemeinsamen Attribute), ergänzt durch assoziative Objekte. Abschließend wird „normalisiert“, i. e., es werden die Redundanzen entfernt.

Im Information Modeling wird der Begriff „*Objekt*“ wie folgt verwendet: Ein Objekt stellt ein Symbol dar, das im Modell ein oder mehrere Exemplare eines Gegenstandes der realen Welt darstellt. Beispielsweise werden in [Shlaer und Mellor, 1988] Objekte mit Entitäten gleichgesetzt, während in der object behavior analysis (OBA; [Rubin und Goldberg, 1992]) Systemverhalten identifiziert wird und dieses dann zur Ableitung von Systemrollen und Verantwortlichkeiten verwendet wird. Im Folgenden verfährt man wie in der ursprünglichen Methode.

Obwohl diese Vorgehensweise der Zerlegung oftmals bereits als „objektorientiert“ bezeichnet wird, fehlen dazu wesentliche Konzepte (vgl. [Coad und Yourdon, 1990]):

- **Methoden (services):** die Möglichkeit, das Verhalten eines Objekts gemeinsam mit den Attributen gekapselt darzustellen,
- **Nachrichten (messages):** die Kommunikation via enge und abgesicherte Schnittstellen zwischen Objekten bzw. Prozessen,
- **Vererbung:** die explizite Darstellung von Gemeinsamkeiten bei Attributen und Methoden, die auch innerhalb der Hierarchie weitergegeben werden kann,
- **Struktur:** Generalisierungs-/Spezialisierungsstrukturen sowie Dekompositions-/Aggregationsstrukturen haben im Analyseergebnis nicht die zentrale Bedeutung, die ihnen zukommen sollte.

Für eine nähere Analyse des Information Modeling wird auf die Literatur verwiesen (z. B. [Loomis *et al.*, 1987], [Shlaer und Mellor, 1988]). Von zum Teil denselben Autoren wurde diese Methode schrittweise in Richtung *objektorientierter Zerlegung* weiterentwickelt.

Objektorientierte Zerlegung

Die Ideen einer „objektorientierten“ Vorgehensweise werden in der Softwareentwicklung erst in den neunziger Jahren des letzten Jahrhunderts in den Mittelpunkt gestellt, sind aber be-

reits sehr alt. So wurde die Klassifizierung von Begriffen (meist einfach mit Objekten gleichgesetzt) der realen Welt schon von Aristoteles (384–322 v. Chr.) empfohlen, um Ordnung in unübersichtliche Sachverhalte zu bringen. Insbesondere geht es dabei darum, Objekte durch ein hierarchisches Schema von Begriffen so zu beschreiben und zu ordnen (zu „klassifizieren“), dass die Begriffe auf tieferen Ebenen spezieller und konkreter sind als die auf höheren Ebenen. Deren allgemeine Eigenschaften werden aber auf den unteren Ebenen beibehalten („geerbt“).

Wie [Elzer, 1994] anmerkt, ist „die Auswahl der für ein bestimmtes Anwendungsgebiet relevanten Klassen ein nichttriviales Problem. Nicht ohne Grund ist die vor etwa zwei Jahrhunderten begonnene Klassifizierung der Tiere und Pflanzen noch nicht abgeschlossen. Es besteht also die Gefahr, dass man bei unbedachter Anwendung der Technik des objektorientierten Designs in ähnlich langwierige und fruchtlose Diskussionen gerät wie bei einem Design nach einem klassischen Verfahren, wenn man dort z.B. ohne tiefergehende Kenntnisse der Anwendung versucht, Hauptfunktionen festzulegen“.

Der Begriff „objektorientiert“ ist häufig Gegenstand kontroverser Diskussionen, die dadurch begründet sind, dass der Begriff „Objekt“ aus zwei sehr verschiedenen Gebieten stammt und dort entsprechend unterschiedlich verwendet wurde:

- Wie bereits erwähnt, bedeutet ein Objekt im Information Modeling die Darstellung eines Gegenstands der realen Welt sowie einer Anzahl von Exemplaren dieses Gegenstands.
- In objektorientierten Programmiersprachen bedeutet ein „Objekt“ ein zur Laufzeit erzeugtes Exemplar eines Gegenstandes mit bestimmten Prozessen und Werten, das durch eine statische Beschreibung, der so genannten „Klasse“, festgelegt wird.

Die objektorientierte Zerlegung verbindet die bewährten Konzepte des Information Modeling mit denen der objektorientierten Programmierung. Die dieser Analysetechnik zu Grunde liegenden Bausteine sind:

- Objekte und Klassen² (beschrieben durch gekapselte Attribute und Methoden),
- Klassifikationsstruktur (Generalisierungs-/Spezialisierungsstrukturen),
- Aggregationsstruktur (Gesamt-/Teilstrukturen),
- Vererbung,
- Kommunikation mittels Nachrichten, die beim empfangenden Objekt den Aufruf einer Methode bewirken.

Die objektorientierte Zerlegung übernimmt folgende Konzepte des Information Modeling:

- Objekte (Klassen) und deren multiple Exemplarisierung („Instanzierung“),
- Relationen,

²Oft werden die Begriffe „Objekt“ und „Klasse“ in der objektorientierten Zerlegung synonym verwendet, seltener im Kontext der objektorientierten Programmierung.

- Struktur (Aggregation).

Folgende Konzepte werden aus der objektorientierten Programmierung übernommen:

- Kapselung von Attributen und Methoden,
- Behandlung von Attributen und Methoden als gekapseltes Ganzes,
- Klassifikation,
- Kommunikation (exklusiv) via Nachrichten,
- explizites Festschreiben von Gemeinsamkeiten via Vererbung.

Die Methode von Abbot zur Bestimmung einer guten Objektstruktur

In der objektorientierten Zerlegung werden Objekte und deren Attribute analog zum Information Modeling festgelegt. Zusätzlich werden jedoch den Objekten diejenigen Methoden zugeordnet, die exklusiv dieses Objekt und seine Attribute manipulieren. Zur Bestimmung einer guten Objektstruktur und damit Klassifikation wird vielfach die *Methode von Abbot* empfohlen (vgl. z. B. [Pressman, 2004]). Diese Methode umfasst

1. das Erstellen einer verbalen Problembeschreibung (aus der Zielbeschreibung),
2. das Identifizieren von Objekten, Attributen und Methoden: *Hauptwörter* sind mögliche Kandidaten für Objekte; *Eigenschaftswörter* und Phrasen mit Hauptwörtern im Genitiv („... die Größe des Fensters ...“) sind mögliche Kandidaten für Attribute; *Zeitwörter* sind mögliche Kandidaten für Methoden,
3. das Suchen nach geeigneten Basisklassen (Hierarchisierung, Abstraktion) sowie
4. das Vervollständigen der Attribute und Methoden. (Es empfiehlt sich, dabei immer etwas allgemeiner zu sein als nötig, um zu möglichst wieder verwendbaren Klassen zu kommen.)

In verbalen Beschreibungen werden jedoch oft Synonyme zur Vermeidung von Wortwiederholungen verwendet (z. B. Hauptwort/Substantiv/Wort; Größe/Ausmaß). Ob bestimmte Wörter synonym verwendet werden oder doch einen bestimmten Bedeutungsunterschied aufweisen, muss im Kontext geklärt werden und stellt eine häufige Fehlerquelle dar. Für die saubere Modellierung des Problems ist die Elimination von Synonymen entscheidend. Unterstützt wird diese durch das Führen eines genauen Datenlexikons (vgl. Abschnitt 9.2.5.2) sowie durch Reviews des Analyse(zwischen)ergebnisses (z. B. auf symmetrische bzw. austauschbare Generalisierungs- und Spezialisierungsbeziehungen oder einen gemeinsamen Oberbegriff ohne Spezialisierungsmerkmal).

Tipp: Oft ist die Wahl möglich, eine Eigenschaft mit Hilfe einer Relation auf eine eigene Komponente abzubilden oder durch ein Attribut zu modellieren. In der Analysephase ist es im Allgemeinen besser, eigene Komponenten samt den notwendigen Relationen einzuführen, da diese flexibler und besser erweiterbar sind und außerdem selbst Attribute haben können

oder in anderen Relationen verwendet werden können. Im Rahmen des abschließenden Analysereviews oder zu Beginn des Designs können Komponenten, die keine (bzw. in begründeten Fällen sehr wenige) Attribute enthalten, keine direkt zugewiesene Funktionalität besitzen und nur mit einer anderen Klasse in Relation stehen, wieder durch entsprechende Attribute ersetzt werden.

Die entscheidenden Vorteile der objektorientierten Zerlegung sind:

- Unterstützung aller Konzepte zur Beherrschung der Komplexität,
- direkte Abbildung aus dem Problemraum in die Modellwelt und retour,
- Möglichkeit der klaren Trennung zwischen Autor und Anwender,
- Durchgängigkeit der Methode von der Analyse über das Design bis hin zur Implementierung,
- enge Verschmelzung von Analyse und Design (diese unterscheiden sich nur mehr durch die Betrachtungsweise der Objekte).

Da die objektorientierte Zerlegung funktionale Zerlegungskomponenten (via Methoden und Nachrichtenkommunikation) sowie die Möglichkeit zur Datenzerlegung (Objekte mit Attributen) enthält, kann mit ihr auch funktions- und datenorientiert analysiert werden. Sie eignet sich daher als allgemeine Analysemethode, die vor allem bei Beherrschung der objektorientierten Konzepte das bislang mächtigste verfügbare Vorgehen zur Analyse darstellt.

Aktuelle Bestrebungen betonen die Verteilung von Systemen, vor allem die Entkopplung von Teilsystemen (so genannten *Komponenten*), die innerhalb des Teilsystems eine starke Verwebung besitzen, untereinander jedoch nur eine schwache. Dementsprechend wird von *komponentenorientierter Softwareentwicklung* (*component-oriented software development, COS*) und *verteilten Systemen* gesprochen, die aus mehreren, vielfach *nebenläufigen Prozessen* bestehen. Die *aspektorientierte Softwareentwicklung* stellt die Sichten auf Teilsysteme (z. B. Fehlerbehandlung, Zugriffssicherheit) und insbesondere deren einheitliche Behandlung in den Vordergrund.

10.1.3.3 Erstellen von Prototypen

Unterstützung der WAS/WIE-Trennung

Das Ziel einer Systementwicklung kann in der Regel nicht unabhängig von der Lösung festgelegt werden. Gerade davon wird aber beim klassischen Softwarelebenszyklus-Modell ausgegangen. Zur korrekten Situierung der Analyse muss man daher die Wechselwirkungen zwischen „Weg“ und „Ziel“ untersuchen und den Analyseprozess darauf aufbauen. Wie bereits erwähnt, ist es die Aufgabe der Analyse zu beschreiben, was ein System leisten soll, aber nicht, wie diese Leistungen realisiert werden sollen. Für diese „*WAS/WIE-Trennung*“ werden verschiedene Vorteile angeführt [Pressman, 2004]:

- Eine Beschreibung, die sich auf das Ziel der Entwicklung beschränkt, gewinnt an Kürze und Verständlichkeit.

- Die Konzentration auf die Analyse verhindert, dass die Diskussion möglicher Realisierungsalternativen von den eigentlichen Analyseaufgaben ablenkt.
- Die Freiheit der Entwickler wird nicht unnötig eingeschränkt.

Die strikte Trennung von Analyse, Design und Implementierung hat aber auch eine Reihe von Nachteilen [Pressman, 2004]:

- Die mangelhafte Qualität der Analyse: Die Vollständigkeit und Widerspruchsfreiheit des Analyseergebnisses ist in der Regel nur erreichbar, wenn bekannt ist, wie und ob einzelne Systemfunktionen realisierbar sind.
- Die inadäquate Darstellungsform der Analyse: Ein entscheidender Aspekt eines Softwaresystems, nämlich sein dynamisches Verhalten, muss im Sinn der „WAS/WIE-Trennung“ statisch, d. h., verbal beschrieben werden. Dies bläht den Umfang der Analyse-dokumentation auf und führt zu Kommunikationsproblemen zwischen Anwender und Entwickler.

Eine *prototypingorientierte Analyse* gestattet es nun, sowohl die Vorteile der „WAS/WIE-Trennung“ zu erhalten als auch deren Nachteile zu beseitigen. Damit lässt sich das dynamische Verhalten eines Systems in Form eines Prototyps realisieren und muss nicht mit Worten beschrieben werden. Es reduziert sich der Umfang des Analyseberichts, die Vollständigkeit und Widerspruchsfreiheit der Problembeschreibung kann verbessert und das Kommunikationsproblem zwischen Anwender und Entwickler vermindert werden, ohne die Freiheit der Entwickler einzuengen und ohne sich allzu sehr auf Implementierungsdetails konzentrieren zu müssen.

Arten von Prototypen

Prototypen können nach folgenden Kriterien klassifiziert werden (vgl. [Coad und Yourdon, 1990] und [Pomberger und Blaschek, 1996]):

- **Vollständige Prototypen versus unvollständige Prototypen:** Prototypen im herkömmlichen Sinn (z. B. im Automobilbau) stellen im Allgemeinen *vollständige Prototypen* dar, in denen alle wesentlichen Funktionen des geplanten Systems vollständig verfügbar sind. Vollständige Prototypen werden für Softwaresysteme kaum hergestellt. Softwareprototypen sind vielmehr *unvollständige Prototypen*, die es gestatten, die Brauchbarkeit und Machbarkeit einzelner Aspekte (z. B. Benutzerschnittstelle, Systemarchitektur, Systemkomponenten) des geplanten Systems zu untersuchen.
- **Breite und flache Prototypen versus enge und tiefe Prototypen:** *Breite und flache Prototypen* geben eine Übersicht über alle benötigte Funktionalität, ohne Detailfunktionalität genauer abzubilden. Hingegen konzentrieren sich *enge und tiefe Prototypen* auf einen Teilbereich, der im Detail erforscht wird. Speziell für das Ziel einer möglichst durchgehenden, evolutionären Prototypunterstützung ist es empfehlenswert, bereits in der Zielerforschungs- und Analysephase mit breiten und flachen Prototypen zu beginnen, die dann in den komplexesten und kritischsten Stellen vertieft bzw. durch eingebundene enge und tiefe Prototypen erweitert werden.

- **Revolutionäre Prototypen versus evolutionäre Prototypen:** Von einem *revolutionären Prototypen* (*Wegwerfprototypen*) spricht man dann, wenn die Implementierung des Prototyps bei der Implementierung des Zielsystems nicht weiterverwendet wird, sondern der Prototyp „nur“ als ablauffähiges Modell dient. Die oben angesprochenen Funktionserforschungsprototypen sowie die Architekturprototypen stellen im Allgemeinen Wegwerfprototypen dar.

Das Ziel und die Ergebnisse von Wegwerfprototypen müssen sehr genau dokumentiert werden, da ja der Prototyp selbst nicht in das Produkt eingeht. Andernfalls kann es leicht passieren, dass der Auftraggeber enttäuscht ist, wenn die besonders attraktiv gestaltete Benutzerschnittstelle und die hervorragende Interaktivität (Antwortzeiten!) im Produkt funktionalen Anforderungen und tatsächlichen Rechenzeiten weichen müssen.

Von *evolutionären Prototypen* (*wieder verwendbaren Prototypen*) spricht man, wenn wesentliche Teile des Prototyps bei der Implementierung des Zielsystems übernommen werden können.

Offensichtlich ist im Allgemeinen der Wiederverwendbarkeit von evolutionären Prototypen gegenüber Wegwerfprototypen der Vorzug zu geben. Wie bereits in Abschnitt 6.4 angemerkt, finden daher revolutionäre Prototypen meist nur in den frühen Projektentwicklungsphasen Verwendung.

10.1.3.4 Sprechdisziplin

Die Erfahrung zeigt, dass gerade in den ersten Projektabschnitten die Phasen des Projektentwicklungszyklus (Zielerforschung, Analyse, Design, teilweise Implementierung) zeitlich nicht strikt getrennt oder aufeinanderfolgend gesehen werden können. Auch können diese Aufgaben nicht personell aufgeteilt werden. Gerade im iterativ-inkrementellen Vorgehen wird bewusst laufend zwischen diesen Phasen hin und her gesprungen und so ein laufendes Reengineering durchgeführt.

Um nicht bei Konzeptlosigkeit anzukommen, ist daher die *Sprechdisziplin* eminent wichtig. Jeder Projektmitarbeiter muss neu eingeführte Begriffe festlegen. Werden diese Begriffe von der Projektgruppe (vorläufig oder definitiv) akzeptiert, so müssen sie von der ganzen Gruppe im definierten Sinn ab diesem Zeitpunkt verwendet werden. Bei jeder Aussage muss dazugesagt werden, im Rahmen welcher Phase (Sichtweise) diese getätigt wird. Eine Aussage wie etwa „Dazu brauche ich eine Liste.“ hat in der Analyse eine andere Semantik und eine andere Konsequenz als im Design.

Einfache Gedächtnisstützen für die Zuordnung einer Aussage zu einer bestimmten Phase sind die Fragen, die mit Aussagen zu jeder Phase beantwortet werden:

- **Zielerforschung:** WAS ist das (unveränderliche) Gesamtziel des Projekts?
- **Analyse:** WAS ist zu erstellen? WAS sind die Einzelteile des Systems?
- **Design:** WIE soll das Ergebnis (für den Benutzer) reagieren und aussehen? WIE sollen die Einzelteile aussehen, zusammengebaut sein und miteinander kommunizieren?
- **Implementierung:** WIE wird das Ergebnis realisiert?

Bestimmte Aussagen sind in bestimmten Phasen verboten. Wie etwas realisiert wird, darf z. B. eine Analyseaussage nicht beeinflussen. Jeder Projektmitarbeiter muss sich bewusst sein, welcher Bereich des Projekts in einer Phase gesehen werden muss und welcher nicht gesehen werden darf.

Diese saubere Trennung ist oftmals wichtiger als die (im Allgemeinen nicht erreichbare oder zumindest nicht überprüfbare) Vollständigkeit der Aussagen.

10.1.4 Werkzeuge

Zur Dokumentation der Analyseergebnisse werden komfortable *Textverarbeitungssysteme* benötigt, aber auch Werkzeuge zur grafischen Darstellung von Daten- und Informationsflüssen. Im Folgenden setzen wir Vorwissen über Textverarbeitungssysteme voraus und behandeln das Problem der grafischen Darstellung zur Analyseunterstützung.

Wir konzentrieren uns im Folgenden auf Werkzeuge, die – in kleinerem oder größerem Umfang – die objektorientierte Zerlegung unterstützen. Solche Werkzeuge können auch zur Unterstützung funktionsorientierter und datenorientierter Zerlegungen genutzt werden, da diese Bestandteil der objektorientierten Zerlegung sind.

10.1.4.1 Analysewerkzeuge

Auch wenn von den Herstellern computerbasierte *Analysewerkzeuge* (oft wiederum als „*CASE-System*“ apostrophiert) sehr häufig als Universalwerkzeug dargestellt werden, eignen sie sich doch in der Praxis meistens nur zur Dokumentation des Analyse(zwischen)ergebnisses. Die eigentliche Arbeit erfolgt besser und kommunikativer auf dem Papier.

Grundsätzlich kann jede Analyse mit Papier und Bleistift in klassischer Manier durchgeführt und erst später in den Computer übertragen werden. Probleme dabei ergeben sich durch die nur bedingte Änderbarkeit (revolutionärer statt evolutionärer Ansatz) und die rasch entwindende Übersicht.

Folgende Darstellungsarten können diese Nachteile reduzieren bzw. eliminieren:

- **CRC-Karten** [Wirfs-Brock und Wilkerson, 1989]: *Class-Responsibility-Collaboration-Karten* (*CRC-Karten*) dienen zum Auffinden von Klassenaufgaben („*Verantwortlichkeiten*“; „*responsibilities*“) und zur Festlegung der zur Erfüllung dieser Aufgaben erforderlichen anderen Klassen („*Zusammenarbeit*“; „*collaboration*“). Der Klassenname wird Bestandteil des Projektvokabulars und soll so gewählt werden, dass er für den Auftraggeber möglichst klar verständlich ist. Die Verantwortlichkeiten sollen knappe und präzise Phrasen sein, deren Aufgabencharakter durch ein enthaltenes Verb ausgedrückt wird. Die Sammlung dieser Phrasen ergibt die Bedeutung der Klasse. Eine CRC-Karte ist in Abbildung 10.2 dargestellt.

CRC-Karten werden von Analytikern während der Diskussionen benutzt. Karten werden dabei immer wieder verschoben und umgelegt. Rollenspiele (Analytiker/Designer spielen „Klassen“) machen die Bedeutung der Klassen klar.

Vielfach wird angemerkt, dass CRC-Karten keine Hilfe zum Finden von Objekten darstellen, sondern das Reorganisieren und Verallgemeinern bereits gefundener Objekte

Class	
Responsibilities	Collaborators

Abbildung 10.2: CRC-Karte

erleichtern (vgl. z. B. [Coleman *et al.*, 1994]). Dies gilt auch für die nachfolgend vorgestellten Haftnotizen-Objekte.

- **Haftnotizen-Objekte:** Die Gestaltung von *Haftnotizen-Objekten* folgt der Darstellung von Objekten/Klassen in der UML, vgl. Abbildung 10.3.

ObjektName [:Klasse]
Attribut [:Typ]
Methode () [:Typ]

Abbildung 10.3: Haftnotizen-Objekt

Haftnotizen-Objekte werden auf Papierbögen angeordnet und können beliebig umgruppiert werden. Ihr Vorteil gegenüber CRC-Karten ist – neben der gängigeren Objektdarstellung – das Haften auf dem Untergrund, das unbeabsichtigtes Verschieben weitgehend ausschließt und auch jederzeit eine Unterbrechung der Arbeit ermöglicht. Wir empfehlen zur Nachvollziehbarkeit der Entwicklung und Änderung der Analyse, zu geeigneten Zeitpunkten Fotokopien des Analyseblatts anzufertigen.

- **Textuelle Beschreibung durch Szenarien (Scripting):** Ein *Szenario* ist eine Sequenz von Aktionen, die in einem Problembereich auftreten kann [Goldberg und Rubin, 1995]. Indem man Szenarien konstruiert und niederschreibt, legt man das notwendige Verhalten eines Systems fest und weist bestimmten Rollen Verantwortung zu, dieses Verhalten auszuführen. Die Technik, Szenarien zu erstellen, wird – in Anlehnung an die Filmindustrie – *Scripting* genannt. Der Vorteil des Scripting ist, dass der Mensch sein Verständnis in einem Problemraum unter anderem in Form von Szenarien abspeichert. Im so genannten episodischen Gedächtnis werden Sequenzen von Ereignissen festgehalten, die ein Szenario (eine Episode) beschreiben. Sowohl das Erstellen von Szenarien als auch das Verstehen solcher ist für viele Menschen einfacher möglich als das vollständige Aufzählen der Komponenten und Einzelaktionen in einem Problemraum [Cockburn, 2004].

Das Arbeiten auf Papier hat den Vorteil gegenüber dem Arbeiten auf dem Computer, dass jeder Mitarbeiter gleichrangig und bei Bedarf gleichzeitig seine Ideen einbringen kann. Am Bildschirm ist erfahrungsgemäß immer der Bediener dominierend. Durch das Verwenden von

CRC-Karten oder Haftnotizen-Objekten wird vor allem das zyklische Adaptieren der Analyseresultate gefördert. Ohne Wiederverwendbarkeit und Umgruppierbarkeit der logischen Einheiten fallen die Kreativität und der Spieltrieb beim Problemlösen oftmals der Schreibfaulheit zum Opfer.

10.1.4.2 Diagramme der UML für die Analyse

Für die Analysemodelle stellt die UML verschiedene Diagrammarten zur Verfügung, wie in Abbildung 10.4 ersichtlich ist. Für Details wird auf [Oestereich, 2004] verwiesen.

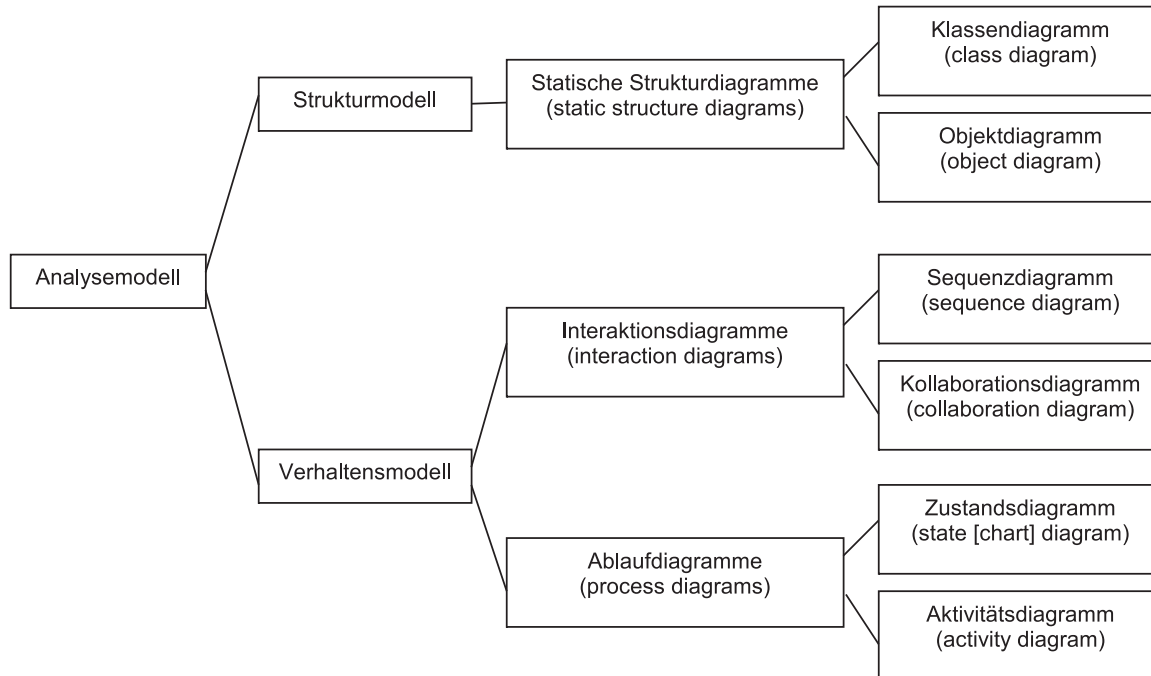


Abbildung 10.4: Diagramme der UML für die Analyse

10.1.4.3 Vorgehen bei der Analyse unter Verwendung der UML

Folgendes schrittweises Vorgehen wird für eine objektorientierte Zerlegung unter Verwendung der Modelle und Diagramme der UML empfohlen (Ausgangsbasis ist die Zielbeschreibung):

- **Entwickeln des Strukturmodells:**

1. Objekte bzw. Klassen aus allen Anwendungsfällen identifizieren (Klassen- bzw. Objektdiagramm verwenden! Immer so allgemein wie sinnvoll möglich bleiben! In der Zielbeschreibung kommen v. a. Objekte vor, im Modell v. a. Klassen!),
2. Assoziationen zwischen Klassen/Objekten hinzufügen,
3. Klassen-/Objektattribute sowie Assoziationsattribute hinzufügen,
4. Klassen durch Generalisierung vereinfachen und hierarchisch strukturieren (Generalisierungen nicht zu früh vornehmen! Zu frühe Generalisierungen basieren oft auf falschen oder unvollständigen Annahmen und führen zu falschen Klassenbäumen!),

5. Modell durch Durchspielen der Anwendungsfälle testen (Anwendungsfälle ggf. aktualisieren!),
6. Klassen zu Paketen gruppieren.

- **Entwickeln des Verhaltensmodells:**

1. Interaktionsfolgen zwischen den Objekten festlegen (Zeitlich strukturierte Anwendungsfälle erstellen!),
2. Ereignisse zwischen Objekten identifizieren und Nachrichten zeichnen (Kollaborationsdiagramm erstellen!),
3. komplexe, nebenläufige Abläufe, an denen viele Objekte beteiligt sind, analysieren und mittels Aktivitätsdiagrammen modellieren,
4. Systemgrenzen und Schnittstellen zur Umgebung festlegen,
5. pro Alternative in einem Anwendungsfall ein Sequenzdiagramm für die Interaktionen erstellen,
6. Zustandsdiagramm(e) für jede Klasse mit (nicht trivialem) dynamischem Verhalten erstellen (Bei komplexem Verhalten die Harel-Notation verwenden!),
7. jede (nach außen wirksame) Aktivität einer Klasse deklarativ beschreiben (Zweck beschreiben, Schnittstelle festlegen, Einschränkungen identifizieren, Performanzansprüche festlegen),
8. auf Basis der Anwendungsfälle Konsistenz und Vollständigkeit des Verhaltensmodells testen.

- **Inkrementelles Verfeinern der Modelle:**

1. verifizieren, dass Klassen, Assoziationen, Attribute und Aktivitäten auf der gewählten Abstraktionsebene konsistent und vollständig sind,
2. Anwendungsfälle so weit detaillieren, dass alle systemrelevanten Sonderfälle und Fehlerfälle (als Varianten der Standardfälle) beschrieben sind,
3. Struktur- und Verhaltensmodell iterativ verfeinern, bis alle festgelegten Anforderungen modelliert sind (Gewähltes Abstraktionsniveau berücksichtigen!).

Zu beachten ist, dass die Schritte zur Erstellung der Modelle je nach Problem auch parallel bzw. in anderer Reihenfolge durchgeführt werden können. Das Ergebnis dieses Analysevorgangs umfasst die Zielbeschreibung, das Strukturmodell, das Verhaltensmodell sowie das adaptierte und erweiterte Datenlexikon und wird im *Analysebericht* (siehe Abschnitt 10.1.5.1) dokumentiert.

10.1.4.4 Werkzeuge des Prototyping

Im Gegensatz zu anderen Techniken zur qualitativen und organisatorischen Verbesserung des Softwareentwicklungsprozesses bedingt Prototyping den Einsatz von Werkzeugen. Die eingesetzten Werkzeuge sind von entscheidender Bedeutung sowohl für den Herstellungs- als auch den Änderungs- und Erweiterungsprozess von Softwareprototypen.

Typische Werkzeugklassen für Prototyping in der Analyse- und Designphase sind (vgl. [Pressman, 2004]):

- **Generatorsysteme:** Generatorsysteme bestehen typischerweise aus zwei Komponenten: einem Editor, mit dem Spezifikationen erstellt werden, und einem Generator/Interpreter/Compiler/Laufzeitsystem zur Syntaxprüfung, zur Transformation der Spezifikation in einen ablauffähigen Prototyp und zu dessen Ausführung.

Die Idealvorstellung ist, dass der Prototypentwickler lediglich spezifizieren muss, was zu geschehen hat; wie das ablauffähige System realisiert wird, darum braucht er sich nicht zu kümmern. Die Implementierung des Prototyps ist Aufgabe des Generators; der Systementwickler hat keinen Einfluss darauf. Der Nachteil der Generatorsysteme liegt häufig in der zu geringen Flexibilität. Prototypen, die nicht in der vorgegebenen Sprache spezifiziert werden können, sind nicht erzeugbar, auch wenn die Abweichungen gering sind.

Formale Spezifikationssprachen existieren bereits seit längerer Zeit, vgl. z.B. *VDM* [Jones, 1990] oder *Z* [Spivey, 1990]. Erst in letzter Zeit werden diese Sprachen um objektorientierte Konzepte erweitert (*Object-Z* [Duke *et al.*, 1996], *Z++* [Lano und Haughton, 1992], *Fresco* [Wills, 1991] etc.) bzw. als UML-Erweiterung spezifiziert [Buschmann, 1998]. Die Anwendung dieser Sprachen ist jedoch sehr zeitaufwändig und verlangt großes formales und mathematisches Verständnis. Außerdem lassen sich derzeit nur Probleme mit eher geringer Komplexität modellieren.

- **Datenbankbasierte Systeme:** Mit zunehmendem Einsatz von Computern steigen vor allem die Anforderungen an die Datenhaltung. Datenbankbasierte Informationssysteme sind oft zentrale Komponenten einer Anwendung. Im Bereich der Informationssysteme treten häufig erhebliche Spezifikations- und Effizienzprobleme auf. Es ist daher nahe liegend, die Analyse von Informationssystemen prototypingorientiert durchzuführen. Objektorientierte Datenbanksysteme eignen sich sehr gut für derartige Prototypingaufgaben.
- **Hypertextsysteme:** Nach [Smith und Weiss, 1988] ist Hypertext „an approach to information management in which data is stored in a network of nodes connected by links“. Mit Hypertextsystemen können auf sehr einfache Weise Verbindungen (Links) zwischen Informationseinheiten und dem Layout von Bildschirmmasken mit einem grafischen Editor hergestellt und damit eigentlich statischen Prototypen ein „dynamisches“ Verhalten hinzugefügt werden.

Kriterien zur Charakterisierung und Beurteilung von Prototypingwerkzeugen

Neben der groben Einteilung der Prototypingwerkzeuge, wie sie sich aus der obigen Aufstellung ergibt, sind für ihre Charakterisierung und Beurteilung eine Reihe von Kriterien von Bedeutung. Die nachfolgend angeführten Kriterien sind nicht unabhängig voneinander, kaum messbar und nur zum Teil objektiv; sie sollen lediglich Anhaltspunkte für eine Beurteilung und Charakterisierung solcher Werkzeuge liefern, siehe [Pomberger und Blaschek, 1996]:

- **Erforderliche Benutzerkenntnisse:** Welche Kenntnisse müssen vorausgesetzt werden (prozedurale Sprache, Anwendung von Prozedurbibliotheken, Spezifikationssprache, Anwendung von grafischen Spezifikationswerkzeugen etc.)?
- **Zeitaufwand bis zum ersten Prototyp:** Liegt bereits eine Standardanwendung vor, die leicht in einen ablauffähigen Prototyp übergeführt werden kann? Wie aufwändig ist die Formulierung der ersten Spezifikation?

- **Aussehen der Benutzerschnittstellen:** Wird ein Standard unterstützt? Werden damit nur bestimmte Anwendungsklassen unterstützt? Wie viel Semantik und Funktionalität ist in den erzeugten Benutzerschnittstellen enthalten? Sind die zu Grunde liegenden Benutzerschnittstellen fest vorgegeben oder sind sie konfigurierbar?
- **Spezifikation der Benutzerschnittstellen:** Auf welcher Abstraktionsebene (z. B. als endlicher Automat, Hypertext, kontextfreie Grammatik etc.) werden Benutzerschnittstellen spezifiziert? Welche Mittel (formale Sprachen; grafische Editoren, die nach dem WYSIWYG-Prinzip oder nach dem Prinzip der „direkten Manipulation“ arbeiten etc.) werden dazu eingesetzt? Wie werden statische Teile (Layouts), wie werden dynamische Teile (Abläufe, Zustandsübergänge) spezifiziert?
- **Verwendung der erzeugten Prototypen:** Können die Prototypen zu fertigen Anwendungen erweitert werden? Kann aus ihnen lediglich ein Programmskelett erzeugt werden? Welche Informationen können wieder verwendet werden und auf welche Weise (einfaches „cut and paste“ von Codeteilen; „höhere“ Verfahren, z. B. Manipulation von Benutzerschnittstellenkomponenten wie Dialogmuster, Dialogskelette, Bildschirm-layouts, etc.; Unterstützung durch Objektverwaltungssysteme)?
- **Unterstützung von Erweiterungen:** Mit welchem Aufwand sind Erweiterungen verbunden, die nur die Benutzerschnittstelle betreffen, und solche, welche die Funktionalität betreffen? Müssen dabei Compile-Link-Go-Zyklen durchlaufen werden, oder liegt ein interpretierendes System zu Grunde? Kann die Benutzerschnittstelle zur Laufzeit verändert werden (z. B. kontextabhängige Menügestaltung, grafische Anzeige generierter Daten, benutzerabhängige Dialogstruktur)? Ist die Konsistenz der anfallenden Dateien und Dokumente gewahrt?
- **Art der funktionalen Erweiterungen:** Kann die Funktionalität der Prototypen schrittweise erweitert werden? Werden im Falle von Generatoren die Erweiterungen auf der Stufe der Spezifikation vorgenommen, oder müssen sie im generierten Code implementiert werden? Wie ist die Qualität (Lesbarkeit, Strukturiertheit etc.) des bei der Codeerzeugung generierten Codes? Ist die Codeerzeugung parametrisierbar (Wahl der Sprache, Wahl des Sprachdialekts, Formatierung)? Wie kann aus funktionalen Teilen des Prototyps auf die Benutzerschnittstelle zugegriffen werden? Können funktionale Erweiterungen, Ausnahmesituationen etc. simuliert werden? Ist es möglich, Prototypen herzustellen, bei denen manche Operationen ausgeführt, andere hingegen nur simuliert werden? Welche Schnittstelle besteht zu Datenbanksystemen?
- **Entwicklungsumgebung:** In welcher Entwicklungsumgebung kann das Prototypingwerkzeug eingesetzt werden, und wie ist es in diese Umgebung integriert? Können Informationen zwischen den Prototypen und anderen Dokumenten ausgetauscht werden (z. B. über ein Datenlexikon)? Welche Softwarewerkzeuge (dynamische Debugger, Werkzeuge für die Versionsverwaltung, Testwerkzeuge, Profilingwerkzeuge etc.) und welche Programmiersprachen stehen zur Verfügung? Können mit dem Prototypingwerkzeug neben den Prototypen weitere Objekte (Dokumentationen, Online-Hilfe-Dateien, Protokolle und Statistiken über Benutzerverhalten etc.) erzeugt werden, und wie können diese verwendet werden?
- **Qualität des Prototypingwerkzeugs:** Wie ist die Effizienz, die Zuverlässigkeit, die Komplexität und die Erlernbarkeit des Prototypingwerkzeugs?

- **Implementierung des Prototypingwerkzeugs:** Auf welchen Konzepten beruht die Implementierung? Ist das Prototypingwerkzeug erweiterbar?

10.1.5 Ergebnisse

10.1.5.1 Analysebericht

Die Ergebnisse der Analyse seitens des Auftragnehmers werden im *Analysebericht* dokumentiert und umfassen die Beschreibung des aus dem Istzustand zu entwickelnden Sollzustand in einer strukturierten, gegliederten Form. Dieser Bericht besteht im Allgemeinen aus der grafischen Darstellung der Problemstrukturierung sowie textuellen Erklärungen und Begründungen der Zerlegung.

Der Analysebericht dient zum Abgleich mit dem vom Auftraggeber erstellten Lastenheft und damit – neben den Planungsergebnissen, vgl. Abschnitt 10.2 – als Basis für die Erstellung eines *Angebots* bzw. für die *Einreichung* bei einer *Ausschreibung*.

10.1.5.2 Lastenheft

Wie z. B. in VDI-Richtlinien festgelegt, ist das *Lastenheft*³ „die Zusammenstellung aller Anforderungen des Auftraggebers hinsichtlich Liefer- und Leistungsumfang“ [VDI, 1991]. Im Lastenheft sind die Anforderungen aus Anwendersicht einschließlich aller Randbedingungen – quantifizierbar und prüfbar – zu beschreiben. Im Lastenheft wird definiert, WAS WOFÜR zu lösen ist.

Ziel des Lastenhefts ist es, die Projektaufgabe und das mit der Bearbeitung der Projektaufgabe verfolgte Ziel operational zu beschreiben, d. h., die Ziele des Projekts und dessen Bearbeitungsablauf müssen so verdeutlicht werden, dass sie von allen Projektmitarbeitern unmissverständlich nachvollzogen werden können. Projektziele sind dabei Sachziele („Was soll mit dem Projekt erreicht werden?“) und primär keine Planungsziele (Zeit-, Kostenziele etc.). Das Lastenheft wird aus diesen Gründen auch als *Aufgabenbeschreibung*, *Anforderungsdefinition* und – missverständlich – als *Systemspezifikation* bezeichnet.

Für das Lastenheft wird in [VDI, 1991] folgende Gliederung vorgeschlagen, wobei projektabhängig Punkte ergänzt oder weggelassen werden können:

1. **Einführung in das Projekt, Projektziele:** Veranlassung, Zielpublikum, Zielsetzung, Projektumfeld, wesentliche Aufgaben, Eckdaten,
2. **Beschreibung des Istzustandes:** System, Prozess, Organisation, Datendarstellung und Mengengerüst (Ist),
3. **Beschreibung des Sollzustandes:** Kurzbeschreibung, Anforderungsliste (Analysegraph; Gliederung in Muss-, Soll- und Kann-Anforderungen), Ablaufbeschreibung, Be-

³Im Englischen wird das Lastenheft oftmals als *system requirements specification* bezeichnet. Zu unterscheiden ist es von der *software requirements specification* der IEEE [IEEE, 1994], welche um weitere IEEE-Dokumente – wie etwa *software development plan*, *quality assurance plan*, *statement of work* – erweitert werden muss.

schränkungen, Abgrenzungskriterien (Nicht Enthaltenes), Datendarstellung und Mengengerüst (Soll), zukünftige Erweiterungen,

4. **Schnittstellen:** Systemebene, Protokolle, Mensch–Rechner, Rechner–Rechner, Rechner–externe Prozesse,
5. **Systemanforderungen:** Datenhaltung und -verarbeitung, Entwicklungs- und Zielhardware, Software, technische Merkmale und Leistungsdaten,
6. **Anforderungen an Inbetriebnahme und Einsatz:** Dokumentation, Testen, Inbetriebnahme, Abnahme, Schulung, Betrieb und Wartung,
7. **Qualitätsanforderungen:** Qualitätsmerkmale, Vorgehen bei der Qualitätssicherung, Nachweis der Produktqualität,
8. **Anforderungen an die Projektentwicklung:** Projektorganisation, Projektplanung (Meilensteine), Projektdurchführung (technische, monetäre, rechtliche Abmachungen),
9. **Anhang:** Begriffe und Definitionen, Abkürzungen, Nomenklatur, Gesetze, Normen und Richtlinien, Liste referenzierter Dokumente.

Das Lastenheft ist vom Auftraggeber vollständig und widerspruchsfrei zu erstellen. Es dient als Ausschreibungs-, Angebots- und/oder Vertragsgrundlage. In der Praxis wird jedoch vielfach der Auftragnehmer vom Auftraggeber mit der *Erstellung des Lastenhefts* beauftragt bzw. die Erstellung gemeinsam durchgeführt. Dies hat den gravierenden Nachteil, dass der Vergleich des Analyseberichts mit dem Lastenheft nur mehr begrenzt zur Überprüfung des Problemverständnisses herangezogen werden kann und die Gefahr besteht, das Projekt auf Basis einer ungenauen, unvollständigen oder gar schwerpunktmäßig falschen Analyse fortzusetzen. Das Lastenheft wird vom Auftraggeber geprüft und freigegeben.

Im Fall der Lastenhefterstellung durch den Auftragnehmer ist es entscheidend, sich in die Rolle und Sprachwelt des Auftraggebers zu versetzen und alle Formulierungen für den Auftraggeber verständlich und relevant zu gestalten. Diese Vorgehensweise unterstützt die WAS/WIE-Trennung, da im Lastenheft nur Anforderungen aus der Sicht des Auftraggebers („WAS“) zu beschreiben sind.

Auch bei agilen Vorgehensmethoden wird im Sinn der beiderseitigen Absicherung vielfach ein Lastenheft erstellt, das allerdings nur die Mindestanforderungen an das Produkt enthält.

Checkliste für das Lastenheft

Nachfolgende Checkliste dient zur Vorprüfung des Lastenhefts durch den Auftragnehmer:

- **Machbarkeit:**
 - Sind die Anforderungen vom Auftragnehmer (gemäß erstelltem Plan) realisierbar?
 - Sind die Anforderungen im vom Auftraggeber geforderten Systemumfeld (Zielpattform) realisierbar?
 - Sind die Anforderungen mit ökonomisch vertretbarem Aufwand („vernünftig“) machbar? Gibt es ökonomisch unvernünftige Anforderungen (z. B. 100 %ige Fehlerfreiheit, Interaktivität bei allem und jedem etc.), selbst wenn sie vom Auftraggeber gefordert bzw. finanziert werden?

- **Vollständigkeit:**
 - Fehlt eine in der Zielerforschung (Erhebungsbericht, bisherige Protokolle, Briefe etc.) erhobene Anforderung ohne explizite Angabe von Gründen?
 - Fehlt eine vom Auftragnehmer als im Sinn des State-of-the-Art unverzichtbar eingestufte Anforderung?
- **Minimalität:**
 - Ist jede im Lastenheft enthaltene Anforderung (in der Zielerforschung, persönlich vom Auftraggeber, durch den State-of-the-Art) auch verlangt?
- **Konsistenz:**
 - Sind die Anforderungen untereinander widerspruchsfrei?
- **Eindeutigkeit:**
 - Wird für jedes Attribut ein eindeutiger, festgelegter Ausdruck verwendet?
 - Bei möglicher Mehrdeutigkeit oder vermutlicher Unklarheit für das Zielpublikum: Ist der Ausdruck im Glossar enthalten und dessen genaue Bedeutung festgelegt?
- **Plausibilität:**
 - Ist jede Anforderung für den Auftragnehmer plausibel?
- **Überprüfbarkeit:**
 - Ist die Realisierung der Anforderungen überprüfbar (qualitativ, quantitativ)?
 - Ist die Realisierung der Anforderungen testbar?
- **Modifizierbarkeit:**
 - Ist das Lastenheft übersichtlich gestaltet (Inhaltsverzeichnis vorhanden, adäquates Glossar enthalten, ggf. Index beigegeben)?
 - Ist jede Anforderung separat angeführt?
 - Sind die Anforderungen eindeutig gekennzeichnet (z. B. durch Nummerierung), um sie in späteren Phasen eindeutig referenzieren zu können?
 - Wurde Redundanz vermieden, soweit dies sinnvoll möglich ist? Sind bei notwendigen Redundanzen entsprechende Querverweise angegeben?

10.1.5.3 Machbarkeitsstudie

Ist die Machbarkeit eines Projekts ungewiss, so wird manchmal zuerst nur eine Teilmenge des Lastenhefts ausgearbeitet, die so genannte *Machbarkeitsstudie* (*Durchführbarkeitsstudie*; *feasibility study*). Diese weist im Allgemeinen die gleiche Struktur auf wie ein Lastenheft, konzentriert sich jedoch in allen Gliederungspunkten darauf festzustellen, welche der Anforderungen technisch und wirtschaftlich durchführbar sind. Es wird entweder das gesamte Projekt auf Machbarkeit untersucht, oder es wird versucht, ein machbares (und für den Auftraggeber sinnvolles) Teilprojekt zu definieren. Neben der Prüfung auf *technische Machbarkeit* und *wirtschaftliche Machbarkeit* ist oftmals auch – bei einer vorgegebenen Gruppe von Projektmitarbeitern – die *personelle Machbarkeit* zu überprüfen.

10.2 Planung

10.2.1 Ziel

Das *Ziel der Planung* ist es, Vorgaben für Aufgaben, Aufwände, Termine, Ressourcen, Kosten, Finanzierung sowie begleitende Maßnahmen (z. B. Qualitätssicherung) und vorsorgende Maßnahmen (z. B. Sicherungs- und Notfallplanung) festzulegen (vgl. auch Abschnitt 3.1).

Die erste planerische Tätigkeit in einem Projekt ist das Festlegen der Projektorganisation (*Organisationsplanung*). Eine genaue Festlegung der Leistungen und des Erbringungszeitraums ist jedoch erst auf Grund einer sorgfältigen Anforderungsanalyse möglich (siehe Abschnitt 10.1); auf diese aufbauend erfolgt eine detaillierte Zeit-, Ressourcen- und Kostenplanung.

Im Allgemeinen fehlen zum Zeitpunkt der ersten Projektplanung ausreichend genaue Informationen über alle Projektanforderungen, gerade auch im Fall eines iterativ-inkrementellen Vorgehens. Man geht daher bei der Projektplanung so vor, dass man für das Gesamtprojekt zunächst nur eine Grobplanung und lediglich für die unmittelbar bevorstehenden Teilaufgaben (z. B. für die nächste Iteration) eine detaillierte Planung durchführt. Daraus folgt, dass die Planung mit dem Projektfortschritt kontinuierlich verfeinert werden muss.

10.2.2 Aufgaben

Die *Aufgaben der Planung* ergeben sich aus dem Planungsziel, i. e., man unterteilt in

- Ziel- und Aufgabenplanung,
- Zeitplanung,
- Ablauf- und Terminplanung,
- Ressourcenplanung (inkludiert Sachmittel- und Personalplanung),
- Kosten- und Finanzplanung,
- begleitende und vorsorgende Maßnahmen.

10.2.2.1 Ziel- und Aufgabenplanung

In der *Zielplanung* werden alle für die Überwachung und Steuerung eines Projekts erforderlichen Zielwerte (insbesondere Leistungs-, Termin-, Kostenziele) festgelegt. Die Vorgaben für das angestrebte Ausmaß und den zeitlichen Bezug der Zielerreichung werden durch die weiteren Planungen festgelegt.

In der *Aufgabenplanung* werden den Anforderungen eines Projekts systematisch Aufgaben zugeordnet. Basierend auf der Analyse des Projekts sind überschaubare und klar abgegrenzte Aufgaben zu definieren, inklusive deren Zusammenhänge untereinander. Keine Aufgabe darf vergessen werden, keine Aufgabe wird im Allgemeinen mehrmals vergeben. Es ist besonders

darauf zu achten, dass die Teilaufgaben so präzise und unabhängig von anderen Teilaufgaben beschrieben werden, dass die Durchführenden aus der Teilaufgabenbeschreibung alleine erkennen können, was zu tun ist.

Zusätzlich zu den Teilaufgaben ist der Entscheidungsvorgang der Aufgabenübergänge festzulegen. Dies bedeutet insbesondere die Art der Bewertung der Ergebnisse („Abnahme“ einer Teilaufgabe) sowie den Prozess der Freigabe einer Folgeaufgabe. Die Aufgabenplanung wird im *Aufgabenplan* festgehalten.

10.2.2.2 Zeitplanung

Mit der *Zeitplanung* (auch *Aufwandsplanung* genannt) wird für jede Teilaufgabe der für ihre Durchführung erforderliche Zeitbedarf ermittelt. Dabei sind der Umfang und der Schwierigkeitsgrad der Aufgabe, die zur Unterstützung zur Verfügung stehenden Sachmittel sowie die Fähigkeiten und Fertigkeiten der für die Durchführung der Aufgabe vorgesehenen Personen zu berücksichtigen. Die verschiedenen Teilaspekte der Planung können also nicht voneinander unabhängig betrachtet werden, Planung ist als iterativer Prozess der verschiedenen Planungsaufgaben zu verstehen.

Die Zeitbedarfsermittlung erfolgt mittels Aufwandsschätzungen (vgl. Abschnitt 10.2.3.1). Um diese nicht vollständig von der Erfahrung der Projektgruppe im Gesamten und des Projektleiters im Besonderen abhängig zu machen, ist eine systematische Datensammlung über abgeschlossene Projekte nötig. Dadurch werden die Schätzergebnisse laufend verbessert. Für Verbesserungsmöglichkeiten individueller Softwareentwickler sei z. B. auf die Methoden des *personal software process* bzw. des *team software process* [Humphrey, 2005] verwiesen.

Für die Zeitplanung ist die erfolgte Aufgabenplanung unbedingt erforderlich. Sie wiederum bildet die Grundlage für die Ablauf- und Terminplanung, die Ressourcenplanung und die Kosten- und Finanzplanung. Iterativ lassen sich in mehreren Schritten die einzelnen Planungen konkretisieren und verbessern.

Um auf Unvorhersehbarkeiten reagieren zu können, ist es entscheidend, einen gewissen *Freiraum* (*slack*; vgl. [DeMarco, 2004]) einzuplanen, d. h., eine 100 %ige Ressourcenauslastung in der Planung zu vermeiden. Allerdings ist dabei auch das *parkinsonsche Gesetz* zu berücksichtigen: „Man braucht immer so viel Zeit wie man hat.“

10.2.2.3 Ablauf- und Terminplanung

Durch Verknüpfen der Aufgaben entsprechend den für den Beginn einer neuen Aufgabe notwendig abgeschlossenen Aufgaben wird im Rahmen der *Ablaufplanung* ein Ablaufgraph erstellt. Auf Grund dieses Graphen werden mit der *Terminplanung* für jede Teilaufgabe Anfangs- und Endtermine festgelegt. Sich auf diese Weise ergebende wichtige Zwischentermine des Gesamtprojekts werden als *Meilensteine* bezeichnet. Die Ablauf- und Terminplanung wird daher manchmal zusammenfassend auch *Meilensteinplanung* genannt.

10.2.2.4 Ressourcenplanung

Aufgabe der *Ressourcenplanung* (*Kapazitätsplanung*) ist es, die kapazitätsmäßigen Voraussetzungen für die Projektdurchführung zu schaffen.

Die Ressourcenplanung ist mit der Terminplanung eng verknüpft. Bei der Ressourcenplanung müssen jedoch zusätzlich qualitative, quantitative, örtliche und organisatorische Aspekte berücksichtigt werden. Für jeden Vorgang ist festzustellen, welche und wie viele Mitarbeiter, Maschinen und Materialien für die Durchführung gebraucht werden und wo er durchgeführt wird. Für die Ermittlung des Kapazitätsbedarfs müssen die geplanten Mengen den Terminen zugeordnet werden. Zu Beginn eines Projekts wird man nur sehr grobe Kapazitätsschätzungen vornehmen können, die später entsprechend verfeinert werden.

In größeren Unternehmen oder wenn häufig mehrere Projekte gleichzeitig abgewickelt werden, ist es zweckmäßig, Kapazitätsklassen (Personal mit gleicher Qualifikation, gleichartige Sachmittel) zu bilden und Ressourcen vorerst nur mit entsprechenden Klassen zu belegen. Dadurch kann leicht festgestellt werden, welche Kapazität in welcher Höhe wann benötigt wird bzw. zur Verfügung steht. Es lässt sich dann durch einen Vergleich des Kapazitätsbedarfs überprüfen, ob die vorhandenen Kapazitäten für den Planungsbedarf ausreichen bzw. gleichmäßig ausgelastet werden.

Die Ressourcenplanung kann weiter unterteilt werden in:

- **Sachmittelplanung:** Mit der Sachmittelplanung werden die zur Unterstützung der Aufgabendurchführung erforderlichen Methoden, Werkzeuge und sonstigen Hilfsmittel (z. B. Computer, Räume) festgelegt.
- **Personalplanung:** Mit der Personalplanung werden die nach Anzahl und Qualifikation für die Projektdurchführung erforderlichen Mitarbeiter („Personalbedarf“) bestimmt und zur Projektgruppe zusammengefasst. Dazu werden den Aufgaben des Aufgabenplans Stellen in der Projektorganisation zugeordnet. Zusätzlich muss die Art der Zusammenarbeit der Projektmitarbeiter geregelt werden.

Ein wesentlicher Einflussfaktor für die Ermittlung des Personalbedarfs ist die Entscheidung darüber, welche Projektaufgaben an eigene Mitarbeiter und welche Projektaufgaben extern (z. B. an Berater, andere Softwareunternehmen) vergeben werden („Eigenfertigung“ bzw. „Zukauf“). Der Personalbedarf wird auch maßgeblich bestimmt durch die Form der Projektorganisation, durch die zeitlichen Planungsziele und durch die zur Unterstützung der Aufgabendurchführung zur Verfügung stehenden Sachmittel. Die Zuteilung der Aufgaben zu konkreten Mitarbeitern (*Personaleinsatzplanung*) sowie die Planung des konkreten Materialbedarfs (*Sachmitteleinsatzplanung*) kann teilweise erst in einer Verfeinerung der Planung während des Designs erfolgen.

Die Ablauf- und Terminplanung sowie die Ressourcenplanung sind einem wechselseitigen Abstimmungsprozess unterworfen und stellen daher eine zusätzliche zyklische Verknüpfung im gesamten, iterativen Prozess der Planung dar.

10.2.2.5 Kosten- und Finanzierungsplanung

Mit der *Kostenplanung* werden die kostenmäßigen Konsequenzen aller vorangegangenen Planungsschritte, insbesondere die Sachmittel- und Personalkosten, ermittelt. Dabei ist zwischen Einzelkosten, die direkt einem Projekt zurechenbar sind, und den nicht direkt zurechenbaren Gemeinkosten zu unterscheiden. Letztere werden über Zuschläge zugeordnet.

Eine *Finanzierungsplanung* lässt sich durch die Verbindung des Kostenplans mit dem Terminplan durchführen. Mit ihr wird festgelegt, welche Ausgaben zu welchen Terminen anfallen werden; es wird somit die Budgetierung und Steuerung der Finanzmittelbereitstellung ermöglicht.

10.2.2.6 Begleitende und vorsorgende Maßnahmen

Mit planerischen Aktivitäten verbunden sind auch projektbegleitende Maßnahmen sowie Vorsorgemaßnahmen für Ausnahme- und Notfälle.

Planungen im projektbegleitenden Bereich umfassen beispielsweise:

- **Testplanung** zu der termingerechten Zurverfügungstellung geeigneter Testdaten sowie einer geeigneten Testdurchführung (z. B. Wiederholbarkeit),
- **Installationsplanung und Schulungsplanung**,
- **Qualitätssicherungsplanung** zur Definition der Qualitätsziele und durchgehenden Einhaltung der Qualitätsmaßnahmen,
- **Berichtswesenplanung und Dokumentationswesenplanung**.

Planungen im Vorsorgebereich umfassen beispielsweise:

- **Gewährleistungsplanung und Wartungsplanung** (im Speziellen in Form einer vorausschauenden Personalplanung bzw. einer projektübergreifenden Planung),
- **Sicherheitsplanung und Sicherungsplanung** zur Absicherung sensibler Projektdaten und -ergebnisse sowie zur Vorsorge für den Fall von Datenverlust,
- **Notfallplanung** zur raschen Reaktionsfähigkeit bei Projektverzug, i. e. drastischem Überschreiten von Kosten- oder Zeitzielen bzw. drastischem Unterschreiten von Leistungszielen,
- **Katastrophenplanung** für als unwahrscheinlich einzustufende Ereignisse, die jedoch für das Projekt einen großen Schaden bedeuten, wie z. B. der Ausfall der Entwicklungsumgebung oder von Arbeitsräumlichkeiten, die Einstellung der Weiterentwicklung von Entwicklungswerkzeugen oder der Ausfall wichtiger Mitarbeiter.

Alle Pläne eines Projekts dienen grundsätzlich der erfolgreichen Projektsteuerung. Zusätzlich sind jedoch gute Planungsunterlagen die entscheidende Komponente für die Überprüfung des Projektfortschrittes. Erst durch die Überprüfung und die Einleitung entsprechender Korrekturmaßnahmen kann eine Planung aktuell gehalten und somit sinnvoll zur erfolgreichen Projektentwicklung verwendet werden.

10.2.3 Techniken

10.2.3.1 Aufwandsschätzung

Die Notwendigkeit von *Aufwandsschätzungen* ergibt sich aus der Forderung nach Termin- und Kostenplanung. Aufwandsschätzungen im Bereich der Softwareentwicklung beruhen mangels exakter Methoden fast immer auf dem Vergleich eines Projekts mit früheren Projekten. Das Problem dabei ist, dass wegen der Einmaligkeit von Softwareprojekten die Zahl der vergleichbaren Projekte meistens gering ist und außerdem selten quantitativ abgesicherte Erfahrungswerte zur Verfügung stehen.

Im Bereich des Software Engineering wurden verschiedene Kostenschätzungsmodelle entwickelt, die sich allerdings in der Praxis nur in äußerst bescheidenem Umfang durchsetzen konnten. Die bekanntesten Modelle sind:

- **SLIM** (*software life cycle management*; [Putnam, 1978]), ein Modell, das Entwicklungszeit und Systemgröße mittels einer „Softwaregleichung“ in Beziehung zueinander stellt, in welche die Anzahl erzeugter Programmbefehle, der Entwicklungsaufwand in Mannjahren, die Entwicklungszeit sowie eine „technologische Konstante“ (mittels der das Verfahren auf die Entwicklungsgruppe „geeicht“ werden muss) eingehen,
- **COCOMO II** (*constructive cost model*; [Boehm *et al.*, 2000]), ein Modell zur Ermittlung eines funktionalen Zusammenhangs zwischen der Anzahl erzeugter Programmzeilen und Kosten mittels Korrelationsanalyse,
- **FP** (*function points*, auch *feature points*; [Albrecht und Gaffney, 1983]), eine Methode, die aus dem geschätzten Funktionalitätsumfang den Projektaufwand ermittelt,
- **OP** (*object points*; [Oriolo, 1990]), eine Adaptierung der function points für objektorientierte Entwicklung aus der Notwendigkeit heraus, dass sich bei verschiedenen Entwicklungsmethoden der Aufwand auf unterschiedliche Weise auf die Entwicklungsphasen verteilt,
- **ProBE** (*proxy-based estimating*; [Humphrey, 2005]), eine Methode, die den Projektaufwand auf Grund des Aufwands vergleichbarer Klassen aus früheren Projekten abschätzt.

Untersuchungen haben gezeigt, dass eine Aufwandsschätzung nach Faustregeln nicht signifikant schlechter ist als solche Kostenmodelle (vgl. z. B. [Reifer, 2002], [Olsina *et al.*, 2002]). Trotzdem wäre der Einsatz von Kostenmodellen nützlich und notwendig, da sie eine gewisse Klarstellung der Annahmen über erwartete Programmgröße, Gruppenproduktivität etc. erzwingen.

10.2.3.2 Netzplantechnik

Ein *Netzplan* ist ein Instrument zur zeitlichen und inhaltlichen Planung von Aufgaben [Zäpfel, 2000a]. Die Definition eines Netzplans nach DIN 69900 lautet „alle Verfahren zur Analyse, Beschreibung, Planung, Steuerung und Überwachung von Abläufen auf der Grundlage der Graphentheorie, wobei Zeit, Kosten, Einsatzmittel und weitere Einflussgrößen berücksichtigt werden können“.

Zentrales Element der Netzplantechnik ist der *Vorgang* (*Aktivität, Tätigkeit*). Dieser entspricht einer bestimmten Projektaufgabe. Er besitzt einen genau definierten Beginn und ein festgelegtes Ende. Zudem hat jeder Vorgang mindestens einen Vorläufer (Ausnahme: „Projektbeginn“) und mindestens einen Nachfolger (Ausnahme: „Projektende“).

Der Abschluss eines Vorgangs (oder mehrerer zusammengehöriger Vorgänge) wird als *Ereignis* bezeichnet und dann besonders kenntlich gemacht, wenn dieses Ereignis für den Fortgang und die Beurteilung des Projektstandes (v. a. seitens des Auftraggebers) besonders wichtig ist. Ein solches Ereignis wird als *Meilenstein* bezeichnet.

Jeder Vorgang erfordert Zeit, nämlich die Zeitspanne von seinem Anfang bis zu seinem Ende. Außerdem benötigt jeder Vorgang Ressourcen, die ihm zugeordnet werden.

Durch Angabe der jeweiligen Vorgänger/Nachfolger zu den Vorgängen wird der Vorgangsgraph (eigentlicher „Netzplan“) erstellt. Folgende zeitliche (logische) Abhängigkeiten können dabei definiert werden:

- **Anfang – Ende (AE):** Ein Vorgang kann beginnen, sobald sein Vorgänger abgeschlossen ist.
- **Anfang – Anfang (AA):** Ein Vorgang beginnt genau dann, wenn auch sein Vorgänger beginnt.
- **Ende – Ende (EE):** Ein Vorgang endet genau dann, wenn sein Vorgänger endet.
- **Ende – Anfang (EA):** Ein Vorgang muss abgeschlossen sein, damit sein Vorgänger beginnen kann.

Vorgänge, deren Verzögerung unmittelbar eine Verzögerung des Gesamtprojekts nach sich ziehen, heißen *kritisch*; die übrigen *unkritisch*. In jedem Netzplan gibt es mindestens einen Pfad vom Beginn zum Ende, auf dem alle Vorgänge kritisch sind (*kritischer Pfad*).

Tipp: Speziell im Bereich der Softwareentwicklung hat sich gezeigt, dass auftretende Verzögerungen im Allgemeinen so weit reichende Konsequenzen haben, dass es wenig ausmacht, ob ein Vorgang kritisch ist oder um ein, zwei Tage länger dauern darf, ohne das Projekt zu verzögern. Die Beachtung kritischer Pfade ist somit bei der Projektplanung noch eher relevant, bei der Kontrolle und Umplanung ist jedoch auf alle betroffenen Vorgänge zu achten.

Die verschiedenen Methoden der Netzplantechnik folgen alle derselben Vorgehensweise und unterscheiden sich vor allem durch ihre Darstellungsart. Die wichtigsten Vertreter sind:

- **Metra-Potential Method (MPM):** Bei dieser Methode werden die Vorgänge als Knoten dargestellt. Die Reihenfolgebeziehungen gehen aus den Anordnungsbeziehungen hervor. Man bezeichnet diese Form des Netzplans daher auch als Vorgangsknoten-Netzplan.
- **Critical Path Method (CPM):** Bei dieser Methode werden die Vorgänge als Kanten und die Ereignisse als Knoten dargestellt. Man bezeichnet diese Form des Netzplans daher auch als Ereignisknoten-Netzplan.
- **Program Evaluation and Review Technique (PERT):** Diese Methode basiert auf einem probabilistischen Verfahren, wobei die Ereignisse als Knoten dargestellt werden.

Für jeden Knoten werden drei Zeitwerte geschätzt und daraus die erwartete Dauer abgeleitet. Diese Vorgehensweise steht im Gegensatz zur CPM, die vor allem dann eingesetzt wird, wenn für die einzelnen Vorgänge bereits übertragbare Erfahrungswerte aus früheren Projekten bekannt sind.

Für Details zur Netzplantechnik sei auf die umfangreiche Literatur verwiesen (z. B. [Zäpfel, 2000a], [Zäpfel, 2000b]).

10.2.3.3 Balkenplantechnik

Balkenpläne sind eine tabellarische, grafische Darstellung von Aufwänden (Zeit, Ressourcen etc.). Sie sind in der Praxis weit verbreitet, da sie anschaulich, übersichtlich und ohne Vorkenntnisse verständlich sind. Sie sind auch unter den Bezeichnungen *Balkendiagramm*, *Gantt-Chart*⁴ bzw. *Histogramm* bekannt.

Die Länge der Balken gibt Zeit, Ressourceneinsatz etc. an, die Lage der Balken die zeitliche Folgebeziehung. Die Balken können aber auch kumulativ aufgetragen oder zur Gegenüberstellung von Plan- und Istwerten verwendet werden.

Balkenpläne können leicht aus Netzplänen abgeleitet werden. Sie stellen eine ideale Ergänzung von Netzplänen dar, da in einem Netzplan direkt keine Ressourcenplanung möglich ist. Die Nachteile eines Balkenplans ohne zugehörigen Netzplan sind:

- Ablauflogische Abhängigkeiten oder Zusammenhänge können nicht dargestellt werden.
- Mit zunehmender Projektgröße, Komplexität bzw. Detaillierung nimmt die Übersichtlichkeit rasch ab.

Für Details zur Balkenplantechnik sei auf die umfangreiche Literatur verwiesen (z. B. [Zäpfel, 2000a], [Zäpfel, 2000b]).

10.2.4 Werkzeuge

Zur computerunterstützten Projektplanung werden eine Vielzahl von *Projektplanungssystemen* (oft als *Projektmanagementsysteme* bezeichnet) angeboten. Für eine Übersicht sei auf [PMI, 1999] verwiesen. Alle Systeme ermöglichen ein grundsätzlich sehr ähnliches Arbeiten mit unterschiedlichem Funktionsumfang in Detailbereichen.

Allgemein kann festgehalten werden, dass, wie bei vielen anderen Softwarewerkzeugen auch, diese Systeme sehr gut geeignet sind zur Dokumentation der Planung bzw. zum Abwägen von Alternativen bzw. zum Ändern von Plänen. Die eigentliche Planung selbst (z. B. Zerlegung des Projekts in Vorgänge, Zeitschätzung) sollte aber bereits erfolgt sein, wenn man mit dem Werkzeug zu „planen“ beginnt.

⁴Benannt nach Henry L. Gantt, einem amerikanischen Ingenieur und Sozialwissenschaftler (1861–1919)

Die allgemeine Vorgehensweise gliedert sich in folgende Abschnitte:

1. Anlegen eines neuen Projekts: Eingeben allgemeiner Projektdaten wie Name, Projektbeginn, zu Grunde liegender Kalender etc.,
2. Eingeben der Vorgänge als Liste, Gliedern der Vorgänge und Gruppieren zu Sammelvorgängen (Aufgabenplanung),
3. Zuordnen einer Dauer zu jedem Vorgang (Zeitplanung),
4. Verknüpfen der Vorgänge (Ablaufplanung),
5. Festlegen von Meilensteinen (Terminplanung),
6. Eingeben und Zuordnen von Ressourcen (Ressourcenplanung),
7. Auflösen von Ressourcenkonflikten und Terminkonflikten (Abgleichen der Terminplanung mit der Ressourcenplanung),
8. Eingeben der Kosten (Kostenplanung),
9. Beseitigen von Kostenüberschreitungen bzw. Finanzierungslücken (Kostenplanung, Finanzierungsplanung).

Folgendes ist zu dieser Vorgehensweise anzumerken:

- ad 2. Die Gliederung der Vorgänge kann auch dazu benutzt werden, Details aus- und einzublenden, was v. a. bei größeren Projekten in Balkenplandarstellung sehr hilfreich ist.
- ad 3. Die Zeiteingabe und -ansicht kann im Allgemeinen unabhängig voneinander mit Einheiten versehen werden (Stunden, Tage, Wochen etc.).
- ad 4. Neben der Standardverknüpfung EA können AA, EE sowie AE verwendet werden; zusätzlich oftmals positive und negative Zeitabstände sowie Fixtermine.
- ad 5. Meilensteine werden vielfach als Vorgänge mit der Dauer 0 definiert.
- ad 6. Für jede Ressource kann bei Bedarf ein eigener Kalender definiert werden.
- ad 7. Kapazitätsabgleiche können manuell (z. B. durch Ressourcenerhöhung, Zuweisung zu anderen Ressourcen, neue Anordnung der Vorgänge, Erweiterung der Arbeitszeiten) oder automatisch vorgenommen werden.

Verschiedene Ansichten können eingestellt werden (Tabellen, Formulare, Grafiken, Kalender, Balkenpläne, Netzpläne); teilweise kann auch die Ansicht unterteilt werden und so z. B. die Vorgangsliste mit einer Terminübersicht als Balkenplan kombiniert werden.

10.2.5 Ergebnisse

Gemäß den verschiedenen Planungsaufgaben werden im Allgemeinen folgende Pläne erstellt (die Ergebnisse der Zielplanung fließen in die anderen Pläne ein bzw. werden in den Analysebericht eingebunden) :

- **Aufgabenplan** (*Projektstrukturplan – PSP, work breakdown structure – WBS*),
- **Terminplan** (enthält Zeit- und Ablaufplan),
- **Meilensteinliste** (als kompakte Terminübersicht, die nur die entsprechenden Sammelvorgänge enthält),
- **Ressourcenplan** (*Ressourcenbelegungsplan, Stellenzuordnungsplan*; evtl. eigener Plan für jede Ressource),
- **Kostenplan** und **Finanzierungsplan**.

Zusätzliche Pläne (z. B. für begleitende und vorsorgende Maßnahmen) können ins Projekthandbuch aufgenommen werden oder werden nur intern verbreitet.

Bei jedem Planungsdokument ist genau zu überdenken, in welchem Detaillierungsgrad es dem Auftraggeber zur Verfügung gestellt wird. Auf jeden Fall ist dem Auftraggeber eine Meilensteinliste mit einer Grobgliederung des Projekts in Form der Sammelvorgänge zugänglich zu machen.

10.3 Risikobehandlung

10.3.1 Ziel

[Gilb, 1996] schreibt sehr plakativ: „Wenn du das Risiko nicht aktiv attackierst [= behandelst], so wird es dich aktiv attackieren.“ Um das *Ziel der Risikobehandlung* definieren zu können, ist es zuerst notwendig, den Begriff des Risikos zu definieren. Außerdem werden vorab Risiko von Chance und Problem abgegrenzt und die wichtigsten Projektrisiken in der Softwareentwicklung präsentiert.

10.3.1.1 Risiko und Projektrisiko

Der Risikobegriff

Ein *Risiko (risk)* ist die Möglichkeit, dass eine Aktivität einen Verlust oder Schaden bewirkt [Webster's, 2005]. Von Risiko spricht man jedoch nur dann, wenn die Folgen ungewiss sind. Ein Risiko stellt also ein potenzielles Problem dar („Die Probleme von morgen sind die Risiken von heute.“ [Kulik, 1996]). Ein *Problem* ist daher ein Risiko, das eingetreten ist.

Ein *Projektrisiko* ist ein potenzielles Problem, das dem Projekt abträglich ist für den Fall, dass es eintritt [Thayer und Fairley, 1997]. Unter dem Projektrisiko wird in der Praxis jedoch meistens das mit der Durchführung des Projekts verbundene *Gesamtrisiko* verstanden, das sich aus dem Zusammenspiel der *Einzelrisiken* ergibt.

Abgrenzung von Risiko, Chance und Problem

[Charette, 1990] unterscheidet zwischen Risiken, Chancen und Problemen: „Ein *Risiko* ist die Wahrscheinlichkeit ungewünschter Konsequenzen eines Ereignisses oder einer Entscheidung. Jedes Risiko hat eine Wahrscheinlichkeit und eine Wirkung. Auswahlmöglichkeiten können beide Faktoren beeinflussen. Risiko entsteht durch die Unsicherheit bezüglich des Zeitverlaufs, der Beherrschung des Projektablaufs und der die Auswahl umgebenden Informationen.

Eine *Chance* (*opportunity*) ist die Möglichkeit, die Erwartungen des Programms zu übertreffen im Gegensatz zum Risiko, das die Möglichkeit darstellt, die Erwartungen an das Programm nicht zu erreichen. Da Erwartungen im Laufe der Zeit steigen, können heutige Chancen zu morgigen Risiken werden.

Ein *Problem* (*problem*) ist die Konsequenz einer vergangenen (Risiko-)Entscheidung oder eines Ereignisses, die/das zur Nichterfüllung der Erwartungen geführt hat. Auch wenn die Konsequenz erst in der Zukunft eintreten wird, wir aber keine Auswahlmöglichkeiten haben und so die negative Konsequenz nicht abwenden können, sprechen wir nicht von einem Risiko, sondern von einem Problem (*vorhersehbares Problem*).“

Die Notwendigkeit von Risiken

Ein Projekt ohne Risiken (und sich dadurch ergebende Probleme) gibt es nicht. „Wenn ein Projekt erfolgreich ist, so ist es das nicht deshalb, weil es keine Risiken und Probleme gab, sondern weil diese erfolgreich behandelt wurden.“ [Rook, 1991]

Ein Risiko stellt selbst auch eine wirtschaftliche Chance dar, wie Peter Drucker anmerkt: „Eines der am striktesten geltenden Gesetze der Wirtschaft, das Gesetz von Böhm-Bawerk, zeigt, dass die verfügbaren Produktionsmittel größere wirtschaftliche Performanz nur durch größere Unsicherheit erlangen, das heißt durch größeres eingegangenes Risiko. Es ist also dumm zu versuchen Risiken vollständig zu eliminieren und es ist auch fraglich sie minimieren zu wollen, es ist aber entscheidend die richtigen Risiken auf sich zu nehmen. ... Wir müssen fähig sein, rational zwischen risikobehafteten Projektpfaden zu entscheiden, anstatt in die Unsicherheit hineinzutappen auf der Basis von Instinkt, Gerüchten oder unvollständigem Wissen, egal wie kompliziert sie mathematisch belegt würde.“ [Drucker, 1975]

10.3.1.2 Risikoeigenschaften

Viele Ereignisse treten während einer Softwareentwicklung auf, welche die Entwicklung negativ beeinflussen können. Ob ein solches Ereignis als Risiko bezeichnet werden kann, lässt sich anhand der folgenden *Risikoeigenschaften* feststellen (vgl. auch [Pfleger, 2001]):

- **Wahrscheinlichkeit:** Die Wahrscheinlichkeit, dass ein für den Projektverlauf negatives Ereignis eintreten wird, muss abgeschätzt werden können. Diese *Risikowahrscheinlichkeit* (*risk probability*) muss größer als 0 (Unmöglichkeit des Eintretens) und kleiner als 1 (Sicherheit des Eintretens) sein. Wenn die Risikowahrscheinlichkeit 1 beträgt, so spricht man von einem – in der Zukunft eintretenden – Problem. Die Ausprägungen des Ereignisses und ihre Wahrscheinlichkeiten sind in einer Wahrscheinlichkeitsverteilungs-

funktion festgehalten, die vielfach auch den Zeitverlauf mit berücksichtigt. Auch die Festlegung dieser Funktion ist normalerweise mit gewissen Unsicherheiten verbunden.

- **Auswirkung:** Ein Ereignis muss resultierende – negative – Konsequenzen (*Risikoauswirkung*; *risk impact*) für das Projekt haben. Die Größenordnung dieser Konsequenzen muss abschätzbar sein (qualitativ oder quantitativ).
- **Zeitraumen:** Es muss eine gewisse Zeit zur Verfügung stehen, dem Risiko vorsorgend („proaktiv“) zu begegnen und die geeignetste Behandlung auszuwählen. Bereits vor dem Eintreten der Problemwirkung kann jedoch ein Risiko auch dadurch zum Problem werden, dass aus Zeitmangel keine Auswahlmöglichkeit der Behandlung mehr besteht.
- **Behandlungsmöglichkeit:** Die *Möglichkeit der Risikobehandlung* (*risk manageability*) beschreibt den Grad, in dem die Risikoauswirkung beeinflusst werden kann. Für jedes Risiko muss festgelegt werden, inwieweit es minimiert oder seine Auswirkungen reduziert werden können bzw. sollen.
- **Kopplung:** Viele Risiken und Chancen sind miteinander verbunden. Diese *Risikokopplung* (*risk coupling*) bewirkt, dass, wenn ein Risiko zu einem Problem wird, es die Wahrscheinlichkeiten anderer Risiken verändern kann, deren Auswirkung ändern kann, deren Behandlungsmöglichkeiten beschränkt oder ihren Zeitrahmen ändert.

10.3.1.3 Die zehn wichtigsten Risiken bei Softwareprojekten

Von Barry Boehm, einem der Pioniere der Risikobehandlung im Softwarebereich, wurde Ende der achtziger Jahre erstmals eine „Hitliste“ der wichtigsten Softwarerisiken aufgestellt [Boehm, 1991], die regelmäßig von ihm aktualisiert wird. Abbildung 10.5 (aus [Boehm, 1998]) vergleicht die zehn wichtigsten Risiken bei Softwareprojekten der achtziger Jahre mit denen Mitte der neunziger Jahre.⁵

Auffällig dabei ist, dass der Prozessbegriff in der älteren Liste gar nicht aufscheint, in der Liste der neunziger Jahre aber – an zweiter Stelle – den „höchsten Neueinstieg“ darstellt. [Hall, 1998] merkt dazu an, dass die Bedeutung des Prozesses für die Softwareentwicklung wahrscheinlich der in den achtziger Jahren am meisten unterschätzte Entwicklungsfaktor war. Das einzige neu dazu gekommene Risikoelement stellt die Verwendung von Legacy-Software dar, also die Verwendung historisch gewachsener, archaischer Systeme. Dieses Problem glaubte man in den achtziger Jahren noch mittels Neuimplementierung lösen zu können, was sich als Fehleinschätzung herausstellte.

10.3.1.4 Risikobehandlung

Risikobehandlung (*risk management*) ist die proaktive Annahme, dass mit einer gewissen Wahrscheinlichkeit in einem Projekt Probleme auftauchen können. Dieses geschieht, indem man diese Wahrscheinlichkeit abschätzt, die Auswirkung der Probleme analysiert und geeignete Problemlösungen im Vorhinein vorbereitet [Bennatan, 2000]. Auf diese Weise wird eine Softwareprojekt-Entwicklung „ohne Überraschungen“ angestrebt.

⁵Für das aktuelle Jahrzehnt wurden noch keine Werte publiziert.

Top 10 Risikoelemente		
	Achtziger Jahre	Neunziger Jahre
1	personelle Defizite	personelle Defizite
2	unrealistische Termin- und Kostenvorgaben	unrealistische Termin- und Kostenvorgaben, ungenügende Beachtung des Prozesses
3	Entwicklung falscher Produkteigenschaften	Defizite bei zugekauften Komponenten (COTS)
4	schlecht gestaltete Benutzerschnittstelle	falsch verstandene Produkteigenschaften
5	„Vergolden“ (Realisieren unnötiger Eigenschaften)	schlecht gestaltete Benutzerschnittstelle
6	schleichende Änderungen der Funktionalitäten	schlechte Architektur, Performanz, Qualität allgemein
7	Defizite bei zugekauften Komponenten	Entwicklung falscher Produkteigenschaften
8	Defizite bei extern vergebenen Aufgaben	Aufbau auf oder Einbeziehung von Legacy-Software
9	(Echt-)Zeitperformanz	Defizite bei extern vergebenen Aufgaben
10	Überfordern der Technologie	Überfordern der Technologie

Abbildung 10.5: Die zehn wichtigsten Risiken bei Softwareprojekten Mitte der achtziger Jahre und Mitte der neunziger Jahre

Die *Hauptziele der Risikobehandlung* sind folgende (teilweise entnommen aus [Ijäs, 1996]):

- Risikoelemente identifizieren, analysieren und priorisieren, bevor sie zu projektgefährdenden Problemen werden können bzw. umfangreiche Umarbeiten verursachen,
- eine balancierte und integrierte Strategie entwickeln, die Risiken zu reduzieren bzw. zu eliminieren und die Ausführung dieser Strategie überwachen,
- Softwareentwicklungskosten reduzieren, indem die Notwendigkeit für Umarbeiten während des Projekts verringert wird,
- den Entscheidungsfindungsprozess weniger subjektiv, systematischer und besser nachvollziehbar gestalten und damit auch die strategische Planung im Projekt unterstützen,
- die Bedeutung jedes Risikos für den gesamten Projekterfolg verdeutlichen,
- die Verantwortung des Auftragnehmers gegenüber dem Auftraggeber bezüglich des Projekts zeigen,
- die Einsicht bei Auftraggeber und Auftragnehmer stärken, dass ein Projekt einen sehr unterschiedlichen Erfolgsgrad aufweisen kann, und die Gründe dafür transparent machen.

Trotzdem wird eine aktive Behandlung von Risiken in Projekten von vielen Unternehmen nicht durchgeführt.⁶ Als Gründe dafür werden genannt [Boehm, 1998] [Patzak und Rattay, 1997]:

⁶Laut einer Umfrage der Fa. Compuware aus dem Jahr 2004 quantifizierten zwei Drittel von 320 befragten Softwareunternehmen in Europa die Risiken nicht [Aigner, 2004].

- die mangelnde Bereitschaft zuzugeben, dass Risiken existieren („Risiken könnten den Eindruck hinterlassen, dass man nicht genau weiß, was man tut.“ „Das Denken an Risiken ist schlecht für erfolgsorientiertes Arbeiten.“),
- die Tendenz, die schwer zu lösenden Projektaufgaben zum Projektende hin zu verschieben („Vielleicht verschwinden die Probleme von selbst.“ „Vielleicht werden die Probleme einfacher, sobald wir die einfachen Aufgaben gelöst haben.“),
- die Einstellung, dass die wesentlichen Risiken ohnedies bekannt sind und nicht beeinflusst werden können (Meist wird den Risiken, die von außen auf das Projekt einwirken und ursächlich nicht beeinflussbar sind, im Vergleich zu Risiken, die bei der Arbeit am Projekt selbst verursacht werden, von den Projektverantwortlichen deutlich mehr Beachtung geschenkt [Patzak und Rattay, 1997].),
- die Annahme, dass Projekte mit zu hohem Risiko ohnedies nicht übernommen werden (Diese Annahme widerspricht dem Streben nach Gewinn, das bei vielen Projekten naturgemäß eine Rolle spielen muss, vgl. Abschnitt 10.3.1.1.),
- die Kosten, die jedenfalls verursacht werden, bevor der Nutzen eintritt.

Im Projekt Risikobehandlung durchzuführen bedeutet vorbereitet zu sein. Die Risikobehandlung ist als eine Form der Versicherung zu verstehen, dass, falls ein Problem auftritt, eine Lösung unmittelbar vorhanden ist. Vielfach ist nämlich ein Problem viel einfacher (und billiger) zu lösen, das rechtzeitig analysiert und abgefangen wurde, als eines, das unerwartet auftritt.

Zu wissen, dass es in jedem Projekt Risiken gibt und wie sie behandelt werden können, ist ein Zeichen professioneller Kompetenz. Daher wird für eine „reife“ Organisation im Sinn der Reifegradmodelle die Risikobehandlung als Kernprozessbereich vorgeschrieben [Adameitz, 1999], so z. B. in CMMI [Kulpa und Johnson, 2003] und ISO/IEC 15504 [ISO, 1999].

Bei agilem Vorgehen ist eine laufende Risikobehandlung sogar noch wichtiger, da durch die Unsicherheit über Produktanforderungen und Projektlaufzeit eine strategische Planung schwer bis unmöglich ist. Durch die häufige Möglichkeit der Adaptierung ergeben sich aber bei agilem Vorgehen durch das Eingehen von Risiken viele Chancen.

Es wurde in der Literatur darauf hingewiesen, dass Gruppen bei der Durchführung der Aktivitäten der Risikobehandlung deutlich bessere Leistungen erbringen als Einzelpersonen [Gemmer, 1997]. Eine risikogesteuerte Vorgehensmethode, wie z. B. das Spiralmodell (vgl. Abschnitt 6.4.3.2), ermöglicht die beste Integration der Risikobehandlung in die Projektentwicklung.

10.3.1.5 Situierung der Risikobehandlung in der Projektentwicklung

Viele Techniken der Projektentwicklung zielen darauf ab, das Projektrisiko zu verringern. So sind beispielsweise Projektplanung, Konfigurationsmanagement und Testen Aktivitäten, die erwiesenermaßen das Projektrisiko verringern [Thayer und Fairley, 1997]. In diesem Sinn erfolgt auch in der Projektentwicklung im Großen eine Risikobehandlung. Und doch besteht im Vorgehen ein entscheidender Unterschied: Probleme werden in der Projektentwicklung dann behandelt, wenn sie auftauchen; die Projektentwicklung ist also *reaktiv*, während die

Risikobehandlung *proaktiv* ist. In [Charette *et al.*, 1997] wird dazu formuliert: „Belohne smarte Arbeit, nicht harte Arbeit!“

Eine proaktive Behandlung aller möglichen Projektprobleme wäre viel zu zeitaufwändig und teuer. Daher muss zur korrekten *Situierung der Risikobehandlung* zwischen allgemeinen Risiken und projektspezifischen Risiken eines Projekts unterschieden werden [Rook, 1991]:

- *Allgemeine Risiken* sind diejenigen, die allen Softwareprojekten gemein sind, wie z. B. der Verlust von Schlüsselpersonal, zu wenig Kommunikation oder die Veranschlagung von zu wenig Testzeit.
- *Projektspezifische Risiken* sind vom konkreten Projekt bestimmt. So kann z. B. ein Zulieferer einen riskanten Liefertermin für die Basissoftware versprechen oder die Benutzerschnittstelle nicht an den Benutzerkreis angepasst sein.

In der Risikobehandlung werden nur projektspezifische Risiken berücksichtigt. Die Trennung ist allerdings vielfach nicht scharf möglich und erfordert Erfahrung sowie gutes Einschätzungsvermögen der Mitarbeiter bei Auftraggeber und Auftragnehmer.

10.3.2 Aufgaben

[Boehm, 1991] teilt das Vorgehen bei der Risikobehandlung in sechs Aufgaben ein, die in Abbildung 10.6 ersichtlich sind:

- Die *Risiko-Bewertung* besteht aus Risiko-Identifikation, Risiko-Analyse und Risiko-Priorisierung.
- Die *Risiko-Beherrschung* besteht aus Risiko-Vorsorgeplanung, Risiko-Überwachung und Risiko-Überwindung.

Eine ähnliche Unterteilung wurde in ISO/IEC 15504 vorgenommen: In ISO/IEC 15504 ist die Risikobehandlung als Teil der Projekt-Prozess-Kategorien inkludiert [ISO, 1999].

Die Risikobehandlung ist nicht ohne Kosten möglich, kann und soll aber kosteneffizient sein, indem der Nutzen überwiegt. Da die Risikobehandlung jedoch auf das Vermeiden von Problemen abzielt, kann gerade eine gute Risikobehandlung in die Situation geraten, die entstandenen Kosten zu rechtfertigen, die für die Analyse von Problemen aufgewendet wurden, die nie eingetreten sind. Eine fundierte Kosten-Nutzen-Analyse ist zur Rechtfertigung der Aktivitäten im Rahmen der Risikobehandlung daher unerlässlich. [Pressman, 2004]

10.3.2.1 Risiko-Identifikation

In der *Risiko-Identifikation* (*risk identification*) wird eine Liste der projektspezifischen Risikoelemente erstellt, die den Projekterfolg gefährden könnten. Der Sinn dabei ist nicht, möglichst viele Risiken aufzulisten, indem höchst unwahrscheinliche (Horror-)Szenarien erfunden werden, sondern diejenigen Risiken zu finden, die eine realistische Chance haben zu einem Problem zu werden und die einen signifikanten Einfluss auf den Projektfortgang haben

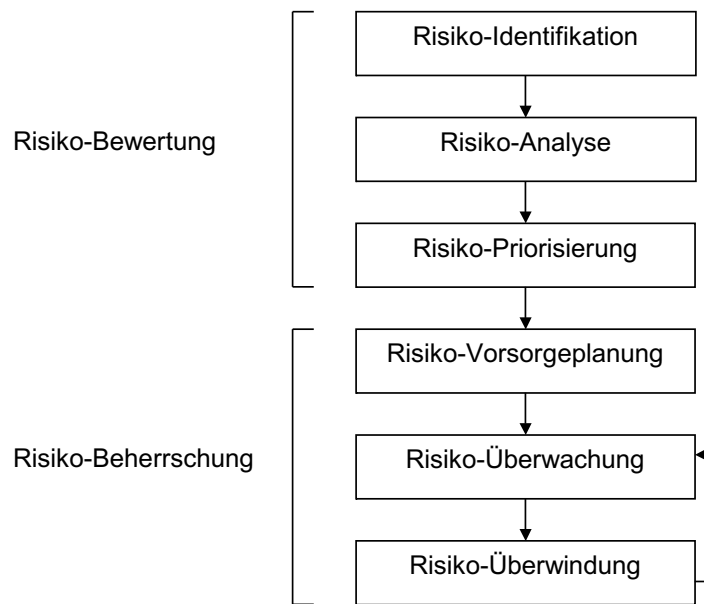


Abbildung 10.6: Aufgabenschritte der Risikobehandlung

[Bennatan, 2000]. Zusätzlich sind immer Reserven für die „unbekannten Unbekannten“ – also das Risiko des Übersehens von Risiken – zu bilden.

Folgende Quellen können zur Risiko-Identifikation herangezogen werden:

- Ein erster Schritt zur Identifikation der möglichen Projektrisiken ist das Durcharbeiten aller bisher erstellten und erhaltenen Dokumente, insbesondere des Analyseberichts bzw. des Lastenhefts sowie des Projektplans. Sofern bereits Vorarbeiten für das Design (z. B. in Form von Prototypen) gemacht wurden, ist auch die Designbeschreibung heranzuziehen.
- Sehr sinnvoll ist es auch, alle vom Auftragnehmer durchgeführten, ähnlich gelagerten Projekte (und hier insbesondere die Projektabschlussberichte) durchzusehen und eine Liste der „lessons learned“ zu erstellen [Donaldson und Siegel, 1997]. Jede der gewonnenen Einsichten ist auf Anwendbarkeit auf das derzeitige Projekt zu prüfen.
- Aus allen identifizierten Attributen in den Dokumenten sind diejenigen herauszusuchen, bei denen sich entweder ihre Abschätzungen (**best**, **worst**, **planned**, **now**) sehr stark unterscheiden oder **planned** sehr weit von **now** entfernt liegt oder sehr nahe bei **best**.
- Alle Schätzwerte mit einer hohen möglichen Abweichung (Maximalwert – Minimalwert) müssen genau untersucht werden.
- In [Gemmer, 1997] wird vorgeschlagen „intelligent“ zu diskutieren. Dies bedeutet, genau zwischen Risiken, Chancen und Problemen zu unterscheiden, Unsicherheiten (betreffen den Prozess) von Wahrscheinlichkeiten (betreffen das Ergebnis) zu trennen und immer die Ursachen der Risiken zu formulieren.

Häufig findet man in Projekten, dass kritische Projektziele, -budgets und -pläne nicht nur sehr vage formuliert werden, sondern dass über den Grad der Unsicherheit der Angaben keinerlei Aussagen gemacht werden! [Gilb, 1996]

10.3.2.2 Risiko-Analyse

Im Rahmen der *Risiko-Analyse* (*risk analysis*; auch *risk estimation*) werden die Schadenswahrscheinlichkeit und das potenzielle Schadensausmaß für jedes identifizierte Risikoelement geschätzt. Zusätzlich werden die Zusammenhänge zwischen den einzelnen Risikoelementen analysiert. Wie in [Bennatan, 2000] angemerkt wird, sollen diese Schätzungen immer von mehr als einem Mitglied der Entwicklungsgruppe durchgeführt werden. Weiters ist es empfehlenswert, die Schätzungen für alle Risikoelemente in einem einzigen Treffen durchzuführen, um die relative Schwere der Auswirkungen auf gleicher Basis zu beurteilen und dadurch die Einschätzungen eher vergleichbar zu halten.

Das potenzielle Schadensausmaß kann entweder quantitativ oder qualitativ bewertet werden. Obwohl vielfach eine qualitative Beurteilung ausreichend erscheinen und auch „ehrlicher“ klingen mag (Einteilung z. B. in vernachlässigbaren, leichten, schweren und katastrophalen Schaden), hat sich eine quantitative Beurteilung sowohl der Schadenswahrscheinlichkeit als auch des Schadensausmaßes am besten bewährt [Bennatan, 2000].

10.3.2.3 Risiko-Priorisierung

Die *Risiko-Priorisierung* (*risk prioritization*) reiht die Risikoelemente entsprechend ihrem Einfluss auf das Projekt. Zur Bestimmung des Einflusses eines Risikoelements hat sich ein kombinierter Faktor aus Schadenswahrscheinlichkeit und potenziellem Schadensausmaß, der *Risikofaktor*, am besten bewährt [Balzert, 2000b]:

1. Jedem Risikoelement wird ein Erwartungswert zwischen 1 und 10 zugewiesen, normalerweise direkt proportional zur Wahrscheinlichkeit.
2. Jedem Risikoelement wird ein Schadenskoeffizient zwischen 1 und 10 zugewiesen (10 bedeutet katastrophales Schadensausmaß).
3. Der Risikofaktor für jedes Risikoelement ergibt sich aus der Multiplikation des Erwartungswerts mit dem Schadenskoeffizienten.

Neben der rein mathematischen Reihung ist aber auch der Hausverstand der an der Priorisierung beteiligten Projektmitarbeiter gefordert. Eine aus dem Projektverständnis heraus erfolgte – nicht mathematisch ermittelte – Umreihung kann durchaus sinnvoll sein, muss aber schriftlich begründet werden. Entscheidend am Ergebnis sind nicht, wie [Gilb, 1996] anmerkt, die exakten Zahlen als quantitative Feststellung, sondern das Vertrauen, das die Projektmitarbeiter in die Zahlen besitzen und wie sie jene interpretieren. Jedenfalls muss bei der Bewertung und der Reihung in der Gruppe Konsens erzielt werden, wobei sich jeder Mitarbeiter bewusst sein muss, dass die Reihung auf der Basis von Schätzungen erfolgt. Daher ist jede Entscheidung am besten schriftlich festzuhalten und ausführlich zu begründen.

Das Ergebnis der Risiko-Priorisierung ist eine „watch list“, eine Liste, in der die identifizierten Risikoelemente gemäß ihrem Risikofaktor geordnet sind. Dabei entschließt man sich vielfach, die weniger hoch priorisierten Risikoelemente nicht in eine Überwachung mit aufzunehmen, sondern sich auf die wesentlichsten N Risiken zu konzentrieren (*Top-N-Liste der Risikoverfolgung*, vgl. Abschnitt 10.3.5.1).

10.3.2.4 Risiko-Vorsorgeplanung

Aufgabe der *Risiko-Vorsorgeplanung* (*risk contingency planning*) ist es, für jedes als behandelnswert klassifizierte Risiko einen Plan festzulegen, der die notwendigen Aktivitäten festschreibt. Im Allgemeinen werden diese Vorsorgepläne (siehe Abschnitt 10.3.5.2) in der Projektbibliothek abgelegt, um bei Bedarf später verwendet zu werden. Wenn der Vorsorgeplan aber bestimmte Vorbereitungen oder Parallelentwicklungen notwendig macht (z. B. die Erstellung eines Softwaresimulators für eine Hardware, die möglicherweise zu spät geliefert wird), so kann sich daraus ein eigenes Subprojekt im Rahmen des Gesamtprojekts ergeben.

Wichtig bei der Vorsorgeplanung ist es, nicht nur den wahrscheinlichsten Fall zu planen, sondern in die Überlegungen auch den schlechtesten und den besten anzunehmenden Fall einzubeziehen und entsprechende Vorgehensalternativen zu planen.

10.3.2.5 Risiko-Überwachung

Wie in [Patzak und Rattay, 1997] erläutert wird, muss die *Risiko-Überwachung* (*risk monitoring*) spätestens am Ende der Analysephase beginnen und stellt keine Einmalaktivität dar. Dabei wird jedes Risiko und der entsprechende Vorsorgeplan überprüft, ob sich in der Einschätzung des Risikos eine Änderung ergeben hat und entweder der bestehende Vorsorgeplan eine Aktivität verlangt oder der Plan adaptiert werden muss.

Eine der häufigsten Fallen bei der Risiko-Überwachung ist die Versuchung für Projektmitarbeiter, sich nicht einzugestehen, dass sich ein Risiko in ein Problem verwandelt hat und damit Aktivitäten getätigt werden müssen [Thayer und Fairley, 1997]. Jedoch kann sich jede kurze Projektverzögerung aufschaukeln („Wie erreicht ein Projekt ein Jahr Verzug? ... Einen Tag jeden Tag.“ [Brooks, 1975]). Es ist daher unbedingt zu vermeiden, den Mitarbeiter, der entdeckt, dass aus einem bestimmten Risiko ein Problem geworden ist, mit Zusatzarbeit zu belasten („Kommen Sie nicht mit einem Problem zu mir, kommen Sie mit einer Lösung!“) oder gar zu bestrafen („Shoot-the-Messenger-Prinzip“).

Im Laufe des Projektfortschritts können zusätzliche Probleme vorhergesehen werden oder Risiken können abgehakt werden. Mit zusätzlichen Informationen im Laufe des Projektfortschritts können vielfach die Wahrscheinlichkeiten und/oder die Auswirkungen einzelner Risiken besser abgeschätzt werden. Daher sollte die Top-*N*-Liste möglichst periodisch aktualisiert werden. Aus den Top-*N* herausfallende Risiken oder ursprünglich auf Grund ihrer geringeren Wichtigkeit nicht aufgenommene Risiken sind dabei nicht vollständig zu entfernen, sondern kommen in eine Warteliste, die bei jeder Aktualisierung der Top-*N*-Liste berücksichtigt wird. Ein guter Zeitpunkt dafür ist die Erhebung der Projektsituation für den Projektfortschrittsbericht durch den Projektleiter (etwa alle zwei Wochen).

10.3.2.6 Risiko-Überwindung

Im Rahmen der *Risiko-Überwindung* (*risk resolution*) wird die in der Risiko-Vorsorgeplanung festgelegte Technik der Risikobehandlung (vgl. dazu das nächste Kapitel) angewendet. In [Donaldson und Siegel, 1997] wird dazu angemerkt, dass in vielen Fällen der Risiko-Überwindung der Auftraggeber mit einbezogen werden muss, entweder durch direkt von ihm

durchzuführende Aktivitäten oder zumindest durch Information über die geänderte Projektsituation. Auch aus diesem Grund erfolgt die Festlegung einer geeigneten *Risiko-Beherrschungsstrategie* beim Auftragnehmer normalerweise eine Hierarchieebene über derjenigen, in der die Risiko-Bewertung stattfindet [Gemmer, 1997].

10.3.3 Techniken

10.3.3.1 Abschätzen des Risikoverhaltens

Die *Techniken der Risikobehandlung* sind, unabhängig vom Projektverlauf, immer dem Risikoverhalten von Auftraggeber und Auftragnehmer anzupassen. Das *Risikoverhalten* (*risk behavior*) von Individuen, Gruppen und Organisationen wird, abhängig von der jeweiligen Situation, in *risikofreudig*, *risikoindifferant* und *risikoscheu* unterteilt [Patzak und Rattay, 1997].

Für die Risikobehandlung ist eine Auseinandersetzung mit dem Risikoverhalten der Entscheidungsträger des Projekts wie auch der sonstigen Projektbeteiligten sehr sinnvoll. Entscheidungen sind häufig die Folge von Verhandlungen. Dabei ist die Kenntnis der Risikoneigung der Verhandlungspartner sehr hilfreich, da in Verträgen speziell die Risikoabgrenzung, d. h., die Zuweisung, wer ein bestimmtes Risiko trägt, vorgenommen wird. Besonders bei externen Projekten sind diese Erkenntnisse für Angebotserstellung und Vertragsverhandlungen von großer Bedeutung.

Gerade bei Geschäften mit großem Auftragsvolumen, wo Entscheidungsfindung und Zuschläge auf dem Konkurrenzweg erfolgen, ist es für den jeweiligen Anbieter erforderlich, sein Risikoverhalten dem des potenziellen Auftraggebers anzupassen, um erfolgreich zu sein. Dies bedeutet, dass in der Angebotsphase Anbieter zumeist risikofreudig sind, sich nach Auftragserhalt aber eher risikoscheu verhalten. Anbieter sind gegenüber Kunden bei Vertragsverhandlungen oft bereit, Konzessionen zu machen und sich vertraglich zu verpflichten, Risiken zu übernehmen. Bei der Durchführung des Projekts nach Beauftragung durch den Kunden wird dann aber (oft zu spät) versucht, die übernommenen Risiken möglichst auf andere Risikoträger abzuwälzen.

Beispiele

Die in den folgenden Kapiteln vorgestellten Techniken der Risikobehandlung werden an zwei Beispielen (adaptiert aus [Boehm, 1998]) veranschaulicht, die Projektrisiken darstellen können:

1. **Risiko eines Autounfalls:** Ein Projektmitarbeiter könnte durch einen Autounfall verletzt werden, während er zur Arbeit fährt.
2. **Risiko eines Plattencrashes:** Die Festplatte eines für das Projekt entscheidenden Computers kann kaputtgehen.

Für jedes dieser Risiken werden nachfolgend pro Technik eine oder mehrere Behandlungsalternativen vorgeschlagen.

10.3.3.2 Risiko-Vermeidung

Die *Risiko-Vermeidung* (*risk avoidance*) eliminiert die Möglichkeit eines bestimmten Risikos durch die Wahl eines alternativen Vorgehens im Projekt, das durch ein niedrigeres Risiko charakterisiert wird.

Die Wahl eines Projektpfades mit geringerem Risiko ist allerdings nicht immer projektdienlich: Ein risikoreicheres Vorgehen kann größere Flexibilität (z. B. im Design) bedeuten, Kosten reduzieren, Gewinne erhöhen oder einen früheren Liefertermin ermöglichen. Wie [Thayer und Fairley, 1997] anmerken, ist die Risiko-Vermeidung in einem Bereich oft mit der Erhöhung des Risikos in einem anderen Bereich verbunden.

Beispiele

1. **Risiko eines Autounfalls:** Der Mitarbeiter verlegt seine Wohnung in die Nähe der Arbeitsstätte und geht zu Fuß. Alternativ benutzt er öffentliche Verkehrsmittel.
2. **Risiko eines Plattencrashes:** Anstelle der Platte wird eine beschreibbare CD/DVD verwendet.

10.3.3.3 Risiko-Reduktion

Die Basis der *Risiko-Reduktion* (*risk mitigation*) ist eine genaue Untersuchung der Risiko-ursachen. Daraus wird ein Risiko-Reduktionsplan erstellt, dessen Aktivitäten umgesetzt werden, bevor das Risiko zu einem Problem wird. Auch durch Akquisition von zusätzlichem Wissen über das Projekt allgemein und das spezifische Risiko im Besonderen (z. B. durch Prototyping, Simulation, inkrementelle Entwicklung; vgl. [Thayer und Fairley, 1997]) kann das Risiko reduziert werden.

Beispiele

1. **Risiko eines Autounfalls:** Die Fahrgeschwindigkeit wird verringert. Die Sicherheitsgurte werden immer angelegt. Ins Auto werden zusätzliche Verstrebungen oder Airbags eingebaut. Gemäß einem Bericht über Crashtests wird ein sichereres Auto angeschafft. Eine sicherere Route zum Arbeitsplatz wird eruiert und in Zukunft gewählt.
2. **Risiko eines Plattencrashes:** Die Platte wird periodisch gesichert. Eine sicherere Platte wird auf Basis eines Vergleichstests der Plattenfehler von Produkten verschiedener Hersteller gekauft. Techniken der Mehrfachhaltung der Daten (z. B. RAID) werden verwendet. Daten werden immer auf CD/DVD oder Band gesichert.

10.3.3.4 Risiko-Annahme

Die *Risiko-Annahme* (*risk assumption*) ist die bewusste Entscheidung, die Konsequenzen eines Risikos zu tragen, falls es zu einem Problem wird. Entscheidend dabei ist, diese Risiko-Annahme bewusst durchzuführen und explizit festzuhalten. Die schriftliche Dokumentation muss neben der Entscheidung auch die Beweggründe für die Risiko-Annahme enthalten.

Risiken werden wegen der daraus resultierenden möglichen Vorteile angenommen. Solche Entscheidungen werden in jedem Projekt getroffen, entscheidend ist es die „richtigen“ Risiken auszuwählen (vgl. Abschnitt 10.3.1.1).

Beispiele

1. **Risiko eines Autounfalls:** Man vertraut auf die eigenen Fahrqualitäten und die bisherige Unfallstatistik für den Arbeitsweg und fährt weiterhin mit dem eigenen Auto.
2. **Risiko eines Plattencrashes:** Die Platte wird, da sie bis jetzt ohne Probleme funktioniert hat, wie bisher verwendet (vielleicht mit dem Versprechen, „morgen“ eine Sicherung durchzuführen).

10.3.3.5 Risiko-Übertragung

Risiko-Übertragung (*risk transfer*) bedeutet den Transfer des Risikos auf andere Verantwortungsbereiche. Es genügt allerdings nicht, rein die Risikoverantwortung zu übertragen, auch die Zuständigkeit, im Fall des eintretenden Problems eine Lösung zu finden und umzusetzen, muss übertragen werden! [Thayer und Fairley, 1997]

Beispiele

1. **Risiko eines Autounfalls:** Eine geeignete Versicherung wird abgeschlossen. Eine gute medizinische Notversorgung wird im Auto mitgeführt. Eine Rechtsschutzversicherung wird abgeschlossen, mit Hilfe derer im Schadensfall der Unfallgegner auf Schadenersatz (wegen ausfallender Arbeitsleistung) verklagt wird.
2. **Risiko eines Plattencrashes:** Man verlässt sich auf Software zur Wiederherstellung des Platteninhaltes. Alternativ wird die Datenspeicherung auf einem von der Systemadministration verwalteten (und damit verantworteten), zentralen Server durchgeführt, der bessere Technik (etwa Redundanz) aufweist. Die Wiederherstellung der Daten ist im Problemfall ebenfalls Aufgabe der Systemadministration.

10.3.3.6 Verwendung von Standardprodukten

Die *Verwendung von Standardprodukten* (*commercial-off-the-shelf products*; *COTS*) stellt ebenfalls eine Technik dar, das Gesamtrisiko einer Projektentwicklung zu reduzieren. Standardprodukte und -bausteine weisen gegenüber eigens für das Projekt entwickelten bzw. im Haus gefertigten Komponenten folgende Vorteile auf (teilweise entnommen aus [Boehm, 1998]):

- Standardprodukte sind bei Bedarf sofort verfügbar und können zu vorhersehbaren Kosten zugekauft werden.
- Ein weit verbreitetes Produkt gewährt eine reife Technologie sowie umfangreiche Unterstützung (seitens des Herstellers, aber auch durch andere Anwender, etwa in Form von Internet-Foren).

- Standardprodukte sind oft plattformunabhängig oder werden plattformübergreifend angeboten. Für Adaptionen bei Plattform-Upgrades ist der Hersteller zuständig.
- Für die Wartung eines Standardprodukts müssen keine Ressourcen eingeplant werden.

Diesen Vorteilen stehen folgende Nachteile gegenüber:

- Lizenzgebühren fallen auf einmal zu Beginn (beim Kauf des Produkts) an.
- Der Auftragnehmer (sowie oftmals auch das erstellte Produkt) ist von einem Zulieferer abhängig.
- Ein Standardprodukt nutzt das festgelegte Zielsystem nicht optimal aus.
- Trotz umfangreicher Funktionalität eines Standardsystems gibt es vielfach Beschränkungen bei für den Auftraggeber benötigter Funktionalität. Dies kann sich auch in einer auf bestimmte Weise durch das Standardprodukt vorgegebenen Gestaltung von (Benutzer-)Schnittstellen auswirken.
- Der Zeitpunkt für Wartungsarbeiten sowie Upgrades des Standardprodukts ist außerhalb der Kontrolle des Projektentwicklers.
- Viele Standardprodukte sind schwer zu skalieren und führen daher „immer“ zu einem komplexen Produkt.
- Standardprodukte verschiedener Zulieferer sind vielfach inkompatibel.

Die Entscheidung, für welche Bereiche in einer Softwareentwicklung Standardkomponenten verwendet werden und für welche nicht, ist daher immer ausreichend schriftlich zu begründen.

10.3.3.7 Techniken zur Behandlung der zehn wichtigsten Risiken bei Softwareprojekten

In [Boehm, 1998] werden neben der Auflistung der wichtigsten Risiken der Softwareentwicklung (vgl. Abbildung 10.5) auch mögliche Techniken zur Behandlung aufgelistet (Liste erweitert aus Erfahrungen des Autors):

1. Risiko personeller Defizite:

- Top-Experten einkaufen,
- Abwanderung von Schlüsselpersonal vertraglich verhindern (Prämiensystem, Konkurrenzausschlussklausel, Firmenbeteiligung),
- regelmäßige Schulungen und Weiterbildung durchführen,
- Supervision und externes Coaching anbieten;

2. Risiko unrealistischer Termin- und Kostenvorgaben sowie ungenügender Beachtung des Prozesses:

- Geschäftsfälle analysieren,
- Design kostenorientiert erstellen,
- Entwicklung inkrementell durchführen,
- Wiederverwendung fördern,
- Anforderungen reduzieren,
- Budget und/oder Zeitplan ausweiten;

3. Risiko von Defiziten bei zugekauften Komponenten:

- Qualifikationstests institutionalisieren,
- Benchmark-Tests definieren,
- Testrahmen erstellen,
- Referenzen des Zulieferers prüfen,
- Kompatibilitäten zugekaufter Komponenten analysieren (untereinander sowie mit dem erstellten Produkt);

4. Risiko falsch verstandener Produkteigenschaften:

- mit dem Auftraggeber auf Win-Win-Basis verhandeln,
- Geschäftsfälle analysieren,
- Projektziele regelmäßig überprüfen,
- zukünftige Anwender interviewen,
- Prototypen erstellen,
- erste Version der Benutzerdokumentation bereits während der Designphase liefern,
- Design und Implementierung kostenorientiert durchführen;

5. Risiko einer schlecht gestalteten Benutzerschnittstelle:

- Anwendungsfälle beschreiben (Szenarios erstellen),
- Benutzerschnittstellen-Prototypen erstellen,
- Benutzerverhalten analysieren (verwendete Funktionalität, Arbeitsstil, Arbeitsablauf, Arbeitsbelastung);

6. Risiko schlechter Architektur, Performanz, Qualität allgemein:

- Architektur-Prototypen erstellen,
- Architektur wieder verwenden, Systemrahmen verwenden,
- regelmäßig Architektur-Reviews durchführen,
- Simulation und Benchmarking einsetzen,
- Programm auf Basis instrumentierten Codes tunen,
- Qualitätssicherung institutionalisieren;

7. Risiko der Entwicklung falscher Produkteigenschaften:

- Sensibilitätsanalyse der Anforderungen durchführen,
- Hemmschwelle für Änderungswünsche bei Anforderungen seitens des Auftraggebers erhöhen (Kostentransparenz, Pönale),
- Entwicklung inkrementell durchführen,
- Geheimnisprinzip hierarchisch anwenden;

8. Risiko der Verwendung von Legacy-Software:

- Migrationsanalyse durchführen,
- Reverse Engineering anwenden (Design der Legacy-Software aufarbeiten),
- Designmuster verwenden (Wrapper, Mediatoren);

9. Risiko bei extern vergebenen Aufgaben:

- Referenzen einholen,
- regelmäßig Zwischenergebnisse reviewen,
- erfolgsbezogene Aufträge erteilen,
- Pönale vertraglich vereinbaren,
- Aufträge wettbewerblich vergeben (Pflichtenhefterstellung, Prototypen),
- Mitarbeiter des Subunternehmens ins Entwicklungsteam einbeziehen (kooptieren);

10. Risiko des Überforderns der Technologie:

- umfangreiche Machbarkeitsanalyse durchführen,
- genaue Kosten-Nutzen-Analyse durchführen,
- externe Peers einbeziehen,
- Prototyping anwenden.

10.3.4 Werkzeuge

Für die Risikobehandlung in der Softwareentwicklung gibt es noch relativ wenige Werkzeuge, deren Verbreitung nicht sehr hoch ist. Viele *Werkzeuge der Risikobehandlung* wurden aus firmeninternen Vorgehensweisen entwickelt und werden daher oftmals nicht weitergegeben. Folgende Ansätze werden verfolgt [Adameitz, 1999]:

- Projektplanungs- und -überwachungswerkzeuge werden um Komponenten zur Risikoanalyse und -überwachung erweitert. So stellt z. B. **Risk+** (<http://www.projectnet.co.uk/pm/pmt/pmtmara3.htm>) eine Erweiterung des Programms **Microsoft Project** dar.
- Eigenständige Programme erlauben das Planen alternativer Projektverläufe abhängig von potenziellen Problemen (in Form von Ereignisbaum-Analysen; z. B. **RiskMaster**; <http://www.agoron.com/~sphygmic/riskmast.htm>) oder unterstützen die quantitative Bewertung von Risiken. Eine Übersicht über solche Produkte ist im Internet verfügbar (http://www.pmi.org/pmi/meme_prod/prdalpha.htm).

- Die Verwendung von Tabellenkalkulationsprogrammen ermöglicht die rasche Erstellung von Analyse-„Werkzeugen“ zur Ermittlung des Gesamtrisikos eines Projektes, vgl. z. B. [Thomsett, 1989].
- In [Adameitz, 1999] wird betont, dass oft einfache Listen, die identifizierte Risiken aufzeichnen, einen wesentlichen Schritt zur Beherrschung von Risiken darstellen: „Niedergeschriebene Risiken sind schwieriger zu ignorieren. Manager reagieren viel eher, wenn Information in geschriebener – vielleicht sogar formal dokumentierter – Form vorliegt, als wenn sie nur mündlich vorgetragen wird. Sie sind plötzlich sehr daran interessiert, ein Risiko zu verfolgen, sobald es einmal in einem Bericht enthalten ist. Ein System der formalen Dokumentation von Risiken macht die Risiken sichtbar für alle Beteiligten an einem Projekt. Es verdeutlicht auch für alle Beteiligten die Bereitschaft jedes einzelnen, ein Risiko proaktiv zu behandeln.“ [Adameitz, 1999]

10.3.5 Ergebnisse

10.3.5.1 Top-N-Liste der Risikoverfolgung

Die *Top-N-Liste der Risikoverfolgung* ist eine Liste, in der die wichtigsten N identifizierten Risikoelemente (oft werden ca. zehn empfohlen, vgl. z. B. [Patzak und Rattay, 1997]) gemäß ihrer Wichtigkeit bezüglich der Überwachung enthalten sind. Im Wesentlichen werden die Risiken daher nach ihrem Risikofaktor geordnet sein.

Diese Liste wird im Rahmen der regelmäßigen Projektkontrolle sowie bei jeder außerplanlichen größeren Änderung der Projektrisiko-Situation (wenn z. B. ein Risiko zu einem Problem wird) aktualisiert. [Balzert, 2000b] empfiehlt dazu, besonderes Augenmerk auf in der Liste aufsteigende und neu aufgenommene Risiken zu legen, um diese möglichst rasch zu lösen, da deren Dynamik (und damit das Potenzial der Projektgefährdung) am höchsten ist.

Die Top- N -Liste enthält für jedes aufgelistete Risikoelement:

- die inhaltliche Beschreibung des Risikoelements,
- die aktuelle Risikobewertung (Risikofaktor),
- den aktuellen Rang in der Liste,
- den vorherigen Rang in der Liste,
- die Dauer des Aufscheinens in der Liste (z. B. in Wochen),
- den Fortschritt bei der Risikobehandlung.

Wenn ein Risiko zu einem Problem wird, so erfordert es normalerweise eine unmittelbare (reaktive) Behandlung und muss in die operationale Projekt(um)planung aufgenommen werden. Es scheidet somit aus der Top- N -Liste der Risikoverfolgung aus.

10.3.5.2 Risiko-Vorsorgeplan

Der *Risiko-Vorsorgeplan* (*risk contingency plan*) beschreibt die Rolle und Vorgehensweise der Risiko-Überwachung und Risiko-Überwindung in Reviews, Prozessadaptierungs-Sitzungen und ähnlichen Projektkontrollschritten. Zusätzlich ist auch die Vorgehensweise für die Inkludierung neu entstandener oder entdeckter Risiken (Prozess der Aktualisierung der Top-*N*-Liste) mit einzubeziehen.

Der Risiko-Vorsorgeplan enthält folgende, für die gesamte Risikobehandlung geltende Anweisungen:

- zuständiges Personal und deren Eingliederung in die Organisation,
- Prozess der Risiko-Bewertung (Risiko-Identifikation, Risiko-Analyse und Risiko-Priorisierung),
- Ablauf der Risiko-Überwachung im Rahmen des Projektentwicklungsprozesses (Fortschrittskontrolle, Reviews etc.),
- Vorgehensweise bei der Aktualisierung der Top-*N*-Liste,
- Vorgehensweise bei der Aktualisierung des Risiko-Vorsorgeplans selbst,
- Risikoindikatoren („Trigger“, i. e. Auslöser) aus Gesamtprojektsicht, die eine Aktualisierung der Risikoplanung (zusätzlich zu den regelmäßigen Terminen) erforderlich machen.

Der Risiko-Vorsorgeplan enthält weiters für jedes Risikoelement, das zumindest ein mittleres Risiko darstellt (aus [Thayer und Fairley, 1997], adaptiert und erweitert):

- inhaltliche Beschreibung des Risikoelements,
- gemachte Annahmen und Beschränkungen,
- Schadenswahrscheinlichkeit (Wahrscheinlichkeit, dass das Risiko zu einem Problem wird; Schätzgrundlage und Schätzmethode; Vertrauen in die Schätzung),
- potenzielles Schadensausmaß (Kosten und inhaltliche Konsequenzen; jeweils bester, schlechtester und wahrscheinlichster Fall),
- Zusammenhang mit anderen Risiken (Kopplung, verstärkende oder abschwächende Wirkung),
- vorhandene Alternativen innerhalb des derzeitigen Projektplans (jeweils mit Risiko, Kosten und Ablauf),
- empfohlene Risikobehandlungstechnik,
- notwendige Arbeitsschritte zur Risikobehandlung (mit Zeitplan, wenn umfangreicher evtl. mit vollständigem Subprojektplan),
- notwendigen Umfang der Einbeziehung (Information, Aktivitäten) des Auftraggebers,

- Auswirkungen der Risikobehandlung auf den Gesamtprojektplan,
- verwendeten Risikoindikator inklusive zu erreichendes Limit, ab dem das Risiko zum Problem wird,
- Kriterien, das Risiko als nicht länger existent einzustufen,
- Kriterien, das eingetretene Problem als erledigt zu betrachten (inklusive evtl. notwendiger Einverständniserklärung des Auftraggebers).

11 Design

Dieses Kapitel beschreibt, wie ein Analysemodell so zu einem Designmodell konkretisiert und erweitert wird, dass daraus in der Implementierung effizient Code erzeugt werden kann. Dabei wird auch auf Möglichkeiten und Arten der Wiederverwendung eingegangen. Die Beschreibung der Dokumente des Designs schließt dieses Kapitel ab, wobei der Benutzerdokumentation im Hinblick auf die Zufriedenheit des Auftraggebers und dem Pflichtenheft im Hinblick auf eine saubere Projektabwicklung besondere Beachtung geschenkt wird.

11.1 Ziel

Während die Analyse die Erwartungen des Benutzers an ein System in Bezug auf Funktionalität festlegt, legt das Design fest, WIE diese Funktionalitätsanforderungen für die Implementierung aufbereitet werden. Das Ziel des Designs ist es, ein detailliertes Modell des zu entwickelnden Systems und damit die Grundlage für die Implementierung zu erstellen.

Das Analysemodell wird im Design durch das Hinzufügen von Implementierungsbeschränkungen adaptiert (z. B. detaillierte Festlegung der Zielplattform, Gestaltung der Benutzerschnittstelle oder Festlegung der Datenhaltung). Zusätzlich werden Entscheidungen getroffen, ob bestimmte Bibliotheken oder Systemrahmen verwendet werden. Die Phase des Designs wird daher auch als *Systemgestaltung* oder (*System-*)*Entwurf* bezeichnet. In der Praxis ist die Abgrenzung des Designs zu Analyse und Implementierung oft verwischt, und gerade bei iterativ-inkrementellem Vorgehen wird zwischen diesen Phasen häufig hin und her gewechselt.

11.2 Aufgaben

Während des Designs wird ein *Designmodell* erstellt, das die *physische Sicht* (*physical view*) des zu entwickelnden Systems beschreibt. Diese Sicht stellt eher die Entwicklersicht dar, muss aber durch eine geeignete Wahl der Begriffe auch aus Anwendersicht verständlich sein.

Das Designmodell umfasst:

- **Systemmodell (system model):** Das Systemmodell beschreibt die Systemarchitektur aus Entwicklersicht. Spezielle Betonung liegt hier auf der *Verteilungssicht (deployment view)* sowie der *Nebenläufigkeitssicht (concurrency view)* des Systems.
- **Komponentenmodell (component model):** Das Komponentenmodell beschreibt die durch die Verteilung entstandenen Komponenten, die aus untereinander eng verwobenen und nach außen hin durch klare, kompakte Schnittstellen gekapselten Klassen bestehen. Man beachte, dass der Komponentenbegriff hier in einer anderen Bedeutung verwendet wird als im Bereich der komponentenorientierten Softwareentwicklung (vgl. Abschnitt 10.1.3.2).

Die *Aufgaben des Designs* lassen sich dementsprechend inhaltlich gliedern in

1. das *Systemdesign* und
2. das *Komponentendesign*.

Früher wurde Design nicht inhaltlich, sondern nach dem Detaillierungsgrad unterschieden in *Grobdesign (Primärdesign, first design, preliminary design)* und *Feindesign (Detaildesign)*. Diese Unterscheidung spiegelt jedoch die – veraltete – strikte Phasentrennung wider; moderne Ansätze gehen alle nach der *schrittweisen Verfeinerung* [Wirth, 1971] vor. Diese bedeutet eine schrittweise Konkretisierung des Designs, wobei Details so spät wie möglich festgelegt werden, was im Allgemeinen zu einem sehr sauberen Design führt. Vor allem wird dadurch die häufig notwendige Adaptierung und Erweiterung des Designs erleichtert.

Tipp: Ein gutes Design des Gesamtsystems und vor allem der Architektur ist entscheidend für den weiteren Projektverlauf. Dieser Bereich des Designs ist nämlich kaum in kleinen Schritten verbesserbar. Die vielfach gelobte Technik des *refactoring* (vgl. Abschnitt 12.1.3.4) ist nur im Komponentendesign sinnvoll anwendbar!

11.2.1 Systemdesign

Das *Systemdesign* umfasst im Wesentlichen drei Aufgabenbereiche:

1. Modellieren der Architektur,
2. Modellieren der Schnittstellen,
3. Modellieren der Datenhaltung.

Dabei sind Designprioritäten festzulegen und oft Kompromisse zu schließen. Viele dieser Entscheidungen sind strategisch (z. B. die Festlegung der Architekturschichten oder der Systemverteilung) und können später nur mit unverhältnismäßig hohem Aufwand geändert werden.

11.2.1.1 Modellierung der Architektur

Im Rahmen der Modellierung der Architektur werden im *Architekturdesign* das Strukturmodell und das Verhaltensmodell der Analyse verfeinert, adaptiert und erweitert. Dazu ist die Verwendung von evolutionären Prototypen besonders geeignet.

Erweiterung des Strukturmodells

Aus dem Strukturmodell wird die Systemarchitektur entwickelt, wobei die durch die Analyse vorgegebenen Beschränkungen sowie die festgelegten Implementierungsbeschränkungen (z. B. die gewählte (Ziel-)Hardware und Systemsoftware) beachtet werden müssen. Die Gliederung in Teilbereiche und Untersysteme geschieht nun nicht mehr problemorientiert, sondern lösungsorientiert. Auf diese Weise wird das Modell für die Implementierung vorbereitet (z. B. durch das Zusammenfassen aller Visualisierungskomponenten oder durch die Modellierung eines zentralen Managers für die Datenhaltung).

Entscheidend ist, bereits zu Beginn der Architekturmodellierung wieder zu verwendende Komponenten (bzw. ganze Untersysteme) einzuplanen und das restliche Architekturmodell darauf abzustimmen. Dies kann z. B. durch die geeignete Generalisierung neu zu entwickelnder Komponenten bzw. die Spezialisierung vorhandener Komponenten geschehen. Bei geplanter Anbindung an bestehende Systeme ist auf entsprechende Systemkompatibilität zu achten, und geeignete Schnittstellen sind einzuplanen.

Erweiterung des Verhaltensmodells

In Erweiterung des vor allem die Außenwirksamkeit beschreibenden Verhaltensmodells der Analyse fokussiert die Modellerweiterung während des Designs auf die Zerlegung des Systems in Untersysteme inkl. deren zeitliches Zusammenspiel (sequenziell ablaufende, probleminhärente parallele, parallel aktivierbare und einander ausschließende Komponenten). Dabei ist, wie auch bei der Erweiterung des Strukturmodells, auf die Umsetzung der logischen Systemgliederung in physische Untersysteme („*verteiltes System*“, insbesondere gekennzeichnet durch seine Hardwarearchitektur bzw. Netzwerkarchitektur) zu achten.

Gemäß dieser Systemverteilung erfolgt sodann die Abbildung auf entsprechende Prozesse. Neben den Spielarten der Nebenläufigkeit ist dabei auch das (Echt-)Zeitverhalten bestimmter Prozesse zu beachten.

11.2.1.2 Modellierung der Schnittstellen

Schnittstellen zwischen einzelnen Prozessen des geplanten Systems sowie zu externen Maschinen und Abläufen (zur Anbindung an bestehende Systeme) werden im Design lediglich durch deren Handhabung für Aufbau, Zugriff etc. sowie durch ihr (funktionales) Übertragungsprotokoll beschrieben. Die detaillierte Modellierung der Schnittstellen und ihres Verhaltens erfolgt in der Implementierung. Nur bei Echtzeitsystemen und allgemein in der Prozessdatenverarbeitung (*embedded systems*) ist vielfach bereits im Design ein höherer Spezifikationsaufwand zu treiben.

Eine Sonderstellung während des Designs nimmt jedoch die Festlegung der Kommunikation mit dem Benutzer ein. Aufgabe dieses *Benutzerschnittstellendesigns* ist eine vollständige Beschreibung aller Interaktionsabläufe zwischen dem System und einem Benutzer. Sowohl alle Eingaben (mögliche Eingabemasken, Befehle, Aufrufe etc.) wie auch alle Ausgaben (Ausgabemasken, Reaktionen auf Eingaben, Abfragen, Warnhinweise, Fehlermeldungen etc.) sind vollständig und in für den Auftraggeber nachvollziehbarer Form zu beschreiben.

Da speziell die dynamischen Abhängigkeiten zwischen Benutzereingaben und daraus resultierende Ausgaben schwer auf Papier (i. e. statisch) darstellbar sind, war die Modellierung von Benutzerschnittstellen eines der ersten Einsatzgebiete für Prototyping (*Benutzerschnittstellenprototyping*). Prototypen für die Benutzerschnittstelle ermöglichen auch eine frühzeitige Abklärung der Funktionalität, die das zu entwickelnde System aufweisen wird, sowie wie diese Funktionalität für den Anwender zugänglich ist. Eine erste „Schulung“ der zukünftigen Benutzer ist somit bereits möglich.

Da Softwaresysteme bislang nur sehr wenige Sinne des Menschen nutzen (und dies selten gleichzeitig), sind Benutzerschnittstellen besonders sorgfältig zu gestalten [Elzer, 1994]. Die Gestaltung guter Benutzerschnittstellen ist Aufgabe der *Ergonomie*, die sich systematisch mit den menschlichen Eigenschaften befasst, die in Betracht gezogen werden müssen, wenn ein technisches System für den Menschen vernünftig bedienbar sein soll. Für eine gute Übersicht über Softwareergonomie wird beispielsweise auf [Herczeg, 2004] verwiesen.

11.2.1.3 Modellierung der Datenhaltung

Für ein gutes Design muss frühzeitig die *Ressourcenverwaltung* organisiert werden. Bei klassischen Softwareentwicklungen, die nicht intensiv oder vorrangig mit externer Hardware zusammenarbeiten, bedeutet dies insbesondere eine präzise Definition und saubere Modellierung der *Datenhaltung*. In diesen Aufgabenbereich fallen auch Sicherung, Schutz und Archivierung der systemspezifischen Daten.

Die Datenhaltung umfasst

- die Verwaltung von *Datenstrukturen* im Hauptspeicher,
- die Verwaltung von für den Systemablauf benötigten oder erzeugten *Dateien*,
- die Anbindung von Datenbanken zur sicheren Ablage persistenter Daten sowie
- den Export und Import von Daten.

Für jeden Zugriff auf externe Daten sind die Transaktionsgrenzen zu definieren, die Sequenzen von Datenzugriffsoperationen (*Transaktionen*) zusammenfassen, die das System von einem konsistenten Zustand in den nächsten überführen. Im Fall einer Unterbrechung ist der Zustand vor Beginn der nicht vollständig ausgeführten Transaktion wieder herzustellen (*Rollback*).

Gerade wegen der derzeit primär vorkommenden Kopplung objektorientierter Systeme mit relationalen Datenbanken sei darauf hingewiesen, dass die für eine methodisch saubere Datenmodellierung gewünschte *Normalisierung* des relationalen Datenmodells, die auf Grund der Reduktion von Datenredundanz sowie der höheren Konsistenzsicherheit von Modifikationen

sehr erstrebenswert erscheint, aus objektorientierter Sicht nicht zu weit getrieben werden darf. Saubere objektorientierte Designs sind normalerweise weit entfernt davon, normalisiert zu sein. Eine unter der objektorientierten Modellschicht liegende, künstlich eingeführte Normalisierung führt vielfach zu einer großen Anzahl von Tabellen und zu beträchtlichen Performanzverlusten im Laufzeitverhalten [Coad und Yourdon, 1990].

Für weiterführende Informationen zum Bereich der Datenmodellierung und Datenhaltung wird auf die Literatur (z. B. [Date, 2003]) verwiesen.

11.2.2 Komponentendesign

Das *Komponentendesign* umfasst für jede Komponente folgende zwei Aufgaben:

1. Modellieren der Struktur einer Komponente,
2. Modellieren des Verhaltens einer Komponente.

Wiederum sind für Designentscheidungen Prioritäten festzulegen und bei Konflikten Designkompromisse zu schließen.

11.2.2.1 Modellierung der Komponentenstruktur

Zur Abgrenzung des Umfelds für jede Komponente werden zu Beginn die Abhängigkeiten und Schnittstellen zwischen den Komponenten festgelegt sowie der Kontrollfluss zwischen den Komponenten spezifiziert.

Bevor nun eine Komponente modelliert wird (durch Adaptieren, Erweitern und Verfeinern des Analysemodells), sind die (wieder) zu verwendenden Komponenten (Bibliotheken, Rahmen, Muster, ...) festzulegen bzw. auszuwählen. Anschließend wird die Definition der Komponente (Module, Klassen) durch Ergänzung von Attributen, Operationen und Schnittstellen vervollständigt. Dabei müssen Strukturen umgesetzt werden, die nicht direkt implementierbar sind (z. B. Mehrfachvererbung in Einfachvererbung, objektorientiertes Design eines Datenbankanschlusses in prozedurales Design).

Iterativ wird dabei parallel das Strukturmodell adaptiert. Gemeinsamkeiten von Komponenten werden durch Generalisierung dokumentiert und gegebenenfalls abstrakte Komponenten eingefügt. Abhängigkeiten von Komponenten werden durch Assoziation oder Aggregation realisiert. Das Design wird mit den Wiederverwendungs-Komponenten (z. B. Klassenbibliotheken) abgestimmt sowie in für die Implementierungsumgebung (und Implementierungssprache) adäquate Teilbereiche zerlegt.

Eine Komponente des Designs kann während der Implementierung weiter zerlegt werden und durchaus als eine Gruppe von Komponenten implementiert werden. Die Designkomponente ist daher als Abstraktion von Implementierungskomponenten zu betrachten und kann als Blackbox der Implementierung gesehen werden, die durch ihr Ein- und Ausgabeverhalten beschrieben wird.

11.2.2.2 Modellierung des Komponentenverhaltens

Das Verhalten einer Komponente wird durch die von ihr zur Verfügung gestellten Methoden bestimmt. Im Allgemeinen wird erst in der Implementierung festgelegt, wie der Ablauf dieser Methoden aussieht. Im Design wird lediglich eine *deklarative Beschreibung* erstellt, welche die Ein- und Ausgabewerte sowie Bedingungen, die gelten müssen, umfasst, also was die Methode benötigt und bewirkt und nicht wie sie es bewerkstelligt. Dies geschieht mittels einer mehr oder weniger formalen textuellen Beschreibung, die in [Meyer, 1997] als „Vertrag“ zwischen der Methode und ihren Verwendern bezeichnet wird. Zur sauberen Strukturierung dieser deklarativen Beschreibung wird an selber Stelle eine Gliederung mittels Schlüsselwörtern vorgeschlagen:

- „**uses**“ bezeichnet alle Eingabeparameter,
- „**changes**“ bezeichnet alle Ausgabeparameter und Übergabeparameter,
- „**informs**“ bezeichnet die erzeugten Nachrichten (inkl. deren Empfänger),
- „**invariant**“ bezeichnet Invarianten, i. e. Bedingungen, die während der gesamten Methodenausführung gelten müssen¹,
- „**requires**“ bezeichnet Vorbedingungen (preconditions), i. e. Bedingungen, die vor dem Aufruf der Methode gelten müssen,
- „**ensures**“ bezeichnet Nachbedingungen (postconditions), i. e. Bedingungen, die nach dem Ablauf der Methode gelten,
- „**produces**“ bezeichnet erzeugte Ergebnisse der Methode („Rückgabewerte“).

Nur wenn ein Algorithmus eine wesentliche Herausforderung für ein Projekt darstellt oder in Bezug auf Performanzbeschränkungen (z. B. Zeit, Speicher) kritisch ist (und nicht verfügbar ist oder zugekauft werden soll), wird bereits im Design eine *funktionale Beschreibung* seines Ablaufs zusammen mit der mindestens erforderlichen Performanz erstellt. So weit wie möglich sind dabei jedoch Detailbeschreibungen der Implementierungsphase vorzubehalten.

Die *Schnittstellenbeschreibung* umfasst neben dem Methodennamen Ein- und Ausgabewerte, Seiteneffekte (globale Werte, Dateien etc.) sowie das Fehlerverhalten. Zur exakten Festlegung von Schnittstellen einer Komponente können im Komponentenmodell explizit *Schnittstellen* für Klassen definiert werden (in der UML mit Hilfe der so genannten *Lollipop-Notation*), die das gewünschte Verhalten einer Komponente durch die Zusammenstellung ausgewählter Methoden beschreiben.

11.3 Techniken

11.3.1 Erweitern der Analysemodelle

Im Design werden das Analysemodell und die zugehörigen Dokumente verfeinert, adaptiert und erweitert. Während bei traditionellem Vorgehen eine „vollständige“ Erstellung des De-

¹Invarianten sind z. B. in der Programmiersprache „Eiffel“ realisiert, vgl. [Meyer, 1997].

signs das Ziel ist, wird bei agilem Vorgehen Wert darauf gelegt, das Design so einfach wie möglich zu halten. Das Design soll nur die Bereiche umfassen, die von der aktuellen Testsuite abgedeckt sind. Im Gegensatz zu ursprünglich sehr extremen Ansätzen, das Design rein im Code ausdrücken zu wollen (vgl. [Beck und Andres, 2005]), wird nun zunehmend die Wichtigkeit eines guten Design-Grundgerüsts (*Big-Picture-Design*) erkannt, das dann sukzessive angepasst und erweitert wird, vgl. [Stephens und Rosenberg, 2003].

Während das Analysemodell im *Problemraum* (Sichtweise des Auftraggebers) modelliert ist, wird es im Design in den zugehörigen *Lösungsraum* (Realisierungssicht des Auftragnehmers) transformiert. Folgende Schritte sind dazu zu setzen:

- Die Beschreibung des Istzustands wird gemäß der erweiterten Einsicht in das Problem adaptiert.
- Der Sollzustand wird mit Rücksicht auf die Machbarkeit gemäß den Projektbeschränkungen genauer definiert bzw. aktualisiert.
- Die Schnittstellen werden detailliert beschrieben.
- Die Systemanforderungen werden verfeinert und erweitert (v. a. um verwendete Systemsoftware und Basisbibliotheken sowie für die Datenhaltung notwendige Komponenten).
- Im Analysemodell werden die Komponenten, Attribute und Operationen des Modells dem Designumfeld und der gewählten Realisierung entsprechend erweitert (vgl. beispielsweise [Oestereich, 2004]).
- Der Umfang und der Detaillierungsgrad der Dokumentation wird genau festgelegt. Begleitende Maßnahmen für Inbetriebnahme und Einsatz (Vorgehensschritte, technische Unterstützung, Schulung etc.) werden dokumentiert.
- Der Zeitplan wird überarbeitet und verfeinert.
- Unklarheiten bei den Projektdurchführungsbestimmungen werden beseitigt.

Bei iterativ-inkrementellem Vorgehen erfolgen diese Schritte in jeder Iteration.

Diese Vervollständigungen werden im *Pflichtenheft* festgehalten (vgl. Abschnitt 11.5.2). Dieses Dokument ist vom Auftraggeber genau zu prüfen und abzunehmen.² Meistens ist diese Prüfung seitens des Auftraggebers die letzte umfassende Überprüfung, ob das Projekt zielgerichtet voranschreitet, bevor der Auftraggeber den ersten Produktprototyp bzw. die erste Version des Produkts erhält. Daher ist ein genaues und überdachtes Arbeiten des Auftragnehmers gerade in der Designphase von entscheidender Bedeutung für den Erfolg eines Projekts.

²Bei agilem Vorgehen ist die Erstellung eines Pflichtenhefts oft optional. Sie wird als Kundenanforderung gesehen und zeitlich eingeplant. Für den Fall, dass kein Pflichtenheft erstellt wird, ist dafür Sorge zu tragen, mit dem Auftraggeber geeignete vertragliche Vereinbarungen zu treffen (z. B. via Angebot/Auftrag oder beiderseits unterfertigte Besprechungsprotokolle).

Generalisierung und Promotion von Wissen

Während für den Auftraggeber die verschiedenen Einsatzsituationen des Produkts entscheidend sind, ist für den Auftragnehmer das Erkennen von Ähnlichkeiten bei Komponenten und Vorgängen schon allein wegen möglicher Entwicklungseinsparungen eine wesentliche Aufgabe. Gerade diese Technik zur Meisterung der Komplexität und Rationalisierung der Produktentwicklung wird im Softwarebereich durch objektorientierte Ideen sehr stark gefördert:

- Die *Generalisierung* und *Spezialisierung* von Komponenten führt zur Erstellung von hierarchischen Klassifikationsstrukturen (*Taxonomien*). Manchmal reicht ein generalisierter Begriff zur Lösung der Aufgaben und das Produkt (bzw. Teile davon) kann allgemeiner eingesetzt werden als ursprünglich geplant. Durch das Erweitern der Taxonomie können verwandte Spezialisierungen gefunden werden, die manchmal Lücken in der Problembeschreibung aufdecken. Mittels Generalisierungen können Modelle und Benutzerschnittstellen in Bereichen, deren Ähnlichkeit dem Auftraggeber vielleicht ursprünglich gar nicht bewusst war, vereinheitlicht werden. Durch Analyse neu gefundener Oberbegriffe können Gemeinsamkeiten von Komponenten gefunden und so die Problembeschreibung vereinheitlicht und vereinfacht werden.
- Wenn bestimmte Eigenschaften und/oder Operationen für alle Spezialisierungen eines Oberbegriffs gelten, so können sie einheitlich im gemeinsamen Oberbegriff festgelegt werden. Dieses Wissen wird in der Taxonomie „promoviert“. Auf diese Weise werden Oberbegriffe mächtiger, und Spezialisierungen lassen sich mit weniger Aufwand erstellen. Durch die Zusammenfassung gleicher Operationsabläufe wird ein einheitliches Verhalten erzielt, und es reduziert sich der Implementierungs- und Testaufwand.

Topdown-Design versus Bottomup-Design

Topdown-Design bedeutet das Vorgehen vom Abstrakten zum Konkreten, d. h., das Modellieren durch sukzessives Zerlegen von großen Komponenten in mehrere kleine. Dies geschieht je nach Designparadigma bei der funktionalen Zerlegung durch Zerlegen des Kontrollflusses, bei der strukturierten Analyse durch Zerlegen der Abhängigkeiten von Eingabe und Ausgabe, beim objektorientierten Vorgehen durch Entflechten von Nachrichtenkommunikation sowie bei verteilter Entwicklung durch Delegation von Verantwortlichkeiten.

Bottomup-Design bedeutet das Vorgehen vom Konkreten zum Abstrakten, d. h., das (rekursive) Zusammenfassen kleiner Komponenten zu größeren bis zur Modellierung des gesamten Problems, günstigenfalls durch Verwenden vorgefertigter Bausteine aus Bibliotheken, Rahmen, Muster o. Ä. In der Praxis werden Topdown-Design und Bottomup-Design fast immer gemischt eingesetzt, je nach Eignung wird Konkretisierung oder Abstraktion angewendet. Je nach Verfügbarkeit wieder verwendbarer Bausteine überwiegt derzeit im Softwaredesign meistens der Bottomup-Anteil.

11.3.2 Erstellen von Prototypen

Wie in der Analyse (vgl. Kapitel 10) führt *Prototyping* auch im Design zu besserem Verständnis zwischen Auftraggeber und Auftragnehmer. Im Design wird vor allem *experimentelles Prototyping* und *evolutionäres Prototyping* eingesetzt. Die Haupteinsatzgebiete sind

- *Architekturdesign* und
- *Benutzerschnittstellendesign*.

Während *Architekturprototypen* erstellt werden, um die Realisierbarkeit oder die Performanz eines bestimmten Designs abzutesten, sind *Benutzerschnittstellenprototypen* für die Kommunikation mit dem Auftraggeber von großer Bedeutung. Sie ermöglichen eine erste Präsentation des Aussehens des Produkts inklusive der Bedienung einzelner Bereiche. Durch Rückmeldungen des Auftraggebers kann die Ergonomie verbessert und die Bedienung an Kundenwünsche angepasst und somit die spätere Akzeptanz des Produkts in Bezug auf die Benutzerinteraktion gesichert werden. Durch das Festlegen und Benennen der Komponenten der Benutzerschnittstelle (Eingabe und Ausgabe) gemeinsam mit dem Auftraggeber werden das Produkt problemadäquat gestaltet und das Know-how sowie die speziellen Wünsche seitens der späteren Anwender des Produkts korrekt im Design abgebildet.

Gerade das *evolutionäre Prototyping* muss während des Designs (und später während der Implementierung) zeitlich und inhaltlich laufend überwacht werden, um nicht anstelle eines Produkts einen „ewigen Prototypen“ zu erstellen. [Coad und Yourdon, 1990] schlagen dazu vor, die Anzahl der Iterationszyklen für jeden Prototyp auf maximal zwei bis drei zu beschränken.

Eine Dokumentation in Form eines „Papierprototypen“ soll erstellt und immer aktuell gehalten werden, in dem festgehalten wird, welche Teile des Prototypen auch im Endprodukt in der gleichen Form enthalten sein werden und wo welche Kompromisse und Abstriche gemacht wurden. Gleichzeitig wird mittels regelmäßiger Meilensteine eine Form von „Rapid Prototyping“ [Coad und Yourdon, 1990] angeregt, bei der in regelmäßigen Zeitabständen der Prototyp mit dem Auftraggeber durchdiskutiert wird, um Notwendigkeiten von Wünschen zu trennen, schleichende Funktionalitätserweiterungen aufzuzeigen, das Projektteam durch die Präsentation der erzielten Ergebnisse zu motivieren sowie den Auftraggeber vor einem „Implementierungsschock“ (durch die erstmalige Verfügbarkeit des Produkts bei der Auslieferung der Endversion) zu bewahren. Diese Vorgehensweise bildet die Grundlage aller „agilen“ iterativ-inkrementellen Vorgehensmethoden.

11.3.3 Modellgesteuertes Entwickeln

Modellgesteuertes Entwickeln (*model-driven development – MDD*) basiert auf dem Gedanken, dass ein Modell eines Systems so erstellt werden kann, dass daraus das reale System durch reine Transformation erzeugt werden kann [Mellor *et al.*, 2003]. Die Grundlage zum modellgesteuerten Entwickeln wurde mit der Standardisierung der UML 2 [Oestereich, 2004] erarbeitet, die als Teil der Norm die so genannte *modellgesteuerte Architektur* (*model-driven architecture – MDA*; <http://www.omg.org/mda/index.htm>) enthält. Die Vorgehensweise beim modellgesteuerten Entwickeln mit MDA ist folgende [Versteegen, 2003]:

1. In einem *plattformunabhängigen Modell* (*platform-independent model – PIM*) werden Anforderungen, Modelle und Prozesse erfasst.
2. Die Rahmenbedingungen werden mittels der *meta object facility* (*MOF*) definiert, einem Bestandteil der UML 2-Norm zur Beschreibung von Metadaten und Modellen sowie deren Transformation.

3. Aus PIM und MOF wird ein *plattformspezifisches Modell* (*platform-specific model – PSM*) für eine bestimmte Implementierungsplattform erstellt.
4. Auf Basis dieses detaillierten Modells wird automatisch ein ausführbarer Code generiert. Dies ist möglich, da die UML in der Version 2 eine vollständige („computationally complete“) Programmiersprache darstellt [Mellor *et al.*, 2003].

Diese Vorgehensweise hat folgende Vorteile:

- Die Modellierung erfolgt plattformunabhängig. MDA trennt Modell und Implementierungsdetails, sodass anwendungsorientierte Modelle über verschiedene Implementierungen und Plattformen hinweg (wieder-)verwendet werden können.
- Die Anwendung kann vollständig auf Modellbasis simuliert werden. So ist z. B. *Tau Generation 2* (<http://www.telelogic.com>) das erste Werkzeug, das nicht nur UML 2-Modelle generiert und (statisch) analysieren lässt. Es simuliert die gesamte Anwendung automatisch auf Basis der UML 2-Beschreibung³ [Versteegen, 2003].
- Das Verteilen auf verschiedene Implementierungen und Plattformen (*deployment*) kann ebenfalls in MOF formuliert und damit automatisiert werden. Entscheidend ist, dass dabei automatisch vollständiger, lauffähiger Code erzeugt wird [Ambler, 2003].

Das Konzept von MDD/MDA ist noch zu jung, um seine Erfolgsaussichten realistisch beurteilen zu können. Die Gefahr ist groß, analog zu den Prozessmonstern der neunziger Jahre wieder einen Moloch zu entwickeln, der (theoretisch) alles kann und von niemandem verwendet wird. Bereits jetzt fühlen sich jedoch die nach „minimalem Design“ strebenden Verfechter der agilen Vorgehensmethoden angegriffen, vgl. [Ambler, 2003]. Außerdem ist unklar, wie z. B. Benutzerschnittstelle und Datenbankanschluss, ohne die kaum ein modernes Programm auskommt, vernünftig in der UML formuliert werden können. Jedenfalls stellt die modellgesteuerte Entwicklung die bereits länger erwartete Gegenbewegung zur agilen Allianz dar und rückt die Bedeutung eines guten (und gut überdachten) Designs, das sicher in den agilen Vorgehensmethoden zu kurz gekommen ist, wieder in den Mittelpunkt (vgl. Abschnitt 6.3.1 und insbesondere Abbildung 6.2).

11.3.4 Anwenderseitiges Entwickeln

Das *anwenderseitige Entwickeln* (*enduser computing*) bezieht den Anwender direkt in das Design ein. Dies geschieht entweder dadurch, dass Softwareexperten des Auftraggebers in die Entwicklergruppe des Auftragnehmers integriert werden, oder aber dadurch, dass dem Auftraggeber nur ein Werkzeug geliefert wird, mit dem er seine Anwendung (durch Konfigurieren, Maßschneiden und Befüllen mit Daten und Abläufen) selbst „erstellt“. Gerade der Durchbruch des Konzepts der Systemrahmen (application frameworks) macht die zweite Alternative zu einem immer häufiger eingeschlagenen Lösungsweg (siehe [Artmann, 1997] für ein Anwendungsbeispiel).

³In [Selic, 2003] wird dieser Schritt in seiner Bedeutung verglichen mit dem Schritt der funktionalen Abstraktion von Assemblersprachen zu prozeduralen Programmiersprachen wie Pascal oder C.

In [Pomberger und Blaschek, 1996] wird festgestellt, dass die Integration von Anwendern am ehesten Sinn macht beim Prototyping von Benutzerschnittstellen. Bei der Modellierung der Architektur trägt auf jeden Fall der Entwickler die alleinige Verantwortung für das Ergebnis. Der Anwender kann nur dadurch sinnvoll eingebunden werden, dass er Anwendungsfälle zur Verfügung stellt und mit diesen das Design durchspielt. Generell wird anwenderseitiges Entwickeln eher abgelehnt, da es in der Regel zu Insellösungen führt, alternative bzw. innovative Lösungen verhindert, meist zu redundanten Systemen führt und Ergonomie, Effizienz, Integrität u. dgl. von Software großes Spezialwissen erfordern, das beim Anwender meist nicht vorhanden ist.

11.3.5 Wiederverwendung

Wiederverwendung (reuse) bedeutet Entwicklung durch (bzw. nach) Investition („development by investment“). Ein großer Anteil wieder verwendeter Komponenten bei einem Produkt hat im Allgemeinen einen reduzierten Entwicklungsaufwand zur Folge. Trotzdem führt dies zu höherer Produktqualität (niedrige Fehlerrate, hohe Performanz, ...) unter der Voraussetzung, dass die wieder verwendeten Komponenten selbst von hoher Qualität sind. Um bei der Implementierung vorhandene Komponenten effizient wieder verwenden zu können, muss bereits das Design auf Wiederverwendung hin ausgerichtet sein.

Wiederverwendung ist bei jedem Vorgehen möglich und sinnvoll, wird aber gerade beim objektorientierten Vorgehen besonders gefördert und unterstützt. Gerade beim objektorientierten Design wird versucht, möglichst viele bestehende Komponenten unverändert oder adaptiert wieder zu verwenden (Idee der *Bausteinbibliothek*) bzw. das Design auf einem vorhandenen Rahmen aufzubauen (Idee des *Systemrahmens*).

Die Voraussetzung für die Wiederverwendung von Komponenten ist die *Wiederverwendbarkeit (reusability)* dieser Komponenten! Es ist also bereits beim Design neuer Komponenten darauf zu achten, inwieweit diese möglicherweise in Zukunft wieder verwendet werden könnten. Gerade Schnittstellen und Verallgemeinerungen (taxonomisch übergeordnete Komponenten) sind dementsprechend zu gestalten. Die Prüfung des Designs auf Wiederverwendbarkeit durch externe Experten bringt hierzu vielfach entscheidende Anregungen. Wesentlich ist die explizite Dokumentation möglicher Wiederverwendbarkeit (mögliche Anwendungen, notwendige Adaptierungen und Erweiterungen) für jede einzelne Komponente.

Folgende Arten der Wiederverwendung werden eingesetzt:

- Wiederverwenden von Bausteinen,
- Verwenden von Mustern,
- Abwandeln von Schablonen und
- Konkretisieren von Systemrahmen.

Diese Möglichkeiten werden nachfolgend näher erläutert.

11.3.5.1 Wiederverwenden von Bausteinen

Die *Wiederverwendung von Bausteinen* basiert auf dem Prinzip des unveränderten Verwendens vorhandener Komponenten. Folgende Bausteinsammlungen werden unterschieden:

- **Toolkits** waren die historisch ersten explizit für Wiederverwendung zusammengestellten Komponenten. Ein Toolkit ist eine Sammlung von Funktionen für einen bestimmten Zweck. Sie sind offen zugänglich, benutzen vielfach globale Variablen und erlauben keinerlei Adaptierung (außer durch Änderung des Quellcodes).
- **Modulbibliotheken** (*libraries*) trennen Schnittstelle und Implementierung. Sie sind dadurch sicherer und erlauben die Festlegung der Sichtbarkeit einzelner Funktionen.
- **Klassenbibliotheken** (*class libraries*) erlauben das Erben der Fähigkeiten der Bibliothek und gleichzeitig die Adaptierung auf eigene Bedürfnisse durch die Erweiterung via Subklassen. Zusätzlich erfüllen sie das „Open/Close-Prinzip“ [Meyer, 1997]: Sie sind geschlossen gegenüber existierenden Anwendungen, jedoch offen für Änderungen und Erweiterungen. Unterschieden werden vielfach
 - *Kernbibliotheken* (*core libraries*), die oft zur Spracherweiterung dienen und nur wenige, elementare Klassen zur Verfügung stellen,
 - *Universalbibliotheken* (*general purpose libraries*), die für viele allgemeine Probleme aus den unterschiedlichsten Anwendungsbereichen Klassen zur Verfügung stellen (und daher vielfach einen sehr hohen Einarbeitungsaufwand bedingen),
 - *Fachbibliotheken* (*application libraries*), die spezifisch für bestimmte Anwendungsgebiete oder Teilbereiche von Softwaresystemen gestaltet werden, wie z. B. Benutzerschnittstellenbibliotheken (*user interface libraries*).

Da viele Programmierer einerseits ihre Ergebnisse gerne anderen gegenüber abschotten, andererseits Skepsis gegenüber der Verwendung fertigen Codes aufweisen, wird vielfach angemerkt, ein „Prämiensystem“ sowohl für die Erstellung von Klassen mit hoher Wiederverwendbarkeit als auch für die Verwendung verfügbarer Klassen einzuführen (vgl. z. B. [Pomberger und Blaschek, 1996]). Diese Beeinflussung der Codeerstellung kann aber auch zu unangebrachter Verwendung von Klassen und damit zur unnötigen Vergrößerung der Programmkomplexität führen, wie z. B. in [Goldberg und Rubin, 1995] angeführt ist.

11.3.5.2 Verwenden von Mustern

Designmuster (*design patterns*; [Gamma et al., 1997]) bieten eine schematische Lösung für ein häufig wiederkehrendes Problem. Sie stellen eine Möglichkeit zur indirekten Wiederverwendung von Wissen dar, da durch das Muster ein Design nur vorgeschlagen wird, das beliebig maßgeschneidert werden kann. Neben produktbezogenen Mustern werden teilweise auch Vorgehensmuster vorgeschlagen, um die Wiederverwendung von Prozesserfahrungen zu ermöglichen (*Prozessmuster*, *process pattern*; vgl. [Ambler, 1999a], [Ambler, 1999b]).

Analysemuster (*analysis patterns*; [Fowler, 1997]) stellen Schemata für die Anforderungsanalyse zur Verfügung. Neben dem Problem der oft sehr unterschiedlichen Anwendungsgebiete stellt hierbei auch die vielfach mangelnde Bereitschaft zur Wissensweitergabe eine Hürde

dar, da Analysemuster Domänenwissen darstellen, das oft ungern externen Personen zur Verfügung gestellt wird.

11.3.5.3 Abwandeln von Schablonen

Schablonen werden in der modernen Softwareentwicklung mittels abstrakter Klassen oder mittels Schnittstellenklassen zur Verfügung gestellt.

Abstrakte Klassen können nur durch Abwandlung mittels einer Subklasse realisiert werden. Dabei weisen *reine abstrakte Klassen* nur die Schnittstellenbeschreibungen der Methoden auf und dienen so zur Standardisierung des Protokolls der Klassenverwendung. Viele abstrakte Klassen weisen jedoch eine Teilimplementierung der Methoden auf, die das Standardverhalten der Klasse beschreiben, das von den Subklassen überschrieben werden kann.

Schnittstellenklassen können im Unterschied zu abstrakten Klassen durch beliebige Klassen (und nicht nur durch Subklassen) realisiert werden. Klassen, die eine bestimmte Schnittstelle realisieren (*supplier classes*), können darüber hinaus zusätzliche Methoden aufweisen; Klassen, die eine Schnittstelle benutzen (*clients*), müssen nicht alle zur Verfügung gestellten Methoden tatsächlich benutzen.

Wieder verwendbare Zusammenstellungen von Klassen, die untereinander stark gekoppelt sind, nach außen hin aber eine eng begrenzte, wohl definierte Schnittstelle aufweisen, werden als *Komponente* bezeichnet. Ihre Schnittstelle wird unterteilt in eine anbietende Schnittstelle und eine minimal geforderte Schnittstelle. Komponenten werden in *Komponentenbibliotheken* (*component libraries*) zusammengestellt (z. B. *Java Beans*; [Chan und Lee, 1998]). Aktive Komponenten besitzen eine eigene Intelligenz und können selbsttätig Aufgaben verrichten. Sie werden als (*intelligente*) *Agenten* bezeichnet (vgl. [Zhong *et al.*, 2002]).

11.3.5.4 Konkretisieren von Systemrahmen

Das größte Problem vieler Klassenbibliotheken ist, dass sie keine Gesamtstruktur einer Anwendung definieren und damit beispielsweise die Konsistenz von Benutzerschnittstellen nur auf der Ebene einzelner Bausteine garantieren können und nicht auf der Anwendungsebene.

Ein *Systemrahmen* (*application framework*) ist eine Sammlung miteinander verwobener Objekte, welche die Grundfunktionalität einer Anwendung bietet, aber einfach zu spezifischen Anwendungen spezialisiert werden kann [Schmucker, 1986]. Ein Systemrahmen stellt also eine besondere Form einer Klassenbibliothek dar, die zusätzlich zu den Bausteinen auch das Grundgerüst einer vollständigen Anwendung enthält. Diese so genannte Outside-in-Konstruktion ermöglicht es, das vorgegebene Rahmenprogramm mit objektorientierten Methoden zu einer sinnvollen fachspezifischen Anwendung zu erweitern (Konzept der *generischen Anwendung*).

Durch die Verlagerung des Kontrollflusses der Anwendung in die Bibliothek wird auch die Systemarchitektur standardisiert und ermöglicht so eine bessere Wiederverwendung des Systemdesigns (*design reuse*). Durch Einschubmethoden (*hooks*) können anwendungsspezifische Methoden in den Systemrahmen eingeschoben werden, die dann vom Systemrahmen aufgerufen werden (so genanntes „Hollywood-Prinzip“: „Don’t call us, we’ll call you!“).

Der Vorteil von Systemrahmen ist, dass neben dem Komponentendesign auch das Systemdesign wieder verwendet wird und damit die Standardisierung der Systemarchitektur vorangetrieben wird. Demgegenüber stehen als Nachteile, dass Systemrahmen einen hohen Einarbeitungsaufwand bedeuten und auf Grund ihres spezifischen Anwendungsgebiets oft zu starr sind, wodurch Hilfslösungen (workarounds) notwendig werden.

Für spezielle Anwendungsbereiche werden spezialisierte Systemrahmen angeboten (z. B. data management frameworks, mechanical engineering frameworks).

11.3.6 Empfehlungen für ein gutes Design

Für eine umfassende Zusammenstellung von Designempfehlungen, sowohl was den Designvorgang betrifft als auch was ein gutes Designergebnis charakterisiert, wird auf die umfangreiche Literatur verwiesen (z. B. [Balzert, 2000a], [Buschmann, 1998] oder [Gamma *et al.*, 1997]). Nachfolgend werden nur einige Grundregeln angegeben, die für ein gutes Designergebnis essenziell sind:

- Entwickle modulare Systeme. Ein Modul ist eine Sammlung von Komponenten (enthalten Daten und/oder Algorithmen)⁴, das mit der Außenwelt nur über eine klar definierte Schnittstelle kommuniziert. Damit setzt die Benutzung eines Moduls keine Kenntnisse seines inneren Aufbaus voraus und die Implementierung eines Moduls keine oder nur beschränkte Kenntnisse über seine spätere Benutzung.
- Befolge die Regeln „Eine Klasse, eine Abstraktion!“ sowie „Eine Methode, eine Funktionalität!“.
- Befolge das Geheimnisprinzip (*information hiding*; [Parnas, 1972]), nach dem Daten nur denjenigen Methoden bekannt sein dürfen, die sie zu ihrer Implementierung brauchen. Erlaube insbesondere von außen keinen direkten Zugriff auf Objektvariablen. Dies bietet Sicherheit gegen unbeabsichtigte Eingriffe und verbessert die Abstraktion durch das Verbergen von Details.
- Begrenze die Tiefe von Vererbungsbäumen. Als Wurzel und, wo immer sinnvoll, für innere Knoten verwende abstrakte Klassen. Abstrakte Klassen dienen zur Definition oder zum Herausheben von Gemeinsamkeiten und besitzen abstrakte Methoden, die entweder gar nicht oder mit einem vorgegebenen Verhalten implementiert werden, das bei Bedarf überschrieben werden kann. Stelle sicher, dass dieselbe Baumtiefe denselben Abstraktionsgrad repräsentiert.
- Betreibe Wiederverwendung durch Komposition (Objekte übernehmen „Rollen“) an Stelle von Wiederverwendung durch Vererbung. Nur wenn die vollständige Schnittstelle der Superklasse auch für die Subklasse Sinn macht, ist eine Wiederverwendung durch Vererbung sinnvoll.
- Verwende Designmuster und Systemrahmen, um durch Übernahme von bewährten Vorlagen eine gute System- und Komponentenstruktur sicherzustellen.

⁴Ein *Modul* kann von einer *Datenkapsel* abgegrenzt werden dadurch, dass Module auch nur Methoden enthalten können, eine Datenkapsel (Sammlung von abstrakten Datenstrukturen bzw. abstrakten Datentypen) jedoch immer Daten enthält.

- Halte die Anzahl der Objektinteraktionen gering. Versuche Cluster zu bilden, innerhalb derer eine enge Interaktionsverwebung besteht, die aber untereinander nur lose gekoppelt sind (Idee der *Softwarekomponenten*; vgl. z. B. [Szyperski, 1998]).

11.4 Werkzeuge

11.4.1 Computergestützte Designwerkzeuge

Viele der in Abschnitt 10.1.4.1 vorgestellten Analysewerkzeuge ermöglichen auch computerunterstütztes Design, indem sie die Erstellung des Systemmodells und des Komponentenmodells unterstützen. Die Unterstützung durch diese Werkzeuge muss gerade in der Designphase auf die beim Auftragnehmer vorherrschenden Designmethoden (funktionsorientiert, datenorientiert, objektorientiert etc.) und Vorgaben durch verwendete Hardware und Software sowie die vom Auftraggeber gewünschten Notationen und Dokumente abgestimmt werden können. Konfigurierbare und adaptierbare Werkzeuge (so genannte *Meta-CASE-Systeme*) sind daher starren Systemen auf jeden Fall vorzuziehen.

Folgende Kriterien können für die Auswahl eines CASE-Systems herangezogen werden:

- Die Designmodelle sollen grafisch-interaktiv erstellt und geändert werden können.
- Die verwendeten Darstellungen sollen dem „Prinzip der geringsten Überraschung“ folgen. Der Leser soll die Bedeutung der Darstellungen aus dem Kontext herleiten können, wobei jeder Darstellung die nahe liegendste Bedeutung zugewiesen ist.
- Es soll möglich sein, den Zwischenstand des Designs auszudrucken und diese Darstellungen einfach manuell zu bearbeiten, um z. B. Ideen auch ohne Computer durchzudiskutieren. Dazu müssen die Darstellungen leicht mit der Hand erstellbar sein.
- Alle in der gewählten Methode notwendigen Designschritte sollen mit dem Werkzeug computerunterstützt durchgeführt werden können. Alle Ergebnisdokumente sollen mit dem Werkzeug in adäquater Form und Qualität erstellt werden können.
- Das Werkzeug soll die Eingaben auf Konsistenz mit dem Modell prüfen können.
- Das Werkzeug soll für die Größe des zu erstellenden Produkts sowie die Anzahl der Gruppenmitglieder (Mehrbenutzerbetrieb!) geeignet sein.
- Das Werkzeug soll über eine Versionsverwaltung verfügen.

Um die Designergebnisse effizient in der Implementierungsphase weiterverwenden zu können, ist darauf zu achten, dass aus dem Designmodell Codeblöcke und Rahmen für die eingesetzte Entwicklungsumgebung in der gewählten Programmiersprache erstellt werden können (*Forward Engineering*). Um das Designmodell auch während der Implementierung auf dem aktuellen Stand halten zu können, ist es von Vorteil, wenn aus vorhandenem Code das Design extrahiert werden kann (*Reverse Engineering*⁵). *Offene CASE-Systeme* erlauben die

⁵Werkzeuge, die das kombinierte Forward Engineering und Reverse Engineering beherrschen, werden als *Roundtrip Engineering*-Werkzeuge bezeichnet.

Integration von Werkzeugen in die Entwicklungsumgebung mit darauf abgestimmter Benutzeroberfläche.

11.4.2 Diagramme der UML für das Design

Im Design werden die in der Zielerforschung und der Analyse erstellten Diagramme adaptiert, erweitert bzw. verfeinert. Zusätzlich stellt die UML Diagrammarten zur Verfügung, die vorrangig für die Designmodelle Verwendung finden (Abbildung 11.1):

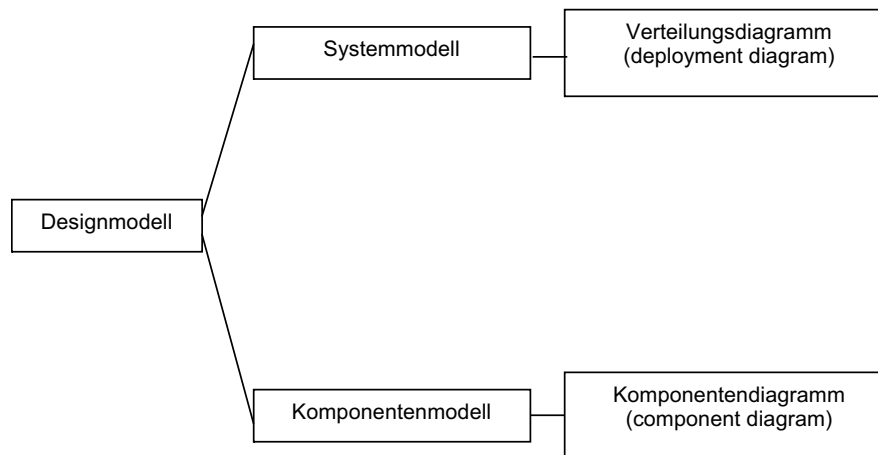


Abbildung 11.1: Diagramme der UML für das Design

- **Verteilungsdiagramm (deployment diagram):** Das Verteilungsdiagramm stellt die geplante Konfiguration eines Systems in Bezug auf Verteilung (Hardwarekomponenten, Softwarekomponenten) sowie Nebenläufigkeiten (Prozesse und deren Abhängigkeiten) dar. Dabei ist zu beachten, Objekte geeignet zu kategorisieren, sodass stabile Systemstrukturen und ein Maximum an Änderungslokalität erreicht werden. In [Kappel *et al.*, 2003] wird vorgeschlagen, Objekte in drei Kategorien einzuteilen,
 1. Problembereichsobjekte (entity objects),
 2. Schnittstellenobjekte (boundary objects),
 3. Steuerungsobjekte (control objects).

Das Verteilungsdiagramm wird häufig durch eine *textuelle Architekturbeschreibung* erläutert. Zur Darstellung der Systemarchitektur werden vielfach ergänzend Interaktionsdiagramme und Ablaufdiagramme (vgl. Abschnitt 10.1.4.2) verwendet.

- **Komponentendiagramm (component diagram):** Das Komponentendiagramm beschreibt Cluster eng gekoppelter Softwarekomponenten sowie deren Abhängigkeiten untereinander. *Pakete* können dabei Komponenten enthalten und umgekehrt. Daher wird das Komponentendiagramm in [Fowler, 2004b] auch als *package diagram* bezeichnet. Über *Schnittstellen* wird die Kommunikation einer Komponente mit den anderen beschrieben.

Komponentendiagramme werden immer dann eingesetzt, wenn Systeme so komplex sind, dass die zugehörigen Klassendiagramme zu unübersichtlich werden (i. e. nicht

mehr auf eine A4-Seite passen). Zusätzlich kann durch die explizite Beschreibung von Schnittstellen die Abhängigkeiten von Komponenten untereinander gering und leichter adaptierbar gehalten werden.

11.4.3 Vorgehen beim Design unter Verwendung der UML

Folgendes schrittweises Vorgehen wird für ein objektorientiertes Design unter Verwendung der Modelle und Diagramme der UML empfohlen (Ausgangsbasis ist der Analysebericht):

- **Entwickeln des Systemmodells:**

1. Implementierungsbeschränkungen festlegen (z. B. Zielplattform),
2. Systemarchitektur mit gewählter Hardware und Systemsoftware abstimmen und in Untersysteme zerlegen (Verteilungsdiagramm erstellen!),
3. Datenhaltung organisieren (Hauptspeicher, Dateien, Datenbanken); Persistenz von Komponenten (verwendete Mechanismen, Abbildungsvorschriften, Transaktionsgrenzen) festlegen (in Verteilungsdiagramm eintragen!),
4. (Echt-)Zeitverhalten und Nebenläufigkeiten der Untersysteme festlegen (sequenzielle Abhängigkeiten, Parallelitäten, ausschließende Komponenten, Synchronisationspunkte; Sequenzdiagramm erstellen!),
5. Benutzerschnittstelle gestalten (Eingabe und Ausgabe; Benutzerschnittstellenprototypen bauen!).

- **Entwickeln des Komponentenmodells:**

1. Abhängigkeiten und Schnittstellen zwischen den Komponenten festlegen (Komponentendiagramm erstellen!),
2. zu verwendende Komponenten (Bibliotheken, Rahmen, Muster, ...) festlegen bzw. auswählen (in Klassen-/Objektdiagramm eintragen, Komponentendiagramm erweitern!),
3. Strukturmodell adaptieren, erweitern und verfeinern (abstrakte und abgeleitete Komponenten, Seiteneffekte, Nebenläufigkeiten, Sichtbarkeiten; Klassen-/Objektdiagramm verfeinern!),
4. Komponentendefinition vervollständigen (Attribute, Operationen für jedes Modul bzw. jede Klasse, Schnittstellen ergänzen; Klassen-/Objektdiagramm erweitern!),
5. Gemeinsamkeiten von Komponenten durch Generalisierung dokumentieren (für einheitliches Verhalten, bessere Wartung und bessere Wiederverwendbarkeit); neu zu erstellende Komponenten auf wieder verwendbare abstimmen (Klassen-/Objektdiagramm adaptieren!),
6. Abhängigkeiten von Komponenten (durch Assoziation oder Aggregation) realisieren sowie deren Navigation optimieren (in Klassendiagramm und Kollaborationsdiagramm eintragen!),
7. Kontrollfluss zwischen den Komponenten festlegen (Kollaborationsdiagramm erstellen bzw. erweitern!),
8. Steuer- und Zustandsvariablen aus den Ablaufdiagrammen in das Klassendiagramm übertragen,

9. Operationen deklarativ und funktional spezifizieren, entsprechende Algorithmen in Bezug auf Performanz auswählen bzw. spezifizieren (Zustands- bzw. Aktivitätsdiagramm erstellen).

- **Festlegen von Designkompromissen und Designprioritäten.**

Wiederum ist zu beachten, dass die Schritte zur Erstellung der Modelle je nach Problem parallel bzw. in anderer Reihenfolge durchgeführt werden können. Das Ergebnis wird im *Designbericht* dokumentiert, der den Analysebericht ergänzt (vgl. Abschnitt 11.5.1).

11.4.4 Werkzeuge des Prototyping

Im Allgemeinen finden im Design dieselben Prototypingwerkzeuge Anwendung wie in der Analyse, speziell für das Architekturprototyping. Für eine Klassifikation und Kriterien zur Auswahl sei daher auf Abschnitt 10.1.4.4 verwiesen. Da der Erstellung von Benutzerschnittstellenprototypen im Design eine zentrale Bedeutung zukommt, wird im Folgenden auf Werkzeuge für diese Aufgabe näher eingegangen.

11.4.4.1 Beschreibungssprachen für Benutzerschnittstellen

Mithilfe von *Benutzerschnittstellen-Beschreibungssprachen* (*user interface description languages*; *UIL*) kann der statische Teil einer Benutzeroberfläche textlich beschrieben werden. Die Spezifikationsdatei definiert die grafischen Objekte (z. B. Menüs, Buttons, Textfelder) durch Angabe der Merkmalsausprägungen der zugehörigen Attribute (z. B. Größe, Farbe, Fonts, Position). Zusätzlich kann angegeben werden, wie Zustandsänderungen (z. B. Drücken eines Buttons) an die Anwendung weitergegeben werden. Auf diese Weise wird die Schnittstelle zur Anwendung definiert. Da eine UIL nur die Eigenschaften der Benutzerschnittstellen definiert, braucht der Designer nicht auf den Kontrollfluss Rücksicht nehmen.

Der größte Nachteil gegenüber interaktiven Systemen besteht darin, dass Änderungen an der Schnittstellenbeschreibung nicht unmittelbar sichtbar werden. Man muss – wie bei der konventionellen Programmierung – die Spezifikationsdatei zunächst in einen Zwischencode übersetzen, diesen Zwischencode mit dem (vorgefertigten bzw. automatisch erstellten) Systemrahmen binden und kann erst dann die Benutzerschnittstelle testen. Da bei einer Benutzerschnittstelle der visuelle Eindruck von entscheidender Bedeutung ist, die optimale Wirkung auf den Benutzer aber erst durch oftmaliges Ausprobieren verschiedener Attributbelegungen herausgefunden werden kann, ist diese Vorgehensweise ein zeitaufwändiges Unterfangen.

11.4.4.2 Grafische Generatoren zur Erstellung von Benutzerschnittstellen

Grafische Benutzerschnittstellen-Generatoren (*graphic user interface builder*; *GUI builder*) gewährleisten durch interaktive Eingabemöglichkeiten die rasche Änderbarkeit sowohl der Eigenschaften als auch des logischen Aufbaus der Oberflächenobjekte. Der Designer kann durch interaktive Manipulationen die Benutzerschnittstelle aus einer Reihe von Basisobjekten zusammenbauen. Das Aussehen der Schnittstelle kann durch Modifizieren verschiedener Attributwerte ebenfalls interaktiv unterstützt an die Benutzerwünsche angepasst werden.

Auf Grund dieser Flexibilität ist der Designer in der Lage, Aussehen und Funktionalität der Schnittstelle durch Experimentieren optimal an die Bedürfnisse des Anwenders anzupassen.

Als zusätzliches Feature bieten einige dieser Systeme einen so genannten *Dialogmanager* an. Der Dialogmanager ist für die Kommunikation zwischen der grafischen Benutzerschnittstelle und der Anwendung zuständig und nimmt dem Oberflächendesigner einen großen Teil der Arbeit zum Verwalten der Benutzerschnittstellen-Komponenten ab. Da es auch möglich ist, den Dialogmanager interaktiv zu betreiben, kann bereits in der Prototypingphase mit geringem Aufwand sehr viel Funktionalität in den Prototypen gepackt werden.

11.4.5 Werkzeuge des Compilerbaus

Die Motivation für die Verwendung *attributierter Grammatiken* für Designaufgaben ist die, dass in vielen Fällen der Programmablauf wesentlich durch den Strom der Eingabedaten gesteuert wird. Wenn man daher die Struktur dieser Daten durch eine Grammatik beschreibt und diese durch semantische Aktionen anreichert, kann man mittels Werkzeugen des Compilerbaus (z. B. Compiler-Compiler, vgl. [Dobler und Pirklbauer, 1990]) daraus ein Programm erstellen lassen, das dem gewünschten Ablauf entspricht. Diese Technik ist anwendbar, wenn im Wesentlichen nur ein Eingabestrom vorliegt und die Eingabeinformation genügend strukturiert ist [Pomberger und Blaschek, 1996].

Die Vorteile dieser Vorgehensweise sind die gut strukturierte Modularisierung des Problems, die präzise Problembeschreibung, aus der sich eine exakte technische Dokumentation herleiten lässt, sowie die Möglichkeit der automatischen Programmgenerierung. Nachteile sind die Unverständlichkeit für viele Auftraggeber (Compilerbaukenntnisse sind zum Verstehen attributierter Grammatiken notwendig) und die Beschränkung auf nur einen Eingabestrom.

Attributierte Grammatiken werden z. B. in [Coleman *et al.*, 1994] zur Beschreibung der zugelassenen Benutzerinteraktionen und deren Reihenfolge verwendet. Für weiter gehende Informationen zu attributierten Grammatiken und Compilerbauwerkzeugen sei beispielsweise auf [Aho *et al.*, 1986] verwiesen.

11.5 Ergebnisse

Designdokumente dürfen nicht als Selbstzweck erstellt werden, z. B. um mangelhaftes Design durch umfangreiche Dokumente zu kaschieren. Sie müssen (im Fall interner Dokumente) dem Auftragnehmer mehr nützen als Aufwand verursachen bzw. (im Fall externer Dokumente) explizit – als *Anforderung* – vom Auftraggeber gefordert und deren Erstellung daher auch – als *Aufgabe* – eingeplant sein.

11.5.1 Designbericht

Der *Designbericht* ist eine Erweiterung des Analyseberichts um die Designmodelle sowie gegebenenfalls erstellte Prototypen. Der Designbericht in seiner vollständigen Form wird normalerweise nicht dem Auftraggeber zur Verfügung gestellt, er dient der internen Dokumentation des Entwicklungs-Know-hows. Auszüge daraus sowie die Prototypen können jedoch dem Auf-

traggeber zur besseren Verdeutlichung der Inhalte des Pflichtenhefts zur Verfügung gestellt werden.

11.5.2 Pflichtenheft

Gemäß [VDI, 1991] ist das *Pflichtenheft* (*system specification*) die Beschreibung der Realisierung aller Anforderungen des Lastenhefts. Das Pflichtenheft enthält das Lastenheft. Im Pflichtenheft werden die Anwendervorgaben detailliert und die Realisierungsanforderungen beschrieben. Im Pflichtenheft wird definiert, WIE und WOMIT die Anforderungen zu realisieren sind. Das Pflichtenheft beschreibt also sämtliche Aufgaben und Pflichten, die der Auftragnehmer im Rahmen des Projekts übernehmen muss. Es wird daher auch *Leistungsverzeichnis* genannt.

Da das Pflichtenheft eine Erweiterung des Lastenhefts darstellt, wird der Inhalt des Lastenhefts wie folgt im Pflichtenheft adaptiert:

1. **Einführung in das Projekt, Projektziele:** keine Änderungen zugelassen (bei Änderungen neues Projekt),
2. **Beschreibung des Istzustands:** Ergänzungen bzw. Streichungen, Kommentare,
3. **Beschreibung des Sollzustands:** Verfeinerung, Festlegung der zu erfüllenden und nicht zu erfüllenden Anforderungen,
4. **Schnittstellen:** Adaptierung, Detaillierung (z. B. Benutzerschnittstelle: Snapshots, Dokumentation, Interaktionsabläufe),
5. **Systemanforderungen:** Detaillierung der Systembedingungen (z. B. detaillierte Anforderungen an die Hardware), notwendige Systemsoftware (z. B. Compiler, Bibliotheken), Datenhaltungsmodell (Verwendung von Datenbanken, Datensicherung, Datenschutz und Datenarchivierung),

NEU: **Design:** Systemdesign, Komponentendesign,

6. **Anforderungen für Inbetriebnahme und Einsatz:** Verfeinerung, Adaptierung (z. B. Umfang und Detailliertheitsgrad der Dokumentation),
7. **Qualitätsanforderungen:** evtl. Adaptierungen,
8. **Anforderungen an die Projektentwicklung:** Adaptierung sowie Verfeinerung des Zeitplans, Konkretisierung der Projektdurchführungsbestimmungen (z. B. Lieferung der Abnahmedaten, Durchführung der Abnahme, Garantie und Wartung, Rechte und Pflichten von Auftraggeber und Auftragnehmer – etwa bzgl. des Quellcodes; diese Bedingungen können aus dem Auftrag entnommen werden, bzw. es kann auf den Auftrag verwiesen werden),
9. **Anhang:** Erweiterung.

Das Pflichtenheft wird in der Regel nach Auftragserteilung vom Auftragnehmer erstellt, falls erforderlich unter Mitwirkung des Auftraggebers. Zur Absicherung des Auftragnehmers ist im

Pflichtenheft explizit anzuführen, was im Projektergebnis alles nicht enthalten sein wird (v. a. in Bezug auf Ergebnisse der Zielerforschung, vorangegangene Kontakte mit dem Auftraggeber, Lastenheft). Probleme treten jedoch häufig auch dann auf, wenn sich Spezifikationen des Auftraggebers nachträglich als zu ungenau herausstellen. Sich genau an die Spezifikation zu halten ist in diesem Fall nicht empfehlenswert, da der Auftraggeber sicher vielfach mit dem Ergebnis nicht zufrieden wäre. Besser ist es, die – ungenügende – Spezifikation als Minimalanforderung zu betrachten und gemeinsam mit dem Auftraggeber Verfeinerungen auszuarbeiten. Solche nachträgliche Adaptierungen bzw. Erweiterungen des Pflichtenhefts sind immer schriftlich festzuhalten und vom Auftraggeber und vom Auftragnehmer zu unterzeichnen; im Allgemeinen sind sie mit einer Vergrößerung des Projektumfangs verbunden.

Der Auftragnehmer prüft bei der Erstellung des Pflichtenhefts die Widerspruchsfreiheit und Realisierbarkeit der im Lastenheft genannten Anforderungen (so er nicht bereits an der Erstellung des Lastenhefts beteiligt war). Das Pflichtenheft bedarf der Genehmigung durch den Auftraggeber („Abnahme“). Nach der Abnahme wird das Pflichtenheft die verbindliche Vereinbarung für die Realisierung und Abwicklung des Projekts für Auftraggeber und Auftragnehmer.

Checkliste für das Pflichtenheft

Diese Checkliste dient zur Vorprüfung des Pflichtenhefts durch den Auftragnehmer. Ein erneutes Durchgehen der Checks des Lastenhefts für das Pflichtenheft ist sehr empfehlenswert.

- **Kompatibilität:**

- Ist das Design mit dem Lastenheft kompatibel?
- Sind Anforderungsänderungen zum Lastenheft explizit im Pflichtenheft dokumentiert?

- **Technische Qualität:**

- Wird das Design vom Auftragnehmer als gut beurteilt?
- Sind bei mit „nicht gut“ beurteilten Designbereichen die Gründe bekannt, akzeptiert und dokumentiert?

- **Realisierbarkeit:**

- Ist das Design in dem im Pflichtenheft festgelegten Umfeld vom Auftragnehmer in ein Produkt umsetzbar?
- Wird die Zeitplanung als realistisch eingestuft?

11.5.3 Benutzerdokumentation

Um die Designergebnisse während der Implementierung leicht zugänglich zu haben, ist es sehr sinnvoll, die Benutzerdokumentation (bzw. die erste Version davon) bereits während der Designphase zu erstellen. Darüber hinaus können durch ein Review der Benutzerdokumentation mit dem Auftraggeber Designfehler bereits vor der Implementierung erkannt werden.

Die Benutzerdokumentation umfasst folgende Komponenten:

1. **Allgemeine Produktbeschreibung:** Die allgemeine Produktbeschreibung informiert über den Anwendungsbereich des Produkts und über dessen Funktionalität bzw. nicht abgedeckte Funktionalität. Sie enthält Angaben über benötigte Hard- und Software-Ressourcen, organisatorische und infrastrukturelle Voraussetzungen sowie ggf. einfache Anwendungsbeispiele.

Ein Interessent soll anhand der allgemeinen Produktbeschreibung den Zweck des Produkts verstehen und feststellen können, wie er das Produkt am besten nutzen kann. Einzelheiten zum Produkt sind nicht erwünscht, ein Überblick über die gesamte verfügbare Benutzerdokumentation soll jedoch gegeben werden.

2. **Installationsanleitung:** Die Installationsanleitung beschreibt, was an Hardware und Software zur Installation notwendig ist, sowie den Vorgang der Installation selbst. Sie umfasst auch eine Liste aller zum Lieferumfang gehörigen Dokumente und Komponenten.
3. **Anweisungen zur Inbetriebnahme:** Diese umfassen den Aufruf des Systems, eine Liste dabei möglicher Fehler mit deren Behandlung, den Zugang zu Hilfsfunktionen sowie eventuell ein Beispiel.
4. **Kurzeinführung (Tutorial):** Das Ziel der Kurzeinführung ist es, einem Neuling rasch den Betrieb des Systems mit seinen wesentlichsten Funktionalitäten zu ermöglichen. Daher ist dieses Dokument am besten so aufgebaut, dass der Inhalt Schritt für Schritt am System nachvollzogen werden kann. Ein Tutorial kann auch (oder ergänzend) in Online-Form verfügbar sein.
5. **Benutzerhandbuch:** Das Benutzerhandbuch gibt eine Einführung in das Programm für den täglichen Gebrauch. Die Beschreibung muss dabei so erfolgen, dass nur die für den Benutzer erkennbare Funktionalität (via Benutzerschnittstelle) dargestellt wird, die interne Funktionalität bzw. interne Abläufe werden nicht angesprochen.

Das Benutzerhandbuch enthält die Beschreibung der Systemfunktionalität in leicht verständlicher Form sowie die jeweils notwendigen Eingaben und erzielbaren Ausgaben. Alle Ein- und Ausgabedaten werden erläutert und begründet, am besten anhand von im typischen Gebrauch vorkommenden Beispielen. Eine Liste aller Fehlermeldungen soll das Ermitteln von Fehlerursachen und Maßnahmen zu deren Behebung ermöglichen. Ein Kapitel soll sich mit den Möglichkeiten zur individuellen Anpassung des Systems an den Benutzer beschäftigen.

Das Benutzerhandbuch soll so geschrieben sein, dass es sequenziell gelesen werden kann, wobei die Sprache für den Anwender verständlich sein muss. Dies ist besonders wichtig bei der Wahl und Darstellung von Beispielen sowie bei der Erläuterung von Fachbegriffen. Verwendete Fachbegriffe sollen zusätzlich in einem eigenen Kapitel zusammengestellt und erklärt werden.

Für den Ersteller des Systems ist es einfacher, ein Feature-orientiertes Handbuch zu erstellen, d. h., eine Beschreibung gegliedert nach den Funktionalitäten. Für Benutzer hingegen sind aufgabenorientierte Beschreibungen oftmals am leichtesten verständlich [Sohr, 1983]. Ein aufgabenorientiertes Handbuch beschreibt die einzelnen Abläufe, die der Benutzer durchführen kann, aus der Sicht und in der Reihenfolge und Strukturierung des Benutzers. Dies führt im Allgemeinen zu sehr umfangreichen Handbüchern,

die trotzdem nur einen Teil der möglichen Anwendungen beschreiben. Daher wird als ein Kompromiss oftmals ein Feature-orientiertes Handbuch um ein aufgabenorientiertes Kapitel ergänzt, in dem häufig vorkommende, repräsentative Aufgaben detailliert beschrieben werden.

6. **Referenzhandbuch:** Das Referenzhandbuch dient als möglichst detailliertes Nachschlagewerk der Systemfunktionen. Es beschreibt Details der Lösung, Datenformate, Fehlerlisten und Fehlerursachen und wird in der Implementierungsphase um Implementierungsdetails ergänzt. Genaue Beispiele sind dazu vielfach notwendig. In der Testphase werden Grenzen des Systems und Beschränkungen der Benutzung des Systems (in der jeweiligen Version) hinzugefügt.

Wenn ein Benutzer im Referenzhandbuch nachschaut, so benötigt er normalerweise Information zu einer Systemeigenschaft oder zu einem Systemverhalten möglichst rasch. Das Referenzhandbuch wird daher meistens selektiv und Modul für Modul gelesen. Es ist im Allgemeinen in eher nüchternem Stil gehalten. Augenmerk muss jedoch auf eine äußerst klare Strukturierung und einen ausführlichen Index gelegt werden.

7. **Referenzkarte (quick reference guide):** Eine Referenzkarte enthält die essenzielle Information des Referenzhandbuchs in Kurzform („auf einer Seite“). Eine gut aufgebaute Online-Hilfe kann die Referenzkarte ersetzen.
8. **Online-Hilfe:** Die Online-Hilfe deckt im Idealfall den Inhalt des Benutzerhandbuchs sowie des Referenzhandbuchs ab, wobei der Zugriff jedoch immer selektiv und modulweise erfolgt. Idealerweise sind die einzelnen Komponenten des Systems direkt mit den dazugehörigen Komponenten der Online-Hilfe verknüpft, und zwar unter Berücksichtigung des gerade aktuellen Systemstatus („kontextsensitive Online-Hilfe“). Moderne Online-Hilfen verfügen über Hypertext-Links, mit denen zu weiteren Hilfebereichen verzweigt werden kann. Auch bei der Online-Hilfe ist ein ausführlicher Index sehr wichtig.
9. **Supplement, Readme-Datei:** Kurzfristige Änderungen oder Ergänzungen zur Benutzerdokumentation werden in einem beigefügten Supplement angegeben, welches in die nächste Version der Dokumentation eingearbeitet wird. Informationen in letzter Minute können in eine so genannte „Readme-Datei“ geschrieben werden, welche in elektronischer Form der Lieferung beigefügt wird und vom Anwender vor der Installation des Systems gelesen werden soll. Allerdings sollte man sich darauf nie verlassen und bei bedeutenden Informationen besser die Dokumentation aktualisieren, auch wenn damit eine Verzögerung der Produktauslieferung in Kauf genommen wird.

Da eine Benutzerdokumentation nur so gut ist, wie sie vom Benutzer empfunden wird, ist der Schreibstil von entscheidender Bedeutung. In [Sommerville, 2004] werden u. a. folgende Regeln angeführt:

- Aktive Formulierungen sind passiven vorzuziehen.
- Mehrere kurze Sätze sind besser als ein langer. Ein Satz soll eine Tatsache beschreiben.
- Fortlaufend nummerierte Aussagen bleiben besser im Gedächtnis. Absätze sollten kurz sein; sieben Zeilen ist ein guter Richtwert.
- Komplexe Sachverhalte sind durch mehrere Sichten mit verschiedener Schwerpunktsetzung für den Benutzer leichter verständlich.

- Verwendete Begriffe sind zu definieren, speziell Fachbegriffe und Begriffe mit vielen „offensichtlichen“ Bedeutungen (z. B. Objekt).
- Hierarchisch nummerierte Überschriften schaffen Übersicht. Gut formulierte und sauber strukturierte Überschriften führen zu einem gut lesbaren Inhaltsverzeichnis.

Checkliste für die Benutzerdokumentation

Folgende Anforderungen sind an eine gute Benutzerdokumentation zu stellen:

- **Vollständigkeit:**
 - Sind alle im Pflichtenheft angeführten Komponenten und Funktionalitäten abgedeckt?
- **Verständlichkeit:**
 - Ist der Stil der Dokumentation klar verständlich und präzise?
 - Ist die Dokumentation in der Sprache des Benutzers verfasst?
- **Strukturiertheit:**
 - Ist der Aufbau klar verständlich? Ist der Detaillierungsgrad auf jeder Ebene adäquat?
- **Modularität:**
 - Ist die Beschreibung jeder Komponente bzw. jeder Funktion für sich verständlich, sodass selektives Lesen möglich ist?
 - Sind Querverweise dort und nur dort vorhanden, wo sie sinnvoll sind?
- **Benutzbarkeit:**
 - Ist die Dokumentation als Nachschlagewerk verwendbar, d. h., ist die gewünschte Information rasch auffindbar?
 - Sind Inhaltsverzeichnis, Index und Glossar vorhanden und ausführlich genug?
- **Aktualität:**
 - Stimmt die Dokumentation mit der referenzierten Programmversion überein?
 - Passt die schriftliche Dokumentation zur Online-Hilfe bzw. stimmen die Querverweise der Online-Hilfe und der schriftlichen Dokumentation überein?

Diese Fragen verursachen großteils einen hohen Prüfaufwand und sind nicht vollständig beantwortbar. Wie in [Kaner *et al.*, 1999] angegeben ist, kann eine Person pro Stunde 3 bis 5 Seiten Dokumentation überprüfen; höhere Werte lassen auf eine ungenügende Prüfung schließen. Dennoch ist die Prüfung der Benutzerdokumentation essenziell. Gerade Unstimmigkeiten zwischen System und Dokumentation stellen nämlich die vom Benutzer am raschesten zu entdeckenden und unangenehmsten Fehler dar. Diese Fehler sind einfach nachzuvollziehen und kaum zu dementieren; entweder es muss das Programm geändert werden oder die Dokumentation oder beides. Daher ist eine durch sorgfältige Prüfung entstandene hochwertige Benutzerdokumentation ein wichtiger Grundstein für den Erfolg eines Projekts.

12 Implementierung

Dieses Kapitel beschreibt, welche Tätigkeiten aus Projektsicht notwendig sind, um effizient qualitativ geeigneten Code zu erzeugen. Der Bereich des Debuggens wird ebenfalls der Implementierung zugerechnet, da hierbei die Fehlersuche und Korrektur durch den Implementierer selbst erfolgt. Abschließend wird auf die Bedeutung eines lauffähigen Programms eingegangen, also eines Programms von solcher Qualität, dass es für das Testen durch andere Personen als den Implementierer selbst geeignet ist.

12.1 Codieren

12.1.1 Ziel

Das *Ziel des Codierens* ist das Umsetzen der Designergebnisse in ein Programm mittels einer Programmiersprache bzw. einer Programmierumgebung. Das Ergebnis des Codierens ist der (*Quell-Code* (*source code*)). Code wird im Allgemeinen inkrementell, d. h., in Teilen, erzeugt. Diese Teile müssen übersetzt, zusammengebaut und getestet werden, um ein lauffähiges Programm zu ergeben. Aus den Erkenntnissen durch die Implementierung (und die testweise Ausführung des Codes) werden dann wiederum Rückschlüsse für das Design gezogen.

12.1.2 Aufgaben

Aus dem Ziel des Codierens, also dem schrittweisen, korrekten Umsetzen des Designs, ergeben sich folgende Aufgaben:

- Umsetzen des Systemdesigns: Codieren des Daten-/Objektmodells samt Schnittstellen,
- Umsetzen des Komponentendesigns: Konkretisieren der Klassen, insbesondere von deren Methoden unter Ausnützung der Vererbung bestehender Methoden; Erstellen neuer Klassen durch Ableiten bzw. Aggregieren bestehender Klassen,
- Einzeltest jeder Komponente,
- Zusammenbau zum Gesamtsystem, Debuggen bis zur Funktionsfähigkeit.

Grundlegende Entwicklungsschritte der Programmierung

Den Beginn der Programmierung (um 1940) kennzeichnete sehr große *Maschinennähe*. Es wurde in Maschinensprache (später Assembler) programmiert; detaillierte Kenntnisse der Systemhardware waren notwendig. Im Allgemeinen war zur Umsetzung eines Problems in ein Programm sehr großes Abstraktionsvermögen notwendig.

Eine erste Verbesserung ergab sich um 1960 durch die Möglichkeit, *Unterprogramme* zu verwenden. Diese entstanden aus dem Wunsch, Speicherplatz zu sparen. Es ergab sich daraus aber auch ein erster Abstraktionsschritt bei der Programmierung: Mehrfach verwendbare Teile wurden zusammengefasst und einheitlich gestaltet. Außerhalb des gerade erstellten Programms waren jedoch solche Programmteile kaum wieder verwendbar, das gesamte Programmsystem war sehr eng verwoben und kaum durchschaubar („Spagetticode“).

Um 1970 begann das Zeitalter der *strukturierten Programmierung*. Programmiersprachen wie Algol und Pascal ermöglichten es dem Programmierer, seine Programme mit einer auch für andere verständlichen Ablauflogik zu schreiben. Eine klare Zerlegung eines Problems in Teilprobleme führte zur Strukturierung der Programme in Prozeduren (*funktionale Zerlegung*), vgl. z. B. die Methode der schrittweisen Verfeinerung [Wirth, 1971].

Der nächste Entwicklungsschritt war durch die *modulorientierte Programmierung* gekennzeichnet (um 1980). Aus der Anforderung heraus, dass große Softwaresysteme nicht mehr in einem Stück von einem einzelnen Programmierer erstellt werden können, ergab sich die Notwendigkeit der Abgrenzung von einzelnen Teilaufgaben und der Spezifikation von Schnittstellen (*Datenkapselung*). Diese *datenorientierte Zerlegung* war z. B. mit Programmiersprachen wie Modula-2 oder Ada möglich.

Um 1990 begann das Zeitalter der *objektorientierten Programmierung*. Das objektorientierte Paradigma, das am sinnvollsten durchgehend von der Analyse (*objektorientierte Zerlegung*) bis zur Implementierung angewendet wird, führte zu Programmsystemen, die durch gute Erweiterbarkeit und hohe Wiederverwendbarkeit der einzelnen Komponenten gekennzeichnet sind. An Programmiersprachen stehen objektorientierte Erweiterungen bestehender Sprachen (z. B. C++), aber auch eigene Entwicklungen (z. B. Smalltalk, Java) zur Verfügung.

Die Schlagworte Anfang des neuen Jahrtausends sind *komponentenorientierte Programmierung* (Reduzierung der Klassenvernetzung), *agentenorientierte Programmierung* (Verteilung und Autonomisierung der Ablauflogik) sowie *aspektororientierte Programmierung* (Extraktion der Umsetzung bestimmter Sichten, wie z. B. Fehlerbehandlung oder Objektpersistenz, aus der Gesamtlogik). Für nähere Informationen zu diesen Konzepten und zur Entwicklung der Programmierung allgemein sei z. B. auf [Rechenberg und Pomberger, 2002] verwiesen.

12.1.3 Techniken

12.1.3.1 Transformieren der Designmodelle

Das Design in guten Code zu transformieren ist die klassische Kernaufgabe der Programmierung. Im Folgenden seien die wichtigsten Richtlinien für eine gute Codierung angegeben. Für eine Einführung in die Programmierung, insbesondere der Algorithmik, sei z. B. auf [Cormen *et al.*, 2001] verwiesen.

Eine gute Codierung zeichnet sich aus durch:

- das Widerspiegeln der Strukturen des Designs in der Implementierung,
- das Realisieren der Abstraktionsebenen des Designs in der Implementierung,
- das Realisieren klarer Schnittstellen zwischen den Komponenten eines Systems,
- das Verwenden guter Bibliotheken bzw. Wiederverwenden von bewährtem Code,
- das Festlegen sicherer Schnittstellen durch das Vermeiden von öffentlichen Komponenten (Erzwingen der Zugriffe über Methoden),
- das Halten der Konsistenz der Implementierung mit dem Datenlexikon,
- das Einhalten eines guten Programmierstils in Bezug auf Strukturiertheit, einheitliche Form und Ausdruckskraft,
- das sorgfältige Überlegen und Dokumentieren von Änderungen (Ist die Änderung gerechtfertigt? Was sind die Auswirkungen und Kosten der Änderung?),
- das saubere Verwalten verschiedener Varianten (Welche sind wirklich notwendig?) und Revisionen des Programmsystems,
- das weitestgehende Vermeiden von Optimierung während des Codierens; diese lenkt vom Codieren ab, vergrößert die Fehlerwahrscheinlichkeit und zahlt sich nur an wenigen, oft sehr spät erkennbaren Stellen wirklich aus.

12.1.3.2 Instrumentieren des Codes

Als *Instrumentieren eines Codes* bezeichnet man das Versehen des Codes mit zusätzlichen Codeteilen, die im Allgemeinen Hilfsinformation ausgeben.

Instrumentieren wird zu folgenden Zwecken durchgeführt:

- **Feststellen der Codelokalität zur Laufzeit:** Zur Eingrenzung der Programmstellen bei Abstürzen, Endlosschleifen etc. werden *Proben* in den Code eingefügt. Proben sind Codezeilen, die eine Information ausgeben, dass die betreffende Programmstelle gerade durchlaufen wird. Solche Proben können normalerweise ein- und ausgeschaltet werden.
- **Prüfen der Pfadabdeckung:** Um prüfen zu können, ob alle Pfade eines Programms (bzw. eines kritischen Teils) durchlaufen werden, wird jeder Pfad mit einer Probe instrumentiert. Wenn alle Proben mindestens einmal ausgegeben wurden, wurde jeder Pfad durchlaufen.
- **Dynamische Analyse:** Codeteile werden eingefügt, um eine Zeitmessung, eine Ausführungszählung o. Ä. für bestimmte Programmbereiche durchführen zu können (*Profiling*), vgl. z.B. [Dobler und Huber, 1991].
- **Prüfen der Testqualität:** Beim *Bebuggen* werden absichtlich Fehler in den Code eingebaut, um Tester zu motivieren, Fehler zu finden. In [Kaner *et al.*, 1999] wird

argumentiert, dass Tester eher bemüht sind, Fehler in einem Programm zu finden, wenn sie wissen, dass Fehler enthalten sind.

Um die Testadäquatheit von Testdaten zu prüfen, werden so genannte *Mutationen* eingeführt. Mutationen sind kleine Programmänderungen, die sich in irgendeinem Test auswirken sollten. Ist dies nicht der Fall, so ist die Menge bzw. Detailliertheit der Testbeispiele vermutlich nicht adäquat [Beizer, 1995].

Teilweise sind für diese Zwecke spezielle Werkzeuge verfügbar: *Instrumentierer* unterstützen das Instrumentieren des Codes, während *Abdeckungsmonitore* (*coverage monitors*) Probenausgaben sammeln und diese analysieren und zusammenfassen.

Das Hauptproblem des Instrumentierens ist, dass in den Code Teile eingefügt werden. Einerseits können dadurch neue Fehler im Code verursacht werden, andererseits wird mit dem instrumentierten Programm ein anderes Programm getestet als das, welches später ausgeliefert wird. Trotzdem kann gesagt werden, dass generell der Nutzen des Instrumentierens diese Nachteile mehr als ausgleicht.

12.1.3.3 Einfügen logischer Bedingungen

Programmierer wissen oft, dass bestimmte Bedingungen an einer bestimmten Programmstelle erfüllt sein müssen. An solchen Programmstellen können *Assertionen* eingefügt werden, dies sind logische Aussagen, die diesen Bedingungen entsprechen. Assertionen verhalten sich in Programmen so, dass nichts (Sichtbares) getan wird, wenn die Assertionsbedingung erfüllt ist, dass aber eine Information bzw. eine Fehlermeldung zusammen mit der Lokalität der Assertion ausgegeben wird, wenn eine Assertionsbedingung nicht erfüllt ist. Typischerweise werden solche Assertionen bedingt übersetzt, sodass Produktversionen in voller Geschwindigkeit laufen können, aber aus demselben Quellcode erzeugt werden wie die Version mit Assertionen. Die Problematik dieser Vorgehensweise wird in [Coleman *et al.*, 1994] verglichen mit Seeleuten, die während der Übung an Land mit Schwimmwesten arbeiten, während sie die tatsächliche Arbeit auf See ohne solche durchführen müssen.

Zusätzlich zu allgemeinen *Bedingungen* (*conditions*) werden auch *Vorbedingungen* (*preconditions*; Eingangsprüfung bei Methoden), *Nachbedingungen* (*postconditions*; Ausgangsprüfung bei Methoden) und *Invarianten* (*invariants*; Gültigkeit während einer gesamten Methodenausführung) eingeführt, z. B. in der Programmiersprache Eiffel [Meyer, 1997]. Manche Bedingungen gelten jedoch für ein ganzes System oder eine ganze Klasse. Gute Lokalitäten für solche Bedingungen sind vielfach schwer anzugeben: In Eiffel werden hierzu Klasseninvarianten angeboten; in C++ und Java wird als einzige Möglichkeit zur Bedingungsprüfung die Assertion (Befehl `assert`) angeboten. Um mit diesem Befehl verschiedene Bedingungen sauber prüfen zu können, ist es ratsam, Makros und Verwendungsregeln für Assertionen festzulegen.

Um das interne Testen von Assertionen sichtbar zu machen, kann es manchmal sinnvoll sein, die Assertionsprüfungen mit Proben zu versehen und so Information über die geprüften Assertionen auszugeben.

12.1.3.4 Code-Restrukturierung

Code-Restrukturierung (refactoring) ist die disziplinierte Änderung eines existierenden Codes mit dem Zweck der Designverbesserung, ohne das funktionale Verhalten zu verändern [Fowler, 1999]. Durch die Restrukturierung soll der Code einfacher und verständlicher gestaltet werden, damit eine nachfolgende Erweiterung leichter möglich ist.

Die Technik der Code-Restrukturierung wurde erstmals im Rahmen der Entwicklung von Smalltalk propagiert [Goldberg und Robson, 1989]. Populär wurde sie in den letzten Jahren durch die enge Verknüpfung mit *eXtreme Programming*, um den im Rahmen dieser Vorgehensmethode sehr rasch entwickelten, aus Design Sicht oft unsauberen Code zu verbessern.

Um zu erkennen, wann und wie Code-Restrukturierung angewendet werden soll, ist in [Fowler, 1999] ein Katalog von 22 Mustern *schlechten Codes (bad smells)* angegeben. Beispiele solchen schlechten Codes sind duplizierter Code, sehr große Klassen, zu lange Parameterlisten oder zu lange Methoden. Für jedes Codemuster ist eine detaillierte Beschreibung der notwendigen Code-Restrukturierung festgelegt.

Ein bemerkenswerter Ansatz ist, dass auch die Existenz von Kommentaren als Indikator für eine notwendige Code-Restrukturierung angesehen wird. So wird z.B. vorgeschlagen, dass

- ein Kommentar, der einen Codeblock erklärt, darauf hinweist, diesen Codeblock als Methode auszulagern;
- ein Kommentar, der zur Erklärung einer kurzen Methode notwendig ist, nahe legt, diese Methode besser zu benennen;
- ein Kommentar, der eine Bedingung bzw. die Annahme eines benötigten Status des Systems beschreibt, besser durch eine Assertion ersetzt wird.

Obwohl die empfohlene Restrukturierung in all diesen Situationen durchaus vernünftig ist, wird eine derartige Praxis des Hinterfragens von Kommentaren in der Realität vermutlich dazu führen, dass (noch) weniger kommentiert wird als bisher. Der Code wird also schlecht sein, auf Grund der fehlenden Kommentare der Restrukturierungsbedarf aber weniger einfach erkannt und, was schwerer wiegt, der Code noch schlechter zu verstehen sein.

Für eine erfolgreiche Code-Restrukturierung ist es essenziell, einerseits ein lauffähiges Programm daraus generieren zu können und andererseits das Verhalten dieses Programms (möglichst automatisch) testen zu können. Erst dadurch kann sichergestellt werden, dass das funktionale Verhalten des Codes nach der Restrukturierung dem ursprünglichen Verhalten entspricht. Die Möglichkeit, eine Suite von Einzeltests für die zu restrukturierende Komponente weitestgehend automatisiert durchlaufen zu lassen (vgl. Abschnitt 12.2.4.4), ist daher unbedingte Voraussetzung für eine kontrollierte Code-Restrukturierung.

Da viele Techniken der Code-Restrukturierung Änderungen von Schnittstellen nach sich ziehen (selbst einfache, häufig verwendete, wie „Methode umbenennen“), ist der Einsatz von Konfigurationsverwaltungssystemen ebenfalls unabdinglich.

[Boehm, 2002] merkt als Problem der Code-Restrukturierung an, dass der Restrukturierungsaufwand gerade bei unerfahrenen Softwareentwicklern überproportional zur Anzahl von

Anforderungen steigt. Zudem stammen bei den untersuchten Projekten „... 80 % der notwendigen Codeänderungen aus Architekturbrüchen (wie z. B. Verletzung der Komponententrennung zur Performanzsteigerung), Fehlertoleranzproblemen oder Sicherheitsproblemen, in welchen Fällen kein noch so umfangreiches Restrukturieren den ursprünglichen Code noch retten kann“ [Boehm, 2002]. Dies gibt auch Kent Beck in [Beck, 2002] zu: „Ist Refactoring nicht einfach durchführbar, muss mehr Wert auf das Design gelegt werden.“ Damit entsteht natürlich ein Henne-Ei-Problem, das verhindert werden kann, wenn man Code-Restrukturierung so einsetzt, wie sie in [Fowler, 1999] auch beabsichtigt ist: zur (lokalen) Verbesserung des Komponentendesigns und nicht zur Entwicklung einer guten Gesamtarchitektur eines Systems. Dafür sind ein gutes – vom Code abstrahiertes – Vorstellungsvermögen und ein entsprechendes Rüstzeug zur Erstellung von (System-)Designmodellen unumgänglich, auch wenn dies nicht in die Welt mancher eXtreme-Programmierer passt.

12.1.3.5 Erstellen von Testrahmen

Der *Bau von Testrahmen* (*Gerüstbau; scaffolding*) für Software umfasst temporäre Programme und Daten, die dem Entwickler Zugang zu Codeteilen in Form ausführbarer Programme geben. Testrahmen erlauben darüber hinaus das Überprüfen von Code ohne das Risiko, Fehler durch das Interagieren mit anderen Codeteilen einzubauen. Voraussetzung dafür ist natürlich ein gut getesteter, „fehlerfreier“ Testrahmen. Der Programmierungsumfang für Testrahmen kann beträchtlich sein: Schätzungen gehen von 50 % des zu überprüfenden Codes bis hin zur gleichen Codemenge für den Testrahmen wie für den zu prüfenden Code.

Komponenten von Testrahmen sind:

- **Stumpf:** Ein Stumpf (*stub*) ist eine Routine, die nur aus Dummycode besteht, sodass sie von einer übergeordneten Routine, die getestet werden soll, aus aufgerufen werden kann. Ein Stumpf kann
 - nichts tun und sofort zur übergeordneten Routine zurückspringen,
 - die Eingangsdaten prüfen,
 - Diagnosemeldungen ausgeben (z. B. die Eingangsdaten),
 - durch eine Probe über die Lokalität informieren,
 - Zeitverhalten messen oder Aufrufe zählen,
 - aber auch eine „quick-and-dirty“-Version der zu entwickelnden Routine sein.
- **Treiber:** Ein Treiber (*driver*) ist eine Dummyroutine, welche die zu testende Routine aufruft. Dieser kann
 - die zu testende Routine mit fixen Eingabedaten aufrufen,
 - interaktiv eine Eingabe verlangen oder Eingabedaten aus einer Datei lesen,
 - die Routine iterativ mit einer bestimmten Menge von Eingabedatensätzen testen.
- **Dummydatei:** Eine Dummydatei (*dummy file*) ist eine kleine Version einer realen Datei, welche dieselbe Struktur und Komponenten wie die reale Datei aufweist, nur eben eine kleine Menge ausgewählter Daten. Dies bringt folgende Vorteile:
 - Durch die geringe Datenmenge ist der Inhalt genau bekannt und besser verfolgbar.
 - Es besteht eine größere Chance, dass die Datei fehlerfrei aufgebaut ist.

Wichtig ist es, die Komponenten des Testrahmens auch nach dem Zusammenbau des Codes aufzuheben, um bei einer erneuten Notwendigkeit der Verwendung eines Testrahmens darauf zugreifen zu können. Teilweise kann der Testrahmen im fertigen Code enthalten bleiben und durch geeignete Makros, Präprozessor-Befehle oder Kommentare einfach ausgeblendet werden. Eine weitere Alternative ist die Erstellung von zwei Varianten des Codes im Rahmen der Versionsverwaltung, einer mit Testrahmen und einer ohne.

12.1.3.6 Testgesteuerte Entwicklung

Die *testgesteuerte Entwicklung* (auch *testgetriebene Entwicklung* genannt; im Englischen *test first development* bzw. *test-driven development*) ist eine Softwareentwicklungsmethode, bei der immer der Test vor dem entsprechenden Teil der Anwendung erstellt (= programmiert) wird. Die Testfallerstellung ist dabei eine zweistufige:

1. Der Kunde (Auftraggeber) erstellt *Systemtests* für die Anwendungsfälle (unterstützt durch den Auftragnehmer i. e. Entwickler).
2. Auf Basis dieser Systemtests erstellt der Entwickler einer zugehörigen Komponente entsprechende *Komponententests*.

Gerade in der iterativ-inkrementellen Entwicklung dienen die so erstellten Testsuites oft als Ersatz für eine genaue Produktspezifikation. Zur automatisierten Ausführung der Testsuites werden Testrahmensysteme eingesetzt, z. B. *xUnit* für Komponententests (vgl. Abschnitt 12.2.4.4), oder *FITness* für Systemtests (vgl. Abschnitt 13.4). Dabei wird immer zuerst ein (Komponenten-)Testfall erstellt, der mit dem aus dem vorhandenen Code erzeugten Programm einen Fehler bewirkt. Anschließend wird der Code um gerade so viel erweitert, dass der Testfall keinen Fehler mehr bewirkt. Dies wird solange mit neu hinzugefügten Testfällen wiederholt, bis keine weiteren Testfälle mehr erzeugt werden können (aus Zeitgründen, weil alle Ideen für Tests erschöpft sind etc.). Falls der Code durch die stark inkrementelle Erstellung unsauber ist, wird er anschließend restrukturiert (vgl. Abschnitt 12.1.3.4).

Dieses Erstellen des Tests vor dem Code führt für den Entwickler, der den Test ja selbst erstellt, zu besserem Verständnis, wie die zu entwickelnde Komponente verwendet wird bzw. warum sie überhaupt erstellt wird. Testgesteuerte Entwicklung beeinflusst daher viel mehr die Art und Weise der Entwicklung (Design, Implementierung) einer Komponente als dass sie hilft, geeignete Testfälle zu finden. Testgesteuerte Entwicklung ist daher primär KEINE Technik des Testens, sondern eine Technik der Entwicklung!

Wie in [Ambler, 2002] festgestellt wird, ist es – ähnlich wie für eine gute Code-Restrukturierung – für eine gute testgesteuerte Entwicklung sehr empfehlenswert, zuerst ein „Big-Picture“-Design der zu implementierenden Funktionalität zu erstellen. Die detaillierten Designaktivitäten können dann im Rahmen der testgesteuerten Implementierung durchgeführt werden. Lösungsideen werden auf diese Weise rasch auf ihre Umsetzbarkeit und Tragfähigkeit überprüft [Pichler, 2004].

Die Hauptvorteile der testgesteuerten Entwicklung sind:

- Die existierenden Testfälle ermöglichen es, Verbesserungsarbeiten am Code vorzunehmen, (meist) ohne dabei unbemerkt Fehler in den Code einzubringen.

- Für jede Komponente existiert eine Sammlung zugehöriger Komponententests, welche die Komponente einzeln testen können. Änderungen an der Komponente können schnell und unmittelbar überprüfbar durchgeführt werden.
- Durch das laufende, automatische Testen nach dem Hinzufügen eines immer nur sehr kleinen Codestückes ist es kaum mehr notwendig, Programme im Debugger durchzugehen.
- Etwaige Fehler werden sehr rasch lokalisiert, und ihre Behebung kann durch die Komponententests unmittelbar überprüft werden. Auch können eventuelle Folgefehler der Behebung sofort erkannt werden.

Entscheidend dabei ist, immer nur die kleinstmögliche Menge Code hinzuzufügen, bevor die Testsuite wieder ausgeführt wird. [Westphal, 2005] merkt dazu an: „Wenn Sie mehr Code schreiben, als Ihr Testfall in Wirklichkeit verlangt, schreiben Sie [...] zu wenige Tests. Sie schreiben dann Code, der unter Umständen von keinem Test [ab]gesichert wird.“ Umgekehrt können aber nie alle erdenklichen Testfälle in eine Testsuite aufgenommen werden, da der Aufwand sonst ins Unermessliche steigt.

Folgende Probleme löst die testgesteuerte Entwicklung nicht:

- Bei der Auswahl von Testfällen wird keinerlei Unterstützung geboten. Diese erfolgen daher vielfach intuitiv, was zu groben Fehleinschätzungen der Produktqualität eines Codes führen kann (vgl. dazu die amüsante Geschichte über „Die trügerische Sicherheit des grünen Balkens in JUnit“ in [Fraikin *et al.*, 2004]). Die Kombination der testgesteuerten Entwicklung mit solidem Wissen über Methoden der Testfallerstellung ist daher unumgänglich [Fraikin *et al.*, 2004].
- Die Qualität einer Software hängt wesentlich von der Qualität der Testfälle ab. Deren Qualität im Hinblick auf die Anforderungen des Auftraggebers zu beurteilen ist dem Entwickler vielfach unmöglich.
- Neue Anforderungen wirken sich oft auf bereits bestehende Testfälle aus. Dieser Zusammenhang ist meist nur sehr schwer zu erkennen. Das Ergebnis ist die trügerische Annahme, das Programm laufe korrekt, weil kein Testfall einen Fehler meldet. In Wirklichkeit sind jedoch die Testfälle veraltet. Selbst für den Fall, dass erkannt wird, dass Testfälle geändert werden müssen, ist es sehr schwierig festzustellen, welche zu ändern sind und wie dies systematisch durchzuführen ist. Man müsste „Testfälle für die Testfälle“ erstellen, was zu einem pyramidenartigen Ansteigen des Aufwands für die Testfallerstellung (und auch für die Ausführung) führt.¹
- Selbst der ausgiebigste Test einzelner Bausteine garantiert kein fehlerfreies Gesamtsystem. Integrations- und Systemtest dürfen daher nicht vernachlässigt werden [Fraikin *et al.*, 2004].

¹Auf diesen sehr kritischen Umstand wird in kaum einer Publikation im Dunstkreis des „agilen Hypes“ hingewiesen. Es wird im Gegenteil absolute Sicherheit versprochen (z. B. „Once you get a test working, you know it is working now and forever.“), was in den Augen des Autors manche der Protagonisten der agilen Vorgehensmethoden in die Nähe von Marktschreibern rückt.

12.1.3.7 Kontinuierliche Integration

Die Technik der *kontinuierlichen Integration* (*continuous integration*) hält Entwickler dazu an, ihren Code nach jeder kleinen (erfolgreichen) Änderung in die gemeinsame Codebasis einzupflegen. Auf diese Weise ist „laufend“ der letzte Stand der Entwicklung für alle Entwickler zugänglich. Dieses Vorgehen wird z. B. von Microsoft seit langem angewendet und wurde ursprünglich als *tägliche Integration* (*daily build*) bezeichnet. Mittlerweile wurde erkannt, dass eine Integration nach jedem Implementierungsschritt, der eine theoretisch lauffähige Version produziert, sinnvoll ist.² Agile Vorgehen sprechen daher von einer „Integration mindestens einmal pro Tag, am besten mehrmals“ [Highsmith, 2002].

Wichtig ist es, nur Versionen einzupflegen, die vom Implementierer getestet wurden und so weit lauffähig sind, dass andere Entwickler konstruktiv damit arbeiten können, um ihren eigenen Entwicklungsfortschritt einzupflegen.

Folgende Voraussetzungen müssen erfüllt sein, um eine effiziente kontinuierliche Integration zu ermöglichen [Fowler und Foemmel, 2005]:

- Es muss *eine* gemeinsame Codebasis an einer für alle Entwickler laufend zugänglichen Stelle gehalten werden.
- Das Übersetzen und Binden des Programms mit der letzten Version aller Dateien muss einfach und vollautomatisch („auf Knopfdruck“) möglich sein.
- Das entwicklerseitige Testen auf Funktionsfähigkeit muss möglichst hoch automatisiert sein (vgl. Abschnitt 12.2.4.4).
- Die neueste lauffähige Programmversion muss für jeden Entwickler automatisch zur Verfügung gestellt werden können.

In der Praxis ist der Einsatz von Werkzeugen für die kontinuierliche Integration unumgänglich: Die Versionen der Programmkomponenten können nur in einem Konfigurationsverwaltungssystem übersichtlich administriert werden. Das Übersetzen und Binden erfolgt entweder in einer integrierten Programmierumgebung oder via spezieller Bindewerkzeuge (im Open-Source-Bereich etwa *Ant*, <http://ant.apache.org>, oder *CruiseControl*, <http://cruisecontrol.sourceforge.net>).

Zu beachten ist, dass eine kontinuierliche Integration, gerade bei sehr kurzen Iterationszyklen einer iterativ-inkrementellen Entwicklung, die Produktivität auch hemmen kann. Es ist nämlich nicht nur die Software laufend aktuell zu halten, sondern auch die am Ende des Zyklus mitzuliefernden Dokumente, die Online-Hilfe etc.

12.1.4 Werkzeuge

Entwicklungsumgebungen für Software (manchmal auch als *Programmierungsumgebungen* bezeichnet) vereinigen wesentliche Codierwerkzeuge unter einer einheitlichen Benutzeroberfläche. Teilweise sind auch Werkzeuge für Debuggen und andere Phasen (Analyse, Design,

²Dies wird auch dadurch ermöglicht, dass die Rechnerleistung mittlerweile so angestiegen ist, dass die Zeit für das Übersetzen und Binden eines Programms in den meisten Fällen vernachlässigbar kurz ist, zumindest bei einem inkrementellen Build.

Testen etc.) enthalten. Entwicklungsumgebungen können darüber hinaus eine Vielzahl kleinerer, nützlicher Werkzeuge enthalten.

Editoren sind Systeme zur (Quell-)Texterstellung, die günstigenfalls neben Standardfunktionen zur Texterfassung das eigentliche Schreiben von Programmen besser unterstützen [Pomberger und Blaschek, 1996]: Mittels *Makrofunktionen* können wiederkehrende Komponenten oder Gerüste für ganze Komponenten in den Quellcode eingefügt werden. Strukturorientierte Editoren zeigen ein Programm in einer einheitlichen (konfigurierbaren) Form an und erlauben eine Speicherung in dieser Form, *syntaxgesteuerte Editoren* besitzen Information über die verwendete Programmiersprache und verhindern eine unzulässige Syntax bereits bei der Eingabe.

Für wiederkehrende, gleichartige Aufgaben eignen sich auch *Programmgeneratoren*, z. B. Rahmengeneratoren, die leere Objekte oder Programme mit einheitlicher Dokument- und Kommentarstruktur erzeugen. Andere Generatoren, z. B. Benutzerschnittstellen-Generatoren (user interface builder) oder Compiler-Compiler, erstellen aus einer grafischen oder formalen Beschreibung ganze Codeteile automatisch.

Gerade moderne Programmierstile (objektorientierte oder verteilte, agentenorientierte Programmierweise) zeichnen sich durch eine komplexe Zusammenhangstruktur und oft durch einen großen Gesamtcodeumfang aus. Da in diesem Fall die Dateistruktur immer mehr in den Hintergrund tritt, haben sich zur Orientierung im Quellcode so genannte *Browser* („Schmökterer“) bewährt, mit denen die Programmstruktur in grafischer Form oder als Text angezeigt und zu einzelnen Komponenten verzweigt werden kann.

Compiler übersetzen den Quellcode in ausführbaren Programmcode („Maschinensprache“). Ein guter Compiler muss zuverlässig und rasch effizienten Code erzeugen. Im Fall von Compilerfehlern muss ein guter Compiler möglichst genaue Information über die Fehlerursache liefern und die Fehlerstelle(n) markieren. Nicht nur zur Übersetzungszeit erkennbare Syntaxfehler sollen jedoch erkannt werden, auch für Laufzeitfehler soll vorgesorgt werden: Im Bedarfsfall sollen Überschreitungen von Wertebereichen, unzulässige Index- und Adresswerte oder Divisionen durch Null und dergleichen überprüft werden können.

Sinnvoll – und ein Hauptvorteil von Entwicklungsumgebungen – ist eine Kopplung von Editor und Compiler zur Anzeige der vom Compiler ermittelten Programmierfehler direkt im editierten Quellcode, sodass der Fehler ohne umständliche Suche behoben werden kann.

Linker dienen zur Erstellung ausführbarer Programmsysteme aus einzeln übersetzten Programmmodulen. Teilweise sind Linker Bestandteil des Compilers. Manchmal findet man aber auch getrennte Linker vor, die dann im Allgemeinen Module verschiedener Programmiersprachen zu einem Programmsystem zusammenbinden können. Das vom Übersetzen getrennte Binden („Linken“) hat den großen Vorteil, dass Modul für Modul inkrementell entwickelt und übersetzt werden kann und so große Programmsysteme effizient erstellt werden können. Außerdem werden lange Übersetzungszeiten zur Übersetzung des gesamten Quellcodes auf einmal vermieden. Ein aus mehreren oder vielen Modulen bestehender Quellcode besitzt jedoch vielfach eine komplexe Abhängigkeitsstruktur zwischen den Modulen. Wenn sich die Schnittstelle eines Moduls ändert, müssen alle Module, die diese Schnittstelle benutzen, neu übersetzt werden. Die kontrollierte Neuübersetzung eines Multimodulsystems wird in allen modernen Entwicklungsumgebungen automatisiert durchgeführt.

12.1.5 Ergebnisse

12.1.5.1 Quellcode

Die Notwendigkeit, die Gestaltung von *Quellcode* (*source code*) bestimmten Regeln zu unterwerfen, ergibt sich aus folgenden Grundkriterien moderner Softwareentwicklung:

- Lesbarkeit von Code,
- Wartbarkeit von Code (für den Autor selbst sowie für andere),
- Integrierbarkeit von Code verschiedener Autoren,
- Wiederverwendbarkeit von Code.

Daraus ergibt sich, dass sowohl die Gestaltung des Programmcodes als auch die der Kommentierung einer einheitlichen Struktur unterliegen müssen. Da die Leistung eines Arbeitsteams wichtiger ist als eine herausragende Einzelleistung, ist auch ein einheitlicher Programmierstil im Team wichtiger als eine individuelle ästhetische Glanzleistung. Formatierwerkzeuge („Prettyprinter“) können verwendet werden, um Programmen ein einheitliches Aussehen zu verleihen, das vielfach durch Parameter gesteuert werden kann, vgl. z. B. [Blaschek und Sametinger, 1989].

Bei der Kommentierung muss darauf geachtet werden, dass die Kommentierung (größtenteils) das Design widerspiegelt, die Codierung die Implementierung. Gute Kommentare begründen, WARUM etwas getan wird. Dies erklärt auch die unterschiedliche Schwerpunktsetzung von Codierstil (z. B. sprechende Namen) und Kommentierstil (Zusammenhänge des Designs). Das eine kann nicht das andere ersetzen!

Vielfach ist es nicht sinnvoll, den Quellcode in gedruckter Form an den Kunden auszuliefern. Auf jeden Fall ist auch bei Auslieferung in elektronischer Form ein Verzeichnis aller Dateien zu erstellen sowie ein Modulbaum mit Zusammenhangstruktur, um z. B. eine sinnvolle Lesereihenfolge vorzugeben. Verwendete Komponenten (Klassenbibliotheken, Systemrahmen) sind explizit zu referenzieren. Sofern eine Dokumentation zu diesen Komponenten nicht explizit verfügbar ist und beigelegt wird, ist anzugeben, wie die Dokumentation bzw. der Quellcode (bzw. notwendige Teile davon) verfügbar gemacht werden kann.

Tipp: Es sei hier nochmals explizit darauf hingewiesen, dass es für den Projektabschluss und vor allem für die Nutzung und Wartung des Softwaresystems von entscheidender Bedeutung ist, rechtzeitig festzulegen, ob der Quellcode an den Kunden ausgeliefert wird. Im Falle der Auslieferung des Quellcodes ist genau zu spezifizieren, inwieweit der Kunde Änderungen am Quellcode (und seien sie auch noch so klein) vornehmen darf und ob diese Änderungen Einfluss auf die Quellcodewartung haben. Vielfach werden Fehler nach solchen Quellcodeänderungen von Auftragnehmer und Auftraggeber gegenseitig jeweils dem Partner angelastet und können dazu führen, dass das System unwartbar und damit unbrauchbar wird.

12.1.5.2 Systemdokumentation

Die *Systemdokumentation* beschreibt den Aufbau des Softwaresystems sowie die dazu verwendeten einzelnen Komponenten. Der Zweck der Systemdokumentation ist daher

- die Erklärung der Implementierung,
- die Dokumentation des Entwicklungs-Know-hows zur Weitergabe an andere Entwickler bzw. Wartungsmitarbeiter,
- die Schaffung einer Grundlage für die Wartung während des Betriebs,
- die Unterstützung der Kommunikation zwischen den Softwareingenieuren bereits während der Entwicklung.

Die Systemdokumentation umfasst daher folgende Komponenten:

- **Beschreibung der Systemstruktur:** gewählte Zerlegung und die sich daraus ergebenden Strukturen und Schnittstellen; je nach Programmierparadigma Programmablaufplan, Aufrufhierarchie, Komponentenhierarchie bezüglich Taxonomie (Klassenbaum) und Aggregation,
- **Beschreibung des dynamischen Verhaltens:** Systemverhalten, Interaktionsmöglichkeiten, Systemausgaben (teilweise Verweis auf Details in der Benutzerdokumentation),
- **Beschreibung der Komponenten:** je nach Programmierparadigma Funktionenliste, zentrale Algorithmen, Datenmodell, Klassenbeschreibung (teilweise Verweis auf den Quellcode),
- **Beschreibung der verwendeten externen Datenhaltung:** bei Verwendung externer Dateien Beschreibung der Dateien, bei Verwendung einer Datenbank Beschreibung des Datenmodells,
- **Beschreibung der zentralen Begriffe:** Datenlexikon,
- **Beschreibung der zur Installation notwendigen Schritte:** Installationshandbuch, Ergänzung der Benutzerdokumentation um Fehlerbeschreibungen, Troubleshooting etc.

Eine gute Dokumentation

- verbessert die Brauchbarkeit,
- erhöht die Zuverlässigkeit (auch bei einem schlechten System reduziert eine genaue Verwendungsbeschreibung die Anzahl der Benutzerfehler),
- verbessert die Installierbarkeit,
- erhöht die Wartbarkeit

eines Softwareprodukts. Wie in [Kaner *et al.*, 1999] angemerkt wird, reduziert eine sorgfältige Dokumentation auch die rechtliche Verantwortung der Auftragnehmer (z. B. bezüglich Benutzerfehler, unerwarteten Systemverhaltens).

Wichtig ist, dass in der Systemdokumentation nicht nur die Resultate dokumentiert werden, sondern auch der Entstehungsprozess und damit die Gründe (WARUM) für die Realisierung

in der dokumentierten Form. Bedingt durch die Komplexität moderner Softwaresysteme eignet sich sequenzieller Text immer weniger zur Dokumentation. Eine moderne Systemdokumentation kann z. B. adäquat als Hypertext in Verbindung mit automatisch extrahierter Information aus dem Quellcode erstellt werden, vgl. z. B. *doc++* (<http://docpp.sourceforge.net>) oder *javadoc* (<http://java.sun.com/products/jdk/javadoc>).

Tipp: Dem Autor dieses Buchs erscheint es nicht sinnvoll, das Pflichtenheft als „... Bestandteil der Systemdokumentation ... für die Systemdokumentation von der Vertragsform in die Form einer Aufgabenstellung umzuarbeiten“ [Pomberger und Blaschek, 1996]. Das Pflichtenheft ist ein verbindliches Dokument, dessen Umarbeitung durchaus zu rechtlichen Problemen und ggf. Streitigkeiten mit dem Auftraggeber führen kann. Auf das Pflichtenheft soll daher in der Systemdokumentation verwiesen werden; es einzuarbeiten bringt keinerlei Vorteile.

Natürlich ist die Systemdokumentation mit dem Ende des Codierens noch nicht vollständig und korrekt und damit nicht abgeschlossen. Die Systemdokumentation soll daher mit einer Versionsnummer versehen werden und immer die beschriebene Version der Software (Versionsnummer, Erzeugungsdatum) enthalten. Die Systemdokumentation ist somit laufend zu adaptieren. Nach Fertigstellung des Produkts sind die Systemdokumentation und die übrigen in den Phasen entstandenen Dokumente nochmals zu überprüfen, gegebenenfalls zu ergänzen bzw. zu korrigieren sowie stilistisch und im Layout zu vereinheitlichen.

Tipp: Wenn die Systemdokumentation bereits während der Codierung erstellt wird (und nicht, wie leider vielfach üblich, zum letztmöglichen Zeitpunkt vor dem Projektende), können Projektmitarbeiter, die Information über die Implementierung benötigen, als „Kontrollleser“ bewirken, dass die Systemdokumentation nicht nur für den Autor verständlich und brauchbar ist, sondern auch für andere Entwickler und den Auftraggeber.

12.2 Debuggen

12.2.1 Ziel

Debuggen ist der Prozess des Findens der Ursache eines Fehlers oder eines Mangels (z. B. Programmierunsauberkeit, zu große Komplexität) in einem Programm inklusive entsprechender Behebung. Im Allgemeinen wird daher das Debuggen eines Programms vom Ersteller des Codes selbst durchgeführt.

Wie in [Kaner *et al.*, 1999] angeführt, ist „das Beheben von Fehlern viel billiger, wenn Programmierer ihre eigenen Fehler finden. Es gibt keine Kommunikationskosten. Sie müssen einen Fehler niemandem Anderen erklären. Sie müssen ihn nicht in eine Fehlerverfolgungsdatenbank eintragen. Tester und Projektleiter müssen den Fehlerstatus und die Behebung des Fehlers nicht überwachen ... , und schlussendlich blockiert oder stört der Fehler nicht die Arbeit eines anderen Projektmitarbeiters.“ Debuggen wird also von der Phase des Testens dadurch unterschieden, dass Letzteres von einer Person durchgeführt wird, die nicht mit dem Ersteller des Codes identisch ist.

12.2.2 Aufgaben

Die Aufgaben des Debuggens umfassen:

1. **Stabilisieren** des Fehlers/Mangels (Sammeln von Daten und Wissen durch wiederholbare Experimente),
2. **Lokalisieren** der Fehler-/Mangelursache,
3. **Ausarbeiten** der Korrektur (inkl. Überlegen der Konsequenzen),
4. **Durchführen** und Dokumentieren der Korrektur,
5. **Testen** der Korrektur,
6. **Prüfen**, ob weitere ähnliche Fehler/Mängel enthalten sind.

Daraus ergibt sich eine weitere Abgrenzung zum Testen: Bei diesem werden Fehler nur gefunden und nicht behoben (im Allgemeinen steht auch der Quellcode gar nicht zur Verfügung). Oftmals wird der Fehler nicht einmal genau lokalisiert.

Tipps zum effizienten Finden und Beheben von Fehlern

Die folgenden Tipps stammen teilweise aus [Stacey, 1995], teilweise aus Erfahrungen des Autors. „Fehler“ ist dabei als Oberbegriff für Fehler und Mangel zu verstehen.

- Verfeinere und diversifiziere den Testfall, der den Fehler hervorrief. Den Fehler auf verschiedene Arten zu reproduzieren hilft, die Ursache für den Fehler herauszufinden.
- Anstatt zu lange zu grübeln, frage Kollegen. Andere Personen können neue Perspektiven und Erfahrungen in die Fehlerfindung einbringen.
- Schränke den Bereich ein, in dem der Fehler liegen muss. Eliminiere systematisch Codeteile, die den Fehler nicht enthalten.
- Sei besonders achtsam bei Codeteilen, in denen bereits Fehler gefunden wurden. Prüfe besonders fehlerbehaftete Routinen regelmäßig (führe dazu zumindest für jedes Modul einen Fehlerzähler ein).
- Prüfe zuerst den Code, der kürzlich erstellt oder geändert wurde.
- Erweitere den verdächtigen Codebereich. Schau die Umgebung an, auch wenn du sicher bist, dass der Fehler genau in einem bestimmten Bereich liegen muss, aber noch nicht entdeckt werden konnte.
- Entwickle den Code Stück für Stück und teste dazwischen.
- Setze ein Zeitlimit für „quick & dirty“ Debugging. Wenn der Fehler in dieser Zeit nicht gefunden werden kann, so ist er als schwierig einzustufen und benötigt systematisches Vorgehen.

- Prüfe auf Standardfehler. Verwende Standardfehlerlisten, stelle aber sukzessiv deine individuelle Fehler-„Hitliste“ zusammen.
- Präsentiere das Problem einem Kollegen. Das Präsentieren des Codes bewirkt ein organisiertes Durchdenken und kann alleine dadurch zur Lösungsfindung beitragen.
- Vertraue den Informationen des Compilers (Fehlerart, Fehlerursache) nie zu 100 %. Achte besonders auf Informationen bei Folgefehlern.
- Prüfe Fehlerbehebungen sehr sorgfältig. Die Chance, bei der Behebung einen neuen Fehler einzubauen, ist größer als 50 %.
- Behebe das Problem, nicht das Symptom, auch wenn die Zeit drängt. Dazu ist es notwendig, das gesamte Fehlerproblem zu verstehen und nicht nur die lokale Ursache. Eine einzelne Symptombeseitigung kann die Fehlerfindung zusätzlich erschweren.
- Verstehe das Programm und nicht nur das Problem. Wenn das Programm sehr umfangreich ist, soll zumindest die Nachbarschaft des Problems (statisch, dynamisch) durchgesehen werden.
- Beseitige immer nur einen Fehler auf einmal.

Fehler als Chancen zur Leistungssteigerung

Fehler sollten nicht nur als unangenehm und frustrierend empfunden werden. Sowohl durch die Art der Fehler als auch den Prozess der Beseitigung kann man sehr viel zur Steigerung der eigenen Programmierleistung lernen [Stacey, 1995]:

- Versuche, die Fehlerursache zu ermitteln: Ist der Fehler in Analyse, Design, Codierung, beim Debuggen oder beim Testen eingeführt worden? Daraus lässt sich ein besseres Verständnis für das Projekt und den Löseprozess gewinnen.
- Sammle deine Fehler, klassifiziere sie und versuche, gerade deine häufigsten Fehler vorrangig auszumerzen. Bei der persönlichen Fehlersuche kannst du diese Liste zur Ideenfindung benutzen.
- Erkenne, wie du Fehler findest. Ermittle, wie weit das Wissen über deine Debugstrategie dir bei zukünftigen, ähnlichen Problemen hilft, Zeit einzusparen.
- Verstehe, wie du Fehler behebst. Versuche, deine Fehlerbehebung zu systematisieren (persönlicher Fehlerbehebungsprozess).

12.2.3 Techniken

12.2.3.1 Statische Programmanalyse

Aufgabe der *statischen Programmanalyse* ist die Fehlersuche ohne direkte Ausführung des Programms. Methoden dazu befassen sich mit syntaktischen und semantischen Analysen des Programms. Das ultimative Ziel der statischen Analyse, die Programmverifikation, ist bislang nur für akademische Beispiele durchführbar. Die wesentlichen derzeit verfügbaren Techniken werden im Folgenden erläutert.

Desk Checking

Desk Checking, also das manuelle Durchgehen des Programms am Schreibtisch, wird im Allgemeinen ohne Computer durchgeführt und umfasst das sorgfältige Lesen und Analysieren des Codes. Dies geschieht durch den Programmator oder durch eine andere Person, oft anhand konkreter Beispiele. In der Praxis ist es günstig, das Programm auch am Computer lauffähig zur Verfügung zu haben, um Testläufe zu machen, wenn der Programmablauf an bestimmten Stellen durch das Lesen nicht klar ist. Manchmal wird auch ein Compiler verwendet, um Syntaxfehler oder ungültige Anweisungen zu finden. Auf ähnliche Weise kann ein Linker verwendet werden, um Komponenten zu entdecken, die nicht referenziert werden oder zwar referenziert werden, aber nicht definiert wurden.

Im Vergleich zu technischen Reviews (siehe dazu den folgenden Abschnitt) wird beim Desk Checking ein größerer Umfang an Code durchgearbeitet und dementsprechend normalerweise auch weitaus mehr Zeit aufgewendet. Einer der wichtigsten Punkte, die betrachtet werden, ist, ob der Code für den Leser Sinn macht. Wenn der Code verwirrend ist, ist vermutlich auch das der Codierung zu Grunde liegende Design nicht klar genug. Wenn der Code klar ist und zusätzlich die betrachteten Beispiele einen plausiblen Programmablauf vermuten lassen, hat der getestete Code das Desk Checking bestanden.

Technische Reviews

Technische Reviews sind Präsentationen von Code in einer Gruppe („Review-Team“), um Fehler aufzuspüren. Sie stellen eine Spezialform der im gesamten Projekt gebräuchlichen *Reviews* (Abschnitt 7.1.3) dar. Die Korrektur von Fehlern ist *nicht* Aufgabe dieser Treffen!

Review-Teams sollen klein sein (in [Kaner *et al.*, 1999] wird eine Richtzahl von 7 Personen angegeben). Sie umfassen auch Personen, die nicht an Design und Implementierung beteiligt waren. Die Teammitglieder arbeiten die zur Verfügung gestellten Dokumente vorher durch und stellen alle auftauchenden Fragen während des Treffens. Folgende Typen von technischen Reviews sind gebräuchlich:

- **Walkthrough:** Bei dieser Technik simuliert der Implementierer das Programm. Er zeigt Schritt für Schritt, was das Programm mit den Testdaten macht, die vom Review-Team geliefert werden. Diese Simulation zeigt, wie die Komponenten des Programms zusammenspielen, und kann wegen der Notwendigkeit der Erklärung durch den Programmierer zu besserem Verständnis führen als der Code allein. Redundante oder umständlich codierte (aber auch zu sehr „optimierte“) Programmteile werden dadurch identifiziert; viele vergessene Details werden aufgezeigt.
- **Codeinspektion:** Bei dieser Technik prüfen die Teammitglieder jede Codezeile gegen vorher erstellte Checklisten. Diese Prüfung kann sich auf verschiedene Aspekte konzentrieren, z. B. auf Einhalten der Programmierrichtlinien, Fehlerbehandlung, Inklusion von Prüfbedingungen oder andere, genau festgelegte Teilbereiche. Mehrere Inspektionen, die je einen Aspekt behandeln, sind dabei einer umfangreichen Inspektion vorzuziehen.
- **Codereview:** Bei dieser Technik analysieren die Teammitglieder den Code vorab und bringen eine Liste mit Anmerkungen zum Treffen mit. Während des Treffens erklären

sie ihre Anmerkungen und Einwände und weisen speziell auf Codebereiche hin, die für sie verwirrend oder unklar sind. Das Ziel des Codereviews ist die Erstellung einer gemeinsamen Fehlerliste, wobei sichergestellt werden muss, dass jeder Fehler vom Codierer verstanden wird. Das Beheben der Fehler ist Aufgabe des Codierers, jedoch nicht während des Treffens!

Vorteile von technischen Reviews sind die gründlichere Prüfung eines Programms dadurch, dass bei einem noch nicht lauffähigen Programm mehr Bereitschaft für eine durchgehende Analyse vorhanden ist (“First do reviews, then compile!”), sowie die Erklärung des Programms für andere Personen als den Autor (jeder lernt beim Lesen fremder Programme).

Technische Reviews für Code sollen gemacht werden, wenn der Code produziert wird. Dies ermöglicht dem Review-Team, Programmierrichtlinien und andere Standards von Beginn der Implementierung an einzuführen und deren Einhaltung zu überprüfen. Viele Probleme können so bereits vor dem Debuggen und vor allem vor dem – aus Projektsicht für die Fehlerbehebung teuren – Testen erkannt und behoben werden. In [Humphrey, 2005] wird angegeben, dass bis zu 80 % aller logischen Fehler auf diese Weise bereits vor dem ersten Debug- oder Testlauf gefunden werden können. In der gleichen Quelle wird angegeben, dass bei technischen Reviews durchschnittlich ca. 150 Codezeilen pro Stunde analysiert werden können.

Komplexitätsanalyse

Aufgabe der *Komplexitätsanalyse* ist die Bewertung der Codestücke eines Programms mittels Komplexitätszahlen. Dem liegt die Annahme zu Grunde, dass in komplexen Codeteilen die Fehlerwahrscheinlichkeit am höchsten ist [Kaner *et al.*, 1999]. Aus diesen Zahlen werden im Allgemeinen Maße für die Programmqualität sowie die Produktivität abgeleitet. Diese stellen die für die Projektentwicklung eigentlich relevanten Maßzahlen dar.

Folgende Komplexitätszahlen für Code sind verbreitet anzutreffen:

- **Lines of Code (LOC):** Das Ermitteln der Anzahl der Codezeilen (LOC, manchmal auch SLOC – „source lines of code“ bzw. KLOC – „kilo-lines of code“) ist vermutlich die einfachste und daher auch die am weitesten verbreitete Komplexitätszahl. Ihre Sinnhaftigkeit wird – wie übrigens auch die der übrigen Zahlen – in der Literatur heftig diskutiert (vgl. z.B. [Goldberg und Rubin, 1995]). Einigkeit besteht darüber, dass Kosten- und Produktivitätswerte, die von LOC abgeleitet werden, für die jeweilige Programmiersprache und die verwendete Entwicklungsumgebung kalibriert werden müssen. Im längerfristigen Durchschnitt produziert jedoch jeder Programmierer, unabhängig von Sprache und Umgebung, in der Größenordnung von ca. 10 LOC pro Arbeitstag, ein Wert, der vorab vielfach als unglaublich niedrig angesehen wird. Viele aber, die eine wesentlich höhere Zahl von LOC pro Arbeitstag produzieren, erstellen Code, der den Test- und Wartungsaufwand derart erhöht, dass die vermeintliche Produktivitätssteigerung wieder aufgehoben wird. Für eine umfangreiche Aufarbeitung und Anwendung dieser Maßzahl sei auf [Humphrey, 2005] verwiesen.
- **Software Sciences:** In [Halstead, 1977] wird aus der Anzahl der verwendeten Operanden, der Anzahl der verwendeten Operatoren sowie der Summe der Anzahl aller

Benutzungen der Operanden und Operatoren die „innere Komplexität“ eines Codes berechnet, die zur Feststellung eines „Programmvolumens“ und der „Programmschwierigkeit“ herangezogen werden. Trotz der langjährigen Verfügbarkeit konnte dieses Maß bisher nicht seine praktische Bedeutung unter Beweis stellen.

- **Cyclomatic Complexity:** In [McCabe, 1976] wird ein Maß vorgestellt, das die Verzweigkeit des Codes misst und daraus die Komplexität ableitet. In [Kearney *et al.*, 1986] wird dargestellt, dass dieses Komplexitätsmaß, wie auch jenes von Halstead, ohne jegliche theoretische Basis ist und vor allem bei der objektorientierten Programmierung keinesfalls adäquat ist.
- **Andere Maße:** Eine Vielzahl weiterer Komplexitätszahlen (und Qualitätszahlen) lassen sich in der Literatur finden. Unter anderem werden Distanzen von Datenreferenzierungen, Kommentardichte, durchschnittliche Länge der Variablennamen, Schachtelungstiefe sowie die Anzahl von Funktionen, Klassen, Friends pro Klasse, Inline-Definitionen und der Umfang der Schnittstellen genannt.

In [Goldberg und Rubin, 1995] wird zu diesen Maßen angemerkt, dass speziell bei der objektorientierten Programmierung die definierten Maße auf keinem realistischen Modell beruhen und ihre Definition eher davon geleitet worden sein dürfte, was leicht zu messen ist, und nicht davon, was Sinn ergibt.

Grundsätzlich ist anzumerken, dass in einer Ingenieursdisziplin das Messen des Projektfortschritts unverzichtbar ist und dass gilt, eine schlechte Messung sei besser als gar keine. Wie in [Humphrey, 2005] angemerkt wird, soll jede Maßzahl mittels statistischer Methoden auf ihre Korrelation mit den tatsächlichen Ergebnissen überprüft werden. Wegen des Aufwandes und der Wiederholbarkeit sollen Maßzahlen durch Werkzeuge automatisch ermittelbar sein.

Tipp: Bewährt hat sich eine Festlegung von vier Werten für jede Maßzahl (vgl. dazu [Goldberg und Rubin, 1995]):

1. „Worst“ beschreibt den schlechtesten, gerade noch akzeptierbaren Wert. Bei einem noch schlechteren Wert ist das Projekt gescheitert.
2. „Plan“ legt den Wert oder das Intervall fest, das zu Projektende angestrebt wird.
3. „Best“ stellt den aktuellen Stand der Technik dar.
4. „Current“ ist ein gemessener Wert aus einem vergleichbaren System oder Projekt, um die übrigen drei Werte besser einschätzen zu können.

Strukturanalyse

Aufgabe der *Strukturanalyse* ist es, Strukturanomalien des analysierten Objekts zu finden. Solche Anomalien sind z.B. Codeteile, die vom Programmanfang aus nicht erreicht werden können („toter Code“), Anweisungen, von denen aus das Programmende nicht erreichbar ist (Endlosschleifen, Deadlocks), Einsprünge in Schleifen oder unerlaubte Kontrollstrukturen.

Datenflussanalyse

Aufgabe der *Datenflussanalyse* ist es, Datenflussanomalien des analysierten Objekts zu finden. Solche Anomalien sind z. B. Objekte, denen vor ihrer Verwendung kein Wert zugewiesen wurde, oder Objekte, die mit Werten belegt, aber nie verwendet werden.

12.2.3.2 Dynamische Programmanalyse

Während bei der statischen Programmanalyse der Code überprüft wird, ohne im Allgemeinen das Programm auszuführen, wird bei der *dynamischen Programmanalyse* der Code ausgeführt und getestet. Der Code selbst wird dabei nicht notwendigerweise überprüft. Die wesentlichen Techniken der dynamischen Programmanalyse werden im Folgenden vorgestellt.

Postmortem-Analyse

Die *Postmortem-Analyse* ist das Auswerten der Ausgaben eines instrumentierten Codes nach einem Testlauf. In vielen Entwicklungsumgebungen können darüber hinaus auch nach einem Testlauf, insbesondere nach einem Absturz, die zuletzt gültige Wertebelegung der Variablen sowie die zuletzt bestehende Aufrufhierarchie analysiert werden.

Singlestepping

Das *Singlestepping* ist das schrittweise Durchgehen eines Programms, im Allgemeinen mit Hilfe eines Debuggers (siehe Abschnitt 12.2.4.5). Dabei kann der beschrittene Pfad in der Kontrollstruktur analysiert werden; vielfach können auch Wertebelegungen von Variablen eingesehen werden. Das Setzen von „Breakpoints“ erlaubt das Ausführen des Programms bis an eine kritische Programmstelle, an der automatisch gestoppt wird. So genannte „Watchpoints“ erlauben das Setzen bedingter Breakpoints, z. B. zur Unterbrechung, wenn der Wert einer bestimmten Variablen nicht mehr innerhalb eines bestimmten Intervalls ist.

Topdown-/Bottomup-Test

Topdown-Test und Bottomup-Test nutzen die im Testrahmenbau (vgl. Abschnitt 12.1.3.5) erstellten Komponenten. Beim *Bottomup-Test* werden dabei Komponenten der untersten Ebene zuerst getestet, indem die für sie erstellten Testtreiber (z. B. mittels *xUnit*, vgl. Abschnitt 12.2.4.4) verwendet werden. Eine analoge Situation besteht bei der Verwendung von Klassenbibliotheken sowie Wiederverwendung von Code. Sind die Basiskomponenten getestet, wird derselbe Prozess auf der nächsthöheren Ebene fortgesetzt.

Beim *Topdown-Test* werden die Komponenten auf höchster Ebene zuerst getestet. Für die Komponenten der nächstniedrigeren Ebene werden deren Stümpfe verwendet. Sobald der Test abgeschlossen ist, wird die nächstniedrigere Ebene in der gleichen Weise getestet. Diese Testsituation taucht auch bei Systemrahmen auf. In der Praxis wird eine Komponente nach ihrer Erstellung getestet. Je nach Bereich und Ansatz geschieht dies manchmal bottomup, manchmal topdown.

Beide Testtechniken sind inkrementelle Tests, da inkrementell Ebene für Ebene getestet wird und sukzessive auf mehr getesteten Code aufgebaut werden kann. Im Gegensatz dazu wird beim „Big-Bang-Test“ das gesamte Programm auf einmal getestet. Diese Technik ist nach moderner Auffassung des Software Engineering abzulehnen.

Whitebox-Test

Wenn der Programmierer während der Codierungsphase sein Programm selbst testet, so kennt er den Quellcode und hat Zugriff darauf. Er testet und analysiert also sein Programm „von innen“. Diese Art von Testen wird daher *Whitebox-Test* genannt. Andere Bezeichnungen dafür sind *Glassbox-Test*, *Strukturtest* und *innerer Test*. Im Gegensatz dazu wird beim Blackbox-Test der Quellcode nicht eingesehen, also als Blackbox betrachtet (siehe dazu Kapitel 13). Da Whitebox-Testen vom Programmierer selbst durchgeführt wird und normalerweise Komponenten während der Implementierung (vor und nach ihrer Integration) getestet werden, wird allgemein Whitebox-Testen der Implementierung zugeordnet (vgl. z. B. [Kaner *et al.*, 1999]).

Da beim Whitebox-Test die Codestruktur bekannt ist, ist ein Hauptziel dieses Testens das Abdecken des gesamten Codes. Als Abdeckungskriterien werden vorrangig verwendet:

- Die **Zeilenabdeckung** hat den einmaligen Durchlauf jeder Codezeile als Ziel. Ein mehrmaliger Durchlauf mit verschiedenen Daten bringt keine Abdeckungsverbesserung.
- Die **Verzweigungsabdeckung** hat den mindestens einmaligen Durchlauf aller Verzweigungsalternativen als Ziel. Dies wird manchmal als „vollständige Abdeckung“ des Codes bezeichnet. In [Beizer, 1995] wird allerdings darauf hingewiesen, dass vollständige Abdeckung nichts mit vollständigem Testen zu tun hat!
- Die **Bedingungsabdeckung** bezweckt den Durchlauf mit allen Alternativen, bei denen eine Bedingung bzw. deren Komponenten wahr bzw. falsch sind.

Das Hauptproblem beim Abdecken des Codes ist, dass man naturgemäß Problembereiche, die im Code nicht berücksichtigt sind, nicht testet.

Die Hauptvorteile des Whitebox-Testens sind:

- Der Programmierer kann das Programm in kleinen Stücken schreiben und sofort testen. Zwischenresultate aus Modulen können für weitere Tests verwendet werden.
- Wenn der Quellcode zur Verfügung steht, können dafür sehr präzise Testfälle geschrieben werden.
- Nur der Programmierer weiß, welcher Codebereich von den bisherigen Tests bereits abgedeckt wurde. Er kann entsprechende zusätzliche Tests entwerfen, die den noch nicht abgedeckten Code testen.
- Der Programmierer kann Testfälle entwerfen, die auf die verwendeten Algorithmen speziell abgestimmt sind.
- Der Programmierer kennt den Kontrollfluss des Programms und kann Testfälle für einen bestimmten Kontrollpfad entwerfen, um spezielle Situationen zu prüfen.

- Der Programmierer weiß, wo welche Daten geändert werden dürfen. Durch Verfolgen von Änderungen an inkorrekten Stellen kann er fehlerhafte Datenänderungen feststellen.
- Rein interne Grenzwerte für Datenwerte, die von außen nicht zugänglich sind, weiß nur der Programmierer, und diese können auch nur von ihm direkt getestet werden.

12.2.3.3 Effizienz der Techniken beim Finden von Fehlern

In [Grady, 1992] werden für die verschiedenen Arten und Bereiche des Testens von Programmen Effizienzwerte angegeben, vgl. Tabelle 12.1, die auf dem Datenmaterial von Hewlett Packard aus den Jahren 1986 bis 1992 beruhen. Die wesentliche – selbst für den Testverantwortlichen bei Hewlett Packard überraschende – Erkenntnis dieser Untersuchungen war die, dass die Effizienz des Whitebox-Testens nicht wesentlich höher als die des Blackbox-Testens ist, Whitebox-Testen aber technischen Reviews bei weitem unterlegen ist. Im resultierenden Umfang ist dies auch nicht mit der Einstellung von Programmierern zu erklären, selbst entdeckte Fehler nach außen hin geheim zu halten. Im angesprochenen Unternehmen wurden nach dieser Erkenntnis technische Reviews beträchtlich ausgedehnt, was zu einer wesentlichen Qualitätssteigerung bei ausgelieferter Software führte [Grady, 1996].

Art des Tests	Effizienz (gefundene Fehler pro Stunde und Person)
Reguläre Verwendung	0,21
Blackbox-Testen	0,28
Whitebox-Testen	0,32
Technisches Review	1,06

Tabelle 12.1: Effizienz der verschiedenen Testarten

12.2.4 Werkzeuge

12.2.4.1 Statische Analysatoren

Statische Analysatoren unterstützen vor allem die Komplexitätsanalyse, indem sie statische Programmmzusammenhänge ermitteln und in geeigneter Form darstellen (Aufrufgraph, Objektbäume etc.). Teilweise sind Algorithmen zur Ermittlung von Komplexitätszahlen bereits inkludiert. In Ansätzen wird in manchen Analysatoren auch die Programmverifikation unterstützt (Generierung von Eingangs-, Ausgangsbedingungen, Invarianten).

12.2.4.2 Listengeneratoren

Zur besseren Übersicht und Analyse von Programmen sind Generatoren gebräuchlich, die aus dem Quellcode verschiedene Listen erzeugen. Beispiele solcher Listen sind:

- **Komponentenlisten** zur Übersicht über Module, Klassen etc. eines Softwaresystems inklusive ihrer Schnittstellen,
- **Aufruflisten** von Prozeduren, Methoden bzw. der Nachrichten-Kommunikation,

- **Kreuzreferenzlisten** zur Darstellung der Querverweise zwischen Komponenten,
- **Programmstrukturpläne** zur Verdeutlichung der Ablaufstruktur.

12.2.4.3 Speicherprüfer

Speicherprüfer (*memory checker*) geben Auskunft über verdächtige Speicherzugriffe und Speicherverwendungen. Einfache Hilfen sind das Anzeigen des jeweils aktuell freien Speicherplatzes bzw. des größten freien Speicherblocks. Eine wesentliche Prüfhilfe stellen *memory leak finders* dar, die das Feststellen falscher Speicheranforderungen und -freigaben ermöglichen. Dadurch können vor allem Programmteile entdeckt werden, die sukzessive Speicher anfordern, ohne ihn wieder freizugeben, und auf diese Weise immer mehr Speicherressourcen verbrauchen.

12.2.4.4 Testrahmensysteme

Testrahmensysteme sind Bibliotheken oder eigenständige Komponenten von Testoperationen (Testaufrufmethoden mit Parameterzuweisung, Testsuite-Ausführmethoden, Ergebnisprüfer etc.). Sie erleichtern und vereinheitlichen das Schreiben von Testtreibern und automatisieren jeden gewünschten Teilschritt nach Bedarf. Testrahmensysteme führen im Allgemeinen zu sauber entworfenen und änderungsfreundlichen Testsuites [Astels, 2003].

Auf Grund der einfachen Möglichkeit der Testautomatisierung unterstützen Testrahmensysteme ideal die *testgesteuerte Entwicklung*. Gemäß dem dabei angewendeten zweistufigen Verfahren (Systemtests durch die Anwender, Komponententests durch die Entwickler, vgl. Abschnitt 12.1.3.6) stehen auch Testrahmensysteme für beide Ebenen zur Verfügung:

- *Testrahmensysteme für Systemtests* sind für das Blackbox-Testen konzipiert. Sie werden in Abschnitt 13.4 behandelt.
- *Testrahmensysteme für Komponententests* verlangen Zugriff auf und Verständnis für den Quellcode des zu testenden Programmteils [Link, 2005] und sind daher Werkzeuge des Debuggens. In diesem Kapitel werden daher nur diese Testrahmensysteme näher betrachtet.

Die Grundidee der testgesteuerten Entwicklung ist die, dass Entwickler für geschriebene Codeteile auch selbst die zugehörigen Testfälle entwerfen (und zwar idealerweise sogar *vor* der Erstellung des Codes). Testrahmensysteme, wie z. B. *JUnit* (<http://junit.org>) als bekanntester Vertreter der Familie der „xUnit“-Testrahmensysteme³, stellen einerseits einen Klassenrahmen zur Verfügung, um darin die Testfälle – in Form von Programmcode – rasch erstellen zu können, andererseits sind sie auch ein vollständiger Testtreiber für die zu testenden Codeteile. Mit JUnit geschieht daher das Erweitern des Testrahmens in zwei einfachen Schritten:

³Ein Überblick über verfügbare Testrahmensysteme für verschiedene Sprachen und Programmierungsumgebungen ist auf http://www.adaptionsoft.com/tddapg_xunit.html angegeben.

1. Jede selbst erstellte Klasse erhält eine Testklasse zugeordnet, die ihr grundsätzliches Verhalten von einer einheitlichen, abstrakten Testklasse erbt. In diese spezifische Testklasse werden alle Komponententests (Methodenname „test...“) als definiertes Eingabe/Ausgabeverhalten programmiert.
2. Eine zentrale Klasse verwaltet die verschiedenen Testsuites (vielfach je eine pro Entwickler). Neue zu testende Klassen werden in diese Testsuite-Klasse eingetragen, die zugehörigen Testmethoden eruiert JUnit automatisch bei Einhaltung der entsprechenden Namenskonvention. Diese Testsuites können einzeln oder kombiniert aufgerufen werden. Teilweise geschieht dies zentral durch einen eigenen Prozess, sobald eine der registrierten Testklassen geändert wurde [Hunt und Thomas, 2003].

Um solcherart auf Komponentenebene getestete Codeteile regelmäßig auch auf Systemebene testen zu können, ist eine werkzeuggestützte, *kontinuierliche Integration* unumgänglich, vgl. Abschnitt 12.1.3.7. Das Verwenden von Testrahmensystemen muss als (teilweiser) Ersatz des Debuggers sowie als zusätzliche Qualitätssicherungsmaßnahme gesehen werden und darf keinesfalls als Ersatz für das eigentliche (Blackbox-)Testen dienen! Dies entkräftet auch das häufig vorgebrachte Argument, dass Testfälle wertlos seien, die vom Implementierer des Codes selbst geschrieben werden.

Was als Kritikpunkt bleibt, ist die oftmals hohe Investition für die Erstellung der Komponententests, die schwer in ihrem Umfang gerechtfertigt werden kann, da ja kurzfristig kein messbarer Mehrwert für den Kunden entsteht. Erst durch die langfristig höhere Qualität des Produkts (vor allem besser an die Kundenwünsche angepasste und stabilere Funktionalität) kann der Nachweis für den Nutzen des Einsatzes von Testrahmensystemen in der Implementierung erbracht werden [Hunt und Thomas, 2003]. Im Bereich der Open-Source-Entwicklungsumgebungen (wie z. B. Eclipse; <http://www.eclipse.org>) sind entsprechende Testrahmensysteme bereits integraler Bestandteil der Umgebungen. In kommerziellen Umgebungen können sie – derzeit noch ausschließlich – durch Open-Source-Lösungen nachgerüstet werden (z. B. *NUnit* für Microsoft Visual Studio .NET; <http://nunit.org>).

12.2.4.5 Debugger

Ein *Debugger* ist ein spezielles Programm, welches das Ausführen des zu testenden Programms in einem speziellen Modus ermöglicht, in dem der Programmablauf verfolgt werden kann, um die Ursache von Fehlern zu finden. Meistens sind dazu spezielle Übersetzungen des Quellcodes notwendig, um das ausführbare Programm mit entsprechendem Code („Debuginformation“) zu instrumentieren. Folgende Funktionen soll ein guter Debugger bieten [Pomberger und Blaschek, 1996]:

- Analyse nach einem Programmabbruch (Postmortem-Analyse),
- Anzeige der durchgegangenen Programmstellen im Quellcode,
- Zugriff auf die gespeicherten Daten via Variablennamen (inklusive Auflösen von Verkettungen und Strukturen),
- Darstellung von Aufrufketten beim fehlerhaften Aufruf einer Methode (so können die Ursachen für ungültige Parameterwerte ermittelt werden),

- Unterstützung von Singlestepping inklusive Breakpoints,
- Analyse dynamischer Daten- und Objektstrukturen zur Verfolgung von Beziehungen zwischen Objekten sowie direkt und indirekt referenzierten Datenwerten,
- interaktive Änderung der Datenwerte zur (temporären) Fehlerkorrektur während des Durchlaufs.

12.2.5 Ergebnisse

12.2.5.1 Lauffähiges Programm

Ein *lauffähiges Programm* ist der übersetzte Quellcode (als Gesamtprogramm, manchmal aber auch in Form einer Bibliothek), der nach dem Debuggen vom Ersteller für das Testen freigegeben wird. Spätestens ab diesem Zeitpunkt sind Versionsnummern zu führen, die im gesamten Projekt bekannt zu machen sind.

Einem lauffähigen Programm sollen in elektronischer Form Informationen beiliegen, auf welche Weise das Programm installiert und gestartet werden kann. In dieser Datei können auch nötigenfalls kurzfristige, wichtige Änderungen oder Ergänzungen der aktuellen Dokumentation enthalten sein (vgl. dazu aber die Anmerkungen in Abschnitt 11.5.3).

12.2.5.2 Fehlerliste

Fehlerlisten sind strukturierte Sammlungen begangener oder möglicher Fehler. Umfassende, gut strukturierte Fehlerlisten sind schwierig zu erstellen. Daher ist es sinnvoll, auf Standardlisten zurückzugreifen und diese sukzessive um eigene Fehler und Fehlerklassen zu ergänzen. In [Kaner *et al.*, 1999] ist z. B. eine Fehlerliste mit mehr als 400 Fehlern enthalten. Teilweise wird auch vorgeschlagen, Fehlerklassifikationslisten auf möglichst breiter Basis zu erstellen, z. B. unter Nutzung des Internets. Auf individueller Basis wird die Bedeutung von Fehlerlisten für die Verbesserung des persönlichen Softwareentwicklungsprozesses in [Humphrey, 2005] ausführlich erläutert.

Die Gültigkeit eines allgemein anerkannten Prinzips für die Programmierleistung eines einzelnen kann mittels Fehlerlisten und Fehlerstatistiken rasch nachgewiesen werden: Die Majorität an Fehlern bei jedem Softwareentwickler lässt sich auf eine relativ kleine Anzahl von – individuell unterschiedlichen – Fehlerarten zurückführen. Dieses so genannte *Pareto-Prinzip* (vgl. z. B. [McCabe und Schulmeyer, 1987]) ermöglicht es, durch Sortieren der Fehlerarten nach Häufigkeit die individuellen Schwächen bei der Softwareerstellung rasch zu erkennen und entsprechende Maßnahmen zu setzen.

Eine Fehlerliste kann als Anleitung zur Strukturierung der Fehlersuche dienen, aber auch zur Generierung von Ideen bei scheinbar unlösbaren Problemen. Durch Extraktion der relevanten Fehlerklassen und Ergänzen projektspezifischer Fehler(klassen) können aus Fehlerlisten projektspezifische Checklisten erstellt werden. Umfassende Fehlerlisten können zur raschen Einschätzung projektspezifischer Testpläne verwendet werden. Auch wenn für Tests sehr wenig Zeit vorhanden ist, kann mittels Fehlerlisten eine geeignete Schwerpunktsetzung bei gleichzeitiger breiter Abdeckung möglicher Fehler gewährleistet werden.

13 Testen

In diesem Kapitel wird der Bereich des Testens strukturiert präsentiert und erläutert, warum der Aufwandsanteil des Testens im gesamten Projektentwicklungszyklus so hoch ist. Zusätzlich verfolgt dieses Kapitel auch den Zweck der Einführung einer sauberen Nomenklatur des Testens, da gerade im Bereich des Testens viele Begriffe in der Literatur unterschiedliche Verwendung finden.

13.1 Ziel

Das *Ziel des Testens* eines Softwaresystems besteht darin, „die Wechselwirkungen der Systemkomponenten unter realen Bedingungen zu prüfen, möglichst viele Fehler aufzudecken und so sicherzustellen, dass die Systemimplementierung die Systemspezifikation erfüllt“ [Pomberger und Blaschek, 1996]. Testen soll systematisch die verschiedenen Klassen von Fehlern aufzeigen in minimaler Zeit und mit minimal notwendigem Aufwand.

Oft wird als Ziel des Testens angegeben, dass es dazu dient, die Funktionsfähigkeit der Software über den gesamten Anwendungsbereich zu zeigen. Testen wird daher vielfach fälschlicherweise definiert als ein Prozess, der zeigen soll, dass ein Softwaresystem keine Fehler enthält. [Kaner *et al.*, 1999] formulieren dazu gegensätzlich, dass Testen definiert werden soll als die Aufgabe, „ein Programm mit der Absicht auszuführen, Fehler zu finden“. Es kann jedoch nur die Anwesenheit von Fehlern gezeigt werden, die Abwesenheit von Fehlern, also Fehlerfreiheit, lässt sich nie zeigen! [Dijkstra, 1976] Diese unmögliche Forderung wird von „Praktikern“ oft als „100 %ige Fehlerfreiheit“ im Lastenheft erhoben, sie frustriert und demotiviert jedoch den Programmierer.

Jedes Programm enthält konsequenterweise Fehler. Dieser Umstand wird in [Halstead, 1977] auf eine „mentale Irrtumsrate“ zurückgeführt, die zwar individuell verschieden, über die Zeit jedoch für jede Person relativ konstant ist. Als erreichbare (und damit anstrebenswerte) Aufgabe ist daher das *Finden von Fehlern* als Ziel vorzugeben, denn mit jedem gefundenen Fehler wird das Programm brauchbarer. Aus dieser Einsicht ergibt sich jedoch auch die Konsequenz, dass schwer definierbar ist, wann ein Softwareprodukt fertig gestellt ist. In [Kaner *et al.*, 1999] wird diesbezüglich festgehalten, dass „Softwareprodukte nie freigegeben werden, sie entkommen“.

Testen ist ein destruktiver Prozess und wird daher als nicht kreativ und psychologisch negativ besetzt eingestuft. Im Gegensatz dazu gelten die vorangehenden Phasen der Entwicklung, und dabei insbesondere die Implementierung, als kreativ und sind daher positiv besetzt und eine weit anstrebenswertere Tätigkeit (vgl. dazu [Elzer, 1994]).

Daher ist es wichtig, auf den notwendigen und positiven Beitrag des Testens zum Erfolg eines Projekts hinzuweisen: Gutes Testen führt zur Erhöhung der Produktqualität und damit der Kundenzufriedenheit. Fehler werden gefunden und behoben. Die scheinbar destruktive Tätigkeit ist, im ganzen Projektentwicklungszyklus gesehen, eine sehr konstruktive Aufgabe, die das Projektergebnis entscheidend verbessert. Testen ist also auch eine wesentliche Maßnahme zur Qualitätssicherung.

Der Implementierer soll nicht testen!

Einem Tester muss man die richtige Einstellung vermitteln: Er weist dem Implementierer (Designer, Analytiker, ...) nach, dass dessen Werk Fehler enthält. Wie dazu in [Myers, 1979] angegeben wird, sucht jemand konsequenter nach Fehlern, wenn es seine Aufgabe ist, Fehler zu finden, als wenn er nachweisen soll, dass ein Programm fehlerfrei ist. Aus diesem Grund ist es nicht sinnvoll, dass ein Implementierer sein eigenes Programm testet! Der Implementierer wäre seiner Arbeit gegenüber viel zu wenig destruktiv. Seine konzeptuellen Fehler während der Implementierung wiederholen sich beim Testen und klassifizieren viele dieser Fehler als „korrektes“ Programmverhalten. Zusätzlich gibt sich ein Implementierer viel mehr Mühe bei seiner Arbeit, wenn er weiß, dass jemand anderer sein Ergebnis testet. Eine weitere, gerade für die Dokumentation und Verbesserung des Entwicklungsprozesses negative Erkenntnis ist die, dass ein Implementierer von ihm gefundene Fehler (speziell schwerwiegendere) zwar behebt, aber im Allgemeinen weder die Existenz des Fehlers dokumentiert noch die Zeit zur Behebung korrekt angibt.

Ein Test wird daher als „erfolgreich“ bezeichnet, wenn er einen Fehler aufdeckt. Ein Test, der keinen Fehler findet, gilt als „nicht erfolgreich“, in [Kaner *et al.*, 1999] wird er sogar als „Zeitverschwendung“ bezeichnet. Dennoch muss man bedenken, dass, wenn der Implementierer einigermaßen sorgfältig und vorsichtig gearbeitet hat, auch ein Großteil der bestausgedachten Testfälle keinen Fehler entdecken wird.

Der Tester soll nicht korrigieren!

Wenn beim Testen entdeckte Fehler vom Tester korrigiert werden, so hindert ihn dies an der Fortsetzung der Fehlersuche. Außerdem bedeutet für ihn das Finden von mehr Fehlern mehr Arbeit bei der Korrektur, was die Fehlerentdeckungsrate beträchtlich senkt. Ein Tester soll daher die entdeckten Fehler dokumentieren und dem Implementierer zur Korrektur überlassen. Der Tester ist auch mit der Implementierung im Allgemeinen zu wenig vertraut und würde durch die Analyse der Codestruktur zu sehr von seiner Tätigkeit abgelenkt. Im Gegensatz zum Whitebox-Testen des Implementierers im Rahmen des Debuggens (vgl. Abschnitt 12.2.3.2) betreibt daher der Tester *Blackbox-Testen*, er „schaut nicht in das Testobjekt hinein“. Dieses Testen wird daher auch als *äußeres Testen* bezeichnet und konzentriert sich auf das für den Anwender erkennbare Verhalten eines Testobjekts. Dies führt zu ganz anderen Testklassen und Testabläufen als beim Debuggen und ergänzt daher dessen Aufgaben.

13.2 Aufgaben

13.2.1 Erkennen von Analyse- und Designfehlern

Da der Tester das Programm „von außen“ verstehen muss, beschäftigt er sich zwangsläufig intensiv mit der Benutzerschnittstelle. Natürlich treten dabei viele Implementierungsfehler (Programmabstürze, nicht anwählbare Elemente etc.) auf. Bedienungsprobleme, Inkonsistenzen sowie fehlerhafte Bedienungsabläufe (z. B. das Fehlen eines Menüeintrags zum Beenden des Programms) weisen jedoch häufig auf Designfehler (manchmal auch Analysefehler) hin. Auch wenn ein nicht beherrschbarer Bedienungsablauf so implementiert wurde, wie er im Pflichtenheft spezifiziert wurde, wird dieser dadurch nicht brauchbarer und ist als Fehler festzuhalten.

13.2.2 Erkennen von Implementierungsfehlern

Das Erkennen von Implementierungsfehlern beginnt bereits mit den ersten informellen Tests. Erfolgreiche Tests sollen jedenfalls für Wiederholungszwecke aufgehoben werden. Beim Beginn der Testzyklen mit der geplanten Testfallsammlung werden Implementierungsfehler strukturierter und nach Klassen systematisch gesucht und erkannt. Die ersten Erfahrungen dienen zur vorläufigen Abschätzung des Testaufwands in Bezug auf Testbereiche, Ressourcen, Aufwand und Zeit.

Speziell Erfolg versprechend ist es, die möglichen Fehler und deren Behandlung durchzutes-ten. Dabei versucht man, möglichst viele verschiedene Fehler im Programm zu generieren. Neben der Variation der Eingabemöglichkeiten stellt hier die Liste der Fehlermeldungen des Programms eine gute Hilfe dar: Jede Fehlermeldung sollte zumindest einmal erzeugt werden.

Ein weiterer Bereich, der oft sehr fehlerbehaftet ist, ist die Kompatibilität zweier Programme oder verschiedener Programmversionen mit Datei- oder Datenbankformaten. Diese wird beim so genannten *Kompatibilitätstesten* überprüft.

13.2.3 Bestimmen des Produktstatus

13.2.3.1 Alphatesten

Alphatesten ist das Testen der Funktionalität eines Systems bottomup, d. h., die funktionalen Anforderungen an ein System werden einzeln, unabhängig voneinander überprüft. Das Produkt ist dabei im Allgemeinen als Gesamtes noch nicht sehr stabil. Für eine *Alphaversion* eines Softwaresystems muss die Kernfunktionalität in testbarer Qualität implementiert sein. Das Produkt kann v. a. in Bezug auf Benutzerinteraktion verwendet werden. Detailfunktionalitäten fehlen jedoch noch bzw. werden sukzessive integriert.

Die Umsetzung des Designs muss bereits so weit fortgeschritten sein, dass keine ernsthaften Implementierungsrisiken mehr bestehen dürfen. Manchmal wird gefordert, dass bereits die gesamte Funktionalität implementiert sein muss, jedoch noch schwere Fehler enthalten sein dürfen (vgl. z. B. [Kaner *et al.*, 1999]). Durch den Blackbox-Status des Alphatestens ist jedoch für den *Alphatester* im Wesentlichen nicht überprüfbar, ob eine Funktionalität tatsächlich be-

reits implementiert und nur fehlerhaft ist oder ob nur ein Gerüst und entsprechende Stümpfe für die Funktionalität existieren. Die Forderung nach funktionaler Vollständigkeit einer Alphaversion ist nicht überprüfbar und daher nicht zweckmäßig.

13.2.3.2 Betatesten

Betatesten ist das Testen eines Systems im Gesamten anhand realer Beispielsabläufe in realen Umgebungen. Dazu muss ein Softwaresystem den Status erreicht haben, dass es für entwicklungsexterne Anwender verwendbar, brauchbar und nicht verärgern ist. Die Produktqualität muss also von externen Anwendern anerkannt werden können, wobei nicht unbedingt bereits die volle Funktionalität zur Verfügung stehen muss.

Zur Durchführung des Betatestens sind zwei Alternativen möglich:

1. Spezielle „Betatester“ fungieren als „Testpiloten“, die mit der *Betaversion* praktische Beispiele aus dem zukünftigen Anwendungsgebiet bearbeiten und alle auftretenden Fehler oder Probleme dokumentieren und an die Entwickler weiterleiten. Betatester sind entweder ausgewählte Pilotanwender aus dem zukünftigen Kundenkreis bzw. des Mitarbeiterstabs des Auftraggebers oder aber eigens für diese Aufgabe angestellte Mitarbeiter.

Die Betatester wissen, dass es sich noch um kein fertiges Produkt handelt und dass sie an ihre Arbeit mit der gebotenen Vorsicht gehen müssen (z. B. durch häufiges Speichern von Zwischenergebnissen, Skepsis gegenüber Dokumentation und Texten der Benutzerschnittstelle). Eine gute Kooperation mit den Entwicklern ist daher in beiderseitigem Interesse essenziell.

2. Betatest-Beispiele werden vom Auftraggeber dem Auftragnehmer zur Verfügung gestellt zusammen mit einer genauen Ziel- und Ablaufbeschreibung für jedes Beispiel. Mitglieder der Entwicklungsgruppe des Auftragnehmers übernehmen (als Teilaufgabe) die Rolle von Betatestern. Dabei müssen sie dennoch engen Kontakt mit dem Auftraggeber pflegen, um das Umfeld der Anwendungsbeispiele zu verstehen und richtig einschätzen zu können. Durch die direkte Integration in die Entwicklergruppe ergeben sich Zeitvorteile beim Berichten und Erläutern von Fehlern sowie beim Beurteilen von Behebungsvorschlägen.

Viele Betatest-„Fehler“ hängen von der Einschätzung des Benutzers ab und werden daher von Auftraggeber und Auftragnehmer verschieden beurteilt. Betatesten ohne externe Betatester soll daher vermieden werden, wenn Projektumfang, Projektstruktur und die Zeitplanung dies zulassen.

13.2.4 Performanztest – Testen der Produktperformanz

Performanz wird bereits während des Debuggens überprüft (vgl. Abschnitt 12.1.3.2). Während dies aber aus der Sicht des Implementierers geschieht, geschieht das Testen der Performanz beim Blackbox-Testen aus der Sicht und der Antwortzeiterfordernisse des Anwenders. Folgende Aufgaben werden von Performanztests erfüllt:

- **Ermitteln notwendiger Performanzverbesserung:** Durch Tests wird festgestellt, welche der am häufigsten verwendeten Komponenten ein inakzeptables Zeitverhalten aufweisen. Aus diesen Tests werden Performanzvorgaben abgeleitet, die dann von den Implementierern durch Tunen der entsprechenden Systemkomponenten erreicht werden sollen.
- **Speichertests** prüfen die Anforderung von Speicher (Hauptspeicher, externer Speicher) durch das Programmsystem. Bei spezifizierten Grenzen wird versucht, diese zu überschreiten, andernfalls wird versucht, das System in einen Zustand bzw. Zyklus dauernder Speicheranforderung („memory leak“) zu bringen.
- **Volumentests** loten den maximalen Umfang der Aufgaben aus, der vom Programm gerade noch bewältigt werden kann (z. B. maximale Dateilänge einer Textverarbeitung, Belastbarkeit des Eingabepuffers).
- **Belastungstests** testen das Systemverhalten durch sukzessives Erhöhen der Systemaktivität (zusätzliche Prozesse, Netzwerkbelastung etc.). Dadurch wird ausgelotet, bis zu welchen Hardware- und Systembedingungen das Softwareprodukt stabil bleibt. In diesen Tests ist auch die Auslotung der Schnittstellenbelastbarkeit enthalten (z. B. extrem hohe Mausektivität, Client/Server-Betrieb mit ungewöhnlich hoher Senderate).
- **Benchmarkvergleiche** vergleichen entweder verschiedene Produktversionen miteinander, um Verbesserungen der Produktperformanz quantifizieren zu können, oder sie vergleichen das Produkt mit Konkurrenzprodukten, um Produktvorteile zu identifizieren bzw. Nachbesserungen bei Performanznachteilen einzufordern (siehe dazu für Details [Beizer, 1995]).

13.2.5 Abnahmevortest – Vorbereiten der Abnahme

Der *Abnahmevortest* nimmt die Abnahmesituation beim Auftraggeber vorweg. Um den Abnahmevorgang möglichst problemlos ablaufen lassen zu können, sind folgende Bereiche vorab zu prüfen:

- Test aller Konfigurationen, in denen das System korrekt funktionieren muss,
- korrekter Durchlauf der vertraglich festgelegten Abnahmesuite,
- Installierbarkeit auf der Zielumgebung gemäß der Installationsbeschreibung,
- Möglichkeit der Inbetriebnahme des installierten Systems,
- Vollständigkeit und Korrektheit der Dokumentation (Systemdokumentation, Benutzerdokumentation),
- Einfrieren des Aussehens der Benutzerschnittstelle für die Abnahme („user interface freeze“), Übereinstimmung mit der Lieferversion der Dokumentation.

Die *Abnahmesuite* dient zur Überprüfung der Brauchbarkeit des Systems durch den Auftraggeber. Dieser muss sich vertraglich bereit erklären, dass bei erfolgreichem Durchlauf der Abnahmesuite das Produkt die geforderten Anforderungen erfüllt („abgenommen ist“) und

in Zukunft auftretende Fehler oder Produktmodifikationen extra zu bezahlen sind, so im Vertrag nicht anderes festgelegt wurde (z. B. eine zeitlich begrenzte Garantie). Idealerweise ist die Abnahmesuite vom Auftraggeber zu erstellen. In der Praxis wird sie meistens von Auftraggeber und Auftragnehmer gemeinsam erstellt, was dem Auftragnehmer eine gewisse Einflussnahme auf Schwerpunkte und inhaltliche Details ermöglicht.

Vielfach wird seitens des Auftraggebers versucht, die Abnahmesuite „offen zu halten“, d. h., laufend weitere Testfälle zu spezifizieren – bis hin zur Generierung zusätzlicher neuer Testfälle während der Abnahme. Dadurch wird die Abnahme hinausgezögert, manchmal sogar ganz verhindert. Die Verhandlungen über den Abnahmemodus und die Festlegung der Abnahmesuite sollten daher im Projekt so früh wie möglich erfolgen. Bei der Festlegung von Abnahmebedingungen sowie des Abnahmeablaufs ist die Beiziehung von Rechtsexperten sehr ratsam.

Für Details zum eigentlichen *Abnahmetest* wird auf Abschnitt 14.3.4 verwiesen.

13.3 Techniken

13.3.1 Testfalldesign

Grundsätzlich ist nicht derjenige Tester der beste Tester, der die meisten Fehler findet oder der die meisten Implementierer verärgert, sondern derjenige mit der höchsten Fehlerbehebungsrate (d. h., der die Fehler so findet und aufbereitet, dass sie effizient behoben werden können). Dazu ist es entscheidend, für jeden Fehler einen solchen Testfall zu finden, der den Fehler und dessen Konsequenzen so offensichtlich aufzeigt, dass der Fehler auf keinen Fall toleriert werden kann. Ein guter Testfall

- macht Programmfehler offensichtlich,
- besitzt eine hohe Wahrscheinlichkeit, einen Fehler zu entdecken,
- ist weder zu einfach noch zu komplex.

Guten Testern wird allgemein ein großes Maß an Intuition zugestanden; Testen ist aber über weite Bereiche ein sehr systematischer Prozess [Kaner *et al.*, 1999]. Folgende Testtechniken sind dabei am gängigsten:

- **Bildung von Äquivalenzklassen:** Aus der Klasse ähnlicher Tests ist derjenige auszuwählen, der am wahrscheinlichsten einen Fehler findet und möglichst dessen Behebung unterstützt.
- **Grenzwertanalyse:** Grenzen der Wertebereiche von Parametern haben die größte Fehlerwahrscheinlichkeit [Beizer, 1995]. Daher sind alle Eingabewerte mit den kleinsten und größten zulässigen Werten sowie mit den gerade nicht mehr zulässigen Werten zu versehen und das System damit zu testen. Weitere Grenzwerte können bestimmt werden durch Nulldurchgänge, Grenzen des mathematischen Definitionsbereichs, Grenzen der internen Zahlendarstellung, Feldgrenzen und minimale/maximale Schleifendurchlaufzahlen (soweit im Blackbox-Test bekannt). Um die Grenzwertanalyse auch für

Ergebniswerte durchzuführen, ist anzustreben, auch solche Eingabewerte zu testen, bei denen sich Extremwerte für die Ausgabe ergeben.

- **Analyse des Ursache-Wirkungs-Zusammenhangs:** Um feststellen zu können, ob ein Test einen Fehler liefert oder nicht, ist es essenziell, für jeden Testfall zu definieren, welche Ergebnisse er liefern soll. Die tatsächlichen Ergebnisse sind genau mit den erwarteten zu vergleichen. Dies geschieht am besten werkzeugunterstützt (z. B. durch Dateivergleichswerkzeuge), um Fehler durch inkorrekte Vergleiche weitgehend zu vermeiden.

13.3.2 Testsuitedesign

Das *Testsuitedesign* umfasst das Erstellen eines Verzeichnisses geeigneter Testfälle. Diese sollen beim Blackbox-Testen alle funktionalen Anforderungen an das System (aus externer Sicht) abdecken. Es ist unmöglich, die Testsuite so zu gestalten, dass die gesamten möglichen Anwendungen des Systems getestet werden. [Kaner *et al.*, 1999] weisen darauf hin, dass man nie alle Tests durchführen kann, die man möchte und dass man durch den Projektdruck kaum je alle Tests durchführen kann, die man sich als unbedingt notwendige Mindestsuite festgelegt hat.

Beim Testsuitedesign ist darauf zu achten, bei den funktionalen Anforderungen nicht nur Testfälle zu spezifizieren, die ein korrektes Ergebnis liefern müssen (*Konformanztests*), sondern auch Testfälle, die Fehler hervorrufen (*Nonkonformanztests*). Zusätzlich sind – nicht-funktionale – *Qualitätstests* (z. B. Stabilität bei unzulässiger Eingabe) zu spezifizieren.

13.3.2.1 Erstellen von Testfällen

Folgende Vorgehensweisen können zur Erstellung von Testfällen angewendet werden:

- Aus dem Lasten- und Pflichtenheft sowie aus der Benutzer- und Systemdokumentation lassen sich viele Tests ableiten. Mittels Grenzwerttabellen für die einzelnen Eingabewerte können die am besten geeigneten Kombinationen von Werten für Konformanztests und Nonkonformanztests ermittelt werden. Diese Tabellen werden laufend durch neue Einsichten aus Testergebnissen erweitert.
- Durch interaktives Arbeiten mit dem zu testenden System können neue Ideen für Tests und entsprechende Tests generiert werden.
- Wenn ein Referenzprogramm vorhanden ist (z. B. wenn das neue System ein altes ersetzt, das aber zumindest in Teilbereichen die gleiche Funktionalität aufweist), kann dieses Programm zur Generierung korrekter Ausgaben für Testfälle verwendet werden.
- Konkurrenzprodukte oder kurzfristig geschriebene Spezialprogramme können verwendet werden, um das gewünschte Verhalten des zu testenden Systems in Teilbereichen festzulegen.
- Als korrekt eingestufte Ergebnisse können gespeichert und bei nächsten Testläufen als Referenzergebnisse verwendet werden.

- Jedes Ergebnis eines Tests soll möglichst in elektronischer Form aufgehoben werden, um es später für Vergleichszwecke oder zur Ableitung verbesserter Tests verwenden zu können.

13.3.2.2 Auswahl von Testfällen

Folgende Kriterien können zur Auswahl von Testfällen für die Testsuite verwendet werden [Kaner *et al.*, 1999]:

- Tests für Bereiche, in denen bis jetzt die meisten Fehler gefunden wurden¹,
- Tests auf Fehler, die dem Anwender zuerst auffallen oder die für den Anwender am verheerendsten sind,
- Tests für Programmbereiche, die am häufigsten verwendet werden,
- Tests für Funktionalitäten, welche die besonderen Merkmale eines Produkts (z. B. Alleinvorteile gegenüber Konkurrenzprodukten) darstellen,
- Tests für Bereiche, in denen Fehler für den Implementierer am schwierigsten zu beheben sind (in Rücksprache mit dem Implementierer),
- Tests in Bereichen des Systems, die für den Tester am besten verständlich sind.

13.3.3 Testablaufplanung

Die Planung des Testablaufs kann beginnen, sobald Analyse- oder Designdokumente verfügbar sind, spätestens aber mit Verfügbarkeit des Pflichtenhefts. Gerade bei testgesteuerter Entwicklung (vgl. Abschnitt 12.1.3.6) sind die Systemtests Basis für das Vorantreiben der Implementierung und daher bereits während der Analyse, spätestens aber während der ersten Designschritte zu erstellen.

13.3.3.1 Testvorbereitung

Mit der Testvorbereitung kann bereits während der Analyse und des Designs begonnen werden. Auch wenn noch kein Testprodukt zur Verfügung steht, können vom Tester bereits folgende Aktivitäten gesetzt werden:

- Einlesen in Projektziel und Produktzweck,
- Reviewen der existierenden Dokumente (Lastenheft, Pflichtenheft, Kommunikation mit dem Auftraggeber, Organisationsdokumente),
- Einfordern und Analysieren der Testdaten des Auftraggebers,

¹Wie in [Myers, 1979] festgestellt wird, „leben Fehler in Kolonien“, sodass in stark fehlerbehafteten Codebereichen auch das Beheben von Fehlern sehr fehleranfällig ist.

- Ausarbeiten (bzw. Konkretisieren) der Abnahmebedingungen und des Abnahmevergangs,
- Reviewen der Benutzerschnittstelle,
- Reviewen der Benutzerdokumentation,
- Analysieren von Konkurrenzprodukten.

Sobald eine erste, noch instabile Produktversion verfügbar ist, kann man sich mit dem Produkt und seiner Anwendung bereits vertraut machen. Eine erste Version der Testsuite wird erstellt und dem Projektleiter zur Überprüfung gegeben. Designfehler können bereits aus dem Pflichtenheft und der Benutzerdokumentation erkannt und rückgemeldet werden. Für das Produkt wird nun eine Testumgebung eingerichtet (Hardware- und Systemsoftwarekonfiguration, notwendige Zusatzprodukte, Geräte) und auf Kompatibilität untereinander und mit der im Pflichtenheft festgelegten Zielkonfiguration überprüft.

13.3.3.2 Qualifikationstest

Wenn ein Tester eine neue Programmversion erhält, ist es sinnvoll, zuerst kurz abzutesten, ob die Version überhaupt stabil genug für ausführliche Tests ist. Dazu wird eine kleine *Qualifikationstestsuite* erstellt (*qualification test suite*, auch *acceptance test suite*). Diese Suite muss im *Qualifikationstest* vom Programm korrekt verarbeitet werden, um für einen Testdurchlauf zugelassen zu werden. Um die Anzahl der Zurückweisungen zu reduzieren und um Zeit zu sparen, kann die Qualifikationstestsuite den Implementierern zugänglich gemacht werden. Dann muss der Tester allerdings davon ausgehen, dass das Programm zwar wahrscheinlich genau auf die Qualifikationstests hin geprüft wurde, dass unter Umständen dadurch aber andere Programmteile weniger genau überprüft worden sind, als dies ohne Bekanntgabe der Qualifikationstestsuite der Fall wäre.

Qualifikationstests sollen nur Hauptanwendungsbereiche umfassen („mainstream testing“). Die Fehler, die diese Tests aufzeigen sollen, müssen auch vom Implementierer ohne Diskussion als schwerwiegend eingestuft werden. Eine Qualifikationstestsuite, über deren Inhalt dauernd diskutiert wird, ist für ein Projekt schlechter als keine!

Die Anzahl der gefundenen Fehler im Qualifikationstest kann auch als Maß für den Testaufwand im nachfolgenden Zyklus verwendet werden: Je mehr Fehler auf Anhieb gefunden werden, desto mehr stecken normalerweise noch im Programm.

Da Qualifikationstests sehr häufig durchgeführt werden, ist eine Automatisierung dieser Tests oft sehr sinnvoll. Auch wenn Qualifikationstests teilweise spontan erfunden und durchgeführt werden, ist es sehr wichtig, den Test zu protokollieren, selbst wenn diese Unterlagen bei Qualifikationstests vielfach nur für den Tester und dessen Prozessverbesserung verwendet werden.

13.3.3.3 Testzyklus

Sobald eine erste Version des Produkts verfügbar ist, die den Qualifikationstest bestanden hat, wird die festgelegte Testsuite iterativ durchlaufen, wobei normalerweise die Testsuite

kontinuierlich erweitert wird. Ein Durchlauf wird *Testzyklus* genannt. Folgende Schritte sind in einem Testzyklus empfehlenswert:

1. **„Start with a bang!“** [Kaner *et al.*, 1999]: Es ist vor allem im Hinblick auf die eigene Zeitplanung empfehlenswert, gleich zu Beginn eine möglichst große Anzahl von Fehlern zu entdecken und Fehlerberichte an die Implementierer zu übermitteln, z. B. indem man im vielversprechendsten Bereich mit der Fehlersuche beginnt. Die Implementierer sind dadurch von Beginn an mit der Fehlerbehebung beschäftigt, und dem Projektleiter gegenüber wird die Arbeit und Produktivität des Testers sichtbar gemacht.
2. **Schätze Stabilität und Zuverlässigkeit des Produkts ab:** Instabile Bereiche des Produkts, die sich für ein intensives Testen nicht eignen, sollen möglichst bald identifiziert und den Implementierern mitgeteilt werden. Aus Anzahl, Quelle und Art der ersten Fehler kann die momentane Produktzuverlässigkeit abgeschätzt und damit die Prognose für die Anzahl der noch nötigen Testzyklen aktualisiert werden.

In [Beizer, 1990] wird auf Basis von statistischen Erhebungen die durchschnittliche Anzahl von Fehlern, die in einem Programm enthalten sind, das von den Implementierern zum Testen freigegeben worden ist, mit 1 bis 3 Fehler pro 100 Befehle angegeben. Derselbe Autor, ein erfahrener Programmierer und Tester, gibt jedoch seine eigene Fehlerrate über den ganzen Softwareentwicklungszyklus hinweg in [Beizer, 1995] mit *1,5 Fehler pro ausführbarem Befehl* an! Fehler dürfen und sollen daher auch (und gerade) in Softwarekomponenten erwartet werden, von denen der Implementierer behauptet, sie seien „praktisch fehlerfrei“.
3. **Teste auf offensichtliche Fehler und fehlerhafte Dokumentation:** Fehler, die ein Anwender in den ersten Stunden der Verwendung findet, bleiben besonders in Erinnerung und sind dadurch besonders ärgerlich. Durch eine Simulation des ersten Arbeitstages mit dem Produkt können solche Fehler entdeckt werden. Grobe Inkonsistenzen zwischen Produkt und Dokumentation sind ebenfalls Fehler, die einen Benutzer sehr ärgern, wenn er sie entdeckt (vgl. Abschnitt 11.5.3). Diese Fehler sollen rasch gemeldet werden, damit Produkt, Dokumentation oder beides korrigiert werden können.
4. **Teste alle Testfälle der Testsuite:** Dabei sollen alle Ergebnisse, aber auch Besonderheiten beim Ablauf oder Erkenntnisse zur Verbesserung der Testfälle selbst, in elektronischer Form dokumentiert werden.
5. **Dokumentiere die Ergebnisse des Testzyklus:** Dieser Bericht enthält eine detaillierte Beschreibung aller in diesem Zyklus neu gefundenen Fehler sowie eine Auflistung aller noch im Produkt enthaltenen, bereits bekannten Fehler.
6. **Adaptiere die Testsuite** gemäß den Erkenntnissen des Testzyklus.

In [Kaner *et al.*, 1999] wird angegeben, dass bei jedem durchschnittlichen Projekt mindestens acht Testzyklen bis zur Auslieferung der ersten Produktversion eingeplant werden müssen. Nach Abschluss des Testens ist genügend Zeit (mindestens eine Woche) einzuplanen, um die Erkenntnisse aus dem gerade abgeschlossenen Test für zukünftige Tests aufzubereiten sowie den eigenen Testprozess zu verbessern.

Inkrementelles Testen versus Big-Bang-Testen

Da ein Softwaresystem komponentenweise erstellt wird, ist es sehr sinnvoll, diese schrittweise Entwicklung des Systems durch schrittweises Testen der Komponenten zu unterstützen. Gerade bei testgesteuerter Entwicklung ist es essenziell, aus den Systemtests während der Implementierung laufend neue Komponententests ableiten zu können und so die Implementierung voranzutreiben, vgl. Abschnitt 12.1.3.6.

Diese Methode des *inkrementellen Testens* bedingt den Bau von Testrahmen und Stümpfen ähnlich wie in der Implementierung (Abschnitt 12.1.3.5) und macht damit im Allgemeinen Zugang zu Quellcode und Übersetzungsprozessen für den Tester notwendig. Die Vorteile sind jedoch ein rascherer Beginn des Testens, leichtere Lokalisierung der Fehlerursache durch die inkrementelle Vorgehensweise sowie bessere Testabdeckung durch die modulare Sichtweise beim Erstellen von Testfällen.

Im Gegensatz zum inkrementellen Testen wird manchmal, um den Bau von Testrahmen zu vermeiden, *Big-Bang-Testen* empfohlen, bei dem der erste wirkliche Test erfolgt, wenn das Gesamtsystem in seiner ersten vollständigen Version vorliegt. Diese Vorgehensweise besitzt jedoch die schwerwiegenden Nachteile, dass sehr spät mit dem Testen begonnen wird, Fehlerursachen kaum zu finden sind sowie über Verantwortlichkeiten für Fehler oft heftig diskutiert wird. Vielfach bleibt unerwähnt, dass ohne inkrementelles Testen oftmals der Status nie erreicht wird, dass eine erste testbare Gesamtversion zur Verfügung steht. Big-Bang-Testen ist daher mit einem ingenieurmäßigen Vorgehen unvereinbar.

13.3.4 Verifikation – Testen der Funktionalität

Gemäß [Boehm, 1984] bedeutet der Testbegriff *Verifikation* (*verification*) “to establish the truth of correspondence between a software product and its specification”. Verifikation im Rahmen des Testens ist also das Überprüfen der Funktionalität eines Softwaresystems im Vergleich zum Pflichtenheft und ggf. zusätzlicher Spezifikationsunterlagen (Protokolle, Designdokumente). Dieser Test wird daher auch *Funktionalitätstest* (*functional test*) genannt.²

Folgende Techniken werden unterschieden:

- Der *Komponententest* (*component test*, *Klassentest*; auch *Einzeltest*, *unit test*) testet jede Komponente als eine unabhängige Einheit.
- Der *Modultest* (*Programmtest*) testet ein Modul als Sammlung abhängiger Komponenten. Das Modul wird jedoch unabhängig gegenüber anderen Modulen getestet.
- Der *Integrationstest* testet den Zusammenbau einzelner Komponenten zum System. Dies geschieht iterativ durch das Testen von Kombinationen von Komponenten und Modulen zu größeren Modulen, bis schließlich das Gesamtsystem getestet wird. Bei mehreren Prozessen wird zuerst paarweise, dann gruppenweise und schließlich das gesamte Prozessnetzwerk getestet.

²Der Begriff *Funktionstest* ist zu vermeiden, da der Begriff „Funktion“ in der Softwareentwicklung im Allgemeinen nicht mit Funktionalität gleichgesetzt wird, sondern ein Programmkonstrukt assoziiert.

Folgende Besonderheiten sind beim *Testen von objektorientiertem Code* zu beachten:

- Vererbung, dynamische Bindung und Polymorphismus können den Code schwerer verständlich machen.
- Saubere Kapselung kann auch Fehler „kapseln“, sodass sie seltener auftreten und schwerer gefunden werden.
- Objekte speichern typischerweise Zustände. Es kann mühsam sein, Eingabefälle zu finden, um alle Objekte in alle möglichen Zustände und deren Kombinationen zu bringen.
- Bei einem großen System von Objekten kann es zeitaufwändig und mühsam sein, das Objekt in den gewünschten Ausgangszustand für einen bestimmten Test zu bringen.
- Bei der Verwendung von Rahmen mit vorgefertigtem Code (z.B. Systemrahmen) ist ein Integrationstest von der Komponente zum Gesamtsystem oft schwierig.

Das Testen objektorientierten Codes ist also im Allgemeinen nicht einfacher oder weniger zeitaufwändig als das Testen anderen Codes!

13.3.5 Validierung – Testen des Gesamtsystems

Gemäß [Boehm, 1984] bedeutet der Testbegriff *Validierung* (*validation*) “to establish the fitness or worth of a software product for its operational mission”. Validierung im Rahmen des Testens ist also das Überprüfen der Funktionalität eines Softwaresystems auf Brauchbarkeit gemäß den Benutzeranforderungen und wird daher auch *Systemtest* oder *Gesamttest* genannt.

Folgende Techniken werden unterschieden:

- Beim *Subsystemtest* wird ein Subbereich des Softwaresystems, der vom Anwender (größtenteils) unabhängig von den anderen Bereichen verwendet wird, getestet. Neben der Brauchbarkeit eines Subsystems wird auch auf Schnittstellenfehler eines Subsystems zum Gesamtsystem geachtet.
- Beim *Integritätstest* wird das aus den Subsystemen gebildete Gesamtsystem auf Brauchbarkeit gemäß vom Auftraggeber spezifizierten Anwendungsfällen getestet. Sind keine Daten vom Auftraggeber verfügbar, so sind die Fälle vom Auftragnehmer bestmöglich nachzustellen. Auf jeden Fall ist der erste Verwendungstag zu simulieren. Welche Aufgaben wird der Anwender am ersten Tag mit dem Softwaresystem bearbeiten? Sind Installations- und Inbetriebnahmedokumentation sowie das Tutorial korrekt? Welche Probleme könnte ein Benutzer am ersten Tag bei der Bedienung des Systems haben? Der erste Tag ist für die Einstellung einem Softwaresystem gegenüber ausschlaggebend. Beim Integritätstest werden typische Anwendungsfälle getestet, das Arbeiten mit dem System geschieht also aus Anwendersicht, nicht aus Reviewer- oder Testersicht. Es werden nicht ursächlich Fehler gesucht. Neben dem Softwaresystem ist die Dokumentation genau auf Brauchbarkeit zu untersuchen; zusätzlich können Vergleichstests mit Konkurrenzprodukten durchgeführt werden.

[Kaner *et al.*, 1999] empfehlen, mit einem Integritätstest nicht eine Gruppe von Testern, sondern eine erfahrene Einzelperson zu betrauen, wobei für ein durchschnittlich komplexes Single-User-Programm etwa zwei Wochen für das Testen notwendig sind.

- Eine *Zertifizierung* wird im Allgemeinen von externer Stelle durchgeführt und kann in verschiedenem Umfang durchgeführt werden. Der Testumfang wird im Zertifikat angegeben zusammen mit dem erreichten Standard des Softwaresystems, des Entwicklungsprozesses oder des Testprozesses. Häufig zertifizierte Produkte sind Compiler; Entwicklungs- und Testprozesse werden beispielsweise in der Luftfahrtindustrie oder im Krankenhauswesen in der Regel zertifiziert.

Ein validiertes System bedeutet nicht automatisch ein verifiziertes (und umgekehrt!). Beide Testaktivitäten sind also in Kombination durchzuführen, vgl. Abbildung 13.1. Daher wird im Englischen anstelle von Testen auch manchmal von *validation & verification* (*V & V*) gesprochen, vgl. z. B. [Boehm, 1984].

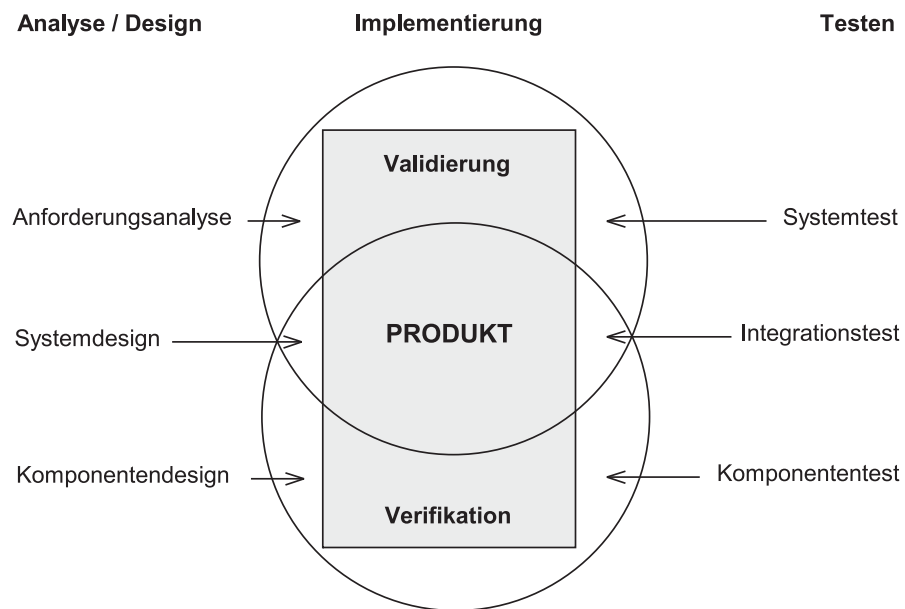


Abbildung 13.1: Zuordnung der Testaktivitäten zu den Entwicklungsaktivitäten

13.3.6 Regressionstesten

Wie z. B. in [Martin und McClure, 1983] festgestellt wird, zeigen Statistiken, dass weniger als die Hälfte aller Fehler beim ersten Behebungsversuch auch tatsächlich behoben werden, selbst wenn der Implementierer die Behebung bereits „getestet“ hat. Daher ist es sinnvoll eine *Regressionstestsuite* festzulegen, die jedes Mal abgetestet wird, wenn eine neue Programmversion zum Testen freigegeben wird.

Regressionstesten (*Retesten*) wird aus zwei Gründen durchgeführt:

- zur Prüfung, dass ein Fehler tatsächlich korrigiert wurde,
- zur Sicherstellung, dass bei der Fehlerkorrektur kein neuer Fehler eingeführt wurde.

Die Regressionstestsuite wird bei jedem Testzyklus zumindest um diejenigen Tests erweitert, die Fehler aufgezeigt haben. Im Speziellen sind auch vom Anwender gemeldete Fehler mit

entsprechenden Tests in der Regressionstestsuite zu berücksichtigen, da ein erneutes Auftreten bereits gemeldeter Fehler beim Anwender starkes Misstrauen an der Testorganisation und der Fehlerbehebungscompetenz der Entwickler wecken kann.

Wichtig ist es, immer die gesamte Regressionstestsuite durchzulaufen, damit man – weitgehend – sicherstellen kann, dass durch die Behebung eines Fehlers nicht andere eingeführt wurden bzw. bisher nicht sichtbare Fehler zu Tage treten. Daher ist es sinnvoll, zumindest das Regressionstesten weitestgehend zu automatisieren. Dies geschieht z. B. durch automatischen Durchlauf der Testfälle in der Testsuite, Sammeln der Ergebnisse in Files und Vergleichen dieser Files mit den gewünschten Ergebnissen (z. B. Ergebnissen aus früheren, als korrekt beurteilten Testläufen oder Ergebnissen aus speziell geschriebenen Testprogrammen; vgl. Abschnitt 12.2.4.4). Ein weiterer Vorteil der Automatisierung ist der, dass dadurch Versionen rasch auf behobene Fehler abgetestet werden können und die Implementierer schnell eine Rückmeldung erhalten, ob ihre Fehlerbehebung erfolgreich war. Diese rasche Rückmeldung ist ein entscheidender Motivationsfaktor für die Implementierer, Aufträge zur Fehlerbehebung wirklich ernst zu nehmen!

Da eine Regressionstestsuite möglichst oft durchlaufen werden soll, muss sie kompakt sein, dabei aber den ganzen Anwendungsbereich des getesteten Programmsystems abdecken. Es ist darauf zu achten, redundante Tests zu entfernen und Tests auch nach möglichst geringer Laufzeit auszuwählen. Wenn ein Fehler, der mit mehreren Tests geprüft wurde, einige Regressionszyklen lang als behoben klassifiziert wurde, so soll die Anzahl der diesen Fehler prüfenden Tests entsprechend reduziert werden. Manchmal ist es auch möglich, einzelne Tests zu kombinieren und so den Testdurchlauf effizienter zu gestalten.

13.3.7 Testautomatisierung

Mittels manuellem Testen kann immer nur ein kleiner Ausschnitt des Verhaltens eines Softwaresystems überprüft werden. Fehler können auch nach dem Testen in das Programm eingefügt werden, was erneutes (manuelles) Testen bedingt. Manuelles Testen ist selbst fehleranfällig und müsste deshalb einer laufenden Überprüfung unterzogen werden.

Daher gibt es bereits seit längerem Bestrebungen, das Testen zu automatisieren. Von *Testautomatisierung* wird gesprochen, wenn zumindest einer der folgenden Schritte programmgesteuert („automatisch“) ausgeführt werden kann [Vigenschow, 2004]:

- Testfälle erzeugen,
- Testfälle durchlaufen,
- Ergebnisse überprüfen,
- Resultate darstellen (visualisieren) und protokollieren.

Das automatische Generieren von (sinnvollen) Testfällen ist eher als Teilgebiet der Programmverifikation (vgl. Abschnitt 12.2.3.1) zu sehen. Die weiteren Punkte können aber mit Testrahmensystemen für Systemtests automatisiert werden, sehr ähnlich zu Testrahmensystemen für Komponententests (Abschnitt 12.2.4.4).

Gerade im Fall einer testgesteuerten Entwicklung ist die möglichst umfassende Automatisierung aller Systemtests vorrangiges Ziel. Viele Performanztests, wie z. B. Belastungstests oder Speichertests, sind ohne Testautomatisierung kaum durchführbar. Allerdings ist das Implementieren der Testfälle derart, dass sie automatisch ausgeführt werden können, sehr aufwändig: [Kaner *et al.*, 1999] sprechen davon, dass das Automatisieren eines Testszenarios³ zehnmal so viel Arbeit erfordert wie ein manuelles Durchführen derselben Tests. Neben den Amortisationskosten wird auch die Amortisationszeit häufig unterschätzt. Erst bei häufigem Retesten amortisiert sich die Testautomatisierung in vernünftig planbarer Zeit.

Testen wird seitens der Entwickler oft zugunsten der Testautomatisierung vernachlässigt (gerade im Umfeld der testgesteuerten Entwicklung mittels Testrahmensystemen, bei denen die Entwickler die Testfälle „programmieren“ dürfen). Es werden zu viele simple Testfälle automatisiert. Alte Testfälle werden aus Bequemlichkeit häufig wiederholt, anstatt neue zu entwickeln. Für komplexere Testfälle wird der Trainingsbedarf für die Werkzeuge unterschätzt. Derartige Testfälle werden daher kaum bis gar nicht erstellt. Ein weiteres Problem der automatischen Durchführung von Tests wird in [Microsoft, 1990] angesprochen: Der Ablauf und das Ergebnis eines automatisierten Tests werden weit weniger genau analysiert als beim manuellen Testen. Zusätzlich fehlen vielfach die Testprotokolle bzw. sind für den Menschen kaum lesbar. Automatisches Testen muss daher sehr detailliert geplant und organisiert werden.

13.4 Werkzeuge

Werkzeuge zur Unterstützung des Testens sind derzeit vor allem im Open-Source-Bereich anzutreffen. Einzelne Standardprodukte (z. B. TestDirector von Mercury, <http://www.mercury.com>) finden nur in größeren Unternehmen Verwendung und sind kaum in die weit verbreiteten Entwicklungsumgebungen (wie etwa Microsoft Visual Studio .NET) eingebunden. Am ehesten findet man integrierte *Testwerkzeuge* wiederum im Open-Source-Bereich (z. B. in Eclipse), da hier neben der freien Verfügbarkeit auch die Integration von Werkzeugen durch deren Entwickler selbst vorgenommen werden kann.

Die Gründe für den Mangel an weit verbreiteten, professionellen Testwerkzeugen sind schwer eruierbar. Eine wesentliche Rolle dürfte aber die lange Unterbewertung der Bedeutung eines geplanten und strukturierten Testens sein sowie die Einstellung, dass ein Auftraggeber froh sein muss, wenn überhaupt Zeit zum Testen bleibt. Darüber hinaus unterscheiden sich verschiedene Projektentwicklungen in den späteren Projektphasen (wie eben dem Testen) deutlich stärker voneinander als in den früheren (wie Analyse oder Design), was eine effiziente Werkzeugunterstützung erschwert.

In einer Studie zu den Defiziten der Testinfrastruktur [NIST, 2002] stellte das amerikanische National Institute of Standards and Technology fest, dass der Wirtschaftszweig der Softwareentwicklung in den letzten Jahren zwar konstruktiv stark aufgerüstet hat (Sprachen, Prozesse, ...), dass aber im analytischen Bereich wie vor 20 Jahren vorgegangen wird und auch die Werkzeugunterstützung kaum verbessert wurde bzw. vorhandene Werkzeuge kaum verwendet werden. Wenn der Softwareentwicklungsprozess an nur einer Stelle verbessert werden könne, müsse man unbedingt beim werkzeuggestützten Testen beginnen.

³Ein *Testszenario* besteht aus mehreren zusammengehörigen Testfällen.

Folgende *Werkzeuge zur Testunterstützung* stehen zur Verfügung:

- **Dateivergleichswerkzeuge** stellen Unterschiede zwischen zwei Dateien fest, wobei manche Werkzeuge sogar kontextsensitiv arbeiten, d. h., Gleichheit nicht durch bitweisen Vergleich, sondern durch inhaltlichen Vergleich festgestellt wird. Unterschiede gibt es auch in der Qualität der Information, wodurch die Dateien verschieden sind bzw. wie eine Datei in die andere übergeführt werden kann.
- **Formatkonvertierer** erlauben das Transformieren von Dateien (mit Text, Grafiken, Bitmaps, ...) und dadurch ein rascheres Erstellen von Eingabedateien bzw. Analysieren von Ergebnisdateien. Gute Formatkonvertierer sind in weiten Bereichen parametrisierbar bzw. erlauben die Beschreibung von Ausgangs- und Zielformaten mittels spezieller Sprachen.
- **Bildschirmspeicherprogramme (screen grabber)** erlauben das Abspeichern oder Ausdrucken des Bildschirms (oder eines bestimmten Ausschnitts oder Fensters) zu jeder Zeit während des Ablaufs eines Programms. Fehlermeldungen oder Fehler in der Bildschirmgestaltung können auf diese Weise viel genauer dokumentiert werden.
- **Monitorprogramme** erlauben das Abspeichern von Benutzerinteraktionssequenzen, die einerseits eine genaue Analyse der Benutzerinteraktionen – speziell unmittelbar vor einem Fehler – ermöglichen, andererseits zu einem späteren Zeitpunkt als Eingabe für eine genaue Wiederholung eines Ablaufs verwendet werden können.
- **Testrahmensysteme** für Systemtests (z. B. FIT; <http://fit.c2.com>) erlauben es dem Anwender, Testfälle tabellenartig als Text zu beschreiben (Eingabe, Ausgabe) und mit der Programmlogik zu verknüpfen. Datengesteuerte Tests lassen sich damit sehr effizient und langfristig stabil automatisieren. Neuere Systeme setzen Webanwendungen zur Kommunikation mit dem Benutzer ein und erlauben dadurch die Anbindung an verschiedene Sprachen und Entwicklungsumgebungen (z. B. FITnesse; <http://www.fitnesse.org>).
- **„capture & replay“-Testsysteme** stehen für interaktionsintensive Anwendungen zur Verfügung (z. B. WinRunner; <http://www.mercury.com>). Sie erfassen – im Gegensatz zu den „unintelligenten“ Monitorprogrammen – während der Bedienung Benutzereingaben und Steuerungsanweisungen (z. B. auch Mausklicks) und verknüpfen diese mit der aufgerufenen Programmlogik. Diese Systeme sind jedoch sehr anfällig für Schnittstellenänderungen (so genügt teilweise eine Änderung der Bildschirmauflösung, um eine Testsequenz erneut erfassen zu müssen).
- **Fehlermeldesysteme** ermöglichen das (meist webbasierte) Erfassen von Testfällen und dazugehörigen, entdeckten Fehlern. Mittels E-Mail o. Ä. werden Entwickler und Projektleiter verständigt sowie die entsprechenden Übersichten aktualisiert. Solche Systeme erlauben eine umfassende Kontrolle, ob gemeldete Fehler auch termingerecht behoben werden (sowie von wem, durch wen getestet etc.). Derartige Systeme sind vielfach in Konfigurations- und Versionsverwaltungssysteme integriert (vgl. Abschnitt 7.2.3).
- **Diagnoseprogramme** ermöglichen die Ermittlung genauer Daten über Hardwarekonfiguration und den Status verwendeter Systemsoftware. Manche Programme erlauben die Ermittlung dieser Daten auch während des Betriebs eines anderen Programms und

können daher parallel während des Ablaufs des zu testenden Programms verwendet werden.

- **Standardüberwachungsprogramme** prüfen ein Programm auf die Einhaltung festgelegter Standards, z. B. Programmierrichtlinien, Kommentarumfang oder Einhalten bestimmter Einschränkungen des zulässigen Sprachumfangs.

13.5 Ergebnisse

13.5.1 Testplan

Ein *Testplan* ist gemäß [IEEE, 1998] ein „Dokument zur Beschreibung von Aufgabenumfang, Vorgehensweise, Ressourcen und Ablauf der beabsichtigten Testaktivitäten“. Er legt die zu testenden Systembereiche und Systemeigenschaften fest sowie die Testaktivitäten, wer diese durchführt und welche Risiken in welchem Umfang in der Planung berücksichtigt sind.

Viele Auftraggeber erwarten vom Auftragnehmer ein Produkt, „das funktioniert“. Ob und wie es getestet wurde, ist für sie zweitrangig. So wichtig auch ein Testplan ist, meistens bleibt er ein internes Dokument, dessen Inhalt und zeitliche und finanzielle Auswirkungen dem Auftraggeber in direkter Form gar nicht verständlich gemacht werden können. Durch die Bestrebungen zur Qualitätssicherung und dabei besonders durch die Verpflichtung zur Prüfung der Qualität bei Zulieferern wird sich diese Einstellung ändern, das ist aber ein sehr langwieriger Prozess.

Folgende Inhalte umfasst ein Testplan:

- Beschreibung des Produktziels und damit des Testziels (ein Prototyp zum Nachweis der Machbarkeit wird anders getestet als ein Turn-Key-Produkt),
- Testumfang (Beschreibung dessen, was und was *nicht* getestet wird; im Allgemeinen durch die Festlegung einer Testsuite in einem eigenen Dokument spezifiziert, vgl. Abschnitt 13.5.2),
- Beschreibung der durch das Testen abgedeckten Risiken (z. B. Probleme beim Netzwerkbetrieb, Datenkonsistenz bei Abstürzen) sowie der offenen Risiken (inklusive durch das Testen entstehende Risiken),
- Beschreibung der Teststrategie,
- Beschreibung des Testansatzes (Vorgehensweise, Hauptaktivitäten, Techniken, Verwendung von Werkzeugen) und der Testkriterien (z. B. für Testabbruch, Testende),
- Beschreibung der notwendigen Testumgebung (Hardware, Software, zugreifbare Datenquellen etc.),
- Ressourcenplan und Ressourcenbelegungsplan (Wer testet welchen Bereich womit? Dies inkludiert auch externe Tester.),
- Ablaufplan mit Meilensteinen und zeitlich aufgeschlüsselter Ressourcenverwendung,

- Festlegung der an den Auftraggeber zu liefernden Testdokumentation (Art, Inhalt und Umfang).

Die Erstellung eines Testplans benötigt viel Zeit. Sie soll daher bereits parallel zur Arbeit an Analyse und Design durchgeführt werden. Hilfreich ist dabei, auf einer stabilen Produktspezifikation (z. B. Pflichtenheft) aufbauen zu können.

Während des Testens muss der Testplan laufend aktualisiert werden. Entscheidende Faktoren für eine manchmal beträchtliche Erweiterung des Testplans sind

- Änderungen der Produkthanforderungen,
- Übersehen von Anwendungsbereichen des Produkts,
- Übersehen von Testklassen bei der Planung (z. B. Belastungstests),
- Übersehen von Fehlerklassen bei der Planung (z. B. Fehlerbehandlung).

13.5.2 Testsuite

Eine *Testsuite* ist ein Verzeichnis von Testfällen, geordnet nach Testklassen. Sie wird daher auch als *Testfallverzeichnis* bzw. nicht ganz sauber als *Testscript* bezeichnet. Die Testsuite bildet die standardisierte Testmenge für einen Testzyklus. Testsuiten werden darüber hinaus für den Qualifikationstest eines Produkts, für Regressionstests sowie für die Abnahme erstellt.

Diese Klassen umfassen

- **Konformanztests**, die vom Programmsystem korrekt verarbeitet werden müssen,
- **Nonkonformanztests**, die das spezifizierte Fehlverhalten ergeben müssen,
- **Qualitätstests** (z. B. Kompatibilitätstests, Zuverlässigkeitstests oder Performanztests, vgl. Abschnitt 13.2.4).

Konformanztests und Nonkonformanztests werden auch als *funktionale Tests* bezeichnet, Qualitätstests als *nichtfunktionale Tests*.

Die Beschreibung eines Testfalls besteht aus folgenden Komponenten:

- zu testendes System bzw. zu testende Komponente,
- Zweck des Tests,
- besondere Anforderungen an die Testumgebung,
- Abhängigkeit von anderen Testfällen bzw. deren Ergebnissen (ggf. durch Vorgabe einer notwendigen Testreihenfolge),
- Eingabebeschreibung (Werte, Dateinamen, geforderte Speicherzustände, Systemeinstellungen, Zustände weiterer notwendiger Programme wie z. B. einer Datenbank, Zeitabhängigkeiten),

- schrittweise Beschreibung des Testablaufs,
- erwartetes Ergebnis (Ausgabewerte, Nachrichten, Ausdrücke, Dateien; inklusive erwarteter Antwortzeiten),
- Kriterien, die festlegen, ob ein Testfall korrekt oder fehlerhaft bearbeitet wurde (evtl. inklusive einer Beschreibung möglichen oder angestrebten Fehlverhaltens).

13.5.3 Testlisten

Testlisten werden zumeist von den Testern für den persönlichen oder gruppeninternen Gebrauch angefertigt und ermöglichen es, eine rasche Übersicht über bestimmte Produkteigenschaften zu bekommen bzw. beim Testen wesentliche Produktbereiche nicht zu vergessen. Im Wesentlichen sollten diese Listen alle aus einer vollständigen Produktspezifikation (z. B. dem Pflichtenheft) erstellt werden können.

Folgende Listen werden u. a. angefertigt:

- Funktionalitätenliste (Liste der erfüllten Anforderungen gemäß Produktstruktur),
- Liste möglicher Fehlermeldungen (nötigenfalls aus dem Code extrahiert),
- Liste aller Eingabemasken und sonstiger Dialoge,
- Liste der vom Programmsystem generierbaren Ausgabemasken, Berichte und Listen,
- Liste aller zu erstellenden Dokumente (v. a. externe, für den Auftraggeber bestimmte; zum Zweck der Evaluierung und des Aufbaus von Know-how jedoch auch interne),
- Liste zum Betrieb notwendiger Hardware- und Softwarekomponenten,
- Liste unterstützter (kompatibler) Hardware und Software.

13.5.4 Fehlerbericht

Ein *Fehlerbericht* ist eine standardisierte Beschreibung eines entdeckten Fehlers zusammen mit allen Angaben, welche die Behebung des Fehlers begünstigen. Ein Fehlerbericht sollte so geschrieben werden, dass der Fehler auch behoben wird, d. h., dass der betroffene Implementierer seine Arbeit auf Grund des Fehlerberichts verrichten kann und sich nicht durch Schreibstil o. Ä. gekränkt fühlt. Entscheidend dazu ist es, den Fehler so zu beschreiben, dass er mit möglichst wenigen, klaren Schritten reproduziert werden kann. Ein nicht reproduzierbarer Fehler sollte nur als Hinweis auf einen möglichen Problembereich weitergegeben werden; eine unmittelbare Behebung eines solchen Fehlers darf der Tester nicht erwarten.

In [Kaner *et al.*, 1999] wird ein Formular für einen Fehlerbericht vorgeschlagen, das folgende Felder enthält:

- Fehlerberichtsnummer,
- getestetes System bzw. Systembereich (inklusive Programmversion),

- Art des Problems (Implementierungsfehler, Designfehler, Verbesserungsvorschlag, Dokumentationsinkonsistenz, Hardwareinkompatibilität, Anfrage wegen unklaren Verhaltens),
- Schwere des Fehlers,
- kurze Problembeschreibung,
- Vorgehen zur Reproduktion,
- Idee/Vorschlag zur Behebung (optional),
- Tester,
- Datum,
- Priorität der Behebung.

Wichtig ist es, den Fehlerbericht sofort zu schreiben, wenn ein Fehler auftritt. Wenn das Fehlverhalten „noch auf dem Bildschirm sichtbar“ ist, können die dem Fehler vorausgehenden Aktivitäten sowie der Fehler selbst viel exakter beschrieben werden als zu einem späteren Zeitpunkt. Viele Fehler werden deshalb nicht behoben, da die Arbeitsschritte des Testers, die den Fehler herbeigeführt haben, zu spät und dadurch zu ungenau protokolliert werden und der Fehler daher nicht mehr reproduziert werden kann.

13.5.5 Fehlerlogbuch

Das *Fehlerlogbuch*, manchmal auch im Wesentlichen synonym mit *Testlogbuch* bezeichnet, ist eine zeitlich geordnete Auflistung aller gefundenen Fehler und ihrer Behebung.

Ein Eintrag im Fehlerlogbuch besteht aus:

- Fehlernummer (eindeutig; entsprechend der Nummer des Fehlerberichts),
- Datum der Fehlermeldung,
- Fehler (Kurzbeschreibung; inklusive Fehlerklasse),
- Lokalität des Fehlers (so genau wie eruierbar),
- Tester,
- Datum der Behebung,
- Implementierer der Behebung,
- Kommentar (zur Behebung bzw. zur Fehlerursache).

14 Produkt-Einführung

Dieses Kapitel beschreibt alle notwendigen Tätigkeiten, die nach der Erstellung eines ausgetesteten Produkts bis zu einem erfolgreichen Projektabschluss geleistet werden müssen. Notwendige Tätigkeiten zur Evaluierung des Projektverlaufs und Archivierung der Projektergebnisse und -erkenntnisse schließen das Kapitel ab.

14.1 Ziel

Das *Ziel der Produkt-Einführung* (*introduction*) ist der Abschluss eines Projekts durch die erfolgreiche Aufnahme des Betriebs eines Softwareprodukts durch den Auftraggeber. Für den Auftragnehmer liegt das Ziel der Produkt-Einführungsphase in der Erweiterung des Projekt-Know-hows durch das Auswerten der Planabweichungen und Erkenntnisse während des Projekts. Oft wird in dieser Phase bereits das Entwicklungsteam des Auftragnehmers verkleinert.

Nachdem ein Softwareprodukt beim Anwender eingeführt worden ist, befindet es sich bei ihm in *Betrieb* (*use*) und unterliegt der *Wartung* und *Pflege* (vgl. beispielsweise [Balzert, 2000b]). Diese Bereiche sind nicht mehr unmittelbarer Bestandteil der Produkt-Einführungsphase, werden aber auf Grund des oftmals fließenden Übergangs in dieses Kapitel eingeschlossen.

14.2 Aufgaben

14.2.1 Lieferung

Die *Lieferung* (*delivery*) umfasst die *Übergabe* (*turnover*) durch den Auftragnehmer sowie die *Übernahme* (*takeover*) durch den Auftraggeber. Dabei wird das Gesamtprodukt – inklusive der gesamten zu liefernden Dokumentation – entweder an den Auftraggeber gesendet oder persönlich übergeben.

Wesentlich ist es, ein geeignetes Medium zu wählen, das einerseits eine effiziente und einfache Installation und Inbetriebnahme des Produkts ermöglicht, aber auch einen professionellen optischen Eindruck macht. Der erste Eindruck des Systems (Verpackung, Labels, Kopier-

qualität der Dokumente etc.) ist oft entscheidend für die Einstellung des Auftragnehmers zum Produkt und damit für die gesamte Produkt-Einführungsphase.

Durch die Lieferung wird dem Auftraggeber signalisiert, dass das Projektergebnis und somit die Lösung seines Problems nun vorliegt und er daher ein brauchbares Produkt in Händen hält. Es ist daher Aufgabe der Lieferung, zusätzlich zur Erbringung eines brauchbaren Produkts den Auftraggeber dabei zu unterstützen, sich mit dem System vertraut zu machen und sich bei der Einführung und beim Einsatz des Produkts wohl zu fühlen. Eine diesbezüglich gewonnene, positive Einstellung zum Produkt durch den Auftraggeber beeinflusst die Verwendung des Produkts und damit auch die Bereitschaft zum positiven Abschluss des Projekts entscheidend.

Vielfach wird seitens des Auftragnehmers vor der Lieferung des Produkts eine schriftliche Stellungnahme vom Projektleiter eingeholt, inwieweit er das Produkt für zuverlässig und lieferfähig hält [Pfleeger, 2001]. *Zuverlässigkeit* wird dabei definiert durch die Wahrscheinlichkeit, dass der Auftraggeber in der ersten Zeit der Anwendung auf (schwere) Fehler stößt. In [Kaner *et al.*, 1999] wird dazu empfohlen, die einzelnen Komponenten des Softwaresystems nach vier Zuverlässigkeitsgraden einzustufen:

1. **Niedrige Zuverlässigkeit** ist gegeben, wenn das Produkt bekannte, schwere Fehler enthält, die der Auftraggeber wahrscheinlich finden wird.
2. **Hohe Zuverlässigkeit** ist gegeben, wenn das Produkt sauber getestet wurde und alle bekannten schweren Fehler beseitigt werden konnten. Die noch enthaltenen Fehler werden sehr wahrscheinlich vom Kunden nicht in der ersten Zeit gefunden.
3. **Mittlere Zuverlässigkeit** liegt zwischen niedriger und hoher Zuverlässigkeit.
4. **Unbekannte Zuverlässigkeit** entsteht entweder durch ungenügendes Testen, sodass die Zuverlässigkeit des Produkts nicht ausreichend eingeschätzt werden kann, oder aber durch extensives Testen, bei dem nichts Außergewöhnliches gefunden wird, aber der Eindruck bleibt, dass trotzdem schwere Fehler im Produkt verblieben sind. [Kaner *et al.*, 1999] empfehlen, die Bedenken in diesem Fall eher nicht laut zu äußern, sofern nicht ein konkreter Vorschlag geliefert werden kann, wie man den Verdacht erhärten oder ausräumen kann.

Auf jeden Fall ist es sehr sinnvoll, beim Auftragnehmer (unternehmensweit oder zumindest projektintern) eine *minimale Zuverlässigkeit* schriftlich festzulegen, die erreicht werden muss, um das Produkt ausliefern zu können. In Streitfällen ist ein solches Dokument zusammen mit den Protokollen und Unterlagen zur Überprüfung dieser minimalen Zuverlässigkeit Entscheidungsgrundlage für das weitere Vorgehen.

Für den Projektleiter ist es wichtig zu verstehen, dass seine Rolle darin besteht, die Unternehmensführung über die mit einer Lieferung zu einem bestimmten Zeitpunkt verbundenen Risiken klar zu informieren. Ob das Produkt unter diesen Rahmenbedingungen ausgeliefert wird, wird normalerweise nicht vom Projektleiter entschieden, sondern von der im Unternehmen dafür zuständigen Abteilung (Verkauf, Marketing etc.). Es spielen dabei nicht nur die Produkteigenschaften eine Rolle, sondern auch die Situation am Markt sowie die Planung des Auftraggebers. Ein fehlerbehaftetes Produkt zeitgerecht auszuliefern ist unter bestimmten Umständen entscheidender als eines mit höchster Zuverlässigkeit zu einem zu späten Zeitpunkt.

14.2.2 Installation und Inbetriebnahme

Die *Installation* (*installation*) eines Produkts bedeutet die Einrichtung eines Produkts in der *Zielumgebung* (*target platform*) des Auftraggebers zum Zweck des Betriebs [Balzert, 2000b]. Dabei wird zwischen *simulierter* (i. e. nicht produktiver) und *echter* (produktiver) *Zielumgebung* unterschieden. Die Installation umfasst:

- die Überprüfung der Zielhardware auf Vollständigkeit und Funktionsfähigkeit,
- die Konfiguration der Hardware und Software (Anschluss der notwendigen Geräte, Einrichten der Netzwerk- und anderer Kommunikationsverbindungen, Einrichten von Dateien und Datenbank-Zugriffsmöglichkeiten etc.),
- kundenspezifische Installation (Installation und Konfiguration der Betriebssystemoptionen, Einspielen und Adaptieren der Software (*Maßschneidern*, *tailoring*), Einrichten der notwendigen Treiber, Konfigurieren der kundenspezifischen Softwareoptionen,
- Prüfung der Geräte samt der darauf verwendeten Software (mit Ausnahme des neu installierten Produkts) auf Funktionsfähigkeit.

Die *Inbetriebnahme* (*invocation test*) ist die Vorbereitung des installierten Produkts für den Regelbetrieb. Er darf nicht mit dem – im Betrieb häufig erfolgenden – *Starten* (*Aufrufen*) des Produkts verwechselt werden. Die Inbetriebnahme umfasst:

- das Starten und Testen jeder Anwendungskomponente des Produkts,
- des Erstellen von Testausdrucken für jede Komponente,
- das erneute Testen der auf der Zielumgebung verfügbaren (Standard-)Programme, um abzusichern, dass deren Betrieb auch parallel oder/und sequenziell mit dem neuen Softwaresystem möglich ist.

Sehr sinnvoll ist es, vor der Installation ein *Backup* der derzeitigen Systemumgebung anzufertigen, um bei einem eventuellen Fehlschlagen der Installation den Zustand der Systemumgebung vor der Installation zuverlässig wieder herstellen zu können. Genauso ist es empfehlenswert, nach der Installation ein Backup der installierten Version anzufertigen (diese enthält nun bereits die korrekten Verzeichnisnamen, Kundendaten etc.). Dieses Backup kann einerseits als definierter Zustand für einen Neubeginn bei Problemen während der Inbetriebnahme dienen, andererseits stellt es auch die Referenzumgebung für während der Inbetriebnahme entdeckte Fehler dar. In diesem Zusammenhang prüft der *Wiederinbetriebnahmetest* (*recovery test*), ob nach einer Unterbrechung das System mit den vorliegenden Beschreibungen wieder in Betrieb genommen werden kann und ob dann noch alle Daten korrekt verfügbar sind.

14.2.3 Schulung

Die *Schulung* (*training*) weist die seitens des Auftraggebers mit dem Produkt befasste Personengruppe (bzw. eine Auswahl daraus) in die geeignete Verwendung der Software ein. Dies

geschieht teilweise bereits vor der Lieferung des Produkts (auf Testversionen, meist beim Auftragnehmer), parallel zur Inbetriebnahme des Produkts sowie vielfach auch noch während der Anfangszeit des Betriebs. Schulung kann durch weitere Aktivitäten, wie z. B. Consulting oder Hotline-Hilfe, unterstützt werden.

Bei der Schulung muss bedacht werden, dass meistens verschiedene *Rollen von Bedienern* des Produkts beim Auftraggeber geschult werden müssen:

- Der *Benutzer* (*Anwender*, *user*) verwendet die Hauptfunktionen des Systems, um die im Lastenheft beschriebenen Anwendungsfälle und Probleme des Auftraggebers zu lösen. Für einen Benutzer ist es beispielsweise jedoch nicht notwendig (es wäre im Gegenteil oft verwirrend) zu wissen, wer mit ihm gleichzeitig mit dem Produkt arbeitet oder wann wo welche Information abgelegt wird.

Die *Benutzerschulung* konzentriert sich daher darauf, die Anwendungsfunktionalität und die notwendige Reihenfolge der Abläufe zu trainieren. Man muss bei dieser Schulung allerdings darauf achten, dass es nicht nur regelmäßige Benutzer des Produkts gibt und dass Benutzer, die das Produkt selten verwenden, viele der Abläufe bis zur nächsten Produktverwendung wieder vergessen. Zusätzlich werden vom Auftragnehmer geschulte Benutzer mit der Zeit durch nicht geschulte ergänzt oder ersetzt. Eine gute Online-Hilfe für unregelmäßige Benutzer sowie Schulungsunterlagen (z. B. ein gut aufgebautes Tutorial) für neue Benutzer sind daher für den kontinuierlichen Betrieb einer Software unverzichtbar. Oft werden diese Dokumente auch von regelmäßigen Benutzern verwendet, um Details aufzufrischen.

- Der *Betreuer* (*administrator*) erhält die Betriebsfähigkeit des Produkts durch die Erteilung von Zugriffsrechten, das Anpassen von Benutzeroberflächen, das Anlegen notwendiger Dateien und Datenbankzugänge und ähnliche unterstützende Tätigkeiten. Vielfach ist er auch für das Sichern und Archivieren von Datenbeständen zuständig.

Die *Betreuerschulung* umfasst daher hauptsächlich die Einweisung in diese unterstützenden Tätigkeiten bzw. ihrer spezifischen Verknüpfung mit dem Produkt. Ein Betreuer muss verstehen, wie das Produkt arbeitet, nicht nur, was das Produkt tut. Da ein Betreuer die Ergebnisse seiner Tätigkeiten vielfach mittels der Durchführung von Benutzeraktivitäten überprüfen muss, ist es sehr sinnvoll, Betreuer auch an der Benutzerschulung teilnehmen zu lassen.

Für Details zur Schulung sei beispielsweise auf [Pfleeger, 2001] verwiesen.

14.2.4 Abnahme und Abschluss

Die *Abnahme* (*acceptance*) ist der rechtskräftige Schritt der Annahme eines Produkts als Ergebnis des Projekts durch den Auftraggeber (bzw. bei Standardsoftware durch eine Zertifizierungsstelle). Eine erfolgreiche Abnahme führt automatisch zum *Abschluss* (*finalization*) des Projekts.

Die Abnahme beim Auftraggeber ist kein umfassender Systemtest, sie dauert im Allgemeinen maximal einen Tag [Kaner *et al.*, 1999]. In [Perry, 2002] werden das Erstellen von Abnahmetestsuiten und Taktiken beim Verhandeln über deren Inhalt mit dem Auftraggeber detailliert beschrieben. Wie beispielsweise in [Balzert, 2000b] angemerkt wird, stellt die

Abnahme immer einen Kompromiss zwischen optimalem („fehlerfreiem“) und akzeptablem (mit tolerierbaren Fehlern behaftetem) Produkt dar.

Tipp: Aus Erfahrung des Autors ist, anders als in mancher Literatur dargestellt (vgl. z. B. [Balzert, 2000b]), eine Abnahme nur nach erfolgreicher Installation und Inbetriebnahme des Produkts in der realen (evtl. simulierten) Zielumgebung des Auftraggebers sinnvoll. Erst wenn der Betrieb des Produkts im Umfeld des Auftraggebers möglich ist, ist dieser zu einem Abschluss des Projekts bereit.

Falls ein größeres System in einem verzweigten Netzwerk installiert werden soll oder während der Abnahme möglicherweise auftretende Fehler den Betrieb beim Auftraggeber beeinträchtigen könnten, erfolgt eine so genannte *Werkabnahme* in einer speziell dafür vom Auftraggeber eingerichteten Testumgebung. Es ist Aufgabe des Auftraggebers sicherzustellen, dass die in der Testumgebung vorherrschenden Bedingungen möglichst dem realen Umfeld entsprechen. Das Risiko einer nicht der (realen) Zielumgebung entsprechenden Testumgebung trägt der Auftraggeber.

Eine *erfolgreiche Abnahme* bedeutet das korrekte Verarbeiten aller Testfälle in der vom Auftraggeber erstellten oder genehmigten Abnahmesuite. Auch nach erfolgreicher Abnahme befinden sich Fehler im Produkt, sowohl bekannte als auch unbekannte. Auch während der Abnahme können Fehler auftreten, die im *Abnahmeprotokoll* dokumentiert werden. Sind diese nach Ansicht beider Seiten nicht betriebsgefährdend, so gilt trotz dieser Fehler die Abnahme als erfolgreich. Diese Fehler – sowie je nach Vereinbarung weitere, im Rahmen der ersten Betriebszeit gefundene Fehler – werden im Rahmen der *Garantie* kostenlos behoben. Die Dauer der Garantie ist frei zwischen Auftraggeber und Auftragnehmer vereinbar. Kurze Zeiten (zwei bis drei Monate) sind dabei üblich, vielfach wird auch keinerlei Garantie vereinbart. Die Behebung der entdeckten sowie im Betrieb gefundenen Fehler erfolgt in diesem Fall im Rahmen der – für den Auftraggeber kostenpflichtigen – Wartung des Produkts.

14.2.5 Wartung

Die *Wartung von Software (maintenance)* umfasst alle Arbeiten zur Produktverbesserung, die nach Abschluss der Entwicklungsarbeiten an einem Softwaresystem vorgenommen werden. Laut [IEEE, 1991] zählen dazu das Beheben von Fehlern, die im Produkt bereits zum Zeitpunkt der Lieferung enthalten waren, sowie Änderungen zur Leistungsverbesserung oder um das Produkt an ein geändertes Umfeld anzupassen. Zu beachten ist dabei, dass zusätzlich zur Software auch Dokumentation, Schulungsunterlagen etc. gewartet werden müssen.

Es ist praktisch unmöglich, ein Softwareprodukt zu entwickeln, das keine Wartung erfordert, zumal es nicht möglich ist, die Korrektheit eines Softwaresystems durch Tests nachzuweisen. Viele Fehler werden erst beim tatsächlichen Einsatz erkannt und können daher erst nach den Entwicklungsphasen beseitigt werden. Häufig kommen auch während des Betriebs neue Benutzeranforderungen dazu, die ebenfalls Systemänderungen nach sich ziehen. In diesem Sinn ist der Wartungsbegriff für Software teilweise abweichend von dem für andere technische Produkte definiert, da bei diesen Produkten Wartung als Reparatur bei Abnützungserscheinungen definiert wird.

Abbildung 14.1 (aus [Mayr, 1999]) zeigt, wie der Projektentwicklungszyklus um die „Phasen“ *Betrieb* und *Wartung* ergänzt werden kann. Der Betrieb wird dabei der Auftraggeberseite

zugeordnet, während die Wartung auf Auftragnehmerseite platziert ist. Auf diese Weise wird verdeutlicht, dass einerseits viele Entwicklungsanforderungen nicht aus dem Wunsch nach neuen Softwaresystemen entstehen, sondern aus dem Betrieb bestehender. Andererseits durchlaufen auch viele Wartungsarbeiten (vor allem solche, die größere Änderungen bedingen) die gleichen Projektentwicklungsphasen wie Neuentwicklungen.

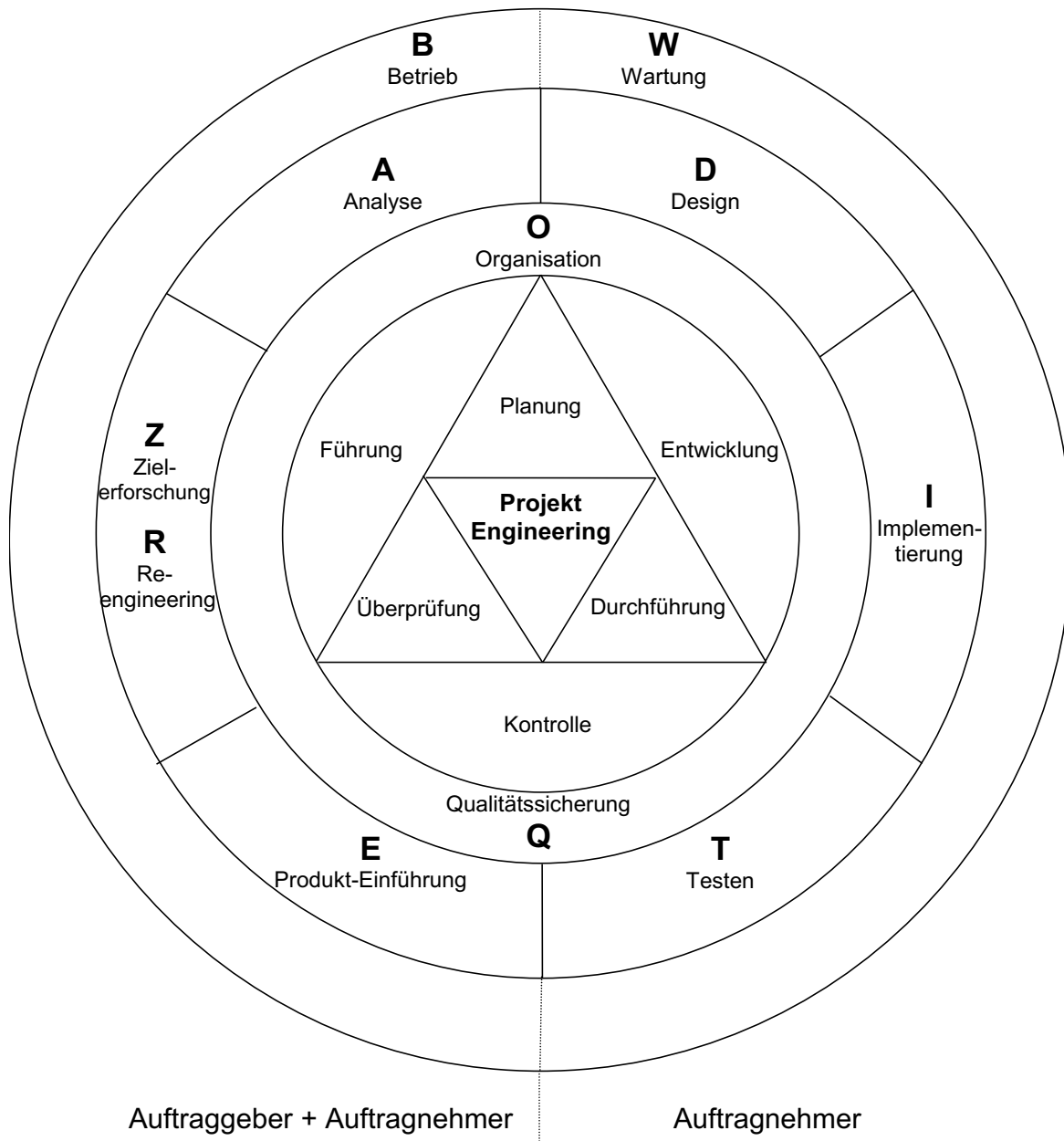


Abbildung 14.1: Ergänzung des Projektentwicklungszyklus um Betrieb und Wartung

Die Wartung im weiteren Sinn, wie oben angeführt, wird in der Literatur teilweise in *Wartung* (im engeren Sinn) und in *Pflege* eingeteilt (vgl. z.B. [Sneed, 1988]), wobei die Wartung die korrigierenden Tätigkeiten umfasst und die Pflege die progressiven (i.e. erweiternden) Tätigkeiten.

Aktivitäten der *Wartung von Software* sind:

- **Stabilisierung:** Die Stabilisierung dient der Lokalisierung und Behebung von Fehlerursachen auf Grund der bekannten Fehlerwirkung. Da es unmöglich ist, alle Programmpfade eines komplexen Softwaresystems und alle möglichen Kombinationen von Eingabedaten zu testen, werden Fehler oft erst während des Betriebs von Softwareprodukten erkannt. Sie können daher nur im Rahmen der Softwarewartung behoben werden. Fehler können jedoch auch im Rahmen der Wartung ins System eingebracht werden. Diese durch Wartung verursachten Fehler, die so genannten *second level defects*, machen nach kurzer Zeit die Mehrzahl der Fehler aus, wie [Grady, 1992] anmerkt. Grund dafür ist die schlechte Programmierung (bzw. das schlechte Design) und die dadurch verursachte Fehleranfälligkeit des ursprünglichen Softwareprodukts.

Die Stabilisierung ist ereignisgesteuert, d.h., schwer planbar und vom Aufwand her schlecht kontrollierbar, da der Zeitpunkt nicht vorhersehbar ist, zu dem ein Fehler das erste Mal auftritt.

- **Optimierung:** Neu eingesetzte Software ist nicht nur unzuverlässig, sie verbraucht auch mehr Ressourcen (Zeit, Speicher, ...) als zur Erfüllung ihrer Aufgaben notwendig wären. Der Grund ist, dass Softwareprodukte selten vor der ersten Freigabe optimiert werden. Sobald ein Produkt funktionsfähig ist, wird es freigegeben; die Optimierung (Leistungsverbesserung) bleibt der Wartung vorbehalten.

Zur Optimierung gehören alle Aktivitäten, die dazu dienen, die Leistung des Produkts zu verbessern oder den Ressourcenverbrauch zu reduzieren. Zum Teil sind auch Restrukturierungen erforderlich, um die Leistungsverbesserungen zu erreichen [Balzert, 2000b].

Aktivitäten der *Pflege von Software* sind:

- **Anpassung:** Anpassungen werden bedingt durch Änderungen im Projektumfeld, Änderungen in der technischen Umgebung (z.B. Versionsänderung von Systemsoftware) oder Änderungen in der Funktionalität bzw. den Rahmenbedingungen (z.B. durch Gesetzesänderungen). Zusätzlich sind vielfach Schnittstellen anzupassen: Erst der Betrieb eines Softwaresystems zeigt die tatsächliche Güte der Benutzerschnittstelle und der Schnittstellen zu anderen Systemen.

Man spricht in diesen Fällen nicht von einer Fehlerkorrektur, weil der Grund für die Wartungstätigkeit nicht eine Abweichung des Systemverhaltens von dem in der Anforderungsdefinition spezifizierten Verhalten ist, sondern nachträgliche Korrekturwünsche der Benutzer. Ausgangsbasis ist jedenfalls ein konsistentes Produkt, in das gezielt und unter Beibehaltung der Konsistenz Änderungen (und Erweiterungen) eingebracht werden.

- **Erweiterung:** Mit der Dauer des Einsatzes eines Softwaresystems kommen laufend neue Benutzerwünsche dazu, an deren Notwendigkeit die Benutzer oder Realisierbarkeit die Entwickler anfangs gezweifelt haben. Auch organisatorische Umstellungen oder sonstige innerbetriebliche Veränderungen und der Einsatz anderer Softwareprodukte bedingen in vielen Fällen neue Anforderungen an ein Softwareprodukt.

Die Bandbreite bei Erweiterungen kann von kleinen bis zu großen Modifikationen gehen, ist aber vom Umfang und vom Zeitpunkt her jedenfalls planbar.

Der englische Begriff „*maintenance*“ umfasst im Allgemeinen immer beide Tätigkeiten der Produktverbesserung, Wartung und Pflege. In [Lientz und Swanson, 1980] werden die Stabilisierung mit „*corrective maintenance*“, die Anpassung mit „*adaptive maintenance*“ und die Optimierung und Erweiterung mit „*perfective maintenance*“ bezeichnet und kostenmäßig mit 17 %, 18 % bzw. 65 % beziffert.

Manche Produktverbesserungen sind jedoch nicht eindeutig in eine der drei Klassen einzuteilen (z. B. Optimieren der Benutzerschnittstelle durch Hinzufügen von Befehlen, die Bedienungsschritte zusammenfassen). Analysiert man die in der Literatur durchaus verschieden angegebenen Aufwandsverteilungen (vgl. z. B. [Martin und McClure, 1983]), so erkennt man, dass durchgehend der meiste Aufwand auf Pflegeaktivitäten – Anpassungen und Erweiterungen – entfällt (60 bis 80 % laut [Balzert, 2000b]).

Eine einigermaßen genaue Schätzung der Kosten für die Wartung und Pflege eines Softwareprodukts ist praktisch unmöglich, da die Gründe für die Wartung bzw. Pflege kaum a priori abzusehen sind und weitgehend außerhalb des Einflussbereichs der Softwareentwickler liegen.

Tip: In der Praxis werden für Softwarewartung/-pflege pro Jahr vielfach 7 bis 15 % der Entwicklungskosten angesetzt.

In den vergangenen Jahren hat sich das Wartungsproblem in vielen Fällen dadurch verschärft, dass Entwickler und Betreiber von Anwendungssoftware gezwungen sind, diese häufig an neue Versionen der Basissoftware anzupassen. Dabei ist es unerheblich, ob es sich um Betriebssysteme, Grafikpakete, Netzwerksoftware oder Ähnliches handelt. Der Aufwand für diese Art von Anpassung lässt sich auch kaum dadurch umgehen, dass man die Basissoftware „einfriert“, da die Produktpolitik der betreffenden Hersteller dazu führen kann, dass man dann irgendwann auch den Änderungsdienst für den Anschluss neuer Peripheriegeräte verliert oder wichtige Softwareergänzungen nicht nutzen kann, da diese jeweils die neueste Version der betreffenden Basissoftware voraussetzen [Elzer, 1994].

Folgende (präventive) Maßnahmen können die Wartung und Pflege erleichtern:

- Definition von Projektstandards und Qualitätssicherungsmaßnahmen,
- Verwendung von Standardkomponenten und Standardbibliotheken,
- Betonung von Verständlichkeit, Modularität und Erweiterbarkeit sowie Zurückstellung fraglicher Optimierungsentscheidungen,
- Schutz von Daten vor unberechtigt bzw. inkompetentem Zugriff,
- Achten auf guten Programmierstil,
- Wertlegung auf systematische Testvorbereitung sowie genaue Dokumentation der Testbeispiele und des Testvorgangs.

In der Regel werden Wartungskosten unterschätzt (die Pflegekosten sind teilweise durch separat verrechnete Zusatzaufträge besser steuerbar). Die Untersuchungen in [Lientz und Swanson, 1980] decken sich mit anderen Literaturquellen in der Angabe, dass die Kosten für Wartung und Pflege mehr als 50 % der Gesamtkosten – gesehen über die gesamte Produktentwicklungs- und Produktlebenszeit – ausmachen. Es lohnt sich daher, dem Qualitätsmerkmal

Wartungsfreundlichkeit besondere Beachtung zu schenken und Softwaresysteme so zu konstruieren, dass sie wartungsfreundlich sind.

Die Produktivität der Wartung und der Pflege kann verbessert werden, indem Wartungsaktivitäten und Pflegeaktivitäten voneinander getrennt werden. Diese Trennung ist jedoch oftmals schwierig durchzuführen und durchzuhalten, wie [Balzert, 2000b] anmerkt. Wenn ein Programm geändert wird, dann werden in der Regel Fehlerkorrekturen, Optimierungen, Anpassungen und Erweiterungen in einem Durchgang ausgeführt. Da aber Wartungs- und Pflegeaktivitäten unterschiedliche Charakteristika besitzen, sollte eine Trennung auf jeden Fall angestrebt werden.

Betrachtet man die Charakteristika der Pflegeaktivitäten genauer, so stellt man fest, dass Anpassungen und Erweiterungen eines Produkts auch charakteristisch für Weiterentwicklungen bzw. für neue Versionen von Produkten sind. Es ist daher sinnvoll – abgesehen von minimalen Änderungen –, alle Pflegeaktivitäten den normalen Projektentwicklungsprozess durchlaufen zu lassen [Balzert, 2000b], wobei eine gute Versionsverwaltung eine entscheidende Rolle spielt.

Der Aufwand für Wartung und Pflege eines Produkts wächst durchschnittlich um 10 % pro Jahr [Grady, 1992]. Dies wird vor allem durch die Bereitstellung zusätzlicher Funktionalität bedingt. Ältere Softwareprodukte tendieren daher dazu, umfangreicher und schwerer wartbar zu sein. Ab einem bestimmten Zeitpunkt stellt sich daher die Frage,

- ob das Produkt weiter gewartet und gepflegt werden soll,
- ob das Produkt saniert werden soll („*Reengineering*“) oder
- ob es durch ein neues Produkt ersetzt werden soll.

Effektivität und Effizienz von Wartung und Pflege müssen daher laufend überprüft werden, um den Zeitpunkt zu erkennen, wann ein Softwareprodukt das Ende seines Produktlebenszyklus erreicht hat.

14.2.6 Evaluierung

Die *Evaluierung* (*evaluation*) ist die auftragnehmerinterne Nachanalyse und Nachkalkulation des Projekts. In ihr werden die tatsächlichen Werte des Projektaufwands mit den geplanten Werten verglichen sowie die Gründe für die Abweichungen erforscht. Sich daraus ergebende Konsequenzen für den Entwicklungsprozess werden dokumentiert sowie den Aufwandsschätzungen zu Grunde liegende Schätzparameter adaptiert. Auf Grund der Planabweichungen werden die Liste der Projektrisiken sowie deren Wahrscheinlichkeiten adaptiert.

Das Projekt wird kostenmäßig abgerechnet und die Kosten für die Planänderungen werden ermittelt. Aus diesen Werten wird das Modell für die Budgetierung neuer Projekte aktualisiert. Je nach Projektausgang werden Konsequenzen gezogen bzgl. Projektorganisation, des Einsatzes bestimmter Mitarbeiter sowie der Rahmenbedingungen für eine Auftragsannahme. Im Fall positiver Projektergebnisse werden Prämien ausbezahlt. Die Zeitplanung der Mitarbeiter wird normalisiert (Abbau von Zeitausgleich, Adaptierung der Urlaubsplanung).

Soweit es die Unternehmenskultur zulässt, sind bei der Evaluierung alle Projektmitglieder unabhängig von ihrer Stellung in der Organisation gleichberechtigt anzuhören, möglichst moderiert von einer neutralen, nicht ins Projekt involvierten Person. Auf Grund der oftmals hohen Vertraulichkeit der Informationen ist es vielfach jedoch nicht möglich, einen unternehmensfremden Moderator beizuziehen.

14.2.7 Archivierung

Meist ist nach den vielfach eher hektischen letzten Projektphasen (Implementierung, Testen, Produkt-Einführung) die interne Projektdokumentation sowie die Dokumentationsorganisation allgemein vernachlässigt. Die *Archivierung* (*archivation*, manchmal auch *tidying-up* – „Zusammenräumen“ – genannt) umfasst daher das Durchforsten aller elektronischen Dokumente und Papierdokumente, das Eliminieren veralteter Versionen sowie das Aktualisieren der Struktur bzw. der Inhalte der Projektbibliothek.

Das Ziel ist es, die gesamten Projektunterlagen so zusammenzustellen, dass gegebenenfalls andere Mitarbeiter die Arbeit ohne Rückfragen fortführen können. Daher ist es entscheidend, auch das bisher nicht schriftlich festgehaltene („in den Köpfen der Mitarbeiter gespeicherte“) Know-how zu dokumentieren. Auftraggeberspezifische Information wird gelöscht.

Die saubere Archivierung der Projektunterlagen ist sowohl für einen etwaigen Wartungsauftrag für das Produkt als auch zur raschen Wiederauffindung von Know-how für zukünftige Projekte wichtig. Wenn diese Archivierung zeitlich im Rahmen der Produkteinführung erfolgt, so sind einerseits vielfach die direkt involvierten Mitarbeiter noch greifbar, andererseits sind diese gedanklich mit dem Projekt und dessen Organisationsumfeld vertraut und können so effizienter ein sauber geordnetes Projektarchiv erstellen.

14.3 Techniken

14.3.1 Umstellung

Die *Umstellung* (*transfer*) umfasst die Übertragung aller Datenbestände, die Adaptierung der Hardware und Systemsoftware sowie anderer interagierender Software von der bisherigen Betriebsumgebung in die durch das neu eingeführte Produkt entstehende. Speziell die Datenbestände sowie Schnittstellen müssen dazu oftmals geeignet aufbereitet werden. Hierfür sind meistens eigene Konvertierungsprogramme zu entwickeln, die natürlich ebenfalls getestet und in Betrieb genommen werden müssen. Speziell sind dabei Prüfmöglichkeiten für die Richtigkeit der umgestellten Datenbestände zu berücksichtigen; manchmal ist auch eine bidirektionale Umstellmöglichkeit vorzusehen.

Gerade das Übertragen sich laufend ändernder („lebender“) Datenbestände kann dabei viel Aufwand verursachen und bedarf einer genauen Zeitplanung. Folgende Strategien der Umstellung werden angewendet:

- Die *direkte Umstellung* bewirkt die unmittelbare und vollständige Umstellung vom alten System zum neuen zu einem bestimmten Stichtermin. Das alte System wird dabei gestoppt, die Konvertierungen vorgenommen und das neue System auf Basis

des aktuellen Datenbestandes gestartet. Diese Vorgehensweise bedeutet den geringsten unmittelbaren Zusatzaufwand, ist aber auch am risikoreichsten, da im Notfall nicht auf das alte System zurückgegriffen werden kann. Die direkte Umstellung kann bei stabilen Datenbeständen und bei Systemen mit einfachen Rücknahmemöglichkeiten von Operationen durchgeführt werden.

- Die *parallele Umstellung* behält zusätzlich zum Arbeiten mit dem neuen System das alte System bei, wobei alle Datenbestände und notwendigen Schnittstellen verdoppelt werden. Der Vorteil dieser Vorgehensweise ist, dass man die Ergebnisse des neuen Systems direkt mit den Ergebnissen des alten Systems vergleichen kann. Außerdem kann im Fall des Scheiterns der Umstellung sofort zur alten Version zurückgekehrt werden. Demgegenüber stehen die Nachteile, dass der Parallelbetrieb oft großen Aufwand bei der Duplizierung der Daten und Schnittstellen bedeutet, dass die Synchronisation extern beeinflusster aktiver Datenbestände schwierig ist und dass für die Zeit des Parallelbetriebes sowohl seitens der Hardware als auch seitens des Personals doppelte Ressourcen benötigt werden. Außerdem kann es bei dem Personal, das parallel mit dem alten System weiterarbeiten muss, zum Gefühl der Zurückgesetztheit gegenüber den mit dem neuen System betrauten Mitarbeitern und damit zu Motivationsproblemen kommen, da bei erfolgter Umstellung deren Arbeit und auch deren Know-how nicht weiter benötigt wird. Die parallele Umstellung erfolgt bei kritischen Anwendungen und bei Datenbeständen, die nicht oder nur schwer in einem Testumfeld simuliert werden können.
- Die *testweise Umstellung* simuliert das umgestellte Umfeld in einem Testbetrieb, währenddessen der produktive Betrieb mit dem alten System weiter geführt wird. Der Testbetrieb ist nicht in allen Teilbereichen mit dem aktuellen Datenbestand abgestimmt bzw. arbeitet auf nicht aktuellen Datenbeständen, sodass keine Synchronisationsprobleme mit aktiven Datenbeständen entstehen. Der Vorteil beim Arbeiten mit nicht aktuellen Datenbeständen ist, dass man bereits aus dem Betrieb mit dem alten System die Sollergebnisse weiß und diese unmittelbar mit den Ergebnissen des neuen Systems vergleichen kann.

Oftmals ist für die Umstellung auf das neue System ein schrittweiser Übergang vom Testumfeld in das reale Betriebsumfeld möglich, sodass das neue System Schritt für Schritt in (Real-)Betrieb genommen werden kann. Zusätzlich zum Aufwand für die Erstellung der Umgebung für den Testbetrieb fallen auch hier die Kosten für Geräte- und Mitarbeiterressourcen an, die während der Umstellungszeit parallel für das alte System und das neue System benötigt werden. Die testweise Umstellung erfolgt bei schwierig zu testenden Anwendungen sowie bei sensiblen Datenbeständen.

14.3.2 Probetrieb

Der *Probetrieb* (*Probelauf*, *test run*) ermöglicht dem Auftraggeber, das Produkt zu evaluieren. Folgende Tests sind dazu üblich und werden jeweils auf dem tatsächlich für den Einsatz vorgesehenen System durchgeführt:

- **Installationstest:** Ein Installationstest dient dazu, die erfolgreiche Installation des Systems zu überprüfen. Dazu wird eine bestimmte Menge an Tests ausgeführt, die sich oftmals auf Qualitätstests konzentrieren (z. B. Kompatibilitätstests).

- **Leistungstest (Benchmarktest):** Der Leistungstest umfasst spezielle Testfälle, die darauf abzielen, das Systemverhalten und die Systemperformanz auszuloten. Geprüft werden z. B. die Belastung des Rechners und des Netzwerks, die Antwortzeiten bei Dialogen und die Zuverlässigkeit über einen längeren Zeitraum.
- **Pilottest:** Bei einem Pilottest wird das System, im Unterschied zu Leistungstests, für Anwendungsfälle des täglichen Gebrauchs getestet. Er erlaubt dem Benutzer, die von ihm häufig benötigten Systemfunktionen in der im Einsatz üblichen Reihenfolge durchzuführen und dabei auftretende Fehler zu dokumentieren. Oft wird dazu vom Auftraggeber eine Menge von Funktionalitäten festgelegt, die in diesen „Alltagsbeispielen“ enthalten sein müssen. Für einen Pilottest sollen vom Auftraggeber typische Benutzer des späteren Anwendungsumfeldes ausgewählt werden.

Der Probetrieb zielt darauf ab, die vollständige Funktionsfähigkeit des installierten Systems sowie die korrekte Durchführung aller Funktionalitäten in den realen, tatsächlichen Umgebungsbedingungen zu überprüfen. Wenn der Auftraggeber mit den Ergebnissen zufrieden ist bzw. bei Abweichungen deren Gründe ermittelt und dokumentiert wurden, wird der Probetrieb beendet. Ein erfolgreicher Probetrieb ist Voraussetzung für die Abnahme, wobei der Auftraggeber nicht ein fehlerfrei funktionierendes, sondern ein brauchbares System erwarten darf und Abweichungen in Form von Fehlern in Bezug auf die Beeinträchtigung des täglichen Gebrauchs klassifiziert werden müssen.

14.3.3 Kontinuierliche Freigabe

Viele moderne Softwaresysteme weisen Datenbankinhalte (z. B. Datamining- oder Datawarehousingsysteme) oder Webinhalte (Informationsbasen, Wissensbasen) als integralen Bestandteil der Anwendung auf. Dabei ist der Wert der Anwendung (das Programm selbst) untrennbar mit der Aktualität des Dateninhalts (dem „content“) verknüpft. Teilweise bestimmt der Dateninhalt selbst (bzw. dessen Aufbau durch die Benutzer) die weitere Entwicklungsrichtung des Programms. Konsequenterweise lässt sich bei derartigen Anwendungen der Übergang von der Entwicklung zur Wartung vielfach nicht terminisieren; dies ist auch meist nicht sinnvoll. Das Produkt wird daher „kontinuierlich freigegeben“.

In [Pressman, 2000] wird zur Beherrschung dieser *kontinuierlichen Freigabe* (*continuous release, continuous delivery*) vorgeschlagen, ein inkrementelles Prozessmodell festzulegen, das „... dem Entwicklungsteam erlaubt, den Fokus eines Inkrements einzufrieren, sodass eine betriebsfähige Produktversion einer Anwendung generiert werden kann. Das nächste Inkrement wird eventuell Fokusänderungen adressieren, die durch ein Review des vorangegangenen Inkrements vorgeschlagen wurden. Sobald aber das zweite Inkrement festgelegt wurde, wird der Fokus erneut eingefroren.“ [Pressman, 2000] Diese Technik ist nichts anderes als die übliche Freigabepolitik traditioneller Softwareprojekte, nur dass die Zeit zwischen zwei Produktversionen nicht drei bis sechs Monate beträgt, sondern oft nur wenige Tage.

Wartung und Pflege sind insbesondere dann schwierig, wenn die Anwendung durchgehend verfügbar sein muss („24×7 operation“), z. B. bei Webportalen. Bei der Entwicklung derartiger Systeme ist entscheidend, ob trotz der knappen Termine Anforderungen und Designentscheidungen (inklusive Begründungen!) schriftlich festgehalten wurden, damit Entwicklungspfade und die Änderungshistorie nachvollzogen werden können. Die Verwendung eines Werkzeugs für die Konfigurationsverwaltung ist dabei essenziell, siehe Abschnitt 7.2.3.

14.3.4 Abnahmetest

Der *Abnahmetest* (*acceptance test*) ist ein Gesamttest unter realen Einsatzbedingungen, der prüft, ob das Produkt die im Lastenheft spezifizierten Anforderungen des Auftraggebers erfüllt. Der Abnahmetest kann entweder vom Auftraggeber selbst durchgeführt werden oder aber vom Auftragnehmer unter Beisein des Auftraggebers in dessen Einsatzumfeld. Er wird im Allgemeinen mit Echtdaten durchgeführt.

Der Abnahmetest erfolgt gegen eine vorher festgelegte Menge von Testfällen, die so genannte *Abnahmesuite*. Diese Suite wird vom Auftraggeber zusammengestellt, wobei er dazu auch (Beta-)Testfälle des Gesamttests vom Auftragnehmer übernehmen oder abwandeln kann. Im Allgemeinen umfassen die Testfälle Beispiele aus dem tatsächlichen Einsatzbereich beim Auftraggeber.

Die Abnahmesuite muss dem Auftragnehmer rechtzeitig vor Beginn der Abnahme übermittelt werden, damit dieser sie prüfen, gegebenenfalls die Entfernung oder Modifikation bestimmter Testfälle verlangen und danach den Abnahmevortest durchführen kann, vgl. Abschnitt 13.2.5. Sind im Pflichtenheft oder den Verträgen bestimmte Kriterien für die Abnahme allgemein und die Abnahmesuite im Speziellen festgelegt, so obliegt es dem Auftragnehmer, die Einhaltung dieser Kriterien zu prüfen.

Das Ziel des Abnahmetests ist eigentlich die Überprüfung, dass das gelieferte Produkt die im Lastenheft (und den Ergänzungsprotokollen) beschriebene Funktionalität aufweist. In der Realität wird dadurch aber auch aufgedeckt, wenn zwischen beschriebenen und tatsächlich gewünschten (oder mittlerweile adaptierten) Wünschen Unterschiede herrschen [Pfleeger, 2001]. Gleichzeitig stellt sich spätestens jetzt heraus, in welchen Bereichen sich vom Auftragnehmer zugesagtes Systemverhalten (Funktionsumfang, Performanz etc.) nicht im vollen Umfang hat realisieren lassen. Zum Zeitraum der Abnahme wird daher verstärkt über tatsächlich notwendige Anforderungen diskutiert und darüber, wie ausständige Funktionalitäten priorisiert werden sollen. Für diese Aufgabe ist daher genügend Zeit einzuplanen; die Abnahme ist *kein punktuelles Ereignis*!

Auch wenn der Auftraggeber keine Zeit für die Abnahme aufwenden möchte, so ist er doch unbedingt in den Abnahmetest mit einzubeziehen, weil [Suhr und Suhr, 1993]

- nur er die notwendige Funktionalität und deren Adäquatheit feststellen kann,
- zur sinnvollen Durchführung eines Abnahmetests Echtdaten unter realen Bedingungen getestet werden müssen,
- die Akzeptanz der Benutzerschnittstelle nur von den tatsächlichen Benutzern getestet werden kann,
- die Brauchbarkeit der Benutzerdokumentation nur von den Benutzern beurteilt werden kann.

Beim Abnahmetest sollen folgende Punkte beachtet werden (diese stammen teilweise aus [Frühauf *et al.*, 1995]):

- Der Abnahmetest erfolgt beim Auftraggeber durch den Auftraggeber auf Basis des Lasten- bzw. Pflichtenhefts.

- Die Vorgehensweise bei der Abnahme sowie die Kriterien für eine erfolgreiche Abnahme sollten im Pflichtenheft bereits festgelegt worden sein.
- Die Abnahmesuite ist vom Auftraggeber rechtzeitig vor der Abnahme zu liefern. Ein Meilenstein dafür ist bei der Projektplanung vorzusehen.
- Die Benutzerdokumentation (sowie alle sonstige benötigte Produktdokumentation) ist in das Testen mit einzubeziehen.
- Der Abnahmetest dient auch zur Prüfung der Erfüllung der im Lastenheft festgelegten Qualitätsmerkmale des Produkts samt deren Priorisierung.
- Der Abnahmetest erstreckt sich über einen längeren Zeitraum.
- Der Verlauf des Abnahmetests sowie die gefundenen Fehler und Beanstandungen werden in einem Protokoll (*Abnahmeprotokoll*) festgehalten.
- Wird vom Auftraggeber neben dem Abnahmetesten freies Testen mit dem Produkt durchgeführt, so sind dabei entdeckte Probleme und Fehler *nicht* abnahmewirksam. Diese Fehler und die sie verursachenden Testfälle sind jedoch genau zu dokumentieren, sodass im Fall späterer Garantie- oder Wartungsarbeiten die Fehler nachvollzogen und behoben werden können.

Der Abnahmetest endet mit einer Abnahme-Schlussbesprechung, bei der die dokumentierten Fehler auf ihre Auswirkungen hin besprochen und entsprechend gewichtet werden. Sind die Fehler tolerierbar oder können sie nachgebessert werden, so erfolgt die Abnahme des Produkts durch den Auftraggeber [Balzert, 2000b]. Der Auftraggeber muss sich dabei bewusst sein, dass es bei einem Abnahmetest nicht möglich ist, ein Produkt auch nur annähernd vollständig zu testen.

14.3.5 Nachkalkulation

Die *Nachkalkulation* (*post analysis*) umfasst den Soll/Ist-Vergleich von Arbeitsaufwand, Zeit, Abläufen und Terminen, Personaleinsatz und Ressourcenbelegung sowie Kosten und Finanzierung (vgl. Abschnitt 10.2) mit dem Ziel, die Ursachen der Abweichungen zu ermitteln. Als Ergebnis werden einerseits die Kalkulationsvorschriften und Parameter für die Aufwandsschätzverfahren zur Planung adaptiert. Bei größeren Abweichungen im Vorgehen kann auch eine Adaptierung des Entwicklungsprozesses notwendig sein.

14.4 Werkzeuge

Spezielle Werkzeuge zur Durchführung der Einführungsaktivitäten existieren derzeit nicht. Die einsetzbaren Werkzeuge unterstützen hauptsächlich die Überwachung des Systemverhaltens sowie die Protokollierung der Benutzeraktivitäten. Eingesetzt werden beispielsweise

- **Versionsverwaltungswerkzeuge** zum Zusammenbau korrekter Produktversionen,
- **Installationsprogramme (installer)** zur Installation und Konfiguration des Produkts samt all seinen Komponenten und Hilfsprogrammen,

- **Simulatoren** zur Ermöglichung des Testens einzelner Komponenten des Produkts sowie zur Nachbildung externer Komponenten und Geräte,
- **Monitorprogramme (data capturer)** zur Aufzeichnung von Datenflüssen und Steuerungssequenzen, um Abfolgen von Benutzeranweisungen nachvollziehbar zu machen,
- **Schnittstellenprüfer (interface checker)** zur Analyse von Inhalten beim Datenaustausch zur Laufzeit,
- **Logfile-Ersteller** zum Festhalten der vom Produkt durchgeführten Schritte,
- **Protokollierensysteme (protocoller)** zur Zusammenstellung und Aufbereitung der vom System und Benutzer in zeitlicher Abfolge getätigten Aktivitäten.

14.5 Ergebnisse

14.5.1 Betriebsversion

Die *Betriebsversion eines Produkts (production version)* umfasst die erste freigegebene, installierte, in Betrieb genommene und abgenommene Version des Softwaresystems (*release*) zusammen mit aller dazugehörigen Dokumentation. Der Begriff „*Vollversion*“ soll nicht verwendet werden, da meistens die erste Betriebsversion nicht den vollen Funktionsumfang aufweist, jedoch im Rahmen der Abnahme als für den Betrieb tauglich eingestuft wurde.

Bei der mit der Betriebsversion mitgelieferten Dokumentation ist besonders darauf zu achten, dass die Benutzerdokumentation in der letztgültigen Version beigelegt wird. Zusätzlich ist vielfach als Qualitätsnachweis die Testdokumentation (zumindest der Testplan, die Testsuite und ein Protokoll der erfolgreichen Verarbeitung der Testsuite) mitzuliefern. Im Fall der Auslieferung des Quellcodes muss eine aktuelle Systemdokumentation mitgeliefert werden, die spezielle Hinweise für Wartungsarbeiten enthält.

Nur im Notfall (essenzielle Hinweise in letzter Minute) sollen Supplements („Readme-Dateien“) der Betriebsversion elektronisch oder als Ausdruck beigelegt werden. Jedenfalls ist es aber besser, den Benutzer auf diese Weise über kurzfristig notwendige Änderungen (z. B. bei Menüs, Dialogboxen oder Befehlsnamen, neu eingeführten Fehlermeldungen), entdeckte Fehler oder Beschränkungen des Programms (z. B. Grenzen bei der Anzahl handhabbarer Datensätze) zu informieren als ihn diese selbst (und unvorbereitet) finden zu lassen.

14.5.2 Installations- und Inbetriebnahmeprotokoll

Das *Installations- und Inbetriebnahmeprotokoll* (auch *Einführungsprotokoll* genannt) dokumentiert chronologisch alle Vorkommnisse während der Installation und der Inbetriebnahme. Als Gliederung kann dazu der geplante Einführungsablauf dienen. Während die Installation meistens allein vom Auftragnehmer durchgeführt und protokolliert wird, werden die Auftraggeberaktivitäten bei der Inbetriebnahme manchmal separat in einem *Bedienerprotokoll* festgehalten. Dieses Bedienerprotokoll wird vielfach auch während der ersten Betriebstage

weitergeführt, um Fehler und die Schritte, die zu einem Fehler geführt haben, nachvollziehbar zu dokumentieren, aber auch, um aus Bedienerfehlern diejenigen Bereiche feststellen zu können, wo Schulungen bzw. Nachschulungen notwendig sind.

14.5.3 Abnahmeprotokoll

Das *Abnahmeprotokoll* dokumentiert den Ablauf der Abnahme, die Charakteristika der Abnahmeumgebung sowie alle vorgenommenen Eingaben, durchgeführten Tests und dadurch erhaltenen Ergebnisse. Es wird in jedem Fall erstellt, auch wenn die Abnahme nicht als erfolgreich eingestuft wird. Das Abnahmeprotokoll wird vom Auftragnehmer und vom Auftraggeber unterzeichnet. Falls die Ansichten über den Verlauf der Abnahme (insbesondere die Interpretation der Ergebnisse) zwischen Auftraggeber und Auftragnehmer so sehr voneinander abweichen, dass ein gemeinsam unterzeichnetes Abnahmeprotokoll nicht zu Stande kommt, so werden die verschiedenen Standpunkte in einer *Sachverhaltsdarstellung* festgehalten, die als Grundlage zur Klärung auf gerichtlichem oder außergerichtlichem Wege (durch eine Schlichtungsstelle) dienen kann.

14.5.4 Abschlussbericht

Der *Abschlussbericht* (*final report*) eines Projekts ist ein interner Bericht seitens des Auftragnehmers und wird von allen Mitarbeitern unter Anleitung durch den Projektleiter erstellt. Er fasst die wichtigsten Ereignisse des Projekts und die daraus gewonnenen Erkenntnisse zusammen. Dabei sind sowohl die negativen als auch die positiven Verläufe im Projekt festzuhalten. Manchmal wird dieser Bericht auch als Protokoll anlässlich einer *Abschlussbesprechung* verfasst. Wesentliche Bestandteile sind der Bericht der Qualitätssicherung sowie die aus diesem Bereich gewonnenen Erkenntnisse für zukünftige Projekte. Meist werden auch die Ergebnisse der Nachkalkulation in diesem Bericht festgehalten. Oft wird jeder Projektmitarbeiter aufgefordert, seine eigene Einschätzung des Projektverlaufs einzubringen und Vorschläge zur Verbesserung der Projektentwicklung zu machen („three lessons learned“). In [Kaner *et al.*, 1999] werden dazu folgende Ratschläge gegeben:

- Sei konstruktiv. Betone das gut Gelaufene. Erwähne das schlecht Gelaufene sachlich.
- Versuche nicht, das ganze Projekt als problemlos darzustellen. Entscheide selbst, welche Probleme du ansprechen möchtest. Tu dies sachlich und unpersönlich. Streite aufgetretene Probleme nicht ab. Lobe keinesfalls Mitarbeiter für nicht oder schlecht erledigte Arbeiten.
- Vermeide es, den Ruf als Dauernörgler zu bekommen. Zeige nicht mit dem Finger auf andere, die noch schlechter als du gearbeitet haben.
- Verhalte dich nicht defensiv. Suche keine Entschuldigungen.
- Verlange nie, dass jemand anderer aus der Gruppe entfernt oder entlassen werden soll.
- Wenn du glaubst, dass das Produkt schlecht ist, überlege genau, ob du das auf den Tisch bringst. Wärme keine alten, bekannten Fehlergeschichten auf. Überlege, ob du deine Aussagen in einem Monat genauso tätigen würdest.

- Sage nicht in vorauseilendem Gehorsam, dass deine Arbeit schlecht war. Wenn du deine eigene Arbeit kritisierst, erkläre, was geändert werden kann und wie du deine Arbeit bis zum und im nächsten Projekt verbessern kannst.
- Besprich deine Aussagen mit einem Freund, bevor du sie in der Gruppe tätigst.

Eine Abschlussbesprechung bzw. die Erstellung eines Abschlussberichts sollten daher nicht unmittelbar nach der Abnahme erfolgen; es sollten ein paar Tage zur eigenen Reflexion dazwischen liegen.

14.5.5 Projektarchiv

Das im Rahmen der Archivierung erstellte *Projektarchiv* stellt nicht nur die Basis für eine etwaige Wartung dar, es bildet auch die Dokumentation für das im Projekt erarbeitete Know-how sowohl aus Produkt- als auch aus Projektsicht. Falls das Projektarchiv für Wartungszwecke verwendet wird, ist es essenziell, dass von jeder Produktkomponente die verschiedenen Versionen aufbewahrt werden müssen, um daraus die jeweils im Einsatz befindlichen Konfigurationen zusammenstellen zu können.

Vielfach ist es nicht zulässig, auftraggeberspezifische Daten zu archivieren. Werden diese für Wartungszwecke benötigt, so empfiehlt sich eine entsprechende gesonderte schriftliche Vereinbarung. Auf Grund des oft auch auftragnehmerinternen sensiblen Datenmaterials (z. B. Performanz der einzelnen Projektmitarbeiter) empfiehlt sich eine gesicherte Ablage mit entsprechendem Zugriffsschutz, wobei eine auftragnehmerinterne Zugriffspolitik auf archivierte Projekte schriftlich festgelegt werden soll. Bei der Auswahl der Archivformate ist darauf zu achten, dass alle zum Öffnen, Starten bzw. Bearbeiten der Dokumente notwendigen speziellen Werkzeuge (Kompressions- und Dekompressionsprogramme, Formatkonverter etc.) ebenfalls im Archiv enthalten sind und das Archiv in einer Standardumgebung ohne zusätzliche Werkzeuge bearbeitet werden kann.

Literaturverzeichnis

- [Adameitz, 1999] Michael Adameitz. A Short Report on Risk Management. [Web-Seite, http://www.ozemail.com.au/~sqain/meeting_reports/Risk_report.htm, zugegriffen am 22.12.1999], 1999.
- [Aho *et al.*, 1986] Alfred V. Aho, Ravi Sethi, und Jeffrey D. Ullman. *Compilers – Principles, Techniques, and Tools*. Addison Wesley, 1986.
- [Aigner, 2004] Kurt Aigner. Optimaler Business Value durch „Risk Based Testing“. In *Proc. Workshop Softwaretesten und Qualitätskontrolle*. LSZ Consulting, 11. November 2004, Wien, Österreich, 2004.
- [Albrecht und Gaffney, 1983] Allan J. Albrecht und John E. Gaffney. Software Function, Source Lines of Code, and Development Effort Prediction: A Software Science Validation. *IEEE Transactions on Software Engineering*, SE-9(11):639–648, 1983.
- [Alleman, 2002a] Glen Alleman. Can Software Projects Be Managed Using PMBOK-like Processes? Niwot Ridge Consulting [Web-Seite, <http://www.niwotridge.com/Essays/Editorials/PMEditorial.htm>, zugegriffen am 21.3.2005], 2002.
- [Alleman, 2002b] Glen Alleman. Putting Agility in Context. *Information Age – Australia’s IT Trends Review*, 02(8), 2002.
- [Ambler, 1999a] Scott W. Ambler. *More Process Patterns: Delivering Large-Scale Systems Using Object Technology*. Cambridge University Press, 1999.
- [Ambler, 1999b] Scott W. Ambler. *Process Patterns: Building Large-Scale Systems Using Object Technology*. Cambridge University Press, 1999.
- [Ambler, 2002] Scott W. Ambler. *Agile Modeling: Effective Practices for Extreme Programming and the Unified Process*. Wiley, 2002.
- [Ambler, 2003] Scott W. Ambler. Agile Model Driven Development Is Good Enough. *IEEE Software*, 20(5):71–73, 2003.
- [Artmann, 1997] Karl Artmann. Object-Oriented Engineering Frameworks. Diplomarbeit, RISC-Linz, Johannes Kepler Universität, Linz, Österreich, 1997.
- [Astels, 2003] David Astels. *Test Driven Development: A Practical Guide*. Prentice Hall, 2003.
- [Backerra *et al.*, 2002] Hendrik Backerra, Christian Malorny, und Wolfgang Schwarz. *Kreativitätstechniken*. Hanser, 2002.
- [Baker und Mills, 1979] F. Terry Baker und Harlan D. Mills. Chief Programmer Teams. In *Classics in Software Engineering*. Yourdon Press, 1979.
- [Baker, 1972] F. Terry Baker. Chief Programmer Team Management of Production Programming. *IBM Systems Journal*, 11(1), 1972.
- [Balzert, 2000a] Helmut Balzert. *Lehrbuch der Software-Technik: Band 1 – Software-Entwicklung*. Spektrum Akademischer Verlag, Heidelberg Berlin, 2. Auflage, 2000.
- [Balzert, 2000b] Helmut Balzert. *Lehrbuch der Software-Technik: Band 2 – Software-Management, Software-Qualitätssicherung, Unternehmensmodellierung*. Spektrum Akademischer Verlag, Heidelberg Berlin, 2. Auflage, 2000.
- [Baskerville und Levine, 2001] Richard Baskerville und Linda Levine. How Internet Software Companies Negotiate Quality. *IEEE Computer*, 34(5):51–57, 2001.
- [Bauer, 1975] Friedrich L. Bauer. Software Engineering. In *Software Engineering as an Advanced Course*. Springer, 1975.

- [Beck und Andres, 2005] Kent Beck und Dirk Andres. *Extreme Programming Explained*. Addison-Wesley, 2005.
- [Beck und McBreen, 2002] Kent Beck und Pat McBreen. *Questioning Extreme Programming*. Addison-Wesley Longman, 2002.
- [Beck, 2002] Kent Beck. *Test Driven Development by Example*. Addison-Wesley Professional, 2002.
- [Beizer, 1990] Boris Beizer. *Software Testing Techniques*. Van Nostrand Reinhold, New York, 2. Auflage, 1990.
- [Beizer, 1995] Boris Beizer. *Black-Box Testing*. John Wiley & Sons, 2. Auflage, 1995.
- [Bennatan, 2000] Elli M. Bennatan. *On Time, Within Budget: Software Project Management Practices and Techniques*. John Wiley & Sons, 3. Auflage, 2000.
- [Blaschek und Sametinger, 1989] Günther Blaschek und Johannes Sametinger. User-Adaptable Prettyprinting. *Software – Practice and Experience*, 19(7), 1989.
- [Boehm *et al.*, 2000] Barry W. Boehm, Ellis Horowitz, Ray Madachy, Donald Reifer, Bradford K. Clark, Bert Steece, A. Winsor Brown, Sunita Chulani, und Chris Abts. *Software Cost Estimation with COCOMO II*. Prentice Hall, 2000.
- [Boehm und Huang, 2003] Barry W. Boehm und Li Guo Huang. Value-based Software Engineering: A Case Study. *IEEE Computer*, 36(3):33–41, 2003.
- [Boehm, 1984] Barry W. Boehm. Verifying and Validating Software Requirements and Design Specifications. *IEEE Software*, 84(1):75–88, 1984.
- [Boehm, 1988] Barry W. Boehm. A Spiral Model of Software Development and Enhancement. *IEEE Computer*, 21, 1988.
- [Boehm, 1991] Barry W. Boehm. Software Risk Management: Principles and Practices. *IEEE Software*, 91(1):32–41, 1991.
- [Boehm, 1998] Barry W. Boehm. Software Risk Management. Vorlesungsunterlagen CS 577a, 510, Center for Software Engineering, University of Southern California, 1998.
- [Boehm, 2000] Barry W. Boehm. Requirements that Handle IKIWISI, COTS, and Rapid Change. *IEEE Computer*, 33(7):99–102, 2000.
- [Boehm, 2002] Barry W. Boehm. Get Ready for Agile Methods, with Care. *IEEE Computer*, 35(1):64–69, 2002.
- [Bons und van Megen, 1982] Heinz Bons und Rudolf van Megen. Zur Festlegung von Qualitätszielen als Grundlage der Qualitätsplanung und -kontrolle. Technischer Bericht, ACM, German Chapter, Bericht 9, 1982.
- [Brooks, 1975] Frederick P. Brooks. *The Mythical Man-Month. Essays on Software Engineering*. Addison-Wesley, 1975.
- [Brown *et al.*, 2000] William J. Brown, Hays W. McCormick, und Scott W. Thomas. *Anti-Patterns in Project Management*. John Wiley & Sons, 2000.
- [Broy und Rausch, 2005a] Manfred Broy und Andreas Rausch. V-Modell XT Dokumentation: Teil 1 – Grundlagen. Technischer Bericht, Deutsches Bundesministerium des Innern, Berlin, Deutschland, 2005.
- [Broy und Rausch, 2005b] Manfred Broy und Andreas Rausch. V-Modell XT Dokumentation: Teil 3 – Tailoring. Technischer Bericht, Deutsches Bundesministerium des Innern, Berlin, Deutschland, 2005.
- [Buschmann, 1998] Frank Buschmann. *Pattern-orientierte Software-Architektur. Ein Pattern-System*. Addison-Wesley, 1998.
- [Chan und Lee, 1998] Patrick Chan und Rosanna Lee. *The Java Class Libraries – An Annotated Reference*. Addison-Wesley, 2. Auflage, 1998.
- [Charette *et al.*, 1997] Robert Charette, Mary White, und Kevin Adams. Establishing Risk Management in an Operational Maintenance Organization. *IEEE Software*, 97(5):43–50, 1997.
- [Charette, 1990] Robert Charette. *Application Strategies for Risk Management*. McGraw-Hill, 1990.
- [Chen, 1977] Peter S. Chen. *The Entity-Relationship Approach to Logical Data Base Design*. Q.E.D. Information Sciences, Inc., Wellesley, MA, 1977.
- [Coad und Yourdon, 1990] Peter Coad und Edward Yourdon. *Object-oriented Analysis*. Yourdon Press Computing Series, 1990.

- [Cockburn, 1997] Alistair Cockburn. Goals and Use Cases. *JOOP*, 97(9), 1997.
- [Cockburn, 2001] Alistair Cockburn. *Agile Software Development*. Addison-Wesley Professional, 2001.
- [Cockburn, 2004] Alistair Cockburn. *Crystal Clear*. Addison-Wesley Professional, 2004.
- [Coldewey, 2002] Jens Coldewey. Multi-Kulti: Ein Überblick über die Agile Entwicklung. *Objekt-Spektrum*, Seiten 69–77, 2002.
- [Coleman *et al.*, 1994] Derek Coleman, Patrick Arnold, Stephanie Bodoff, Chris Dollin, Helena Gilchrist, Fiona Hayes, und Paul Jeremaes. *Object-Oriented Development – The Fusion Method*. Prentice Hall, 1994.
- [Connell und Shafer, 1989] John L. Connell und Linda I. Shafer. *Structured Rapid Prototyping*. Yourdon Press / Prentice Hall, 1989.
- [Cormen *et al.*, 2001] Thomas H. Cormen, Charles E. Leiserson, und Ronald L. Rivest. *Introduction to Algorithms*. Massachusetts Institute of Technology, McGraw Hill, 2. Auflage, 2001.
- [Cutter, 2000] Cutter. Poor Project Management Number-One Problem of Outsourced E-Projects, Cutter Consortium Research Briefs, Nov 7, 2000. [Web-Seite, <http://www.cutter.com/research/2000/crb001107.html>, zugegriffen am 18.3.2003], 2000.
- [Dart, 2000] Susan Dart. *Configuration Management: The Missing Link in Web Engineering*. Artech House, 2000.
- [Darwen und Date, 1998] Hugh Darwen und Chris J. Date. *Foundation for Object-Relational Databases: The Third Manifesto*. Addison-Wesley, 1998.
- [Date, 2003] Chris J. Date. *An Introduction to Database Systems*. Addison-Wesley, 8. Auflage, 2003.
- [DeMarco, 1978] Tom DeMarco. *Structured Analysis and Systems Specifications*. Yourdon Press, 1978.
- [DeMarco, 2004] Tom DeMarco. *Spielräume: Projektmanagement jenseits von Burn-out, Stress und Effizienzwahn*. Hanser, 2004.
- [Deming, 1986] W. Edwards Deming. *Out of the Crisis*. Massachusetts Institute of Technology, 1986.
- [Dijkstra, 1976] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice Hall, 1976.
- [DIN, 2000] DIN. DIN 69904: Projektmanagementsysteme: Elemente und Strukturen. Technischer Bericht, Deutsches Institut für Normung, 2000.
- [Dobler und Huber, 1991] Heinz Dobler und Gerhard Huber. Dynamic Analysis of Modula-2 Programs. In *Proc. Modula-2 and Beyond*, Johannes Kepler Universität, Linz, Österreich, 1991.
- [Dobler und Pirklbauer, 1990] Heinz Dobler und Klaus Pirklbauer. COCO-2 – A New Compiler-Compiler. *ACM Sigplan Notices*, 25(5), 1990.
- [Donaldson und Siegel, 1997] Scott E. Donaldson und Stanley G. Siegel. *Cultivating Successful Software Development – A Practitioner’s View*. Prentice Hall, 1997.
- [Drucker, 1975] Peter Drucker. *Management*. Heinemann, 1975.
- [Duke *et al.*, 1996] Roger Duke, Paul King, Gordon Rose, und Graeme Smith. The Object-Z Specification Language Version 1. Technischer Bericht, Software Verification Research Centre, University of Queensland, Queensland, Australien, Mai 1996.
- [Ebert, 2004] Christof Ebert. Getting Started with the CMMI. *IEEE Software*, 21(6):92–94, 2004.
- [Elzer, 1987] Peter F. Elzer, Herausgeber. *Proc. IFAC / IFIP Workshop „Experience with the Management of Software Projects“, Heidelberg 1986*, Pergamon Press, 1987.
- [Elzer, 1991] Peter F. Elzer. New Trends in Software for Real-Time Control. In *Proc. 11th IFAC World Congress, Tallinn, Estonia*, Seiten 267–271, Pergamon Press, 1991.
- [Elzer, 1994] Peter F. Elzer. *Management von Softwareprojekten*. Vieweg, 1994.
- [Engels *et al.*, 2003] Gregor Engels, Marc Lohmann, und Annika Wagner. Entwicklungsprozess von Web-Anwendungen. In Gerti Kappel, Birgit Pröll, Siegfried Reich, und Werner Retschitzegger, Herausgeber, *Web Engineering: Systematische Entwicklung von Web-Anwendungen (Kapitel 10)*. dpunkt.verlag, 2003.
- [Fagan, 1976] Michael E. Fagan. Design and Code Inspections to Reduce Errors in Program Development. *IBM Systems Journal*, 15(3), 1976.
- [Fairley, 1985] Richard Fairley. *Software Engineering Concepts*. McGraw Hill, 1985.
- [Favela und Decouchant, 2004] Jesus Favela und Dominique Decouchant. *Groupware Design, Implementation, and Use*. Springer, 2004.

- [Fischer, 1989] Hans-Peter Fischer, Herausgeber. *Personalentwicklung im Werk: Konzepte und Erfahrungen von Mercedes-Benz*. Windmühle, 1989.
- [Floyd, 1984] Christiane Floyd. *A Systematic Look at Prototyping*. In: Approaches to Prototyping. Springer, 1984.
- [Ford und Gibbs, 1996] Gary Ford und Norman E. Gibbs. A Mature Profession of Software Engineering. Technischer Bericht TR 96-004, Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, USA, 1996.
- [Fowler und Foemmel, 2005] Martin Fowler und Matthew Foemmel. Continuous Integration. [Web-Seite, <http://www.martinfowler.com/articles/continuousIntegration.html>, zugegriffen am 28.4.2005], 2005.
- [Fowler und Highsmith, 2001] Martin Fowler und Jim Highsmith. The Agile Manifesto. *Software Development*, 9(8):28–32, 2001.
- [Fowler, 1997] Martin Fowler. *Analysis Patterns – Reusable Object Models*. Addison-Wesley Object-Oriented Software Engineering Series, 1997.
- [Fowler, 1999] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.
- [Fowler, 2004a] Martin Fowler. The New Methology. [Web-Seite, <http://www.martinfowler.com/articles/newMethology.html>, zugegriffen am 20.11.2004], 2004.
- [Fowler, 2004b] Martin Fowler. *UML Distilled*. Addison-Wesley, 2. Auflage, 2004.
- [Fraikin et al., 2004] Falk Fraikin, Matthias Hamburg, Stefan Jungmayr, Thomas Leonhardt, Andreas Schönknecht, Andreas Spillner, und Mario Winter. Die trügerische Sicherheit des grünen Balkens. *ObjektSpektrum*, 04(1):26–30, 2004.
- [Frühauf et al., 1995] Karol Frühauf, Jochen Ludewig, und Helmut Sandmayr. *Software-Prüfung – eine Anleitung zum Test und zur Inspektion*. Teubner, Stuttgart, 1995.
- [Frühauf und Lebsanft, 1995] Thomas Frühauf und Ernst Lebsanft. Die BOOTSTRAP-Methode: ISO 9001-Zertifizierung bei Softwareentwicklern. *SAQ ASPQ Qualität*, 95(1):30–34, 1995.
- [Gamma et al., 1997] Erich Gamma, Richard Helm, Ralph E. Johnson, und John Vlissides. *Design Patterns – Elements of Reusable Object-oriented Software*. Addison-Wesley, 2. Auflage, 1997.
- [Gane und Sarson, 1979] Chris P. Gane und Trish Sarson. *Structured Systems Analysis: Tools and Techniques*. Prentice Hall, 1979.
- [Gareis, 1988] Roland Gareis. Projektmanagement in den USA und in Österreich: Gemeinsamkeiten, Unterschiede, Trends. *Projekt-Management Austria*, 88(5):25–28, 1988.
- [Gareis, 2004] Roland Gareis. *Happy Projects*. Manz, Wien, 2. Auflage, 2004.
- [Gemmer, 1997] Art Gemmer. Risk Management: Moving Beyond Process. *IEEE Computer*, 97(5):33–43, 1997.
- [Gernert und Ahrend, 2001] Christine Gernert und Norbert Ahrend. *IT-Management: System statt Chaos*. Oldenbourg, 2001.
- [Gilb, 1996] Tom Gilb. *Principles of Software Engineering Management*. Addison Wesley, 2. Auflage, 1996.
- [Goldberg und Robson, 1989] Adele Goldberg und David Robson. *Smalltalk-80*. Addison-Wesley Professional, 1989.
- [Goldberg und Rubin, 1995] Adele Goldberg und Kenneth S. Rubin. *Succeeding With Objects: Decision Frameworks for Project Management*. Addison Wesley, 1995.
- [Grady, 1992] Bob Grady. *Practical Software Metrics for Project Management and Process Improvement*. Prentice Hall, 1992.
- [Grady, 1996] Bob Grady. Lessons From Industrial Adoption of Software Engineering Practices. In Nancy R. Mead, Herausgeber, *Proc. 9th Conf. on Software Engineering Education*, Seiten 216–217, 21. – 24. April 1996, Daytona Beach, USA, IEEE Computer Society, 1996.
- [Gregorio und Wakefield, Jr., 1994] Donna D. Gregorio und Edward H. Wakefield, Jr. Guidelines for Defining the Prototyping Process at MITRE. Technischer Bericht, MIT Research and Engineering – MITRE, Bedford, Massachusetts, USA, Oktober 1994.
- [Gregorio, 1996] Donna D. Gregorio. Prototyping Systems. Tutorial-Unterlagen, 9th Conference on Software Engineering Education, 21. – 24. April 1996, Daytona Beach, Florida, USA, MIT Research and Engineering – MITRE, Bedford, Massachusetts, USA, 1996.

- [Grünbacher, 2003] Paul Grünbacher. Requirements-Engineering für Web-Anwendungen. In Gerti Kappel, Birgit Pröll, Siegfried Reich, und Werner Retschitzegger, Herausgeber, *Web Engineering: Systematische Entwicklung von Web-Anwendungen (Kapitel 2)*. dpunkt.verlag, 2003.
- [Grochla, 1982] Erwin Grochla. *Grundlagen der organisatorischen Gestaltung*. Stuttgart, 1982.
- [Hall, 1998] Elaine M. Hall. *Managing Risk – Methods for Software Systems Development*. Addison Wesley Longman, 1998.
- [Halling *et al.*, 2003] Michael Halling, Stefan Biffl, und Paul Grünbacher. An Economic Approach for Improving Requirements Negotiation Models With Inspection. *Requirements Engineering*, 8, 2003.
- [Halstead, 1977] Maurice H. Halstead. *Elements of Software Science*. Elsevier, North-Holland, 1977.
- [Harry und Schroeder, 1999] Mikel Harry und Richard Schroeder. *Six Sigma, The Breakthrough Management Strategy Revolutionizing The World's Top Corporations*. Currency, 1999.
- [Heinrich und Lehner, 2002] Lutz J. Heinrich und Franz Lehner. *Informationsmanagement*. Oldenbourg, 7. Auflage, 2002.
- [Heinrich und Roithmayr, 1992] Lutz J. Heinrich und Friedrich Roithmayr. *Wirtschaftsinformatiklexikon*. Oldenbourg, 1992.
- [Herczeg, 2004] Michael Herczeg. *Software-Ergonomie. Grundlagen der Mensch-Computer-Kommunikation*. Oldenbourg, 2004.
- [Highsmith, 2000] Jim Highsmith. *Adaptive Software Development – A Collaborative Approach to Managing Complex Systems*. Dorset House, 2000.
- [Highsmith, 2002] Jim Highsmith. *Agile Software Development Ecosystems*. Addison-Wesley, 2002.
- [Humphrey, 1987] Watts S. Humphrey. *Method for Assessing the Software Engineering Capability of Contractors*. N.T.I.S., 1987.
- [Humphrey, 1989] Watts S. Humphrey. *Managing the Software Process*. Addison-Wesley, 1989.
- [Humphrey, 1997] Watts S. Humphrey. *Managing Technical People*. Addison-Wesley, 1997.
- [Humphrey, 2002] Watts S. Humphrey. Comments on eXtreme Programming. [Web-Seite, <http://www.computer.org/SEweb/Dynabook/HumphreyCom.htm>, zugegriffen am 20.11.2004], 2002.
- [Humphrey, 2005] Watts S. Humphrey. *PSP – A Self-Improvement Process for Software Engineers*. Addison-Wesley, 2005.
- [Hunt und Thomas, 2003] Andrew Hunt und David Thomas. *Pragmatic Unit Testing in Java with JUnit*. The Pragmatic Programmers, 2003.
- [IEEE, 1991] IEEE. IEEE Software Engineering Terminology, IEEE Std. 610.12-1990. Technischer Bericht, The Institute of Electrical and Electronics Engineers, Inc., New York, USA, 1991.
- [IEEE, 1994] IEEE. IEEE Recommended Practice for Software Requirements Specifications, IEEE Std. 830-1993. Technischer Bericht, The Institute of Electrical and Electronics Engineers, Inc., New York, USA, 1994.
- [IEEE, 1996] IEEE. IEEE / EIA Standard Industry Implementation of International Standard ISO / IEC 12207:1995 for Information Technology – Software Life Cycle Processes (Description), IEEE / EIA Std. 12207.0-1996. Technischer Bericht, The Institute of Electrical and Electronics Engineers, Inc., New York, USA, 1996.
- [IEEE, 1998] IEEE. IEEE Standard for Software Test Documentation, IEEE Std. 829-1998. Technischer Bericht, The Institute of Electrical and Electronics Engineers, Inc., New York, USA, 1998.
- [Ijäs, 1996] Lasse Ijäs. Project Training. Internes Trainingshandbuch, Ahlström Machinery Corp., Varkaus, Finland, 1996.
- [ISO, 1991] ISO. Information Technology – Software Product Evaluation: Quality Characteristics and Guidelines for Their Use, ISO / IEC Std. 9126. Technischer Bericht, International Standardization Organization, 1991.
- [ISO, 1999] ISO. Information Technology – Software Process Assessment, ISO / IEC TR 15504-5:1999. Technischer Bericht, International Standardization Organization, 1999.
- [ISO, 2004] ISO. Software Engineering – Guidelines for the Application of ISO 9001:2000 to Computer Software, ISO / IEC Std. 90003. Technischer Bericht, International Standardization Organization, 2004.
- [Jeffries *et al.*, 2000] Ron Jeffries, Ann Anderson, und Chet Hendrickson. *Extreme Programming Installed*. Addison-Wesley, 2000.

- [Jones, 1990] Cliff B. Jones. *Systematic Software Development using VDM*. Prentice Hall, Englewood Cliffs, NJ, 2. Auflage, 1990.
- [Jones, 1994] Capers Jones. *Assessment and Control of Software Risks*. Yourdon Press Computing Series, 1994.
- [Kaner et al., 1999] Cem Kaner, Jack Falk, und Hung Quoc Nguyen. *Testing Computer Software*. Wiley, 2. Auflage, 1999.
- [Kaplan und Norton, 1996] Robert Kaplan und David Norton. *The Balanced Scorecard: Translating Strategy into Action*. Harvard Business School Press, 1996.
- [Kappel et al., 2003] Gerti Kappel, Birgit Pröll, Siegfried Reich, und Werner Retschitzegger. *Web Engineering: Systematische Entwicklung von Web-Anwendungen*. dpunkt.verlag, 2003.
- [Kearney et al., 1986] Joseph K. Kearney, Robert L. Sedlmeyer, William B. Thompson, Michael A. Gray, und Michael A. Adler. Software Complexity Measurement. *Communications of the ACM*, 29(11):1044–1050, 1986.
- [Kieser und Kubicek, 1983] Alfred Kieser und Herbert Kubicek. *Organisation*. de Gruyter, 2., neubearbeitete und erweiterte Auflage, 1983.
- [Kopetz, 1976] Hermann Kopetz. *Software-Zuverlässigkeit*. de Gruyter, 1976.
- [Koskela, 2000] Lauri Koskela. *An Exploration Towards a Production Theory and Its Application to Construction*. VTT publications, Espoo, Finnland, 2000.
- [Kotonya und Sommerville, 1998] Gerald Kotonya und Ian Sommerville. *Requirements Engineering: Processes and Techniques*. John Wiley & Sons, 1998.
- [Kruchten, 2000] Philippe Kruchten. *The Rational Unified Process, An Introduction*. Addison-Wesley, 2. Auflage, 2000.
- [Kuhn, 1996] Thomas S. Kuhn. *The Structure of Scientific Revolutions*. University of Chicago Press, 3. Auflage, 1996.
- [Kuhrmann, 2005] Marco Kuhrmann. Das V-Modell XT: Einführung und Überblick. Technischer Bericht, Vortrag anlässlich der Präsentation des V-Modells XT an der TU München, 4.2.2005, Institut für Informatik, TU München, Deutschland, 2005.
- [Kulczycki, 1996] Peter Kulczycki. Analysetechniken mit Schwerpunkt OOA. Seminararbeit „Projektbezogene Vertiefung“, Prof. H. Mayr, Fachhochschulstudiengang Software Engineering, Hagenberg, Österreich, Wintersemester 1995/1996, 1996.
- [Kulik, 1996] Peter Kulik. What Is Software Risk Management? (And Why should I Care?). Technischer Bericht, Kulik & Lazarus Consulting, Inc., <http://www.klci.com/accelerate>, 1996.
- [Kulpa und Johnson, 2003] Margaret K. Kulpa und Kent A. Johnson. *Interpreting the CMMI: A Process Improvement Approach*. Auerbach Publications, 2003.
- [Lano und Haughton, 1992] Kevin C. Lano und Howard Haughton. The Z++ Manual. Technischer Bericht, Lloyds Register of Shipping, 29 Wellesley Road, Croydon, UK, 1992.
- [Lientz und Swanson, 1980] Bennet P. Lientz und E. Burton Swanson. *Software Maintenance Management*. Addison-Wesley, 1980.
- [Link, 2005] Johannes Link. *Softwaretests mit JUnit: Techniken der testgetriebenen Entwicklung*. dpunkt.verlag, 2., überarbeitete und erweiterte Auflage, 2005.
- [Loomis et al., 1987] Mary Loomis, Ashwin Shah, und James Rumbaugh. An Object Modeling Technique for Conceptual Design. In *Proc. European Conference on OOP*, 1987.
- [Lowe und Eklund, 2002] David B. Lowe und John Eklund. Client Needs and the Design Process in Web Projects. *Journal of Web Engineering*, 1(1):23–36, 2002.
- [Lowell, 1993] Jay Arthur Lowell. *Improving Software Quality: An Insider's Guide to TQM*. Wiley, 1993.
- [Martin und McClure, 1983] James Martin und Carma L. McClure. *Software Maintenance: The Problem and Its Solutions*. Prentice Hall, 1983.
- [Mayr, 1997] Herwig Mayr. Teaching Software Engineering by Means of a „Virtual Enterprise“. In *Proc. 10th Conf. Software Engineering Education and Training – SEE&T'97*, Seiten 176–184. 13. – 16. April 1997, Virginia Beach, USA, IEEE Computer Science Press, 1997.
- [Mayr, 1999] Herwig Mayr. A Mnemonic Scheme for Teaching Project-Oriented Software Engineering. In *Proc. Software Engineering im Unterricht der Hochschulen SEUH'99*, Seiten 91–102. 25. – 26. Februar 1999, Wiesbaden, Deutschland, Teubner, Stuttgart Leipzig, 1999.

- [Mayr, 2003] Herwig Mayr. Webprojekt-Management. In Gerti Kappel, Birgit Pröll, Siegfried Reich, und Werner Retschitzegger, Herausgeber, *Web Engineering: Systematische Entwicklung von Web-Anwendungen (Kapitel 9)*. dpunkt.verlag, 2003.
- [Mayr, 2004a] Herwig Mayr. „Disciplined Agility“ or How To Teach Agile Software Development to Undergraduate Students. In Nagib Callaos, Herausgeber, *Proc. 8th World Multi-Conference on Systemics, Cybernetics and Informatics (SCI 2004)*, Seiten 156–161. IIIS / IFSR, Orlando, USA, 2004.
- [Mayr, 2004b] Herwig Mayr. Project Management Education for Software Engineering and Web Engineering: Commonalities and Differences. In Freddy Malpica, Herausgeber, *Proc. Intl. Conf. on Education and Information Systems: Technologies and Applications (EISTA 2004)*, Seiten 277–282. IIIS / IFSR, Orlando, USA, 2004.
- [McCabe und Schulmeyer, 1987] Thomas J. McCabe und G. Gordon Schulmeyer. The Pareto Principle Applied to Software Quality Assurance. *Schulmeyer & McManus*, Seiten 178–210, 1987.
- [McCabe, 1976] Thomas J. McCabe. A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-12(4), 1976.
- [McDonald und Welland, 2001] Andrew McDonald und Ray Welland. A Survey of Web Engineering in Practice. Technischer Bericht R-2001-79, Department of Computing Science, University of Glasgow, Glasgow, Schottland, 2001.
- [McMenamin und Palmer, 1984] Steve McMenamin und John Palmer. *Essential Systems Analysis*. Yourdon Press / Prentice Hall, 1984.
- [Mellor *et al.*, 2003] Stephen J. Mellor, Anthony N. Clark, und Takao Futagami. Model-Driven Development. *IEEE Software*, 20(5):14–18, 2003.
- [Meyer, 1997] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, 2. Auflage, 1997.
- [Meyer, 2001] Bertrand Meyer. Software Engineering in the Academy. *IEEE Computer*, 34(5):28–35, 2001.
- [Meyers, 2004] Meyers. *Kompaktlexikon*. Bibliografisches Institut, Mannheim, Deutschland, 2004.
- [Microsoft, 1990] Microsoft. *Microsoft Windows: International Handbook for Software Design*. Microsoft Press, Redmond, WA, 1990.
- [Myers, 1979] Glenford J. Myers. *The Art of Software Testing*. John Wiley & Sons, 1979.
- [Naur und Randell, 1969] Peter Naur und Brian Randell, Herausgeber. *Software Engineering: A Report on a Conference Sponsored by the NATO Science Committee*. NATO, 1969.
- [NIST, 2002] NIST. The Economic Impacts of Inadequate Infrastructure for Software Testing, National Institute of Standards and Technology, Report No. 2002-3. [Web-Seite, <http://www.nist.gov/director/prog-ofc/report02-3.pdf>, zugegriffen am 30.4.2003], 2002.
- [Nuseibeh, 2001] Bashar Nuseibeh. Weaving Together Requirements and Architectures. *IEEE Computer*, 34(3):115–117, 2001.
- [Oestereich, 2004] Bernd Oestereich. *Analyse und Design mit UML 2*. Oldenbourg, 2004.
- [Ogunnaike und Harmon, 1994] Babatunde A. Ogunnaike und Ray W. Harmon. *Process Dynamics, Modeling, and Control*. Oxford University Press, 1994.
- [Olsina *et al.*, 2002] Luis A. Olsina, Guillermo Lafuente, und Oscar Pastor. Towards a Reusable Repository of Web Metrics. *Journal of Web Engineering*, 1(1):61–73, 2002.
- [Oriolo, 1990] Marita Oriolo, Herausgeber. *Schätzverfahren in 4GL- und CASE-Umgebungen; Informatik-Fachberichte 258*. Springer, 1990.
- [Orr, 2002] Ken Orr. CMM Versus Agile Development: Religious Wars and Software Development. Technischer Bericht Cutter Executive Report Nr. 3.7, Cutter Consortium, 2002.
- [Parnas, 1972] David L. Parnas. On the Criteria to Be Used in Decomposing Systems Into Modules. *Communications of the ACM*, 15(12), 1972.
- [Patzak und Rattay, 1997] Gerold Patzak und Günther Rattay. *Projekt Management*. Linde, Wien, 2. Auflage, 1997.
- [Paulk *et al.*, 1993] Mark C. Paulk, Bill Curtis, Mary Beth Chrissis, und Charles Weber. Capability Maturity Model for Software, Version 1.1. Technischer Bericht CMU / SEI-93-TR-24, Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, USA, 1993.
- [Paulk, 2001] Mark C. Paulk. Extreme Programming from a CMM Perspective. *IEEE Software*, 01(6):19–26, 2001.

- [Paulson, 2001] Linda D. Paulson. Adapting Methodologies for Doing Software Right. *IEEE IT Professional*, 01(7/8):13–15, 2001.
- [Perry, 2002] William E. Perry. *Software Testen*. Mitp-Verlag, 2002.
- [Pfleeger, 2001] Shari L. Pfleeger. *Software Engineering*. Prentice Hall, 2. Auflage, 2001.
- [Pichler, 2004] Roman Pichler. Test-driven Development. *Java Spektrum*, 04(6):20–22, 2004.
- [PMI, 1999] Project Management Institute PMI. *Project Management Software Survey*. PMI Books, 1999.
- [PMI, 2005] Project Management Institute PMI. *PMBOK – Project Management Body of Knowledge (2005 Edition)*. PMI Books, 2005.
- [Pomberger und Blaschek, 1996] Gustav Pomberger und Günther Blaschek. *Grundlagen des Software Engineering: Prototyping und objektorientierte Software-Entwicklung*. Carl Hanser, 2., überarbeitete Auflage, 1996.
- [Porter, 1998] Michael E. Porter. *Competitive Advantage: Creating and Sustaining Superior Performance*. Free Press, 1998.
- [Potts, 1993] Colin Potts. Software Engineering Research Revisited. *IEEE Software*, 93(9):19–28, 1993.
- [Pressman, 1998] Roger S. Pressman. Can Internet-Based Applications Be Engineered? *IEEE Software*, 15(5):104–110, 1998.
- [Pressman, 2000] Roger S. Pressman. Can WebApps Be Engineered? Technischer Bericht 2000-05, SPC E-ssentials, Software Productivity Center Inc., 2000.
- [Pressman, 2004] Roger S. Pressman. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, 6. Auflage, 2004.
- [Putnam, 1978] Lawrence H. Putnam. A General Empirical Solution to the Macro Software Sizing and Estimation Problem. *IEEE Transactions on Software Engineering*, 4(4), 1978.
- [Raitkin, 2001] Steven Raitkin. Manifesto Elicits Cynicism. *IEEE Computer*, 34(12):4, 2001.
- [Rausch-Schott und Retschitzegger, 1993] Stefan Rausch-Schott und Werner Retschitzegger. Projektmanagement und Präsentationstechnik. Vorlesungsunterlagen, Abteilung für Informationssysteme des Institutes für Informatik, Johannes Kepler Universität, Linz, Österreich, 1993.
- [Rechenberg und Pomberger, 2002] Peter Rechenberg und Gustav Pomberger. *Informatik-Handbuch*. Hanser, 3. Auflage, 2002.
- [Rechenberg, 1985] Peter Rechenberg. Editorial zu Werkzeuge der Softwaretechnik. *Elektronische Rechenanlagen*, 27(2), 1985.
- [Reifer, 2002] Donald C. Reifer. Estimating Web Development Costs: There Are Differences. *CrossTalk – The Journal of Defense Software Engineering*, 02(6):13–17, 2002.
- [Rook, 1991] Paul Rook. Project Planning and Control. In John A. McDermid, Herausgeber, *Software Engineer's Reference Book*. Butterworth-Heinemann, 1991.
- [Royce, 1970] Walker Royce. Managing the Development of Large Software Systems. In *Proc. WESCON '70*, 1970.
- [Rubin und Goldberg, 1992] Kenneth S. Rubin und Adele Goldberg. Object Behavior Analysis. *Communications of the ACM*, 35(9):48–62, 1992.
- [Sanchez und Canton, 1997] Julio Sanchez und Maria P. Canton. *Patterns, Models, and Application Development : A C++ Programmer's Reference*. CRC Press, 1997.
- [Schmucker, 1986] Kurt J. Schmucker. *Object-oriented Programming for the Macintosh*. Hayden, 1986.
- [Schnupp und Floyd, 1976] Peter Schnupp und Christiane Floyd. *Software*. de Gruyter, 2. Auflage, 1976.
- [Schwaber und Beedle, 2001] Ken Schwaber und Mike Beedle. *Agile Software Development With Scrum*. Prentice Hall, 2001.
- [Selic, 2003] Bran Selic. The Pragmatics of Model-Driven Development. *IEEE Software*, 20(5):19–25, 2003.
- [Shlaer und Mellor, 1988] Sally Shlaer und Steve Mellor. *Object-Oriented Systems Analysis. Modeling the World in Data*. Yourdon Press; Prentice Hall, 1988.
- [Smith und Weiss, 1988] John B. Smith und Stephen F. Weiss. Hypertext. *Communications of the ACM*, 31(7), 1988.
- [Sneed, 1988] Harry Sneed. *Softwarewartung (Einleitung)*. BI-Verlag, Mannheim, 1988.

- [Sohr, 1983] Dana Sohr. Better Software Manuals. *Byte*, 8(5):286–294, 1983.
- [Sommerville, 2004] Ian Sommerville. *Software Engineering*. Addison Wesley, 7. Auflage, 2004.
- [Spalt, 2004] Alfred Spalt. Testen im Softwareentwicklungsprozess. Vorlesungsunterlagen, Fachhochschule Hagenberg, 4232 Hagenberg, Mai, 2004.
- [Spivey, 1990] J. Michael Spivey. *The Z Notation: A Reference Manual*. Prentice Hall, Englewood Cliffs, NJ, 1990.
- [Stacey, 1995] Deb Stacey. Software Engineering Course Notes. Vorlesungsunterlagen, University of Guelph, Guelph, Ontario, Kanada, 1995.
- [Stephens und Rosenberg, 2003] Matt Stephens und Doug Rosenberg. *Extreme Programming Refactored: The Case Against XP*. a! Press, 2003.
- [Suhr und Suhr, 1993] Rini Suhr und Roland Suhr. *Software Engineering: Technik und Methodik*. Oldenbourg, München Wien, 1993.
- [Szyperski, 1998] Clemens Szyperski. *Component Software: Beyond Object-oriented Programming*. Addison-Wesley, 1998.
- [Tanenbaum, 2001] Andrew S. Tanenbaum. *Modern Operating Systems*. Prentice Hall, 2001.
- [Thayer und Fairley, 1997] Richard H. Thayer und Richard E. Fairley. Software Risk Management. In Richard H. Thayer, Herausgeber, *Software Engineering Project Management*. IEEE Computer Society Press, 1997.
- [Thomsett, 1989] Rob Thomsett. *Third Wave Project Management*. Prentice Hall, 1989.
- [Thomsett, 2001] Rob Thomsett. Extreme Project Management. Technischer Bericht Cutter Executive Report Nr. 2.2, Cutter Consortium, 2001.
- [Tichy, 1988] Walter F. Tichy. Tools for Software Configuration Management. *Softwaretechnik-Trend. Mitteilungen der Fachgruppe Software Engineering, Gesellschaft für Informatik*, 88(8):51–70, 1988.
- [Turkle, 1986] Sherry Turkle. *Die Wunschmaschine: Der Computer als zweites Ich*. rororo Computer 8135, 1986.
- [VDI, 1991] VDI. VDI / VDE Norm 3694: Lastenheft / Pflichtenheft für den Einsatz von Automatisierungssystemen. Technischer Bericht, Verein Deutscher Ingenieure / Verband Deutscher Elektrotechniker, Düsseldorf, Deutschland, 1991.
- [Versteegen, 1994] Gerhard Versteegen. Von Amts wegen – Softwareerstellung nach dem V-Modell. *iX Multi User / Multi Tasking Magazine*, 11, 1994.
- [Versteegen, 2003] Gerhard Versteegen. CASE – quo vadis? Werkzeuggestützte moderne Softwareentwicklung. *Entwickler*, 2003(1):16–19, 2003.
- [Vigenschow, 2004] Uwe Vigenschow. *Objektorientiertes Testen und Testautomatisierung in der Praxis*. dpunkt.verlag, 2004.
- [Wallmüller, 2001] Ernest Wallmüller. *Software-Qualitätsmanagement in der Praxis*. Hanser, 2001.
- [Webster's, 2005] Webster's. *Random House Webster's Unabridged Dictionary*. Random House, 2005.
- [Westphal, 2005] Frank Westphal. *Testgetriebene Entwicklung mit JUnit und FIT*. dpunkt.verlag, 2005.
- [Will, 2004] Torsten T. Will. Check it out! CVS-Nachfolger: Versionierung mit Subversion. *c't*, 2004(18):180–185, 2004.
- [Wills, 1991] Alan Wills. Capsules and Types in FRESCO. In *Proc. European Conference on Object-Oriented Programming 1991, Lecture Notes in Computer Science 512*, Seiten 59–76. Springer, 1991.
- [Wirfs-Brock und Wilkerson, 1989] Rebecca Wirfs-Brock und Brian Wilkerson. Object-Oriented Design: A Responsibility-Driven Approach. In Uwe Beck und Winfried Sommer, Herausgeber, *Proc. OOPSLA'89*, 1989.
- [Wirth, 1971] Niklaus Wirth. Program Development by Stepwise Refinement. *Communications of the ACM*, 14(4), 1971.
- [Yourdon, 1989] Edward Yourdon. *Modern Structured Analysis*. Yourdon Press / Prentice Hall, 1989.
- [Yourdon, 1997] Edward Yourdon. *Death March: The Complete Software Developer's Guide to Surviving „Mission Impossible“ Projects*. Prentice Hall, 1997.
- [Zahrnt, 2002] Christoph Zahrnt. *Vertragsrecht für IT-Fachleute*. Hüthig Telekommunikation, 2002.

- [Zäpfel, 1982] Günther Zäpfel. *Produktionswirtschaft*. de Gruyter, 1982.
- [Zäpfel, 2000a] Günther Zäpfel. *Strategisches Produktions-Management*. Oldenbourg, 2000.
- [Zäpfel, 2000b] Günther Zäpfel. *Taktisches Produktions-Management*. Oldenbourg, 2000.
- [Zhong *et al.*, 2002] Ning Zhong, Jiming Liu, und Setsuo Ohsuga. *Intelligent Agent Technology*. World Scientific Publishing Company, 2002.

Index

- Abbot
 - Methode von, 170
- Abdeckung
 - vollständige, 252
- Abdeckungskriterium, 252
- Abdeckungsmonitor, 236
- Ablage
 - Systematisieren der, 110
- Ablauf
 - ereignisorientierter, 166
- Ablaufdiagramm, 224
- Ablaufkontrolle, 54
- Ablauforganisation, 28, 73
 - Dokumentation der, 77
 - Dokumente der, 115
- Ablaufplanung, 184
- Ablaufstruktur
 - Verdeutlichung der, 254
- Abnahme, 46, 274, 280, 288, 289
 - erfolgreiche, 280, 281
 - erfolgte, 293
 - Vorbereiten der, 261
- Abnahme-Schlussbesprechung, 290
- Abnahmeprotokoll, 281, 290, 292
- Abnahmesituation
 - beim Auftraggeber, 261
- Abnahmesuite, 52, 261, 289
- Abnahmetest, 262, 289
- Abnahmevortest, 261, 289
- Abschluss
 - eines Projekts, 46, 277, 280
- Abschlussbericht, 119, 292
- Abschlussbesprechung, 292, 293
- abstrakte Klasse, 222
- Abstraktion, 163, 170
 - Daten-, 163
 - prozedurale, 163
- Abwandlung
 - Gesetze der, 148
- acceptance, 280
- acceptance test, 289
- acceptance test suite, 265
- actor, 154
- Adäquatheit, 123
- Adaptierung, 40
- administrativer Projektleiter, 114
- administrator, 280
- Ähnlichkeit
 - Gesetze der, 148
- Änderbarkeit, 124, 161
- Äquivalenzklasse
 - Bildung von, 262
- Agenda, 103, 116
- Agent, 221
 - intelligenter, 221
- agentenorientierte Programmierung, 234
- Aggregationsstruktur, 168, 169
- Agiles Manifest, 96
- Aktennotiz, 115
- Akteur, 154
 - aktiver, 154
 - maschineller, 155
 - menschlicher, 155
 - passiver, 154
 - Schnittstellenbeschreibungen zu, 157
- Aktivität, 88, 188
- Aktualität, 232
- Akzeptanz
 - der Benutzerschnittstelle, 289
- Algol, 234
- Alphatesten, 259
- Alphatester, 259
- Alphaversion, 259
- Alternative
 - Ankreuzen von, 142
 - Nennung von, 142
 - Reihung von, 142
- American National Standards Institute, 133
- Analysator
 - statischer, 253
- Analyse, 57, 157, 159, 173
 - der Anforderungen, 159
 - der Machbarkeit, 89
 - dynamische, 235, 251
 - moderne strukturierte, 166
 - Postmortem-, 251, 255
 - prototypingorientierte, 172
 - statische, 247
 - strukturierte, 166
- Analysebericht, 177, 180, 225–227
- Analysefehler, 259
 - Erkennen von, 259
- Analysemethode
 - allgemeine, 171
- Analysemodell, 153
 - Erweiterung des, 214
- Analysemuster, 162, 220
- Analysevorgang, 177
- Analysewerkzeug, 106, 174
- analysis pattern, 162, 220

- Anforderung, 159, 227
 - explizite, 159
 - funktionale, 159
 - implizite, 159
 - nichtfunktionale, 159
 - sich ständig ändernde, 162
 - tatsächlich notwendige, 289
 - Verfolgbarkeit von, 162
 - Verwalten von, 161
- Anforderungsanalyse, 38, 102, 138, 159, 160
 - Aufgaben der, 160
 - Eigenschaften der, 160
 - Ergebnisse der, 180
 - Schwierigkeiten der, 161
 - Tätigkeiten der, 160
 - Techniken der, 162
 - Werkzeuge der, 174
 - Werkzeugunterstützung der, 103
 - Ziel der, 159
- Anforderungsbeschreibung, 46
- Anforderungsdefinition, 180
- Anforderungskatalog, 39
- Angebot, 180
 - unverbindliches, 36
 - verbindliches, 36
- Angebotsleger, 36
- Angebotslegung, 36, 39
- Angemessenheit, 151
- Annahme
 - eines Produkts, 280
- Anpassung, 283
- ANSI, 133
- Anti-Pattern, 78
- Antwortenermittlung, 149
- Anweisungen zur Inbetriebnahme, 230
- Anwender, 33, 114, 280
 - Antwortzeiterfordernisse des, 260
 - Motivation der, 40
- Anwendung
 - generische, 221
- Anwendungsfall, 153, 154
 - erweiterter, 156
 - verwendeter, 156
- Anwendungsfallbeschreibung, 155, 156
 - Strukturierung einer, 156
- Anwendungsfalldiagramm, 155, 157
 - Darstellung des, 155
 - Komponenten eines, 155
- Anwendungsfallmodell, 153, 155
- Anwendungsfallmodellierung, 154
- Anwendungsgebiet
 - Verständnis für das, 161
- application framework, 218, 221
- application library, 220
- Arbeitsablauf, 73
- Arbeitsbericht, 119
- Arbeitsbesprechung, 103
- Arbeitsfortschritts-Demonstration, 89
- Arbeitsgruppe, 63
- Arbeitsplan, 38
- Arbeitsversion, 144
- architecture
 - model-driven, 217
- Architektur, 160
 - modellgesteuerte, 217
 - Modellierung der, 211
- Architekturbeschreibung
 - textuelle, 224
- Architekturdesign, 211, 217
- Architekturprototyp, 52, 217
- Archiv, 110
- Archivar, 85, 114
- archivation, 286
- Archivierung, 286
- Archivierungssystem
 - elektronisches, 112
- Arten
 - der Entwicklung, 52
 - der Führung, 50
 - der Kontrolle, 53
- Aspekt, 26
- aspektorientierte Programmierung, 234
- Assembler, 234
- assembly, 44
- Assertion, 236
- Assessment, 131
- Assoziation, 163
 - Gesetze der, 148
- Attribut, 169
- attributierte Grammatik, 227
- Audit, 129
- Auditor, 129
- Aufbauorganisation, 28, 59
 - Dokumente der, 112
- Aufgabe, 227
- Aufgaben
 - der Überprüfung, 45
 - der Anforderungsanalyse, 160
 - der Durchführung, 44
 - der Entwicklung, 51
 - der Erhebung, 138
 - der Führung, 50
 - der Kontrolle, 53
 - der Organisation, 28
 - der Planung, 44, 183
 - der Produkt-Einführung, 277
 - der Qualitätssicherung, 127
 - der Risikobehandlung, 196
 - der Zielfindung, 145
 - des Codierens, 233
 - des Debuggens, 246
 - des Designs, 210
 - des Testens, 259
- Aufgabenbeschreibung, 37, 180
- Aufgabenplan, 184, 191
- Aufgabenplanung, 183
- Aufgabenstellung
 - unsichere, 38
- Aufgabentrennung, 47
- Aufrufliste, 253

- Auftraggeber, 29, 36, 277, 289, 292
 - Abnahmesituation beim, 261
 - Anforderungen des, 37
 - potenzieller, 36, 138
- Auftragnehmer, 29, 36, 277, 292
 - potenzieller, 39, 138
- Auftragserteilung, 37
- Aufwandsplanung, 184
- Aufwandsrisiko, 29
- Aufwandsschätzung, 38, 55, 184, 187
 - nach Faustregeln, 187
- Aufwandsverteilung, 284
- Ausgabegröße
 - Ist-, 73
 - Soll-, 73
- Ausgabemaske, 212
- Ausgabeparameter, 214
- Ausschreibung, 180
- automatisches Portieren, 125

- Böhm-Bawerk
 - Gesetz von, 192
- Backup, 279
- bad smell, 237
- Balanced Scorecard, 132
- Balkendiagramm, 189
- Balkenplan, 189
- Balkenplantechnik, 189
- Basisinnovation, 41
- Basisklasse, 170
- Basisprodukt, 39, 40
- Basisproduktentwicklung, 40
- Baugruppe, 56
- Baustein
 - Wiederverwenden von, 219, 220
- Bausteinbibliothek, 219
- Bauteil
 - genormter, 56
- Bebuggen, 235
- Bedarfsanalyse, 159
- Bedienbarkeit, 123
- Bediener
 - Rollen von, 280
- Bedienerprotokoll, 291
- Bedienungsablauf
 - nicht beherrschbarer, 259
- Bedingung, 236
 - Einfügen logischer, 236
 - Sine-qua-non-, 143
- Bedingungsabdeckung, 252
- behavior
 - emergent, 76
- behavior model, 154
- Belastungstest, 261
- Benchmarktest, 288
- Benchmarkvergleich, 261
- Benutzbarkeit, 123, 232
- Benutzer, 280
- Benutzerdokumentation
 - Brauchbarkeit der, 289
 - Checkliste für die, 232
- Benutzerfreundlichkeit, 122
- Benutzerhandbuch, 230
- Benutzerschnittstelle, 157, 259, 283
 - Akzeptanz der, 289
 - Beschreibungssprachen für, 226
 - Einfrieren des Aussehens der, 261
 - Generatoren zur Erstellung von, 226
- Benutzerschnittstellen-Generator, 242
 - grafischer, 226
- Benutzerschnittstellenbibliothek, 220
- Benutzerschnittstellendesign, 212, 217
- Benutzerschnittstellenprototyp, 217
- Benutzerschnittstellenprototyping, 212
- Benutzerschulung, 280
- Berater, 30
- Bericht, 116
- Berichtsart, 109
- Berichtswesen, 109
- Berichtswesenplanung, 186
- Beschränkung, 154
- Beschreibung
 - deklarative, 214
 - funktionale, 214
 - prozessorientierte, 155
- Beschreibungssprache
 - Benutzerschnittstellen-, 226
- Besprechung, 103
 - externe, 103
 - interne, 103
- Besprechungsdauer
 - maximale, 104
- Besprechungsstruktur, 103
- best practice, 56
- Betatesten, 260
- Betatester, 260
- Betaversion, 260
- Betrachtungsweise
 - ganzheitliche, 26
 - prozessorientierte, 128
 - technologieorientierte, 128
- Betreuer, 280
- Betreuerschulung, 280
- Betrieb, 277, 279, 281
 - Aufnahme des, 277
- Betriebsführungskonzept
 - des Auftraggebers, 40
- Betriebsversion
 - eines Produkts, 291
- bewährte Praktik, 56
- Bewertungsmatrix, 144
- Bezeichnung
 - einheitliche, 110
- Bibliothek, 85, 213, 216, 225
 - Benutzerschnittstellen-, 220
 - Fach-, 220
 - Kern-, 220
 - Klassen-, 220
 - Komponenten-, 221
 - Modul-, 220

- Universal-, 220
- Big-Bang-Test, 252
- Big-Bang-Testen, 267
- Bildschirmspeicherprogramm, 272
- Bildungskoordinator, 114
- Binden, 242
- Bindleistung, 69
- Blackbox, 48
- Blackbox-Test, 252
- Blackbox-Testen, 258
- Blackbox/Whitebox-Sicht
 - der Projektentwicklung, 48
- BOOTSTRAP, 131
- Bottomup-Design, 216
- Bottomup-Test, 251
- boundary object, 224
- Brainstorming, 147
 - mit Osborne-Verfremdung, 148
- Brauchbarkeit, 123, 268
 - eines Subsystems, 268
- Breakpoint, 251
- brookssches Gesetz, 69
- Browser, 242

- Capability Maturity Model, 129
- Capability Maturity Model Integration, 130
- CASE-System, 106, 158, 174
 - Auswahl eines, 223
 - Meta-, 223
 - offenes, 223
- CCITT, 134
- CEN, 133
- CENELEC, 133
- CFD, 167
- Chance, 192
- checking, 45
- Checkliste, 117
 - für das Lastenheft, 181
 - für das Pflichtenheft, 229
 - für die Benutzerdokumentation, 232
 - projektspezifische, 256
- Chefprogrammierer, 65
 - Team, 64, 66
- class library, 220
- Class-Responsibility-Collaboration-Karte, 174
- Cleanroom Engineering, 95
- client, 221
- CMM, 129
- CMMI, 97, 130, 195
- Coaching, 30
- COCOMO II, 187
- code
 - kilo-lines of, 249
 - lines of, 249
 - source, 243
 - source lines of, 249
- Code, 233
 - Instrumentieren des, 235
 - Komplexitätszahlen für, 249
 - schlechter, 237
 - Testen von objektorientiertem, 268
 - toter, 250
 - Verzweigtheit des, 250
- Code-Restrukturierung, 237
 - erfolgreiche, 237
 - Problem der, 237
- Codeinspektion, 248
- Codelokalität
 - Feststellen der, 235
- Codereview, 248
- Codieren
 - Aufgaben des, 233
 - Ergebnisse des, 243
 - Techniken des, 234
 - Werkzeuge des, 241
 - Ziel des, 233
- Codierung
 - gute, 235
- collaboration, 174
- Comité Consultatif International de Téléphone et Télégraphie, 134
- Comité Européen de Normalisation, 133
- Comité Européen de Normalisation Electrique, 133
- commercial-off-the-shelf product, 202
- Compiler, 178, 242
- Compiler-Compiler, 242
- Compilerbau
 - Werkzeuge des, 227
- component diagram, 224
- component library, 221
- component model, 210
- component test, 267
- component-oriented software development, 171
- Computer, 26
- Computer Aided Software Engineering, 106
- computer supported cooperative work, 151
- concurrency view, 210
- condition, 236
- conditional objective, 145
- configuration & versioning management, 111
- constructive cost model, 187
- Consulting, 30
- continuous delivery, 288
- continuous integration, 241
- continuous release, 288
- control flow diagram, 167
- control object, 224
- core library, 220
- Corporate Identity, 114
- COS, 171
- COTS, 202
- coverage monitor, 236
- CPM, 188
- CRC-Karte, 174
- critical path method, 188
- CRM, 80
- CSCW, 151
- customer relationship management, 80
- CVM, 111

- daily build, 241
- data capturer, 291
- data dictionary, 157
- data flow diagram, 166
- Datei, 212
 - Readme-, 291
- Dateivergleichswerkzeug, 263, 272
- Datenflussanalyse, 166, 251
- Datenflussanomalie, 251
- Datenflussdiagramm, 166
- Datenhaltung, 212
 - Modellierung der, 212
- Datenkapsel, 222
- Datenkapselung, 234
- Datenkatalog, 157
- Datenlexikon, 157, 170, 179, 235
 - Erstellung eines, 158
- Datenmodell
 - semantisches, 168
- Datenstruktur, 167, 212
 - abstrakte, 222
- Datentyp
 - abstrakter, 222
- Datenverzeichnis, 157
- Datenzerlegung, 171
- Deadlock, 250
- Debuggen, 245, 258
 - Aufgaben des, 246
 - Ergebnisse des, 256
 - Techniken des, 247
 - Werkzeuge des, 253
 - Ziel des, 245
- Debugger, 251, 255
- defect
 - second level, 283
- DEK, 133
- Dekomposition
 - funktionale, 164
- Dekompositionsstruktur, 168
- delivery, 277
- Deltaspeicherung, 111
- Deming-Rad, 86
- Demonstrationsversion, 144
- deployment, 218
- deployment diagram, 224
- deployment view, 210
- Design, 38, 45, 57, 157, 173
 - Grundgerüst, 215
 - Architektur-, 211, 217
 - Aufgaben des, 210
 - Benutzerschnittstellen-, 212, 217
 - Big-Picture-, 99, 215, 239
 - Bottomup-, 216
 - Detail-, 210
 - Empfehlungen für ein gutes, 222
 - Ergebnisse des, 227
 - Fein-, 210
 - first, 210
 - Grob-, 210
 - Komponenten-, 213
 - objektorientiertes, 225
 - preliminary, 210
 - Primär-, 210
 - System-, 210
 - Techniken des, 214
 - Topdown-, 83, 216
 - Werkzeuge des, 223
 - Ziel des, 209
- design for change, 161
- design pattern, 220
- design reuse, 221
- Designbericht, 226, 227
- Designfehler, 259
 - Erkennen von, 259
- Designkompromiss, 226
- Designmodell, 154, 209, 223, 227
 - Transformieren des, 234
- Designmuster, 220, 222
- Designpriorität, 226
- Designwerkzeug, 106
 - computergestütztes, 223
- Desk Checking, 248
- Detaildesign, 210
- Detailplanung, 37
- Detailziel, 137
- Deutsche Elektrotechnische Kommission, 133
- Deutsches Institut für Normung, 133
- development, 47, 51
 - by investment, 219
 - model-driven, 217
 - test first, 239
 - test-driven, 239
- DFD, 166
- Diagnoseprogramm, 272
- diagram
 - component, 224
 - deployment, 224
 - package, 224
- Dialog, 154
- Dialogmanager, 227
- DIN, 133
- doer, 113
- Dokument
 - Fehlen aktueller, 72
- Dokumentablage, 111
 - elektronische, 112
- Dokumentablagesystem, 110
- Dokumentation, 37, 108, 277
 - mitlaufende, 110
 - Produkt-, 108
 - Projekt-, 108
 - schriftliche, 36
- Dokumentationsanforderung, 110
- Dokumentationsorganisation, 110
- Dokumentationsplanung, 108
- Dokumentationsrichtlinie, 127
- Dokumentationsverwaltung
 - Werkzeuge zur, 106
- Dokumentationswerkzeug, 106
- Dokumentationswesenplanung, 186

- Dokumentausrwertung, 143
- Dokumentenmanagement
 - System, 112
- Dokumentfreigabe, 110
- Dokumentgestaltung, 114
- Dokumentkoordinator, 114, 120
- Dokumentstatus, 110
- Dokumentstruktur, 111, 115
 - einer Agenda, 116
 - einer Aktennotiz, 115
 - einer Gesprächsnotiz, 116
 - einer Telefonnotiz, 116
 - eines Protokolls, 116
- Dokumentverteilung, 110
- Domänenwissen, 221
- driver, 238
- dummy file, 238
- Dummydatei, 238
- Durchführbarkeitsstudie, 182
- Durchführung, 43, 121
 - Aufgaben der, 44
 - Ziel der, 44
- dynamische Analyse, 235
- dynamisches Verhalten, 172

- Easiest-first-Strategie, 45
- Echtdaten, 289
- Echtzeitsystem, 211
- Eclipse, 107
- ECMA, 134
- Editor, 178, 242
 - syntaxgesteuerter, 242
- Effizienz, 124, 151
- Eiffel, 236
- Eigenfertigung, 185
- Eignungsprüfung, 150
- Eindeutigkeit, 182
- Einführungsprotokoll, 291
- Einfluss-Projektorganisation, 60
- Eingabe, 123
- Eingabegröße, 73
- Eingabemaske, 212
- Eingabeparameter, 214
- Einreichung, 180
- Einsatz
 - Anforderungen für den, 181
- Einschubmethode, 221
- Einzelerhebung, 139
- Einzelkämpfer, 63
- Einzelrisiko, 191
- Einzeltest, 267
- embedded system, 211
- emergent behavior, 76
- Endlosschleife, 250
- Endprodukt, 125
- enduser computing, 218
- engineering
 - forward, 223
 - practice, 82
 - process, 82
 - reverse, 223
 - roundtrip, 106, 223
- Enthält-Beziehung, 156
- Entität, 167
- entity, 167
- entity object, 224
- Entity-Relationship-Diagramm, 168
- Entscheidungsfrage, 141
- Entwickeln
 - anwenderseitiges, 218
 - modellgesteuertes, 217
- Entwicklung, 47, 48, 51
 - Arten der, 52
 - Aufgaben der, 51
 - durch Investition, 219
 - eines Produkts, 43
 - Einzel-, 52
 - konkurrierende, 52
 - modellgesteuerte, 99
 - parallele, 52
 - Schwierigkeiten der, 55
 - serielle, 52
 - testgesteuerte, 239, 254, 267, 271
 - testgetriebene, 239
 - Turn-Key-, 30
 - verteilte, 52
 - Ziel der, 51
- Entwicklung der Prozessbedeutung, 76
- Entwicklungsleistung
 - Umfang an, 32
- Entwicklungsmodell, 79
- Entwicklungsprozess, 107
- Entwicklungsprozess-Modell, 80
- Entwicklungsstrategie
 - inkrementelle, 84
- Entwicklungsumgebung, 38, 241
 - Open-Source-, 255
- Entwicklungswerkzeug, 128
- Entwurf, 209
- ERD, 168
- Ereignis, 167, 188
- Ereignisantwort, 167
- Erfüllungsgrad, 38
- erfolgreiche Abnahme, 281
- Ergänzungsfrage, 141
- Ergebnis, 37, 73, 123
 - produktbezogenes, 74
 - prozessbezogenes, 74
 - Wiederverwendbarkeit eines, 162
- Ergebnisse
 - der Anforderungsanalyse, 180
 - der Erhebung, 144
 - der Planung, 191
 - der Produkt-Einführung, 291
 - der Qualitätssicherung, 132
 - der Risikobehandlung, 206
 - der Zielfindung, 157
 - des Codierens, 243
 - des Debuggens, 256
 - des Designs, 227

- des Testens, 273
- Ergonomie, 212
- Erhebung, 138, 151
 - Aufgaben der, 138
 - Einzel-, 139
 - Ergebnisse der, 144
 - Fragebogen-, 141
 - Primär-, 139
 - repräsentative, 139
 - Sekundär-, 139
 - Techniken der, 139
 - Total-, 139, 141
 - Ziel der, 138
- Erhebungsauswertung, 144
- Erlernbarkeit, 123
- Erweiterbarkeit, 124
- Erweitert-Beziehung, 156
- Erweiterung, 283
- essential objective, 145
- estimating
 - proxy-based, 187
- Ethik, 68
- European Computer Manufacturers
 - Association, 134
- evaluation, 285
- Evaluierung, 46, 285
- event, 167
- evolutionäres Prototyping, 91
- evolvierende Funktionalität, 30
- Exemplarisierung
 - multiple, 169
- Experte, 114
- externe Projektbesprechung, 103
- eXtreme Programming, 237
- Führung, 47–49
 - Arten der, 50
 - Aufgaben der, 50
 - kaufmännische, 50
 - Schwierigkeiten der, 55
 - Situierung der, 50
 - technische, 50
 - Ziel der, 49
- Führungssicht
 - kaufmännische, 50
 - technische, 50, 55
- Führungsverständnis, 29
- Fachbibliothek, 220
- Fachexperte, 30
- Fachvorgesetzter, 61
- facilitator, 113
- Fallstudie, 139
- feasibility study, 182
- feature point, 187
- Fehler
 - durch Wartung verursachte, 283
 - Finden von, 253, 257
 - nicht reproduzierbarer, 275
 - projektspezifischer, 256
- Fehleranfälligkeit, 283
- Fehlerbehebungsprozess
 - persönlicher, 247
- Fehlerbericht, 275
 - Formular für einen, 275
- Fehlerliste, 249, 256
- Fehlerlogbuch, 276
 - Eintrag im, 276
- Fehlermeldesystem, 272
- Fehlermeldung, 123
- Fehlerrate, 122
- Fehlersuche
 - Strukturierung der, 256
- Fehlerursache
 - Behebung von, 283
- Fehlerwirkung, 283
- Feindesign, 210
- final report, 292
- finalization, 280
- Finanzierungskontrolle, 54
- Finanzierungsplan, 191
- Finanzierungsplanung, 186
- first design, 210
- Flussdiagramm, 165
- Folgeprojekt, 40
- Formatierwerkzeug, 243
- Formatkonvertierer, 272
- Fortschrittsbewertung, 50
- Forward Engineering, 223
- FP, 187
- Frage
 - abschließende, 141
 - aufschließende, 141
 - direkte, 141
 - Entscheidungs-, 141
 - Ergänzungs-, 141
 - geschlossene, 141
 - indirekte, 141
 - offene, 141
 - W-, 141
- Fragebogen, 139–141
- Fragebogendramaturgie, 142
- Fragebogenerhebung, 141
- Fragebogenkonstruktion, 142
- Fragenreihenfolge, 142
- Fragetypen, 141
- framework
 - application, 221
- Freigabe
 - kontinuierliche, 288
- Freiraum, 69, 184
- Fresco, 178
- function point, 187
- functional test, 267
- Funktion, 104, 165
- funktionale Dekomposition, 164
- funktionale Zerlegung, 234
- Funktionalität, 104
 - ausständige, 289
 - definierte, 30
 - evolvierende, 30

- geforderte, 91
- gelieferte, 91
- notwendige, 289
- Testen der, 267
- Funktionalitätenliste, 275
- Funktionalitätstest, 267
- Funktionserforschungsprototyp, 52
- Funktionstest, 267
- Gantt-Chart, 189
- Gap-Analyse, 131
- Garantie, 37, 262, 281
- Gegensatz
 - Gesetz des, 148
- Geheimnisprinzip, 222
- gelieferte Funktionalität, 91
- general purpose library, 220
- Generalisierung, 176, 216
- Generalisierungsstruktur, 168
- Generator, 178
 - Benutzerschnittstellen-, 242
- Generatorsystem, 178
- Gerüstbau, 238
- Gesamtkosten, 37
- Gesamtplan, 44
- Gesamtprodukt, 277
- Gesamtrisiko, 191
- Gesamtsystems
 - Testen des, 268
- Gesamttest, 268, 289
- Gesamtziel, 137, 145
- Gesetz
 - brookssches, 69
 - parkinsonsches, 184
- Gesetz von Böhm-Bawerk, 192
- Gesichtspunkt
 - verwaltungstechnischer, 30
- Gesprächsnotiz, 116
- Gewährleistungsplanung, 186
- Glassbox-Test, 252
- Glossar, 158
- goal, 137
- Grammatik
 - attributierte, 227
- Grenzwertanalyse, 262
- Grobdesign, 210
- Grobziel, 137, 145
- Groupware, 151
- Groupwaresystem, 151
- Gruppe, 63
- Gruppenarbeit, 66
- Gruppengröße
 - optimale, 69
- Gruppeninterview, 139
- GUI builder, 226
- Hacken, 94, 97
- Hackertum, 94
- Haftnotizen-Objekt, 175
- Hardest-first-Strategie, 45
- Hardware
 - Produktlebensdauer von, 41
- Hardwarearchitektur, 211
- Harel-Notation, 177
- Hausverstand, 198
- Hierarchie, 165
- Hierarchisierung, 170
- Histogramm, 189
- Hollywood-Prinzip, 221
- hook, 221
- Hypertext, 178, 245
- Hypertextsystem, 178
- Ideenbewertungsphase, 147
- Ideengenerierungsphase, 147
- IEC, 133
- IEEE, 74, 134
- Implementierung, 45, 57, 173
- Implementierungsfehler
 - Erkennen von, 259
- Implementierungswerkzeug, 106
- Inbetriebnahme, 279, 291
 - Anforderungen für die, 181
 - Anweisungen zur, 230
 - erfolgreiche, 281
- Inbetriebnahmeprotokoll, 291
- Industriestandard, 134
- information hiding, 163, 222
- Information Modeling, 167
- Informationssystem, 178
- ingenieurmäßige Projektentwicklung, 34, 47
 - Aufgabenbereiche der, 57
 - Schwierigkeiten der, 55
- ingenieurmäßige Softwareentwicklung, 34
- Ingenieursdisziplin, 32
- Initialisierung, 35
- Inkrementgröße, 95
- innerer Test, 252
- Installation, 279, 291
 - erfolgreiche, 281
- Installationsanleitung, 230
- Installationsplanung, 186
- Installationsprogramm, 290
- Installationsprotokoll, 291
- Installationstest, 287
- installer, 290
- Instanzierung, 169
- institutionale Organisation, 27
- institutionale Projektführung, 49
- instrumentale Organisation, 27
- Instrumentieren
 - des Codes, 235
- Instrumentierer, 236
- integration
 - continuous, 241
- Integration
 - kontinuierliche, 241, 255
 - tägliche, 241
- Integrationstest, 267
- Integrierbarkeit, 151

- integrierte Kontrolle, 54
- Integritätstest, 268
- Interaktion
 - nachrichtenbasierte, 167
- Interaktionsdiagramm, 224
- Interaktionsstruktur, 154
- interface, 26
- interface checker, 291
- International Electrotechnical Commission, 133
- International Institute of Electrical and Electronic Engineers, 134
- International Standardization Organization, 129, 133
- interne Projektbesprechung, 103
- Interpreter, 178
- Interview, 139
 - halb strukturiertes, 140
 - nicht strukturiertes, 140
 - strukturiertes, 140
- Interviewer, 139
- Interviewleitfaden, 140
- Interviewprotokoll, 140
- Interviewter, 139
- introduction, 277
- invariant, 236
- Invariante, 236
- invocation test, 279
- Irrtumsrate
 - mentale, 257
- ISO, 74, 129, 133
- ISO 9000 ff., 129
- ISO 9001:2000, 129
- ISO/IEC 15504, 132, 195
- ISO/IEC 90003:2004, 74, 129
- Istanalyse, 37
- Istzustand, 36, 37
 - Beschreibung des, 180
 - Darstellung des, 38
 - Erhebung des, 160
- Iterationsgeschwindigkeit, 95

- Java, 234
- Java Beans, 221
- Joint Venture, 30
- JUnit, 254

- Körpersprache, 140
- Kaizen, 86
 - Prinzip, 86
- Kapazität der Problemlösung, 27
- Kapazitätskontrolle, 54
- Kapazitätsplanung, 185
- Kapselung, 163
 - von Attributen, 170
 - von Methoden, 170
- Katastrophenplan, 114
- Katastrophenplanung, 186
- kaufmännische Führung, 50
- Kernbibliothek, 220
- key process area, 130
- kilo-lines of code, 249
- Klasse, 169
 - abstrakte, 221, 222
 - reine abstrakte, 221
 - wieder verwendbare, 170
- Klassenaufgabe, 174
- Klassenbibliothek, 56, 85, 220, 243
- Klassenmanager, 114
- Klassentest, 267
- Klassifikation, 170
- Klassifikationsstruktur, 169
- Klassifizierung
 - von Begriffen, 169
- KLOC, 249
- Know-how, 43
 - Wiederauffindung von, 286
- Kommunikation, 67
 - in Projekten, 101
 - mündliche, 101
 - mittels Nachrichten, 163, 169
 - schriftliche, 101
 - zwischenmenschliche, 161
- Kommunikationsstruktur
 - im Team, 45
- Kommunikationszeit, 69
- Kompatibilität, 229
- Kompatibilitätstest, 274
- Kompatibilitätstesten, 259
- Kompetenz
 - soziale, 67
- Kompetenztrennung, 47
- Komplexität
 - Beherrschen der, 162
 - innere, 250
- Komplexitätsanalyse, 249, 253
- Komponente, 160, 171, 210, 221
 - aktive, 221
- Komponentenbibliothek, 221
- Komponentendesign, 210, 213
 - Umsetzen des, 233
- Komponentendiagramm, 157, 224
- Komponentenliste, 253
- Komponentenmodell, 210, 223, 225
- komponentenorientierte Programmierung, 234
- Komponentenstruktur
 - Modellierung der, 213
- Komponententest, 239, 267
- Komponentenverhalten
 - Modellierung des, 214
- Konfiguration, 110, 111
- Konfigurationsverwaltung, 111
- Konfigurationsverwaltungssystem, 111, 241
- Konflikt, 67
- Konformanztest, 263, 274
- Konkurrenzprodukt
 - Alleinvorteile gegenüber, 264
- konkurrierende Entwicklung, 52
- Konsistenz, 182
- kontinuierliche Verbesserung, 85
- Kontrolle, 47, 48, 53

- Ablauf-, 54
- Arten der, 53
- Aufgaben der, 53
- der Wirtschaftlichkeit, 54
- Finanzierungs-, 54
- integrierte, 54
- Kapazitäts-, 54
- Kosten-, 54
- Leistungs-, 54
- Leistungsfortschritts-, 54
- Qualitäts-, 54
- Schwierigkeiten der, 56
- technische, 54
- Termin-, 54
- Ziel der, 53
- Kontrollflussdiagramm, 167
- Korrektheit, 122
- Korrekturmaßnahmen
 - bei Planabweichungen, 45
- Kosten, 28
 - für Pflege, 284
 - für Wartung, 284
- Kosten-Nutzen-Analyse, 196
- Kostenentwicklung
 - Kontrolle der, 72
- Kostenkontrolle, 54
- Kostenplan, 191
- Kostenplanung, 186
- Kostensteigerung, 72
- Kreativität, 145
- Kreativitätsgruppe
 - interdisziplinäre, 146
- Kreativitätspotenzial, 145
- Kreativitätstechnik, 145, 146
 - Merkmale der, 146
 - organisatorische Voraussetzungen für, 146
- Kreuzreferenzliste, 254
- kritischer Pfad, 188
- kritischer Vorgang, 188
- KROKUS-Regel, 141
- Kunde, 36
 - Projektbegleitung des, 30
- Kundenanforderung, 154
- Kundenbindungsmaßnahme, 80
- Kundeneinbindung
 - Häufigkeit der, 101
- Kundenzufriedenheit, 258
- Kurzeinführung, 230
- Lösungsmöglichkeit, 55
- Lösungsraum, 215
- Lastenheft, 36, 37, 158, 180, 263
 - Checkliste für das, 181
 - Erstellung eines, 181
- Laufzeit, 69
- Laufzeitsystem, 178
- lean management, 51
- Lebensdauer
 - von Software, 41
- Lebenszyklus
 - Projekt-, 77
 - Software-, 77
- Legacy-Software, 193
- Leistung, 123
 - Bind-, 69
 - Nutz-, 69
- Leistungsbericht, 118
- Leistungsfähigkeit, 69
- Leistungsfortschrittskontrolle, 54
- Leistungskontrolle, 54
- Leistungsprozess
 - eines Unternehmens, 47
- Leistungstest, 288
- Leistungsverbesserung, 283
- Leistungsverzeichnis, 228
- Leitergruppe, 66
- Lesbarkeit, 124
- librarian, 114
- library, 220
 - application, 220
 - class, 220
 - component, 221
 - core, 220
 - general purpose, 220
 - user interface, 220
- Liefertermin, 30
- Lieferung, 277
- life cycle
 - project, 77
 - software, 77
- lines of code, 94, 249
- Linienorganisation
 - Projektleitung in der, 67
- Linken, 242
- Linker, 242
- Listengenerator, 253
- LOC, 249
- Logfile-Ersteller, 291
- logical view, 153
- logische Sicht, 153
- Lollipop-Notation, 214
- Maßnahme
 - analytische, 128
 - begleitende, 186
 - konstruktive, 128
 - organisatorische, 127
 - vorsorgende, 186
- Maßschneidern, 279
- Machbarkeit, 160, 181
 - Abschätzung der, 37
 - Analyse der, 89
 - Feststellung der, 38
 - personelle, 182
 - technische, 182
 - wirtschaftliche, 182
- Machbarkeitsfeststellung, 88
- Machbarkeitsstudie, 182
- mainstream testing, 265
- maintenance, 281, 284

- adaptive, 284
- corrective, 284
- perfective, 284
- Makrofunktion, 242
- management, 47, 49
 - lean, 51
 - requirements, 161
- Maschinennähe, 234
- Maschinensprache, 234, 242
- Masterplan, 44, 75
- Matrix-Projektorganisation, 61
- maturity level, 130
- MDA, 217
- MDD, 99, 217
- Meilenstein, 50, 71, 92, 184, 188
 - Abnahme eines, 39
 - inhaltsbetonter, 75
 - zeitbetonter, 75
- Meilensteinliste, 191
- Meilensteinplanung, 184
- Meilensteintechnik, 39
- memory
 - checker, 254
 - leak, 261
 - leak finder, 254
- message, 168
- meta object facility, 217
- Meta-CASE-System, 223
- Methode, 168, 169
 - abstrakte, 222
- Methode von Abbot, 170
- Methodenkrieg, 153
- Methodik
 - der Beschreibung, 82
 - des Vorgehens, 82
- metra-potential method, 188
- Metrik, 94
- Mikromanagement
 - exzessives, 97
- Mil.Stdd., 134
- Minimalität, 182
- Mitarbeiter
 - Auswahl eines, 36
 - Flexibilität eines, 29
 - hoch qualifizierter, 45
 - Konfliktlösungsfähigkeit eines, 29
 - Kooperationsbereitschaft eines, 29
 - Motivation der, 68
- Mitarbeiterführung, 67, 68
- Mitarbeitermotivation, 68
- Mitglieder
 - Kommunikation der, 67
- model, 26
 - component, 210
 - platform-independent, 217
 - platform-specific, 218
 - system, 210
- model-driven development, 99
- Modell, 26
 - objektorientiertes, 85
 - plattformspezifisches, 218
 - plattformunabhängiges, 217
- modellbasiertes Vorgehen, 79
- Modellierung
 - der Architektur, 211
 - der Datenhaltung, 212
 - der Komponentenstruktur, 213
 - der Schnittstellen, 211
 - des Komponentenverhaltens, 214
- moderne strukturierte Analyse, 166
- Modifizierbarkeit, 182
- Modul, 222
- Modularität, 160, 232
- Modulbibliothek, 220
- Modultest, 267
- MOF, 217
- monitoring, 47, 53
- Monitorprogramm, 272, 291
- Moral, 68
- Morphologischer Kasten, 150
- Motivation
 - der Anwender, 40
 - der Mitarbeiter, 68
 - Faktoren der, 68
- MPM, 188
- Multi-Projektführung, 50
- Multi-Projektorganisation, 113
- Mussziel, 145
- Muster, 213, 216, 225
 - Analyse-, 220
 - Design-, 220
 - Prozess-, 220
 - Verwenden von, 219, 220
- Mutation, 236
- Nachanalyse
 - auftragnehmerinterne, 285
- Nachbedingung, 214, 236
- Nachbesprechung, 103, 104
- Nachkalkulation, 46, 290
 - des Projekts, 285
 - Ergebnisse der, 292
- Nachricht, 168
 - Kommunikation mittels, 163, 169
- Nachrichtenkommunikation, 163
- Nebenläufigkeitssicht, 210
- Netzplan, 187
- Netzplantechnik, 189
- Netzwerkarchitektur, 211
- Nonkonformanztest, 263, 274
- Norm, 132
- Normalisierung, 168, 212
- Normatives Prozessdenken, 76
- Normenauswahl, 135
- Normungsgremium, 133
- Normungsinstitution, 134
- Notfallplan, 114
- Notfallplanung, 186
- Nutzleistung, 69
- Nutzung, 40

- Nutzwertanalyse, 144
- object
 - boundary, 224
 - control, 224
 - entity, 224
- Object Modeling Group, 153
- object point, 187
- Object-Z, 178
- objective, 137
 - capturing, 138
 - conditional, 145
 - determination, 137
 - elicitation, 145
 - essential, 145
 - optional, 145
- Objekt, 25, 167–169
 - Problembereichs-, 224
 - Schnittstellen-, 224
 - Steuerungs-, 224
- objektorientierte Programmierung, 169
- objektorientierte Zerlegung, 168, 176
- objektorientiertes Denken, 92
- objektorientiertes Design, 225
- objektorientiertes Vorgehen, 85
- ÖNORM, 133
- Österreichisches Institut für Normung, 133
- OMG, 153
- Online-Hilfe, 231, 280
- OP, 187
- opportunity, 192
- Optimierung, 40, 283
 - Vermeiden von, 235
- optional objective, 145
- Organigramm, 112
- Organisation, 26, 49
 - Ablauf-, 28, 73
 - Aufbau-, 28, 59
 - Aufgaben der, 28
 - institutionale, 27
 - instrumentale, 27
 - projektbezogene, 59
 - Systemdeterminanten der, 26
 - Ziel der, 26
- Organisationsbegriff, 27
- Organisationsform
 - allgemeine, 59
 - softwareentwicklungsspezifische, 62
 - zweidimensionale, 61
- Organisationshandbuch, 117
- Organisationsmangel, 71
- Organisationsmethode
 - primäre, 163
- Organisationsplanung, 183
- Organisationsstruktur
 - besondere Anforderungen an eine, 62
 - spezielle, 63
- Organisationsverständnis, 29
- Osborne-Verfremdung, 148
- Outside-in-Konstruktion, 221
- package, 155
- package diagram, 224
- packaging, 155
- Paket, 154, 155, 224
- Paketierung, 155
- parallele Entwicklung, 52
- Pareto-Prinzip, 256
- parkinsonsches Gesetz, 184
- Partitionierung, 155
- Pascal, 234
- pattern, 93
 - analysis, 220
 - anti-, 78
 - design, 220
 - process, 220
- Pattern, 78
- pattern matching, 93
- Performanz, 260
- Performanztest, 260, 271, 274
- Performanzverbesserung, 261
- Performanzvorgabe, 261
- Periode, 50
- personal software process, 184
- Personalbedarf, 185
- Personaleinsatzplanung, 185
- Personalplanung, 185
- Personalstruktur
 - im Unternehmen, 36
- Personalvorgesetzter, 61
- PERT, 188
- Peterprinzip, 64
- Pfadabdeckung
 - Prüfen der, 235
- Pflege, 277, 282
 - Aktivitäten der, 283
 - Aufwand für, 285
 - Kosten für, 284
 - Maßnahmen der, 284
 - Produktivität der, 285
 - von Software, 283
- Pflegeaktivität, 285
- Pflegekosten, 284
- Pflichtenheft, 36, 37, 158, 215, 228, 263, 275
 - Checkliste für das, 229
- Phase, 37, 77, 88
- Phasenmodell, 82
- physical view, 154, 209
- physische Sicht, 154, 209
- pilot project, 37
- Pilotanwender, 102, 114, 260
- Pilottest, 288
- PIM, 217
- Plan
 - Master-, 44
- Planabweichung, 285
 - Korrekturmaßnahmen bei, 45
- planning, 44
- Planung, 43, 121
 - Ablauf-, 184
 - Aufgaben der, 44, 183

- Aufgaben-, 183
- Aufwands-, 184
- Berichtswesen-, 186
- des Testablaufs, 264
- Dokumentationswesen-, 186
- Ergebnisse der, 191
- Finanzierungs-, 186
- Gewährleistungs-, 186
- Installations-, 186
- Kapazitäts-, 185
- Katastrophen-, 186
- Kosten-, 186
- mangelhafte, 55
- Meilenstein-, 184
- Notfall-, 186
- Organisations-, 183
- Personal-, 185
- Personaleinsatz-, 185
- projekt übergreifende, 62
- Qualitätssicherungs-, 186
- Ressourcen-, 185
- Sachmittel-, 185
- Sachmitteleinsatz-, 185
- Schulungs-, 186
- sequenzielle, 44
- Sicherheits-, 186
- Sicherungs-, 186
- Techniken der, 187
- Termin-, 184
- Test-, 186
- Wartungs-, 186
- Werkzeuge der, 189
- Zeit-, 184
- Ziel der, 44, 183
- Ziel-, 183
- Planungsaufgabe, 62
- Planungsbüro, 62
- Planungswerkzeug, 106
- Plausibilität, 182
- Portabilität, 125
- Portierbarkeit, 114
- Portieren, 114
 - automatisches, 125
- porting maintainer, 114
- post analysis, 290
- postcondition, 214, 236
- Postmortem-Analyse, 251, 255
- potenzieller Auftraggeber, 36
- PR-Verantwortlicher, 114
- Praktik
 - bewährte, 56
- precondition, 214, 236
- preliminary design, 210
- Prettyprinter, 243
- Primärdesign, 210
- Primärerhebung, 139
- Priorisierung, 45
- Probe, 235
- ProBE, 187
- Probetrieb, 287
- Probelauf, 287
- Problem, 191, 192
 - Abgrenzung des, 35
 - potenzielles, 191
 - vorhersehbares, 192
- Problembereich
 - Abgrenzung des, 160
- Problembereichsmodell, 153
- Problembereichsobjekt, 224
- Problembeschreibung
 - formal korrekte, 159
 - verbale, 170
- Problemdefinition, 149
- Problemlösung
 - Kapazität der, 27
- Problemraum, 215
- Problemspezifikation, 159
- Problemstrukturierung, 167
- Problemzerlegung, 159
- process pattern, 220
- process tailoring, 81
- product
 - commercial-off-the-shelf, 202
- production, 44
- production version, 291
- Produkt, 32, 36, 83, 92
 - Annahme eines, 280
 - Einrichtung eines, 279
 - Entwicklung eines, 43
- Produkt-Einführung, 45, 57
 - Aufgaben der, 277
 - Ergebnisse der, 291
 - Techniken der, 286
 - Werkzeuge der, 290
 - Ziel der, 277
- Produktbeschreibung
 - allgemeine, 230
- Produktdokumentation, 108, 158
- Produktentwicklung
 - Planung einer, 43
 - Qualität in der, 125
 - technische, 34
- Produktentwicklungszyklus, 43, 57
- Produkterstellung, 39
- Produktevaluierung, 287
- Produktherstellung
 - wirtschaftliche, 34
- Produktionsprozess, 62
- Produktivität, 249
 - einer Projektgruppe, 69
- Produktlebensdauer
 - von Hardware, 41
 - von Software, 41
- Produktperformanz
 - Testen der, 260
- Produktqualität, 127
 - Erhöhung der, 258
- Produktspezifikation
 - vollständige, 275
- Produktstatus

- Bestimmen des, 259
- Produktverbesserung, 281
- Produktvergleich, 143, 144
 - Techniken des, 144
- Produktversion
 - instabile, 265
- Produktziel, 273
- Profession
 - des Softwareingenieurs, 68
- Profiling, 235
- program evaluation and review technique, 188
- Programm, 33
 - lauffähiges, 233, 256
- Programmanalyse
 - dynamische, 251
 - statische, 247
- Programmcode
 - ausführbarer, 242
- Programmgenerator, 242
- Programmieren
 - als Kunst, 33
 - als Technik, 33
- Programmierer, 33
 - Individualität der, 55
- Programmierheer, 63, 66
- Programmierleistung, 69
- Programmiersprache, 234
- Programmierstil
 - guter, 284
- Programmierungsumgebung, 241
- Programmierung, 27
 - agentenorientierte, 234
 - aspektorientierte, 234
 - Beginn der, 234
 - Entwicklungsschritte der, 234
 - Kernaufgabe der, 234
 - komponentenorientierte, 234
 - modulorientierte, 234
 - objektorientierte, 169, 234
 - strukturierte, 234
- Programmierzeit, 69
- Programmqualität, 249
- Programmschwierigkeit, 250
- Programmstrukturplan, 254
- Programmtest, 267
- Programmumfang, 69
- Programmverifikation, 247, 253
- Programmvolumen, 250
- project life cycle, 77
- Projekt, 28
 - Ablauf eines, 108
 - Abschluss eines, 277
 - Dokumentation eines, 108
 - Eigenschaften eines, 28
 - Einführung in ein, 180
 - Einteilung von, 29
 - externes, 29, 35
 - Gesamtziel eines, 173
 - internes, 29, 35
 - Kommunikation in, 101
 - modernes Spannungsfeld eines, 30
 - Organisationsmängel bei, 71
 - Schwierigkeit eines, 29
 - Spannungsfeld eines, 30, 32
 - traditionelles Spannungsfeld eines, 30
 - Zielvorgabe eines, 137
- Projekt Engineering, 34, 47
 - Praktiken, 99
 - Aufgabenbereiche des, 57
 - Aufgabenschema des, 57
 - Metriken des, 94
 - Schwierigkeiten des, 55
 - Software-, 34
- projektübergreifende Planung, 62
- projektübergreifende Qualitätssicherung, 80
- Projektablauf, 39
- Projektabschluss, 277, 280
 - externer, 46
- Projektabschnitt, 37
- Projektanbahnung, 35, 138
- Projektansatz
 - holistischer, 66
- Projektanzahl, 29
- Projektarchiv, 293
- Projektassistent, 65
- Projektaufsicht, 53, 113, 118, 119
- Projektauftrag, 113, 138
- Projektausgangslage, 138
- Projektbedeutung, 29
- Projektbeginn, 28, 37
- Projektbegriff, 28
- Projektbericht, 116
- Projektbesonderheit, 28
- Projektbesprechung, 103
 - externe, 103
 - interne, 103
- Projektbezeichnung, 36
- Projektbibliothek, 65, 110, 119
 - Führen der, 120
- Projektbudget, 113
- Projektdauer, 28
- Projektdokumentation, 108
- Projektdurchführung, 38
- Projekteigenschaft, 37
- Projektende, 28, 30, 37
- Projektentwicklung, 35, 47, 49
 - Anforderungen an die, 181
 - Blackbox/Whitebox-Sicht, 48
 - ingenieurmäßige, 34, 47, 55
 - Regelkreis der, 49
 - Reifegrad der, 56
 - Schwachstellen der, 72
 - Schwierigkeiten der, 55
 - Software-, 34
- Projektentwicklungsprozess, 127, 285
- Projektentwicklungszeit, 69
- Projektentwicklungszyklus, 35, 57
- Projektergebnis, 39, 108
- Projektführer, 114
- Projektführung, 49

- Inkompetenz der, 71
- institutionale, 49
- Multi-, 50
- Situierung der, 50
- Projektfortschritt, 50
 - Messen des, 94, 250
- Projektfortschrittsbericht, 46, 53, 118, 199
- Projektfortschrittskontrolle, 72, 118
 - Mangel an, 72
- Projektgliederung, 38
- Projektgrundlage
 - Prüfen der, 127
- Projektgruppe, 36, 66, 105, 185
- Projekthandbuch, 117
- Projektinitialisierung, 35
- Projektintensität, 29
- Projektkomplexität, 28
- Projektkontinuität, 29
- Projektkontrolle, 53
- Projektkooperation
 - Art der, 29
- Projektkoordinator, 113
- Projektkosten, 29
- Projektlebenszyklus, 77
- Projektleiter, 32, 60, 67, 103, 113, 278, 292
 - administrativer, 114
 - Kompetenzen des, 49
 - stellvertretender, 114
- Projektleitung
 - in der Linienorganisation, 67
- Projektmanagement, 49
- Projektmanagementsystem, 189
- Projektmanager, 51, 114
- Projektmitarbeiter, 60
 - überlasteter, 29
 - ausgelasteter, 29
 - hauptberuflicher, 29
 - sporadischer, 29
 - teilzeitlicher, 29
 - unterbeschäftigter, 29
- Projektorganisation, 57, 59
 - Einfluss-, 60
 - Festlegen der, 183
 - Matrix-, 61
 - Multi-, 113
 - reine, 59
- projektorientierte Teilorganisation, 67
- Projektpartner, 35
- Projektphase, 58
- Projektplan, 46
- Projektplanung, 44
 - Fehlen von, 71
 - Komplexität der, 44
- Projektplanungssystem, 189
- Projektpolitik, 50, 113
- Projektpriorität, 113
- Projektprozess-Modell, 80
- Projektregelung, 49
- Projektrestriktion, 35
- Projektrisiko, 29, 191, 285
- Projektschritt, 44
- Projektschwierigkeit, 29
- Projektsekretär, 65
- Projektsicherheit, 39
- Projektsicht
 - betriebswirtschaftliche, 30
 - soziologische, 32
 - technische, 30
- Projektstab, 63
- Projektstandard, 71, 284
- Projektsteuerung, 49
- Projektstrukturplan, 191
- Projekttagebuch, 118
- Projektträger, 49, 119
- Projektumfang, 28
 - Abgrenzung des, 161
 - Abschätzung des, 160
- Projektumfeld, 138
- Projektverantwortung, 30
- Projektverlauf, 109
- Projektverzug, 37
- Projektvorbereiten, 138
- Projektziel, 38, 180
 - Umsetzung des, 51
- Projektzweck, 28
- protocoller, 291
- Protokoll, 104, 116
 - Besprechungs-, 46
 - Stichwort-, 104
 - Volltext-, 104
- Protokollführer, 104
- Protokolliersystem, 291
- Prototyp, 86, 87, 92
 - Arten von, 172
 - Auslieferung von, 92
 - breiter und flacher, 172
 - enger und tiefer, 172
 - evolutionärer, 173, 211
 - revolutionärer, 173
 - unvollständiger, 172
 - vollständiger, 172
 - Wegwerf-, 173
 - wieder verwendbarer, 173
- Prototyping, 86, 87, 216
 - Benutzerschnittstellen-, 212
 - evolutionäres, 91, 216, 217
 - experimentelles, 88, 216
 - exploratives, 88
 - herkömmliches, 87
 - Rapid, 217
 - revelationäres, 88
 - Werkzeuge des, 177, 226
- Prototypingwerkzeug, 106, 157, 226
 - Beurteilung von, 178
- Prozedur, 234
- prozedurale Abstraktion, 163
- Prozess, 28, 73
 - Adaptierung eines, 93
 - Bedeutung des, 193
 - definierter, 74, 93

- der Softwareentwicklung, 75
 - destruktiver, 258
 - empirischer, 75
 - funktionaler, 165
 - geregelter, 73
 - gruppendynamischer, 67
 - nebenläufiger, 171
 - normative Bedeutung, 76
 - Optimierung eines, 93
 - transformierende Bedeutung, 76
 - wertgenerierende Bedeutung, 76
- Prozessanalyse, 99
- Prozessanpassung, 81, 82
- Prozessanwendung
 - systematische, 93
- Prozessbedeutung
 - Entwicklung der, 76
- Prozessdatenverarbeitung, 211
- Prozessdenken
 - Normatives, 76
- Prozessfunktion, 165
- Prozessidentifikation, 94
- Prozesskette, 73
- Prozessmodell, 79, 82
 - Topologie, 79, 80
 - Gültigkeitsbereich, 79
 - Skalierung eines, 81
 - Vereinfachung eines, 81
- Prozessmuster, 220
- Prozessökonomie, 96, 97
- Prozessoptimierung, 93
- prozessorientiert, 128
- prozessorientierte Betrachtungsweise, 128
- prozessorientiertes Vorgehen, 79
- Prozessreife, 131
- Prozessskalierung, 81
- Prozessverbesserung, 93, 99, 131
- Prozessverständnis, 94
- Prozesswissen
 - normatives, 76
 - transformationales, 76
 - wertgenerierendes, 76
- PSM, 218
- PSP, 191
- qualification test suite, 265
- Qualifikationstest, 265, 274
- Qualifikationstestsuite, 265
- Qualität, 121
 - bestmögliche, 121
 - der Zwischenprodukte, 125
 - des Endprodukts, 125
 - in der Produktentwicklung, 125
 - mangelnde, 71
 - technische, 229
 - totale, 130
 - von Software, 121
- Qualitätsanforderungen, 121, 181
- Qualitätsbewertung, 121, 131
- Qualitätsgesteuerter
 - Softwareentwicklungsprozess, 126
- Qualitätsgrad, 131
- Qualitätshandbuch, 132
- Qualitätskontrolle, 54
 - Mangel an, 72
- Qualitätslenkung, 127, 132
 - analytische Maßnahmen der, 128
 - konstruktive Maßnahmen der, 128
 - organisatorische Maßnahmen der, 127
- Qualitätsmanagement, 127
- Qualitätsmanager, 114
- Qualitätsmerkmal, 121
 - der Übertragung, 126
 - der Anwendung, 125
 - der Transformation, 125
 - der Wartung, 126
 - des Entwicklungsprozesses, 76
 - Festlegen der, 127
 - Priorisierung, 126
 - Prioritäten der, 127
 - von Endprodukten, 125
 - von Software, 121
 - von Zwischenprodukten, 125
- Qualitätsmodell, 79
- Qualitätsplanung, 127, 132
- Qualitätsprüfung, 127
- Qualitätssicherer, 114
- Qualitätssicherung, 119, 121, 258, 273, 292
 - analytische Maßnahmen zur, 128
 - Aufgaben der, 127
 - Ergebnisse der, 132
 - konstruktive Maßnahmen zur, 128
 - Maßnahmen der, 37, 132
 - organisatorische Maßnahmen zur, 127
 - projektübergreifende, 80
 - Techniken der, 128
 - Ziel der, 127
- Qualitätssicherungsmaßnahme, 284
 - Einbettung von, 90
- Qualitätssicherungsplanung, 186
- Qualitätstest, 263, 274, 287
- Quellcode, 124, 233, 243
 - Auslieferung des, 291
 - Orientierung im, 242
- quick reference guide, 231
- Rückkopplung, 84, 150
 - nicht-lineare, 76
- Rücklaufquote, 143
- RAD, 95
- Rahmen, 213, 216, 219, 223, 225
- Randbedingung, 161
- Rapid Application Development, 95
- Rapid Prototyping, 217
- Readme-Datei, 231, 291
- Realisierbarkeit, 161, 229
 - Prüfung der, 49
- Realisierungsplan, 38
- Realisierungsrisiko, 29
- Rechnerunabhängigkeit, 125

- Rechte, 37
- recovery test, 279
- Redundanz
 - Vermeiden von, 110
- Reengineering, 57, 173, 285
- refactoring, 210, 237
- Referenzhandbuch, 231
- Referenzkarte, 231
- Referenzprogramm, 263
- Regelbetrieb, 279
- Regelkreis, 44
- Regressionstest, 274
- Regressionstesten, 269
- Regressionstestsuite, 269
- Reifegrad, 130, 131
- Reihenfolgeplanung, 45
- reine Projektorganisation, 59
- Relation, 169
- release, 291
- repository, 157
- requirement, 159
- requirements analysis, 159
- requirements management, 161
- responsibility, 174
- Ressource, 28, 38, 124
- Ressourcenbelegungsplan, 191, 273
- Ressourcenplan, 38, 191, 273
- Ressourcenplanung, 185
- Ressourcenverwaltung, 212
- Ressourcenverwendung, 273
- Retesten, 269
- reusability, 219
- reuse, 219
 - design, 221
- Reverse Engineering, 223
- Review, 53, 104, 248
 - Team, 248
 - technisches, 248
- Revision, 111, 235
- Richtlinie, 132, 133
- Risiko, 191, 192
 - Abgrenzung von, 192
 - aktive Behandlung eines, 194
 - allgemeines, 196
 - Aufwands-, 29
 - Behandlung der wichtigsten, 203
 - Notwendigkeit von, 192
 - Projekt-, 29
 - projektspezifisches, 196
 - Realisierungs-, 29
 - Verwertbarkeits-, 29
 - wichtigste bei Softwareprojekten, 193
 - Zeit-, 29
 - Zielsetzungs-, 29
- Risiko-Übertragung, 202
- Risiko-Überwachung, 199
- Risiko-Überwindung, 199
- Risiko-Analyse, 198
- Risiko-Annahme, 201
- Risiko-Beherrschung, 196
- Risiko-Beherrschungsstrategie, 200
- Risiko-Bewertung, 196, 200
- Risiko-Identifikation, 196
- Risiko-Priorisierung, 198
- Risiko-Reduktion, 201
- Risiko-Vermeidung, 201
- Risiko-Vorsorgeplan, 207
- Risiko-Vorsorgeplanung, 199
- Risikoabgrenzung, 200
- Risikoanalyse, 90
- Risikoauswirkung, 193
- Risikobegriff, 191
- Risikobehandlung, 45, 191, 193
 - Aufgaben der, 196
 - Ergebnisse der, 206
 - Möglichkeit der, 193
 - Situierung der, 196
 - Techniken der, 200
 - Werkzeuge der, 205
 - Ziel der, 191, 194
- Risikoeigenschaft, 192
- Risikofaktor, 198
- Risikoindikator, 207
- Risikokopplung, 193
- Risikopunkt, 92
- Risikoverhalten, 200
 - Abschätzen des, 200
 - risikofreudiges, 200
 - risikoindifferentes, 200
 - risikoscheues, 200
- Risikowahrscheinlichkeit, 192
- risk, 191
 - analysis, 198
 - assumption, 201
 - avoidance, 201
 - behavior, 200
 - contingency plan, 207
 - contingency planning, 199
 - coupling, 193
 - estimation, 198
 - identification, 196
 - impact, 193
 - manageability, 193
 - management, 193
 - mitigation, 201
 - monitoring, 199
 - prioritization, 198
 - probability, 192
 - resolution, 199
 - transfer, 202
- Robustheit, 123
- Rollback, 212
- Rolle, 112
 - des Bedieners, 280
- Rollenspiel, 174
- Rollenwechsel, 66
- Roundtrip Engineering, 106, 223
- Routine, 27
- Routinisierung, 27

- Sachmitteleinsatzplanung, 185
- Sachmittelplanung, 185
- Sachverhaltsdarstellung, 292
- scaffolding, 238
- Schablone, 93
 - Abwandeln von, 219, 221
- Schablonenkatalog, 93
- schablonenmäßiges Vorgehen, 93
- Schablonenmodell, 93
- Schiedsstelle, 113
- Schmökerer, 242
- Schnittstelle, 26, 44, 157, 161, 163, 181, 214, 224, 283
 - anbietende, 221
 - Benutzer-, 157
 - funktionale, 165
 - geforderte, 221
 - Modellierung von, 211
 - Spezifikation einer, 234
 - System-, 157
- Schnittstellenbelastbarkeit, 261
- Schnittstellenbeschreibung, 155, 157, 214
- Schnittstellenfehler
 - eines Subsystems, 268
- Schnittstellenklasse, 221
- Schnittstellenobjekt, 224
- Schnittstellenprüfer, 291
- Schnittstellenprototyp, 52
- schrittweise Verfeinerung, 74, 210, 234
- schrittweises Vorgehen, 82
- Schulung, 37, 279
- Schulungsplan, 38
- Schulungsplanung, 186
- Schulungsunterlage, 280
- screen grabber, 272
- Scripting, 175
- Scrum, 78
- SEI, 74
- Sekundärerhebung, 139
- Selbstverwirklichung, 32
- sequenzielles Vorgehen, 77
- serielle Entwicklung, 52
- service, 168
- Shoot-the-Messenger-Prinzip, 199
- Sicherheitsbeauftragter, 114
- Sicherheitsplanung, 186
- Sicherungsplanung, 186
- Sicht, 26
 - logische, 153
 - physische, 154, 209
- Simulator, 291
- Sine-qua-non-Bedingung, 143
- Singlestepping, 251
- Six Sigma, 132
- Skalierung, 142
- skill, 51
- slack, 69, 184
- SLIM, 187
- SLOC, 249
- Smalltalk, 234, 237
- Software, 87
 - Produktlebensdauer von, 41
 - Qualität von, 121
 - Qualitätsmerkmale von, 121
 - value-added, 76
- software development
 - component-oriented, 171
 - quality-driven, 126
- Software Engineering, 32–34
 - Computer Aided, 106
- Software Engineering Institute, 74
- software life cycle, 77
- software life cycle management, 187
- software requirements specification, 180
- Software-CMM, 129
- Software-Projekt Engineering, 34
- Software-Projektentwicklung, 34
- Softwareentwicklung
 - agile, 78
 - aspektorientierte, 171
 - besondere Eigenschaften der, 62
 - evolutionäre, 87
 - Grundkriterien moderner, 243
 - humanzentrierte, 67
 - ingenieurmäßige, 34
 - komponentenorientierte, 171
 - Normen und Richtlinien in der, 135
 - Prozess der, 75
 - prozessorientierte, 76
- Softwareentwicklungsmethode
 - agile, 107
 - bottomup-orientierte, 85
 - topdown-orientierte, 88
- Softwareentwicklungsprozess, 128
 - qualitätsgesteuerter, 126
- Softwareentwicklungsumgebung, 106
- Softwareerstellung, 33
- Softwareingenieur, 40, 114
 - Profession des, 68
- Softwarekomponente, 223
- Softwarekrise, 32
- Softwarelebenszyklus, 77
 - Modell, 171
- Softwareprodukt, 277
 - Optimierung eines, 40
 - Wartung eines, 281
- Softwareprojekt
 - Erfolgsstatistik von, 57
- Softwareprototyp, 87
- Softwareprozessmessung, 132
- Softwarequalität, 127
- Softwarerisiken
 - wichtigste, 193
- Softwaresystem, 33
 - dialogorientiertes, 123
 - Einmaligkeit eines, 55
 - Korrektheit eines, 281
 - Leistungsfähigkeit eines, 123
- Softwaretechnik, 33
- Sollkonzept, 37

- Sollziel, 145
- Sollzustand, 37
 - Beschreibung des, 180
 - Darstellung des, 38
 - Erhebung des, 160
- source code, 233, 243
- source lines of code, 249
- soziale Kompetenz, 67
- Soziologie, 27
- Spagetticode, 234
- Speicherprüfer, 254
- Speichertest, 261
- Spezialisierung, 66, 216
- Spezialisierungsstruktur, 168
- Spezialist, 65
- Spezifikation, 122, 165
- Spezifikationssprache
 - formale, 178
- SPICE, 132
- Spin-off, 39
- Spiralmodell, 89, 195
- Spitzenqualität, 121
- Spontandefinition, 149
- Sprechdisziplin, 173
- Stärken-Schwächen-Profil, 131
- Störgröße, 73
- Stabilisierung, 283
- Stakeholder, 102
- Standard, 110, 132
- Standard-Überwacher, 114
- Standardüberwachungsprogramm, 273
- Standardbeschreibungssprache
 - für Softwareentwicklungen, 153
- Standardbibliothek, 284
- Standardisierung, 27
- Standardkomponente, 284
- Standardprodukt
 - Verwendung eines, 202
- Statusbesprechung, 103
- Stelle, 112
- Stellenbeschreibung, 112
- Stelleninhaber, 112
- Stellenzuordnungsplan, 191
- Stellungnahme, 278
- stellvertretender Projektleiter, 114
- Stellvertretung, 112
- Steuerungsaufgabe, 44
- Steuerungsobjekt, 224
- Stichprobe, 141
- Stichwortprotokoll, 104
- Strategie
 - Easiest-first-, 45
 - Hardest-first-, 45
- structure model, 153
- Struktogramm, 165
- Struktur, 168
 - Aggregations-, 168
 - Dekompositions-, 168
 - Generalisierungs-, 168
 - Spezialisierungs-, 168
- Strukturanalyse, 250
- Strukturanomalie, 250
- strukturierte Analyse, 166
 - mit Datenflussdiagrammen, 166
 - mit Kontrollflussdiagrammen, 167
- Strukturiertheit, 232
- Strukturierung
 - hierarchische, 159
- Strukturmodell, 153, 176, 211
 - Erweiterung des, 211
- Strukturtest, 252
- stub, 238
- Stumpf, 238
- Subauftragnehmer, 38
- Subkomponente, 26
- Subprojekt, 39
- Subsystem, 155
 - Gliederung in, 155
- Subsystemtest, 268
- Supersystem, 155
- Supplement, 231, 291
- supplier class, 221
- SW-CMM, 129
- Synektik, 148
- System, 25
 - Abbildung eines, 26
 - brauchbares, 288
 - computerbasiertes, 40
 - datenbankbasiertes, 178
 - Einzelteile des, 173
 - embedded, 211
 - komplexes, 155
 - soziales, 26
 - Strukturen eines, 59
 - Strukturierung eines, 28
 - Sub-, 155
 - Super-, 155
 - validiertes, 269
 - verifiziertes, 269
 - verteiltes, 171, 211
 - Ziel eines, 26
- system model, 210
- system requirements specification, 180
- system specification, 159, 228
- Systemanalyse, 138, 161
- Systemanforderungen, 181
- Systemarchitektur, 153, 210, 211
- Systembeschreibung, 154
- Systembetreuer, 114
- Systembeziehung, 26, 28
- Systemdesign, 210
 - Umsetzen des, 233
 - Wiederverwendung des, 221
- Systemdeterminante, 26
 - der Organisation, 26
- Systemdokumentation, 243
 - Komponenten der, 244
 - moderne, 245
- Systemeigenschaft, 87
- Systemelement, 26, 28

- Systementwicklung, 87
- Systementwurf, 209
- Systemgestaltung, 209
- Systemgrenze, 155
- Systemmodell, 210, 223, 225
- Systemperformanz, 288
- Systemrahmen, 56, 218, 219, 221, 222, 226, 243, 251, 268
 - Konkretisieren von, 219, 221
- Systemschnittstelle, 157
- Systemspezifikation, 159, 180
- Systemtest, 239, 268
- Systemverhalten, 26, 153, 160, 288
 - zugesagtes, 289
- Systemziel, 26
 - Quantifizierung eines, 26
- Szenario, 175
- Tätigkeit, 188
 - humanzentrierte, 67
- Tagesordnung, 103
- Tagesordnungspunkt, 103
- tailoring, 279
- takeover, 277
- target platform, 279
- task, 51
- Taskforce, 59
- Taxonomie, 163, 216
- Team, 63
- team software process, 184
- Teamarbeit, 66
- Teamgeist, 32
- Teamorganisation
 - hierarchische, 63
- Teamstärke
 - volle, 45
- Teamzusammensetzung, 105
- Techniken
 - der Anforderungsanalyse, 162
 - der Erhebung, 139
 - der Planung, 187
 - der Produkt-Einführung, 286
 - der Qualitätssicherung, 128
 - der Risikobehandlung, 200
 - der Zielfindung, 146
 - des Codierens, 234
 - des Debuggens, 247
 - des Designs, 214
 - des Testens, 262
- technische Führung, 50
- technologieorientierte Betrachtungsweise, 128
- Teilarbeitsbereich, 44
- Teilauftrag, 38
- Teilorganisation
 - projektorientierte, 67
- Teilprojekt, 38
 - machbares, 182
 - parallel laufendes, 40
- Teilsystem, 26
 - Entkopplung von, 171
- Telefonnotiz, 116
- template, 93
- Termin, 28
- Terminüberschreitung, 72
- Terminkontrolle, 54, 108
- Terminplan, 191
- Terminplanung, 184
- Terminverzug, 71
- Test
 - Abnahme-, 262, 289
 - Abnahmevor-, 261, 289
 - Belastungs-, 261
 - Benchmark-, 288
 - Big-Bang-, 252
 - Blackbox-, 252, 258
 - Bottomup-, 251
 - Einzel-, 267
 - funktionaler, 274
 - Funktionalitäts-, 267
 - Funktions-, 267
 - Gesamt-, 268, 289
 - Glassbox-, 252
 - inkrementeller, 252
 - innerer, 252
 - Installations-, 287
 - Integrations-, 267
 - Integritäts-, 268
 - Klassen-, 267
 - Kompatibilitäts-, 274
 - Komponenten-, 267
 - Konformanz-, 263, 274
 - Leistungs-, 288
 - Modul-, 267
 - nichtfunktionaler, 274
 - Nonkonformanz-, 263, 274
 - Performanz-, 260, 274
 - Pilot-, 288
 - Programm-, 267
 - Qualifikations-, 265, 274
 - Qualitäts-, 263, 274, 287
 - Regressions-, 274
 - Speicher-, 261
 - Struktur-, 252
 - Subsystem-, 268
 - System-, 268
 - Topdown-, 251
 - Volumen-, 261
 - Whitebox-, 252
 - Wiederinbetriebnahme-, 279
- test first development, 239
- test run, 287
- test suite
 - acceptance, 265
 - qualification, 265
- test-driven development, 239
- Testablauf
 - Beschreibung des, 275
 - Planung des, 264
- Testansatz, 273
- Testaufwand

- Abschätzung des, 259
- Testautomatisierung, 106, 270
- Testbarkeit, 124
- Testdokumentation, 274, 291
- Testen, 45, 57
 - äußeres, 258
 - Aufgaben des, 259
 - Big-Bang-, 267
 - Blackbox-, 258
 - der Funktionalität, 267
 - des Gesamtsystems, 268
 - Ergebnisse des, 273
 - inkrementelles, 267
 - manuelles, 270
 - schrittweises, 267
 - Techniken des, 262
 - von objektorientiertem Code, 268
 - Werkzeuge des, 271
 - werkzeuggestütztes, 271
 - Whitebox-, 258
 - Ziel des, 257
- Testfall
 - Auswahl eines, 264
 - automatisches Generieren eines, 270
 - Beschreibung eines, 274
 - Erstellen eines, 263
- Testfalldesign, 262
- Testfallerstellung
 - zweistufige, 239
- Testfallgenerierung, 106
- Testfallverzeichnis, 274
- Testklasse, 274
- Testliste, 275
- Testlogbuch, 276
- Testmenge
 - standardisierte, 274
- Testorganisation
 - Misstrauen an der, 270
- Testpilot, 260
- Testplan, 273
 - Erstellung eines, 274
- Testplanung, 186
- Testqualität
 - Prüfen der, 235
- Testrahmen
 - Bau von, 238, 251, 267
- Testrahmensystem, 254, 272
 - für Komponententests, 254
 - für Systemtests, 254, 270, 272
- Testreihenfolge
 - notwendige, 274
- Testscript, 274
- Teststrategie, 273
- Testsuite, 274
 - Verantwortlicher, 114
- Testsuitedesign, 263
- Testsystem
 - capture & replay, 272
- Testscenario, 271
- Testumfang, 273
- Testumgebung, 265, 273
- Testunterstützung
 - Werkzeuge zur, 272
- Testvorbereitung, 264, 284
- Testwerkzeug, 106, 271
 - integriertes, 271
- Testzeit, 69
- Testziel, 273
- Testzyklus, 266
- Textverarbeitungssystem, 174
- tidying-up, 286
- Tool-Smithing, 85, 93
- Toolkit, 220
- Top-N-Liste
 - der Risikoverfolgung, 198, 206
- Topdown-Design, 83, 216
- Topdown-Test, 251
- Total Quality Management, 132
- totale Qualität, 130
- Totalerhebung, 139, 141
- TQM, 132
- traceability, 103, 162
- training, 279
- Transaktion, 212
- Transaktionsgrenze, 212
- transfer, 286
- Transparenz, 151
- Treiber, 238
- Triage, 145
- Trigger, 207
- Turn-Key-Entwicklung, 30
- turnover, 277
- Tutorial, 230, 280
- Twin-Peaks-Modell, 91
- Überforderung, 68
- Übergabe, 277
- Übergabeparameter, 214
- Übernahme, 277
- Überprüfbarkeit, 182
 - des Zielerfüllungsgrades, 30
- Überprüfung, 43, 121
 - Aufgaben der, 45
 - des Projektfortschrittes, 186
 - Ziel der, 45
- Übertragbarkeit, 125
- UIL, 226
- Umformulierung, 149
- UML, 153
- UML 2, 153
- UML-Diagramme
 - für das Design, 224
 - für die Analyse, 176
 - für die Zielerforschung, 154
- Umplanung, 44
- Umstellung, 286
 - direkte, 286
 - parallele, 287
 - Strategien der, 286
 - testweise, 287

- Undo-Funktion, 123
- Unified Modeling Language, 153
- unit test, 267
- Universalbibliothek, 220
- unkritischer Vorgang, 188
- Unternehmen
 - Leistungsprozess eines, 47
 - Ziele des, 29
- Unternehmensführung, 278
- Unternehmensorganisation, 59
- Unternehmensprozess-Modell, 79
- Unterprogramm, 234
- Unterprojekt, 44, 52
- Untersystem, 26, 225
 - Kopplung von, 26
- Unterteilung
 - horizontale, 26
 - vertikale, 26
- Ursache-Wirkungs-Zusammenhang
 - Analyse des, 263
- use, 277
- use case, 153, 154
 - diagram, 155
 - model, 153
- user, 280
- user interface
 - builder, 242
 - builder, graphic, 226
 - description language, 226
 - freeze, 261
 - library, 220
- V & V, 269
- V-Modell, 77, 81
- V-Modell XT, 81
- validation, 268
- validation & verification, 269
- Validierung, 46, 160, 268
- Validierungsprozess, 83
- value-added software, 76
- value-driven requirements engineering, 102
- Variante, 111, 235
- Varietät, 27
- VDA, 134
- VDE, 134
- VDI, 134
- VDM, 178
- Veränderung
 - technologische, 55
- Verantwortlichkeit, 154, 174
- Verarbeitung, 73
- Verband Deutscher Elektrotechniker, 134
- Verbesserung
 - kontinuierliche, 85
- Verbesserungswesen, 143
- Verein der Deutschen Automobilhersteller, 134
- Verein Deutscher Ingenieure, 134
- Vererbung, 163, 168–170
- Verfeinerung
 - schrittweise, 74, 210, 234
- Verfremdung
 - Gesetze der, 148
- Verfremdungsphase, 148
- Verhalten
 - dynamisches, 172
 - emergentes, 76
 - lokales, 154
 - problemlösendes, 27
 - routiniertes, 27
 - systemglobales, 154
- Verhaltenskategorisierung, 164
- Verhaltensmodell, 154, 177, 211
 - Erweiterung des, 211
- verification, 267
- Verifikation, 160, 267
- Versionierung, 111
- Versionsverwaltung, 108, 111, 285
- Versionsverwaltungssystem, 111
- Versionsverwaltungswerkzeug, 290
- Verständlichkeit, 123, 232
- verteilte Entwicklung, 52
- Verteilungsdiagramm, 224
- Verteilungssicht, 210
- Verwertbarkeitsrisiko, 29
- Verzweigungsabdeckung, 252
- 24×7 operation, 122, 288
- view, 26
 - concurrency, 210
 - deployment, 210
 - logical, 153
 - physical, 154, 209
- Vision, 138
- vision statement, 138
- Vollständigkeit, 123, 127, 182, 232
- Volltextprotokoll, 104
- Vollversion, 291
- Volumentest, 261
- Vorbedingung, 214, 236
- Vorbesprechung, 103
- Vorgang, 188
 - kritischer, 188
 - unkritischer, 188
- Vorgehen
 - agiles, 109
 - agiles, Nachteile, 98
 - agiles, Vorteile, 98
 - Dokumenten-getriebenes, 109
 - inkrementelles, 83
 - iterativ-inkrementelles, 90, 173
 - iteratives, 85
 - modellbasiertes, 79
 - objektorientiertes, 85
 - prozessorientiertes, 79
 - schablonenmäßiges, 93
 - schrittweises, 82
 - sequenzielles, 77, 82
 - traditionelles, 109
 - Werkzeug-unterstütztes, 109
 - zyklisches, 89
- Vorgehenshandbuch, 38, 117

- Vorgehensmethode, 82
 - agile, 32, 96
 - Kombination von, 82
 - traditionelle, 96
- Vorgehensmodell, 77, 127
- Vorgehensprozess, 82
- Vorprojekt, 36, 37, 138
 - Ergebnisse des, 38
- Vorprojektsbericht, 39
- Vorstudie, 37

- W-Frage, 141
- W-Technik, 149
- Walkthrough, 248
- Wartung, 37, 40, 277, 281
 - Aktivitäten der, 283
 - Aufwand für, 285
 - im engeren Sinn, 282
 - im weiteren Sinn, 282
 - Kosten für, 284
 - Maßnahmen der, 284
 - Produktivität der, 285
 - von Software, 281, 283
- Wartungsaktivität, 285
- Wartungsauftrag, 286
- Wartungsbegriff
 - für Software, 281
- Wartungsfreundlichkeit, 124, 285
- Wartungskosten, 284
- Wartungsplanung, 186
- WAS/WIE-Trennung, 160, 171
 - Unterstützung der, 171
- Wasserfallmodell, 84, 90
- watch list, 198
- Watchpoint, 251
- WBS, 191
- Wegwerfprototyp, 173
- Weiterbildung, 37
- Weiterentwicklung
 - Sättigungskurve der, 41
- Werkabnahme, 281
- Werkzeug, 44, 106
- Werkzeuge
 - der Anforderungsanalyse, 174
 - der Planung, 189
 - der Produkt-Einführung, 290
 - der Risikobehandlung, 205
 - der Zielfindung, 151
 - des Codierens, 241
 - des Debuggens, 253
 - des Designs, 223
 - des Prototyping, 177, 226
 - des Testens, 271
- Werkzeugeinsatz, 106
- Werkzeugkoordinator, 114
- Werkzeugnutzung
 - sinnvolle, 107
- Werkzeugschmieden, 85
- Whitebox, 48
- Whitebox-Test, 252
- Whitebox-Testen, 258
- Widerspruchsfreiheit, 127
- Wiederinbetriebnahmetest, 279
- Wiederverwendbarkeit, 219
- Wiederverwendung, 85, 162, 165, 219
 - von Bausteinen, 220
- Wirtschaftlichkeit, 124, 160
- work breakdown structure, 191
- work-in-progress demonstration, 89
- Wunschziel, 145

- Z, 178
- Z++, 178
- Zahlungsziel, 37
- Zeilenabdeckung, 252
- Zeitplan, 38
- Zeitplanung, 184
- Zeitrisiko, 29
- Zeitverhalten
 - inakzeptables, 261
- Zerlegung
 - datenorientierte, 234
 - funktionale, 234
 - hierarchische, 164
 - objektorientierte, 168, 176, 234
- Zerlegungskomponente
 - funktionale, 171
- Zerlegungstechnik, 164
- Zertifizierung, 269
- Zertifizierungsaudit, 131
- Ziel
 - Adaptieren eines, 46
 - der Überprüfung, 45
 - der Anforderungsanalyse, 159
 - der Durchführung, 44
 - der Entwicklung, 51
 - der Erhebung, 138
 - der Führung, 49
 - der Kontrolle, 53
 - der Organisation, 26
 - der Planung, 44, 183
 - der Produkt-Einführung, 277
 - der Qualitätssicherung, 127
 - der Risikobehandlung, 191, 194
 - der Zielfindung, 145
 - des Codierens, 233
 - des Debuggens, 245
 - des Designs, 209
 - des Testens, 257
 - Detail-, 137
 - eines Systems, 26
 - Gesamt-, 137
 - Grob-, 137
- Zieladaptierung, 46
- Zielbeschreibung, 157, 170, 176
 - textuelle, 154
- Zieldefinition, 35
- Zielerfüllungsgrad
 - Überprüfbarkeit des, 30
- Zielerforschung, 57, 137, 173

- Erweiterung der, 160
- Zielerforschungsmodell, 153
- Zielerreichung
 - Messen des Grades der, 94
- Zielfindung, 151, 157
 - Aufgaben der, 145
 - Ergebnisse der, 157
 - Techniken der, 146
 - Werkzeuge der, 151
 - Ziel der, 145
- Zielgruppe, 141
- Zielkonflikt, 145
- Zielplanung, 183
- Zielsetzungsrisiko, 29
- Zielumgebung, 38, 279
 - echte, 279
 - simulierte, 279
- Zielvorstellung
 - persönliche, 27
- Zukauf, 185
- Zusammenarbeit, 174
 - interdisziplinäre, 70
- Zusatzprojekt, 40
- Zuverlässigkeit, 122, 278
 - hohe, 278
 - minimale, 278
 - mittlere, 278
 - niedrige, 278
 - unbekannte, 278
- Zuverlässigkeitsgrad, 143, 278
- Zwischenprodukt, 39, 125
- zyklisches Vorgehen, 89

Projekt Engineering

Herwig Mayr



Bewertung des Projektkommunikations					
	Erstellung	Lesen	Verstehen	Überprüfen	
Texte	1	2	3	4	5
Medien					
Verfahren					

Teilnehmer	Organisation	Wissensbasis	Ergebnisse
Informations-Systeme	IT		
Management-Systeme	Management		
Marketing-Systeme	Marketing		
Finanz-Systeme	Finanz		
Personal-Systeme	Personal		
Rechts-Systeme	Recht		
Umwelt-Systeme	Umwelt		

Erstellung	Lesen	Verstehen	Überprüfen
1	2	3	4

Erstellung	Lesen	Verstehen	Überprüfen
1	2	3	4



Dipl.-Ing. Dr. Herwig Mayr
ist Professor für Software
Engineering an der Fachhoch-
schule Hagenberg, Österreich.
Seit 1987 ist er als selbst-
ständiger IT-Berater tätig.

Dieses Buch gibt eine fundierte Einführung in die ingenieurmäßige Softwareentwicklung in Form von (Gruppen-) Projekten. Es fokussiert auf die vielfach vorhandene (Ausbildungs-)Lücke zwischen Softwareerstellung und Projektmanagement. Dabei wird ein moderner prozessorientierter Ansatz der Softwareentwicklung aus technischer Sicht verfolgt. Das Hauptaugenmerk liegt auf dem gesamten Entwicklungszyklus von der Erforschung der Ziele und Anforderungen des Auftraggebers über die eigentliche Entwicklung bis hin zur erfolgreichen Einführung und zum Betrieb des Softwareprodukts beim Kunden (integrierte technische, wirtschaftliche und soziale Betrachtungsweise).

Dieses Buch ist gedacht für Studierende im technischen und wirtschaftlichen Bereich, Lehrende/Vortragende über teamorientierte Softwareprojekte und für (Software-)Entwickler als Hilfe bei der ingenieurmäßigen Abwicklung von Softwareprojekten in Projektgruppen.

Ein durchgängiges Beispielprojekt
mit Übungsaufgaben begleitet
das Buch im Internet unter:
www.projekt-engineering.at



ISBN 3-446-40070-2



Fachbuchverlag Leipzig

www.fachbuch-leipzig.hanser.de